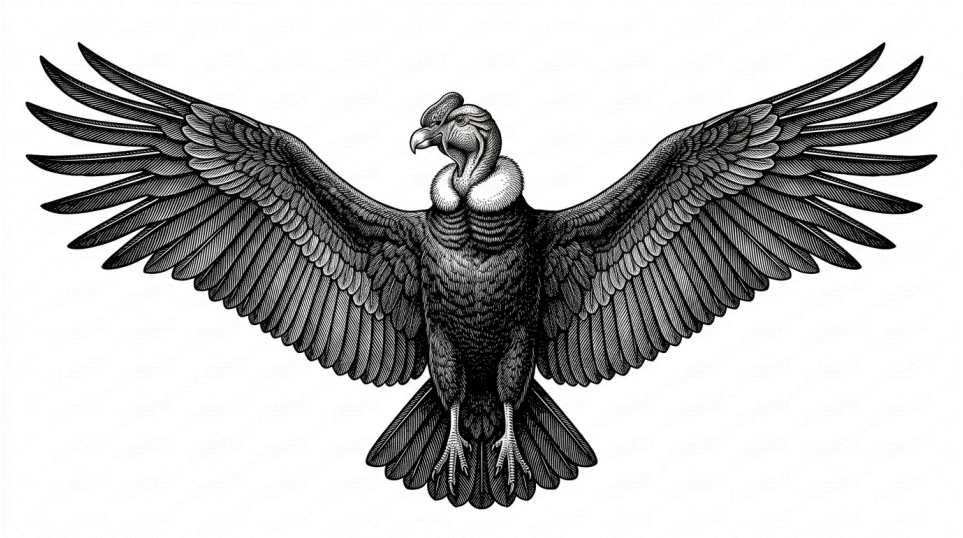
Cómo ser un

DATA ENGINEER

Una mirada a fondo de los fundamentos y los procesos prácticos para llegar de cero a Data Engineer



Luis Galvis

2026

Cómo ser un

DATA ENGINEER

Una mirada a fondo de los fundamentos y los procesos prácticos para llegar de cero a Data Engineer

Luis Galvis

2026

"A mis padres, por ser el cimiento inquebrantable, el sacrificio silencioso y la luz que impulsó cada uno de mis pasos. A mi pareja, por su amor incondicional, su infinita paciencia y su aliento constante en cada noche de estudio y creación. Y a todos los ingenieros y científicos que buscan comprender la esencia profunda de los datos más allá de las herramientas efímeras."

— Luis Galvis

Cómo ser un Data Engineer

Primera edición · Octubre de 2026

© 2026 Luis Galvis. Todos los derechos reservados.

Ninguna parte de esta obra puede ser reproducida, almacenada en sistemas de recuperación ni transmitida, total o parcialmente, por ningún medio, sin la autorización previa y por escrito del autor.

Los fragmentos de código fuente (PySpark, SQL, Python, HCL) se distribuyen bajo la licencia de código abierto MIT y pueden reutilizarse libremente en entornos productivos, conservando el aviso de licencia.

Escrito y compilado en Ibagué, Tolima, Colombia · 2026

Índice general

PARTE I · PANORAMA Y FUNDAMENTOS DE LA PROFESIÓN 8

Cap. 0 Data Engineering Roadmap y Panorama de la Profesión 9
Cap. 0A Cimientos de Plataforma: Linux, Git, Docker y Redes 33
Cap. 0B Bases de Datos Relacionales (OLTP) y Modelado Transaccional 46
Cap. 0C Python para Ingeniería de Datos: Tipado, APIs y Resiliencia 57
Cap. 0D Entorno de Laboratorio Reproducible y Generación de Datos 75
Cap. 1 Fundamentos de SQL Declarativo y Computación Analítica 85
Cap. 2 Agrupaciones Multidimensionales, Window Functions y Optimización 105
Cap. 2A DuckDB y Cómputo In-Process: Cuándo No Usar Computación Distribuida 117
Cap. 2B Algoritmos y Estructuras de Datos Esenciales para Data Engineers 127

PARTE II · COMPUTACIÓN DISTRIBUIDA CON APACHE SPARK 137

Cap. 3 De Hadoop y MapReduce al Cómputo en Memoria 138
Cap. 4 Arquitectura de Clúster: Driver, Workers y Ejecutores 143
Cap. 5 Entorno de Configuración y SparkSession 149
Cap. 6 Abstracciones de Cómputo: De RDDs a DataFrames 154

PARTE III · INGESTA, FORMATOS Y ESQUEMAS 159

Cap. 7 Ingesta de Datos: Archivos, JDBC Batch y Fuentes Incrementales 160
Cap. 8 Formatos de Archivo: Parquet, ORC y Avro 168
Cap. 9 Tipos de Datos y Casting Riguroso en Memoria 176

PARTE IV · TRANSFORMACIÓN CON LA API DATAFRAME 182

Cap. 10 Filtrado y Expresiones Condicionales 183
Cap. 11 Transformaciones de Columnas y Operaciones con Esquemas 188
Cap. 12 Joins Distribuidos y Estrategias de Cruce 194

PARTE V · EL MOTOR INTERNO Y LA OPTIMIZACIÓN EN SPARK 204

Cap. 13 Evaluación Perezosa (Lazy Evaluation) y el Grafo DAG 205
Cap. 14 El Optimizador Catalyst y Spark SQL 212
Cap. 15 Gestión de Memoria RAM y Estrategias de Caché 219
Cap. 16 Particionado, Shuffling y Mitigación de Data Skew 225

PARTE VI · ARQUITECTURA LAKEHOUSE Y TRANSACCIONES ACID 232

Cap. 17 Arquitectura Medallón y Modelado Dimensional 233

Cap. 18 Lakehouse y Formatos de Tabla Abiertos: Delta Lake y Apache Iceberg 247

Cap. 18A Plataforma Databricks: Arquitectura, Delta Live Tables, Photon y Workflows 258

Cap. 19 Gobernanza, Linaje y Unity Catalog en Lakehouse 273

Cap. 20 Catalogación, Descubrimiento y Contratos de Datos (Data Contracts) 282

PARTE VII · PROCESAMIENTO EN TIEMPO REAL (STREAMING) 291

Cap. 21 Fundamentos de Streaming y Apache Kafka 292

Cap. 22 Spark Structured Streaming y Change Data Capture (CDC) 303

PARTE VIII · ORQUESTACIÓN, MODELADO Y DATAOPS 315

Cap. 23 Orquestación de Flujos: Apache Airflow y Azure Data Factory 316

Cap. 23A Orquestación Avanzada con Apache Airflow: Arquitectura Interna, Dynamic Mapping y Datasets 328

Cap. 24 Transformación Modular con dbt, Snapshots e Idempotencia 339

Cap. 25 Infraestructura como Código (IaC) con Terraform y OpenTofu 350

Cap. 26 Integración y Despliegue Continuo (CI/CD) para Pipelines de Datos 359

Cap. 26A Pruebas de Software en Datos: Pytest, Chispa y Testcontainers 369

PARTE IX · OBSERVABILIDAD Y CALIDAD DE DATOS 378

Cap. 27 Observabilidad, Monitoreo y Logging en Sistemas Distribuidos 379

Cap. 28 Calidad de Datos (Data Quality) y Validación Automatizada 388

PARTE X · ARQUITECTURA CLOUD, SYSTEM DESIGN E INTELIGENCIA ARTIFICIAL 398

Cap. 29 Cloud Data Architecture y Optimización de Costos FinOps 399

Cap. 29A Microsoft Fabric y OneLake: Arquitectura SaaS y el Futuro Analítico en Azure 410

Cap. 29B Arquitectura de Plataformas de Datos en AWS: S3 Data Lake, Glue, EMR, Athena y Redshift 418

Cap. 29C Arquitectura de Plataformas de Datos en Google Cloud (GCP): BigQuery, Dataflow, Pub/Sub y Dataproc 427

Cap. 29D Plataforma Snowflake Empresarial: Arquitectura Multi-Cluster, Snowpark, Dynamic Tables, Snowpipe y Polaris Catalog 442

Cap. 30 System Design para Data Engineers y Preparación para Roles Senior 454

Cap. 31 Seguridad, Privacidad y Gobernanza Regulatoria de Datos 469
Cap. 32 Bases de Datos NoSQL y Motores Especializados 480
Cap. 33 Ingeniería de Datos para Inteligencia Artificial y LLMs 491

PARTE XI · PROYECTO INTEGRADOR Y PLATAFORMA DE PRODUCCIÓN 506

Cap. 34 Proyecto Integrador: Plataforma Lakehouse y Portafolio de Producción 507

PARTE XII · APÉNDICES DE REFERENCIA, GLOSARIO Y EPÍLOGO 524

Cap. 35 Apéndice A — Glosario Enciclopédico de Términos y Siglas de Ingeniería de Datos (A-Z) 525

Cap. 36 Apéndice B — El Manual Definitivo de Carrera, Empleo y Preparación de Entrevistas 534

Cap. 37 Epílogo — El Futuro de la Ingeniería de Datos y Conclusión de la Obra .. 553

Guía del lector

Esta obra está concebida como un manual riguroso de ingeniería que acompaña la progresión integral desde los fundamentos de la computación analítica hasta el diseño de sistemas de datos a escala empresarial.

El texto se organiza en doce partes temáticas. Comienza con los cimientos de Linux, Git, Docker, bases de datos relacionales OLTP, Python moderno y SQL declarativo avanzado; continúa con el cómputo distribuido en Apache Spark; profundiza en la optimización física de memoria y shuffles; aborda la arquitectura Lakehouse con Delta Lake y Unity Catalog; trata el procesamiento en tiempo real con Kafka y Structured Streaming; cubre DataOps, Terraform, observabilidad, calidad con Great Expectations, FinOps, System Design, plataformas empresariales (AWS, Azure Fabric, GCP, Databricks y Snowflake), seguridad regulatoria (GDPR y Ley 1581), persistencia políglota NoSQL, ingeniería de datos para IA y LLMs; culmina con un proyecto integrador (capstone) de extremo a extremo y se complementa con un glosario enciclopédico de términos, guías de estimación de escala y un epílogo de ingeniería.

Cada capítulo incluye objetivos de aprendizaje, definiciones formales, notas de producción, anti-patrones de la industria, un resumen técnico, ejercicios de revisión con solución paso a paso y lecturas recomendadas canónicas. Se recomienda ejecutar los bloques de código y verificar los planes de ejecución en el laboratorio reproducible provisto.

PARTE I

Panorama y fundamentos de la profesión

Capítulo 0 · Data Engineering Roadmap y Panorama de la Profesión

Capítulo 0A · Cimientos de Plataforma: Linux, Git, Docker y Redes NÚCLEO

Capítulo 0B · Bases de Datos Relacionales (OLTP) y Modelado Transaccional NÚCLEO

Capítulo 0C · Python para Ingeniería de Datos: Tipado, APIs y Resiliencia NÚCLEO

Capítulo 0D · Entorno de Laboratorio Reproducible y Generación de Datos NÚCLEO

Capítulo 1 · Fundamentos de SQL Declarativo y Computación Analítica NÚCLEO

Capítulo 2 · Agrupaciones Multidimensionales, Window Functions y Optimización NÚCLEO

Capítulo 2A · DuckDB y Cómputo In-Process: Cuándo No Usar Computación Distribuida NÚCLEO

Capítulo 2B · Algoritmos y Estructuras de Datos Esenciales para Data Engineers RECOMENDADO

CAPÍTULO 0

Data Engineering Roadmap y Panorama de la Profesión

> *"El rol del Data Engineer no es simplemente escribir consultas SQL; es diseñar sistemas de distribución robustos, resilientes e idempotentes capaces de mover y transformar petabytes con latencia predecible."*

0.1 Intuición Inicial: El Acueducto Invisible de los Datos

Imagina abrir el grifo de tu cocina y recibir agua potable limpia, a presión constante y sin interrupciones. No piensas en los kilómetros de tuberías subterráneas, las plantas de tratamiento químico, los tanques de decantación ni las bombas de presión que evitan que la red colapse.

En una empresa contemporánea, el analista de negocio y los tableros de Power BI son como ese grifo: esperan abrir una herramienta analítica y observar números limpios, coherentes y actualizados al segundo. Sin embargo, detrás de esa interfaz existe un torrente de datos caótico: bases de datos transaccionales saturadas, archivos JSON semiestructurados con formatos rotos, registros de clientes duplicados y millones de eventos por segundo llegando desde aplicaciones móviles.

El Ingeniero de Datos (Data Engineer) es el profesional que diseña, construye, monitorea y repara esa gigantesca red de acueductos digitales. Su misión no es contemplar el agua en el vaso, sino garantizar que la información fluya desde los sistemas de origen hacia los destinos analíticos de forma pura, oportuna y sin inundar los sistemas operativos. Si un pipeline falla a las 3:00 de la madrugada, es el Data Engineer quien recibe la alerta y subsana el problema antes de que los tomadores de decisiones inicien su jornada laboral.

> **Repositorio Oficial de Laboratorio en GitHub:**
> Todo el código fuente, los entornos de Docker Compose, los scripts de generación de datos y las soluciones completas a los ejercicios de esta obra se encuentran alojados y mantenidos en el repositorio oficial de código abierto:
> **https://github.com/luisgalvis/como-ser-un-data-engineer**
> *(Consulte el Capítulo 0D para las instrucciones detalladas de clonación y despliegue local).*

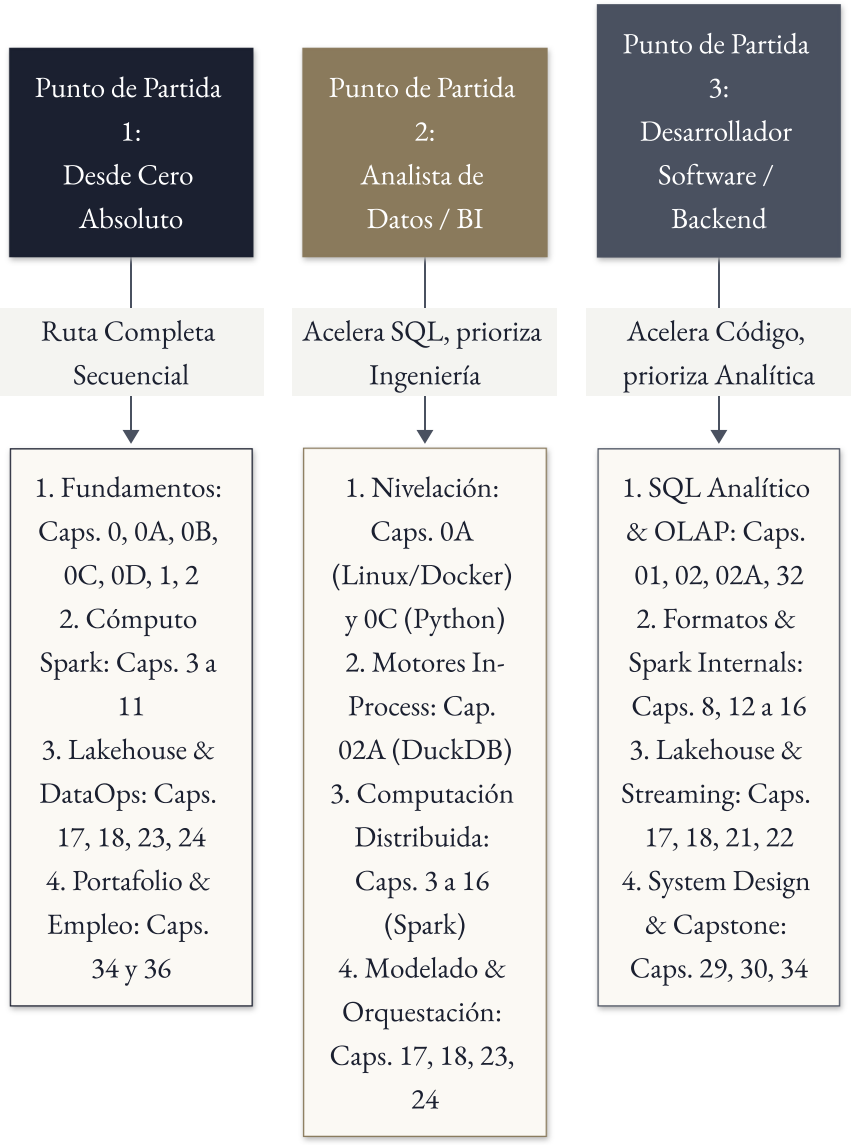
Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución y fronteras del rol de Data Engineer frente a analistas de datos, científicos de datos e ingenieros de machine learning.
- Identificar las tres rutas de lectura recomendadas según su punto de partida técnico (desde cero absoluto, analista en transición o programador backend).
- Distinguir qué capítulos constituyen el **Núcleo de Empleabilidad** para obtener un primer trabajo frente a los módulos recomendados y avanzados.
- Conocer la jerarquía física de memoria, los fundamentos web HTTP/APIs y la estadística operativa mínima necesaria para la disciplina.
- Trazar un plan de estudio deliberado de 9 a 12 meses con dedicación horaria honesta y una estrategia pragmática de certificaciones oficiales.

Rutas de Lectura según tu Punto de Partida

Para maximizar el aprovechamiento de esta obra, identifique su perfil inicial y siga la secuencia curricular adaptada a sus fortalezas previas:



Cuestionario de Auto-Diagnóstico Inicial: ¿Dónde me encuentro hoy?

Responda con honestidad a las siguientes 10 preguntas para confirmar cuál es su ruta de estudio óptima:

1. ¿Sabe clonar un repositorio con Git, crear ramas (*branches*) y resolver conflictos básicos en terminal? `[ ] Sí [ ] No`
2. ¿Puede escribir un archivo `docker-compose.yml` y levantar servicios aislados (como PostgreSQL o MinIO) en su máquina? `[ ] Sí [ ] No`

3. ¿Escribe funciones en Python con tipado estático (`typing`), manejo estructurado de excepciones y pruebas en `pytest`? `[ ] Sí [ ] No`
4. ¿Ha consumido APIs REST con librerías HTTP gestionando respuestas con paginación por cursor y códigos de estado (200, 429, 500)? `[ ] Sí [ ] No`
5. ¿Domina consultas SQL complejas empleando Expresiones de Tabla Comunes (CTEs) y funciones de ventana (`ROW_NUMBER`, `LEAD`, `LAG`)? `[ ] Sí [ ] No`
6. ¿Comprende la diferencia fundamental entre una base relacional normalizada en 3NF y un esquema analítico en estrella de Kimball? `[ ] Sí [ ] No`
7. ¿Ha ejecutado transformaciones sobre DataFrames de Apache Spark (PySpark) y analizado planes de ejecución en la interfaz Spark UI? `[ ] Sí [ ] No`
8. ¿Conoce los mecanismos de consistencia transaccional ACID y control de versiones (*Time Travel*) de Delta Lake o Apache Iceberg? `[ ] Sí [ ] No`
9. ¿Ha programado y orquestado flujos de trabajo (*DAGs*) productivos en Apache Airflow bajo el paradigma TaskFlow? `[ ] Sí [ ] No`
10. ¿Ha modelado canalizaciones de transformación en dbt aplicando pruebas automáticas de unicidad y no-nulos? `[ ] Sí [ ] No`

Interpretación de Resultados:

- **0 a 3 respuestas afirmativas: Ruta 1 (Cero Absoluto).** Su prioridad es avanzar de forma estrictamente lineal por los capítulos marcados como `[NÚCLEO EMPLEABILIDAD]`, sin saltarse los fundamentos de terminal, Linux, Python básico y SQL inicial.
- **4 a 6 respuestas afirmativas (con fortaleza en SQL/BI): Ruta 2 (Analista en Transición).** Puede avanzar rápido en SQL básico y centrar el 80% de su energía en Python modular, Docker, computación distribuida con Spark y orquestación con Airflow.
- **4 a 6 respuestas afirmativas (con fortaleza en código/DevOps): Ruta 3 (Desarrollador Backend en Transición).** Puede obviar la sintaxis básica de Python y terminal. Su foco crítico debe ser el SQL analítico avanzado, el modelado dimensional de Kimball, el tuning de memoria en Spark y la arquitectura Lakehouse.
- **7 o más respuestas afirmativas:** Diríjase de inmediato a las partes de optimización física avanzada (Partes V a X), al Proyecto Integrador Capstone y al Apéndice B para preparar entrevistas de nivel Mid-Level o Senior.

Clasificación de Capítulos por Nivel de Prioridad

A lo largo del libro, cada capítulo incluye una insignia editorial que delimita su alcance curricular:

- `[NÚCLEO EMPLEABILIDAD]`: Temas indispensables para superar entrevistas técnicas junior y desempeñarse en el primer trabajo (Linux, Git, Docker, Python, SQL, DuckDB, Spark básico,

Delta Lake, Airflow, dbt y los Proyectos de Portafolio).

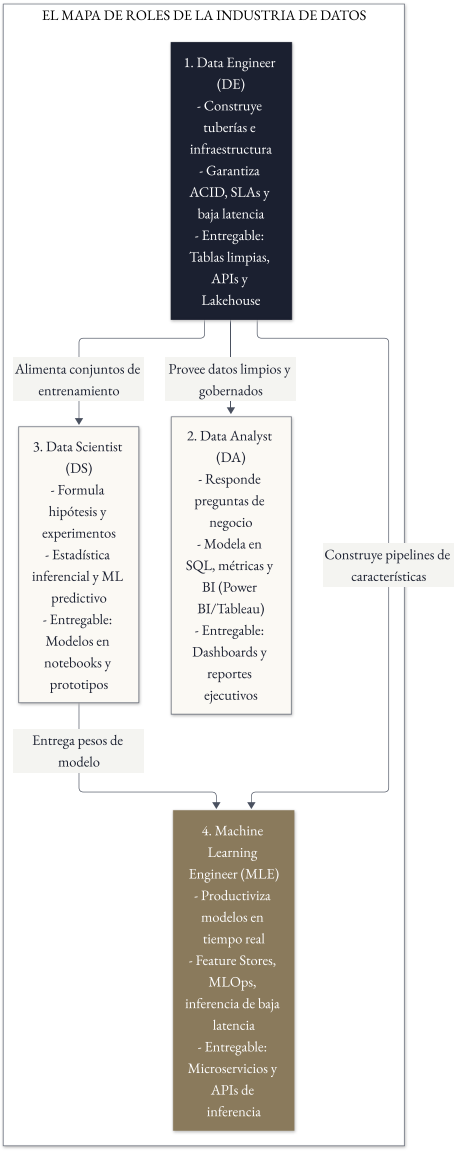
- `[RECOMENDADO]`: Herramientas que diferencian su perfil y aceleran la transición a puestos intermedios (Kafka, Terraform, Great Expectations, nubes públicas AWS/Azure/GCP, Databricks y Snowflake).
- `[AVANZADO]`: Conceptos de optimización física profunda, arquitectura distribuida y liderazgo técnico (Catalyst, Shuffling a fondo, Data Vault 2.0, Flink, MLOps, RAG y System Design senior).

Profundiza: El Fundamento Formal de los Sistemas de Datos

> **Definición 0.1 (Ingeniería de Datos como Disciplina de Sistemas Distribuidos)**
> La ingeniería de datos contemporánea es la rama de la ingeniería de software responsable del diseño, implementación, escalabilidad y observabilidad de plataformas para la recolección, transformación, almacenamiento y gobierno de activos de información a gran escala.
>
> Su objetivo fundamental es garantizar que los datos fluyan desde sistemas operacionales heterogéneos hacia capas analíticas con alta disponibilidad, latencia predecible, consistencia transaccional (semántica ACID) e integridad estructural verificable.

0.2 El Mapa de Roles de la Industria Analítica

Para evitar la habitual confusión en las ofertas de empleo, delimitamos formalmente las fronteras entre los cuatro roles principales del ecosistema:



- **Data Engineer (DE):** Arquitecto y constructor de las plataformas. Si el pipeline se detiene a la medianoche, si los datos contienen duplicados o si la consulta tarda 4 horas en vez de 3 minutos, es su responsabilidad directa.
- **Data Analyst (DA):** Traduce datos limpios a decisiones corporativas. Depende de que el DE le entregue dimensiones conformadas y tablas de hechos agregadas en la capa Gold.
- **Data Scientist (DS):** Descubre patrones probabilísticos y correlaciones para predecir comportamientos futuros.
- **Machine Learning Engineer (MLE):** Toma el modelo matemático validado por el científico y lo despliega como un microservicio escalable con monitoreo de deriva (*drift*).

0.3 Fundamentos Previos: Computadores, Redes y Estadística Mínima

Antes de manipular millones de registros en clústeres distribuidos, todo ingeniero debe comprender las restricciones físicas impuestas por el hardware y la red.

1. La Jerarquía de Memoria y la Física de la Latencia

En sistemas de datos, el tiempo requerido para acceder a un dato delimita el rendimiento máximo de cualquier algoritmo. A continuación se presentan las latencias de hardware normalizadas a escala humana (suponiendo que un ciclo de CPU equivalga a 1 segundo):

Nivel de Memoria / Operación	Latencia Física Real	Tiempo Equivalente Humano
Ciclo de reloj de CPU (Registros)	0.3 ns	1 segundo
Caché L1	0.9 ns	3 segundos
Caché L2	2.8 ns	9 segundos
Caché L3	12.9 ns	43 segundos
Memoria RAM Principal	100 ns	5.5 minutos
Lectura de Disco SSD NVMe (Bloque 4 KB)	10.000 ns (10 µs)	9.2 horas
Lectura de Disco Duro Magnético (HDD Seek)	10.000.000 ns (10 ms)	1.05 años
Transferencia en Red LAN (Mismo Datacenter)	500.000 ns (0.5 ms)	19.3 días
Llamada de Red WAN (Transcontinental)	60.000.000 ns (60 ms)	6.3 años

> **Lección Fundamental de Ingeniería:** Acceder a la memoria RAM es aproximadamente **100.000 veces más veloz** que buscar en un disco magnético, y transferir datos por la red es la operación más costosa de un clúster. Cuando Spark ejecuta un *Shuffle*, debe serializar datos en memoria, escribirlos a disco local y transmitirlos por la red a otros nodos. **Optimizar un pipeline consiste esencialmente en evitar transferencias de red y serializaciones innecesarias.**

2. Fundamentos de la Web: HTTP, REST y APIs

Gran parte de la ingesta de datos transaccionales se produce consumiendo interfaces de programación de aplicaciones (**APIs REST**) sobre el protocolo **HTTP/HTTPS**:

- **Métodos HTTP:** `GET` (lectura idempotente), `POST` (creación no idempotente), `PUT` (reemplazo idempotente), `PATCH` (modificación parcial).
- **Códigos de Estado Críticos en Pipelines:**
  - `200 OK` / `201 Created`: Solicitud exitosa.
  - `400 Bad Request` / `422 Unprocessable`: Datos con formato o contrato inválido (error en el payload).
  - `401 Unauthorized` / `403 Forbidden`: Problemas de token o permisos de acceso.
  - `429 Too Many Requests`: Se ha excedido la cuota de peticiones (*Rate Limit*). Requiere retroceso exponencial con variación aleatoria (*Exponential Backoff con Jitter*).
  - `500 Internal Error` / `502 Bad Gateway` / `503 Service Unavailable`: Falla transitoria del servidor de origen (permite reintentos automáticos).
- **Formato JSON:** El estándar dominante en eventos web. Formato semiestructurado con tipado débil que requiere validación estricta de esquemas antes de incorporarse al almacenamiento columnar.

3. Estadística Mínima para el Data Engineer

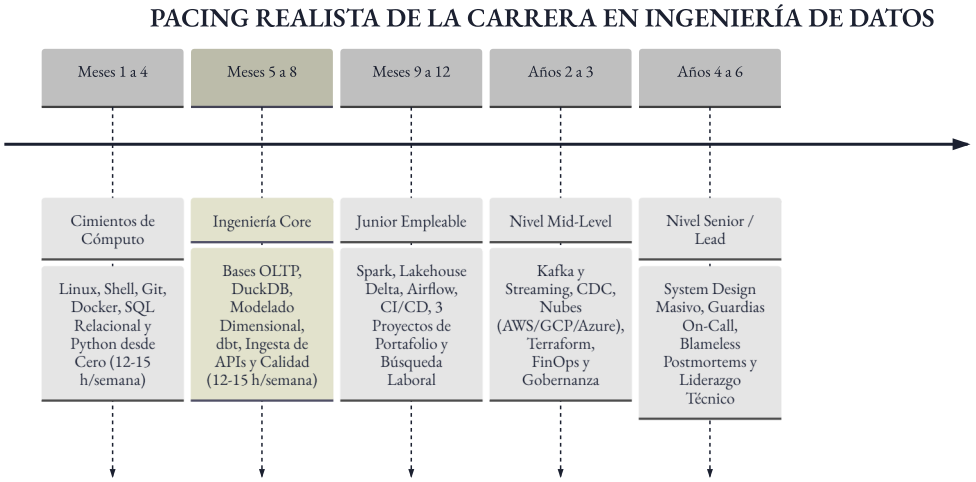
El Data Engineer no formula modelos econométricos, pero debe dominar tres conceptos estadísticos indispensables para no corromper datos de negocio ni diseñar alertas erróneas:

1. **La Trampa de la Media frente a la Mediana:**
   En distribuciones con asimetría severa (*skewness* o colas pesadas, como duración de consultas o volumen de compras), la media aritmética se ve fuertemente distorsionada por valores atípicos (*outliers*). En ingeniería de datos, **los acuerdos de nivel de servicio (SLAs) nunca se miden por el promedio, sino mediante percentiles**.
2. **Percentiles Operativos ($p50$, $p95$, $p99$):**
   - $p50$ (Mediana): El 50% de las ejecuciones concluyen por debajo de este umbral.
   - $p95$ y $p99$: El 95% o 99% de las ejecuciones terminan en este límite. El percentil 99 expone el peor escenario sufrido por los usuarios en momentos de saturación.
3. **Detección de Anomalías Volumétricas:**
   Para detectar caídas abruptas en la ingesta o duplicaciones accidentales, se monitorean la **Desviación Estándar (σ)** y el **Rango Intercuartílico ($IQR = Q_3 - Q_1$)**. Si el volumen diario de registros se desvía en más de 3σ respecto al promedio móvil mensual, el pipeline debe emitir una alerta preventiva antes de consolidar la capa Silver.

0.4 Pacing Profesional Realista: De Cero a Empleabilidad

Prometer que una persona sin formación técnica previa alcanzará el nivel de "Senior Data Engineer" en 12 meses es una afirmación irreal. La ingeniería de sistemas distribuidos exige asimilar abstracciones complejas, depurar fallos en memoria y acumular experiencia ante incidentes reales.

El camino profesional honesto y probado en la industria se estructura en tres hitos:



Plan de Estudio Mensual con Entregables Semanales (Hito 1: Junior Empleable)

Dedicación sugerida: **12 a 15 horas semanales** de estudio deliberado y práctica en laboratorio. Cada mes concluye con entregables verificables en código:

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 1	**Cimientos de Entorno y Linux** Cap. 00A (Linux/Docker/Git)	• **Semana 1:** Terminal Linux configurada con `zsh`/`bash`, scripts shell con pipes (`grep`, `awk`, `sed`) y control de versiones con Git (ramas, commits convencionales). • **Semana 2:** Contenerización con Docker: Dockerfile multi-stage, volumes y redes bridge. • **Semana 3:** Orquestación multicontenedor con `docker compose` levantando Postgres y MinIO con variables de entorno en `.env`. • **Semana 4:** Entregable Mes 1: Repositorio con un pipeline en Shell/Docker que descarga un dataset público, valida checksum SHA-256 y lo carga en PostgreSQL.
Mes 2	**Bases Relacionales y Python Moderno** Cap. 00B (OLTP) y 00C (Python Cero)	• **Semana 5:** Modelado relacional normalizado (3NF), índices B-Tree, constraints (`CHECK`, `FK`) y análisis de `EXPLAIN ANALYZE` en PostgreSQL. • **Semana 6:** Python moderno: entorno virtual aislado con `uv`, tipado estático estricto (`mypy`), estructuras inmutables y decoradores. • **Semana 7:** Manejo de datos estructurados: parseo robusto de JSON con Pydantic v2, lectura de CSVs y llamadas HTTP con `httpx` implementando backoff exponencial. • **Semana 8:** Entregable Mes 2: Paquete Python probado con `pytest` al 90% de cobertura que extrae datos de una API pública con paginación y valida esquemas con Pydantic.
Mes 3	**SQL Analítico Avanzado y DuckDB** Cap. 01, 02 y 02A (DuckDB)	• **Semana 9:** SQL declarativo: agregaciones avanzadas (`GROUPING SETS`, `ROLLUP`, `CUBE`) y expresiones de tabla comunes (`WITH RECURSIVE`). • **Semana 10:** Funciones de ventana analíticas: `ROW_NUMBER()` para deduplicación, `RANK()`, `DENSE_RANK()`, `LEAD()`, `LAG()` y marcos móviles (`ROWS BETWEEN`). • **Semana 11:** Motor in-process DuckDB: lectura directa de Parquet, inferencia de esquemas, exportación columnar y persistencia de catálogo local. • **Semana 12:** Entregable Mes 3: Script analítico en DuckDB + SQL que procesa 5 millones de registros de e-commerce en menos de 2 segundos generando un reporte de cohortes de retención.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 4	**Estructuras de Datos y APIs para DE** Cap. 02B (Estructuras de Datos)	• **Semana 13:** Algoritmos clave en DE: hashing, sets y diccionarios para uniones $O(N)$ en memoria sin bibliotecas pesadas. • **Semana 14:** Generadores y streams en Python: procesamiento perezoso (*lazy evaluation*) con `yield` para procesar archivos de 10 GB en máquinas con 2 GB de RAM. • **Semana 15:** Concurrencia controlada con `asyncio` y `ThreadPoolExecutor` para descargas paralelas masivas respetando límites de tasa (*rate limits*). • **Semana 16:** Entregable Mes 4: Extractor asíncrono con pool de hilos que descarga 50.000 registros mediante lotes de 100 sin desbordar memoria ni recibir errores 429.
Mes 5	**Fundamentos de Spark y Arquitectura** Cap. 03 a 06 (Arquitectura Clúster)	• **Semana 17:** Arquitectura distribuida: Driver, Cluster Manager, Executors, memoria JVM (Storage vs. Execution memory) y ciclo de vida de jobs. • **Semana 18:** RDDs y evaluación perezosa: transformaciones estrechas vs. anchas (*Narrow vs. Wide dependencies*) y costo físico del Shuffle. • **Semana 19:** Entorno PySpark local con Docker Compose: configuración de master/workers e interacción con PySpark CLI. • **Semana 20:** Entregable Mes 5: Benchmark comparativo de rendimiento entre un job con shuffle masivo (`groupByKey`) frente a una reducción local (`reduceByKey`).
Mes 6	**PySpark DataFrames y Formatos Columnar** Cap. 07 a 12 (Ingesta y Joins)	• **Semana 21:** Formato Apache Parquet: compresión Snappy/Zstandard, estadísticas de columnas (*min/max pushdown*) y particionamiento físico. • **Semana 22:** DataFrames API: transformaciones optimizadas por Catalyst Optimizer y Tungsten Engine. • **Semana 23:** Joins distribuidos: Shuffle Hash Join, Sort Merge Join y optimización con Broadcast Join (`broadcast()`). • **Semana 24:** Entregable Mes 6: Pipeline en PySpark que ingesta logs crudos, aplica broadcast join con una tabla dimensional y persiste en Parquet particionado por fecha.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 7	**Lakehouse, Delta Lake y Modelado** Cap. 17 (Medallón) y 18 (Delta Lake)	• **Semana 25:** Arquitectura Medallón: definición formal de capas Bronze (crudo), Silver (conformado/limpio) y Gold (agregado de negocio). • **Semana 26:** Modelado dimensional Kimball: tablas de hechos (*Facts*), dimensiones conformadas, claves subrogadas y dimensiones que cambian lentamente (SCD Tipo 1 y 2). • **Semana 27:** Delta Lake transaccional: registro `_delta_log`, ACID con `MERGE INTO`, Time Travel y optimización física con `OPTIMIZE` y `Z-ORDER`. • **Semana 28:** Entregable Mes 7: Pipeline Lakehouse completo Bronze → Silver → Gold implementando SCD Tipo 2 idempotente con `MERGE INTO` en Delta Lake.
Mes 8	**Orquestación con Airflow y Modelado con dbt** Cap. 23, 23A (Airflow) y 24 (dbt)	• **Semana 29:** Apache Airflow: arquitectura (Scheduler, Webserver, Triggerer, Workers), operadores básicos y diseño de DAGs idempotentes. • **Semana 30:** Manejo avanzado de dependencias: `TaskGroups`, reintentos automáticos, SLAs, conexiones seguras y variables en Airflow. • **Semana 31:** dbt Core: estructura de proyecto, modelos staging, intermediate y marts; documentación interactiva y linaje automático. • **Semana 32:** Entregable Mes 8: DAG de Airflow orquestando un proyecto dbt que ejecuta tests de unicidad y no-nulidad antes de actualizar los marts de analítica.
Mes 9	**DataOps, Calidad y Contratos** Cap. 26, 26A (CI/CD) y 28 (GX/Contratos)	• **Semana 33:** Contratos de datos (*Data Contracts*): definición de especificaciones semánticas y cuarentena de registros inválidos (*Dead-Letter Queue*). • **Semana 34:** Validación automatizada con Great Expectations / Soda / Pydantic Contracts: assertions de negocio y generación de reportes de calidad. • **Semana 35:** CI/CD para datos: GitHub Actions ejecutando linters (`ruff`, `sqlfluff`) y pruebas unitarias con Testcontainers en cada Pull Request. • **Semana 36:** Entregable Mes 9: Pipeline con GitHub Action que bloquea el merge si los esquemas de datos entrantes violan el contrato establecido.

Mes	Eje Temático y Capítulos	Entregables Semanales Requeridos
Mes 10	**Infraestructura Cloud Básica e IaC** Cap. 13 (AWS), 25 (Terraform)	• **Semana 37:** Fundamentos de AWS para datos: S3, IAM (principio de menor privilegio), Glue Data Catalog y Athena. • **Semana 38:** Infraestructura como Código: aprovisionamiento de buckets S3, roles IAM y bases de datos con Terraform modular. • **Semana 39:** Gestión de costos (FinOps): alertas de presupuesto de $5 USD en AWS, ciclos de vida de objetos en S3 y destrucción con `terraform destroy`. • **Semana 40:** Entregable Mes 10: Módulo Terraform reutilizable que levanta un Data Lakehouse en S3 con políticas IAM estrictas y alerta de costo configurada.
Mes 11	**Construcción de los 3 Proyectos de Portafolio** Cap. 34 (Proyectos Portafolio)	• **Semana 41:** Proyecto 1 (Local Zero-Cost): DuckDB + dbt + Airflow + Streamlit documentado de punta a punta. • **Semana 42:** Proyecto 2 (API Ingestion & Lakehouse): Extractor API con backoff → S3/MinIO → PySpark → Delta Lake con cuarentena de datos. • **Semana 43:** Proyecto 3 (Streaming o Cloud): Kafka básico o pipeline serverless en AWS/Databricks con métricas de rendimiento y costos. • **Semana 44:** Entregable Mes 11: Tres repositorios públicos en GitHub con README profesional, diagramas Mermaid, instrucciones `docker compose up` y videos demo.
Mes 12	**Tramo de Empleo y Procesos de Selección** Cap. 36 (Estrategia Laboral)	• **Semana 45:** Optimización de CV en formato X-Y-Z bilingüe (español/inglés) adaptado a sistemas ATS, perfil de LinkedIn y presencia en GitHub. • **Semana 46:** Práctica intensiva de Live-Coding: 50 problemas de SQL analítico (LeetCode/StrataScratch) y 30 ejercicios de algoritmos en Python. • **Semana 47:** Simulacros de entrevista de System Design de datos y resolución de pruebas para llevar a casa (*Take-Home Assignments*). • **Semana 48:** Entregable Mes 12: Estrategia de postulación activa (10 postulaciones semanales con mensajes personalizados y búsqueda de referidos internos).

Listas de Verificación: «Ya Puedo Hacer Esto» por Hito

Utilice estas listas de control como su estándar de autoevaluación. No avance al siguiente bloque formativo hasta haber marcado afirmativamente cada competencia mediante evidencia en su propio código:

Hito 1: Cimientos y Junior Empleable (Checklist de 12 Competencias)

- ☐ **1. Control de Entorno:** Sé clonar repositorios, resolver conflictos de ramas en Git y estructurar proyectos en Python usando `uv` y entornos virtuales limpios.
- ☐ **2. Contenedores:** Sé escribir un `Dockerfile` optimizado y un `docker-compose.yml` para levantar PostgreSQL, MinIO y servicios locales con variables de entorno protegidas en `.env`.
- ☐ **3. Ingesta Robusta:** Sé construir un script en Python que consuma una API REST con paginación, maneje códigos HTTP 429 con retroceso exponencial (*exponential backoff*) y valide tipos con Pydantic.
- ☐ **4. SQL Analítico:** Sé escribir consultas usando funciones de ventana (`ROW_NUMBER()`, `LEAD()`, `LAG()`, `DENSE_RANK()`) para deduplicar registros y calcular métricas acumuladas.
- ☐ **5. Motores Modernos:** Sé utilizar DuckDB para consultar archivos Parquet directamente desde disco o S3 sin requerir una base de datos externa.
- ☐ **6. Computación Distribuida:** Comprendo la diferencia entre transformaciones estrechas y anchas en Apache Spark, y sé explicar cómo y cuándo ocurre un *Shuffle*.
- ☐ **7. PySpark en Práctica:** Sé cargar datos en PySpark, aplicar esquemas estrictos (`StructType`), realizar transformaciones limpias y ejecutar optimizaciones mediante *Broadcast Joins*.
- ☐ **8. Almacenamiento Lakehouse:** Sé operar tablas Delta Lake, ejecutar operaciones `MERGE INTO` para actualización idempotente (upserts) y consultar versiones históricas con Time Travel.
- ☐ **9. Modelado Dimensional:** Sé diseñar un esquema en estrella (*Star Schema*) con tablas de hechos transaccionales y dimensiones con manejo de SCD Tipo 2.
- ☐ **10. Orquestación:** Sé crear un DAG en Apache Airflow con dependencias claras, reintentos configurados y tareas completamente idempotentes.
- ☐ **11. Calidad y Contratos:** Sé definir reglas de calidad que aíslen registros con errores en una cola de cuarentena (*Dead-Letter Queue*) sin detener el pipeline.
- ☐ **12. Portafolio Demostrable:** Cuento con un repositorio público en GitHub con CI/CD automatizado, tests con `pytest` y documentación clara que cualquier reclutador puede clonar y ejecutar.

Hito 2: Especialización Mid-Level (Checklist de 10 Competencias)

- ☐ **1. Optimización Spark Avanzada:** Sé diagnosticar problemas de sesgo de datos (*Data Skew*) mediante el Spark UI y resolverlos aplicando técnicas de *Salting* o *Liquid Clustering*.
- ☐ **2. Event Streaming:** Sé configurar un productor y consumidor en Apache Kafka, diseñar esquemas con Schema Registry y explicar la semántica *At-Least-Once* vs. *Exactly-Once*.

- ☐ **3. Spark Streaming:** Sé procesar flujos continuos en PySpark Structured Streaming aplicando *Watermarking* para gestionar datos desordenados o tardíos.
- ☐ **4. Infraestructura como Código:** Sé escribir plantillas de Terraform modulares para aprovisionar buckets, políticas IAM de menor privilegio y clústeres analíticos.
- ☐ **5. CI/CD Empresarial:** Sé construir pipelines en GitHub Actions que ejecuten pruebas de integración usando Testcontainers y validen calidad de código con linters antes del despliegue.
- ☐ **6. Modelado con dbt:** Sé estructurar proyectos dbt con Jinja macros, paquetes reutilizables, snapshots para SCD2 y pruebas genéricas/singulares.
- ☐ **7. Nube Pública:** Domino los servicios esenciales de al menos una nube líder (AWS S3/Glue/Athena o Azure Data Lake/Data Factory o GCP GCS/BigQuery).
- ☐ **8. FinOps y Costos:** Sé implementar políticas de retención, particionado eficiente y monitoreo de costos para garantizar la sostenibilidad financiera de la plataforma.
- ☐ **9. Contratos de Datos:** Sé acordar e implementar data contracts semánticos entre equipos de software transaccional y el equipo de datos.
- ☐ **10. Resolución de Incidentes:** Sé interpretar trazas de error complejas (OOM, Garbage Collection pauses, Deadlocks) y redactar análisis post-incidente (*postmortems*).

Hito 3: Nivel Senior y Liderazgo Técnico (Checklist de 8 Competencias)

- ☐ **1. System Design Masivo:** Sé liderar entrevistas de arquitectura de datos diseñando sistemas capaces de procesar decenas de terabytes diarios bajo estrictos SLAs de latencia.
- ☐ **2. Trade-offs de Arquitectura:** Sé justificar técnicamente la elección entre Lambda vs. Kappa, Delta vs. Iceberg, y Batch vs. Micro-batch según costos y requisitos del negocio.
- ☐ **3. Gobierno y Seguridad:** Sé diseñar políticas de cifrado en reposo y en tránsito (mTLS, KMS), control de acceso basado en roles (RBAC/ABAC) y cumplimiento normativo (GDPR / Ley 1581).
- ☐ **4. Observabilidad de Plataforma:** Sé instrumentar pipelines con OpenLineage, métricas Prometheus y trazas distribuidas para monitorear la salud de extremo a extremo.
- ☐ **5. Gestión de Guardias On-Call:** Sé diseñar esquemas de rotación on-call, manuales de respuesta rápida (*runbooks*) y matrices de escalamiento de severidad.
- ☐ **6. Datos para Inteligencia Artificial:** Sé diseñar canalizaciones para ingestión masiva de embeddings, integración con bases vectoriales y feature stores para inferencia en tiempo real.
- ☐ **7. Mentoría y Cultura de Ingeniería:** Sé guiar a ingenieros junior y mid-level, realizar revisiones de código de alto estándar y promover la cultura de *blameless postmortems*.
- ☐ **8. Alineación de Negocio:** Sé traducir necesidades estratégicas de directores ejecutivos en hojas de ruta técnicas viables y cuantificables en retorno de inversión (ROI).

Rúbrica de Autoevaluación de Proyectos de Portafolio

Todo proyecto incluido en su portafolio de GitHub debe evaluarse con la siguiente matriz. Para considerar un proyecto listo para postulación laboral, **debe alcanzar un puntaje mínimo de Nivel Producción (4 o 5 puntos) en las cinco dimensiones**:

Dimensión Evaluada	Insuficiente (1–2 Puntos)	Aceptable (3 Puntos)	Nivel Producción Senior (4–5 Puntos)
1. Idempotencia y Reejecución	El pipeline falla si se corre dos veces seguidas o duplica filas en la tabla de destino (`INSERT` ciego).	El pipeline usa `TRUNCATE` y recarga todo, o requiere borrar archivos manualmente antes de volver a correr.	El pipeline usa `MERGE INTO`, particionamiento determinista o reemplazo dinámico. Se puede reejecutar N veces con idéntico resultado sin duplicados.
2. Pruebas Automatizadas y Calidad	No hay pruebas unitarias ni validación de esquemas. El código se prueba corriendo el script completo a mano.	Tiene 2 o 3 pruebas con `pytest` para funciones básicas, pero no valida el esquema ni la calidad de los datos de entrada.	Suite completa con `pytest` (>85% cobertura), tipado estático (`mypy`), validación con Pydantic/Great Expectations y aislamiento a cuarentena (*Dead-Letter Queue*).
3. Infraestructura y Reproducibilidad	«Funciona en mi máquina». Requiere instalar múltiples herramientas en el host sin versiones fijas.	Incluye un `Dockerfile` básico, pero faltan servicios auxiliares o requiere configurar credenciales manuales.	Reproducible en un solo comando: `docker compose up -d`. Usa `.env.example`, versiones fijadas, dependencias bloqueadas con `uv.lock` y scripts de generación de datos.
4. Documentación y Arquitectura (README)	README vacío o generado por plantilla. Solo contiene el título del proyecto y dos líneas de texto.	Describe qué hace el código y lista las tecnologías usadas, pero carece de diagramas de arquitectura y métricas.	Diagrama de flujo en Mermaid, justificación técnica de decisiones de diseño, instrucciones paso a paso probadas y métricas cuantificables de rendimiento y costos.
5. Manejo de Errores y Observabilidad	El script aborta inmediatamente con excepciones no controladas; no hay logs ni alertas.	Usa bloques genéricos `try/except: pass` y mensajes `print()` simples por consola.	Logging estructurado (`loguru`/`logging` estándar con formato JSON), captura de errores con retries automáticos, alertas de fallo y aislamiento de registros corruptos.

0.5 Mapa Pragmático de Certificaciones Oficiales

Las certificaciones no sustituyen a los proyectos demostrables en código, pero actúan como un filtro clave en plataformas de reclutamiento automatizado (ATS) para validar el conocimiento formal de un estándar.

> **Estrategia Recomendada para Principiantes:**
> **Concéntrese en un máximo de UNA certificación oficial para obtener su primer empleo.** Intentar aprobar tres o cuatro certificaciones simultáneas mientras se parte de cero dispersa el esfuerzo y genera un costo económico innecesario. Obtenga una credencial clave y reserve las certificaciones especializadas para cuando su empleador financie los exámenes.

Certificación Oficial	Proveedor	Costo Oficial	Nivel Sugerido	Enfoque Principal
AWS Certified Data Engineer – Associate (DEA-C01)	AWS	$150 USD	Junior - Mid	*(Recomendada para primer empleo).* Ingesta, S3, Glue, EMR, Athena, Redshift y seguridad KMS/IAM.
Databricks Certified Data Engineer Associate	Databricks	$200 USD	Junior - Mid	*(Recomendada para primer empleo).* Apache Spark, PySpark, Delta Lake, Lakehouse y Lakeflow básico.
dbt Certified Developer	dbt Labs	$200 USD	Junior - Mid	Modelado dimensional Kimball, Jinja, macros, tests genéricos y snapshots.
Microsoft Fabric Analytics Engineer Associate (DP-600)	Microsoft	$165 USD	Junior - Mid	OneLake, modelos semánticos de Power BI, Delta Lake y T-SQL analítico.
Microsoft Fabric Data Engineer Associate (DP-700)	Microsoft	$165 USD	Junior - Mid	Ingesta Fabric, Data Factory, Lakehouse Spark y orquestación empresarial. *(Sustituye al retirado DP-203).*
SnowPro Core Certification	Snowflake	$175 USD	Junior - Mid	Almacenamiento columnar en micro-particiones, virtual warehouses, Time Travel y Roles.
HashiCorp Certified: Terraform Associate (003)	HashiCorp	$70.50 USD	Mid	Infraestructura como Código (IaC), State Management, módulos y proveedores cloud.
Databricks Certified Data Engineer Professional	Databricks	$200 USD	Mid - Senior	Optimización física de Spark UI, Unity Catalog empresarial, streaming y CI/CD avanzado.
Google Cloud Professional Data Engineer (GCP PDE)	Google Cloud	$200 USD	Mid - Senior	BigQuery a escala masiva, Apache Beam en Cloud Dataflow y Cloud Composer.

0.6 Construcción del Repositorio de Portafolio en GitHub

Para captar el interés de comités técnicos y líderes de ingeniería, el portafolio en GitHub debe alejarse radicalmente de notebooks aislados de Kaggle o scripts lineales sin pruebas. Debe reflejar la estructura de un repositorio de producción corporativo:

```
repositorio-lakehouse-produccion/
├── .github/
```

```
|   └── workflows/
|       └── ci_cd.yml                # Pipeline CI/CD: linter, tests y validación
IaC
├── infrastructure/
|   └── terraform/                    # Infraestructura como código (S3, IAM,
Clústeres)
|       ├── main.tf
|       └── variables.tf
├── src/
|   ├── ingestion/                    # Clientes API con backoff o productores
Kafka
|   |   ├── api_client.py
|   |   └── schemas.py
|   ├── transformations/              # Pipelines modulares en PySpark o DuckDB
|   |   ├── bronze_to_silver.py
|   |   └── silver_to_gold.py
|   └── quality/                      # Suites de validación y aislamiento de
cuarentena
|       └── data_contracts.py
├── dbt_project/                      # Modelado dimensional de capa Gold
|   ├── models/
|   └── dbt_project.yml
├── orchestrator/                     # DAGs de Apache Airflow
|   └── dags/
|       └── lakehouse_daily_dag.py
├── tests/                            # Pruebas unitarias e integración con pytest
|   ├── test_transformations.py
|   └── test_quality_rules.py
├── scripts/
|   └── data_generator.py             # Generador de datos sintéticos reproducible
├── docker-compose.yml                # Entorno local reproducible (Postgres,
MinIO, Spark)
├── pyproject.toml                    # Gestión moderna de dependencias con uv
└── README.md                         # Documentación técnica ejecutiva y
arquitectura
```

Elementos Críticos del `README.md` de Alto Impacto

1. **Diagrama de Arquitectura Claro:** Diagramas en Mermaid o esquemas gráficos que ilustren el flujo de datos desde el origen operacional hasta el consumo analítico.
2. **Justificación Técnica de Decisiones:** Explicar con rigor *por qué* se eligió Delta Lake frente a Parquet plano, o por qué se empleó una ventana móvil frente a un re-cálculo total.
3. **Métricas Tangibles de Rendimiento y Costo:** Demostrar el impacto de la optimización (ej. *«Se redujo el volumen de shuffle en un 65% mediante broadcast joins y salting, recortando el tiempo de ejecución de 40 a 14 minutos»*).
4. **Reproducibilidad en un Solo Comando:** Cualquier ingeniero debe poder clonar el repositorio, ejecutar `docker compose up -d` y reproducir los pipelines localmente sin configuraciones manuales complejas.

0.7 Estrategia de Preparación para Entrevistas Técnicas

Los procesos de selección en empresas internacionales se articulan habitualmente en cuatro fases técnicas diferenciadas:

1. Prueba de SQL en Vivo (Live-Coding)

- **Temas evaluados:** Funciones de ventana avanzadas (`ROW_NUMBER()` para deduplicación, `DENSE_RANK()`, `LEAD/LAG`), agregaciones condicionales, uniones relacionales complejas y expresiones de tabla comunes (CTEs) legibles.
- **Criterio de evaluación:** Corrección del resultado, manejo impecable de valores `NULL`, legibilidad y consideración del costo computacional del plan de ejecución.

2. Algoritmos y Manipulación de Datos en Python

- **Temas evaluados:** Procesamiento en memoria mediante generadores (`yield`), manejo de estructuras JSON anidadas, operaciones en $O(N)$ o $O(N \log N)$ y agrupaciones sin bibliotecas externas.
- **Criterio de evaluación:** Selección eficiente de estructuras de datos (diccionarios y conjuntos para búsquedas en tiempo constante $O(1)$), tipado estático y cobertura de casos límite (*edge cases*).

3. Computación Distribuida y Fundamentos de Arquitectura

- **Preguntas clásicas:** *«¿Qué etapas componen exactamente una fase de Shuffle en Spark?»*, *«¿Cómo diagnosticarías y resolverías un fallo OutOfMemory por sesgo de datos (Data Skew)?»*, *«¿Cómo se garantiza la semántica exactly-once al persistir eventos de Kafka en Delta Lake?»*.
- **Criterio de evaluación:** Claridad conceptual sobre la relación entre memoria RAM, discos locales, red y recolectores de basura en la JVM.

4. Entrevista de System Design de Datos (Nivel Senior)

- Ejercicio de 45 a 60 minutos donde el candidato lidera el diseño de una plataforma completa a escala (ej. *«Diseñe la plataforma analítica de telemetría para 500 millones de dispositivos IoT»*).
- **Estructura para responder:**
  1. *Aclaración de Requisitos y Alcance (5 min):* Delimitar requerimientos funcionales y no funcionales (latencia, frescura, throughput).
  2. *Estimación Numérica de Escala (5 min):* Calcular volumen de almacenamiento diario/anual, ancho de banda de red y memoria RAM.

3. *Diseño de Alto Nivel (15 min):* Trazar el diagrama modular (Ingesta → Mensajería → Procesamiento → Almacén → Consumo).
4. *Inmersión Profunda en Componentes Críticos (15 min):* Justificar la selección de tecnologías, estrategias de particionado y políticas de tolerancia a fallos.
5. *Manejo de Errores y Cuellos de Botella (5 min):* Diseñar recuperación ante caídas de nodos y monitoreo de salud del sistema.

0.8 El Inglés Técnico como Multiplicador Profesional

En el mercado global de datos, el dominio del **inglés técnico profesional** es el principal multiplicador salarial y el puente de acceso a posiciones remotas internacionales pagadas en dólares estadounidenses:

- **Acceso a la Verdad Técnica:** La documentación de proyectos de la Apache Software Foundation, especificaciones RFCs, discusiones de arquitectura en GitHub y literatura fundacional se publican exclusivamente en inglés.
- **Vocabulario Operativo Estandarizado:** Los debates cotidianos de producción giran en torno a términos en inglés (*shuffling, stragglers, backpressure, idempotency, data drift, watermarking, tombstoning, dead-letter queues*). Dominar esta terminología con soltura y precisión es un indicador inmediato de madurez profesional.

0.9 Resumen y Puntos Clave

- **De consumidores a constructores:** El Data Engineer no consume datos para armar gráficos ejecutivos; construye las plataformas escalables, idempotentes y observables sobre las cuales opera la corporación.
- **Rutas personalizadas:** El aprendizaje debe adaptarse al punto de partida técnico (desde cero absoluto, analista en transición o desarrollador backend) priorizando los capítulos clasificados como `[NÚCLEO EMPLEABILIDAD]`.
- **Cimientos físicos:** La física del hardware impone que la memoria RAM sea 100.000 veces más rápida que el disco magnético y que la transferencia por red sea el factor más costoso. Todo diseño distribuido debe minimizar transferencias innecesarias.
- **Pacing honesto:** La transición desde cero hasta la empleabilidad junior requiere entre 9 y 12 meses de estudio deliberado (12–15 horas/semana); el nivel senior consolidado exige entre 4 y 6 años de experiencia productiva real.

- **Estrategia pragmática:** Enfóquese en dominar los cimientos y obtener un máximo de una certificación inicial clave respaldada por un portafolio reproducible en GitHub con Docker Compose, CI/CD y pruebas unitarias.

Ejercicios de Revisión

Ejercicio 0.1: Estimación de Transferencia y Física de Latencia

Suponga que una consulta en Apache Spark sobre un clúster sin optimizar provoca un *Shuffle* no intencionado que redistribuye 120 GB de datos intermedios a través de una red local con un ancho de banda efectivo disponible de 1 Gbps (gigabit por segundo).

1. Calcule el tiempo teórico mínimo que consumirá únicamente la transferencia de red de dicho Shuffle (sin contar serialización ni escritura a disco).
2. Compare dicho tiempo con el acceso a 120 GB si los datos ya estuvieran en memoria RAM local con un ancho de banda de bus de 40 GB/s.

Solución Paso a Paso:

1. **Conversión de unidades y tiempo de red:**
   $120\ \text{GB} = 120 \times 8\ \text{gigabits} = 960\ \text{Gigabits}$.
   A un ancho de banda de 1 Gbps:

$$\text{Tiempo de red} = \frac{960\ \text{Gb}}{1\ \text{Gbps}} = 960\ \text{segundos} = 16\ \text{minutos}$$

2. **Tiempo de acceso en bus de memoria RAM:**

$$\text{Tiempo en RAM} = \frac{120\ \text{GB}}{40\ \text{GB/s}} = 3\ \text{segundos}$$

3. **Conclusión de ingeniería:** La transferencia de red tarda 960 segundos frente a 3 segundos en bus local (un factor de más de $300\times$ más lento). Esta disparidad ilustra por qué un broadcast join o un particionamiento adecuado transforma consultas de 20 minutos en procesos de pocos segundos.

Lecturas Recomendadas

1. **Reis, J. & Housley, M. (2022).** *Fundamentals of Data Engineering: Plan and Build Robust Data Systems.* O'Reilly Media.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems.* O'Reilly Media.
3. **AWS Training and Certification (2024).** *AWS Certified Data Engineer – Associate (DEA-C01) Exam Guide.* Amazon Web Services.
4. **Databricks Learning (2024).** *Databricks Certified Data Engineer Associate Exam Study Guide.* Databricks Inc.

CAPÍTULO 0A

Cimientos de Plataforma: Linux, Git, Docker y Redes NÚCLEO

> *"Un ingeniero de datos que no domina la terminal de Linux, el aislamiento de contenedores en Docker y el control de versiones con Git es como un cirujano que desconoce la anatomía básica: puede saber qué operación desea realizar, pero carece de la destreza instrumental para manipular el sistema sin provocar un desastre en producción."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Operar con fluidez en entornos **Linux** mediante la línea de comandos, dominando tuberías, redirecciones de flujo y herramientas de procesamiento de texto (`grep`, `sed`, `awk`, `jq`).
- Automatizar tareas periódicas en el sistema operativo mediante el demonio `cron` y gestionar procesos y memoria (`ps`, `top`, `kill`, `systemd`).
- Implementar flujos de trabajo profesionales con **Git** (estrategias de ramificación, Conventional Commits, `rebase` interactivo frente a `merge` y resolución limpia de conflictos).
- Diseñar contenedores reproducibles con **Docker**, optimizando imágenes mediante construcciones multi-etapa (*multi-stage builds*), volúmenes persistentes y redes aisladas.
- Orquestar entornos locales de ingeniería de datos compuestos por múltiples servicios mediante **Docker Compose**.
- Comprender los conceptos esenciales de **redes**: direccionamiento CIDR, subredes, puertos, resolución DNS, cortafuegos (*Security Groups*), VPCs y cifrado TLS/mTLS en tránsito.

> **Definición 0A.1 (La Pila de Ejecución del Data Engineer)** En la arquitectura de sistemas distribuidos, la infraestructura no es un detalle abstracto delegado exclusivamente al equipo de operaciones. Todo pipeline de datos —sea un clúster de Spark sobre Kubernetes, un broker de Kafka o un contenedor de Airflow— se ejecuta sobre núcleos **Linux**, se versiona en **Git**, se empaqueta en imágenes de **Docker** y se comunica a través de **redes TCP/IP**.
>
> La estabilidad y el rendimiento de un pipeline dependen directamente de cómo interactúa el código de datos con el sistema operativo anfitrión y los límites de recursos de CPU, memoria y sockets de red.

0A.1 Linux y Terminal para Ingeniería de Datos

El 99% de las plataformas de datos distribuidas se ejecutan sobre distribuciones Linux (Ubuntu, Debian, Red Hat Enterprise Linux, Alpine). La terminal no es una alternativa a la interfaz gráfica: es la interfaz nativa de automatización.

1. Flujos Estándar, Redirecciones y Tuberías (*Pipes*)

En Linux, todo proceso interactúa con tres descriptores de archivo estándar:

- `stdin` (Descriptor 0): Entrada estándar de datos.
- `stdout` (Descriptor 1): Salida estándar de resultados.
- `stderr` (Descriptor 2): Salida estándar de errores y diagnósticos.

```bash
# Redirigir stdout a un archivo (sobrescribiendo)
python pipeline.py > salida.log

# Redirigir stdout y stderr a un archivo combinado (modo append)
python pipeline.py >> ejecucion.log 2>&1

# Conectar la salida de un proceso con la entrada de otro mediante Pipe (|)
cat transacciones_raw.csv | grep "COMPLETADO" | cut -d',' -f1,4 >
ventas_filtradas.csv
```

2. Procesamiento de Texto de Alto Rendimiento en Terminal

Antes de cargar gigabytes en Spark o Python, la terminal permite inspeccionar y transformar datos crudos en milisegundos:

- **`grep` (Búsqueda por patrones regex):**

```bash
# Buscar errores en logs ignorando mayúsculas/minúsculas y contando ocurrencias
grep -i "OutOfMemory" /var/log/spark/*.log | wc -l
```

- `sed` **(Editor de flujos para transformaciones y sustituciones):**

```bash
# Reemplazar delimitadores de punto y coma por comas en archivos CSV masivos
sed 's/;/,'/g transacciones.txt > transacciones_limpias.csv
```

- `awk` **(Procesador analítico de columnas estructuradas):**

```bash
# Sumar el valor de la columna 3 para registros cuyo país (columna 2) sea 'CO'
awk -F',' '$2 == "CO" { suma += $3 } END { print "Total Ventas Colombia: " suma }' ventas.csv
```

- `jq` **(El estándar de oro para manipular JSON en línea de comandos):**

```bash
# Extraer campos anidados y filtrar eventos de telemetría provenientes de una API
curl -s https://api.empresa.com/eventos | jq '.data[] | select(.monto > 500) | {id: .transaccion_id, valor: .monto}'
```

3. Permisos de Archivos y Administración de Procesos

- **Permisos POSIX:** El comando `chmod` altera los permisos para Usuario (u), Grupo (g) y Otros (o). En scripts ejecutables de producción:

```bash
chmod 750 run_ingest.sh   # rwxr-x--- (Lectura, escritura y ejecución para dueño; lectura y ejecución para grupo)
```

- **Procesos y Memoria:** Identificar y finalizar procesos colgados en servidores:

```bash
# Localizar el identificador de proceso (PID) de un driver de PySpark zombi
ps aux | grep "pyspark"

# Matar el proceso de forma forzada enviando la señal SIGKILL (9)
kill -9 <PID>

# Monitorear consumo de RAM, CPU y Swap en tiempo real
htop
```

4. Automatización con `cron` y Acceso Remoto con SSH

El demonio `cron` programa tareas periódicas mediante una sintaxis de 5 campos temporales (`minuto hora día-mes mes día-semana`):

```bash
# Editar el crontab del usuario actual
crontab -e

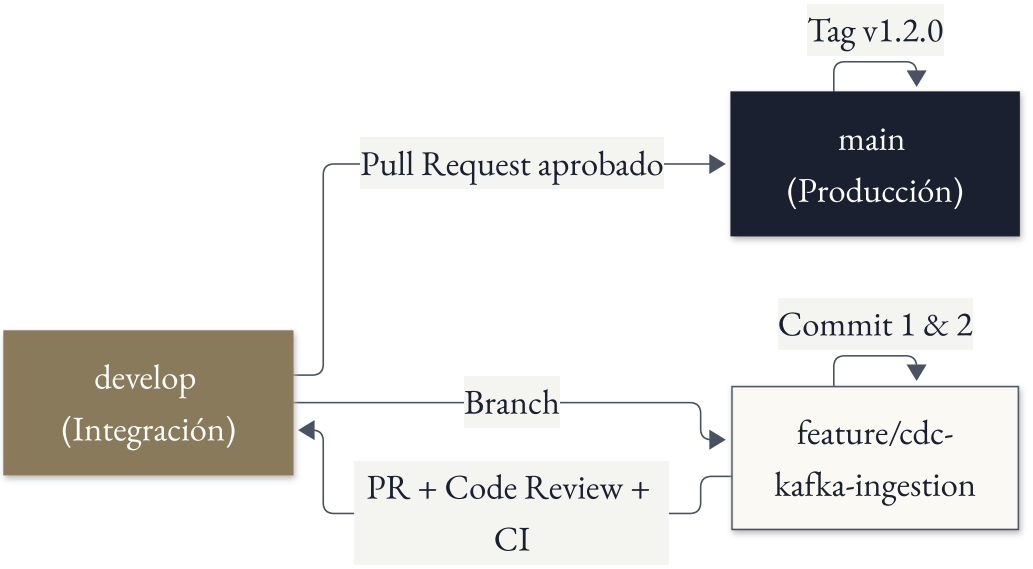
# Ejecutar el script de ingesta todos los días a las 02:30 AM UTC
30 2 * * * /usr/bin/bash /opt/pipelines/ingesta_diaria.sh >> /var/log/ingesta.log
2>&1
```

Para conectarse a instancias de cómputo en la nube sin exponer credenciales inseguras se utilizan pares de claves criptográficas SSH:

```bash
# Conexión segura usando clave privada RSA/Ed25519
ssh -i ~/.ssh/data_cluster_key.pem ubuntu@10.0.1.45
```

0A.2 Git Profesional y Estrategias de Colaboración

En la ingeniería de datos moderna, los pipelines son código fuente (*Pipelines as Code*). Toda alteración debe ser trazable, reproducible y sujeta a revisión por pares.



1. Estrategia de Ramas y Pull Requests (Trunk-Based vs GitFlow)

- **Trunk-Based Development:** Es la práctica recomendada para equipos ágiles de ingeniería de datos. Los ingenieros crean ramas de vida corta (`feat/nueva-fuente-ventas`) a partir de la rama principal (`main`), ejecutan pruebas automáticas y fusionan en cuestión de horas o días tras aprobación de un Pull Request (PR).
- **Conventional Commits:** Estandarizar los mensajes de confirmación permite generar changelogs automáticos y versionado semántico:
  - `feat: agregar particionado dinámico en carga de capa Silver`
  - `fix: corregir manejo de valores nulos en deduplicación de clientes`
  - `perf: optimizar broadcast join en cruce de catálogo de productos`
  - `test: agregar validaciones unitarias de esquema con Chispa`

2. `git rebase` interactivo frente a `git merge`

- `git merge`: Crea un commit de unión explícito que une dos historiales. Puede generar historiales enmarañados si se abusa en ramas locales.
- `git rebase`: Reescribe el historial aplicando los commits locales encima de la última versión de la rama base, manteniendo un historial lineal y limpio.

```bash
# Actualizar la rama local con los últimos cambios de main de forma lineal
git checkout feat/nueva-fuente
git fetch origin
git rebase origin/main

# Limpiar y condensar (squash) commits locales antes de abrir el PR
git rebase -i HEAD~3
```

0A.3 Docker y Contenedores para Plataformas de Datos

La clásica excusa *"en mi máquina local sí funcionaba"* es inaceptable en producción. Un contenedor encapsula la aplicación junto con sus bibliotecas de sistema, intérprete de Python, dependencias de Java (JARs de Spark/Hadoop) y variables de entorno, garantizando ejecución idéntica en local, en Kubernetes y en la nube.

1. Dockerfile Multi-Etapa (*Multi-Stage Build*)

Las construcciones multi-etapa permiten compilar dependencias en una imagen temporal pesada y copiar únicamente los binarios limpios a la imagen final ligera, reduciendo el tamaño de la imagen de 1.5 GB a menos de 200 MB y mitigando vulnerabilidades de seguridad:

```dockerfile
# Etapa 1: Constructor (Builder)
FROM python:3.11-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Instalar uv para resolución ultrarrápida de dependencias
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

COPY pyproject.toml ./
RUN uv pip install --system --no-cache -r pyproject.toml

# Etapa 2: Imagen Final de Producción (Runtime)
FROM python:3.11-slim AS runtime

WORKDIR /app

# Instalar Java OpenJDK para controladores locales de PySpark
RUN apt-get update && apt-get install -y --no-install-recommends \
    openjdk-17-jre-headless \
    curl \
```

```
    && rm -rf /var/lib/apt/lists/*

ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
ENV PATH="${JAVA_HOME}/bin:${PATH}"

# Copiar paquetes instalados desde el builder
COPY --from=builder /usr/local/lib/python3.11/site-packages
/usr/local/lib/python3.11/site-packages
COPY src/ /app/src/

# Usuario sin privilegios por seguridad
RUN useradd -m -u 1001 datauser
USER datauser

ENTRYPOINT ["python", "-m", "src.pipelines.main"]
```

2. Volúmenes Persistentes y Redes Aisladas

- **Volúmenes (`-v` o `volumes:`):** Los contenedores son efímeros por diseño; si un contenedor de PostgreSQL se destruye, los datos almacenados en su interior desaparecen. Los volúmenes montan directorios del host o almacenamiento administrado por Docker para persistir datos.
- **Redes Bridge (`networks:`):** Permiten la comunicación interna entre contenedores por nombre de servicio mediante DNS interno de Docker, sin necesidad de exponer puertos al exterior de la máquina física.

0A.4 Composición de Infraestructura Local con Docker Compose

Para reproducir localmente un laboratorio de ingeniería de datos (con una base transaccional PostgreSQL y un almacenamiento de objetos S3 local compatible mediante MinIO), se utiliza **Docker Compose**:

YAML

```
version: '3.8'

services:
  # Base de datos relacional operacional (OLTP)
  postgres_oltp:
    image: postgres:16-alpine
    container_name: lab_postgres
    environment:
      POSTGRES_DB: transaccional_db
      POSTGRES_USER: ${POSTGRES_USER:-admin_datos}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD} # Inyectado de forma segura desde
archivo .env local
    env_file:
      - .env
```

```
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - red_datos

  # Almacenamiento de objetos local S3-compatible (Data Lake)
  minio_s3:
    image: minio/minio:latest
    container_name: lab_minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minio_admin
      MINIO_ROOT_PASSWORD: MinioSecretKey2026!
    ports:
      - "9000:9000"   # Puerto API S3
      - "9001:9001"   # Consola Web UI
    volumes:
      - miniodata:/data
    networks:
      - red_datos

volumes:
  pgdata:
  miniodata:

networks:
  red_datos:
    driver: bridge
```

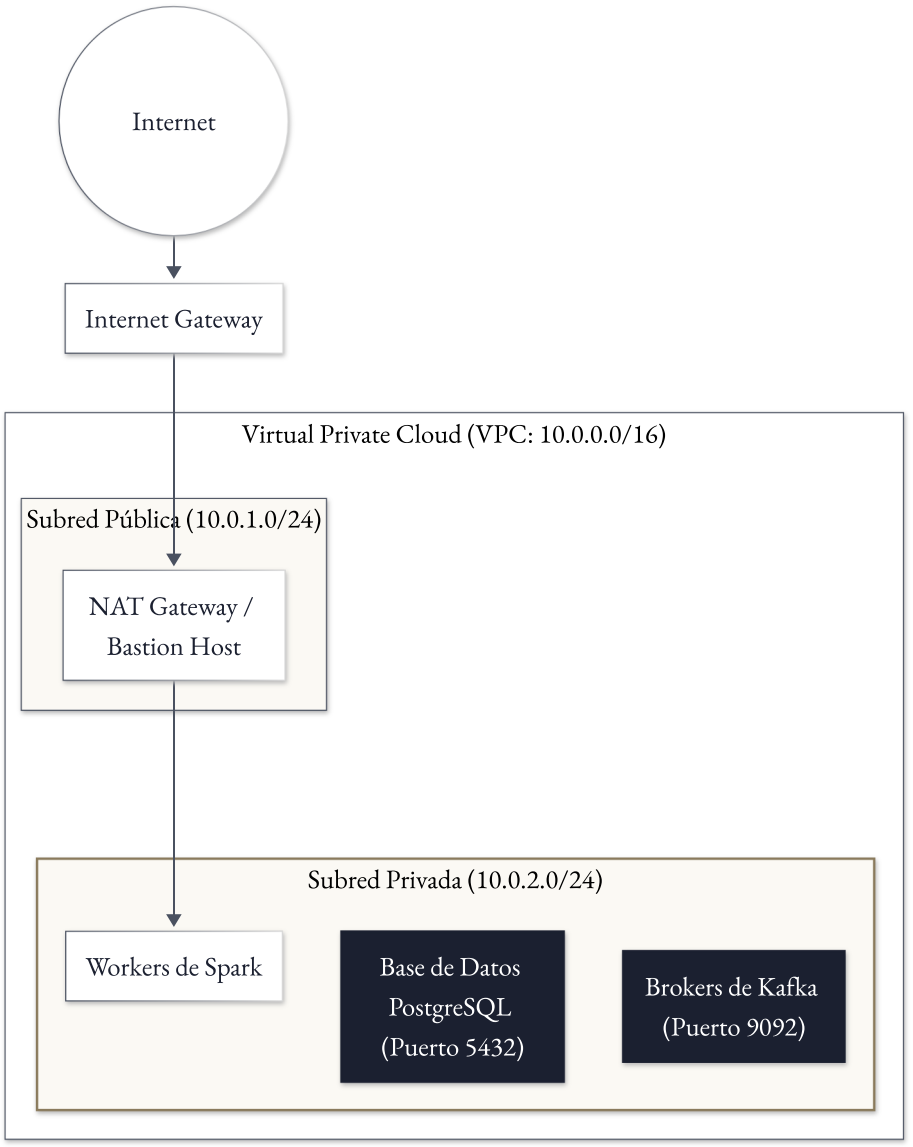
Para levantar todo el ecosistema de laboratorio en segundo plano, basta ejecutar un único comando en la terminal:

```
BASH
```

```
docker compose up -d
```

0A.5 Redes Básicas y Seguridad para Plataformas de Datos

Todo sistema distribuido opera sobre redes de comunicaciones. Conocer la arquitectura de redes previene fallos de conectividad comunes en la nube:



1. Direccionamiento CIDR y Subredes

- **CIDR (*Classless Inter-Domain Routing*):** Representa bloques de direcciones IP mediante la notación `IP/máscara`.
  - `10.0.0.0/16`: Red privada con $2^{32-16} = 65.536$ direcciones IP disponibles (espacio típico para una VPC).
  - `10.0.1.0/24`: Subred con $2^{32-24} = 256$ direcciones IP.
- **Aislamiento en Subredes Privadas:** Los clústeres de Spark, bases de datos y brokers de Kafka **nunca deben tener direcciones IP públicas**. Deben residir en **subredes privadas**,

comunicándose hacia internet exclusivamente a través de un **NAT Gateway** y recibiendo tráfico administrativo mediante un *Bastion Host* (Jump Server) o túneles VPN.

2. Cortafuegos de Red (*Security Groups*) y Puertos Estándar

Los cortafuegos definen reglas de entrada (*Inbound*) y salida (*Outbound*) basadas en protocolos y puertos:

- `Puerto 22`: SSH (Acceso a terminal segura).
- `Puerto 443`: HTTPS / TLS (Consumo de APIs y transferencia de objetos S3/ADLS).
- `Puerto 5432`: PostgreSQL.
- `Puerto 9092`: Apache Kafka.
- `Puerto 7077 / 4040`: Apache Spark Master y Spark Web UI.
- **Regla de Mínimo Privilegio:** Jamás abra un puerto de base de datos a `0.0.0.0/0` (todo internet). Restrinja la regla al bloque CIDR exacto de los nodos ejecutores de Spark.

3. Cifrado en Tránsito: TLS y mTLS

- **TLS (*Transport Layer Security*):** Cifra los datos transmitidos entre el cliente y el servidor mediante certificados criptográficos, impidiendo ataques de interceptación (*Man-In-The-Middle*).
- **mTLS (*Mutual TLS*):** Exige que **tanto el servidor como el cliente** presenten certificados válidos firmados por una Autoridad de Certificación (CA) de confianza. Es el estándar de oro para comunicar microservicios y productores con clústeres de Apache Kafka en arquitecturas de cero confianza (*Zero Trust*).

0A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutar Contenedores de Producción como Usuario `root`
Dejar que los procesos dentro del contenedor corran como `root` representa un riesgo grave de seguridad: un atacante que explote una vulnerabilidad en una biblioteca de Python podría escapar del contenedor y obtener privilegios totales sobre la máquina anfitriona (*host breakout*).
Corrección: Declare siempre `USER datauser` en su `Dockerfile`.

Anti-patrón 2: Almacenar Credenciales Secretas en Imágenes Docker o Repositorios Git
Escribir contraseñas de bases de datos o claves de acceso a AWS directamente en un `Dockerfile`, script o archivo de configuración versionado en Git es una de las principales causas de brechas de seguridad.

Corrección: Inyecte secretos en tiempo de ejecución mediante variables de entorno, gestores de secretos (AWS Secrets Manager, HashiCorp Vault) o herramientas de montaje seguro (`docker run --env-file .env`).

Anti-patrón 3: Ignorar el Archivo `.dockerignore` y `.gitignore`
Construir una imagen Docker sin un archivo `.dockerignore` copia accidentalmente entornos virtuales locales pesados (`.venv`), cachés de Python (`__pycache__`) y claves privadas a la imagen, aumentando el tiempo de construcción y exponiendo datos sensibles.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0A.1 (Filtrado y Agregación Eficiente con Herramientas de Terminal):
Se tiene un archivo de registro crudo de 20 GB denominado `conexiones_db.log` con el siguiente formato separado por espacios:
`TIMESTAMP IP_ORIGEN USUARIO ESTADO_CONEXION LATENCIA_MS`
Escriba un comando de terminal Linux de una sola línea (*one-liner*) que:

1. Filtre únicamente las conexiones fallidas (`ESTADO_CONEXION = 'FAILED'`).
2. Extraiga la dirección IP de origen (columna 2).
3. Calcule las 5 direcciones IP con mayor cantidad de intentos fallidos, ordenadas de mayor a menor.

Solución 0A.1:

```bash
grep "FAILED" conexiones_db.log | awk '{print $2}' | sort | uniq -c | sort -nr | head -n 5
```

Explicación Paso a Paso:

- `grep "FAILED"` descarta de inmediato millones de registros exitosos sin cargarlos en memoria.
- `awk '{print $2}'` proyecta únicamente la columna con la dirección IP.
- `sort` agrupa las IPs consecutivamente para que `uniq` pueda operar.
- `uniq -c` contabiliza las ocurrencias de cada IP única.
- `sort -nr` realiza un ordenamiento numérico (`-n`) en orden descendente (`-r`).
- `head -n 5` trunca la salida a las 5 principales fuentes de ataques o errores de conexión.

Ejercicio 0A.2 (Resolución de Conflictos en Git):
Dos ingenieros de datos modificaron simultáneamente el archivo `pipeline_ventas.py`. Al intentar hacer `git rebase origin/main`, Git detiene la ejecución notificando un conflicto en la línea donde se define la ruta de salida de almacenamiento. Explique el procedimiento exacto paso a paso para resolver el conflicto y completar el rebase sin perder cambios.

Solución 0A.2:

1. Identificar el archivo en conflicto ejecutando `git status`.
2. Abrir `pipeline_ventas.py` y localizar los marcadores de conflicto insertados por Git:

```python
<<<<<<< HEAD
RUTA_SALIDA = "s3://bucket-analitica/silver/ventas_v2/"
=======
RUTA_SALIDA = "s3://bucket-analitica/silver/ventas_particionadas/"
>>>>>>> feat/nueva-ruta
```

3. Editar el archivo manualmente conservando la lógica arquitectónica correcta y eliminando los marcadores `<<<<<<<`, `=======` y `>>>>>>>`.
4. Marcar el archivo como resuelto en el área de preparación (*staging*): `git add pipeline_ventas.py`.
5. Continuar el proceso de rebase ejecutando: `git rebase --continue` (¡Nunca ejecutar `git commit` durante un rebase activo!).

0A.7 Resumen y Puntos Clave

- **Linux es el sustrato universal:** Las herramientas nativas de terminal (`grep`, `sed`, `awk`, `jq`) permiten realizar transformaciones exploratorias y diagnósticos de fallos órdenes de magnitud más rápido que escribir scripts ad-hoc.
- **Git profesional:** Flujos basados en ramas cortas, Conventional Commits y `rebase` interactivo mantienen historiales limpios y trazables para auditorías de código.
- **Contenedores reproducibles:** Las imágenes multi-etapa de Docker minimizan la superficie de ataque y garantizan que los pipelines se ejecuten exactamente igual en cualquier entorno.
- **Redes y seguridad de datos:** Las plataformas analíticas deben residir en subredes privadas aisladas con reglas de cortafuegos de mínimo privilegio y canales cifrados mediante TLS/mTLS.

Lecturas Recomendadas

1. **Nemeth, E., Snyder, G., Hein, T. R., Whaley, B. & Mackin, D. (2017).** *UNIX and Linux System Administration Handbook (5th Edition).* Addison-Wesley.
2. **Chacon, S. & Straub, B. (2014).** *Pro Git (2nd Edition).* Apress (Disponible libre en línea).
3. **Mouat, A. (2015).** *Using Docker: Developing and Deploying Software with Containers.* O'Reilly Media.
4. **Kurose, J. F. & Ross, K. W. (2021).** *Computer Networking: A Top-Down Approach (8th Edition).* Pearson.

CAPÍTULO 0B

Bases de Datos Relacionales (OLTP) y Modelado Transaccional NÚCLEO

> *"Casi la totalidad de los datos analíticos que fluyen a través de un Lakehouse nacieron originalmente en una base de datos relacional orientada a transacciones. Si un ingeniero de datos no comprende los principios de normalización, la mecánica de los árboles B-Tree y los niveles de aislamiento transaccional, es incapaz de diseñar procesos de extracción e ingesta que no degraden o bloqueen los sistemas operacionales de la empresa."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la fundamentación matemática del modelo relacional y las formas normales canónicas (**1NF**, **2NF**, **3NF**).
- Distinguir las razones arquitectónicas por las cuales los sistemas **OLTP** normalizan rigurosamente sus esquemas mientras que los sistemas **OLAP** desnormalizan intencionadamente.
- Analizar la estructura interna de los índices **B-Tree** frente a índices **Hash**, comprendiendo su costo de mantenimiento en operaciones de escritura.
- Desglosar las cuatro propiedades **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad) y su implementación física en discos y diarios de transacciones (*Write-Ahead Logging / WAL*).
- Dominar los cuatro niveles de aislamiento transaccional estándar y diagnosticar las anomalías de concurrencia: *Dirty Reads*, *Non-Repeatable Reads*, *Phantom Reads* y *Write Skew*.
- Explicar el funcionamiento del Control de Concurrencia Multiversión (**MVCC**) en PostgreSQL y la necesidad de mantenimiento mediante `VACUUM`.

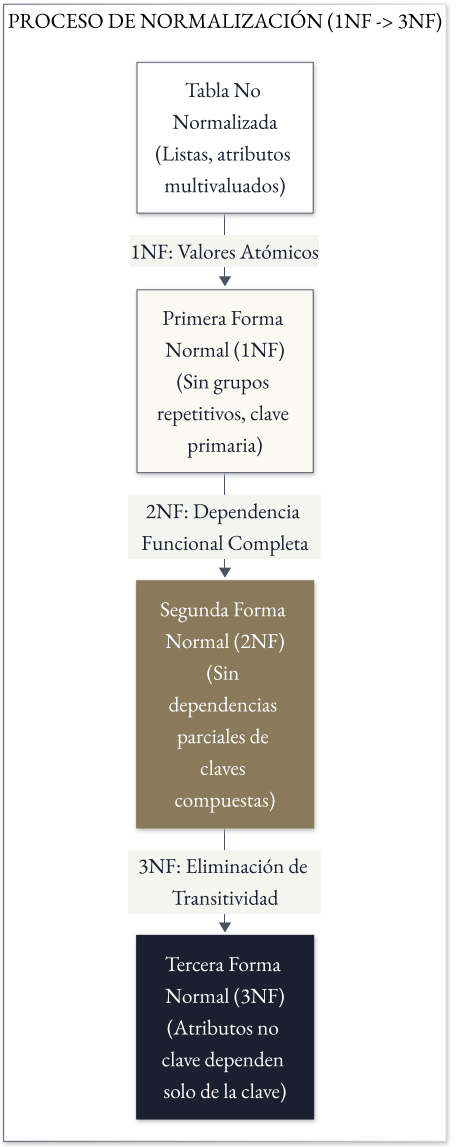
Definición 0B.1 (Sistemas OLTP frente a Sistemas OLAP)

- **OLTP (*Online Transaction Processing*):** Sistemas diseñados para procesar cientos o miles de operaciones simultáneas por segundo de baja latencia (altas, modificaciones, lecturas puntuales de tuplas individuales). Su prioridad absoluta es la integridad transaccional inmediata, la no redundancia de datos y la concurrencia masiva mediante bloqueos o versiones.
- **OLAP (*Online Analytical Processing*):** Sistemas diseñados para ejecutar consultas complejas que agregan y analizan millones de registros a lo largo de amplios rangos temporales. Su prioridad es el rendimiento en lectura masiva, el escaneo secuencial en disco y la compresión columnar.

0B.1 Normalización Relacional y Eliminación de Anomalías

En los sistemas transaccionales, el objetivo principal del diseño de esquemas es la **eliminación de redundancias** para impedir anomalías operacionales:

- *Anomalía de Inserción:* Imposibilidad de registrar datos de un cliente hasta que este realice una compra.
- *Anomalía de Actualización:* Modificar la dirección de una sucursal en 10.000 filas de facturación, corriendo el riesgo de dejar registros inconsistentes si el proceso falla a la mitad.
- *Anomalía de Eliminación:* Borrar el último pedido de un usuario provocando la pérdida accidental de todos los datos de contacto de dicho usuario.



1. Primera Forma Normal (1NF)

Exige que cada celda de la tabla contenga un único valor **atómico** indivisible y que no existan grupos repetitivos (como columnas `telefono_1`, `telefono_2` o listas serializadas en texto). Cada fila debe poseer una clave primaria única.

2. Segunda Forma Normal (2NF)

Aplica a tablas con claves primarias compuestas: exige cumplir 1NF y que ningún atributo no clave dependa funcionalmente de una **parte** de la clave primaria (*dependencia parcial*). Todo atributo debe depender de la totalidad de la clave.

3. Tercera Forma Normal (3NF)

Exige cumplir 2NF y que ningún atributo no clave dependa transitivamente de la clave primaria a través de otro atributo no clave (*dependencia transitiva*).

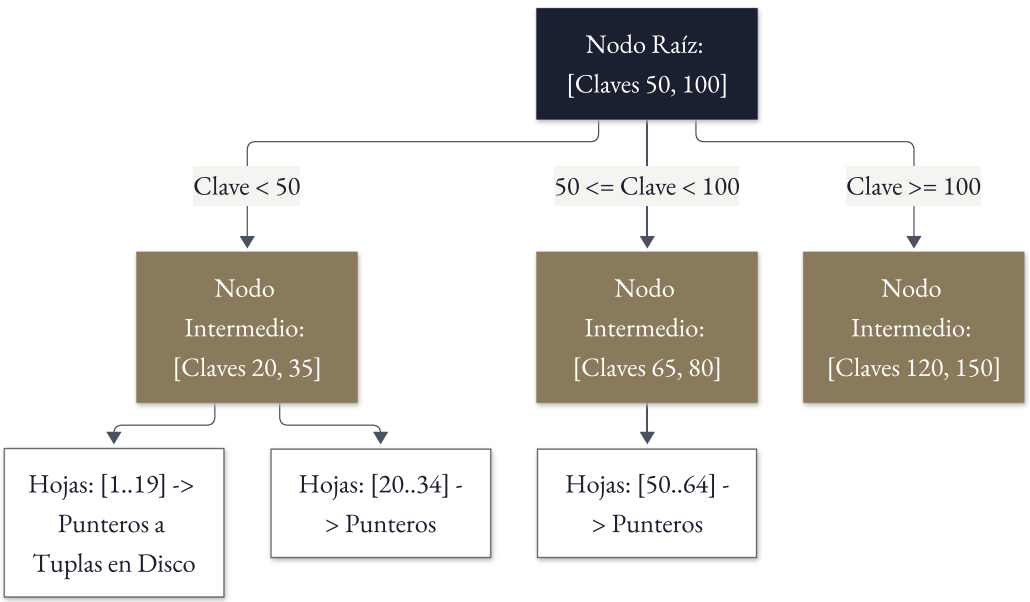
- *Ejemplo:* Si una tabla tiene `(factura_id, cliente_id, ciudad_cliente)`, `ciudad_cliente` depende de `cliente_id`, y este de `factura_id`. Para alcanzar 3NF, la información del cliente se traslada a su propia tabla normalizada `clientes`.

> **¿Por qué OLTP normaliza y OLAP desnormaliza?**
> Normalizar en OLTP garantiza que una actualización modifique exactamente **un solo registro físico en disco**, eliminando inconsistencias. Sin embargo, consultar una tabla en 3NF requiere múltiples `JOINs` costosos. En OLAP (Data Warehouses / Lakehouses), el espacio en disco es económico y las lecturas analíticas son masivas: desnormalizar en esquemas de estrella (*Star Schema*) optimiza drásticamente el escaneo columnar sin bloqueos.

---

oB.2 Estructura Física e Índices: B-Trees frente a Hash

Una base de datos relacional almacena las filas en páginas de disco (típicamente de 8 KB en PostgreSQL). Buscar un registro sin índices fuerza un escaneo secuencial completo de la tabla (*Sequential Scan* o $O(N)$).



1. El Árbol B-Tree (Balanced Tree)

Es el índice predeterminado y más versátil en motores relacionales:

- **Estructura Auto-Balanceada:** Todas las hojas se encuentran a la misma profundidad, garantizando complejidad de búsqueda logarítmica $O(\log N)$.
- **Búsquedas de Rango:** Las páginas hoja están interconectadas mediante punteros bidireccionales en lista doblemente enlazada, lo que permite resolver de forma óptima consultas con predicados de rango (`BETWEEN`, `<`, `>=`).
- **Soporte para Ordenación:** Almacena los elementos ordenados, satisfaciendo cláusulas `ORDER BY` sin necesidad de clasificar en memoria de trabajo (*Sort in-memory*).

2. Índices Hash

Calculan el hash criptográfico o posicional de la clave, ofreciendo búsquedas de igualdad exacta (`=`) en tiempo constante $O(1)$. Sin embargo, son **inútiles para consultas de rango** (`WHERE fecha > '2026-01-01'`) y no admiten ordenación.

3. El Costo de los Índices en Pipelines de Ingesta

Crear índices acelera las lecturas analíticas, pero **penaliza severamente las operaciones de escritura** (`INSERT`, `UPDATE`, `DELETE`). Cada tupla insertada requiere balancear y reescribir los nodos del árbol B-Tree en disco:

- **Regla de Producción:** Durante cargas masivas por lotes (*Bulk Load*) de millones de filas en una base relacional, la mejor práctica consiste en **eliminar los índices secundarios antes de la carga** y reconstruirlos en bloque una vez finalizada la inserción.

0B.3 Propiedades ACID y Mecánica de Durabilidad

Una **transacción** es una secuencia de operaciones relacionales tratadas como una única unidad lógica indivisible:

- **Atomicidad (Atomicity):** Todo o nada. Si falla una sola sentencia dentro del bloque transaccional, el motor deshace todas las operaciones previas mediante un *Rollback*.
- **Consistencia (Consistency):** La transacción solo puede llevar la base de datos de un estado válido a otro estado válido, respetando todas las restricciones de integridad declaradas (claves foráneas, tipos de datos, restricciones `CHECK`).
- **Aislamiento (Isolation):** La ejecución concurrente de transacciones produce el mismo estado que si se hubiesen ejecutado de forma secuencial una tras otra.

- **Durabilidad (Durability):** Una vez que una transacción confirma su éxito (*Commit*), sus cambios son permanentes y no se perderán incluso ante cortes catastróficos de energía.

Write-Ahead Logging (WAL)

Para garantizar la durabilidad sin ralentizar cada transacción esperando a que los bloques de la tabla se sincronicen en el disco de datos, los motores utilizan un diario secuencial denominado **WAL** (*Write-Ahead Log* en PostgreSQL o *Redo Log* en Oracle):

1. Antes de modificar cualquier página de memoria en la caché (*Buffer Pool*), el cambio se escribe secuencialmente al WAL en disco.
2. Al invocar `COMMIT`, el motor solo fuerza el volcado físico del WAL a disco (*fsync* secuencial ultrarrápido).
3. Si el servidor colapsa, durante el arranque el motor lee el WAL y aplica los cambios confirmados sobre los archivos de datos (*Crash Recovery*).

---

oB.4 Niveles de Aislamiento Transaccional y Anomalías de Concurrencia

El estándar ANSI SQL define cuatro niveles de aislamiento crecientes, cada uno protegiendo contra anomalías específicas de concurrencia:

Nivel de Aislamiento	Lectura Sucia (***Dirty Read***)	Lectura No Repetible (***Fuzzy Read***)	Lectura Fantasma (***Phantom Read***)	Desviación de Escritura (***Write Skew***)
Read Uncommitted	Posible	Posible	Posible	Posible
Read Committed *(Default en Postgres)*	**Prevenida**	Posible	Posible	Posible
Repeatable Read	**Prevenida**	**Prevenida**	**Prevenida** *(en Postgres MVCC)*	Posible
Serializable	**Prevenida**	**Prevenida**	**Prevenida**	**Prevenida**

Definición Matemática de las Anomalías:

1. **Lectura Sucia (*Dirty Read*):** La Transacción T_1 modifica una fila pero aún no hace commit. La Transacción T_2 lee ese valor no confirmado. Si T_1 hace rollback, T_2 procesó datos ficticios que jamás existieron formalmente.

2. **Lectura No Repetible (*Non-Repeatable Read*):** La Transacción T_1 lee una fila. La Transacción T_2 modifica o borra dicha fila y confirma su commit. Si T_1 vuelve a leer la misma fila, obtiene valores distintos.
3. **Lectura Fantasma (*Phantom Read*):** La Transacción T_1 consulta un rango de filas con una condición (ej. `WHERE saldo > 1000`). La Transacción T_2 inserta una nueva fila que satisface la condición y confirma. Si T_1 repite la consulta de rango, aparece una nueva fila "fantasma".
4. **Desviación de Escritura (*Write Skew*):** Ocurre cuando dos transacciones leen los mismos datos pero escriben en conjuntos disjuntos violando una regla de negocio global (ej. dos médicos solicitan salir de guardia simultáneamente al verificar que al menos uno de ellos sigue activo; ambas solicitudes se aprueban, dejando la guardia vacía). Solo se previene en **Serializable**.

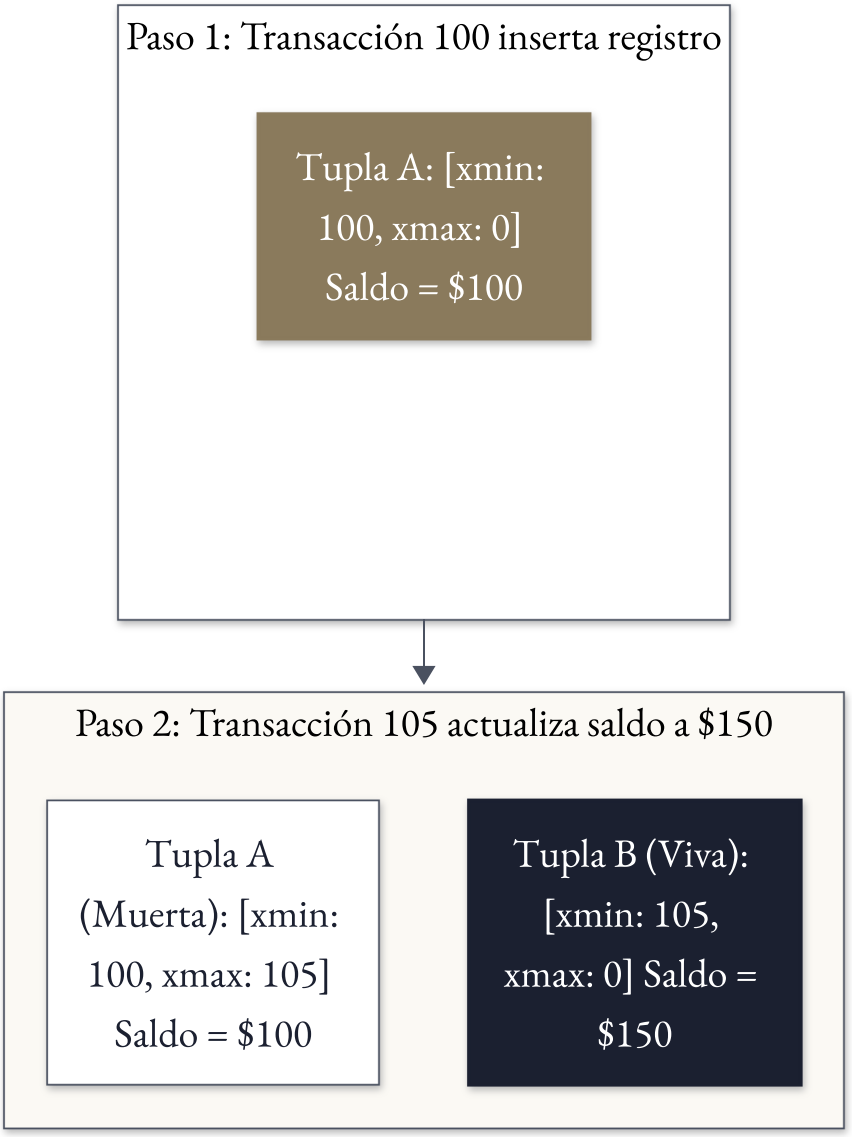
---

oB.5 Control de Concurrencia Multiversión (MVCC) en PostgreSQL

PostgreSQL no utiliza bloqueos de lectura para proteger a los lectores de los escritores. En su lugar, implementa **MVCC** (*Multi-Version Concurrency Control*): **"Los lectores nunca bloquean a los escritores, y los escritores nunca bloquean a los lectores"**.

Cada fila (*heap tuple*) almacena metadatos invisibles en su encabezado:

- `xmin`: El identificador de la transacción que insertó la fila.
- `xmax`: El identificador de la transacción que borró o actualizó la fila (inicialmente 0).



- **Actualizaciones como Inserciones:** En MVCC, un `UPDATE` no sobrescribe el registro original en el disco. Marca la tupla previa asignándole el ID de la transacción actual en `xmax` (convirtiéndola en una tupla "muerta") e inserta una tupla completamente nueva con el nuevo valor.
- **El Proceso `VACUUM`:** Las tuplas muertas ocupan espacio físico en las páginas de disco. El proceso `VACUUM` (y el demonio en segundo plano `autovacuum`) escanea las páginas para reclamar el espacio de tuplas muertas que ya no son visibles para ninguna transacción activa, evitando el crecimiento descontrolado de las tablas en disco (*Table Bloat*).

oB.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Extraer Datos Analíticos Masivos mediante Consultas Directas sobre la Base Primaria OLTP
Ejecutar consultas de extracción analítica masiva (`SELECT * FROM transacciones WHERE anio = 2025`) directamente sobre el nodo primario de la base transaccional durante el día satura el buffer pool y compite por recursos con los clientes de la aplicación.
Corrección: Dirija las extracciones hacia **réplicas de solo lectura** (*Read Replicas* configuradas mediante replicación física por WAL) o utilice herramientas de CDC (*Change Data Capture*) como Debezium para leer los eventos directamente del log sin consultar las tablas vivas.

Anti-patrón 2: Desactivar o Retrasar el Proceso de `autovacuum`
Desactivar `autovacuum` para supuestamente ahorrar CPU en una base de datos de alto tráfico transaccional provoca que las tablas acumulen millones de tuplas muertas. Las consultas se ralentizan dramáticamente al tener que escanear páginas infladas y se corre el riesgo inminente de fallo catastrófico por agotamiento de identificadores de transacción (*Transaction ID Wraparound*).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0B.1 (Diagnóstico de Anomalía de Concurrencia):
Considere el siguiente escenario transaccional en una tabla de cuentas bancarias (`saldo` inicial = $500):

1. La Transacción A inicia en nivel `READ COMMITTED` y consulta el saldo de la cuenta 101, obteniendo $500.
2. La Transacción B inicia, descuenta $100 de la cuenta 101 (nuevo saldo = $400) y ejecuta `COMMIT`.
3. La Transacción A, aún activa, vuelve a ejecutar la misma consulta sobre la cuenta 101 y obtiene $400. ¿Qué anomalía de concurrencia ha ocurrido y a qué nivel de aislamiento debería elevarse la Transacción A para evitarla?

Solución 0B.1:

- Ha ocurrido una anomalía de **Lectura No Repetible (*Non-Repeatable Read* o *Fuzzy Read*)**, donde una misma transacción obtiene valores contradictorios para una misma fila tras el commit de una transacción concurrente.
- Para evitar esta anomalía, la Transacción A debe ejecutarse bajo el nivel de aislamiento `REPEATABLE READ` (o `SERIALIZABLE`). Bajo `REPEATABLE READ`, el motor establece una instantánea (*snapshot*) temporal al momento de la primera consulta, garantizando que

lecturas posteriores dentro de la misma transacción lean exactamente la misma foto inmutable del registro.

Ejercicio 0B.2 (Estructuración de Índices B-Tree Compuestos):
Se tiene una tabla `pedidos` con 100 millones de registros. Las consultas más recurrentes del sistema transaccional tienen la siguiente estructura:
`SELECT * FROM pedidos WHERE cliente_id = 4500 AND fecha_pedido >= '2026-01-01'`
Compare las siguientes dos alternativas de índice B-Tree compuesto y justifique cuál es la óptima:

- Opción 1: `CREATE INDEX idx_pedidos ON pedidos (fecha_pedido, cliente_id);`
- Opción 2: `CREATE INDEX idx_pedidos ON pedidos (cliente_id, fecha_pedido);`

Solución 0B.2:

- **La Opción 2 `(cliente_id, fecha_pedido)` es la óptima.**
- *Justificación Técnica:* En un árbol B-Tree compuesto, los nodos se ordenan jerárquicamente por la primera columna y luego por la segunda.
  - Con la Opción 2, el motor busca por igualdad exacta sobre `cliente_id` (alta selectividad), descendiendo directamente a la rama del cliente específico en $O(\log N)$ y luego escaneando secuencialmente solo el rango de fechas de ese cliente.
  - Con la Opción 1, la primera columna es un predicado de rango (`fecha_pedido >= '2026-01-01'`). Una vez que un B-Tree entra en una búsqueda de rango sobre su primera columna, el motor no puede podar eficientemente por la segunda columna y se ve forzado a escanear todas las entradas que cumplan la fecha en todas las páginas hoja.

0B.7 Resumen y Puntos Clave

- **Diseño para la integridad:** Los sistemas OLTP normalizan (1NF a 3NF) para eliminar anomalías de actualización y garantizar que cada hecho se registre en una sola ubicación física.
- **Mecánica de índices B-Tree:** Ofrecen búsquedas logarítmicas $O(\log N)$ y ordenación eficiente, pero imponen una penalización de mantenimiento en cada inserción o modificación.
- **Garantías ACID y WAL:** La durabilidad se garantiza mediante el volcado secuencial al Write-Ahead Log (*WAL*), permitiendo recuperaciones rápidas ante caídas del servidor.
- **MVCC en PostgreSQL:** Mediante identificadores de transacción `xmin` y `xmax`, PostgreSQL permite lecturas no bloqueantes generando nuevas versiones de tuplas, requiriendo `autovacuum` para reclamar espacio muerto.

- **Aislamiento transaccional:** Conocer las anomalías de concurrencia permite elegir el nivel óptimo sin incurrir en bloqueos excesivos.

Lecturas Recomendadas

1. **Silberschatz, A., Korth, H. F. & Sudarshan, S. (2020).** *Database System Concepts (7th Edition).* McGraw-Hill.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 7: Transactions).* O'Reilly Media.
3. **Stonebraker, M. & Cetintemel, U. (2005).** *"One Size Fits All": An Idea Whose Time Has Come and Gone.* Proceedings of the 21st International Conference on Data Engineering (ICDE).
4. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Concurrency Control and MVCC.*

CAPÍTULO oC

Python para Ingeniería de Datos: Tipado, APIs y Resiliencia NÚCLEO

> *"Escribir scripts exploratorios en un notebook es una habilidad de análisis; construir pipelines modulares, fuertemente tipados, tolerantes a fallos y empaquetados con rigor de ingeniería de software es la verdadera responsabilidad del Data Engineer. En entornos de producción, Python no es un lenguaje de juguete: es el lenguaje de orquestación, integración y transformación de datos por excelencia."*

oC.o Intuición Inicial: La Cinta Transportadora de la Fábrica

Imagina que te encargan procesar las facturas de una tienda pequeña. Tienes diez facturas en papel sobre una mesa: tomas cada una, lees el total con los ojos y anotas la suma en una libreta con lápiz. Si son diez facturas, el método manual funciona sin problemas.

Pero ahora imagina que la tienda se convierte en un gigante de comercio electrónico que procesa 500.000 transacciones por hora provenientes de ocho países distintos. Las facturas ya no vienen en papel: llegan como mensajes JSON a través de conexiones de red inestables, algunas traen importes nulos, otras traen identificadores duplicados y los servidores de origen se saturan con frecuencia arrojando errores temporales. Si intentas resolver esto con un script improvisado que asume que "nada fallará", el sistema colapsará en cuestión de minutos.

Python es la cinta transportadora programable de esa fábrica industrial de datos. En este capítulo comenzaremos desde lo más elemental —cómo guardar un dato en una variable, cómo filtrar una lista de registros, cómo encapsular lógica en funciones y cómo leer archivos de disco— hasta ensamblar componentes de nivel productivo: contratos de datos estrictos con Pydantic, consumo resiliente de APIs REST con reintentos exponenciales y manipulación columnar de alta velocidad con Polars y Apache Arrow.

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Programar en Python moderno desde los fundamentos: variables, colecciones (`list`, `dict`, `set`, `tuple`), estructuras de control, funciones puras y manejo seguro de archivos con *context managers*.
- Aplicar Programación Orientada a Objetos básica y `@dataclass` para modelar entidades de datos estructuradas.
- Aislar dependencias y ejecutar pruebas unitarias automatizadas con `uv` y `pytest`.
- Aplicar el sistema de tipos moderno (`typing`) y validar contratos de datos en tiempo de ejecución con **Pydantic**.
- Diseñar clientes de ingesta HTTP resilientes para **APIs REST** con soporte para autenticación, paginación por cursores y retroceso exponencial (*Exponential Backoff con Jitter*).
- Manipular datos en memoria comparando los paradigmas de **Pandas**, **PyArrow** y **Polars** (ejecución multihilo perezosa en Rust).

Profundiza: La Brecha entre Scripting y Software Engineering

Definición 0C.1 (La Brecha entre Scripting y Software Engineering)
Un *script de datos* se escribe para ejecutarse una sola vez de forma interactiva: asume que la red nunca se corta, que el esquema del JSON de origen es estático y que todos los datos caben cómodamente en la memoria RAM.

Un **pipeline de ingeniería de software para datos** se diseña bajo la premisa del fallo inevitable (*Design for Failure*): valida los datos entrantes contra un contrato estricto, maneja desconexiones transitorias mediante reintentos exponenciales, no filtra credenciales en los logs y garantiza reproducibilidad absoluta de su entorno mediante bloqueos de dependencias (*lockfiles*).

oC.1 Fundamentos de Python para Ingeniería de Datos: De Cero a Código Modular

Para el aspirante a Data Engineer que proviene de cero o de campos no computacionales, dominar Python no significa memorizar sintaxis abstracta: significa comprender cómo estructurar datos en memoria, cómo procesar archivos y cómo escribir código modular y testeable.

1. Tipos de Datos Primitivos y Variables

En Python, una variable es una referencia en memoria a un objeto tipado:

```python
# Tipos primitivos esenciales en pipelines de datos
registro_id = 1048576            # int: enteros de precisión arbitraria
monto_transaccion = 450.75       # float: números de punto flotante IEEE 754 de 64 bits
estado_pago = "COMPLETADO"       # str: cadenas inmutables codificadas en UTF-8
es_valido = True                 # bool: valores booleanos para predicados lógicos
tasa_cambio = None               # NoneType: representa la ausencia explícita de valor
```

2. Estructuras de Datos de Primera Clase para Pipelines

Los flujos de datos manipulan cuatro colecciones fundamentales:

```python
# 1. LISTAS (list): Secuencias ordenadas y mutables. Útiles para acumular registros
transacciones = [120.50, 45.00, 890.10, 15.20]
transacciones.append(300.00)     # O(1) amortizado

# 2. DICCIONARIOS (dict): Tablas hash de clave-valor O(1). El corazón de cada fila o evento JSON
cliente = {
    "cliente_id": 4501,
    "nombre": "Ana Gomez",
    "pais": "CO",
    "activo": True
}
print(cliente.get("correo", "desconocido@correo.com"))  # Acceso seguro ante claves faltantes

# 3. CONJUNTOS (set): Colecciones no ordenadas de elementos únicos. Deduplicación instantánea O(1)
paises_con_ventas = {"CO", "MX", "CL", "PE"}
paises_con_ventas.add("CO")      # No duplica; el tamaño del conjunto sigue intacto
print("US" in paises_con_ventas) # Verificación de pertenencia en O(1)

# 4. TUPLAS (tuple): Secuencias ordenadas e inmutables. Seguras como claves compuestas
coordenada_evento = ("servidor_01", 8080)
```

Comprensiones de Listas y Diccionarios (Comprehensions)

Permiten transformar y filtrar colecciones con sintaxis concisa y optimizada en C:

```python
# Filtrar y transformar ventas superiores a $100 aplicando un impuesto del 19%
ventas_con_iva = [round(monto * 1.19, 2) for monto in transacciones if monto > 100.0]

# Indexar un diccionario de clientes por su ID para búsquedas directas
lista_clientes = [
    {"id": 101, "nombre": "Carlos"},
    {"id": 102, "nombre": "Lucia"}
]
indice_clientes = {c["id"]: c["nombre"] for c in lista_clientes}
```

3. Control de Flujo y Funciones Modulares

Las funciones puras encapsulan lógica de transformación sin efectos secundarios:

```python
def procesar_lote_ventas(montos, umbral_alerta=500.0, **kwargs):
    """Procesa un lote de montos, calculando totales y detectando anomalías."""
    total = 0.0
    alertas = []

    for monto in montos:
        if monto is None or monto < 0:
            continue  # Omitir registros corruptos
        total += monto
        if monto >= umbral_alerta:
            alertas.append(monto)

    return {"total_procesado": round(total, 2), "alertas": alertas}

resultado = procesar_lote_ventas(transacciones, umbral_alerta=200.0)
```

4. Manejo de Archivos en Disco: CSV, JSON y Context Managers

En ingeniería de datos, **nunca se abren archivos sin un administrador de contexto (`with`)**, el cual garantiza el cierre inmediato de los descriptores de archivo incluso ante excepciones no controladas:

PYTHON

```python
import csv
import json

# Lectura y parseo seguro de archivos CSV fila a fila
def leer_transacciones_csv(ruta_archivo):
    registros = []
    with open(ruta_archivo, mode="r", encoding="utf-8") as archivo:
        lector_csv = csv.DictReader(archivo)
        for fila in lector_csv:
            registros.append({
                "id": int(fila["transaccion_id"]),
                "monto": float(fila["monto_usd"]),
                "pais": fila["pais"]
            })
    return registros

# Serialización y guardado de resultados en JSON estructurado
def exportar_resumen_json(datos, ruta_destino):
    with open(ruta_destino, mode="w", encoding="utf-8") as archivo:
        json.dump(datos, archivo, indent=2, ensure_ascii=False)
```

5. Programación Orientada a Objetos (OOP) Esencial para DE

La POO se utiliza en ingeniería de datos para modelar clientes de bases de datos, extractores de APIs y transformadores modulares:

PYTHON

```python
class ExtractorDatosAPI:
    """Clase base reutilizable para clientes de ingesta con encapsulamiento de
estado."""
    def __init__(self, endpoint_base: str, api_key: str):
        self.endpoint_base = endpoint_base.rstrip("/")
        self._api_key = api_key  # Atributo protegido para no exponer
credenciales
        self.registros_procesados = 0

    def construir_url(self, recurso: str) -> str:
        return f"{self.endpoint_base}/{recurso.lstrip('/')}"

    def registrar_metrica(self, cantidad: int) -> None:
        self.registros_procesados += cantidad
        print(f"[{self.__class__.__name__}] Total procesado acumulado:
{self.registros_procesados}")
```

6. Aislamiento de Entornos Virtuales y Dependencias con `uv`

Nunca instale paquetes de ingeniería en el entorno global del sistema operativo:

BASH

```bash
# 1. Crear entorno virtual moderno y ultrarrápido con uv
uv venv .venv --python 3.11

# 2. Activar el entorno virtual:
# En Linux / macOS / WSL:
source .venv/bin/activate
# En Windows PowerShell:
.venv\Scripts\Activate.ps1

# 3. Instalar librerías de producción:
uv pip install pydantic requests pyarrow polars pytest
```

7. Pruebas Unitarias desde Cero con `pytest`

La ingeniería de software aplicada a datos exige validar que las funciones de transformación respeten sus especificaciones matemáticas antes de promover código a producción:

PYTHON

```python
# tests/test_transformaciones.py
import pytest

def normalizar_pais(pais: str) -> str:
    if not pais or not isinstance(pais, str):
        return "UNKNOWN"
    return pais.strip().upper()

# Casos de prueba automatizados con pytest
def test_normalizar_pais_exitoso():
    assert normalizar_pais("  co ") == "CO"
    assert normalizar_pais("mx") == "MX"

def test_normalizar_pais_nulo_o_invalido():
    assert normalizar_pais(None) == "UNKNOWN"
    assert normalizar_pais("") == "UNKNOWN"

def test_normalizar_pais_tipo_invalido():
    assert normalizar_pais(123) == "UNKNOWN"
```

Para ejecutar las pruebas en la terminal:

BASH

```bash
uv run pytest tests/ -v
```

oC.2 Tipado Estático Moderno y Validación con Pydantic

Python es un lenguaje de tipado dinámico, pero la introducción de anotaciones de tipo (*Type Hints*, PEP 484) permite verificar el código estáticamente mediante herramientas como `mypy` y `ruff`, detectando bugs antes del despliegue:

PYTHON

```python
from typing import Optional, List, Dict, Any, Union
from datetime import datetime

def calcular_tasa_churn(
    usuarios_activos: List[int],
    bajas_periodo: List[int],
    factor_ajuste: Optional[float] = None
) -> float:
    if not usuarios_activos:
        return 0.0

    tasa_base = len(bajas_periodo) / len(usuarios_activos)
    if factor_ajuste is not None:
        return tasa_base * factor_ajuste
    return tasa_base
```

Validación en Tiempo de Ejecución con Pydantic (v2)

Las anotaciones de tipo estándar son ignoradas en tiempo de ejecución. **Pydantic** resuelve esta carencia parseando y validando automáticamente estructuras de datos externas (JSONs crudos provenientes de APIs o Kafka):

PYTHON

```python
from pydantic import BaseModel, Field, EmailStr, field_validator
from datetime import date
from typing import Literal

class TransaccionVenta(BaseModel):
    transaccion_id: str = Field(..., description="UUID único de la transacción")
    cliente_id: int = Field(..., gt=0, description="Identificador positivo del
cliente")
    correo: EmailStr
    monto_usd: float = Field(..., ge=0.01, description="Monto mínimo positivo")
    moneda: Literal["USD", "EUR", "COP"] = "USD"
    fecha: date

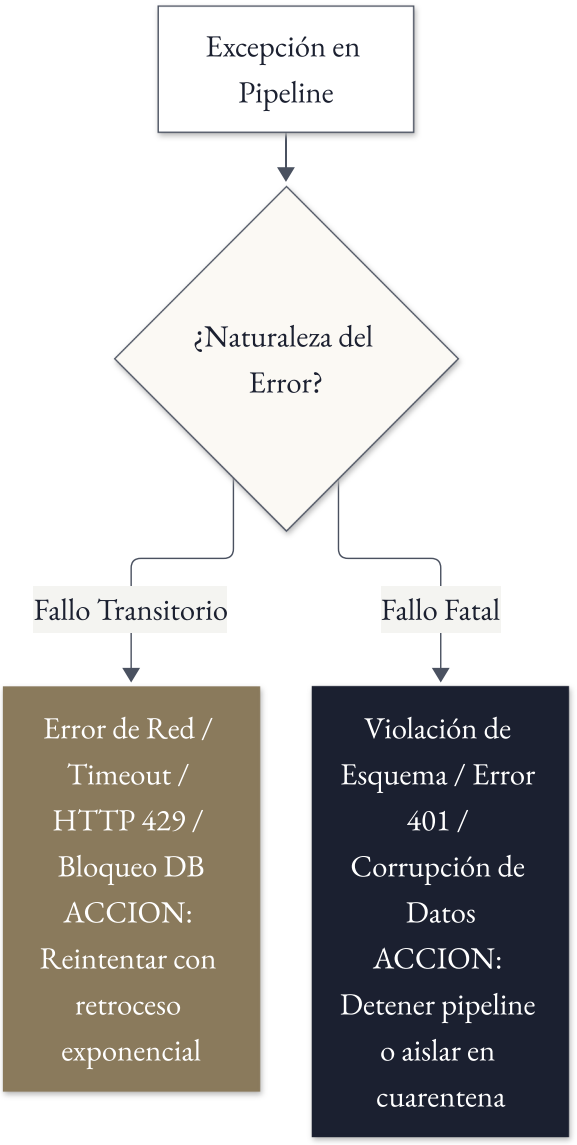
    @field_validator("monto_usd")
    @classmethod
    def validar_monto_maximo(cls, v: float) -> float:
        if v > 1_000_000:
            raise ValueError("Monto excede el límite permitido para procesamiento
automático sin auditoría")
        return round(v, 2)
```

```
# Demostración de parseo y validación estricta
raw_payload = {
    "transaccion_id": "tx-99482-abc",
    "cliente_id": 4501,
    "correo": "cliente.analitica@empresa.com",
    "monto_usd": "1540.508",  # Pydantic coacciona el string a float y aplica el
validador
    "moneda": "USD",
    "fecha": "2026-03-15"
}

try:
    evento_validado = TransaccionVenta.model_validate(raw_payload)
    print(f"Evento válido: ID={evento_validado.transaccion_id}, Monto=
{evento_validado.monto_usd}")
except Exception as e:
    print(f"Error de contrato de datos: {e}")
```

oC.2 Manejo Robusto de Excepciones y Logging Contextual

Un pipeline de datos nunca debe capturar excepciones genéricas silenciosamente (`except: pass`). El manejo de errores debe clasificar las fallas en dos categorías fundamentales:



Excepciones de Dominio Personalizadas y Bloques de Limpieza

PYTHON

```python
import logging
import sys

# Configuración de Logging Estructurado
logging.basicConfig(
    level=logging.INFO,
    format='{"timestamp": "%(asctime)s", "nivel": "%(levelname)s", "modulo": "%
(name)s", "mensaje": "%(message)s"}',
    stream=sys.stdout
)
```

```
logger = logging.getLogger("ingestion_pipeline")

class PipelineDataException(Exception):
    """Excepción base para fallos en el procesamiento de datos."""
    pass

class SchemaContractViolation(PipelineDataException):
    """Fallo fatal: el payload recibido viola el contrato de datos."""
    pass

def procesar_lote_archivos(ruta_archivo: str) -> None:
    manejador_recurso = None
    try:
        logger.info(f"Iniciando apertura de recurso: {ruta_archivo}")
        manejador_recurso = open(ruta_archivo, "r", encoding="utf-8")
        # Lógica de transformación...
    except FileNotFoundError as e:
        logger.error(f"Archivo no localizado en almacenamiento: {ruta_archivo}")
        raise PipelineDataException(f"Recurso inalcanzable: {e}") from e
    except Exception as e:
        logger.critical(f"Fallo imprevisto en la ejecución del lote: {e}",
exc_info=True)
        raise
    finally:
        # Garantía incondicional de liberación de sockets o archivos
        if manejador_recurso and not manejador_recurso.closed:
            manejador_recurso.close()
            logger.info("Recurso liberado exitosamente en bloque finally.")
```

oC.3 Consumo Resiliente de APIs REST: Paginación y Rate Limits

Las APIs transaccionales restringen la cantidad de registros devueltos por petición y limitan la frecuencia de solicitudes mediante cuotas (*Rate Limits*).

1. Estrategias de Paginación

- **Paginación por Desplazamiento (`limit` y `offset`):**
  `GET /api/v1/ventas?limit=100&offset=200`
  Desventaja: Ineficiente para grandes volúmenes ($O(N)$ en la base de datos de origen para saltar filas) y sufre inconsistencias si se insertan nuevas filas mientras se página.
- **Paginación por Cursor (*Cursor-based Pagination*):**
  `GET /api/v1/ventas?limit=100&starting_after=tx_cursor_991823`
  Ventaja: Escanea mediante índices en $O(1)$ y es inmune a inserciones intermedias. Es el estándar en APIs modernas (Stripe, Slack, Shopify).

2. Retroceso Exponencial con Jitter (*Exponential Backoff*)

Cuando un servidor responde con código **HTTP 429 Too Many Requests** o errores de servidor **HTTP 503 Service Unavailable**, bombardearlo con peticiones inmediatas agrava la congestión. Se debe esperar un tiempo exponencial aleatorizado:

$$T_{\text{espera}} = \min(T_{\max}, \text{base} \times 2^{\text{intento}}) + \text{aleatorio}(0, \text{jitter})$$

PYTHON

```python
import time
import random
import requests
from typing import Generator, Dict, Any

class ClienteApiResiliente:
    def __init__(self, url_base: str, token: str, max_reintentos: int = 5):
        self.url_base = url_base
        self.headers = {
            "Authorization": f"Bearer {token}",
            "Accept": "application/json"
        }
        self.max_reintentos = max_reintentos

    def _peticion_con_reintento(self, endpoint: str, params: Dict[str, Any]) ->
Dict[str, Any]:
        url = f"{self.url_base}/{endpoint}"
        intento = 0
        tiempo_espera = 1.0  # Segundo base

        while intento < self.max_reintentos:
            try:
                respuesta = requests.get(url, headers=self.headers,
params=params, timeout=10.0)

                # Éxito
                if respuesta.status_code == 200:
                    return respuesta.json()

                # Manejo de Rate Limit (HTTP 429) o fallos transitorios de
servidor (5xx)
                if respuesta.status_code in (429, 500, 502, 503, 504):
                    intento += 1
                    # Respetar encabezado Retry-After si el servidor lo provee
                    reintento_sugerido = respuesta.headers.get("Retry-After")
                    if reintento_sugerido:
                        demora = float(reintento_sugerido)
                    else:
                        jitter = random.uniform(0.1, 0.5)
                        demora = min(60.0, tiempo_espera * (2 ** intento)) +
jitter

                    time.sleep(demora)
```

```
                    continue

                # Errores fatales de cliente (400, 401, 403, 404)
                respuesta.raise_for_status()

            except (requests.ConnectionError, requests.Timeout) as e:
                intento += 1
                if intento >= self.max_reintentos:
                    raise ConnectionError(f"Se agotaron los reintentos hacia
{url}: {e}") from e
                time.sleep(tiempo_espera * (2 ** intento) + random.uniform(0.1,
0.5))

        raise RuntimeError(f"Fallo definitivo al consultar {url} tras
{self.max_reintentos} intentos.")

    def extraer_todos_los_eventos_por_cursor(self) -> Generator[Dict[str, Any],
None, None]:
        """Extrae la totalidad de los datos utilizando paginación por
cursores."""
        cursor_actual = None
        tiene_mas_datos = True

        while tiene_mas_datos:
            parametros = {"limit": 100}
            if cursor_actual:
                parametros["starting_after"] = cursor_actual

            payload = self._peticion_con_reintento("eventos", parametros)
            registros = payload.get("data", [])

            for registro in registros:
                yield registro

            tiene_mas_datos = payload.get("has_more", False)
            if tiene_mas_datos and registros:
                cursor_actual = registros[-1]["id"]
```

oC.4 Empaquetado Moderno y Entornos Deterministas con `uv`

Durante años, el ecosistema de Python dependió de `virtualenv`, `pip` y `requirements.txt` sin resolver adecuadamente el bloqueo determinista de dependencias indirectas.

La Revolución de `uv` (Astral)

Desarrollado en Rust, `uv` es un administrador de paquetes de última generación que resuelve e instala dependencias entre **10 y 100 veces más rápido que pip**, soportando nativamente el estándar moderno `pyproject.toml` (PEP 517/518/621) y lockfiles universales (`uv.lock`).

```toml
# Archivo: pyproject.toml
[project]
name = "plataforma-datos-lakehouse"
version = "0.1.0"
description = "Pipelines productivos de ingesta y transformación distribuida"
authors = [{ name = "Luis Galvis", email = "autor@empresa.com" }]
requires-python = ">=3.11"
dependencies = [
    "pyspark==3.5.2",
    "delta-spark==3.2.0",
    "pydantic>=2.7.0",
    "requests>=2.31.0",
    "tenacity>=8.2.3",
    "pyarrow>=15.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "chispa>=0.9.4",
    "ruff>=0.4.0",
    "mypy>=1.9.0",
]
```

Comandos Esenciales con `uv` en Producción

```bash
# Crear entorno virtual ultrarrápido con versión específica de Python
uv venv .venv --python 3.11

# Instalar dependencias bloqueadas de forma determinista
uv pip sync uv.lock

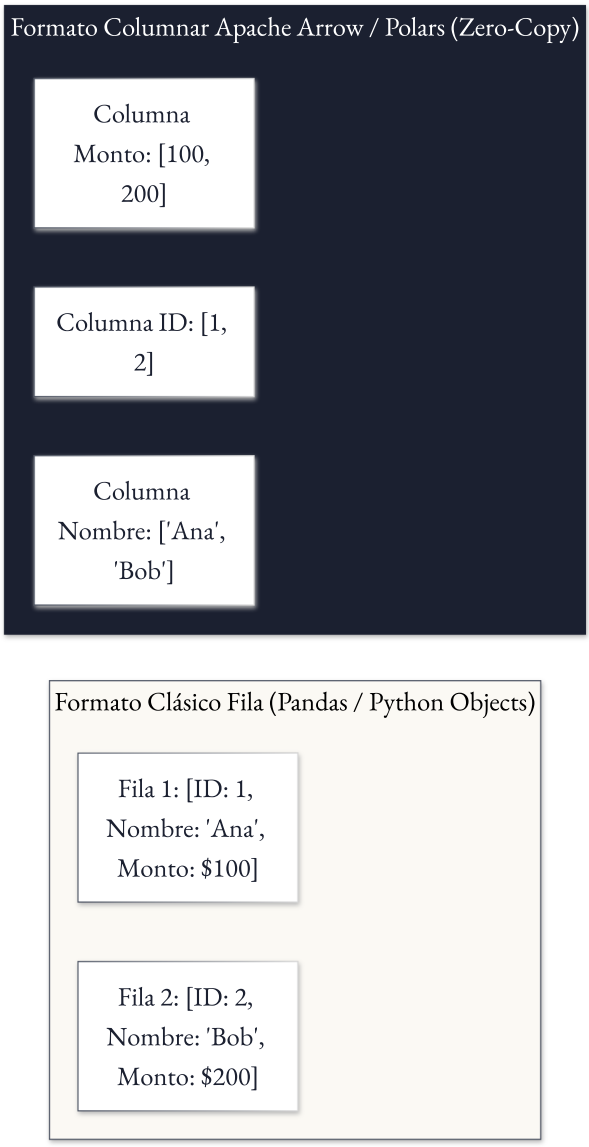
# Compilar un lockfile determinista a partir de pyproject.toml
uv pip compile pyproject.toml -o uv.lock

# Ejecutar herramientas de linteado y tipado estático
uv run ruff check src/
uv run mypy src/
```

---

oC.6 Manipulación Tabular en Memoria: Pandas, Polars y Apache Arrow

Antes de escalar a clústeres de Spark distribuidos, el procesamiento en un solo nodo se ejecuta con **Pandas**, **Polars** y **Apache Arrow**:



- **Apache Arrow:** Es el estándar en memoria para datos tabulares columnares. Ofrece aceleración de instrucciones SIMD (*Single Instruction, Multiple Data*) a nivel de CPU y elimina el costo de serialización (*Zero-Copy*) al transferir datos entre Python, Spark, DuckDB y C++.
- **PyArrow en Ingesta:** Almacena esquemas fuertemente tipados con soporte nativo para compresión y escritura directa a archivos Parquet:

PYTHON

```python
import pyarrow as pa
import pyarrow.parquet as pq

# Definición de esquema estricto en Apache Arrow
esquema_arrow = pa.schema([
    ("transaccion_id", pa.string()),
    ("cliente_id", pa.int64()),
    ("monto_usd", pa.float64()),
    ("fecha", pa.date32())
])

datos = {
    "transaccion_id": ["tx-001", "tx-002"],
    "cliente_id": [101, 102],
    "monto_usd": [450.50, 89.20],
    "fecha": [pa.array(["2026-03-01", "2026-03-02"]).cast(pa.date32())]
}

tabla_arrow = pa.Table.from_pydict(datos, schema=esquema_arrow)

# Escritura directa a disco en formato Parquet con compresión Snappy
pq.write_table(tabla_arrow, "ventas_lote.parquet", compression="snappy")
```

Polars en la Práctica: Rendimiento Multihilo y Evaluación Perezosa

Mientras Pandas opera de forma ansiosa (*Eager Execution*) sobre un único hilo de CPU, **Polars** (escrito en Rust sobre Apache Arrow) es el estándar moderno para transformación de datos en una sola máquina:

1. **Paralelismo Nativo:** Ejecuta transformaciones en paralelo a través de todos los núcleos disponibles, sorteando las limitaciones del Global Interpreter Lock (GIL) de Python.
2. **Evaluación Perezosa (*LazyFrames*):** Optimiza el plan de consulta fusionando filtros (*Predicate Pushdown*) y proyectando únicamente las columnas requeridas (*Projection Pushdown*) antes de leer los datos de disco.
3. **Motor Streaming (*Streaming Engine*):** Procesa archivos Parquet de decenas de gigabytes que exceden la memoria RAM de la máquina procesándolos en micro-lotes sin desbordar el sistema:

```python
import polars as pl

# 1. Definir consulta perezosa con LazyFrame (no ejecuta nada en memoria todavía)
q_lazy = (
    pl.scan_parquet("data/ventas.parquet")
    .filter(pl.col("estado_pago") == "COMPLETADO")
    .filter(pl.col("monto_usd") > 50.0)
    .group_by("pais", "categoria_producto")
    .agg([
        pl.len().alias("total_transacciones"),
        pl.col("monto_usd").sum().round(2).alias("ingresos_totales"),
        pl.col("monto_usd").mean().round(2).alias("ticket_promedio")
    ])
    .sort("ingresos_totales", descending=True)
)

# 2. Inspeccionar el plan de ejecución optimizado
print(q_lazy.explain())

# 3. Materializar el resultado (o usar collect(streaming=True) si los datos
superan la RAM)
df_resultado = q_lazy.collect()
print(df_resultado.head(5))
```

oC.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cargar Archivos Masivos con `pandas.read_csv()` en Producción
Pandas crea objetos de Python con alto sobrecosto en memoria: un archivo CSV de 5 GB puede llegar a consumir 25 GB de RAM física, provocando la terminación del proceso por el sistema operativo (*OOM Killer*).
Corrección: Procese archivos en fragmentos (*chunks* con `chunksize`), utilice DuckDB/PyArrow, o migre directamente a Apache Spark.

Anti-patrón 2: Capturar `Exception` sin Registrar el Traceback
Escribir `except Exception as e: logger.error(e)` oculta el número de línea, la jerarquía de llamadas y el tipo exacto de error, imposibilitando la depuración en sistemas distribuidos.
Corrección: Incluya siempre `exc_info=True` en la llamada a `logger.error()` o utilice `logger.exception()`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 0C.1 (Generador de Extracción con Manejo de Errores):
Diseñe una función en Python con tipado estático completo que lea un archivo JSON línea a línea (*JSON Lines / NDJSON*) utilizando un generador (`yield`). La función debe validar cada línea contra un modelo de Pydantic. Si una línea presenta corrupción sintáctica o violación de esquema, debe registrar un mensaje de advertencia y continuar procesando las siguientes líneas sin interrumpir el flujo.

Solución 0C.1:

PYTHON

```python
import json
import logging
from typing import Generator, Optional
from pydantic import BaseModel, ValidationError

class RegistroEvento(BaseModel):
    evento_id: str
    usuario_id: int
    accion: str

def extraer_eventos_ndjson(ruta_archivo: str) -> Generator[RegistroEvento, None,
None]:
    with open(ruta_archivo, "r", encoding="utf-8") as f:
        for numero_linea, linea in enumerate(f, start=1):
            linea_limpia = linea.strip()
            if not linea_limpia:
                continue

            try:
                payload = json.loads(linea_limpia)
                evento = RegistroEvento.model_validate(payload)
                yield evento
            except json.JSONDecodeError as err_json:
                logging.warning(f"Línea {numero_linea} corrupta (JSON inválido):
{err_json}")
            except ValidationError as err_val:
                logging.warning(f"Línea {numero_linea} viola el contrato de
datos: {err_val}")
```

---

oC.7 Resumen y Puntos Clave

- **Tipado estático y Pydantic:** Transforman Python en un entorno robusto para validar contratos de datos en las fronteras de entrada del pipeline.
- **Manejo de errores por severidad:** Los fallos transitorios se reintentan con retroceso exponencial; los fallos fatales se aíslan para no corromper capas posteriores.

- **APIs resilientes:** El consumo de servicios externos exige paginación por cursores, control estricto de timeouts y respeto a los encabezados de *Rate Limit*.
- **Empaquetado moderno con `uv`:** Garantiza la resolución determinista de dependencias con velocidades de instalación órdenes de magnitud superiores a las herramientas tradicionales.
- **Apache Arrow en memoria:** Proporciona un formato columnar de alto rendimiento y transferencias *zero-copy* entre subsistemas de datos.

Lecturas Recomendadas

1. **Ramalho, L. (2022).** *Fluent Python: Clear, Concise, and Effective Programming (2nd Edition).* O'Reilly Media.
2. **Colvin, S. et al. (2024).** *Pydantic V2 Documentation and Architecture.* Pydantic Documentation.
3. **Apache Arrow Project (2024).** *Apache Arrow: A Cross-Language Development Platform for In-Memory Data.* Apache Software Foundation.
4. **Astral Software (2024).** *uv: Extremely Fast Python Package Installer and Resolver.* Astral Documentation.

CAPÍTULO oD

Entorno de Laboratorio Reproducible y Generación de Datos NÚCLEO

> *"Un libro técnico de ingeniería que propone código sobre archivos inexistentes sin ofrecer un laboratorio reproducible es un ejercicio de frustración. Para que el conocimiento arraigue, cada consulta SQL, cada script de PySpark y cada pipeline de orquestación presentado en esta obra debe ser ejecutable y verificable en su propio computador sin costo alguno."*

oD.o El Repositorio Oficial del Libro

Todo el código fuente de los cincuenta y dos capítulos, los archivos de orquestación de Docker Compose, las suites de pruebas de pytest y el generador determinista de datos están versionados en el repositorio oficial de código abierto:

```bash
# 1. Clonar el repositorio oficial
git clone https://github.com/luisgalvis/como-ser-un-data-engineer.git

# 2. Ingresar al directorio del proyecto
cd como-ser-un-data-engineer
```

Enlace directo al repositorio: **https://github.com/luisgalvis/como-ser-un-data-engineer**

Objetivos de Aprendizaje

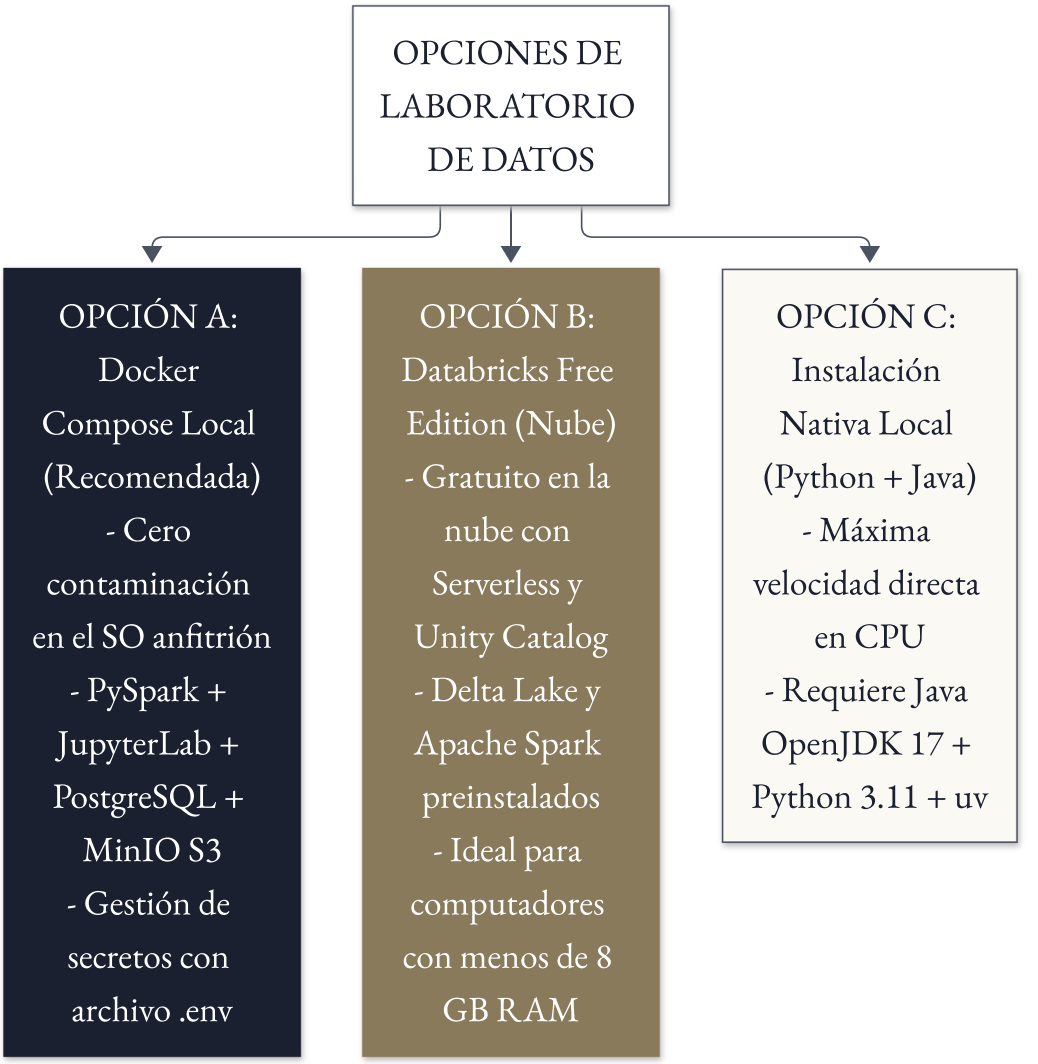
Al finalizar este capítulo, el lector será capaz de:

- Configurar un entorno de ingeniería de datos local y aislado mediante **Docker Compose**, o en la nube mediante **Databricks Free Edition**.
- Aplicar buenas prácticas de seguridad operativa segregando secretos y credenciales en un archivo `.env` local sin exponer contraseñas en control de versiones.
- Configurar entornos Windows mediante **WSL2 (Windows Subsystem for Linux)** y ajustar la gestión de memoria RAM en `.wslconfig` para evitar saturación de recursos.
- Ejecutar el generador oficial de datos (`python scripts/generar_datos.py`) para materializar en disco la suite completa de los 8 datasets de prueba utilizados a lo largo del libro.
- Validar la conectividad de Apache Spark y la lectura de archivos Parquet y JSON semiestructurados mediante un script automatizado de auditoría.

> **Definición 0D.1 (El Principio de Reproducibilidad en la Ingeniería)**
> La reproducibilidad computacional exige que cualquier ingeniero, en cualquier sistema operativo, pueda clonar el repositorio del proyecto, ejecutar un único comando de configuración y obtener idénticos esquemas, registros y resultados numéricos sin depender de archivos propietarios o configuraciones opacas.

0D.1 Opciones de Entorno de Laboratorio

El lector puede elegir la alternativa que mejor se adapte a su infraestructura disponible:



Opción A: Laboratorio Completo con Docker Compose (Recomendada)

La especificación moderna de Docker Compose (sin la directiva obsoleta `version: '3.8'`) permite levantar la infraestructura completa con variables de entorno desacopladas.

1. Archivo de Variables de Entorno (`.env.example` → `.env`)

Cree un archivo `.env` en la raíz del proyecto para no exponer contraseñas en texto plano:

INI

```ini
# .env - Variables de configuración del laboratorio local
POSTGRES_DB=laboratorio_db
POSTGRES_USER=data_engineer
POSTGRES_PASSWORD=LabPasswordSeguro2026!
POSTGRES_PORT=5432

MINIO_ROOT_USER=minio_admin
MINIO_ROOT_PASSWORD=MinioSecretPassword2026!
MINIO_API_PORT=9000
MINIO_CONSOLE_PORT=9001

JUPYTER_PORT=8888
SPARK_UI_PORT=4040
```

2. Definición de Servicios (`docker-compose.yml`)

YAML

```yaml
# docker-compose.yml (Especificación Docker Compose Moderna)
services:
  # Entorno interactivo con Apache Spark 3.5 y Delta Lake 3.2
  spark-jupyter:
    image: quay.io/jupyter/pyspark-notebook:spark-3.5.0
    container_name: lab_spark_notebook
    environment:
      JUPYTER_ENABLE_LAB: "yes"
      PYSPARK_SUBMIT_ARGS: "--packages io.delta:delta-spark_2.12:3.2.0 pyspark-
shell"
    ports:
      - "${JUPYTER_PORT:-8888}:8888"   # JupyterLab Web UI
      - "${SPARK_UI_PORT:-4040}:4040"   # Spark Web UI
    volumes:
      - ./libro:/home/jovyan/work/libro
      - ./data:/home/jovyan/work/data
      - ./scripts:/home/jovyan/work/scripts
    networks:
      - red_laboratorio

  # Base de datos relacional para ejercicios OLTP, JDBC y CDC
  postgres:
    image: postgres:16-alpine
    container_name: lab_postgres
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - red_laboratorio

  # MinIO: Almacenamiento de Objetos S3 local (Data Lake)
```

```
  minio:
    image: quay.io/minio/minio:RELEASE.2024-01-31T20-20-33Z
    container_name: lab_minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    ports:
      - "${MINIO_API_PORT:-9000}:9000"
      - "${MINIO_CONSOLE_PORT:-9001}:9001"
    volumes:
      - miniodata:/data
    networks:
      - red_laboratorio

volumes:
  pgdata:
  miniodata:

networks:
  red_laboratorio:
    driver: bridge
```

- **Instrucciones de Despliegue:**

  ```bash
  # Copiar plantilla de variables
  cp .env.example .env

  # Iniciar todos los contenedores en segundo plano
  docker compose up -d
  ```

  Acceda a `http://localhost:8888` en su navegador para interactuar con JupyterLab y ejecutar notebooks de Spark.

Guía Especial para Usuarios de Windows y WSL2

Para desarrolladores en Windows, la experiencia más estable y cercana a producción se obtiene combinando **Docker Desktop** con **WSL2 (Windows Subsystem for Linux)**:

1. **Habilitar WSL2 y Ubuntu:**
   Abra PowerShell como Administrador y ejecute:

   ```powershell
   wsl --install -d Ubuntu-24.04
   ```

2. **Configuración de Límites de Memoria (`.wslconfig`):**
   Por defecto, WSL2 puede llegar a consumir el 80% de la memoria física del equipo durante

operaciones pesadas de Spark. Para evitar que el sistema operativo Windows se ralentice, cree el archivo `C:\Users\\.wslconfig`:

```ini
[wsl2]
memory=8GB            # Límite máximo de RAM asignada a WSL2
processors=4          # Límite de núcleos de CPU
swap=4GB              # Archivo de paginación swap
localhostForwarding=true
```

Reinicie WSL2 desde PowerShell ejecutando: `wsl --shutdown`.

3. **Manejo de Saltos de Línea (CRLF vs LF):**
   Los scripts shell de Linux (`.sh`) fallan con errores crípticos del tipo `\r: command not found` si Git los descarga con saltos de línea de Windows (`CRLF`). Configure Git para respetar saltos de línea Unix:

```bash
git config --global core.autocrlf input
```

4. **Ejecutar dentro del File System de Linux:**
   Para obtener el máximo rendimiento de lectura y escritura en disco (I/O), clone el repositorio dentro del sistema de archivos de WSL2 (ej. `/home/su_usuario/proyectos/`), en lugar de montarlo desde `/mnt/c/`.

Opción B: Databricks Free Edition (Nube sin Costo)

Si su computador dispone de recursos limitados (menos de 8 GB de RAM) o desea experimentar directamente con una plataforma de datos moderna:

1. Regístrese en el portal oficial de **Databricks Free Edition** (`https://community.cloud.databricks.com/` o `https://www.databricks.com/try-databricks`).
2. Cree un clúster de trabajo (*Single Node*, Databricks Runtime 15.4+ LTS con Apache Spark 3.5+ y Delta Lake preinstalado).
3. Clone el repositorio oficial directamente en Databricks utilizando **Workspace Git Folders / Repos**.
4. Trabaje sobre el catálogo predeterminado de **Unity Catalog** (`workspace.default`), familiarizándose con la gobernanza empresarial y linaje desde el primer día.

Opción C: Instalación Nativa Local (Python + PySpark)

Para ejecutar los scripts directamente en su terminal local (Linux, macOS o WSL2):

1. **Instalar Java OpenJDK 17 (Requisito fundamental de la JVM de Spark):**

```bash
# En Ubuntu / Debian / WSL2:
sudo apt-get update && sudo apt-get install -y openjdk-17-jdk-headless
```

2. **Crear el entorno virtual con `uv` e instalar dependencias:**

```bash
uv venv .venv --python 3.11
source .venv/bin/activate  # En Windows: .venv\Scripts\activate
uv pip install pyspark==3.5.2 delta-spark==3.2.0 pyarrow duckdb polars pytest
```

---

oD.2 Generación Oficial de Datasets de Prueba

Para garantizar que los archivos de datos existan en disco y contengan exactamente la distribución matemática descrita en los capítulos de optimización, joins y calidad, ejecute el generador oficial determinista:

```bash
python scripts/generar_datos.py
```

Inventario de Datasets Generados en `data/`

Archivo	Volumen	Formato	Propósito Pedagógico en el Libro
`data/ventas.parquet`	10.000 filas	Parquet / Snappy	Transacciones de comercio electrónico. Base para filtros, agregaciones y particionado temporal.
`data/clientes.parquet`	1.500 filas	Parquet / Snappy	Catálogo maestro de clientes. Empleado en uniones relacionales (Broadcast Joins) y SCD Tipo 2.
`data/eventos_telemetria.json`	1.000 eventos	JSON semiestructurado	Cargas de IoT y clickstream con campos anidados para prácticas de parsing de esquemas.
`data/interacciones.parquet`	25.000 filas	Parquet / Snappy	Eventos de navegación web (clickstream) para análisis de sesiones y funciones de ventana.
`data/usuarios.parquet`	2.000 filas	Parquet / Snappy	Datos con información de identificación personal (PII) para ejercicios de GDPR, enmascaramiento y Ley 1581.
`data/pedidos.parquet`	20.000 filas	Parquet / Snappy	Transacciones detalladas de ventas para indexación, modelado dimensional y joins pesados.
`data/eventos_anuales.parquet`	50.000 filas	Parquet / Snappy	Eventos históricos distribuidos en múltiples años para demostrar poda de particiones (*Partition Pruning*).
`data/transacciones_sesgadas.parquet`	100.000 filas	Parquet / Snappy	Dataset con **sesgo extremo de datos (85% Zipf)** y payload de 256 bytes para reproducir cuellos de botella y *Shuffle Spill* en la interfaz Spark UI.

oD.3 Script de Verificación de Integridad del Laboratorio

Para cerciorarse de que su sesión de Apache Spark está correctamente configurada y puede procesar los archivos generados sin problemas de memoria ni permisos, ejecute:

PYTHON

```python
# scripts/verificar_laboratorio.py
from pyspark.sql import SparkSession

def verificar_laboratorio():
    print("Iniciando auditoría del entorno de Apache Spark...")
    spark = SparkSession.builder \
        .appName("AuditoriaLaboratorio") \
```

```
        .master("local[*]") \
        .config("spark.sql.shuffle.partitions", "4") \
        .getOrCreate()

    print(f"Versión de Apache Spark activa: {spark.version}")

    ruta_ventas = "data/ventas.parquet"
    df_ventas = spark.read.parquet(ruta_ventas)

    total_filas = df_ventas.count()
    print(f"Archivo '{ruta_ventas}' leído exitosamente. Total registros:
{total_filas}")

    df_ventas.createOrReplaceTempView("ventas_test")
    resumen = spark.sql("""
        SELECT pais, COUNT(transaccion_id) as total_tx, ROUND(SUM(monto_usd), 2)
as facturacion
        FROM ventas_test
        GROUP BY pais
        ORDER BY facturacion DESC
        LIMIT 3
    """)
    resumen.show()
    print("¡Laboratorio verificado y 100% operativo para el estudio del libro!")

if __name__ == "__main__":
    verificar_laboratorio()
```

oD.4 Resumen y Puntos Clave

- **Cero código sobre el vacío:** Todo el código del libro es inmediatamente reproducible clonando el repositorio oficial de GitHub: `https://github.com/luisgalvis/como-ser-un-data-engineer`.
- **Seguridad desde el primer día:** Docker Compose utiliza la especificación moderna con variables de entorno desacopladas en `.env`, eliminando credenciales en texto plano.
- **Compatibilidad Windows:** La configuración de WSL2 y límites de memoria en `.wslconfig` protege el sistema anfitrión frente a bloqueos de recursos.
- **Suite completa de datos:** El script `scripts/generar_datos.py` produce deterministamente 8 datasets calibrados para reproducir desde agregaciones básicas hasta *spills* severos por sesgo de datos (*data skew*).

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide (Chapter 2: A Gentle Introduction to Spark).* O'Reilly Media.

2. **Docker Documentation (2024).** *The Compose Specification & Environment Variables in Compose.* Docker Docs.
3. **Microsoft Learn (2024).** *Advanced Settings Configuration in WSL2 (.wslconfig).* Microsoft Documentation.
4. **Databricks Learning (2024).** *Databricks Free Edition & Unity Catalog Quickstart.* Databricks Documentation.

CAPÍTULO 1

Fundamentos de SQL Declarativo y Computación Analítica NÚCLEO

> *"SQL en la ingeniería de datos no es una simple sintaxis para consultar tablas estáticas; es la interfaz declarativa universal que permite expresar qué resultado matemático se requiere, delegando en el motor computacional la generación del plan de ejecución físico óptimo."*

1.1 Intuición Inicial: Pedir un Café en Lugar de Operar la Máquina

Cuando entras a una cafetería, le dices al barista: «*Por favor, un café americano grande, bien caliente y sin azúcar*». No necesitas entrar a la cocina a calibrar el molinillo de granos, medir los gramos de café molido, hervir el agua a 92 grados centígrados ni bombear la presión del agua a través del filtro. Describes con exactitud el resultado final que deseas (**QUÉ quieres**), y el barista se encarga de ejecutar la secuencia mecánica precisa (**CÓMO prepararlo**).

SQL (Structured Query Language) funciona exactamente bajo este principio declarativo. En lugar de programar bucles `for`, abrir archivos en bajo nivel, asignar punteros de memoria y recorrer registros uno a uno (como se haría en C o Java imperativo), en SQL describes las condiciones lógicas de los datos que necesitas. El motor de la base de datos (PostgreSQL, DuckDB o Apache Spark) dispone de un optimizador interno —el barista experto— que analiza los índices, el tamaño físico de las tablas y la memoria disponible para construir el plan de ejecución más rápido posible.

Si comienzas desde cero, este capítulo te guiará paso a paso: desde seleccionar tus primeras columnas y filtrar filas, hasta realizar uniones relacionales complejas, resolver casos de lógica trivaluada con valores nulos e introducir los fundamentos del modelado dimensional analítico.

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la diferencia fundamental entre el paradigma declarativo de SQL y la programación procedural/imperativa.
- Escribir consultas SQL analíticas desde cero (`SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`) dominando el orden lógico de ejecución del motor.
- Aplicar con precisión todos los tipos de uniones relacionales (`INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`, `LEFT SEMI`, `LEFT ANTI`) y gestionar los efectos de la lógica trivaluada ante valores `NULL`.
- Construir transformaciones modulares legibles mediante Expresiones de Tabla Comunes (CTEs) y CTEs recursivas.
- Dominar operaciones de sincronización e inserción atómica mediante `MERGE INTO`.
- Comprender los principios del **Modelado Dimensional de Ralph Kimball** (tablas de hechos, dimensiones y esquemas en estrella) frente a la normalización transaccional 3NF.

Profundiza: El Fundamento Formal del Álgebra y Cálculo Relacional

Definición 1.1 (Cálculo Relacional y Paradigma Declarativo)
En la teoría matemática de bases de datos formulada por Edgar F. Codd (1970), SQL se fundamenta en el **cálculo relacional de tuplas**: el ingeniero especifica el predicado analítico y la proyección relacional deseada (*qué calcular*), abstrayéndose por completo de la secuencia de pasos mecánicos (*cómo calcularlo*).

En contraste con la programación imperativa —donde el programador administra bucles, punteros de memoria y buffers de red—, el paradigma declarativo traslada la carga de optimización al planificador del motor (Catalyst en Spark, optimizadores basados en costos en PostgreSQL o DuckDB). Este analiza la consulta, evalúa reglas de equivalencia algebraica y compila un árbol de ejecución físico adaptado a los recursos disponibles.

1.2 Consultas SQL desde Cero: Anatomía y Orden Lógico de Ejecución

Para un ingeniero de datos, dominar SQL va más allá de recordar palabras clave: requiere asimilar el modelo relacional formal propuesto por Edgar F. Codd y comprender el orden exacto en que los motores de bases de datos compilan y ejecutan las instrucciones.

1. El Modelo Relacional Básico

- **Relación (Tabla):** Conjunto no ordenado de tuplas únicas con un esquema definido.
- **Tupla (Fila o Registro):** Instancia única de una entidad en el mundo real.
- **Atributo (Columna):** Dimensión con un tipo de dato y restricciones específicas.

2. Proyección y Filtrado Básico (`SELECT` y `WHERE`)

La consulta elemental extrae y transforma columnas de una tabla:

```sql
-- Selección de columnas específicas con alias y transformaciones aritméticas
SELECT
    transaccion_id,
    cliente_id,
    pais,
    cantidad * precio_unitario AS subtotal_usd,
    fecha_transaccion
FROM ventas_analitica.ventas
WHERE estado_pago = 'COMPLETADO'
  AND (pais = 'CO' OR pais = 'MX')
  AND fecha_transaccion >= '2026-01-01'
  AND monto_usd BETWEEN 50.0 AND 5000.0;
```

Predicados y Operadores de Filtrado Esenciales

- **Igualdad y Desigualdad:** `=`, `<>` (o `!=`).
- **Comparaciones:** `>`, `<`, `>=`, `<=`.
- **Conjuntos y Rangos:** `IN ('CO', 'PE', 'CL')`, `BETWEEN 10 AND 50`.
- **Coincidencia de Patrones de Texto:** `LIKE 'TX-%'` (empieza con TX-), `LIKE '%@empresa.com'`.
- **Evaluación de Nulos:** `IS NULL` y `IS NOT NULL` (nunca use `= NULL`).

3. Agrupaciones y Filtrado Post-Agregación (`GROUP BY` y `HAVING`)

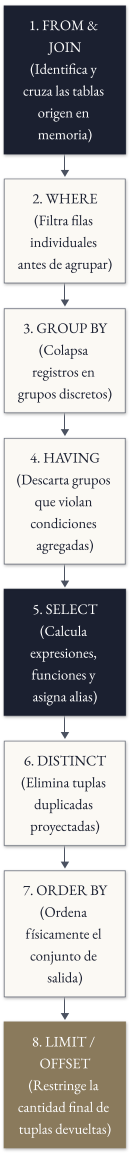
Una de las confusiones más comunes al iniciar es distinguir el momento de filtrado de filas frente al de grupos:

- `WHERE`: Filtra **filas individuales antes** de que se realice cualquier agrupación o cálculo de agregación.
- `GROUP BY`: Colapsa múltiples filas que comparten valores idénticos en una sola fila resumen por grupo.
- `HAVING`: Filtra **grupos resultantes después** de haber calculado las funciones de agregación (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`).

```sql
SELECT
    pais,
    categoria_producto,
    COUNT(transaccion_id) AS total_pedidos,
    ROUND(SUM(monto_usd), 2) AS facturacion_total,
    ROUND(AVG(monto_usd), 2) AS ticket_promedio
FROM ventas_analitica.ventas
WHERE estado_pago = 'COMPLETADO'      -- 1. Filtra filas individuales primero
GROUP BY pais, categoria_producto     -- 2. Agrupa por país y categoría
HAVING COUNT(transaccion_id) >= 100  -- 3. Conserva solo grupos con más de 100
ventas
   AND SUM(monto_usd) > 50000.0
ORDER BY facturacion_total DESC       -- 4. Ordena el resultado agregado
LIMIT 10;                             -- 5. Extrae los 10 primeros registros
```

4. El Orden Lógico de Procesamiento de una Consulta SQL

Un error habitual es creer que la base de datos ejecuta la consulta en el orden en que se escribe (`SELECT` primero). En realidad, el optimizador procesa la consulta en una secuencia lógica matemática estricta:



> **Por qué importa esto al depurar:** Debido a que `WHERE` se evalúa en el paso 2 y `SELECT` se evalúa en el paso 5, **no es posible usar un alias definido en el `SELECT` dentro de la cláusula `WHERE`** (provocará un error de columna desconocida). Comprender este orden evita horas de frustración al estructurar pipelines.

1.3 Lenguaje de Definición (DDL) y Manipulación de Datos (DML)

Toda plataforma analítica interactúa con esquemas relacionales a través de dos subsistemas fundamentales:

1. DDL (Data Definition Language)

Permite definir, alterar y destruir estructuras tabulares, tipos de datos y restricciones de integridad:

```SQL
-- Creación de esquema y tabla analítica con restricciones (PostgreSQL / DuckDB / Databricks)
CREATE SCHEMA IF NOT EXISTS ventas_analitica;

CREATE TABLE ventas_analitica.clientes (
    cliente_id BIGINT NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    correo VARCHAR(200) NOT NULL,
    pais_iso CHAR(2) NOT NULL,
    segmento VARCHAR(50) DEFAULT 'ESTANDAR',
    fecha_registro DATE NOT NULL,
    CONSTRAINT pk_clientes PRIMARY KEY (cliente_id)
);
```

2. DML (Data Manipulation Language) y la Operación Atómica `MERGE INTO`

El DML gobierna la inserción, actualización y eliminación de registros:

```SQL
-- Inserción directa de registros
INSERT INTO ventas_analitica.clientes (cliente_id, nombre, correo, pais_iso, segmento, fecha_registro)
VALUES
    (101, 'Ana Torres', 'ana.torres@empresa.com', 'CO', 'ENTERPRISE', '2025-01-15'),
    (102, 'Carlos Ruiz', 'carlos.ruiz@empresa.com', 'MX', 'ESTANDAR', '2025-02-20'),
    (103, 'Elena Mora', 'elena.mora@empresa.com', 'CO', 'PRO', '2025-03-05');
```

En pipelines analíticos modernos, la inserción y actualización manual (`UPDATE` y `DELETE` tradicionales) son sustituidas por la sentencia atómica **`MERGE INTO`** (estándar ANSI SQL:2006 y pilar central de formatos Lakehouse como Delta Lake e Iceberg):

```sql
-- Sincronización atómica tipo Upsert (Update + Insert) usando MERGE INTO
MERGE INTO ventas_analitica.clientes AS destino
USING ventas_analitica.novedades_staging AS origen
ON destino.cliente_id = origen.cliente_id
WHEN MATCHED AND origen.correo <> destino.correo THEN
    UPDATE SET
        destino.correo = origen.correo,
        destino.segmento = origen.segmento
WHEN NOT MATCHED THEN
    INSERT (cliente_id, nombre, correo, pais_iso, segmento, fecha_registro)
    VALUES (origen.cliente_id, origen.nombre, origen.correo, origen.pais_iso,
origen.segmento, origen.fecha_registro);
```

1.4 Expresiones Condicionales, Funciones de Fecha y JSON

Lógica Condicional con `CASE WHEN` y Manejo de Nulos

En SQL, la sentencia `CASE` es una expresión escalar determinista:

```sql
SELECT
    cliente_id,
    monto_usd,
    CASE
        WHEN monto_usd >= 10000 THEN 'ALTO VALOR'
        WHEN monto_usd >= 1000 THEN 'VALOR MEDIO'
        ELSE 'MINORISTA'
    END AS clasificacion_cliente,
    COALESCE(descuento_usd, 0.0) AS descuento_aplicado,
    NULLIF(dias_mora, 0) AS mora_efectiva -- Convierte ceros en NULL para no
distorsionar promedios
FROM ventas_analitica.facturacion;
```

Aritmética de Fechas y Marcas de Tiempo

Las transformaciones analíticas requieren manipular dimensiones temporales con precisión de zona horaria:

```sql
SELECT
    pedido_id,
    fecha_hora_creacion,
    -- Truncamiento a inicio de mes
    DATE_TRUNC('month', fecha_hora_creacion) AS mes_periodo,
    -- Extracción de componentes
    EXTRACT(YEAR FROM fecha_hora_creacion) AS anio_fiscal,
    -- Diferencia temporal en días
    CAST(fecha_entrega AS DATE) - CAST(fecha_hora_creacion AS DATE) AS
dias_transito,
    -- Suma de intervalos
    fecha_hora_creacion + INTERVAL '30 days' AS vencimiento_garantia
FROM ventas_analitica.pedidos;
```

Procesamiento de Estructuras Jerárquicas JSON

En arquitecturas de ingesta moderna, los eventos provienen de APIs o Kafka en formato JSON. SQL analítico provee operadores especializados:

```sql
-- Sintaxis PostgreSQL (operadores de ruta jsonb -> y ->>)
SELECT
    evento_id,
    payload_json->>'usuario_id' AS usuario_id,
    payload_json->'dispositivo'->>'sistema_operativo' AS so_cliente,
    CAST(payload_json->>'monto' AS NUMERIC(10,2)) AS monto_transaccion
FROM ventas_analitica.eventos_raw;

-- Sintaxis estándar Spark SQL / DuckDB equivalente
SELECT
    evento_id,
    get_json_object(payload_str, '$.usuario_id') AS usuario_id,
    get_json_object(payload_str, '$.dispositivo.sistema_operativo') AS so_cliente
FROM ventas_analitica.eventos_raw;
```

1.5 Subconsultas y Operaciones de Conjuntos

Subconsultas Escalares y Correlacionadas

Una subconsulta es **correlacionada** cuando depende de una fila procesada por la consulta externa, evaluándose conceptualmente para cada registro:

```sql
-- Obtener pedidos cuyo monto supera el promedio de su respectivo país
(Subconsulta correlacionada)
SELECT p1.pedido_id, p1.pais_id, p1.monto_usd
FROM ventas_analitica.pedidos AS p1
WHERE p1.monto_usd > (
    SELECT AVG(p2.monto_usd)
    FROM ventas_analitica.pedidos AS p2
    WHERE p2.pais_id = p1.pais_id
);
```

Operadores de Existencia: `EXISTS` frente a `IN`

- `IN`: Evalúa una lista completa de valores. Si la subconsulta contiene un valor `NULL`, evaluar `NOT IN` puede devolver un conjunto vacío de forma inesperada debido a la lógica trivaluada (`NOT (x IN (1, NULL))` evalúa a `UNKNOWN`).
- `EXISTS`: Evalúa la presencia de al menos una tupla que satisfaga la condición mediante un predicado booleano, permitiendo al optimizador detener el escaneo (*short-circuit evaluation*):

```sql
-- Identificar clientes con al menos una compra en los últimos 90 días
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
WHERE EXISTS (
    SELECT 1
    FROM ventas_analitica.pedidos AS p
    WHERE p.cliente_id = c.cliente_id
      AND p.fecha_creacion >= CURRENT_DATE - INTERVAL '90 days'
);
```

Operaciones de Conjuntos (`UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`)

Operación	Semántica Relacional	Costo Computacional
`UNION ALL`	Concatena los registros de dos consultas preservando duplicados.	**Bajo ($O(N)$):** No requiere ordenar ni crear tablas hash.
`UNION`	Concatena y elimina tuplas duplicadas.	**Alto ($O(N \log N)$):** Fuerza una fase de deduplicación o hash de conjunto.
`INTERSECT`	Devuelve solo los registros presentes en ambos conjuntos.	Requiere coincidencia de todas las columnas.
`EXCEPT` / `MINUS`	Devuelve los registros del primer conjunto que no existen en el segundo.	Útil para pruebas de conciliación y calidad de datos.

1.6 Expresiones de Tabla Comunes (CTEs) y CTEs Recursivas

Las **CTEs** (*Common Table Expressions*, introducidas mediante la cláusula `WITH`) transforman consultas anidadas impenetrables en secuencias lógicas modulares, legibles y reutilizables.



```sql
WITH ventas_validas AS (
    SELECT
        pedido_id,
        cliente_id,
        monto_usd,
        DATE_TRUNC('month', fecha_creacion) AS mes
    FROM ventas_analitica.pedidos
    WHERE estado = 'COMPLETADO'
),
metricas_cliente AS (
    SELECT
        cliente_id,
        mes,
        COUNT(pedido_id) AS total_pedidos,
        SUM(monto_usd) AS gasto_mensual
    FROM ventas_validas
    GROUP BY cliente_id, mes
)
SELECT
    c.nombre,
    c.pais_iso,
    m.mes,
    m.total_pedidos,
    m.gasto_mensual
FROM metricas_cliente AS m
INNER JOIN ventas_analitica.clientes AS c ON m.cliente_id = c.cliente_id
WHERE m.gasto_mensual > 1000;
```

CTEs Recursivas para Estructuras Jerárquicas

En ingeniería de datos es común modelar jerarquías organizacionales, árboles de categorías o cadenas de linaje (*data lineage*). Una CTE recursiva consta de un **miembro ancla** y un **miembro recursivo** unidos por `UNION ALL`:

```sql
-- Recorrido de una jerarquía de categorías de productos (Árbol relacional)
WITH RECURSIVE jerarquia_categorias AS (
    -- Miembro Ancla: Categorías raíz (sin padre)
    SELECT
        categoria_id,
        nombre_categoria,
        categoria_padre_id,
        1 AS nivel_profundidad,
        CAST(nombre_categoria AS VARCHAR(1000)) AS ruta_arbol
    FROM ventas_analitica.categorias
    WHERE categoria_padre_id IS NULL

    UNION ALL

    -- Miembro Recursivo: Hijos unidos con la iteración previa
    SELECT
        c.categoria_id,
        c.nombre_categoria,
        c.categoria_padre_id,
        j.nivel_profundidad + 1,
        CAST(j.ruta_arbol || ' > ' || c.nombre_categoria AS VARCHAR(1000))
    FROM ventas_analitica.categorias AS c
    INNER JOIN jerarquia_categorias AS j ON c.categoria_padre_id = j.categoria_id
)
SELECT categoria_id, nombre_categoria, nivel_profundidad, ruta_arbol
FROM jerarquia_categorias
ORDER BY ruta_arbol;
```

1.7 Tipos de Uniones (JOINs) y el Peligro de la Lógica Trivaluada (3VL)

El operador relacional `JOIN` combina tuplas de dos relaciones en función de una condición de acoplamiento:



1. `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN` y `FULL OUTER JOIN`

- `INNER JOIN`: Devuelve únicamente tuplas que satisfacen el predicado en ambas tablas.

- `LEFT OUTER JOIN`: Preserva todas las tuplas de la tabla izquierda. Si no hay coincidencia en la derecha, rellena sus columnas con `NULL`.
- `FULL OUTER JOIN`: Preserva todas las tuplas de ambos extremos, insertando `NULL` en los lados sin correspondencia.

2. Uniones de Filtrado: `LEFT SEMI JOIN` y `LEFT ANTI JOIN` vs. `EXISTS` / `NOT EXISTS`

Ampliamente utilizadas en motores analíticos distribuidos (Spark SQL, Trino, Databricks, DuckDB, Snowflake):

- `LEFT SEMI JOIN`: Retorna las columnas de la tabla izquierda para aquellas filas que tienen al menos una correspondencia en la derecha. **A diferencia del `INNER JOIN`, nunca duplica filas de la tabla izquierda**, incluso si hay múltiples coincidencias en la derecha. En SQL estándar (como PostgreSQL), su equivalente canónico es `WHERE EXISTS (...)`.
- `LEFT ANTI JOIN`: Retorna únicamente las filas de la tabla izquierda que **no tienen correspondencia** en la derecha. Es la alternativa más limpia y eficiente al peligroso `WHERE id NOT IN (...)`.

> **Nota de Portabilidad:** La sintaxis explícita `LEFT ANTI JOIN` y `LEFT SEMI JOIN` es nativa de motores OLAP modernos (Spark SQL, DuckDB, Databricks, Trino, BigQuery). En motores relacionales tradicionales como PostgreSQL o SQL Server estándar, no existe la palabra clave `ANTI JOIN`; en ellos la forma estándar ANSI equivalente y óptima es `WHERE NOT EXISTS (...)`:

```sql
-- En Spark SQL, DuckDB, Trino y Snowflake:
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
LEFT ANTI JOIN ventas_analitica.pedidos AS p ON c.cliente_id = p.cliente_id;

-- Equivalente estándar ANSI (PostgreSQL, MySQL, SQL Server):
SELECT c.cliente_id, c.nombre
FROM ventas_analitica.clientes AS c
WHERE NOT EXISTS (
    SELECT 1 FROM ventas_analitica.pedidos AS p
    WHERE p.cliente_id = c.cliente_id
);
```

3. La Lógica Trivaluada (Three-Valued Logic - 3VL) y Claves `NULL`

En SQL, una comparación lógica puede evaluar a `TRUE`, `FALSE` o `UNKNOWN` (desconocido).

- **Regla Crítica:** `NULL = NULL` evalúa a `UNKNOWN`, no a `TRUE`.
- En un `INNER JOIN`, dos tuplas que tengan `NULL` en su clave de unión **nunca coinciden**.

- Si una clave foránea contiene `NULL`, dichas filas desaparecen de un `INNER JOIN`. En un `FULL JOIN`, cada fila con clave `NULL` formará una tupla independiente emparejada con nulos en el lado opuesto.

SQL

```sql
-- Demostración de igualdad segura frente a NULL (Null-Safe Equality):
-- 1. En Spark SQL y MySQL (operador <=>):
SELECT *
FROM tabla_a AS a
INNER JOIN tabla_b AS b ON a.codigo <=> b.codigo;

-- 2. En SQL Estándar ANSI y PostgreSQL (predicado IS NOT DISTINCT FROM):
SELECT *
FROM tabla_a AS a
INNER JOIN tabla_b AS b ON a.codigo IS NOT DISTINCT FROM b.codigo;
```

1.8 De SQL Estándar a Spark SQL y Vistas Temporales

En Apache Spark, la interfaz SQL es de primera clase y comparte exactamente el mismo optimizador Catalyst que la API DataFrame:

PYTHON

```python
from pyspark.sql import SparkSession

# Inicializar sesión de Spark
spark = SparkSession.builder \
    .appName("Capitulo01-SQL-Analitico") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset estructurado
df_ventas = spark.read.parquet("data/ventas.parquet")

# Registrar vista temporal en el catálogo en memoria de Spark
df_ventas.createOrReplaceTempView("v_ventas")

# Consulta analítica avanzada con CTEs y Anti-Join
consulta_spark_sql = """
    WITH clientes_activos AS (
        SELECT cliente_id, SUM(monto_usd) AS volumen_total
        FROM v_ventas
        WHERE fecha_transaccion >= '2026-01-01'
        GROUP BY cliente_id
    )
    SELECT
        v.pais,
        COUNT(DISTINCT v.cliente_id) AS total_clientes,
        SUM(v.monto_usd) AS facturacion_pais
    FROM v_ventas AS v
    LEFT SEMI JOIN clientes_activos AS ca ON v.cliente_id = ca.cliente_id
```

```
    GROUP BY v.pais
    ORDER BY facturacion_pais DESC
"""

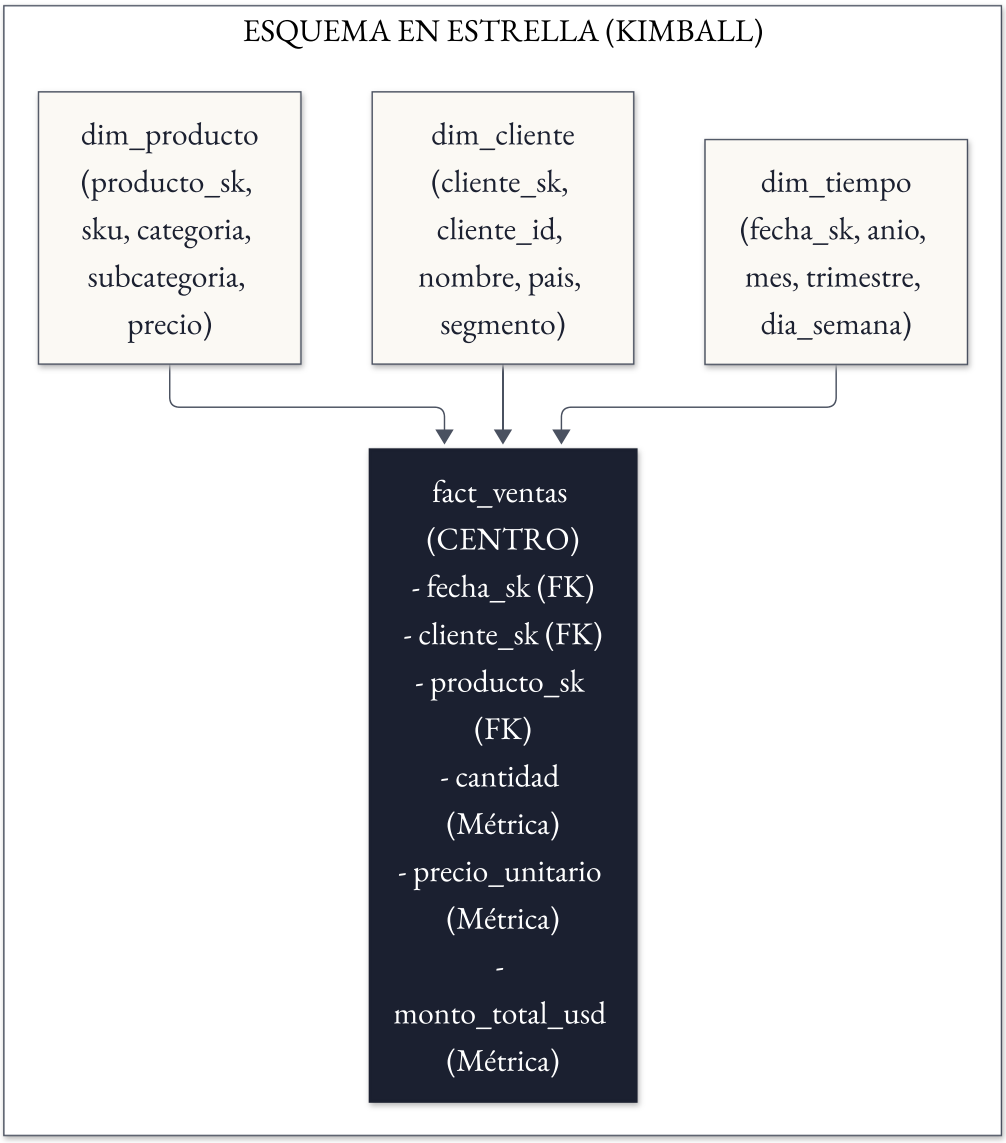
df_resultado = spark.sql(consulta_spark_sql)
df_resultado.show(5)
```

1.9 Introducción al Modelado Dimensional: Kimball vs. Normalización 3NF

Uno de los saltos cualitativos que diferencian a un programador de un verdadero ingeniero de datos es comprender **por qué estructuramos los datos de forma diferente para operar que para analizar**.

1. La Dualidad de Sistemas: OLTP frente a OLAP

- **Sistemas Transaccionales (OLTP):** Diseñados en **Tercera Forma Normal (3NF)**. Su meta es evitar redundancia: si un cliente cambia de correo electrónico, se modifica una única fila en una tabla aislada para evitar inconsistencias y bloqueos de escritura. Sin embargo, responder a una pregunta como *«¿Cuáles fueron las ventas por categoría en Colombia el último trimestre?»* requiere unir 8 o 10 tablas normalizadas con un costo computacional prohibitivo.
- **Sistemas Analíticos (OLAP / Data Warehouse):** Diseñados bajo la metodología dimensional de **Ralph Kimball**. Su meta es optimizar la lectura analítica masiva y la simplicidad semántica, organizando los datos en **Tablas de Hechos** y **Tablas de Dimensiones**.



2. Anatomía de una Tabla de Hechos (*Fact Table*)

Una tabla de hechos registra eventos numéricos medibles que ocurren en el negocio:

- **El Grano (*Granularity*):** Es el principio más sagrado de Kimball. Define con precisión quirúrgica qué representa **una sola fila** de la tabla. Por ejemplo: «*Una fila representa una línea individual de producto en un ticket de compra completado*». Si el grano se mezcla o confunde, los cálculos de agregación (`SUM`, `AVG`) duplicarán importes.
- **Métricas Aditivas:** Valores numéricos que pueden sumarse válidamente a través de todas las dimensiones (`monto_usd`, `cantidad_unidades`).

- **Claves Foráneas:** Identificadores que apuntan a las dimensiones que contextualizan el hecho.

3. Anatomía de una Tabla de Dimensiones (*Dimension Table*)

Las dimensiones proporcionan el contexto descriptivo respondiendo a las preguntas de negocio: *¿Quién compró?*, *¿Dónde?*, *¿Cuándo?*, *¿Qué artículo?*.

- **Claves Subrogadas (*Surrogate Keys*):** Claves primarias artificiales (enteros secuenciales o hashes `cliente_sk`) gestionadas por el almacén de datos, independientes de la clave natural del ERP origen (`cliente_id`). Permiten manejar miembros desconocidos (clave `-1`) y registrar cambios históricos de atributos (dimensiones lentamente cambiantes o SCD).
- **Esquema en Estrella (*Star Schema*) frente a Copo de Nieve (*Snowflake*):**
  En un esquema en estrella, las dimensiones están completamente desnormalizadas en una sola tabla (ej. `dim_producto` incluye su categoría y departamento directamente). En el esquema copo de nieve, las dimensiones se normalizan parcialmente (ej. `dim_producto` se une a `dim_subcategoria` que a su vez se une a `dim_categoria`).
  Veredicto de Producción: Los motores analíticos modernos (DuckDB, Snowflake, Databricks, BigQuery) están optimizados para formatos columnares donde el almacenamiento es barato y los saltos entre múltiples tablas degradan el rendimiento. **El Esquema en Estrella es el estándar dominante de la industria.**

4. Consulta Analítica Canónica sobre Esquema en Estrella

Observe cómo una consulta que requeriría 8 uniones en una base relacional 3NF se reduce a simples cruces directos contra la tabla central de hechos:

SQL

```sql
SELECT
    d_cli.pais,
    d_prod.categoria,
    d_tiem.anio,
    d_tiem.mes,
    COUNT(f.venta_id) AS total_transacciones,
    ROUND(SUM(f.monto_total_usd), 2) AS facturacion_total
FROM fact_ventas AS f
INNER JOIN dim_cliente AS d_cli
    ON f.cliente_sk = d_cli.cliente_sk
INNER JOIN dim_producto AS d_prod
    ON f.producto_sk = d_prod.producto_sk
INNER JOIN dim_tiempo AS d_tiem
    ON f.fecha_sk = d_tiem.fecha_sk
WHERE d_tiem.anio = 2026
  AND d_cli.pais IN ('CO', 'MX', 'CL')
GROUP BY d_cli.pais, d_prod.categoria, d_tiem.anio, d_tiem.mes
ORDER BY facturacion_total DESC;
```

1.10 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Uso Inseguro de `NOT IN` con Columnas Anulables
Si la subconsulta de un `NOT IN` devuelve una sola tupla con valor `NULL`, toda la condición externa evalúa a `UNKNOWN` y la consulta devuelve cero filas.
Corrección: Utilice siempre `LEFT ANTI JOIN` o `WHERE NOT EXISTS (...)`.

Anti-patrón 2: Producto Cartesiano Inadvertido (`CROSS JOIN`)
Omitir la cláusula `ON` en un join genera una combinación de cada fila izquierda con cada fila derecha ($M \times N$ tuplas). Si la tabla izquierda tiene 1 millón de registros y la derecha 1 millón, el motor intentará generar 10^{12} filas (1 billón de registros), saturando la memoria y colapsando el clúster.

Anti-patrón 3: Funciones No SARGables en la Cláusula `WHERE`
Aplicar funciones sobre columnas indexadas o particionadas (por ejemplo, `WHERE YEAR(fecha_registro) = 2026`) impide que el optimizador aplique poda de particiones (*partition pruning*) o busque en índices B-Tree.
Corrección: Escriba predicados SARGables (*Search Argument Able*): `WHERE fecha_registro >= '2026-01-01' AND fecha_registro < '2027-01-01'`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 1.1 (Deduplicación Analítica e Identificación de Huérfanos):
Se tienen dos tablas en un Data Lakehouse: `transacciones` (id, cliente_id, monto, fecha) y `clientes_registrados` (cliente_id, nombre, pais).
Escriba una consulta SQL estándar que cumpla con los siguientes dos requerimientos:

1. Extraer todas las transacciones de clientes que **no figuren** en la tabla de `clientes_registrados` (transacciones huérfanas).
2. Garantizar que la consulta sea inmune a valores `NULL` en `clientes_registrados.cliente_id` y no utilice `NOT IN`.

Solución 1.1:

```SQL
SELECT
    t.id AS transaccion_id,
    t.cliente_id,
    t.monto,
    t.fecha
FROM transacciones AS t
LEFT ANTI JOIN clientes_registrados AS c ON t.cliente_id = c.cliente_id;
```

Explicación Técnica: El operador `LEFT ANTI JOIN` evalúa la ausencia de concordancia relacional a nivel de álgebra relacional. Al comparar mediante punteros de unión física (Hash Anti Join), es inmune a la presencia de valores `NULL` en la tabla de la derecha y no sufre de la degradación a `UNKNOWN` de `NOT IN`.

Ejercicio 1.2 (CTE Recursiva y Cálculo de Jerarquías):
En una estructura de empleados donde cada tupla tiene `(empleado_id, jefe_id, salario)`, escriba una CTE recursiva que determine la cadena de mando completa para el empleado con `empleado_id = 500`, mostrando su nivel de profundidad hasta el Director General (quien tiene `jefe_id IS NULL`).

Solución 1.2:

```SQL
WITH RECURSIVE cadena_mando AS (
    -- Miembro Ancla: El empleado de interés
    SELECT
        empleado_id,
        jefe_id,
        salario,
        1 AS nivel
    FROM empleados
    WHERE empleado_id = 500

    UNION ALL

    -- Miembro Recursivo: Ascendemos hacia el jefe directo
    SELECT
        e.empleado_id,
        e.jefe_id,
        e.salario,
        cm.nivel + 1
    FROM empleados AS e
    INNER JOIN cadena_mando AS cm ON e.empleado_id = cm.jefe_id
)
SELECT *
FROM cadena_mando
ORDER BY nivel ASC;
```

1.11 Resumen y Puntos Clave

- **SQL es un lenguaje declarativo:** La separación entre especificación lógica (*qué*) y ejecución física (*cómo*) permite al motor optimizar planes de consulta sin alterar la intención de negocio.
- **Operación atómica `MERGE INTO`:** Es el pilar fundamental para cargas incrementales y sincronizaciones *Upsert* idempotentes en tablas transaccionales.

- **CTEs y Legibilidad:** Permiten construir pipelines de transformación modulares en lugar de complejas subconsultas anidadas.
- **Joins de Filtrado:** `LEFT SEMI JOIN` y `LEFT ANTI JOIN` proporcionan una semántica limpia y altamente optimizada para comprobaciones de existencia y exclusión.
- **Lógica trivaluada (3VL):** Valores `NULL` en comparaciones booleanas producen `UNKNOWN`, lo que exige extrema precaución en uniones y cláusulas `NOT IN`.
- **Modelado Dimensional (Kimball):** Mientras OLTP normaliza en 3NF para transacciones atómicas, los sistemas analíticos organizan datos en esquemas en estrella (hechos medibles rodeados de dimensiones de contexto) para maximizar la velocidad de lectura analítica.

Lecturas Recomendadas

1. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
2. **Date, C. J. (2019).** *SQL and Relational Theory: How to Write Accurate SQL Code (3rd Edition).* O'Reilly Media.
3. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 2: Data Models and Query Languages).* O'Reilly Media.
4. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* ACM SIGMOD.

CAPÍTULO 2

Agrupaciones Multidimensionales, Window Functions y Optimización

NÚCLEO

> *"Al agregar datos reducimos la cardinalidad de un conjunto, pero en sistemas analíticos y distribuidos desatamos una de las operaciones físicas más complejas: el reagrupamiento en memoria y el intercambio de particiones a través de la red (shuffle). Dominar las funciones analíticas de ventana y las extensiones multidimensionales es lo que distingue a un generador de consultas de un verdadero ingeniero de datos."*

Objetivos de Aprendizaje

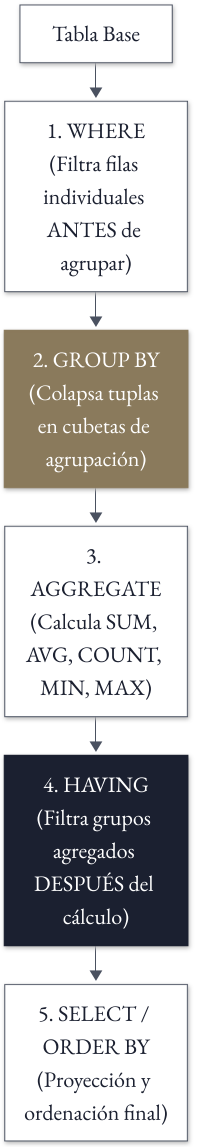
Al finalizar este capítulo, el lector será capaz de:

- Dominar la semántica de agregación con `GROUP BY` y diferenciar con precisión matemática los alcances de `WHERE` frente a `HAVING`.
- Implementar agregaciones multidimensionales para capas analíticas utilizando `ROLLUP`, `CUBE` y `GROUPING SETS`.
- Realizar transformaciones de rotación tabular mediante operadores `PIVOT` y `UNPIVOT`.
- Explicar y aplicar la anatomía completa de las **Window Functions** (*Funciones de Ventana*): cláusulas `PARTITION BY`, `ORDER BY` y especificación de marcos (`ROWS` frente a `RANGE BETWEEN`).
- Utilizar con solvencia funciones de ranking (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`) y de desplazamiento (`LAG`, `LEAD`).
- Resolver problemas críticos de ingeniería como deduplicación por versión más reciente (*latest record*) y cálculo de métricas acumulativas continuas.
- Inspeccionar planes de ejecución con `EXPLAIN` para identificar operadores de agregación física (*HashAggregate* frente a *SortAggregate*).

> **Definición 2.1 (La Mecánica del Marco de Ventana)** A diferencia de una agregación clásica con `GROUP BY` —la cual colapsa múltiples filas en una sola tupla de resumen perdiendo la identidad individual del registro—, una **Window Function** (*Función de Ventana*) calcula un valor analítico sobre un subconjunto de filas relacionadas (*el marco o ventana*), **preservando intacta cada fila individual en la salida**.
>
> Esta propiedad permite yuxtaponer métricas agregadas (totales acumulados, promedios móviles, rankings) junto con los atributos atómicos de cada tupla sin requerir auto-uniones (*self-joins*) costosas.

2.1 Agrupaciones Fundamentales y Filtrado: `WHERE` frente a `HAVING`

El procesamiento de una consulta relacional con agregación sigue un orden lógico estricto:


Tabla Base
1. WHERE (Filtra filas individuales ANTES de agrupar)
2. GROUP BY (Colapsa tuplas en cubetas de agrupación)
3. AGGREGATE (Calcula SUM, AVG, COUNT, MIN, MAX)
4. HAVING (Filtra grupos agregados DESPUÉS del cálculo)
5. SELECT / ORDER BY (Proyección y ordenación final)

```sql
SELECT
    pais_iso,
    categoria_id,
    COUNT(pedido_id) AS total_pedidos,
    SUM(monto_usd) AS facturacion_total,
    AVG(monto_usd) AS ticket_promedio
FROM ventas_analitica.pedidos
WHERE estado = 'ENTREGADO' -- Predicado de fila: se descartan pedidos no entregados ANTES del shuffle
GROUP BY pais_iso, categoria_id
HAVING SUM(monto_usd) >= 50000 -- Predicado de grupo: filtra agregaciones ya calculadas
ORDER BY facturacion_total DESC;
```

2.2 Extensiones Multidimensionales: `ROLLUP`, `CUBE` y `GROUPING SETS`

En el modelado analítico para inteligencia de negocios (BI), es habitual generar subtotales y totales generales en una sola pasada sobre los datos, evitando emitir múltiples consultas unidas con `UNION ALL`.

1. `GROUPING SETS`

Permite declarar explícitamente qué combinaciones de dimensiones deben ser agregadas:

```sql
SELECT
    pais_iso,
    categoria_id,
    SUM(monto_usd) AS total_ventas
FROM ventas_analitica.pedidos
GROUP BY GROUPING SETS (
    (pais_iso, categoria_id), -- Subtotal por país y categoría
    (pais_iso),               -- Subtotal nacional
    ()                        -- Gran total global
);
```

2. `ROLLUP` (Jerarquía Piramidal)

Calcula agregaciones en forma jerárquica de izquierda a derecha. Si agrupamos por `(anio, mes, dia)`, `ROLLUP` produce:

- `(anio, mes, dia)`
- `(anio, mes)`
- `(anio)`
- `()` (Total general)

```sql
SELECT anio, mes, SUM(monto_usd) AS total_ventas
FROM ventas_analitica.fact_ventas_diarias
GROUP BY ROLLUP (anio, mes);
```

3. `CUBE` (Todas las Combinaciones Posibles)

Calcula todas las combinaciones 2^N posibles de las N dimensiones especificadas. Para `CUBE(pais, canal)` genera: `(pais, canal)`, `(pais)`, `(canal)` y `()`.

2.3 Rotación Tabular: `PIVOT` y `UNPIVOT`

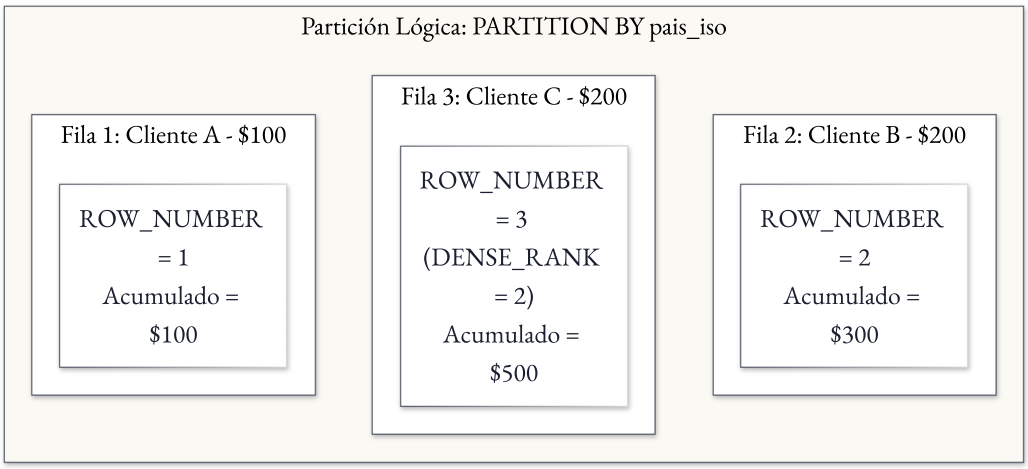
- `PIVOT`: Rota valores de una columna en filas para convertirlos en múltiples columnas (útil para generar matrices analíticas comparativas).
- `UNPIVOT`: Convierte columnas en filas (normalización de datos).

```sql
-- Rotación PIVOT en SQL (DuckDB / Databricks / Snowflake)
SELECT *
FROM (
    SELECT anio, trimestre, monto_usd
    FROM ventas_analitica.ventas_trimestrales
)
PIVOT (
    SUM(monto_usd)
    FOR trimestre IN ('Q1' AS ventas_q1, 'Q2' AS ventas_q2, 'Q3' AS ventas_q3,
'Q4' AS ventas_q4)
)
ORDER BY anio DESC;
```

2.4 Anatomía Completa de las Window Functions

Una función de ventana evalúa una expresión sobre una partición lógica sin comprimir las filas:

FUNCIÓN(expr) **OVER** (**PARTITION BY** claves **ORDER BY** orden



1. Funciones de Ranking

Comparativa entre las principales funciones de jerarquía sobre un conjunto con empates en el salario:

Salario	`ROW_NUMBER()`	`RANK()`	`DENSE_RANK()`	`NTILE(2)`
$10.000	1	1	1	1
$8.000	2	2	2	1
$8.000	3	2 *(empate)*	2 *(empate)*	2
$6.000	4	4 *(salta el 3)*	3 *(sin salto)*	2

- `ROW_NUMBER()`: Asigna un entero secuencial estrictamente único ($1, 2, 3, 4$). Si hay empates, el orden de desempate es arbitrario salvo que se agregue una clave secundaria en el `ORDER BY`.
- `RANK()`: Asigna el mismo rango ante valores idénticos, pero **salta** posiciones en el siguiente valor ($1, 2, 2, 4$).
- `DENSE_RANK()`: Asigna el mismo rango ante empates y **continúa de forma contigua sin huecos** ($1, 2, 2, 3$).
- `NTILE(n)`: Divide la partición en n cubetas casi iguales y asigna el número de cubeta (1 a n), ideal para calcular cuartiles o deciles.

2. Funciones de Desplazamiento y Acceso a Fila (`LAG`, `LEAD`)

Permiten comparar una fila con su predecesora o sucesora sin auto-uniones:

```sql
SELECT
    cliente_id,
    fecha_pedido,
    monto_usd,
    -- Valor de la compra inmediatamente anterior
    LAG(monto_usd, 1, 0.0) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS monto_compra_previa,
    -- Días transcurridos desde la compra anterior
    CAST(fecha_pedido AS DATE) - LAG(CAST(fecha_pedido AS DATE), 1) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS dias_desde_ultima_compra,
    -- Valor de la siguiente compra
    LEAD(monto_usd, 1) OVER (
        PARTITION BY cliente_id
        ORDER BY fecha_pedido ASC
    ) AS monto_compra_siguiente
FROM ventas_analitica.pedidos;
```

3. Marcos de Ventana Físicos (`ROWS`) frente a Lógicos (`RANGE`)

Al invocar agregaciones en ventana (`SUM()`, `AVG()`), el **marco de ventana** delimita qué tuplas entran en el cálculo:

- **Acumulado Histórico Completo:**

```sql
SUM(monto_usd) OVER (
    PARTITION BY cliente_id
    ORDER BY fecha_pedido ASC
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS total_acumulado_hasta_la_fecha
```

- **Media Móvil de los Últimos 7 Días / 7 Eventos:**

```sql
AVG(monto_usd) OVER (
    PARTITION BY tienda_id
    ORDER BY fecha_venta ASC
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS media_movil_7_eventos
```

> **CUIDADO CRÍTICO: La Trampa de `RANGE BETWEEN` por Defecto** Cuando se incluye una cláusula `ORDER BY` sin especificar explícitamente el marco de filas, la norma SQL aplica por defecto: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` `RANGE` evalúa duplicados de orden de forma conjunta: si existen tres transacciones con la misma marca de tiempo idéntica, el acumulado sumará las tres transacciones de golpe en las tres filas, en lugar de acumular fila a fila. **En producción especifique siempre `ROWS BETWEEN` para garantizar determinismo y menor sobrecosto de cálculo**.

2.5 Deduplicación Analítica: `ROW_NUMBER` frente a `dropDuplicates`

Uno de los requerimientos más recurrentes en pipelines Lakehouse es conservar únicamente la versión más reciente de un registro ante múltiples actualizaciones emitidas por sistemas CDC (*Change Data Capture*):

```sql
-- Patrón canónico de deduplicación analítica conservando el registro más reciente
WITH novedades_numeradas AS (
    SELECT
        cliente_id,
        nombre,
        correo,
        direccion,
        fecha_actualizacion,
        ROW_NUMBER() OVER (
            PARTITION BY cliente_id
            ORDER BY fecha_actualizacion DESC, transaccion_id DESC
        ) AS indice_version
    FROM staging.clientes_novedades
)
SELECT
    cliente_id,
    nombre,
    correo,
    direccion,
    fecha_actualizacion
FROM novedades_numeradas
WHERE indice_version = 1;
```

- **Comparativa:** `DISTINCT` o `df.dropDuplicates(["cliente_id"])` descartan registros arbitrariamente si las filas difieren en otras columnas. `ROW_NUMBER()` con orden descendente por marca de tiempo garantiza de manera formal y determinista que solo se conserve el registro de mayor vigencia temporal.

2.6 Implementación en PySpark y Optimización Física con `EXPLAIN`

En PySpark, la API de ventanas se implementa mediante la clase `Window` del módulo `pyspark.sql.window`:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("Capitulo02-WindowFunctions") \
    .master("local[*]") \
    .getOrCreate()

df_pedidos = spark.read.parquet("data/ventas.parquet")

# 1. Definición de especificación de ventana
ventana_cliente = Window \
    .partitionBy("cliente_id") \
    .orderBy(F.col("fecha_transaccion").asc()) \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

# 2. Aplicación de agregación acumulada y ranking
df_analitico = df_pedidos.withColumn(
    "gasto_acumulado",
    F.sum("monto_usd").over(ventana_cliente)
).withColumn(
    "numero_compra",
    F.row_number().over(

Window.partitionBy("cliente_id").orderBy(F.col("fecha_transaccion").asc())
    )
)

# 3. Inspección del plan físico con EXPLAIN
df_analitico.explain(True)
```

Interpretación del Plan Físico

Al ejecutar `explain()`, observe cómo Catalyst organiza los operadores físicos:

1. `Exchange hashpartitioning(cliente_id)`: Realiza el shuffle de red para enviar todas las filas de un mismo `cliente_id` al mismo ejecutor.
2. `Sort [cliente_id ASC, fecha_transaccion ASC]`: Ordena las particiones locales en memoria.
3. `Window [sum(...) ... row_number(...)]`: El operador físico de ventana procesa secuencialmente el flujo de datos ordenado en memoria sin necesidad de reconstruir

estructuras hash adicionales.

2.7 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ventana Global sin `PARTITION BY` sobre Grandes Volúmenes
Declarar `Window.orderBy("monto_usd")` omitiendo el `partitionBy()` fuerza a Spark a enviar **la totalidad de las filas del dataset a un único nodo ejecutor**. Esto satura la memoria del ejecutor, desencadena fallos catastróficos por *OutOfMemory* (OOM) y destruye el paralelismo del clúster.

Anti-patrón 2: Confundir `COUNT(*)` con `COUNT(columna)` ante Valores `NULL`
`COUNT(*)` cuenta el número absoluto de filas de un grupo o ventana, independientemente de su contenido. `COUNT(columna)` cuenta únicamente las tuplas donde dicha columna no es `NULL`. En métricas financieras o de calidad, este descuido altera los promedios e indicadores.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 2.1 (Detección de Transacciones Sospechosas por Desviación de Media Móvil):
En una tabla de eventos financieros `transacciones(id, cuenta_id, monto, timestamp)`, escriba una consulta SQL que calcule:

1. La media móvil del monto de las últimas 5 transacciones de la cuenta (las 4 anteriores y la actual).
2. La diferencia entre el monto actual y dicha media móvil.

Solución 2.1:

```sql
SELECT
    id,
    cuenta_id,
    monto,
    timestamp,
    AVG(monto) OVER (
        PARTITION BY cuenta_id
        ORDER BY timestamp ASC
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS media_movil_ultimas_5,
    monto - AVG(monto) OVER (
        PARTITION BY cuenta_id
        ORDER BY timestamp ASC
        ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
    ) AS desviacion_respecto_media
FROM transacciones;
```

Ejercicio 2.2 (Cálculo de Tasa de Retención con `LAG`):
Se tiene un histórico de suscripciones `pagos(pago_id, usuario_id, fecha_pago, monto)`. Escriba una consulta analítica que etiquete como `'REACTIVADO'` a cualquier usuario cuya compra actual se haya realizado más de 60 días después de su compra previa.

Solución 2.2:

```sql
WITH compras_previas AS (
    SELECT
        pago_id,
        usuario_id,
        fecha_pago,
        LAG(fecha_pago) OVER (
            PARTITION BY usuario_id
            ORDER BY fecha_pago ASC
        ) AS fecha_pago_anterior
    FROM pagos
)
SELECT
    pago_id,
    usuario_id,
    fecha_pago,
    fecha_pago_anterior,
    CASE
        WHEN fecha_pago_anterior IS NULL THEN 'CLIENTE NUEVO'
        WHEN CAST(fecha_pago AS DATE) - CAST(fecha_pago_anterior AS DATE) > 60
THEN 'REACTIVADO'
        ELSE 'RECURRENTE'
    END AS estado_ciclo_vida
FROM compras_previas;
```

2.8 Resumen y Puntos Clave

- **Preservación de tuplas:** Las funciones de ventana computan métricas agregadas y analíticas preservando intacta la granularidad individual de cada fila.
- **Marcos deterministas:** Utilice siempre `ROWS BETWEEN` en lugar del valor implícito `RANGE BETWEEN` para evitar comportamientos anómalos ante marcas de tiempo duplicadas.
- **Jerarquías con `DENSE_RANK` y `ROW_NUMBER`:** `ROW_NUMBER` es la herramienta matemática por excelencia para deduplicación por registro más reciente, mientras que `DENSE_RANK` evita saltos en listas de mejores puntuaciones.
- **Particionado responsable:** Toda ventana en entornos distribuidos debe incluir una clave lógica de `PARTITION BY` para evitar la serialización de todo el dataset en un único ejecutor.
- **Optimización con `EXPLAIN`:** Verificar los operadores físicos de intercambio y ordenamiento (*Exchange* y *Sort*) permite diagnosticar si el plan de consulta incurre en shuffles innecesarios.

Lecturas Recomendadas

1. **Date, C. J. (2019).** *SQL and Relational Theory: How to Write Accurate SQL Code (Chapter 8: Window Functions and Relational Algebra).* O'Reilly Media.
2. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* ACM SIGMOD.
3. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Window Function Processing and Syntax.*

CAPÍTULO 2A

DuckDB y Cómputo In-Process: Cuándo No Usar Computación Distribuida NÚCLEO

> *"Uno de los errores de diseño más costosos en la ingeniería de datos contemporánea es utilizar un cañón distribuido para cazar una mosca local. Durante una década, la industria convenció a los equipos de que cualquier análisis requería un clúster de Spark de ocho nodos. En la práctica, más del 90% de los conjuntos de datos analíticos caben en la memoria RAM de una sola máquina moderna. DuckDB demostró que el cómputo embebido, columnar y vectorizado en un solo nodo puede procesar decenas de gigabytes en segundos a una fracción del costo y complejidad de un clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la fundamentación arquitectónica de los motores analíticos en proceso (**In-Process OLAP**) frente a sistemas distribuidos masivos.
- Analizar los costos ocultos de la computación distribuida (serialización de red, sobrecosto de coordinación del Driver y latencia de shuffles).
- Ejecutar consultas SQL analíticas de alto rendimiento directamente sobre archivos **Parquet**, CSV y JSON sin cargarlos previamente en tablas físicas.
- Conectar DuckDB con almacenamiento de objetos en la nube (Amazon S3 / Azure ADLS) mediante la extensión `httpfs` con transferencias *Zero-Copy* hacia Apache Arrow y Pandas.
- Implementar canalizaciones analíticas Serverless de bajo costo utilizando **AWS Lambda** y DuckDB.
- Aplicar una matriz de decisión rigurosa para seleccionar entre **Pandas**, **Polars**, **DuckDB** y **Apache Spark**.

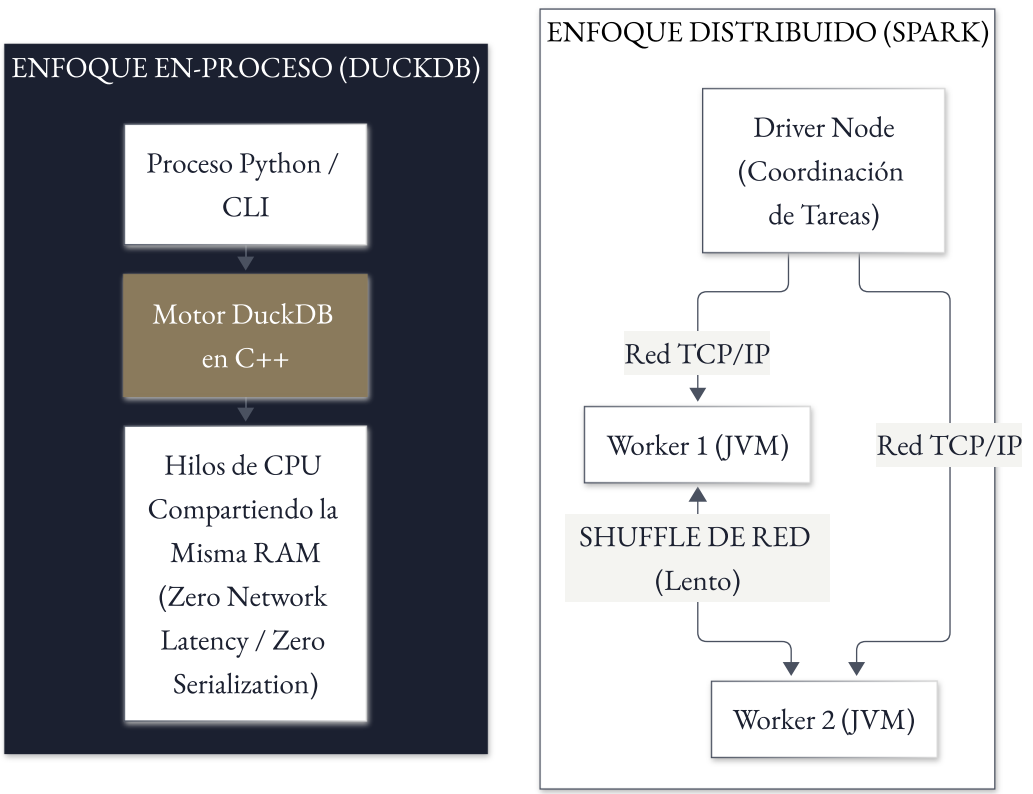
Definición 2A.1 (Base de Datos Analítica En-Proceso - In-Process OLAP) DuckDB es un sistema de gestión de bases de datos relacionales enfocado en procesamiento analítico (OLAP), diseñado para ejecutarse **embebido dentro del mismo espacio de memoria del proceso anfitrión** (*In-Process*), análogo a lo que SQLite representa para cargas transaccionales (OLTP).

No requiere la instalación de un servidor independiente, carece de dependencias externas complejas y procesa datos mediante un **motor columnar vectorizado en C++** con paralelismo multihilo (*Morsel-Driven Parallelism*) y capacidad de procesamiento fuera de memoria (*Out-of-Core Execution*).

2A.1 El Mito del Big Data y el Costo Oculto de los Clústeres

A principios de la era de Hadoop y Spark, los servidores estándar disponían de 4 a 8 GB de RAM. Procesar 50 GB de datos forzaba obligatoriamente a distribuir la carga entre diez máquinas.

Hoy, un computador portátil de desarrollo cuenta con 16 a 64 GB de memoria RAM, y una sola instancia en la nube (como una máquina `r6i.4xlarge` en AWS) ofrece **128 GB de RAM y 16 núcleos de CPU por menos de $1 USD por hora**.



El Impuesto de la Distribución (*The Distribution Tax*)

Cuando una consulta se ejecuta sobre un clúster distribuido, el motor debe asumir sobrecostos físicos inevitables:

1. **Serialización y Deserialización:** Convertir objetos de memoria a secuencias de bytes para transmitirlos por la red.
2. **Latencia de Red:** Transferir gigabytes de particiones intermedias entre switches de red durante un *Shuffle*.
3. **Sobrecosto de Coordinación:** El Driver debe compilar el DAG, programar miles de tareas en hilos remotos y gestionar fallos de comunicación.
4. **Tiempo de Arranque (*Cold Start*):** Un clúster de Spark o EMR tarda entre 3 y 8 minutos en aprovisionar sus máquinas virtuales antes de procesar el primer byte.

Si el volumen de datos a procesar cabe en la memoria de un solo nodo (menos de 100–200 GB), **un motor en proceso como DuckDB suele ser entre 3 y 10 veces más rápido que un clúster de Spark**, costando hasta un 95% menos en infraestructura cloud.

2A.2 Arquitectura Interna de DuckDB

DuckDB alcanza su rendimiento excepcional mediante tres pilares de ingeniería de sistemas:

1. Motor Columnar Vectorizado

A diferencia de los motores tradicionales que procesan una tupla a la vez mediante el modelo de iteradores de Volcán (*Volcano Iterator Model*), DuckDB procesa **vectores de datos contiguos en memoria** (típicamente fragmentos de 2.048 valores). Esto maximiza la utilización de la caché L1/L2 del procesador y permite al compilador emitir instrucciones SIMD (*Single Instruction, Multiple Data*).

2. Paralelismo Guiado por Porciones (*Morsel-Driven Parallelism*)

DuckDB divide el flujo de datos en pequeñas porciones (*morsels*) de aproximadamente 100.000 tuplas. Los hilos de trabajo de la CPU reclaman dinámicamente estas porciones de una cola compartida sin bloqueos pesados (*lock-free queues*), garantizando una distribución de carga homogénea entre todos los núcleos disponibles.

3. Procesamiento Fuera de Núcleo (*Out-of-Core Execution*)

Si una consulta analítica requiere procesar un archivo Parquet de 80 GB en una máquina con solo 16 GB de RAM física, muchos motores colapsan con excepciones de *OutOfMemory*. DuckDB cuenta con un gestor de buffers propio que vuelca bloques intermedios a disco temporalmente

(*Disk Spilling Streaming*), permitiendo resolver consultas analíticas masivas que superan la memoria disponible sin interrumpir la ejecución.

2A.3 Consultas Directas sobre Parquet, CSV y S3

Una de las facultades más poderosas de DuckDB es la capacidad de consultar archivos crudos sin necesidad de crear tablas previas ni importar datos:

```SQL
-- Consultar directamente archivos Parquet locales con SQL analítico estándar
SELECT
    pais,
    categoria_producto,
    COUNT(transaccion_id) AS total_ventas,
    ROUND(SUM(monto_usd), 2) AS facturacion_total,
    ROUND(AVG(monto_usd), 2) AS ticket_promedio
FROM 'data/ventas.parquet'
WHERE estado_pago = 'COMPLETADO'
  AND fecha_transaccion >= '2026-01-01'
GROUP BY pais, categoria_producto
ORDER BY facturacion_total DESC
LIMIT 5;
```

Integración en Python y Transferencia Zero-Copy hacia Arrow

DuckDB se integra de forma transparente con el ecosistema de Python. Puede leer DataFrames de Pandas o tablas de Apache Arrow y devolver resultados sin copiar datos en memoria (*Zero-Copy Memory Sharing*):

```python
import duckdb
import pyarrow.parquet as pq

# Conexión en memoria volátil (o persistente indicando una ruta de archivo)
conexion = duckdb.connect(database=":memory:")

# 1. Consulta SQL directa sobre múltiples archivos Parquet con patrones glob
consulta_sql = """
    SELECT
        DATE_TRUNC('month', CAST(fecha_transaccion AS DATE)) AS mes,
        COUNT(DISTINCT cliente_id) AS clientes_unicos,
        SUM(monto_usd) AS volumen_total
    FROM 'data/ventas.parquet'
    GROUP BY 1
    ORDER BY 1 ASC
"""

# 2. Retorno directo como DataFrame de Pandas o Tabla Arrow
df_resultado_arrow = conexion.execute(consulta_sql).arrow()
df_resultado_pandas = conexion.execute(consulta_sql).df()

print(df_resultado_pandas)
```

Consultas Federadas sobre Amazon S3 / Azure ADLS

Mediante la extensión oficial `httpfs`, DuckDB lee archivos remotos en almacenamiento de objetos en la nube, descargando únicamente los metadatos y las columnas requeridas (*Projection and Predicate Pushdown* sobre HTTP):

PYTHON

```python
import duckdb

con = duckdb.connect()

# Habilitar extensión HTTP/S3
con.execute("INSTALL httpfs; LOAD httpfs;")

# Configurar credenciales cloud seguras mediante el gestor de secretos de DuckDB
# (Lee automáticamente variables de entorno AWS_ACCESS_KEY_ID o roles IAM de la instancia)
con.execute("""
    CREATE SECRET secreto_s3 (
        TYPE S3,
        PROVIDER CREDENTIAL_CHAIN,
        REGION 'us-east-1'
    );
""")

# Consulta remota que descarga únicamente los bytes necesarios de S3
df_s3 = con.execute("""
    SELECT categoria, SUM(monto) AS total
    FROM 's3://mi-bucket-lakehouse/silver/pedidos/*.parquet'
    WHERE anio = 2026
    GROUP BY categoria
""").df()
```

2A.4 Matriz de Decisión: Pandas vs. Polars vs. DuckDB vs. Apache Spark

Para elegir la herramienta adecuada en un pipeline de datos, utilice la siguiente matriz técnica comparativa:

Criterio	Pandas	Polars	DuckDB	Apache Spark
Paradigma Primario	DataFrames en memoria (Python).	DataFrames vectorizados en Rust.	Base de datos relacional SQL en C++.	Cómputo distribuido horizontal en clúster.
Interfaz de Consulta	API imperativa/métodos de Python.	API funcional de expresiones encadenadas.	**SQL Estándar ANSI** de primera clase.	PySpark DataFrame API y Spark SQL.
Escalabilidad Máxima	~5–10 GB (limitado por RAM del nodo).	~100–300 GB (Single-Node multihilo).	**~100–500 GB (Single-Node con Out-of-Core)**.	**Petabytes (Multi-Node horizontal)**.
Manejo de Memoria	Alto consumo (objetos Python, copias).	Excelente (Apache Arrow nativo en Rust).	Excelente (gestor de buffers con volcado a disco).	Tungsten on-heap/off-heap con JVM.
Tiempo de Arranque	< 1 segundo.	< 1 segundo.	**< 10 milisegundos**.	3 a 7 minutos en clúster cloud.
Caso de Uso Óptimo	Análisis exploratorio rápido y ciencia de datos básica.	Transformaciones de alta velocidad en una máquina.	**Pipelines analíticos SQL, serverless, pruebas CI/CD.**	**Cargas masivas que exceden un solo servidor físico.**

2A.5 Pipelines Serverless de Bajo Costo con AWS Lambda y DuckDB

Un patrón de arquitectura moderno consiste en ejecutar pipelines analíticos ligeros sobre **AWS Lambda** utilizando la capa precompilada de DuckDB.

En lugar de mantener un clúster de EMR o Databricks encendido 24/7 (con costos mínimos de cientos de dólares al mes), una función Lambda con 2 GB de memoria ejecuta transformaciones de 5 GB de datos en 8 segundos por **menos de $0.0001 USD por ejecución**:

```python
# Handler de AWS Lambda para transformación de lotes diarios
import os
import duckdb

def lambda_handler(event, context):
    bucket = event.get("bucket", "empresa-datos-crudos")
    fecha_proceso = event.get("fecha", "2026-03-31")

    con = duckdb.connect(database=":memory:")
    con.execute("INSTALL httpfs; LOAD httpfs;")

    # Ingesta, filtrado, deduplicación y exportación directa de Parquet a Parquet
    query = f"""
        COPY (
            SELECT
                transaccion_id,
                cliente_id,
                monto_usd,
                fecha_transaccion
            FROM 's3://{bucket}/raw/{fecha_proceso}/*.parquet'
            WHERE monto_usd > 0
            QUALIFY ROW_NUMBER() OVER (PARTITION BY transaccion_id ORDER BY
fecha_transaccion DESC) = 1
        ) TO 's3://{bucket}/silver/transacciones_limpias_{fecha_proceso}.parquet'
        (FORMAT PARQUET, COMPRESSION 'ZSTD');
    """

    con.execute(query)
    return {"status": "SUCCESS", "fecha": fecha_proceso}
```

2A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Forzar el Uso de Apache Spark para Datasets Menores a 50 Gigabytes
Crear un clúster de Spark para procesar una exportación diaria de 5 GB incurre en minutos de aprovisionamiento de máquinas, facturación mínima de nodos y sobrecosto de shuffling.
Corrección: Utilice DuckDB o Polars en un contenedor ligero de Docker o una máquina virtual pequeña; el pipeline tardará segundos en lugar de minutos y costará centavos.

Anti-patrón 2: Tratar a DuckDB como una Base de Datos Transaccional OLTP Concurrente
Intentar utilizar DuckDB como el backend de una aplicación web donde cientos de usuarios realizan transacciones concurrentes simultáneas (`INSERT` y `UPDATE` de milisegundos) provocará bloqueos de base de datos.
Corrección: DuckDB está optimizado para lecturas analíticas masivas y cargas por lotes (OLAP); para concurrencia transaccional multi-usuario utilice PostgreSQL o MySQL.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 2A.1 (Agregación Multi-Archivo y Detección de Anomalías con DuckDB):
Se tienen cientos de archivos mensuales particionados en disco: `data/particiones/ventas_*.parquet`.
Escriba una consulta SQL pura ejecutable en DuckDB que lea todos los archivos Parquet en una sola pasada y obtenga:

1. El total de transacciones por categoría.
2. La facturación acumulada.
3. El porcentaje que representa cada categoría sobre el gran total nacional utilizando una función de ventana `SUM() OVER ()`.

Solución 2A.1:

```SQL
WITH metricas_categoria AS (
    SELECT
        categoria_producto,
        COUNT(transaccion_id) AS total_transacciones,
        SUM(monto_usd) AS facturacion_categoria
    FROM 'data/particiones/ventas_*.parquet'
    WHERE estado_pago = 'COMPLETADO'
    GROUP BY categoria_producto
)
SELECT
    categoria_producto,
    total_transacciones,
    ROUND(facturacion_categoria, 2) AS facturacion_categoria,
    ROUND(
        (facturacion_categoria / SUM(facturacion_categoria) OVER ()) * 100,
        2
    ) AS porcentaje_facturacion_nacional
FROM metricas_categoria
ORDER BY facturacion_categoria DESC;
```

2A.7 Resumen y Puntos Clave

- **Desmitificación de la escala:** La mayor parte de las cargas analíticas caben en la RAM de un solo servidor; el cómputo distribuido impone penalizaciones de red y coordinación que deben justificarse formalmente.
- **DuckDB como motor OLAP en proceso:** Ofrece ejecución columnar vectorizada en C++, paralelismo multihilo y capacidad fuera de núcleo (*Out-of-Core*) sin requerir servidores ni clústeres.

- **Acceso directo a formatos abiertos:** Permite ejecutar SQL analítico sobre archivos Parquet locales o en S3/ADLS con poda de columnas y predicados sin duplicación de datos.
- **Complementariedad con Spark:** DuckDB optimiza pipelines locales, serverless (Lambda) y pruebas en CI/CD; Apache Spark se reserva para volúmenes masivos que exceden la capacidad física de una máquina individual.

Lecturas Recomendadas

1. **Raasveldt, M. & Mühleisen, H. (2019).** *DuckDB: An Embeddable Analytical Database.* Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 10: Batch Processing).* O'Reilly Media.
3. **DuckDB Foundation (2024).** *DuckDB Documentation: Performance Tuning, Extensions, and Python API.* DuckDB.org.

CAPÍTULO 2B

Algoritmos y Estructuras de Datos Esenciales para Data Engineers RECOMENDADO

> *"Un técnico memoriza la sintaxis de las herramientas de moda; un ingeniero de datos comprende los algoritmos y estructuras de datos fundamentales que gobiernan el comportamiento de la memoria, el disco y la red. Comprender la complejidad asintótica Big-O, los filtros probabilísticos de Bloom, el ordenamiento topológico en grafos DAG y la arquitectura de los árboles LSM-Tree es lo que permite diagnosticar por qué un pipeline colapsa y diseñar sistemas capaces de procesar petabytes con latencia predecible."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Analizar la complejidad temporal y espacial mediante la **notación asintótica Big-O**, prediciendo el comportamiento de transformaciones masivas de datos.
- Explicar el funcionamiento matemático de las **estructuras de datos probabilísticas**: Filtros de Bloom (*Bloom Filters*), HyperLogLog y Count-Min Sketch.
- Implementar y comprender el **Algoritmo de Ordenamiento Topológico** (*Kahn's Algorithm*) que subyace a planificadores de DAGs en Apache Airflow, dbt y Spark Catalyst.
- Contrastar la arquitectura de actualización en el sitio de los **árboles B-Tree** frente a la estructura secuencial de solo adición de los **árboles LSM-Tree** (*Log-Structured Merge-Trees*).
- Diseñar pipelines analíticos evitando trampas algorítmicas de complejidad cuadrática $O(N^2)$ en uniones y agregaciones.

Definición 2B.1 (Complejidad Asintótica y Escalabilidad) La **notación Big-O** describe el límite superior del consumo de recursos (tiempo de procesamiento de CPU o espacio en memoria RAM/disco) a medida que el tamaño de entrada N tiende a infinito ($N \to \infty$).

En la ingeniería de datos, un algoritmo con complejidad $O(N)$ o $O(N \log N)$ escala linealmente con el crecimiento del negocio; un algoritmo con complejidad $O(N^2)$ (como un producto cartesiano inadvertido o una subconsulta correlacionada mal optimizada) se vuelve computacionalmente inviable en cuanto el dataset supera unos pocos miles de registros.

2B.1 Notación Asintótica Big-O en Sistemas de Datos

En el análisis de pipelines, clasificar el costo de cada operación previene cuellos de botella catastróficos:



Complejidad	Ejemplo en Ingeniería de Datos	Comportamiento con 10^6 registros
$O(1)$ — **Constante**	Búsqueda por clave primaria en Redis o diccionario en memoria.	1 operación (~0.1 ms).
$O(\log N)$ — **Logarítmico**	Búsqueda por clave indexada en un árbol B-Tree de PostgreSQL.	≈ 20 comparaciones de nodos en disco.
$O(N)$ — **Lineal**	Escaneo secuencial (*Seq Scan*), filtrado con `WHERE` en Parquet.	$1.000.000$ de filas procesadas (segundos).
$O(N \log N)$ — **Cuasilineal**	Shuffling, ordenamiento distribuido con `ORDER BY`, *Sort-Merge Join*.	$\approx 20.000.000$ de operaciones (minutos).
$O(N^2)$ — **Cuadrático**	`CROSS JOIN` accidental entre tablas o auto-uniones sin filtro.	10^{12} **operaciones (colapso de clúster por OOM).**

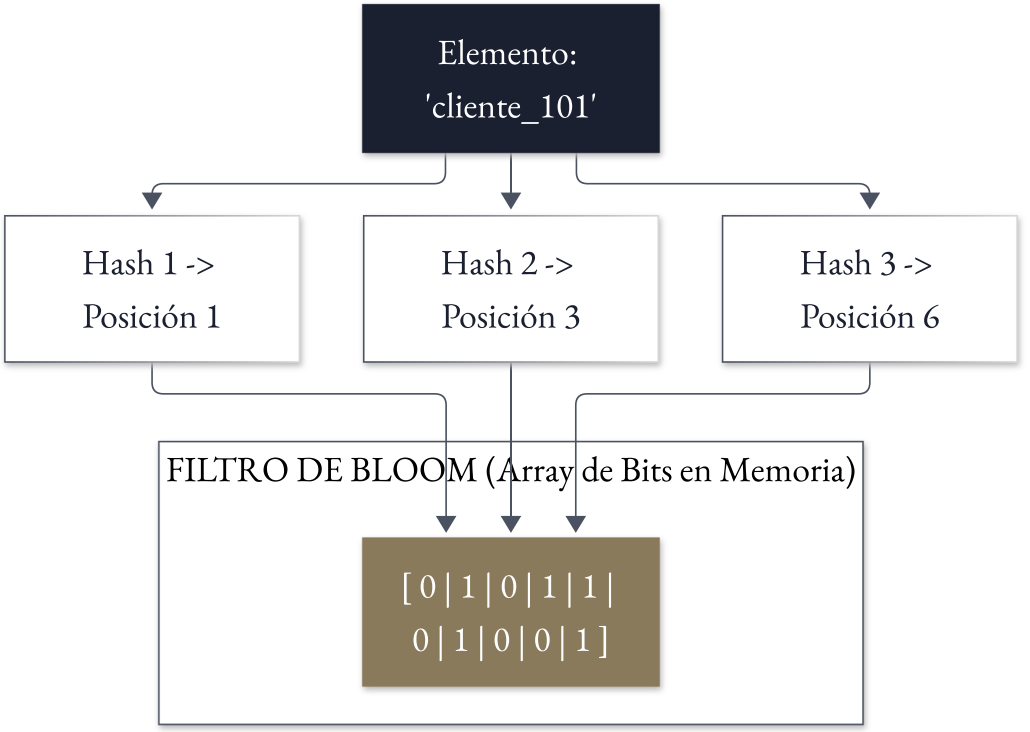
2B.2 Estructuras de Datos Probabilísticas

Cuando el volumen de datos alcanza los terabytes, las estructuras de datos deterministas tradicionales consumen demasiada memoria RAM. Los sistemas modernos adoptan algoritmos probabilísticos que sacrifican una precisión despreciable a cambio de reducciones masivas de memoria:

1. Filtros de Bloom (Bloom Filters)

Un Filtro de Bloom es un vector de m bits inicializado en ceros, acompañado de k funciones hash independientes:

- **Inserción de un elemento:** Se calculan las k funciones hash sobre el elemento y se activan los bits correspondientes fijándolos en 1.
- **Consulta de pertenencia:** Se calculan las k funciones hash sobre el elemento buscado y se inspeccionan los bits resultantes.



> **PROPIEDAD MATEMÁTICA FUNDAMENTAL DE LOS FILTROS DE BLOOM:**
>
> - Si el filtro responde **NO**: Con certeza absoluta matemática el elemento **NO existe** en el conjunto (*Cero falsos negativos*).
> - Si el filtro responde **SÍ**: El elemento **probablemente existe**, pero existe una pequeña probabilidad medible de un **falso positivo** debido a colisiones en los bits.

Aplicación en Spark y Parquet:

Apache Parquet y Apache Spark utilizan filtros de Bloom integrados en los footers de los archivos. Al ejecutar un join o un filtro `WHERE id = 'XYZ'`, el ejecutor evalúa el filtro de Bloom en memoria: si devuelve "NO", **Spark descarta el archivo Parquet entero sin leer un solo bloque de datos del disco o S3** (*Bloom Filter Data Skipping*).

2. HyperLogLog (HLL)

Permite estimar el número de elementos únicos (*Count Distinct*) en flujos de datos de miles de millones de eventos utilizando únicamente **12 KB de memoria fija**, aprovechando la distribución estadística de ceros a la izquierda en representaciones hash de los datos.

3. Count-Min Sketch

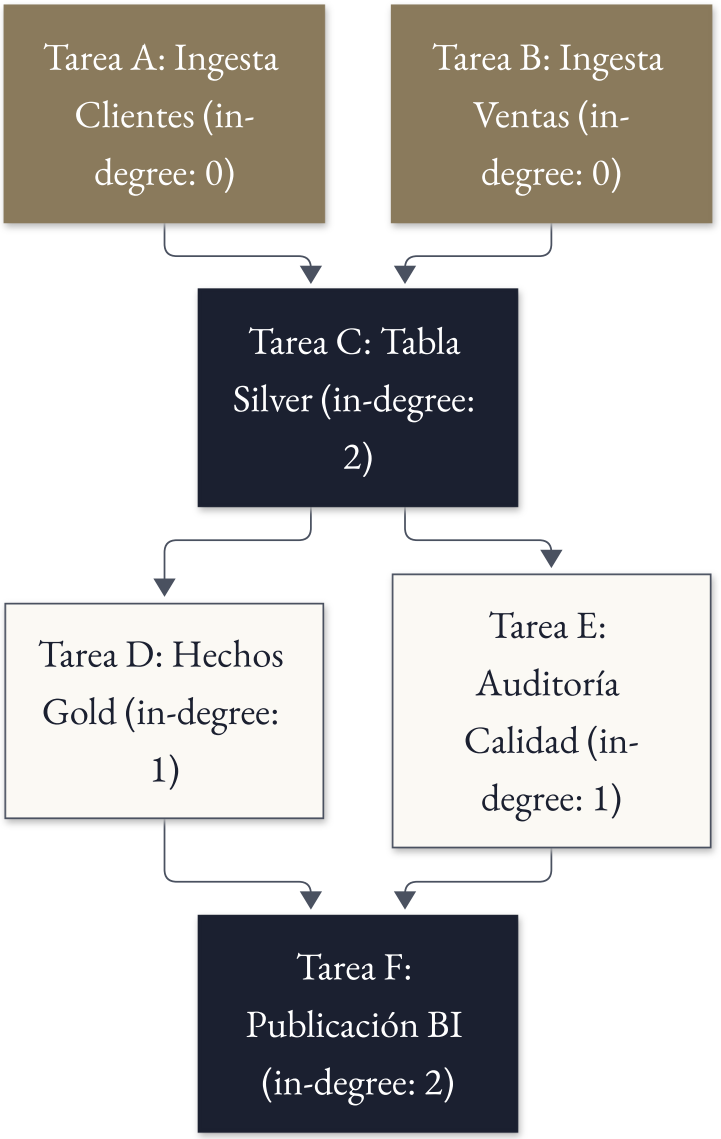
Una matriz de frecuencias sub-lineal que estima el número de apariciones de eventos en tiempo real, fundamental para detectar usuarios fraudulentos (*Heavy Hitters*) o ataques de denegación de servicio en streaming continuo.

2B.3 Grafos Dirigidos Acíclicos (DAG) y Ordenamiento Topológico

En la ingeniería de datos, los flujos de transformación no son secuencias lineales simples: son **Grafos Dirigidos Acíclicos (DAG)**:

- **Dirigido:** Cada dependencia tiene un sentido temporal unívoco (la Tarea A debe ejecutarse antes que la Tarea B).
- **Acíclico:** Es imposible que la Tarea A dependa de B y que B dependa de A (ausencia de bucles infinitos).

Los orquestadores de datos (Apache Airflow, dbt y el compilador Catalyst de Spark) deben resolver una pregunta fundamental: **¿En qué orden exacto deben ejecutarse las tareas para maximizar el paralelismo sin violar dependencias?**



El Algoritmo de Kahn para Ordenamiento Topológico (*Topological Sort*)

El algoritmo resuelve el orden de ejecución en tiempo $O(V + E)$ (donde V es el número de tareas y E el número de dependencias):

PYTHON

```python
from collections import deque, defaultdict
from typing import Dict, List

def resolver_ordenamiento_topologico(grafo_dependencias: Dict[str, List[str]]) ->
List[str]:
    # 1. Calcular el grado de entrada (in-degree) de cada nodo
```

```
    grados_entrada = {nodo: 0 for nodo in grafo_dependencias}
    for nodo, vecinos in grafo_dependencias.items():
        for vecino in vecinos:
            grados_entrada[vecino] += 1

    # 2. Cola con nodos que tienen in-degree == 0 (listos para ejecución
inmediata)
    cola_ejecucion = deque([nodo for nodo, grado in grados_entrada.items() if
grado == 0])
    orden_final = []

    while cola_ejecucion:
        nodo_actual = cola_ejecucion.popleft()
        orden_final.append(nodo_actual)

        # Reducir in-degree de sus dependientes
        for dependiente in grafo_dependencias.get(nodo_actual, []):
            grados_entrada[dependiente] -= 1
            if grados_entrada[dependiente] == 0:
                cola_ejecucion.append(dependiente)

    # 3. Detección formal de ciclos
    if len(orden_final) != len(grafo_dependencias):
        raise ValueError("Error de Arquitectura: Se detectó una dependencia
circular cíclica en el pipeline.")

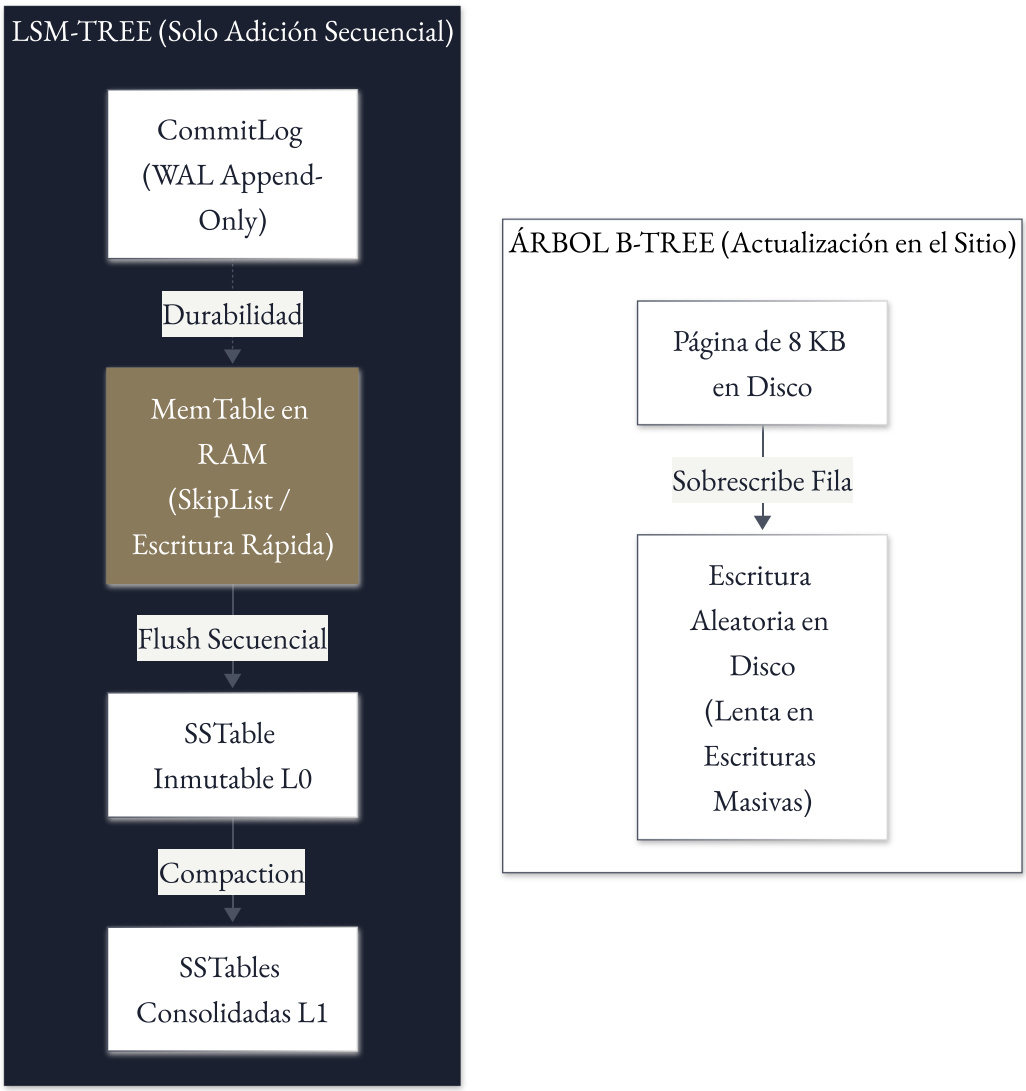
    return orden_final

# Demostración práctica
pipeline_dbt = {
    "stg_clientes": ["int_clientes"],
    "stg_pedidos": ["int_pedidos"],
    "int_clientes": ["fact_ventas"],
    "int_pedidos": ["fact_ventas"],
    "fact_ventas": []
}

secuencia = resolver_ordenamiento_topologico(pipeline_dbt)
print("Orden Topológico de Ejecución:", secuencia)
```

2B.4 Motores de Almacenamiento: B-Trees frente a LSM-Trees

La elección de una base de datos analítica o transaccional depende de cómo su motor interno persiste los datos en disco:



1. Árboles B-Tree (Optimización para Lectura / OLTP)

- Modifican las páginas en el sitio (*In-Place Updates*).
- Sobrescritura aleatoria en disco: excelente para lecturas puntuales por clave en milisegundos, pero pésimo para flujos continuos de escritura masiva debido a fragmentación de páginas (*Page Splits*) y bloqueos.

2. Árboles LSM-Tree (Optimización para Escritura / Big Data)

Motores como **Apache Cassandra**, **RocksDB** (usado por Spark Streaming y Kafka Streams), **Google Bigtable** y el protocolo de logs transaccionales de **Delta Lake** utilizan **Árboles Estructurados por Mezcla de Logs (*Log-Structured Merge-Trees*)**:

1. **MemTable:** Toda nueva escritura se aloja en una estructura ordenada en memoria RAM (típicamente una *SkipList* o *Red-Black Tree*), registrándose en paralelo en un *CommitLog* secuencial para durabilidad.
2. **SSTables (*Sorted String Tables*):** Cuando la MemTable se llena, se vuelca al disco como un archivo inmutable ordenado por clave. **Las escrituras son 100% secuenciales**, alcanzando el máximo throughput físico del almacenamiento.
3. **Compactación en Segundo Plano (*Compaction*):** Un proceso asíncrono toma múltiples SSTables, fusiona las claves repetidas conservando el valor más reciente y elimina registros marcados con borrado lógico (*Tombstones*), reorganizando los archivos en niveles (*Leveled Compaction*).

2B.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Filtrado en Memoria con Colecciones de Búsqueda Lineal $O(N)$
Al realizar deduplicación o exclusión en Python dentro de una función UDF de datos, verificar si un elemento existe en una lista simple (`if x in lista_elementos:`) impone una complejidad de $O(N)$ por cada fila evaluada, elevando el costo de un dataset de 1 millón de registros a un trillón de comprobaciones ($O(N^2)$).
Corrección: Convierta la colección de búsqueda a un conjunto (`set()`) o diccionario (`dict()`) basado en tablas hash con búsqueda en tiempo constante $O(1)$.

Anti-patrón 2: Ignorar Ciclos en Grafos de Dependencia de Tareas
Permitir que dos modelos de dbt o dos tareas de Airflow dependan mutuamente de forma indirecta provoca que el planificador se bloquee indefinidamente en un interbloqueo (*deadlock*).
Corrección: Valide la aciclicidad del grafo durante el pipeline de CI/CD ejecutando el algoritmo de Kahn sobre el manifiesto de dependencias.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 2B.1 (Cálculo de Tasa de Falsos Positivos en Filtros de Bloom):
Un ingeniero de datos desea indexar 1.000.000 de clientes únicos en un Filtro de Bloom para descartar lecturas innecesarias en un clúster de Spark. Dispone de un array de bits de $m = 10.000.000$ bits (aproximadamente 1.2 MB de RAM) y utiliza $k = 7$ funciones hash independientes.
La fórmula matemática aproximada para la tasa de falsos positivos p es:

$$p \approx \left(1 - e^{-k \cdot n / m}\right)^k$$

Calcule la probabilidad de que una consulta reciba un falso positivo.

Solución 2B.1:

1. Razón de bits por elemento:

$$\frac{n}{m} = \frac{1.000.000}{10.000.000} = 0.1$$

2. Evaluación del exponente:

$$-k \cdot \frac{n}{m} = -7 \cdot 0.1 = -0.7$$

3. Cálculo de la potencia exponencial:

$$e^{-0.7} \approx 0.496585$$

4. Sustitución en la fórmula de probabilidad:

$$1 - e^{-0.7} = 1 - 0.496585 = 0.503415$$

$$p \approx (0.503415)^7 \approx 0.00819 \quad (\mathbf{0.82\%})$$

- **Conclusión Técnica:** Con solo **1.2 Megabytes de memoria RAM**, el clúster puede descartar con una precisión de acierto del **99.18%** si un cliente no está en el archivo Parquet, reduciendo drásticamente las lecturas a disco y la saturación de I/O.

2B.6 Resumen y Puntos Clave

- **Complejidad Big-O:** Dicta la viabilidad matemática de los pipelines; las operaciones deben mantenerse dentro de complejidades $O(1)$, $O(N)$ o $O(N \log N)$.
- **Filtros de Bloom:** Ofrecen pruebas de pertenencia ultrarrápidas con cero falsos negativos, permitiendo poda masiva de datos en Parquet y Spark (*Data Skipping*).
- **Ordenamiento Topológico:** El algoritmo de Kahn permite a orquestadores de DAG (Airflow, dbt) resolver el orden óptimo de ejecución paralela libre de dependencias cíclicas.
- **LSM-Trees vs. B-Trees:** Los árboles B-Tree optimizan lecturas puntuales en disco mediante actualizaciones locales; los árboles LSM-Tree maximizan el rendimiento de escritura masiva

mediante registros secuenciales inmutables en disco (SSTables) y compactación.

Lecturas Recomendadas

1. **Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2022).** *Introduction to Algorithms (4th Edition).* The MIT Press.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 3: Storage and Retrieval: Hash Indexes, SSTables and LSM-Trees).* O'Reilly Media.
3. **Bloom, B. H. (1970).** *Space/Time Trade-offs in Hash Coding with Allowable Errors.* Communications of the ACM.

PARTE II

Computación distribuida con Apache Spark

Capítulo 3 · De Hadoop y MapReduce al Cómputo en Memoria NÚCLEO

Capítulo 4 · Arquitectura de Clúster: Driver, Workers y Ejecutores NÚCLEO

Capítulo 5 · Entorno de Configuración y SparkSession NÚCLEO

Capítulo 6 · Abstracciones de Cómputo: De RDDs a DataFrames RECOMENDADO

CAPÍTULO 3

De Hadoop y MapReduce al Cómputo en Memoria NÚCLEO

> *"Antes de Spark existía Hadoop, que introdujo el paradigma de MapReduce para procesar grandes volúmenes de datos en clústeres. El problema fundamental era que procesaba en disco: cada etapa leía y grababa constantemente, lo que hacía todo lento y costoso. Spark revolucionó la industria llevando el cómputo directamente a la memoria RAM."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Analizar la arquitectura fundacional de Apache Hadoop (HDFS, MapReduce, YARN) y sus limitaciones de E/S.
- Entender la transición del procesamiento por lotes centrado en disco hacia la computación distribuida en memoria (*in-memory computation*).
- Describir cómo Apache Spark logra órdenes de magnitud de mayor rendimiento en algoritmos iterativos y analítica interactiva.
- Evaluar cuándo utilizar procesamiento por lotes tradicional y cuándo adoptar motores de memoria de baja latencia.

Definición 3.1 (Escribir en la Pizarra vs. Grabar en Tablas de Piedra) Imagina un equipo de matemáticos resolviendo un problema complejo de 20 ecuaciones sucesivas.

El enfoque de Hadoop MapReduce es como obligar a los matemáticos a tallar el resultado de cada ecuación en una pesada tabla de piedra (disco HDFS), guardarla en la bodega y luego sacarla para la siguiente ecuación. Tallar la piedra toma el 95% del tiempo total.

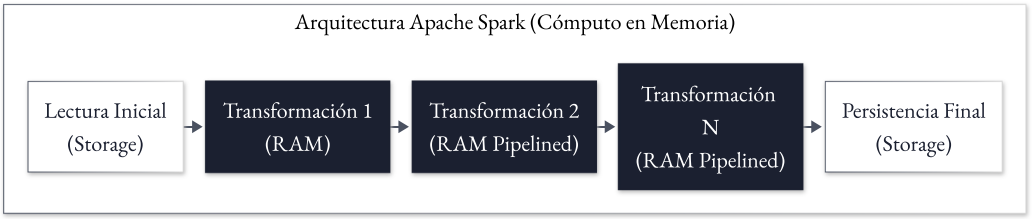
El enfoque de Apache Spark es dotar a cada matemático de una pizarra acrílica (memoria RAM): calculan de inmediato, encadenan las operaciones sin soltar el marcador y solo tallan en piedra el resultado final de la ecuación 20 cuando el trabajo está terminado.

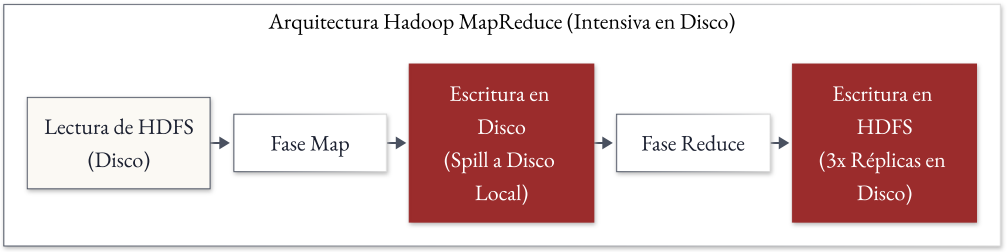
3.1 El Paradigma Fundacional: Hadoop y MapReduce

A principios de la década de 2000, el volumen de datos de la web superó la capacidad de las bases de datos relacionales tradicionales y los sistemas multiprocesador de memoria compartida (*SMP*). En respuesta, Google publicó sus artículos seminales sobre *Google File System* (GFS) y *MapReduce*, sentando las bases de lo que se convertiría en **Apache Hadoop**, desarrollado por Doug Cutting y Mike Cafarella.

Hadoop democratizó el análisis de Big Data permitiendo a las organizaciones procesar petabytes en clústeres de servidores comerciales de bajo costo (*commodity hardware*), apoyándose en tres pilares:

1. **HDFS (Hadoop Distributed File System):** Sistema de archivos distribuido que divide archivos masivos en bloques (típicamente 128 MB) y los replica en múltiples nodos del clúster para tolerancia a fallos.
2. **YARN (Yet Another Resource Negotiator):** Capa de gestión de infraestructura que orquesta CPU y memoria entre aplicaciones concurrentes.
3. **MapReduce:** Modelo de programación funcional dividido en dos primitivas esenciales:
    - **Map:** Transforma y proyecta registros locales sin dependencias externas.
    - **Reduce:** Agrupa y sintetiza los valores intermedios tras una fase obligatoria de shuffle.





3.2 Profundización Técnica: El Cuello de Botella de E/S y la Revolución de Spark

El talón de Aquiles de MapReduce es la **persistencia obligatoria entre fases**. Si un pipeline analítico requería encadenar 5 trabajos consecutivos de MapReduce (por ejemplo en algoritmos de machine learning iterativo o consultas complejas), el sistema debía escribir a HDFS y replicar tres veces por la red tras cada paso intermedio.

En 2009, en el laboratorio AMPLab de la Universidad de California en Berkeley, Matei Zaharia y su equipo diseñaron **Apache Spark** con una premisa revolucionaria: *retener las colecciones de datos en la memoria RAM distribuida entre operaciones sucesivas*.

Al evitar la lectura y escritura constante en disco, Spark alcanzó velocidades hasta **100 veces superiores en memoria** y 10 veces superiores en disco frente a MapReduce, además de introducir el concepto de **DAG** para ejecutar operaciones en cadena sin serializaciones innecesarias.

> **Consideraciones de Producción 3.1: Cómputo Iterativo en Memoria** El verdadero poder de la computación en memoria se evidencia en algoritmos iterativos (como K-Means, Logistic Regression o PageRank), donde los datos se leen una sola vez y se reutilizan en memoria en cada ciclo.

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Capitulo03-Hadoop-vs-Spark") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset masivo una única vez
df_base = spark.read.parquet("data/interacciones.parquet")

# Persistir en la memoria RAM del clúster (In-memory computation)
# Esto replica el salto histórico frente a Hadoop: cero lecturas de disco
posteriores
df_base.cache()

# Algoritmo iterativo: Múltiples pasadas sobre los mismos datos en memoria
for iteracion in range(3):
    umbral = iteracion * 10
    total_segmento = df_base.filter(col("score") > umbral).count()
    print(f"Iteración {iteracion + 1} completada en memoria RAM: {total_segmento}
registros")

# Liberar memoria al finalizar el ciclo
df_base.unpersist()
```

3.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Creer que Hadoop está Muerto

Un error conceptual común es afirmar que "Hadoop ya no existe". Si bien el motor de cómputo *MapReduce* ha sido reemplazado en la gran mayoría de casos por Spark, el sistema de archivos distribuido (HDFS) y el gestor de recursos (YARN) continúan siendo la columna vertebral de miles de clústeres on-premise corporativos.

Anti-patrón 2: Cachear Datos sin Suficiente Capacidad de RAM

Llamar a `df.cache()` indiscriminadamente sobre datasets que superan la memoria disponible del clúster provocará que Spark expulse particiones a disco (*Spill to Disk*) o genere pausas prolongadas del recolector de basura de Java (*Garbage Collection Overhead*).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 3.1 (Comparativa Arquitectónica):

Elabora una tabla comparativa entre Hadoop MapReduce y Apache Spark analizando: (1) Medio de almacenamiento intermedio, (2) Modelo de tolerancia a fallos, y (3) Soporte para procesamiento en streaming en tiempo real.

Solución:

Criterio	Hadoop MapReduce	Apache Spark
Almacenamiento Intermedio	Disco rígido (HDFS con triple réplica)	Memoria RAM distribuida (con spill a disco solo si se llena)
Tolerancia a Fallos	Replicación física de archivos en disco	Linaje lógico del DAG (reconstrucción de particiones perdidas)
Streaming	No soportado de forma nativa (solo batch)	Nativo mediante Spark Structured Streaming

3.4 Resumen y Puntos Clave

- **Hadoop abrió el camino:** Resolvió el almacenamiento masivo con HDFS y demostró que la computación distribuida en hardware comercial era viable.
- **El salto a la memoria:** Apache Spark eliminó el cuello de botella de E/S de disco, habilitando la analítica iterativa, pipelines en cadena y streaming en un único motor unificado.
- **Tolerancia a fallos sin replicación en disco:** A través del linaje de operaciones (DAG), Spark puede reconstruir datos perdidos sin pagar el costo de replicar cada paso intermedio en almacenamiento físico.

Lecturas Recomendadas

1. **White, T. (2015).** *Hadoop: The Definitive Guide (4th Edition).* O'Reilly Media.
2. **Zaharia, M. et al. (2010).** *Spark: Cluster Computing with Working Sets.* HotCloud '10.

CAPÍTULO 4

Arquitectura de Clúster: Driver, Workers y Ejecutores NÚCLEO

> *"Un clúster de Spark no es una máquina gigante, sino una orquesta coordinada: el Driver es el director que interpreta la partitura; el Cluster Manager es el administrador del teatro que asigna las sillas; y los Ejecutores son los músicos que tocan en paralelo en cada nodo trabajador."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la topología física y lógica de un clúster de computación distribuida.
- Identificar las responsabilidades exclusivas del **Driver Program** frente a los **Worker Nodes**.
- Comparar los diferentes administradores de clúster soportados por Spark: Standalone, Apache YARN y Kubernetes.
- Analizar el ciclo de vida y la configuración interna de la JVM de los **Ejecutores** (*Executors*).

Definición 4.1 (La Flota de Transporte y la Torre de Control) Imagina una mega-empresa de mudanzas encargada de trasladar un rascacielos entero:

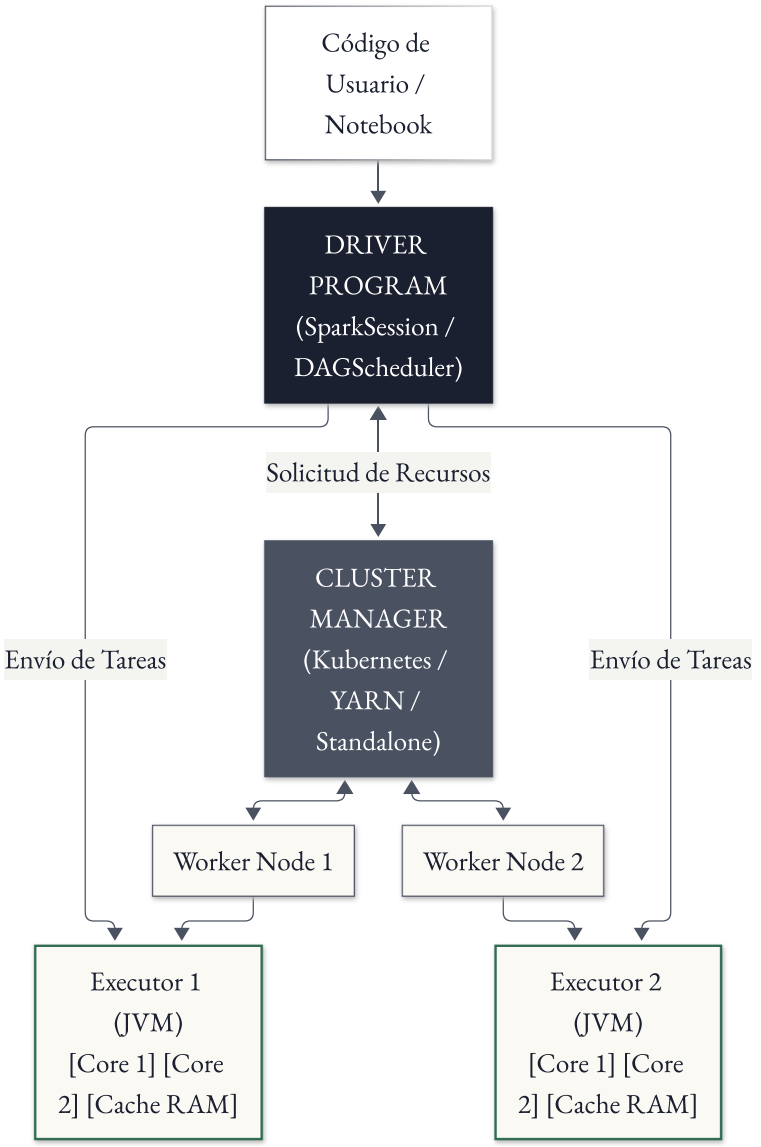
- El **Driver Program** es el jefe de operaciones en la torre de control: revisa el inventario, descompone la mudanza en tareas pequeñas y supervisa el avance general.
- El **Cluster Manager** es la agencia de alquiler de vehículos: el jefe le pide 20 camiones con 4 operarios cada uno, y la agencia los despacha según disponibilidad.
- Los **Worker Nodes** son los estacionamientos físicos donde se encuentran los camiones.
- Los **Ejecutores** (*Executors*) son los camiones con sus operarios: cargan las cajas asignadas (particiones de datos), suben los muebles al ascensor (tareas de cómputo) y reportan a la torre cuando finalizan.

4.1 La Anatomía de un Clúster de Spark

En la ingeniería de datos a escala petabyte, ninguna máquina individual dispone de suficiente memoria RAM o capacidad de CPU para almacenar y procesar la información. El **Cluster Computing** interconecta un conjunto de servidores independientes para que operen como un sistema unificado y tolerante a fallos.

La arquitectura de una aplicación Spark se basa en el patrón maestro/esclavo (*master/worker*):

1. **Driver Program (El Cerebro):**
    - Es el proceso que ejecuta la función `main()` del programa del usuario.
    - Crea y administra la sesión de Spark (`SparkSession` / `SparkContext`).
    - Convierte el código del usuario en un grafo lógico y físico (DAG).
    - Programa y distribuye las tareas individuales (*Tasks*) a través del planificador de tareas (*Task Scheduler*).
    - Recopila los resultados de las acciones de agregación final.
2. **Cluster Manager (El Asignador de Recursos):**
    - Servicio de infraestructura pluggable que adquiere y libera recursos del clúster físico.
    - Spark soporta los siguientes modos principales:
        - **Standalone:** Gestor nativo simple incluido en Spark.
        - **Apache YARN:** Gestor corporativo tradicional en entornos Hadoop.
        - **Kubernetes (K8s):** El estándar moderno de la industria cloud-native basado en contenedores.
        - **Mesos (obsoleto):** su soporte fue declarado obsoleto en Spark 3.2 y eliminado en Spark 4.0; no debe usarse en proyectos nuevos.
3. **Worker Nodes (La Infraestructura Física):**
    - Servidores o instancias de cómputo que ejecutan los procesos esclavos asignados por el Cluster Manager.
4. **Ejecutores (Executors - La Fuerza de Trabajo):**
    - Procesos JVM (*Java Virtual Machine*) que residen dentro de los Worker Nodes.
    - Cada ejecutor tiene asignada una cantidad fija de memoria RAM y núcleos de CPU (*cores*).
    - Ejecutan las tareas asignadas por el Driver y almacenan en memoria/disco los datos cacheados del DataFrame.



4.2 Profundización Técnica: Ciclo de Vida y Asignación de Recursos

Los ejecutores existen **exclusivamente durante el ciclo de vida de la aplicación Spark** que los solicitó:

1. **Aislamiento estricto:** Cada aplicación dispone de sus propios procesos ejecutores. Dos aplicaciones concurrentes en el mismo clúster no comparten espacio de memoria JVM, lo que garantiza seguridad y previene corrupciones cruzadas.

2. **Asignación Dinámica (Dynamic Allocation):** Mediante `spark.dynamicAllocation.enabled = true`, Spark solicita ejecutores adicionales cuando la cola de tareas crece y los devuelve al Cluster Manager cuando el clúster entra en inactividad, optimizando costos en la nube.
3. **Tolerancia a fallos de ejecutores:** Si un Worker Node se apaga o un ejecutor falla por un error del kernel, el Cluster Manager lanza inmediatamente un nuevo ejecutor en otro nodo. El Driver reasigna las tareas pendientes y reconstruye las particiones perdidas a través del linaje del DAG.

> **Consideraciones de Producción 4.1: Inspección de Ejecutores Activos** PySpark **no** expone la lista de ejecutores en `statusTracker()` (ese método existe solo en la API de Scala/Java). La vía soportada para auditar ejecutores, memoria y núcleos es la **REST API de monitoreo** que el Driver publica junto a la Spark UI.

PYTHON

```python
import requests
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Capitulo04-ArquitecturaCluster") \
    .getOrCreate()

sc = spark.sparkContext

# 1. Identidad de la aplicación y paralelismo por defecto
print(f"Spark Version: {sc.version}")
print(f"Master URL: {sc.master}")
print(f"Application ID: {sc.applicationId}")
print(f"Paralelismo por defecto: {sc.defaultParallelism}")

# 2. Ejecutores conectados, consultados a la REST API del Driver
url = f"{sc.uiWebUrl}/api/v1/applications/{sc.applicationId}/executors"
ejecutores = requests.get(url, timeout=10).json()

print(f"\nEjecutores registrados (incluye al driver): {len(ejecutores)}")
for ex in ejecutores:
    print(f" -> id={ex['id']:>6}  host={ex['hostPort']}  cores={ex['totalCores']}
"
          f"tareas activas={ex['activeTasks']}  memoria máx={ex['maxMemory'] /
1024**2:.0f} MiB")
```

4.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutores "Demasiado Gordos" (Fat Executors)

Configurar ejecutores gigantes con 64 GB de RAM y 16 núcleos en una sola JVM tiende a producir pausas largas de *Garbage Collection* (GC) y cuellos de botella de I/O en HDFS. La heurística más difundida (popularizada por Cloudera y reflejada en guías de ajuste de Spark) es usar **alrededor de 5 núcleos por ejecutor** y memorias del orden de decenas de GB; es un punto de partida empírico, no una ley, y debe validarse midiendo GC en la Spark UI.

Anti-patrón 2: Ejecutores "Demasiado Enanos" (Tiny Executors)

Asignar 1 núcleo y 1 GB de RAM por ejecutor desperdicia memoria en la sobrecarga base de la JVM de cada proceso y elimina los beneficios de procesar múltiples hilos en memoria compartida.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 4.1 (Cálculo de Dimensionamiento de Clúster):

Dispones de 10 nodos trabajadores en AWS (instancias de 16 vCPUs y 64 GB de RAM cada una). Dejando 1 vCPU y 2 GB por nodo para el sistema operativo y el demonio de YARN/K8s, ¿cuántos ejecutores de 5 núcleos deberías solicitar y con cuánta memoria por ejecutor?

Solución: Cálculo paso a paso:

1. *Recursos disponibles por nodo:* 15 vCPUs y 62 GB de RAM.
2. *Ejecutores por nodo:* 15 vCPUs / 5 núcleos por ejecutor = **3 ejecutores por nodo**.
3. *Memoria bruta por ejecutor:* 62 GB / 3 = ~20.6 GB.
4. *Reserva de overhead de memoria (10%):* ~2 GB para `spark.executor.memoryOverhead`.
5. *Configuración final:* `spark.executor.instances = 29` (dejando 1 ejecutor reservado para el Driver ApplicationMaster), `spark.executor.cores = 5`, `spark.executor.memory = 18g`.

4.4 Resumen y Puntos Clave

- **Separación de roles:** El Driver coordina y planifica; el Cluster Manager asigna hardware; los Ejecutores procesan particiones en paralelo.
- **Aislamiento JVM:** Cada aplicación opera en un entorno de memoria desacoplado de las demás.
- **Dimensionamiento balanceado:** Ni ejecutores gigantescos ni diminutos; 4 a 5 núcleos por ejecutor es el equilibrio óptimo de rendimiento y recolección de basura.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide.* O'Reilly Media. (Capítulo 15: How Spark Runs on a Cluster).
2. **Ryza, S. (2014).** *How-to: Tune Your Apache Spark Jobs (Part 2).* Cloudera Engineering Blog.

CAPÍTULO 5

Entorno de Configuración y SparkSession NÚCLEO

> *"SparkSession es el punto de entrada unificado al universo de Spark: permite crear y manipular DataFrames, ejecutar consultas SQL, cargar archivos Parquet o JSON y definir los parámetros de paralelismo y memoria que gobernarán el clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución desde los contextos dispersos de Spark 1.x (`SparkContext`, `SQLContext`, `HiveContext`) hacia el punto de entrada unificado `SparkSession` de Spark 2.x y 3.x.
- Dominar el patrón de diseño Builder para inicializar y parametrizar aplicaciones PySpark de forma segura.
- Utilizar `SparkConf` para inyectar configuraciones dinámicas desde archivos JSON o variables de entorno.
- Configurar parámetros críticos de memoria, serialización y paralelismo antes de disparar cómputo en el clúster.

Definición 5.1 (La Llave Maestra del Edificio Inteligente) En las versiones tempranas de Spark, operar el sistema era como llevar un manojo de cinco llaves metálicas pesadas: necesitabas la llave del ascensor (SparkContext), la llave de la sala de conferencias (SQLContext) y la llave del archivo histórico (HiveContext). Si olvidabas una, no podías trabajar.

SparkSession es una tarjeta digital de acceso unificado: una sola llave inteligente que te abre todas las puertas del edificio, conecta con el catálogo de datos, ejecuta SQL y lee cualquier formato de archivo con un único apretón de manos.

5.1 El Punto de Entrada Unificado: SparkSession

En Apache Spark, `SparkSession` actúa como el "corazón" programático de la aplicación. Introducida en Spark 2.0 y consolidada como el estándar indiscutible de la industria, unifica todas las capacidades del motor bajo una misma interfaz fluida.

Con una instancia de `SparkSession` puedes:

1. Crear y manipular DataFrames a partir de fuentes externas o colecciones nativas de Python.
2. Ejecutar código SQL directamente sobre los datos mediante `spark.sql()`.
3. Leer archivos en formatos columnares o estructurados (`parquet`, `json`, `csv`, `orc`, `delta`, `avro`).
4. Acceder y alterar dinámicamente configuraciones internas del runtime mediante `spark.conf`.

El Patrón Builder

Debido a que una aplicación distribuida requiere configurar múltiples parámetros antes de que el motor inicie, SparkSession se construye utilizando el **patrón Builder**:

PYTHON

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiPipelineDeDatos") \
    .config("spark.sql.shuffle.partitions", "200") \
    .getOrCreate()
```

El método `.getOrCreate()` garantiza que si ya existe una sesión activa en la JVM (por ejemplo al trabajar en Databricks, Jupyter o EMR), Spark reutiliza la sesión existente en lugar de lanzar un segundo contexto conflictivo.

5.2 Profundización Técnica: SparkConf y el Puente Py4J

¿Cómo se comunica Python con un motor escrito en Scala y Java? PySpark utiliza una pasarela llamada **Py4J**:

1. Tu código Python invoca métodos en el Driver.
2. Py4J traduce cada llamada a un socket TCP local comunicándose con la JVM subyacente.

3. `SparkConf` es la clase que define los parámetros inmutables del clúster antes de que arranque la JVM. Parámetros como el tamaño del Driver (`spark.driver.memory`) o el master URL (`spark.master`) **deben definirse en SparkConf antes de inicializar la sesión**; una vez encendida la JVM, no pueden modificarse en caliente.

> **Consideraciones de Producción 5.1: Carga de Configuraciones desde JSON** En proyectos de ingeniería de producción nunca se deben quemar configuraciones en duro en el código. El patrón profesional consiste en separar la configuración en un archivo externo (JSON o YAML) e inyectarla dinámicamente a través de SparkConf.

PYTHON

```
import json
import os
from pyspark import SparkConf
from pyspark.sql import SparkSession

```python
# 1. Configuración de parámetros en SparkConf
conf = SparkConf()
conf.set("spark.app.name", "PipelineIngestaProduccion")
conf.set("spark.master", "local[*]")
conf.set("spark.sql.shuffle.partitions", "16")
conf.set("spark.sql.adaptive.enabled", "true")

# Nota técnica: KryoSerializer optimiza la serialización de RDDs y clases
Java/Scala personalizadas;
# en la API de DataFrames, Spark utiliza internamente los codificadores binarios
de Tungsten.
conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")

# 2. Inicializar SparkSession aplicando el objeto SparkConf
spark = SparkSession.builder \
    .config(conf=conf) \
    .getOrCreate()
```

5.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Intentar cambiar memoria en tiempo de ejecución

Escribir `spark.conf.set("spark.executor.memory", "8g")` después de llamar a `getOrCreate()` no produce ningún efecto. La memoria de los ejecutores es asignada por el Cluster Manager en el instante del arranque del proceso JVM. Toda asignación de hardware debe pasarse vía `spark-submit --executor-memory 8g` o en el `SparkConf` inicial.
```

Anti-patrón 2: No cerrar la sesión al finalizar scripts por lotes
En pipelines orquestados por Airflow o scripts standalone, olvidar llamar a `spark.stop()` mantiene la aplicación activa en YARN o Kubernetes, consumiendo recursos y bloqueando cuotas de cómputo del clúster innecesariamente.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 5.1 (Serialización Kryo frente a Codificadores de Tungsten):
¿Por qué configurar `spark.serializer = org.apache.spark.serializer.KryoSerializer` es crítico cuando se opera con RDDs o tipos personalizados, pero tiene un impacto menor en pipelines que utilizan exclusivamente la API DataFrame?

Solución 5.1:

- **En la API DataFrame:** Spark emplea el motor **Tungsten**, el cual almacena y serializa los datos en formato binario compacto fuera del modelo de objetos tradicional de Java mediante codificadores (*Encoders*) y representaciones en memoria `UnsafeRow`. Por ello, las filas de un DataFrame no se serializan con Java ni con Kryo durante las operaciones analíticas estándar.
- **En RDDs y tipos no gestionados:** Cuando se manipulan colecciones de objetos Java/Scala arbitrarios o RDDs con `groupByKey` o `reduceByKey`, el serializador predeterminado de Java introduce una sobrecarga masiva de metadatos de clases. En este escenario, **Kryo** es esencial: serializa objetos binariamente de forma compacta (hasta 10 veces más rápido y con una reducción de tamaño del 50% al 75% frente a Java), descongestionando la memoria y la red durante la fase de shuffle.

5.4 Resumen y Puntos Clave

- **Punto de entrada único:** `SparkSession` unifica APIs estructuradas, ejecución SQL y conectividad de almacenamiento.
- **Inmutabilidad de arranque:** Configuraciones críticas de memoria y master se definen al inicio mediante `SparkConf` o flags de `spark-submit`.
- **Cierre disciplinado:** Toda aplicación por lotes debe finalizar con `spark.stop()` para liberar los recursos asignados por el clúster.

Lecturas Recomendadas

1. **Documentación Oficial de Apache Spark:** *Spark Configuration Guide* (`https://spark.apache.org/docs/latest/configuration.html`).
2. **Damji, J. et al. (2020).** *Learning Spark (2nd Edition).* O'Reilly Media. (Capítulo 2: Downloading Apache Spark and Getting Started).

CAPÍTULO 6

Abstracciones de Cómputo: De RDDs a DataFrames RECOMENDADO

> *"El RDD es el ladrillo fundamental de la computación distribuida en Spark: una colección inmutable, particionada y tolerante a fallos. Sin embargo, su desconocimiento de los esquemas obligó a la industria a evolucionar hacia la API de DataFrames para desbloquear la optimización automática de Catalyst."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la definición formal y las propiedades matemáticas de un **RDD** (*Resilient Distributed Dataset*).
- Describir los tres mecanismos fundamentales del RDD: Inmutabilidad, Particionado y Linaje (*Lineage Graph*).
- Explicar por qué los **DataFrames** superaron a los RDDs en analítica empresarial (conocimiento de esquemas y motor Tungsten).
- Identificar los escenarios especializados donde aún es pertinente descender a nivel de RDDs en producción.

Definición 6.1 (Cajas Opacas vs. Estanterías Etiquetadas) Un RDD es como almacenar tus pertenencias en cajas de cartón completamente selladas y sin etiquetas: el transportista (Spark) sabe que hay 1.000 cajas y puede mover 100 en cada camión, pero no tiene ni idea de qué hay adentro. Si le pides buscar la edad de un cliente, tiene que abrir cada caja, deserializar el contenido y revisar a ciegas.

Un DataFrame es como colocar los datos en una estantería transparente de vidrio con divisiones milimétricas y etiquetas de tipos (un esquema relacional). Spark puede ver desde lejos que la columna 'edad' está en el tercer compartimento y leer directamente los enteros sin abrir el resto de los datos.

6.1 Fundamentos y Anatomía del RDD

Un **RDD** (*Resilient Distributed Dataset*) es la abstracción de datos fundamental y más básica en Apache Spark. Se define como una colección **inmutable, particionada y tolerante a fallos** de elementos que pueden ser procesados en paralelo en un clúster de computadoras.

Sus tres pilares fundamentales son:

1. **Resilient (Resiliente / Tolerante a fallos):** Si un nodo que contiene una partición de datos falla físicamente a mitad de un cómputo, Spark no necesita reiniciar todo el proceso desde cero. Utilizando el grafo de dependencias o linaje (*Lineage*), el motor recalcula exclusivamente la partición perdida en otro nodo.
2. **Distributed (Distribuido):** Los datos no residen en una sola máquina; están divididos en múltiples **particiones lógicas** dispersas en la memoria RAM y discos de los Worker Nodes del clúster.
3. **Dataset (Colección de datos):** Representa una colección de objetos fuertemente tipados (en Scala/Java) o estructuras genéricas en Python.

Inmutabilidad y Linaje

Los RDDs son **estrictamente inmutables**: una vez creado un RDD, su contenido no puede modificarse en caliente. Para alterar datos, se aplica una transformación que genera un **nuevo RDD**. Esta propiedad garantiza:

- Ausencia de condiciones de carrera (*race conditions*) en ejecuciones paralelas multihilo.
- Facilidad de serialización y replicación en memoria.
- Registro exacto del grafo de derivación para tolerancia a fallos.



6.2 Profundización Técnica: ¿Por qué los DataFrames Reemplazaron a los RDDs?

A pesar de su elegancia matemática, los RDDs presentan limitaciones severas para analítica a gran escala:

1. **Caja Negra de Tipos:** Spark no comprende la estructura interna de los objetos dentro de un RDD. No sabe si contiene un string, una clase personalizada o un float; por ende, no puede optimizar expresiones relacionales.
2. **Sobrecarga de Serialización en Python:** En PySpark, cada elemento de un RDD debe serializarse con `pickle` de Python a la JVM y viceversa, degradando la velocidad de CPU dramáticamente.
3. **DataFrames al rescate:** Un **DataFrame** es conceptualmente un RDD de objetos `Row` acompañados de un **Esquema Estricto** (nombres y tipos de columnas). Conocer el esquema permite al motor **Catalyst** optimizar consultas y al motor **Tungsten** almacenar datos en formato binario compacto (*UnsafeRow*), reduciendo drásticamente la sobrecarga de objetos Java al gestionar memoria estructurada directamente en el heap (y opcionalmente fuera del heap con `spark.memory.offHeap.enabled`).

> **Consideraciones de Producción 6.1: Operaciones con RDDs vs. Conversión a DataFrames**
> Aunque en la inmensa mayoría de los casos analíticos utilizará DataFrames, descender a la API de RDDs es útil para procesar datos desestructurados (como archivos de texto crudos o logs de red sin delimitadores).

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder \
    .appName("Capitulo06-RDDs-a-DataFrames") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext

# 1. Crear un RDD a partir de una lista cruda en memoria
rdd_crudo = sc.parallelize([
    ("prod_101", 15),
    ("prod_102", 40),
    ("prod_101", 35),
    ("prod_103", 20)
])

# 2. Transformaciones funcionales de bajo nivel sobre RDD
# reduceByKey suma los valores asociados a cada clave en paralelo
rdd_totales = rdd_crudo.reduceByKey(lambda x, y: x + y)
print("Resultado en RDD:", rdd_totales.collect())

# 3. Transición profesional: Convertir RDD a DataFrame con esquema estricto
esquema = StructType([
    StructField("producto_id", StringType(), False),
```

```
    StructField("cantidad_total", IntegerType(), False)
])

df_optimizado = spark.createDataFrame(rdd_totales, schema=esquema)
df_optimizado.show()
```

6.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Usar RDDs para Transformaciones Tabulares
Escribir transformaciones complejas con funciones lambda de Python sobre RDDs (ej. `rdd.map(lambda x: ...).filter(lambda x: ...)`) desactiva por completo el optimizador Catalyst y genera una penalización severa de rendimiento frente a la API de DataFrames nativa optimizada por Tungsten.

Anti-patrón 2: Pérdida del linaje por sobreescritura incorrecta
Intentar mutar variables en bucles esperando que Spark actualice un RDD existente ignora el principio de inmutabilidad y puede generar estados inconsistentes en la lógica de negocio.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 6.1 (Linaje vs. Replicación):
Si un nodo trabajador con 4 particiones de un RDD se destruye a mitad de un job, describe exactamente los pasos que toma Spark para recuperarse sin reiniciar la aplicación completa.

Solución 6.1:

1. El Driver detecta la desconexión del ejecutor mediante el mecanismo de latidos (*heartbeats*).
2. El Driver consulta el **grafo de linaje** registrado en memoria para identificar exactamente qué transformaciones originaron esas 4 particiones específicas.
3. El Driver re-programa esas 4 tareas en otros ejecutores vivos disponibles.
4. Los nuevos ejecutores leen los datos originales de la fuente y re-ejecutan únicamente la cadena de transformaciones necesaria para regenerar esas 4 particiones, continuando el flujo sin interrupción para el usuario.

6.4 Resumen y Puntos Clave

- **El bloque constructivo:** RDD es la unidad atómica de tolerancia a fallos e inmutabilidad en Spark.

- **El salto a DataFrames:** Proporcionan esquemas estructurados, permitiendo que Catalyst y Tungsten optimicen el uso de CPU y gestionen memoria compacta binaria (on-heap por defecto y off-heap opcional).
- **Regla de oro:** Utilice DataFrames para todo procesamiento relacional y analítico; reserve RDDs únicamente para manipulación de streams de texto no estructurado o librerías legacy.

Lecturas Recomendadas

1. **Zaharia, M. et al. (2012).** *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.* NSDI '12.
2. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide.* O'Reilly Media. (Capítulo 12: Resilient Distributed Datasets).

PARTE III

Ingesta, formatos y esquemas

Capítulo 7 · Ingesta de Datos: Archivos, JDBC Batch y Fuentes Incrementales NÚCLEO

Capítulo 8 · Formatos de Archivo: Parquet, ORC y Avro NÚCLEO

Capítulo 9 · Tipos de Datos y Casting Riguroso en Memoria NÚCLEO

CAPÍTULO 7

Ingesta de Datos: Archivos, JDBC Batch y Fuentes Incrementales NÚCLEO

> *"La ingesta es el primer puerto de entrada al lago de datos: a través del DataFrameReader podemos montar rutas de almacenamiento de objetos, inferir o forzar esquemas estrictos, conectarnos en paralelo a bases de datos relacionales vía JDBC y capturar incrementos controlados mediante columnas de marca de agua."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Utilizar el objeto `DataFrameReader` (`spark.read`) para cargar datos desde almacenamiento de objetos (Amazon S3, Azure ADLS Gen2, GCP GCS) y sistemas de archivos locales.
- Configurar opciones críticas de lectura: delimitadores, encabezados, manejo de caracteres corruptos y modos de fallo (`PERMISSIVE`, `DROPMALFORMED`, `FAILFAST`).
- Comprender la penalización de rendimiento de `inferSchema` y aplicar esquemas explícitos mediante `StructType`.
- Diseñar ingestas paralelas desde motores relacionales mediante **JDBC** (`spark.read.jdbc`) configurando `partitionColumn`, `lowerBound`, `upperBound` y `numPartitions`.
- Implementar patrones de **extracción incremental batch** basados en marcas de agua temporales (*Watermark Columns*).
- Evaluar el rol de conectores gestionados modernos (**Airbyte**, **Fivetran**, SFTP y APIs REST) frente a pipelines Spark nativos.

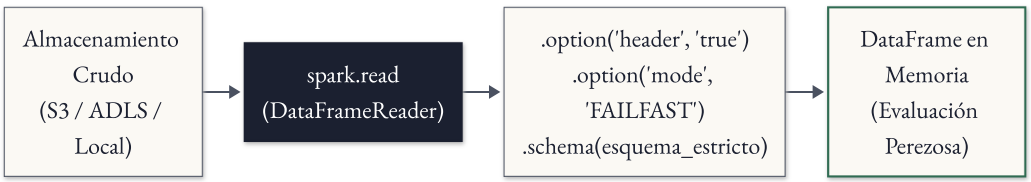
> **Definición 7.1 (La Aduana en el Puerto de Carga Analítico)** Imagine cientos de contenedores marítimos arribando a un puerto de transferencia internacional.
>
> Si el inspector de aduana decide inferir el contenido abriendo cada caja una por una (`inferSchema = True`), se formará una congestión kilométrica y el barco tardará días en desembarcar, consumiendo el doble de combustible e I/O.
>
> Un puerto moderno exige un manifiesto de carga oficial sellado de antemano (un `StructType` explícito): el operario solo valida que los contenedores coincidan con las especificaciones y los transfiere a máxima velocidad hacia las grúas de clasificación sin inspecciones redundantes.

7.1 El Objeto `DataFrameReader` y Formatos de Archivo

En PySpark, la carga de datos se canaliza a través de la propiedad `.read` de la instancia `SparkSession`, la cual retorna un objeto `DataFrameReader`.

El flujo estándar de ingesta sigue tres pasos estructurales:

1. **Definir la ruta fuente:** Puede ser una ruta local del sistema de archivos, un bucket de Amazon S3 (`s3a://`), Azure Data Lake Storage (`abfss://`) o Google Cloud Storage (`gs://`).
2. **Especificar el formato:** Mediante métodos directos (`.csv()`, `.parquet()`, `.json()`, `.orc()`, `.text()`) o el método genérico `.format("formato")`.
3. **Parametrizar opciones de lectura:** Configurar delimitadores (`sep`), presencia de cabeceras (`header`), manejo de valores nulos (`nullValue`) y políticas de parseo ante registros malformados.



Modos de Manejo de Registros Corruptos

Al ingerir fuentes de texto semiestructuradas (como CSV o JSON) es común encontrar filas incompletas o tipos de datos alterados. Spark ofrece tres modos de parseo mediante `.option("mode", ...)`:

1. **PERMISSIVE (Por defecto):** Inserta `null` en los campos corruptos y, si se define una columna especial `_corrupt_record` en el esquema, almacena la cadena cruda malformada para su posterior auditoría.

2. **DROPMALFORMED:** Descarta silenciosamente las filas que no cumplan el esquema.
3. **FAILFAST:** Lanza una excepción inmediata (`SparkException`) y detiene el job en el primer registro corrupto. Es el estándar recomendado para ingestas financieras donde la pérdida de datos es inaceptable.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType

spark = SparkSession.builder \
    .appName("Capitulo07-IngestaArchivos") \
    .master("local[*]") \
    .getOrCreate()

# Definición rigurosa del esquema oficial (Manifiesto de Carga)
esquema_pedidos = StructType([
    StructField("pedido_id", StringType(), False),
    StructField("cliente_id", IntegerType(), False),
    StructField("fecha_creacion", StringType(), True),
    StructField("monto_total", DoubleType(), True),
    StructField("estado", StringType(), True)
])

# Ingesta controlada con DataFrameReader
df_pedidos = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("sep", ",") \
    .option("mode", "FAILFAST") \
    .option("nullValue", "NA") \
    .schema(esquema_pedidos) \
    .load("data/crudo/pedidos_2026.csv")
```

7.2 Ingesta Paralela desde Bases de Datos Relacionales (JDBC)

La extracción de datos desde sistemas relacionales transaccionales (PostgreSQL, MySQL, Oracle, SQL Server) es una de las tareas más comunes del Data Engineer. Utilizar `spark.read.jdbc` sin comprender sus parámetros internos puede colapsar la base de datos origen o generar cuellos de botella extremos:

La Trampa del Hilo Único (Single-Thread Bottleneck)

Si se ejecuta:

```python
# ANTI-PATRÓN: Extracción monohilo
df_clientes = spark.read.jdbc(url=jdbc_url, table="clientes",
properties=db_properties)
```

Spark lanzará **una única tarea en un único ejecutor** para descargar millones de registros a través de una sola conexión JDBC. El resto del clúster permanecerá totalmente ocioso.

Extracción Paralela Particionada por Rango

Para extraer la tabla de forma distribuida en N hilos concurrentes, se debe especificar una columna numérica particionable:

```python
jdbc_url = "jdbc:postgresql://db-oltp.corp:5432/produccion"
db_properties = {
    "user": "analitica_read_only",
    "password": "token_secreto_vault",
    "driver": "org.postgresql.Driver",
    "fetchsize": "10000" # Tamaño del cursor por lote para evitar OOM en la JVM
}

# Ingesta paralela distribuida en 8 ejecutores
df_clientes_paralelo = spark.read.jdbc(
    url=jdbc_url,
    table="clientes",
    column="cliente_id",      # Debe ser columna numérica indexada
    lowerBound=1,             # Valor mínimo esperado
    upperBound=8000000,       # Valor máximo esperado
    numPartitions=8,          # Cantidad de conexiones paralelas simultáneas
    properties=db_properties
)
```

Spark generará internamente 8 consultas SQL concurrentes dividiendo el rango mediante operadores `WHERE`:

- *Tarea 0:* `WHERE cliente_id < 1000000`
- *Tarea 1:* `WHERE cliente_id >= 1000000 AND cliente_id < 2000000`
- ...
- *Tarea 7:* `WHERE cliente_id >= 7000000`

7.3 Extracción Incremental Batch con Marca de Agua (Watermark)

Extraer la tabla transaccional completa todos los días (*Full Table Dump*) es inviable ante tablas con cientos de millones de tuplas. El patrón de **Extracción Incremental** utiliza una columna de auditoría monotónicamente creciente (por ejemplo, `fecha_actualizacion` o `modificado_el`) para consultar únicamente las novedades:

PYTHON

```python
# 1. Recuperar la marca de agua del último lote procesado exitosamente
ultima_marca_tiempo = "2026-10-11 23:59:59"

# 2. Construir predicado de extracción incremental (Pushdown hacia la base relacional)
query_incremental = f"""
    (SELECT cliente_id, nombre, saldo, modificado_el
     FROM cuentas
     WHERE modificado_el > '{ultima_marca_tiempo}') AS novedades_cuentas
"""

df_novedades = spark.read.jdbc(
    url=jdbc_url,
    table=query_incremental,
    column="cliente_id",
    lowerBound=1,
    upperBound=5000000,
    numPartitions=4,
    properties=db_properties
)

# 3. Guardar las novedades en la capa Bronze
df_novedades.write.format("delta").mode("append").save("lakehouse/bronze/cuentas_novedades")
```

7.4 Ingesta Gestionada: Airbyte, Fivetran, SFTP y APIs REST

En la arquitectura empresarial moderna, Spark no es siempre la mejor herramienta para conectar cientos de fuentes heterogéneas:

- **Herramientas ELT Gestionadas (Airbyte / Fivetran):** Especializadas en sincronizar conectores SaaS (Salesforce, Stripe, HubSpot, Google Ads) y bases relacionales directamente hacia almacenamiento de objetos o almacenes MPP, gestionando esquemas, reintentos y paginación sin escribir código manual.
- **Archivos SFTP / Legacy:** Para archivos CSV o XML recibidos por SFTP de proveedores externos, se emplean contenedores ligeros de Python o tareas de Airflow que descargan y

depositan los datos en el bucket Landing antes de activar Spark.

- **Consumo de APIs REST:** Si la fuente expone una API REST con límites de tasa (*Rate Limits*) y paginación por cursor, un script de Python con `requests` y retroceso exponencial (*Exponential Backoff*) es superior a un clúster de Spark.

7.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Creer que `coalesce` Resuelve el Problema de Archivos Pequeños en Ingesta
Leer miles de micro-archivos de 10 KB genera una sobrecarga severa en el Driver durante la fase de listado de metadatos en almacenamiento de objetos. Aplicar `df.coalesce(1)` tras la lectura **no soluciona el cuello de botella**, ya que el Driver ya tuvo que listar y programar todas las particiones. La solución real exige compactar los archivos en reposo (mediante Delta `OPTIMIZE` o procesos de compactación programados) o ajustar `spark.sql.files.maxPartitionBytes` (128 MB por defecto) para que Spark agrupe múltiples archivos pequeños en una sola tarea física de lectura.

Anti-patrón 2: Ejecutar Ingestas JDBC sobre Tablas Transaccionales en Horas Pico sin Índices
Lanzar una consulta `spark.read.jdbc` con `numPartitions = 32` sobre una columna que no tiene índice en PostgreSQL puede provocar bloqueos de tablas y saturación de CPU en la base de datos de producción. Verifique siempre que la columna de particionamiento posea un índice B-tree y ajuste las horas de ejecución a ventanas de bajo tráfico.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 7.1 (Descubrimiento Automático de Particiones):
Si un lago de datos almacena archivos Parquet organizados como `s3://datalake/ventas/anio=2026/mes=10/datos.parquet`, ¿qué sucede si se ejecuta `spark.read.parquet("s3://datalake/ventas/")`?

Solución 7.1:
Spark implementa **Partition Discovery automático**. Al detectar la convención `clave=valor` en las rutas de directorios, Spark escanea los archivos e inyecta dinámicamente las columnas virtuales `anio` (Integer) y `mes` (Integer) en el esquema del DataFrame. Si una consulta posterior incluye `filter(col("mes") == 10)`, Catalyst aplica *Partition Pruning*, descartando la lectura de todos los directorios que correspondan a otros meses sin transferir sus archivos físicos por la red.

Ejercicio 7.2 (Tuning de Conexiones JDBC en Spark):
Un pipeline JDBC tarda 45 minutos en extraer 10 millones de filas desde MySQL. Al revisar la Spark UI, el ingeniero observa un único Stage con una sola tarea (*Task*). Explique la causa y detalle la solución en código.

Solución 7.2:

- **Causa:** La llamada `spark.read.jdbc` no especificó los parámetros de paralelismo (`column`, `lowerBound`, `upperBound`, `numPartitions`). Al omitirlos, Spark utiliza una única conexión JDBC secuencial en un solo ejecutor.
- **Solución:** Especificar una columna numérica indexada (como la clave primaria `id`) y definir el número de particiones:

```python
df = spark.read.jdbc(
    url=url, table="transacciones",
    column="id", lowerBound=1, upperBound=10000000,
    numPartitions=10, properties=props
)
```

Esto divide el trabajo en 10 tareas paralelas, reduciendo el tiempo de extracción de 45 minutos a menos de 5 minutos según la capacidad del servidor de base de datos.

7.6 Resumen y Puntos Clave

- **DataFrameReader:** La interfaz unificada para ingestar archivos y fuentes relacionales con control de esquemas y modos de fallo.
- **Esquemas explícitos:** Suprimen la doble lectura de I/O provocada por `inferSchema` y garantizan tipado determinista.
- **Paralelismo JDBC:** Exige configurar `partitionColumn`, cotas de rango y `numPartitions` sobre columnas indexadas para evitar extracciones monohilo.
- **Extracción incremental:** Minimiza el impacto sobre fuentes operacionales consultando solo registros cuya marca de agua temporal sea superior al último corte exitoso.

Lecturas Recomendadas

1. **Damji, J. et al. (2020).** *Learning Spark: Lightning-Fast Data Analytics* (2nd ed.). O'Reilly Media. (Capítulo 4: "Spark SQL and DataFrames: Built-in Data Sources").

2. **PostgreSQL Global Development Group (2024).** *PostgreSQL Documentation: Concurrency Control and Indexes.* (`https://www.postgresql.org/docs/`).

3. **Apache Spark Documentation (2024).** *Spark SQL Programming Guide: JDBC To Other Databases.* Apache Software Foundation (`https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html`).

CAPÍTULO 8

Formatos de Archivo: Parquet, ORC y Avro NÚCLEO

> *"La elección del formato de archivo es la decisión de arquitectura física más determinante en un lago de datos: define la tasa de compresión, la eficiencia de E/S mediante lectura columnar y la capacidad de evolucionar esquemas en el tiempo."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

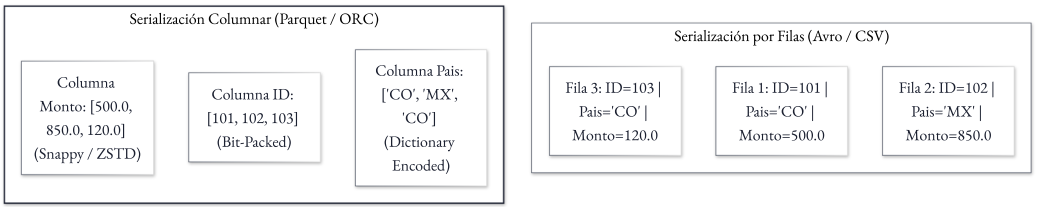
- Diferenciar formalmente entre formatos de almacenamiento orientados a filas (*Row-oriented*) y orientados a columnas (*Columnar-oriented*).
- Describir la estructura interna de Apache Parquet: *File Footer*, *Row Groups*, *Column Chunks* y páginas de datos.
- Comparar los casos de uso óptimos para Apache Parquet, Apache ORC y Apache Avro en arquitecturas de Big Data.
- Aprovechar las estadísticas de bloque (Min/Max) para habilitar poda de datos (*Data Skipping*) y optimizar costos de almacenamiento en la nube.
- Configurar algoritmos de compresión modernos (Snappy, ZSTD, Gzip) según el perfil de carga (escritura frecuente vs. lectura intensiva).

Definición 8.1 (Almacenamiento Columnar y Principio de Proyección Física) Un formato de almacenamiento **orientado a columnas** organiza los datos en disco agrupando secuencialmente todos los valores pertenecientes a una misma columna, en lugar de almacenar tuplas consecutivas de filas. Esta disposición física desacopla el costo de I/O del ancho total de la tabla (*número de columnas*): las consultas analíticas solo transfieren a memoria los bloques de bytes correspondientes a las columnas proyectadas en la consulta (*Column Pruning*).

8.1 Almacenamiento Orientado a Filas vs. Orientado a Columnas

En el diseño de sistemas analíticos modernos coexisten dos filosofías fundamentales de serialización en almacenamiento secundario:

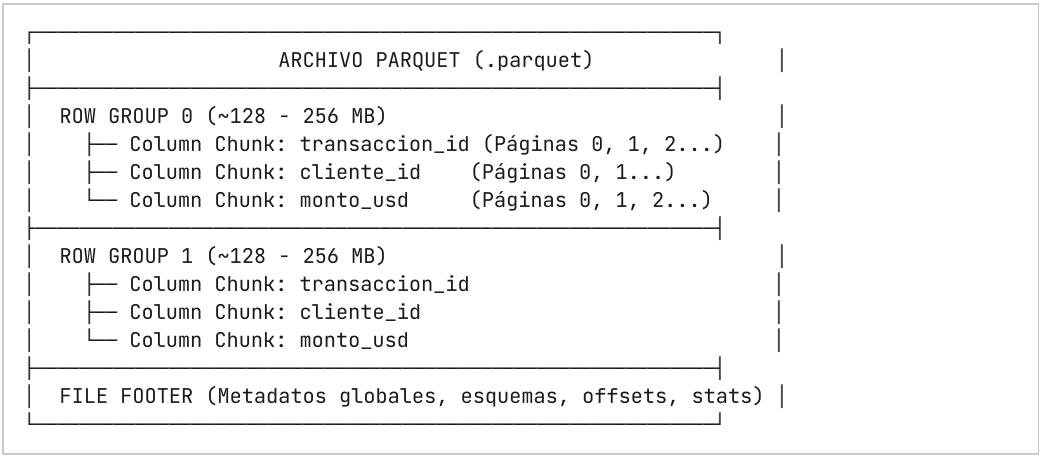
1. **Almacenamiento Orientado a Filas (Row-Oriented - CSV, JSON, Apache Avro):**
   - Cada registro se almacena de forma contigua en bloques secuenciales de disco: `[id_1, nombre_1, balance_1]`, `[id_2, nombre_2, balance_2]`.
   - *Ventaja:* Eficiencia máxima en escrituras atómicas puntuales y recuperación de entidades completas por identificador primario (ideal para cargas transaccionales OLTP y buses de mensajería como Kafka).
   - *Desventaja en analítica:* Para calcular la suma de una columna sobre 1.000 millones de filas en una tabla con 200 columnas, el motor debe leer el 100% de los bytes del archivo, desperdiciando ancho de banda de red y memoria caché.
2. **Almacenamiento Orientado a Columnas (Columnar-Oriented - Apache Parquet, Apache ORC):**
   - Todos los valores de un mismo atributo se agrupan en bloques contiguos: `[id_1, id_2, id_3...]`, `[balance_1, balance_2, balance_3...]`.
   - *Ventaja analítica (OLAP):* Permite **Column Pruning** (lectura selectiva de columnas) y algoritmos de compresión de alta densidad (como *Run-Length Encoding* y codificación por diccionario), reduciendo el tamaño en almacenamiento entre un 60% y un 85% comparado con formatos planos.



8.2 Anatomía Interna de Apache Parquet

Un archivo Apache Parquet es un contenedor binario autosuficiente estructurado jerárquicamente en cuatro capas de granularidad:

1. **File Footer (Pie de Archivo):**
   Ubicado al final del archivo físico. Contiene el esquema completo de la tabla y los punteros de desplazamiento (*byte offsets*) hacia cada grupo de filas. Los lectores de Parquet leen primero los últimos bytes del archivo para conocer la estructura completa sin escanear el cuerpo de datos.
2. **Row Groups (Grupos de Filas):**
   Particiones lógicas horizontales de los datos, típicamente dimensionadas entre 128 MB y 512 MB. Un archivo puede contener múltiples Row Groups.
3. **Column Chunks (Trozos de Columna):**
   Dentro de cada Row Group, los datos de una columna específica se almacenan en un bloque exclusivo contiguo.
4. **Data Pages (Páginas de Datos):**
   Unidad mínima e indivisible de lectura y compresión (típicamente 1 MB). Cada página contiene valores codificados y comprimidos, junto con encabezados de metadatos que almacenan estadísticas locales: valor mínimo, valor máximo, recuento de nulos y cardinalidad.



8.3 Profundización Técnica: Técnicas de Codificación y Poda de Datos (Data Skipping)

La eficiencia extrema de Parquet no radica únicamente en la compresión (ej. Snappy), sino en las técnicas de codificación previa (*Encoding*):

1. **Dictionary Encoding:** Sustituye cadenas de texto repetitivas (ej. nombres de países) por enteros pequeños (0, 1, 2) y mantiene una tabla de correspondencia en el encabezado.
2. **Run-Length Encoding (RLE):** Si un valor booleano o de estado se repite 500 veces seguidas, se almacena como el par `(500, 'APROBADO')` en lugar de escribir 500 registros.

3. **Data Skipping mediante Estadísticas Min/Max:**
   Cuando un motor ejecuta una consulta con filtro:

```sql
SELECT SUM(monto_usd) FROM ventas WHERE transaccion_id = 'TRX-998822';
```

   El motor lee el *File Footer* y los encabezados de cada *Row Group*. Si el rango estadístico de un Row Group indica `min: 'TRX-000001', max: 'TRX-500000'`, el motor **omite por completo la lectura física de ese bloque de 256 MB de disco**, reduciendo el I/O en órdenes de magnitud.

8.4 Comparativa Técnica: Parquet vs. ORC vs. Avro

Dimensión de Comparación	Apache Parquet	Apache ORC	Apache Avro
Orientación	Columnar	Columnar	Filas
Ecosistema Predilecto	Apache Spark, Databricks, Snowflake	Apache Hive, Trino, Presto	Apache Kafka, Mensajería RPC
Estructura de Bloque	Row Groups + Column Chunks	Stripes (típicamente 256 MB)	Sync Markers + Blocks
Soporte de Esquema Complejo	Excelente (anidamientos Dremel)	Excelente (estructuras y mapas)	Excelente (JSON Schema estricto)
Evolución de Esquemas	Adición y eliminación de columnas	Compatible hacia atrás	Sobresaliente (reglas estrictas)
Caso de Uso Primario	Data Lakes, Lakehouses, BI masivo	Almacenes Hive y consultas SQL	Ingesta de streaming y buses CDC

Consideraciones de Producción 8.1: Selección de Algoritmos de Compresión (Snappy vs. ZSTD)

- **Snappy (Predeterminado en Spark):** Optimizado para velocidad extrema de descompresión con consumo moderado de CPU. Ideal para capas intermedias de procesamiento (Silver) donde los datos se leen frecuentemente.
- **ZSTD (Zstandard):** Desarrollado por Meta. Ofrece ratios de compresión significativamente superiores (reducción adicional del 15-30% respecto a Snappy) con niveles de compresión ajustables. Es el estándar recomendado para capas históricas de almacenamiento (Bronze y Cold Storage) para minimizar facturación en la nube.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType

spark = SparkSession.builder \
    .appName("Capitulo08-OptimizacionFormatos") \
    .config("spark.sql.parquet.compression.codec", "zstd") \
    .config("spark.sql.parquet.filterPushdown", "true") \
    .getOrCreate()

# Escritura con compresión ZSTD y configuración de tamaño de Row Group
df_transacciones = spark.read.json("data/transacciones_crudo/")

df_transacciones.write \
    .mode("overwrite") \
    .option("compression", "zstd") \
    .parquet("lakehouse/silver/transacciones_optimizadas/")
```

8.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Almacenar Cargas Analíticas Masivas en Archivos CSV o JSON Comprimidos en Gzip

Gzip no es divisible (*splittable*); un archivo JSON de 50 GB comprimido en `.gz` debe ser procesado secuencialmente por un único ejecutor de Spark, perdiendo todo beneficio de paralelismo en el clúster.

Anti-patrón 2: Generar Millones de Archivos Parquet Diminutos (*Small Files Problem*)

Escribir archivos Parquet de menos de 10 MB multiplica la sobrecarga de lectura del metastore y el sistema de archivos. El tamaño óptimo para archivos Parquet en almacenamiento de objetos (S3, ADLS) oscila entre 128 MB y 1 GB.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 8.1 (Divisibilidad de Archivos y Compresión):
Explica la diferencia entre *Data Splittability* (capacidad de dividir un archivo en bloques paralelos) y compresión de archivos, detallando por qué Parquet es intrínsecamente divisible independientemente del códec empleado.

Solución 8.1:

- **Data Splittability:** Es la propiedad física que permite a un motor distribuido (Spark o MapReduce) asignar múltiples tareas concurrentes para procesar diferentes segmentos de un mismo archivo físico sin coordinar entre sí.
- **Impacto del códec:** En archivos de texto planos como CSV o JSON, si se comprimen con algoritmos como Gzip (`.gz`), el archivo pierde su divisibilidad porque el flujo de bytes comprimido carece de marcadores de sincronización interna (*sync markers*); un único ejecutor debe descomprimir el archivo de principio a fin de forma secuencial.
- **Divisibilidad de Parquet:** Apache Parquet organiza internamente los datos en **Row Groups** delimitados e indexados en el *File Footer*. Aunque cada página interna esté comprimida con Snappy o ZSTD, los límites y offsets en bytes de cada Row Group están explícitamente declarados en los metadatos. Durante la planificación, Spark divide los archivos en particiones lógicas (*Splits*) gobernadas por rangos de bytes mediante `spark.sql.files.maxPartitionBytes` (128 MB por defecto), asignando los Row Groups a las tareas correspondientes según su posición física, lo que permite procesar múltiples bloques en paralelo sin dependencias secuenciales.

Ejercicio 8.2 (Data Skipping mediante File Footer):
¿Cómo interactúa el *File Footer* de Parquet con el optimizador Catalyst y los lectores en los ejecutores para evitar transferencias de red y lecturas innecesarias en almacenamiento de objetos (Amazon S3 / Azure ADLS)?

Solución 8.2:

1. **Empuje de predicados (Catalyst):** Cuando Spark compila una consulta con filtros relacionales (por ejemplo, `WHERE fecha = '2026-10-12'`), Catalyst empuja dichos filtros (*Predicate Pushdown*) al operador físico `FileScan parquet`, delegando la evaluación de la condición a la capa de lectura.
2. **Lectura de metadatos en ejecutores:** El lector columnar de Parquet en el ejecutor realiza una solicitud HTTP de rango de bytes (*HTTP Range Request*) hacia los últimos kilobytes del

archivo en el almacenamiento de objetos para descargar exclusivamente el **File Footer** (`FileMetaData`).

3. **Inspección de estadísticas de bloque:** El lector examina los metadatos de cada **Row Group** (`ColumnChunkMetaData`), los cuales registran los valores mínimo y máximo (`min_value`, `max_value`) y el conteo de nulos para cada columna.
4. **Descarte de grupos de filas (Row Group Skipping):** Si las estadísticas del Row Group indican un rango entre `'2026-10-01'` y `'2026-10-10'`, el lector determina que la condición `fecha = '2026-10-12'` es imposible de satisfacer en ese bloque y **omite la lectura de las páginas de datos completas de ese Row Group**, ahorrando transferencias de red y ciclos de CPU.

Ejercicio 8.3 (Avro en Streaming vs. Parquet en Lakehouse):
En una arquitectura de ingesta en tiempo real conectada a Apache Kafka, ¿por qué es recomendable utilizar Apache Avro en el bus de streaming y convertir a Apache Parquet en la capa de persistencia del Lakehouse?

Solución 8.3:

- **Apache Avro en Streaming (Kafka):**
  - *Orientación a filas:* Kafka transmite eventos transaccionales individuales emitidos uno por uno. Avro serializa registros como tuplas compactas con encabezados ligeros gobernados por Schema Registry, minimizando la latencia de serialización por mensaje a microsegundos.
  - *Evolución de esquemas:* Avro maneja rigurosamente la compatibilidad hacia atrás y hacia adelante (*BACKWARD/FORWARD*), protegiendo a los consumidores ante cambios de campos en los microservicios productores.
- **Apache Parquet en el Lakehouse:**
  - *Orientación a columnas:* Las consultas analíticas de BI y SQL agregan métricas sobre millones de filas proyectando solo un subconjunto de columnas.
  - *Eficiencia analítica:* Parquet ofrece tasas de compresión superiores (reducción típica del 60% al 80% frente a JSON sin comprimir), codificación por diccionario y soporte nativo para *Column Pruning* y *Data Skipping*, optimizando tanto el almacenamiento en reposo como los tiempos de respuesta de consulta interactiva.

8.6 Resumen y Puntos Clave

- **Orientación física:** Los formatos orientados a filas (Avro, CSV) optimizan la escritura atómica y el streaming; los formatos columnares (Parquet, ORC) optimizan la lectura analítica y el cálculo de agregados.
- **Anatomía de Parquet:** Se estructura jerárquicamente en *File Footer*, *Row Groups*, *Column Chunks* y *Data Pages*, permitiendo lectura de metadatos primero y salto de bloques.
- **Poda de datos (Data Skipping):** Las estadísticas de bloque (Min/Max y nulos) permiten a los motores de consulta descartar particiones y grupos de filas completos sin leer sus datos físicos.
- **Compresión balanceada:** Snappy proporciona máxima velocidad de descompresión para capas intermedias; ZSTD ofrece ratios de compresión superiores para capas frías o históricas.

Lecturas Recomendadas

1. **White, T. (2015).** *Hadoop: The Definitive Guide: Storage and Analysis at Internet Scale* (4th ed.). O'Reilly Media. (Capítulo 12: "Avro" y Capítulo 13: "Parquet").
2. **Melnik, S. et al. (2010).** *Dremel: Interactive Analysis of Web-Scale Datasets.* Proceedings of the VLDB Endowment, 3(1-2), 330-339. (El artículo científico seminal de Google en el que se basa la codificación columnar anidada de Apache Parquet).
3. **Apache Parquet Project Documentation (2024).** *Parquet Format Specification, Metadata and Encodings.* Apache Software Foundation (`https://parquet.apache.org/docs/`).

CAPÍTULO 9

Tipos de Datos y Casting Riguroso en Memoria NÚCLEO

> *"Los tipos de datos son el contrato estricto de ingeniería entre tu lógica y el clúster: le indican al motor distribuido cuántos bytes reservar en la memoria RAM de los nodos ejecutores, cómo indexar columnas y cómo compilar el código de bajo nivel mediante el motor Tungsten."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar el catálogo de tipos de datos en PySpark (`StructType`, `IntegerType`, `LongType`, `DecimalType`, `DateType`, `TimestampType`).
- Aplicar la regla de oro financiera: evitar `Float` y `Double` para cálculos monetarios a favor de `DecimalType`.
- Implementar técnicas de casting individual y en bloque dinámico mediante metadatos.
- Anticipar el comportamiento de un cast inválido (`NULL` silencioso o error) según el modo ANSI, y usar `try_cast` para controlarlo.

Definición 9.1 (La Aduana de la Fábrica de Alta Velocidad) Imagina que estás construyendo una línea de producción automatizada en una fábrica de alta velocidad.

Si a un robot calibrado para recibir tornillos metálicos de exactamente 10 milímetros le envías un bloque de arcilla blanda, la maquinaria se detiene de golpe o el producto terminado sale completamente defectuoso.

En Apache Spark y PySpark, los tipos de datos son exactamente ese contrato de tolerancia milimétrica: le dicen al motor cuántos bytes reservar en memoria y cómo operar en microsegundos. Si dejas que el motor "adivine" tipos leyendo strings mal formateados (`inferSchema`), pagas una lectura adicional de los datos y arriesgas que el esquema inferido cambie de una ejecución a otra.

9.1 El Catálogo Esencial de Tipos en PySpark

Para trabajar con los estándares de la industria, siempre importamos los tipos de forma explícita desde `pyspark.sql.types`:

```python
from pyspark.sql.types import (
    StructType, StructField,
    StringType, IntegerType, LongType,
    DoubleType, DecimalType, BooleanType,
    DateType, TimestampType, ArrayType, MapType
)
```

Tipos Numéricos y Cuándo Elegir Cada Uno

1. `IntegerType` **vs.** `LongType`**:**
   - Un entero estándar (`IntegerType`) ocupa **4 bytes** y almacena valores entre -2.147 millones y +2.147 millones.
   - Si tus identificadores transaccionales, números de factura o métricas de eventos superan esta cifra, debes usar `LongType` (**8 bytes**).
   - Si tus datos jamás superarán ese rango, utilizar `LongType` innecesariamente es desperdiciar la mitad de la memoria RAM del clúster en esas columnas.
2. `DoubleType` **vs.** `DecimalType(precision, scale)` **(Regla de Oro):**
   - **En aplicaciones financieras, contables o transaccionales jamás uses Float ni Double para dinero.**
   - Los tipos de punto flotante introducen errores microscópicos de redondeo binario (estándar IEEE 754) que acumulan discrepancias de centavos.
   - Para dinero y montos exactos se debe usar siempre `DecimalType(18, 2)` o `DecimalType(38, 4)`.
3. `DateType`, `TimestampType` **y** `TimestampNTZType`**:**
   - `DateType` almacena únicamente la fecha de calendario (`YYYY-MM-DD`).
   - `TimestampType` almacena un instante físico absoluto (microsegundos desde la época Unix en UTC). Al mostrarse o proyectarse a cadena, Spark lo renderiza en la zona horaria activa de la sesión (`spark.sql.session.timeZone`).
   - `TimestampNTZType` (*Timestamp Without TimeZone*, introducido en Spark 3.4 y consolidado en Spark 4.0): almacena la fecha y hora local exactamente como fue registrada (año, mes, día, hora, minuto, segundo), sin referencia a huso horario ni ajustes de horario de verano (*Daylight Saving Time*).
4. **El Tipo** `VariantType` **(Spark 4.0):**

- En versiones previas, los datos JSON semiestructurados exigían o bien almacenar cadenas de texto sin tipar (`StringType`) o declarar jerarquías complejas y rígidas de `StructType` y `MapType`.
- El tipo **`VARIANT`** almacena documentos semiestructurados en un formato binario compacto con soporte nativo para navegación directa de campos (`col("payload:cliente.nombre")`), triturado columnar en disco (*sub-column shredding*) y evolución libre de esquema sin penalización de rendimiento.

9.2 Profundización Técnica: El Motor Tungsten y la Memoria Off-Heap

Cuando Spark opera con DataFrames y tipos estrictos, no necesita crear un objeto Java por cada celda. El motor **Project Tungsten** codifica las filas en un formato binario compacto (*UnsafeRow*), que por defecto reside en el heap de la JVM pero se gestiona manualmente, sin sobrecarga de objetos. Opcionalmente puede ubicarse **fuera del heap** (*off-heap*) con `spark.memory.offHeap.enabled=true`, lo que reduce la presión sobre el recolector de basura:

- Un `IntegerType` ocupa 4 bytes contiguos dentro de la fila binaria, en lugar de un objeto `java.lang.Integer` que, con su encabezado y alineación, ocupa típicamente 16 bytes.
- Además, el compilador genera código bytecode en tiempo de ejecución (*Whole-Stage Code Generation*) pensado para aprovechar las cachés L1/L2 del procesador del nodo ejecutor.

> **Consideraciones de Producción 9.1: Casting Dinámico en Bloque** Cuando recibes tablas con decenas de columnas, declarar transformaciones manuales una a una ensucia el código. Un ingeniero senior automatiza el proceso mediante metadatos y sanitización previa:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("Capitulo09-TiposDatosCasting") \
    .master("local[*]") \
    .getOrCreate()

# Cargar dataset con columnas en formato texto crudo
df = spark.read.csv("data/ventas_crudo.csv", header=True)

# 1. Metadatos para casting automatizado: columna -> tipo destino
campos_enteros = ["sucursal_id", "categoria_id", "cantidad_articulos"]
```

```
campos_monetarios = ["precio_unitario", "subtotal", "impuesto_iva", "total"]

casts = {c: F.col(c).cast("int") for c in campos_enteros}
casts.update({c: F.col(c).cast("decimal(18,2)") for c in campos_monetarios})

# 2. Una sola proyección con todos los casts (evita encadenar decenas de
withColumn)
df_procesado = df.withColumns(casts)   # Spark 3.3+
# Equivalente en cualquier versión:
# df_procesado = df.select(*[casts.get(c, F.col(c)).alias(c) for c in
df.columns])

# 3. Sanitización defensiva: try_cast devuelve NULL en vez de lanzar error
df_sanitizado = df_procesado.withColumn(
    "edad_limpia", F.expr("try_cast(edad_raw AS INT)")
)

df_sanitizado.printSchema()
```

Nota: encadenar `withColumn` dentro de un bucle genera un plan lógico cada vez más profundo y ralentiza el análisis de Catalyst; el Cap. 11 profundiza en este anti-patrón.

9.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ignorar cómo se comporta un cast inválido

Qué ocurre al convertir `'N/A'`, `'PENDIENTE'` o una cadena vacía a `IntegerType` **depende de la configuración** `spark.sql.ansi.enabled`:

Modo	Valor por defecto en	`CAST('N/A' AS INT)`
ANSI desactivado	Spark 3.x	Devuelve `NULL` en silencio
ANSI activado	Spark 4.0 en adelante	Lanza un error (`CAST_INVALID_INPUT`) y detiene el job

En Spark 3.x, si no supervisas el resultado con aserciones de calidad de datos, inyectarás millones de nulos invisibles en la capa Silver. En Spark 4.0 el mismo código falla de forma ruidosa, lo cual es más seguro pero puede romper pipelines migrados. La herramienta correcta en ambos modos es `try_cast` (SQL) o `F.expr("try_cast(col AS INT)")`, que devuelve `NULL` de forma explícita cuando la conversión no es posible, y después se cuentan o se enrutan esos nulos a cuarentena (Cap. 28). Activar `spark.sql.ansi.enabled=false` para recuperar el comportamiento antiguo debe ser una decisión consciente y documentada, no un parche.

Anti-patrón 2: Inconsistencia de Tipos en Operaciones de Join

Si unes dos DataFrames donde en una tabla `cliente_id` es `StringType` y en la otra es `IntegerType`, Spark inserta un *cast* implícito sobre una de las columnas. El join sigue funcionando, pero ese cast puede impedir el *pushdown* del predicado hacia la fuente o el *partition pruning*, y provoca comparaciones inesperadas (por ejemplo, ceros a la izquierda). Declara los tipos de las claves explícitamente antes del cruce.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 9.1 (Redondeo de Punto Flotante):

Demuestra mediante un script simple en PySpark por qué `DoubleType` produce errores de redondeo en operaciones contables acumuladas frente a `DecimalType(18,2)`.

Solución:

```python
from pyspark.sql.functions import lit, col

df_test = spark.range(1).select(
    (lit(0.1) + lit(0.2)).alias("suma_double"),
    (lit(0.1).cast("decimal(10,2)") +
lit(0.2).cast("decimal(10,2)")).alias("suma_decimal")
)
df_test.show()
# suma_double: 0.30000000000000004  <-- Error de IEEE 754
# suma_decimal: 0.30                <-- Precisión exacta financiera
```

9.4 Resumen y Puntos Clave

- **Contrato de rendimiento:** El tipado explícito optimiza el footprint de memoria en Tungsten y evita fallos en los ejecutores.
- **Finanzas con Decimal:** Nunca uses `Float` o `Double` para transacciones monetarias; usa `DecimalType(precision, scale)`.
- **Casting defensivo:** Usa `try_cast` y valida los nulos resultantes; recuerda que en Spark 4.0 un cast inválido lanza error por defecto (ANSI) mientras que en 3.x devuelve `NULL`.

Lecturas Recomendadas

1. **Goldberg, D. (1991).** *What Every Computer Scientist Should Know About Floating-Point Arithmetic.* ACM Computing Surveys.

2. **Databricks Engineering Blog:** *Deep Dive into Project Tungsten: Bringing Spark Closer to Bare Metal.*

PARTE IV

Transformación con la API DataFrame

Capítulo 10 · Filtrado y Expresiones Condicionales NÚCLEO

Capítulo 11 · Transformaciones de Columnas y Operaciones con Esquemas NÚCLEO

Capítulo 12 · Joins Distribuidos y Estrategias de Cruce NÚCLEO

CAPÍTULO 10

Filtrado y Expresiones Condicionales

NÚCLEO

> *"Filtrar temprano en el flujo de trabajo es la optimización número uno en computación distribuida: cada registro descartado antes de un shuffle o un join representa megabytes de datos que no saturarán las tarjetas de red ni la memoria RAM de los nodos ejecutores."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar la sintaxis idiomática de `filter()` y `where()` mediante objetos de columna (`col`).
- Aplicar operadores lógicos a nivel de bits (`&`, `|`, `~`) y estructurar el encapsulado estricto con paréntesis.
- Construir lógica condicional de negocio avanzada con `when().otherwise()`, `isin()` y verificación de nulos.
- Explicar el funcionamiento interno de **Predicate Pushdown** en el optimizador Catalyst sobre capas de almacenamiento Parquet y Delta Lake.

Definición 10.1 (El Colador Inteligente en la Minería de Oro) Imagina que estás buscando oro en un río caudaloso.

No tiene ningún sentido transportar toneladas de arena, piedras y lodo río abajo en camiones hasta la refinería para recién allí separar las pepitas de oro. Lo inteligente y económico es colocar una criba o colador de alta precisión justo en la orilla del río: descartas el 95% del material estéril de inmediato y solo transportas las partículas valiosas.

En PySpark, filtrar es ese colador en la orilla: previene que millones de filas inútiles viajen por la red o consuman memoria en operaciones pesadas de agregación y cruce.

10.1 Fundamentos y Sintaxis Idiomática

En el ecosistema de PySpark, `filter` es una transformación de estrecha dependencia (*Narrow Transformation*) que evalúa registros basándose en una condición lógica booleana. Dado que los DataFrames son **inmutables**, aplicar un filtro no modifica la colección original, sino que devuelve una nueva abstracción de DataFrame con los registros supervivientes.

En la API disponemos de dos métodos formalmente equivalentes: `filter()` y `where()`. Ambos compilan al mismo operador lógico relacional:

```python
from pyspark.sql.functions import col

# Ambas sentencias son 100% intercambiables
df_activos = df.filter(col("estado_cuenta") == "ACTIVO")
df_activos = df.where(col("estado_cuenta") == "ACTIVO")
```

Operadores a Nivel de Bits y Precedencia

A diferencia de Python tradicional donde se utilizan las palabras reservadas `and`, `or` y `not`, en PySpark **es obligatorio utilizar los operadores a nivel de bits (`&`, `|`, `~`) y envolver cada condición individual entre paréntesis**:

```python
# CORRECTO: Paréntesis obligatorios en cada condición individual
df_colombia_vip = df.filter(
    (col("edad") >= 18) &
    (col("pais") == "Colombia") &
    (col("saldo") > 1000000)
)

# Negación lógica con la tilde de inversión ~
df_cuentas_limpias = df.filter(~col("cuenta_bloqueada"))
```

Si omites los paréntesis, Python evalúa el operador `&` antes que las comparaciones (`>`, `==`) y PySpark lanza `ValueError: Cannot convert column into bool: please use '&' for 'and', '|' for 'or', '~' for 'not' when building DataFrame boolean expressions`. El mismo `ValueError` aparece si usas `and`, `or` o `not` con columnas.

10.2 Profundización Técnica: La Magia de Predicate Pushdown

Cuando aplicas un `filter()` sobre un DataFrame que lee desde archivos Parquet, Delta Lake o tablas particionadas, el optimizador **Catalyst** no lee todas las filas a memoria RAM para filtrarlas después.

En su lugar, **empuja el filtro hasta el lector de la fuente de datos** (*Predicate Pushdown*):

- Spark lee el pie de página (footer) de cada archivo Parquet.
- Compara tu condición contra las estadísticas `[min, max]` de cada Row Group.
- Si estás buscando `edad >= 18` y un Row Group indica `max(edad) = 15`, Spark **descarta el bloque entero sin leer ni descomprimir sus datos**. El ahorro real depende de la selectividad del filtro y de cuán ordenados estén físicamente los datos: con datos ordenados o particionados por esa columna puede ser enorme; con datos aleatorios, casi nulo.

> **Consideraciones de Producción 10.1: Condicionales de Negocio Complejos** Patrones idiomáticos para clasificación con when().otherwise(), filtrado por listas dinámicas con isin() y saneamiento de nulos:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when

spark = SparkSession.builder \
    .appName("Capitulo10-FiltradoCondicionales") \
    .master("local[*]") \
    .getOrCreate()

df = spark.read.parquet("data/clientes.parquet")

# 1. Clasificación categórica basada en múltiples condiciones (CASE WHEN)
df_segmentado = df.withColumn(
    "nivel_riesgo",
    when(col("puntaje_credito") >= 750, "BAJO")
    .when((col("puntaje_credito") >= 600) & (col("puntaje_credito") < 750),
"MEDIO")
    .otherwise("ALTO")
)

# 2. Filtrado eficiente por colecciones con isin()
paises_foco = ["Colombia", "México", "Chile", "Perú"]
df_region = df_segmentado.filter(col("pais_origen").isin(paises_foco))

# 3. Control estricto de nulos y cadenas vacías
df_limpio = df_region.filter(
    col("email").isNotNull() &
    (col("email") != "") &
    (col("estado_cuenta") == "ACTIVO")
)

df_limpio.show(5)
```

10.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Dejar para el final un filtro que podía aplicarse antes

Catalyst reordena muchos casos por ti: un filtro que depende solo de la **clave de agrupación**, como `df.groupBy("pais").agg(F.sum("monto").alias("total")).filter(F.col("pais") == "CO")`, se empuja automáticamente por debajo de la agregación (véase el Ejercicio 13.2) y no provoca un shuffle de todos los países. En cambio, no puede reordenar un filtro que depende de un **resultado agregado** (`.filter(F.col("total") > 1000)`, el equivalente de `HAVING`) ni atravesar UDFs no deterministas. Escribir el filtro de fila lo más temprano posible, `df.filter(F.col("pais") == "CO").groupBy("pais").agg(...)`, hace explícita la intención y no depende de que el optimizador pueda reordenarlo.

Anti-patrón 2: Comparaciones de igualdad con Nulos mediante ==

En SQL y PySpark, `col("x") == None` o `col("x") == "null"` no evalúa como True ante valores nulos (evalúa como Unknown/Null). Utiliza siempre los métodos canónicos `.isNull()` o `.isNotNull()`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 10.1 (Optimización de Predicado):

Explica qué diferencia de ejecución física existe entre:

`df.filter((col("a") > 10) & (col("b") == "X"))` y

`df.filter(col("a") > 10).filter(col("b") == "X")`.

Solución: Respuesta: En tiempo de ejecución física **son exactamente idénticos**. El optimizador Catalyst combina automáticamente múltiples filtros encadenados sucesivos en una única expresión de conjunción lógica booleana (*Filter Pushdown / Combining Operators*), por lo que puedes encadenar llamadas a `.filter()` para mayor legibilidad sin ninguna penalización de rendimiento.

10.4 Resumen y Puntos Clave

- **Criba temprana:** Filtrar al borde de la ingesta es la técnica más rentable para ahorrar costos de cómputo y red en la nube.

- **Operadores bitwise y paréntesis:** En PySpark siempre usa `&`, `|`, `~` con paréntesis en cada condición.
- **Predicate Pushdown:** Catalyst descarga el filtrado a la capa de archivos Parquet/Delta para saltarse la lectura de datos irrelevantes.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide.* O'Reilly Media. (Capítulo 5: Basic Structured Operations).
2. **Databricks Documentation:** *Data Skipping for Delta Lake* (`https://docs.databricks.com/delta/data-skipping.html`).

CAPÍTULO 11

Transformaciones de Columnas y Operaciones con Esquemas NÚCLEO

> *"El módulo pyspark.sql.functions es la caja de herramientas estándar del Data Engineer: funciones vectorizadas de C++ y Java diseñadas para operar a la velocidad de la luz en paralelo sin recurrir a lentas UDFs de Python."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Dominar `withColumn()` y `withColumnRenamed()` para generar nuevas columnas calculadas respetando la inmutabilidad de DataFrames.
- Aplicar funciones estándar de manipulación de texto (`upper`, `trim`, `concat`), fechas (`to_date`, `datediff`) y colecciones (`split`, `explode`).
- Conectar la generación de columnas con la evaluación perezosa y el motor Catalyst.
- Comprender la penalización de rendimiento de las UDFs de Python tradicionales y cómo evitarlas en producción.

Definición 11.1 (La Caja de Herramientas del Maestro Artesano) Imagina que eres un cirujano o un relojero de alta precisión: necesitas herramientas específicas para cada maniobra: pinzas para extraer, bisturí para cortar, lupas para enfocar.

En PySpark, el módulo pyspark.sql.functions es precisamente esa caja de herramientas estándar: cientos de funciones optimizadas, implementadas en Scala/Java y ejecutadas dentro de la JVM del ejecutor, diseñadas para operar sobre millones de registros en paralelo sin tener que reinventar la rueda con bucles lentos de Python.

11.1 Creación y Renombrado de Columnas

Los métodos `withColumn` y `withColumnRenamed` nos ayudan a generar nuevas columnas calculadas o alterar esquemas en un DataFrame:

- Cada vez que los llamamos devuelven una **nueva instancia de DataFrame**, preservando la inmutabilidad y añadiendo un paso al grafo de ejecución perezosa (**DAG**).
- `withColumn(colName, colExpr)`: Añade una nueva columna calculada o reemplaza una existente si tienen el mismo nombre.
- `withColumnRenamed(existingName, newName)`: Simplemente renombra una columna sin alterar sus datos ni añadir lógica computacional.

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, year, current_date

spark = SparkSession.builder.appName("Capitulo11-Transformaciones").getOrCreate()
df = spark.read.parquet("data/usuarios.parquet")

# 1. Crear columna con valor constante literal (lit)
df_dummy = df.withColumn("dummy_col", lit(1))

# 2. Calcular mayoría de edad a partir del año de nacimiento
df_evaluado = df.withColumn(
    "mayor_edad",
    (year(current_date()) - col("anio_nacimiento")) >= 18
)

# 3. Renombrar una columna para normalizar el esquema (el nuevo nombre debe
seguir
#    describiendo lo que contiene: aquí sigue siendo un año, no una fecha)
df_renombrado = df_evaluado.withColumnRenamed("anio_nacimiento",
"nacimiento_anio")
df_renombrado.show(5)
```

11.2 Profundización Técnica: UDFs de Python, Vectorización con Arrow y Funciones de Orden Superior

Muchos desarrolladores que provienen de Python tradicional recurren a `@udf` para cualquier transformación personalizada. **¡Alerta de Rendimiento!** Cuando se ejecuta una UDF estándar de Python:

1. La JVM empaqueta los registros en lotes serializados y los transmite a través de un socket TCP local hacia un proceso daemon de Python (`pyspark.daemon`).

2. El proceso de Python deserializa los datos y evalúa la función fila a fila, quedando completamente fuera de las optimizaciones de Catalyst y Tungsten.
3. El resultado vuelve a serializarse y cruza nuevamente el socket hacia la JVM, multiplicando el consumo de CPU y memoria.

La Alternativa Vectorizada: Pandas UDFs con Apache Arrow

Cuando es imprescindible aplicar código de Python (por ejemplo, inferencia de modelos con Scikit-Learn o transformaciones con SciPy), se deben utilizar **Vectorized Pandas UDFs** (`@pandas_udf`), las cuales transfieren bloques de datos columnares directamente en memoria compartida mediante **Apache Arrow** con serialización de costo prácticamente nulo:

PYTHON

```python
import pandas as pd
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType

@pandas_udf(DoubleType())
def normalizar_monto_arrow(serie_monto: pd.Series) -> pd.Series:
    # Operación vectorizada sobre arrays de NumPy / Pandas en C/SIMD
    return (serie_monto - serie_monto.mean()) / (serie_monto.std() + 1e-9)

df_normalizado = df.withColumn("monto_std",
normalizar_monto_arrow(col("monto_usd")))
```

> **Consideraciones de Producción 11.1: Manipulación Avanzada de Cadenas, Fechas y Arrays**
> Catálogo de transformaciones estandarizadas en producción con el alias canónico `F`:

```python
from pyspark.sql import functions as F
from pyspark.sql.functions import col

# 1. Normalización de cadenas de texto
df_texto = df.withColumn("nombre_limpio", F.upper(F.trim(col("nombre_crudo")))) \
             .withColumn("codigo_completo", F.concat(col("prefijo"), F.lit("-"),
col("folio")))

# 2. Manejo riguroso de fechas
df_tiempo = df_texto.withColumn("fecha_evento", F.to_date(col("fecha_texto"),
"yyyy-MM-dd")) \
                    .withColumn("dias_antiguedad", F.datediff(F.current_date(),
col("fecha_evento")))

# 3. Desanidación de arrays con explode_outer() para preservar arrays vacíos o
nulos
df_tags = df_tiempo.withColumn("etiquetas_array", F.split(col("etiquetas_str"),
","))
df_desanidado = df_tags.withColumn("etiqueta_individual",
F.explode_outer(col("etiquetas_array")))

df_desanidado.show(5)
```

11.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Encadenar docenas de `withColumn` sucesivos
Llamar a `.withColumn()` 50 veces seguidas para agregar 50 columnas crea un árbol de Catalyst con 50 operadores de proyección anidados, ralentizando severamente la fase de análisis del Driver. Para transformaciones masivas, utilice `.withColumns({...})` con un diccionario de expresiones o una única sentencia `.select()` que proyecte todas las columnas en un solo paso.

Anti-patrón 2: Usar literales escalares sin `F.lit()`
Si pasa un valor escalar como segundo argumento (`df.withColumn("tipo", "ACTIVO")`), Spark arrojará una excepción porque espera un objeto de tipo `Column`. Envuelva siempre los escalares con `F.lit("ACTIVO")`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 11.1 (Cálculo sobre Arrays: `explode` frente a Funciones de Orden Superior):
Si un DataFrame tiene una columna `historial_pagos` de tipo `ArrayType(DoubleType())`, escriba el código PySpark para calcular el pago promedio de cada cliente y compare la solución tradicional con `explode` frente al uso de **Higher-Order Functions** (`aggregate`).

Solución 11.1:

1. *Solución tradicional con `explode` (y sus desventajas):*

```python
# Riesgo: explode() descarta filas si el array es vacío o NULL (requiere explode_outer).
# Además, groupBy("cliente_id") desencadena un SHUFFLE masivo innecesario por red.
df_con_shuffle = df.select(
    col("cliente_id"),
    F.explode_outer(col("historial_pagos")).alias("pago")
).groupBy("cliente_id").agg(F.avg("pago").alias("pago_promedio"))
```

2. *Solución óptima con Higher-Order Functions (Cero Shuffle y preserva registros):*

```python
# Con aggregate(), el cálculo se realiza en memoria fila por fila dentro de cada partición,
# eliminando el shuffle y preservando arrays vacíos o nulos mediante condiciones seguras:
df_optimo = df.withColumn(
    "pago_promedio",
    F.when(
        F.size(col("historial_pagos")) > 0,
        F.expr("aggregate(historial_pagos, 0.0D, (acc, x) -> acc + x) / size(historial_pagos)")
    ).otherwise(F.lit(None))
)
```

Ventaja: La solución con `aggregate()` procesa millones de filas en una sola pasada lineal $O(N)$ sin transferir un solo byte por la red del clúster.

11.4 Resumen y Puntos Clave

- **Inmutabilidad y linaje:** `withColumn` y `withColumns` generan nuevos nodos en el plan lógico sin mutar los datos existentes.
- **Vectorización con Arrow:** Las Pandas UDFs (`@pandas_udf`) transfieren bloques columnares a procesos de Python mediante memoria compartida Apache Arrow, superando las limitaciones de serialización de las UDFs estándar.
- **Funciones de orden superior:** Operadores como `aggregate`, `transform` y `filter` permiten procesar arrays directamente en memoria sin necesidad de aplanar con `explode` ni incurrir en shuffles.

Lecturas Recomendadas

1. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide*. O'Reilly Media. Capítulo 6: "Working with Different Types of Data".
2. **Apache Arrow Project (2024).** *In-Memory Analytics and Cross-Language Data Sharing*. Apache Software Foundation (`https://arrow.apache.org/`).
3. **Databricks Engineering (2024).** *Higher-Order Functions in Spark SQL*. Databricks Documentation (`https://docs.databricks.com/sql/language-manual/sql-ref-functions-builtin.html`).

CAPÍTULO 12

Joins Distribuidos y Estrategias de Cruce NÚCLEO

> *"El join distribuido es la prueba de fuego de cualquier arquitectura analítica: un cruce mal planificado satura la red del clúster con terabytes de shuffle y bloqueos por asimetría, mientras que una estrategia de join adecuada transforma consultas de horas en ejecuciones de segundos."*

Objetivos de Aprendizaje

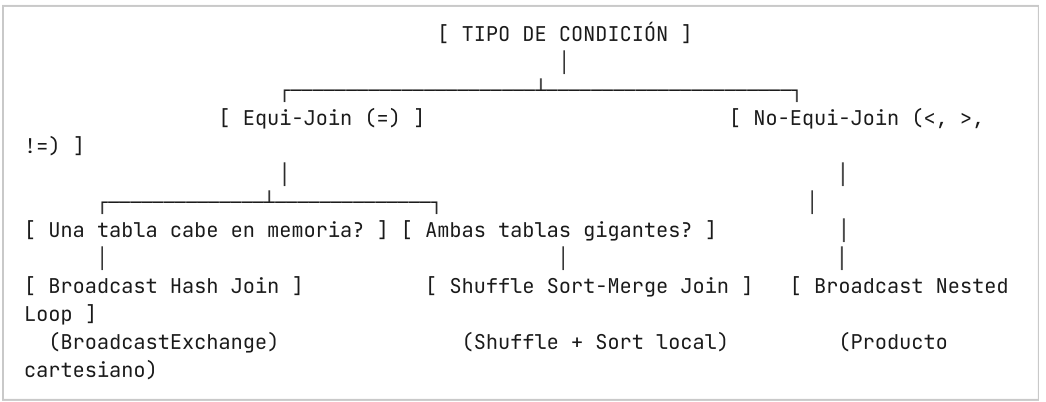
Al finalizar este capítulo, el lector será capaz de:

- Comprender la complejidad algorítmica y física de unir conjuntos de datos distribuidos en una arquitectura sin memoria compartida (*Shared-Nothing*).
- Comparar en profundidad las cuatro estrategias físicas de join en Apache Spark: **Broadcast Hash Join (BHJ)**, **Shuffle Sort-Merge Join (SMJ)**, **Shuffle Hash Join (SHJ)** y **Broadcast Nested Loop Join (BNLJ)**.
- Controlar la selección física de operadores mediante sugerencias del optimizador (*Join Hints*) en SQL y la API DataFrame.
- Diseñar e implementar la técnica de salado de claves (*Salting*) en PySpark para mitigar cuellos de botella severos por sesgo de datos (*Data Skew*).
- Evaluar las limitaciones de las tablas agrupadas en cubos (*Bucket Tables*) en formatos tradicionales frente a formatos abiertos de Lakehouse.
- Auditar planes físicos mediante `.explain()` distinguiendo `ShuffleExchangeExec` de `BroadcastExchangeExec`.

> **Definición 12.1 (Join Distribuido y Costo de Barajado / Shuffle)** En sistemas de computación distribuida compartida (*Shared-Nothing*), un **join distribuido** requiere que las tuplas de ambas tablas que comparten la misma clave de unión (*Join Key*) residan físicamente en la misma partición y en el mismo nodo ejecutor antes de evaluar la condición relacional.
>
> Cuando las tablas no están pre-agrupadas ni particionadas por dicha clave, el motor debe ejecutar una fase masiva de redistribución de red denominada **Shuffle**. Durante el shuffle, cada ejecutor particiona sus datos locales por el valor hash de la clave y transmite fragmentos a través de la red hacia los ejecutores de destino. Esta fase involucra serialización en memoria, transferencias por sockets TCP y volcado temporal en discos locales, constituyendo la operación más intensiva en recursos de todo el pipeline.

12.1 Las Cuatro Estrategias Físicas de Join en Apache Spark

Cuando el optimizador Catalyst compila una operación relacional de cruce (`df_a.join(df_b, "cliente_id")`), evalúa las estadísticas de los datos, el tamaño estimado de las tablas y el tipo de predicado (equi-join con operador `=` o no-equi-join con `<`, `>`, `!=`) para seleccionar una de las siguientes estrategias físicas:



1. Broadcast Hash Join (BHJ - Máximo Rendimiento)

- **Mecanismo:** Si una de las dos tablas tiene un tamaño físico estimado inferior al umbral configurable (`spark.sql.autoBroadcastJoinThreshold`, establecido en 10 MB por defecto), el nodo Driver descarga los datos de la tabla pequeña, construye una tabla hash en memoria y la transmite por red (*broadcast*) a todos los ejecutores del clúster mediante un operador `BroadcastExchangeExec`.
- **Comportamiento en los Workers:** Cada ejecutor mantiene una copia local de la tabla hash de referencia en su memoria RAM. A medida que procesa sus particiones locales de la tabla

masiva, consulta la tabla hash en tiempo constante $O(1)$ por fila.

- **Impacto en Rendimiento: Elimina el `ShuffleExchangeExec` sobre la tabla grande**. No se transmiten terabytes entre ejecutores ni se requiere ordenamiento en disco. La única transferencia de red es el envío unidireccional de la tabla pequeña desde el Driver hacia los Workers.

2. Shuffle Sort-Merge Join (SMJ - El Estándar Robusto)

- **Mecanismo:** Aplicable cuando ambas tablas superan el umbral de broadcast y la condición es una igualdad (*Equi-Join*). Opera en dos fases:
  1. **Fase de Shuffle (Redistribución por Hash):** Spark evalúa el hash de la clave de unión para cada fila y redistribuye los registros de ambas tablas mediante un operador `ShuffleExchangeExec`, garantizando que todas las filas con la misma clave lleguen a la misma partición física.
  2. **Fase de Ordenamiento y Fusión (Sort-Merge):** Cada ejecutor ordena localmente sus particiones por la clave de unión y avanza punteros simultáneos sobre ambas listas para identificar coincidencias en un solo pase lineal con complejidad temporal $O(N + M)$.
- **Impacto en Rendimiento:** Requiere I/O de red masivo y eventual volcado a disco (*Shuffle Spill*), pero es el algoritmo más escalable ante volúmenes arbitrariamente grandes sin desbordar la memoria de los ejecutores.

3. Shuffle Hash Join (SHJ)

- **Mecanismo:** Similar a SMJ en la fase de redistribución por red (`ShuffleExchangeExec`), pero en lugar de ordenar los datos en disco, el ejecutor construye una tabla hash en memoria con la partición de la tabla menor y recorre secuencialmente la partición de la tabla mayor.
- **Requisito y Riesgo:** Cada partición individual de la tabla menor debe caber holgadamente en la memoria heap del ejecutor; de lo contrario, el proceso colapsa con un error de memoria agotada (*OutOfMemoryError*). Está desactivado por omisión en favor de SMJ (`spark.sql.join.preferSortMergeJoin = true`).

4. Broadcast Nested Loop Join (BNLJ)

- **Mecanismo:** Se utiliza cuando el cruce no contiene condiciones de igualdad (por ejemplo, `df_a.monto > df_b.limite`) o cuando se ejecuta un producto cartesiano (`crossJoin`). Transmite una tabla y evalúa bucles anidados para cada registro. Su complejidad temporal es cuadrática $O(N \times M)$, por lo que debe evitarse rigurosamente en producción salvo en tablas de muy baja cardinalidad.

12.2 Join Hints en PySpark y Spark SQL

Cuando el optimizador Catalyst carece de estadísticas precisas o cuando el ingeniero conoce la volumetría de antemano, es posible forzar la estrategia física deseada mediante **Join Hints**:

Sintaxis en la API DataFrame de PySpark

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast, col

spark = SparkSession.builder \
    .appName("Capitulo12-JoinsDistribuidos") \
    .getOrCreate()

df_ventas = spark.read.parquet("lakehouse/silver/ventas/")
df_paises = spark.read.parquet("lakehouse/silver/dim_paises/")

# 1. Sugerencia de Broadcast mediante función nativa o método hint()
df_enriquecido_bhj = df_ventas.join(
    broadcast(df_paises),
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)

# 2. Forzar Shuffle Sort-Merge Join explícitamente
df_enriquecido_smj = df_ventas.hint("merge").join(
    df_paises,
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)

# 3. Forzar Shuffle Hash Join
df_enriquecido_shj = df_ventas.hint("shuffle_hash").join(
    df_paises,
    df_ventas["codigo_pais"] == df_paises["codigo_pais"],
    "inner"
)
```

Sintaxis en Spark SQL Declarativo

En sentencias SQL puras, las sugerencias se declaran dentro de comentarios de bloque especiales `/*+ ... */`:

SQL

```sql
-- 1. Forzar Broadcast Hash Join
SELECT /*+ BROADCAST(p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;
```

```
-- 2. Forzar Shuffle Sort-Merge Join
SELECT /*+ MERGE(v, p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;

-- 3. Forzar Shuffle Hash Join
SELECT /*+ SHUFFLE_HASH(p) */
    v.transaccion_id, v.monto, p.nombre_pais
FROM ventas v
JOIN dim_paises p ON v.codigo_pais = p.codigo_pais;

-- 4. Forzar Broadcast Nested Loop Join (para no-equi-joins con tabla derecha pequeña)
SELECT /*+ BROADCAST(p) */
    v.transaccion_id, v.monto, p.rango_min
FROM ventas v
JOIN rangos_comision p ON v.monto >= p.rango_min;

-- 5. Forzar Cartesian Product con Shuffle Replicate (para tablas grandes sin clave de igualdad)
SELECT /*+ SHUFFLE_REPLICATE_NL(p) */
    v.transaccion_id, v.monto, p.rango_min
FROM ventas v
JOIN rangos_comision p ON v.monto >= p.rango_min;
```

Inspección del Plan Físico: `ShuffleExchangeExec` vs. `BroadcastExchangeExec`

Al inspeccionar el plan con `df_enriquecido_bhj.explain(mode="formatted")`, observe los operadores generados:

```
*(2) BroadcastHashJoin [codigo_pais#12], [codigo_pais#45], Inner, BuildRight
:- *(2) Project [transaccion_id#10, monto#11, codigo_pais#12]
:  +- *(2) Filter isnotnull(codigo_pais#12)
:     +- *(2) Scan parquet [transaccion_id#10,monto#11,codigo_pais#12]
+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]))
   +- *(1) Project [codigo_pais#45, nombre_pais#46]
      +- *(1) Filter isnotnull(codigo_pais#45)
         +- *(1) Scan parquet [codigo_pais#45,nombre_pais#46]
```

Nótese que `BroadcastExchange` recopila la tabla derecha en el Driver y la distribuye a los ejecutores, mientras que la tabla izquierda (ventas) se escanea directamente en el Stage 2 sin ningún `ShuffleExchangeExec`.

12.3 Mitigación de Asimetría de Datos (Data Skew) en Uniones

El fenómeno de **Data Skew** se presenta cuando una porción desproporcionada de las filas comparte la misma clave de unión (por ejemplo, transacciones con cliente nulo o un cliente corporativo mayoritario). En un Sort-Merge Join, esa clave hash única es asignada a una sola

partición de shuffle, sobrecargando a un ejecutor mientras los demás permanecen ociosos:

```
[ Partición 0: 50.000 filas ] ————————> Ejecutor 1 (Termina en 2 seg)
[ Partición 1: 50.000 filas ] ————————> Ejecutor 2 (Termina en 2 seg)
[ Partición 2: 80.000.000 filas (SKEW) ] ——> Ejecutor 3 (Tarda 45 min o falla por
OOM)
```

Solución 1: Adaptive Query Execution (AQE Skew Join)

Desde Apache Spark 3.0, el optimizador adaptativo detecta en tiempo de ejecución las particiones asimétricas tras la fase de shuffle y las divide automáticamente en subtareas más pequeñas:

PYTHON

```python
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
"64MB")
```

Solución 2: Técnica de Salting (Salado de Claves) para Cruces

Cuando el sesgo es extremo o AQE no logra balancear la carga de forma óptima, se aplica **Salting sobre el Join**:

1. A la tabla grande y sesgada se le añade una clave aleatoria (*salt*) entre 0 y $K - 1$.
2. La tabla de búsqueda o dimensión se replica multiplicando cada una de sus filas por los mismos K factores mediante una función de explosión (`explode`).
3. El join se ejecuta sobre la clave compuesta `(clave_original, salt)`, distribuyendo las filas asimétricas de manera uniforme entre K particiones físicas distintas.

PYTHON

```python
from pyspark.sql import functions as F

K = 8  # Factor de dispersión (salting)

# 1. Tabla grande asimétrica (df_ventas con millones de compras del cliente 'C-000')
df_ventas_salted = df_ventas.withColumn(
    "salt",
    F.floor(F.rand() * K).cast("int")
)

# 2. Tabla de referencia (df_clientes): replicar cada registro K veces
df_clientes_replicated = df_clientes.withColumn(
    "salt_array",
    F.array([F.lit(i) for i in range(K)])
).withColumn(
    "salt",
    F.explode("salt_array")
).drop("salt_array")

# 3. Join libre de sesgo sobre la clave compuesta
df_ventas_balanceadas = df_ventas_salted.join(
    df_clientes_replicated,
    on=["cliente_id", "salt"],
    how="inner"
).drop("salt")
```

Este procedimiento garantiza que la clave sesgada se disperse equitativamente entre K tareas de shuffle diferentes, eliminando por completo el cuello de botella de la tarea rezagada (*Straggler Task*).

Consideraciones de Producción 12.1: Bucketed Joins y su Compatibilidad En tablas gestionadas tradicionales de Apache Hive o Apache Parquet (`saveAsTable`), es posible pre-ordenar y pre-agrupar registros mediante `bucketBy`:

```
PYTHON
df.write.bucketBy(32,
"cliente_id").sortBy("cliente_id").saveAsTable("clientes_bucketed")
```

Si dos tablas se unen por la misma clave y ambas comparten idéntico número de cubos, Spark realiza un **Bucket Sort-Merge Join sin Shuffle**.

Aclaración Crítica de Producción: `bucketBy` **no es compatible con rutas directas ni con el formato Delta Lake**. Si intenta invocar `.bucketBy()` al escribir una tabla Delta, el motor ignorará la directiva o arrojará una excepción. En arquitecturas modernas sobre Delta Lake, la técnica estándar para agrupar datos multidimensionalmente y evitar reordenamientos es **Liquid Clustering** (`CLUSTER BY cliente_id`).

12.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Aplicar `broadcast()` a DataFrames que Exceden la Capacidad del Driver
Forzar el broadcast de un DataFrame de gran volumen (por ejemplo, > 4 GB) obliga al nodo Driver a recolectar todas las particiones en su memoria heap local, provocando la interrupción del trabajo por `Driver OutOfMemoryError`. Reserve `broadcast()` estrictamente para tablas de dimensiones que quepan cómodamente en memoria (por lo general < 500 MB).

Anti-patrón 2: Uniones Externas (`FULL OUTER JOIN`) sin Filtrado Previo de Particiones
Ejecutar un `FULL OUTER JOIN` sobre dos tablas transaccionales de escala masiva sin delimitar rangos de fechas produce una explosión de registros nulos en ambos lados y un barajado de datos innecesario que degrada el rendimiento de los ejecutores. Aplique siempre filtros de partición antes del cruce.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 12.1 (Comparativa de Memoria entre BHJ y SMJ):
¿Cuál es la diferencia fundamental en el uso y distribución de memoria entre un Broadcast Hash Join y un Shuffle Sort-Merge Join?

Solución 12.1:

- En **Broadcast Hash Join (BHJ)**: El nodo Driver debe disponer de suficiente memoria heap para ensamblar la tabla completa y serializarla. Posteriormente, cada nodo ejecutor consume memoria adicional en su espacio de almacenamiento/ejecución para albergar la copia íntegra de la tabla hash. La tabla grande se procesa en streaming partición por partición sin requerir memoria extra para ordenamiento ni buffers de red de shuffle.
- En **Shuffle Sort-Merge Join (SMJ)**: El nodo Driver no almacena ningún dato de las tablas. La memoria se consume de manera distribuida en los ejecutores dentro de la región de memoria de ejecución (`Execution Memory`), donde se gestionan los buffers de red para el intercambio de shuffle y las estructuras de ordenamiento en memoria (*AppendOnlyMap* / *Sorter*). Si la memoria de ejecución se satura durante la fase de ordenamiento, los registros se vuelcan a disco local (*Spill*), garantizando estabilidad a expensas de mayor latencia de I/O.

Ejercicio 12.2 (Salting sobre Operaciones de Cruce):
Explique por qué aplicar salting mediante un número aleatorio a la tabla sesgada obliga a replicar las filas de la tabla de referencia mediante `explode()`.

Solución 12.2: Si la tabla grande contiene 10 millones de registros con la clave `cliente_id = 'C-000'` y a cada registro se le asigna un valor de `salt` aleatorio entre 0 y 3, existirán 4 combinaciones posibles de clave compuesta: `('C-000', 0)`, `('C-000', 1)`, `('C-000', 2)` y `('C-000', 3)`. Si la tabla de clientes únicamente tuviera una fila con `cliente_id = 'C-000'`, solo coincidiría con los registros de ventas cuyo salt generado coincidiera con ese único registro, perdiendo el 75% de las transacciones legítimas. Al replicar la fila del cliente en 4 copias idénticas con los salts 0, 1, 2 y 3 mediante `explode()`, se garantiza la semántica relacional completa: cada transacción encontrará su contraparte correspondiente, mientras que la carga de trabajo se reparte equitativamente en 4 particiones físicas de shuffle distintas.

Ejercicio 12.3 (Restricciones de Broadcast en Operaciones `LEFT JOIN`):
En una consulta con un join de tipo `LEFT JOIN` (`df_izq.join(df_der, ..., "left")`), ¿cuál de las dos tablas puede ser transmitida mediante broadcast sin alterar la semántica relacional del resultado? Justifique la respuesta a nivel de planificación física.

Solución 12.3: En un `LEFT JOIN`, **únicamente la tabla derecha (la tabla no preservada) puede ser transmitida mediante broadcast**. *Justificación física:* En la semántica de `LEFT JOIN`, todos los registros de la tabla izquierda deben preservarse obligatoriamente en el resultado final, existan o no coincidencias en la derecha. Si se intentara hacer broadcast de la tabla izquierda, los ejecutores que procesan fragmentos de la tabla derecha no podrían saber de manera independiente y sin coordinación global qué filas de la tabla izquierda no tuvieron coincidencias, lo que obligaría a un intercambio de datos adicional. En cambio, al transmitir la tabla derecha como tabla hash a todos los ejecutores, cada tarea puede recorrer de forma autónoma su partición local de la tabla

izquierda: si la clave existe en la tabla hash derecha, emite la fila unida; si no existe, emite la fila izquierda con valores `NULL` en los campos de la derecha, sin necesidad de transferir ningún dato de la tabla izquierda por la red.

12.5 Resumen y Puntos Clave

- **Criterio de selección:** El optimizador Catalyst elige Broadcast Hash Join cuando una de las tablas es inferior a `spark.sql.autoBroadcastJoinThreshold` (10 MB). Para tablas masivas con equi-joins, la estrategia predeterminada y más estable es Shuffle Sort-Merge Join.
- **Costos físicos:** BHJ elimina el costo de `ShuffleExchangeExec` en la tabla grande pero introduce `BroadcastExchangeExec` desde el Driver. SMJ introduce shuffles en ambas tablas y ordenamiento local en los Workers.
- **Join Hints:** Permiten forzar estrategias cuando las estadísticas de tabla no están disponibles. Se expresan con `.hint("broadcast")` en DataFrames o `/*+ BROADCAST(t) */` en SQL.
- **Manejo de sesgo (Skew):** Adaptive Query Execution (AQE) divide particiones asimétricas en tiempo de ejecución. Ante casos extremos, el salado de claves (*Salting*) dispersa la clave sesgada entre K particiones mediante números aleatorios y duplicación con `explode`.
- **Bucketing vs. Lakehouse:** `bucketBy` permite eliminar shuffles en tablas tradicionales de Parquet/Hive, pero no opera sobre Delta Lake, donde se sustituye por Liquid Clustering.

Lecturas Recomendadas

1. **Chambers, B., & Zaharia, M. (2018).** *Spark: The Definitive Guide: Big Data Processing Made Simple.* O'Reilly Media. Capítulo 8: "Joins".
2. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 1383-1394.
3. **Apache Spark Documentation (2024).** *Performance Tuning: Join Strategy Hints and Adaptive Query Execution.* Apache Software Foundation (`https://spark.apache.org/docs/latest/sql-performance-tuning.html`).

PARTE V

El motor interno y la optimización en Spark

Capítulo 13 · Evaluación Perezosa (Lazy Evaluation) y el Grafo DAG RECOMENDADO

Capítulo 14 · El Optimizador Catalyst y Spark SQL AVANZADO

Capítulo 15 · Gestión de Memoria RAM y Estrategias de Caché AVANZADO

Capítulo 16 · Particionado, Shuffling y Mitigación de Data Skew AVANZADO

CAPÍTULO 13

Evaluación Perezosa (Lazy Evaluation) y el Grafo DAG RECOMENDADO

> *"Spark es como un chef de alta cocina: planifica la comanda y diseña la receta paso a paso, pero no enciende los fogones ni toca un solo ingrediente hasta que el cliente pide que se sirva el plato."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender por qué Apache Spark no ejecuta transformaciones de forma inmediata y cómo esta estrategia maximiza el rendimiento distribuido.
- Diferenciar formalmente entre operaciones de **Transformación** (narrow y wide) y operaciones de **Acción**.
- Analizar la estructura interna de un **DAG** (*Directed Acyclic Graph*), identificando cómo se segmenta en *Jobs*, *Stages* y *Tasks*.
- Interpretar planes de ejecución lógicos y físicos mediante el método `.explain()`.
- Evitar cuellos de botella de re-cómputo y desbordamiento de memoria (*Driver OOM*) causados por llamadas indebidas a acciones.

Definición 13.1 (Evaluación Perezosa y Diferimiento de Cómputo) La **evaluación perezosa** (*Lazy Evaluation*) es una estrategia computacional en la cual las operaciones de transformación sobre estructuras distribuidas (DataFrames y RDDs) no materializan resultados inmediatos en memoria ni generan tráfico en el clúster. En su lugar, cada operación se registra como un vértice o arista dentro de un grafo de dependencias dirigidas (*lineage graph*).

Esta estrategia otorga al motor un panorama global del pipeline analítico antes de iniciar el procesamiento físico. De este modo, el optimizador puede fusionar transformaciones contiguas en un solo pase de datos (*pipeline execution*), anticipar filtros a la etapa de lectura en disco (*Predicate Pushdown*) y descartar columnas irrelevantes (*Column Pruning*), ejecutando el trabajo exclusivamente cuando una acción explícita lo demanda.

13.1 Fundamentos: ¿Qué es la Evaluación Perezosa?

La **evaluación perezosa** (*Lazy Evaluation*) es una de las características angulares de PySpark al trabajar con DataFrames y RDDs. Cuando invocamos métodos de transformación sobre un DataFrame (como `.filter()`, `.select()` o `.withColumn()`), Spark **no procesa ninguna fila inmediatamente**.

En su lugar, Spark registra cada operación como una instrucción dentro de un plan lógico. Los DataFrames en Spark son **inmutables**: cada transformación aplicada no modifica el DataFrame original, sino que genera una nueva abstracción que añade un nodo al linaje de transformaciones (*data lineage*).

Sin una acción que solicite un resultado tangible, los DataFrames son únicamente descripciones matemáticas de un cómputo futuro, no datos materializados en memoria.



La Dualidad Fundamental: Transformaciones vs. Acciones

Para dominar el cómputo distribuido es imperativo distinguir entre estas dos naturalezas de operaciones:

1. **Transformaciones (Perezosas):**
   - *Propósito:* Producen un nuevo DataFrame a partir de uno existente.
   - *Comportamiento:* No desencadenan cómputo físico ni mueven datos a través de la red.
   - *Ejemplos comunes:* `.filter()`, `.where()`, `.select()`, `.withColumn()`, `.groupBy()`, `.join()`, `.drop()`.
2. **Acciones (Disparadoras):**
   - *Propósito:* Obligan a Spark a ejecutar el plan acumulado para devolver un valor concreto al Driver o persistir el resultado en almacenamiento persistente.
   - *Comportamiento:* Materializan el DAG, compilan el plan físico y dividen el trabajo en ejecutores.
   - *Ejemplos comunes:*
     - *Retorno de datos:* `.count()`, `.first()`, `.take(n)`, `.collect()`.
     - *Inspección en consola:* `.show()`.
     - *Escritura en disco:* `.write.parquet()`, `.write.format("delta").save()`.

13.2 Anatomía del Grafo Acíclico Dirigido (DAG)

Un **DAG** (*Directed Acyclic Graph*) es la estructura formal con la que Spark representa el plan de ejecución de una aplicación:

- **Grafo:** Está compuesto por un conjunto de **nodos** (que representan RDDs o DataFrames intermedios generados por operaciones como `filter`, `map` o `groupBy`) y **aristas** directas (que definen las dependencias y el flujo de los datos).
- **Dirigido:** El flujo de procesamiento se desplaza en un único sentido, desde las fuentes de lectura hacia las salidas.
- **Acíclico:** No contiene ciclos ni bucles cerrados. Esta propiedad garantiza que el flujo de ejecución tenga un inicio y un fin rigurosamente acotados, previniendo bucles infinitos en clústeres distribuidos.

13.3 Profundización Técnica: De la Acción al Hardware (Jobs, Stages y Tasks)

Cuando el Driver Program detecta una acción, el motor desglosa el DAG en una jerarquía física de tres niveles de granularidad:

1. **Job:** Una acción invocada en el código desencadena uno o más *Jobs* en el SparkContext (algunas acciones como `take()`, `first()` o lecturas con cabeceras/inferencia de tipos pueden disparar múltiples Jobs secuenciales según la distribución de datos).
2. **Stages (Etapas):** El planificador descompone el Job en Stages analizando las dependencias del grafo:

- **Dependencias Estrechas (Narrow Dependencies):** Operaciones como `filter()` y `select()` donde cada partición del DataFrame padre es consumida por a lo sumo una partición del DataFrame hijo. Se encadenan en memoria sin tocar disco (*Pipelining*) dentro del mismo Stage.
- **Dependencias Amplias (Wide Dependencies):** Operaciones como `groupBy()` o `join()` donde múltiples particiones hijas dependen de datos dispersos en todas las particiones del padre. Exigen redistribuir datos por la red (**Shuffle**). *Cada frontera de shuffle corta el grafo y crea un nuevo Stage.*

3. **Tasks (Tareas):** Es la unidad mínima de ejecución física. Cada Stage se fragmenta en tantas *Tasks* como particiones existan. Una tarea se envía a un hilo de un **Executor** para procesar su partición de datos.

> **Consideraciones de Producción 13.1: Inspección de Linaje y Plan Físico** En entornos productivos, nunca adivines qué hará Spark: utiliza .explain(extended=True) para verificar si Catalyst logró aplicar optimizaciones como Predicate Pushdown (empujar filtros a la fuente de almacenamiento) y Column Pruning (leer únicamente las columnas requeridas).

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, year, current_date

# 1. Inicialización de la sesión de cómputo
spark = SparkSession.builder \
    .appName("Capitulo13-LazyEvaluation-DAG") \
    .master("local[*]") \
    .getOrCreate()

# 2. Carga diferida (Transformación)
# Spark solo lee el esquema/metadatos; no carga filas a memoria
df_clientes = spark.read.parquet("data/clientes.parquet")

# 3. Encadenamiento de transformaciones estrechas (Narrow Transformations)
# Construyen el grafo lógico sin cómputo en ejecutores
df_mayores = df_clientes \
    .withColumn("edad_aprox", year(current_date()) - col("birth_year")) \
    .filter(col("edad_aprox") >= 18) \
    .select("cliente_id", "nombre", "edad_aprox", "pais")

# 4. Inspección del plan optimizado por Catalyst antes de ejecutar
print("=== PLAN DE EJECUCIÓN LÓGICO Y FÍSICO ===")
df_mayores.explain(True)

# 5. Acción Disparadora (Action)
# En este instante preciso se compila el DAG, se lanzan los Jobs y se computa
total_mayores = df_mayores.count()
print(f"Total de clientes mayores de edad: {total_mayores}")
```

13.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Disparar Acciones en Bucle o Múltiples Veces sin Persistencia

Un error clásico es encadenar varias acciones sobre un DataFrame derivado costoso sin llamar a `.persist()` o `.cache()`:

```python
# ¡PELIGRO EN PRODUCCIÓN!
df_filtrado = df_gigante.filter(col("monto") > 1000).groupBy("region").sum()

# Acción 1: Spark re-lee df_gigante y ejecuta todo el DAG
print("Conteo:", df_filtrado.count())

# Acción 2: Spark vuelve a leer df_gigante desde cero y re-ejecuta todo el DAG
df_filtrado.write.parquet("salida/metricas.parquet")
```

Solución de producción: Si vas a reutilizar un DataFrame en más de una acción, invoca `df_filtrado.persist()` antes de la primera acción y libera la memoria con `.unpersist()` al concluir.

Anti-patrón 2: El Peligro de `.collect()` en Grandes Volúmenes

La acción `collect()` transfiere **todas las particiones distribuidas del clúster** a la memoria RAM del nodo Driver. Si el DataFrame pesa 50 GB y el Driver dispone de 8 GB de RAM, la JVM colapsará instantáneamente con un error `java.lang.OutOfMemoryError: Java heap space`. Utiliza siempre `.take(n)` o escribe directamente a disco.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 13.1 (Diagnóstico de Acciones y Transformaciones):

Dado el siguiente bloque de código, determina cuántos *Jobs* independientes de Spark se enviarán al clúster para su ejecución física:

```python
from pyspark.sql import functions as F

df1 = spark.read.csv("ventas.csv", header=True)
df2 = df1.filter(F.col("precio") > 50)
df3 = df2.withColumn("iva", F.col("precio") * 0.19)
print("Paso intermedio listo")
df4 = df3.groupBy("categoria").count()
print(df4.first())
df4.write.mode("overwrite").parquet("reportes/")
```

Solución 13.1:
Al razonar línea por línea sobre el código, se deduce un mínimo de **3 Jobs** enviados al clúster:

1. **Job 1 (Lectura de metadatos):** `spark.read.csv("ventas.csv", header=True)` no es puramente perezosa. Para leer la cabecera y fijar los nombres de columna en el esquema, Spark

ejecuta un primer job pequeño sobre el inicio del archivo (con `inferSchema=True` lanzaría un job adicional que recorre los datos completos).

2. **Transformaciones intermedias (0 Jobs):** `filter`, `withColumn` y `groupBy(...).count()` son transformaciones perezosas que solo actualizan el grafo DAG sin disparar ningún cálculo en el clúster.
3. **Job 2 (Acción de retorno):** `df4.first()` es una acción disparadora. Materializa el plan físico acumulado y extrae la primera tupla (escaneando particiones mediante `limit(1)`).
4. **Job 3 (Acción de escritura):** `df4.write.parquet(...)` es una acción de persistencia que materializa el DAG completo para procesar y escribir todas las particiones en disco. Como `df4` no fue persistido en caché tras `first()`, Spark recalcula el pipeline completo para este segundo job.

Ejercicio 13.2 (Optimización de Linaje):

Explica por qué un filtro ubicado al final de un script PySpark como `df.groupBy("id").agg(F.sum("monto").alias("total")).filter(F.col("id") == 100)` puede ser reorganizado por el optimizador de Spark para ejecutarse antes del agrupamiento. ¿Qué nombre recibe este mecanismo interno de Catalyst y cuándo **no** es posible aplicarlo?

Solución: Respuesta: Este mecanismo se denomina **Predicate Pushdown** (o filtrado anticipado; en Catalyst lo implementan reglas como `PushDownPredicates`). Como Spark conoce todo el DAG antes de ejecutarlo, Catalyst puede desplazar el predicado lo más cerca posible de la fuente, siempre que el resultado sea equivalente. Aquí lo es porque el filtro depende de `id`, que es la **clave de agrupación**: descartar las filas con `id != 100` antes o después de agrupar produce el mismo resultado, y hacerlo antes reduce las filas que entran al shuffle. **No** es posible si el filtro depende de un resultado agregado (`.filter(F.col("total") > 1000)`, equivalente a `HAVING`), porque ese valor solo existe después de agregar.

13.5 Resumen y Puntos Clave

- **La pereza es eficiencia:** Spark pospone la ejecución material para poder ver el panorama completo del cómputo y optimizarlo de forma holística mediante el motor Catalyst.
- **El DAG es el mapa de ruta:** Representa el linaje de transformaciones inmutables; si un nodo del clúster falla, Spark puede reconstruir exactamente las particiones perdidas re-ejecutando únicamente el linaje afectado (*fault tolerance*).
- **Fronteras de Shuffle:** Las transformaciones estrechas (*narrow*) se ejecutan de corrido en memoria; las transformaciones amplias (*wide*) introducen intercambios en red y delimitan los *Stages*.

- **Controla tus acciones:** Nunca llames a `.collect()` sin límite de filas ni encadenes múltiples acciones sobre el mismo DataFrame sin evaluar si amerita ser persistido en memoria.

Lecturas Recomendadas

1. **Zaharia, M. et al. (2016).** *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM.
2. **Chambers, B. & Zaharia, M. (2018).** *Spark: The Definitive Guide*. O'Reilly Media. (Capítulo 2: A Gentle Introduction to Spark; Capítulo 15: How Spark Runs on a Cluster).
3. **Documentación Oficial de Apache Spark:** *Tuning Spark & Catalyst Optimizer Internals* (`https://spark.apache.org/docs/latest/tuning.html`).

CAPÍTULO 14

El Optimizador Catalyst y Spark SQL

AVANZADO

> *"Catalyst es el cerebro matemático de Spark: toma consultas SQL abstractas y las traduce a través de un árbol de operadores lógicos, aplicando optimización basada en reglas y en costos para generar código Java ultra-eficiente en tiempo de ejecución."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender las cuatro fases del pipeline de optimización de Catalyst: *Analysis*, *Logical Optimization*, *Physical Planning* y *Code Generation (Tungsten)*.
- Distinguir con precisión entre la optimización basada en reglas (*Rule-Based Optimization - RBO*), la selección de operadores por umbrales y el optimizador basado en costos (*Cost-Based Optimization - CBO*).
- Inspeccionar y depurar planes de ejecución mediante `.explain(mode="formatted")`.
- Comprender por qué el CBO y el reordenamiento de joins vienen desactivados por omisión en Spark y cómo habilitarlos con `ANALYZE TABLE`.
- Explicar cómo **Adaptive Query Execution (AQE)** re-optimiza el plan físico en tiempo de ejecución utilizando estadísticas reales de shuffle.

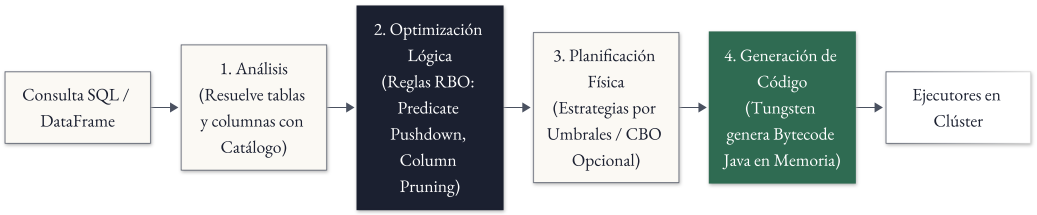
Definición 14.1 (El Compilador de Estrategia Relacional) Imagine a un general militar que recibe directivas en múltiples lenguajes declarativos (SQL, Python, Scala, Java, R).

En lugar de enviar a los soldados a correr a ciegas con cada orden, el general cuenta con Catalyst, un estratega matemático que traduce todas las órdenes a un mapa táctico universal (el árbol sintáctico abstracto o AST). El estratega elimina movimientos redundantes mediante reglas de álgebra relacional, evalúa estrategias físicas viables y compila las órdenes en código binario de máquina ejecutado a máxima velocidad por los nodos del clúster.

14.1 Arquitectura del Motor Spark SQL

Spark SQL es el módulo de Apache Spark para el procesamiento de datos estructurados. A diferencia de la API de bajo nivel de RDDs, Spark SQL proporciona al motor información enriquecida sobre los tipos de datos y las relaciones lógicas de la computación.

El núcleo de esta capacidad reside en **Catalyst**, un optimizador extensible basado en programación funcional en Scala que manipula y transforma árboles de expresiones relacionales:



Las Cuatro Fases de Optimización de Catalyst

1. **Análisis (Analysis):** Comienza con un *Unresolved Logical Plan*. El analizador consulta el catálogo de metadatos (Hive Metastore o Unity Catalog) para resolver identificadores de tablas y columnas, verificar tipos de datos y asignar identificadores internos únicos a cada atributo relacional.
2. **Optimización Lógica (Logical Optimization):** Aplica optimizaciones estándar basadas en reglas (*Rule-Based Optimization - RBO*), transformando el árbol relacional sin alterar su semántica:
   - **Constant Folding:** Evalúa expresiones deterministas en tiempo de compilación (por ejemplo, `1 + 1` se simplifica a `2`).
   - **Predicate Pushdown:** Desplaza filtros relacionales hacia los operadores de escaneo en almacenamiento para descartar filas lo más cerca posible de la fuente.
   - **Projection Pruning:** Descarta columnas que no participan en agregaciones, filtros o proyecciones posteriores.
3. **Planificación Física (Physical Planning):** Traduce el plan lógico optimizado en uno o más planes físicos ejecutables compuestos por operadores de Spark (`SparkPlan`). En esta fase se seleccionan los algoritmos concretos (por ejemplo, `BroadcastHashJoinExec` vs. `SortMergeJoinExec`).
4. **Generación de Código (Code Generation con Tungsten):** Compila el plan físico en bytecode Java altamente eficiente mediante *Whole-Stage Java Code Generation*, colapsando múltiples operadores en un único bucle continuo en memoria y eliminando la sobrecarga de despacho virtual.

14.2 Planificación Física: Umbrales de Tamaño frente a Cost-Based Optimizer (CBO)

Existe una confusión generalizada sobre cómo Catalyst decide las estrategias de cruce en su configuración predeterminada:

La Selección Predeterminada por Umbrales (Threshold-Driven)

Por omisión en Apache Spark, **el optimizador basado en costos (CBO) está desactivado** (`spark.sql.cbo.enabled = false`). La elección entre un Broadcast Hash Join (BHJ) y un Sort-Merge Join (SMJ) **no es realizada por CBO**, sino por una regla heurística del planificador físico basada en umbrales de tamaño en bytes:

- Si las estadísticas de metadatos del catálogo o del escaneo indican que una tabla tiene un tamaño en bytes inferior o igual a `spark.sql.autoBroadcastJoinThreshold` (10 MB por defecto), Catalyst selecciona automáticamente `BroadcastHashJoinExec`.
- Si ambas tablas superan este umbral (o carecen de estimaciones de tamaño en disco), el planificador físico recurre a la estrategia estándar y segura: `SortMergeJoinExec`.

El Optimizador Basado en Costos (CBO) y Reordenamiento de Joins

Para consultas analíticas complejas que involucran múltiples cruces en estrella ($N \geq 3$ tablas), el orden en que se unen las tablas altera drásticamente el volumen de shuffle intermedio. En este escenario interviene el **CBO**:

PYTHON

```python
# Habilitación explícita de CBO y reordenamiento de joins en la SparkSession
spark.conf.set("spark.sql.cbo.enabled", "true")
spark.conf.set("spark.sql.cbo.joinReorder.enabled", "true")
spark.conf.set("spark.sql.cbo.joinReorder.dp.threshold", "12") # Máximo número de tablas para DP
```

Para que CBO opere eficazmente, requiere estadísticas de cardinalidad y distribución de columnas en el metastore, las cuales deben recopilarse manualmente mediante comandos de mantenimiento:

SQL

```sql
-- Recopilación de estadísticas de tamaño y cardinalidad de columnas
ANALYZE TABLE ventas COMPUTE STATISTICS FOR COLUMNS cliente_id, monto;
ANALYZE TABLE clientes COMPUTE STATISTICS FOR COLUMNS cliente_id;
```

Con estas estadísticas, el algoritmo de programación dinámica del CBO calcula el costo estimado de los árboles de unión alternativos y selecciona el orden de ejecución que minimice las tuplas intermedias.

14.3 Inspección de Planes con `.explain()`

Utilice `.explain(mode="formatted")` para auditar cómo Catalyst optimizó la consulta antes de lanzarla en un clúster de producción:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Capitulo14-CatalystOptimizer") \
    .master("local[*]") \
    .getOrCreate()

df_pedidos = spark.read.parquet("data/pedidos.parquet")

# Consulta con filtro y agregación
df_resultado = df_pedidos \
    .select("pedido_id", "cliente_id", "monto_usd", "pais") \
    .filter((col("pais") == "CO") & (col("monto_usd") > 100)) \
    .groupBy("cliente_id") \
    .sum("monto_usd")

# Inspección del plan físico y lógico
df_resultado.explain(mode="formatted")
```

En la salida de `explain`, observe:

- **PushedFilters:** Confirma que el predicado `[EqualTo(pais, 'CO'), GreaterThan(monto_usd, 100.0)]` fue empujado al lector Parquet.
- **ReadSchema:** Muestra que las columnas no utilizadas fueron podadas, leyendo únicamente las requeridas para la computación.

14.4 Adaptive Query Execution (AQE): Optimización Dinámica en Tiempo de Ejecución

Antes de Spark 3.0, Catalyst planificaba las consultas de forma estática antes de comenzar la ejecución. Si los datos reales diferían de las estimaciones del catálogo (por ejemplo, debido a filtros altamente selectivos que reducían el 99% de las filas), el plan estático no podía adaptarse.

Adaptive Query Execution (AQE) resuelve esta limitación re-optimizando el plan físico en tiempo de ejecución en las fronteras de los stages de shuffle:

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = (SparkSession.builder
         .appName("Capitulo14-AQE")
         .master("local[4]")
         .config("spark.sql.shuffle.partitions", "200")
         .getOrCreate())

ventas = spark.range(0, 2_000_000).select(
    (F.col("id") % 50).alias("pais_id"),
    (F.rand(seed=7) * 100).alias("monto"),
)

def correr(aqe: bool):
    spark.conf.set("spark.sql.adaptive.enabled", str(aqe).lower())
    df = ventas.groupBy("pais_id").agg(F.sum("monto").alias("total"))
    df.collect()
    df.explain()

correr(aqe=False)   # El shuffle conserva rígidamente 200 particiones
correr(aqe=True)    # AQE fusiona particiones pequeñas mediante AQEShuffleRead
coalesced
```

Las Tres Capacidades Fundamentales de AQE

1. **Fusión Dinámica de Particiones de Shuffle (`coalescePartitions`):** Si tras una operación `groupBy` el volumen de datos en muchas particiones es mínimo, AQE las consolida dinámicamente en menos particiones para evitar la sobrecarga de programar cientos de tareas triviales.
2. **Conversión Dinámica a Broadcast Join:** Si tras aplicar filtros selectivos el tamaño real de una tabla en un join resulta inferior al umbral de broadcast, AQE transmuta un `SortMergeJoin` planificado inicialmente a un `BroadcastHashJoin`, eliminando el shuffle de la tabla contraria.
3. **Manejo Dinámico de Particiones Sesgadas (`skewJoin`):** Si una partición de shuffle es desproporcionadamente más grande que la mediana, AQE la divide automáticamente en múltiples subtareas paralelas.

14.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Bloquear Catalyst con UDFs Opacas de Python Puro
Cuando se utiliza una función `@udf` de Python en un filtro (`df.filter(mi_udf(col("x")))`), Catalyst no puede inspeccionar su contenido lógico. Como resultado, ese predicado no puede ser empujado a la capa de almacenamiento, obligando al motor a transferir todas las filas a la JVM y luego a los procesos de Python para evaluarlas una a una. Combine siempre filtros con funciones nativas de PySpark para reducir el volumen de datos antes de aplicar transformaciones complejas.

Anti-patrón 2: Asumir que el Reordenamiento de Joins Opera sin Configuración
Creer que Spark optimizará automáticamente el orden de unión de múltiples tablas gigantescas sin haber ejecutado `ANALYZE TABLE` ni habilitado `spark.sql.cbo.joinReorder.enabled = true`. Sin estas directivas, Catalyst ejecuta los joins en el orden sintáctico estricto en que fueron declarados en el código.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 14.1 (Operadores de Whole-Stage Code Generation):
Al ejecutar `df.explain()`, observa el operador `*(1) FileScan parquet [monto_usd#12, pais#13] PushedFilters: [IsNotNull(pais), EqualTo(pais,CO)]`. ¿Qué denota el asterisco `*(1)` al inicio del operador?

Solución 14.1:
El asterisco `*` indica que el operador físico forma parte de una fase de **Whole-Stage Code Generation** gobernada por el motor Tungsten. El número `(1)` representa el identificador del subárbol de operadores fusionados que Catalyst compiló en una única función Java en memoria, ejecutando el procesamiento en un único bucle continuo que aprovecha los registros de la CPU y minimiza el uso de la pila de llamadas.

Ejercicio 14.2 (Optimización Estática frente a Adaptive Query Execution):
¿Por qué una consulta que une una tabla de 100 GB con una tabla de 50 GB puede ejecutarse físicamente como un `BroadcastHashJoin` si AQE está activo, aun cuando ambas tablas superen ampliamente el umbral de 10 MB al inicio?

Solución 14.2:
En la planificación estática inicial, Catalyst estima los tamaños a partir de los metadatos de los archivos en reposo y determina que ambas tablas (100 GB y 50 GB) requieren un `SortMergeJoin`. Sin embargo, si la consulta contiene un filtro altamente restrictivo sobre la tabla de 50 GB (por ejemplo, `WHERE fecha = '2026-10-12' AND sucursal = 'BOG'`), el volumen

real de datos resultante tras el filtrado en la fase de mapeo puede ser de solo 4 MB. Al estar habilitado **AQE** (`spark.sql.adaptive.enabled = true`), el motor recopila los tamaños reales generados en la frontera del stage de shuffle: al detectar que la tabla filtrada tiene únicamente 4 MB (inferior al umbral de 10 MB), AQE re-optimiza el plan físico en caliente y transforma el `SortMergeJoin` en un `BroadcastHashJoin`, suprimiendo la fase de shuffle y ordenamiento de la tabla de 100 GB.

14.6 Resumen y Puntos Clave

- **Pipeline de Catalyst:** Se compone de cuatro fases secuenciales: Análisis, Optimización Lógica (RBO), Planificación Física y Generación de Código (Tungsten).
- **Selección por umbrales:** La elección entre BHJ y SMJ en la configuración estándar responde a `spark.sql.autoBroadcastJoinThreshold`, no a CBO.
- **CBO opcional:** El Cost-Based Optimizer y el reordenamiento de joins requieren activación manual (`spark.sql.cbo.enabled = true`) y estadísticas calculadas mediante `ANALYZE TABLE`.
- **Adaptive Query Execution (AQE):** Representa el estándar moderno de ejecución en Spark, ajustando el número de particiones, dividiendo sesgos de datos y convirtiendo dinámicamente operadores a broadcast en tiempo de ejecución.

Lecturas Recomendadas

1. **Armbrust, M. et al. (2015).** *Spark SQL: Relational Data Processing in Spark.* Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 1383-1394.
2. **Zaharia, M. et al. (2016).** *Apache Spark: A Unified Engine for Big Data Processing.* Communications of the ACM, 59(11), 56-65.
3. **Apache Spark Project Authors (2024).** *SQL Programming Guide: Catalyst Optimizer and Performance Tuning.* Apache Software Foundation (`https://spark.apache.org/docs/latest/sql-performance-tuning.html`).

CAPÍTULO 15

Gestión de Memoria RAM y Estrategias de Caché AVANZADO

> *"La memoria RAM en Spark no es un espacio homogéneo: está rigurosamente dividida entre memoria de almacenamiento para caché y memoria de ejecución para shuffles y joins. Comprender esta frontera dinámica es la diferencia entre un clúster veloz y uno colapsado por OOM y Garbage Collection."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

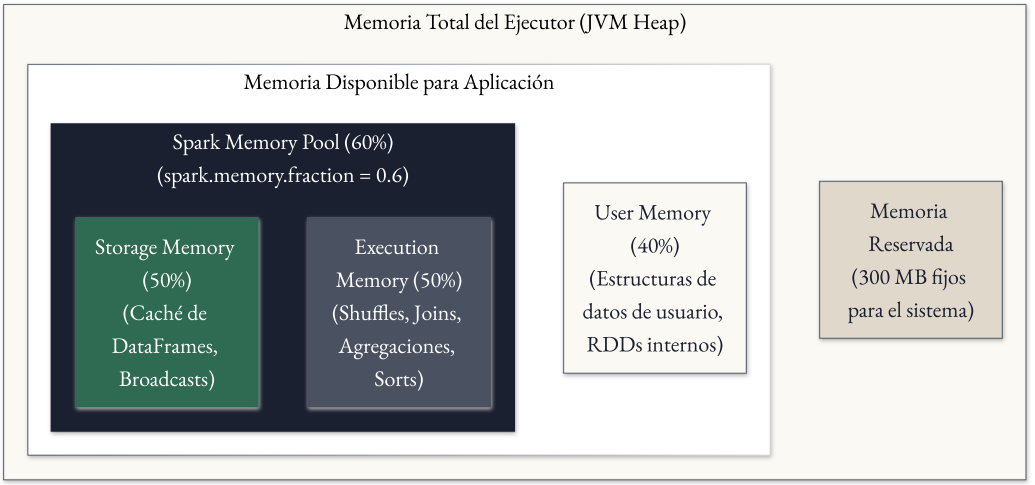
- Comprender la arquitectura de memoria del ejecutor (JVM Heap, Off-Heap, Storage Memory, Execution Memory, User Memory).
- Conocer la frontera dinámica entre *Storage* y *Execution* introducida por el modelo de gestión unificada de memoria de Spark.
- Dominar los niveles de persistencia (`cache()`, `persist()` con `MEMORY_ONLY`, `MEMORY_AND_DISK`, `DISK_ONLY`) y saber cuáles aplican en PySpark.
- Diagnosticar y resolver errores de desbordamiento de memoria (*OutOfMemoryError*) y pausas prolongadas de *Garbage Collection*.

Definición 15.1 (Modelo Unificado de Gestión de Memoria en Spark) A partir de Apache Spark 1.6+, el motor implementa un modelo de asignación unificada de memoria que supera la fragmentación estática tradicional. La memoria heap de cada ejecutor se subdivide dinámicamente entre dos reservas principales: **Execution Memory** (destinada a buffers de barajado, ordenamiento y agregaciones en memoria) y **Storage Memory** (destinada a la persistencia y cacheo de DataFrames y RDDs).

La frontera entre ambas reservas es elástica, pero **asimétrica**: la ejecución puede desalojar bloques en caché hacia disco cuando necesita memoria de cálculo, aunque solo hasta el **umbral protegido** definido por `spark.memory.storageFraction` (50% del pool de Spark por defecto). Por debajo de ese umbral, los bloques cacheados no pueden ser desalojados por la ejecución; en sentido contrario, el almacenamiento nunca puede desalojar memoria de ejecución en uso.

15.1 La Arquitectura de Memoria Unificada de Spark

A partir de Spark 1.6, el motor reemplazó el antiguo modelo estático de memoria por el **Unified Memory Manager**. Cada proceso ejecutor (*Executor JVM*) divide su memoria total en cuatro regiones clave:



La Frontera Dinámica: Storage vs. Execution

1. **Storage Memory (Almacenamiento):** Aloja DataFrames y RDDs cacheados mediante `.cache()` o `.persist()`, así como variables broadcast transmitidas a los nodos.
2. **Execution Memory (Ejecución):** Aloja las estructuras temporales necesarias para computar shuffles, joins, agrupamientos y ordenamientos (*sort buffers*).

3. **El Préstamo Dinámico:**
    - Si la memoria de ejecución no se está utilizando, el almacenamiento puede expandirse para ocupar todo el pool.

Memoria Fuera de la JVM y Procesos de Python (`memoryOverhead`)

En clústeres orquestados por YARN o Kubernetes, el gestor de recursos impone un límite estricto de memoria física por contenedor:

$$\text{Memoria Física del Contenedor} = \text{spark.executor.memory} + \text{spark.executor.memo}$$

- `spark.executor.memoryOverhead`: Es la memoria nativa (fuera del heap de la JVM) reservada para buffers del sistema operativo, memoria off-heap de Tungsten y bibliotecas en C/C++. Por defecto se calcula como $\max(384\ \text{MB}, 0.10 \times \text{spark.executor.memory})$.
- **El Cuello de Botella de PySpark:** Cada ejecutor que procesa transformaciones de Python (como UDFs o Pandas UDFs con Arrow) spawnea subprocesos de Python independientes (`python3` / `pyspark.daemon`). Estos procesos consumen memoria del sistema operativo fuera del heap de la JVM.
- **El Error "Container killed by YARN for exceeding memory limits":** Si los subprocesos de Python acumulan DataFrames o arrays en memoria que superan el margen asignado por `memoryOverhead`, YARN o el OOM Killer de Linux destruyen el contenedor inmediatamente.
- **Solución en Producción:** Incrementar `spark.executor.memoryOverhead` (por ejemplo, al 25% o 30% de la memoria del ejecutor) o limitar la memoria del worker de Python mediante `spark.python.worker.memory` (por defecto 512 MB).

15.2 Profundización Técnica: Niveles de Persistencia (StorageLevel)

El método `.cache()` es un alias abreviado para `.persist(StorageLevel.MEMORY_AND_DISK)` en DataFrames.

> **Regla de Oro: `.cache()` y `.persist()` son Estrictamente Perezosos**
> Invocar `df.cache()` no carga ni un solo byte en la memoria RAM de los ejecutores de forma inmediata. La persistencia es una **instrucción diferida**: Spark registrará que el DataFrame debe guardarse en caché, pero los datos se materializarán en la memoria física únicamente en el instante en que se evalúe la primera acción (`count()`, `show()`, `write()`).

Spark ofrece múltiples niveles de almacenamiento:

- **MEMORY_ONLY:** Guarda los datos como objetos Java deserializados en RAM. Máxima velocidad de lectura, pero si no cabe, las particiones no cacheadas se recalculan desde cero al leerlas.
- **MEMORY_AND_DISK (Recomendado):** Guarda en RAM; si se llena, vuelca el excedente a disco local (*Spill*) sin recalcular el linaje.
- **Variantes `_SER` (`MEMORY_ONLY_SER`, `MEMORY_AND_DISK_SER`):** Almacenan datos serializados en bytes compactos, reduciendo el consumo de RAM a cambio de CPU al deserializar. En PySpark, los DataFrames ya se almacenan internamente en formato binario columnar comprimido gestionado por Tungsten.

> **Consideraciones de Producción 15.1: Caché Estratégica y Liberación de Memoria** Cachear tiene sentido únicamente cuando un DataFrame costoso se va a reutilizar en **múltiples acciones subsecuentes**. Siempre se debe liberar la memoria con `.unpersist()` cuando concluye el bloque de trabajo:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.storagelevel import StorageLevel
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Capitulo15-GestionMemoriaCache") \
    .master("local[*]") \
    .getOrCreate()

# Cargar y aplicar transformaciones complejas
df_pesado = spark.read.parquet("data/eventos_anuales.parquet") \
    .filter(col("tipo_evento").isin(["COMPRA", "REEMBOLSO"])) \
    .groupBy("cliente_id", "categoria") \
    .count()

# 1. Persistir en memoria con desborde a disco (perezoso: no ejecuta I/O aún)
df_pesado.persist(StorageLevel.MEMORY_AND_DISK)

# 2. Acción 1: Conteo total (aquí se materializa la caché en los Workers)
total_filas = df_pesado.count()
print(f"Total agrupado: {total_filas}")

# 3. Acción 2: Exportar a dos destinos distintos reutilizando la caché en RAM
df_pesado.filter(col("count") > 10).write.parquet("salida/clientes_frecuentes/")
df_pesado.filter(col("count") <=
10).write.parquet("salida/clientes_ocasionales/")

# 4. LIBERACIÓN OBLIGATORIA DE MEMORIA EN PRODUCCIÓN
```

```
df_pesado.unpersist()
print("Memoria RAM del clúster liberada exitosamente.")
```

15.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cachear DataFrames que Solo se Usan una Vez
Escribir `df.cache()` y luego invocar una única acción (`df.write.parquet(...)`) es un desperdicio neto de cómputo y memoria: fuerza a Spark a serializar y retener los datos en RAM innecesariamente antes de que el script termine.

Anti-patrón 2: Asumir que `.cache()` es una Acción Inmediata
Creer que llamar a `df.cache()` precalienta la memoria antes de un bloque de código crítico. Como es perezosa, la primera consulta pagará todo el costo de escaneo y cómputo más la sobrecarga adicional de escribir en la memoria de Storage.

Anti-patrón 3: Olvidar `unpersist()` en Pipelines de Larga Duración
En aplicaciones continuas o pipelines diarios, acumular DataFrames cacheados sin despersistir fragmenta la memoria y detona un `java.lang.OutOfMemoryError` en los ejecutores.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 15.1 (Diagnóstico de Spill to Disk):

En el Spark UI observas que una etapa muestra: `Spill (Memory): 12.4 GB` y `Spill (Disk): 3.2 GB`. ¿Qué significa exactamente esta métrica y qué impacto tiene en el tiempo de ejecución?

Solución: Explicación técnica:

Significa que durante un shuffle o join, el pool de Execution Memory del ejecutor se agotó. Spark tuvo que serializar 12.4 GB de datos en RAM y volcarlos como 3.2 GB comprimidos al disco rígido local del Worker Node.

Impacto: Degrada severamente el rendimiento debido al tiempo de E/S de disco. Se soluciona aumentando `spark.sql.shuffle.partitions` (para que cada partición sea más pequeña) o aumentando la memoria RAM por ejecutor (`spark.executor.memory`).

15.4 Resumen y Puntos Clave

- **Memoria dinámica:** Execution y Storage comparten el pool de Spark; la ejecución puede desalojar caché solo hasta el umbral protegido de `spark.memory.storageFraction`.

- **Caché selectiva:** Solo persiste DataFrames que intervengan en múltiples acciones posteriores.
- **Regla de oro:** Todo `.persist()` o `.cache()` en producción debe tener su correspondiente `.unpersist()` al terminar su ciclo útil.

Lecturas Recomendadas

1. **Karau, H. & Warren, R. (2017).** *High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark.* O'Reilly Media. (Capítulo 2: How Spark Works).
2. **Spark Documentation:** *Memory Management Overview* (`https://spark.apache.org/docs/latest/tuning.html#memory-management-overview`).

CAPÍTULO 16

Particionado, Shuffling y Mitigación de Data Skew AVANZADO

> *"El particionado es el plano regulador de la computación distribuida: define cómo se fragmentan los datos entre los ejecutores. Un mal particionado produce cuellos de botella de red y el temido Data Skew, donde un solo ejecutor retrasado frena a todo el clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la mecánica física del **Shuffling**: fase Map, serialización a disco local, transferencia de red y fase Reduce.
- Diferenciar cuándo utilizar `repartition()` (con shuffle total) y cuándo usar `coalesce()` (sin shuffle, solo fusión local).
- Calibrar el número óptimo de particiones de salida (`spark.sql.shuffle.partitions`) buscando particiones de 100 MB a 200 MB.
- Diagnosticar y resolver el **Data Skew** (desequilibrio de datos) utilizando *Salting* y *Adaptive Query Execution (AQE)* (véase la sección 14.3).

Definición 16.1 (La Autopista Congestionada y las Cajas Desiguales) Imagina un equipo de 10 trabajadores empacando frutas en un almacén. Las operaciones locales (lavar, pelar) se hacen en paralelo sin estorbarse.

El Shuffle ocurre cuando se da la orden de que todas las manzanas deben ir a la mesa 1, todas las naranjas a la mesa 2 y todas las uvas a la mesa 3: los 10 trabajadores tienen que caminar por los pasillos al mismo tiempo esquivándose entre sí. Los pasillos se congestionan (la red del clúster).

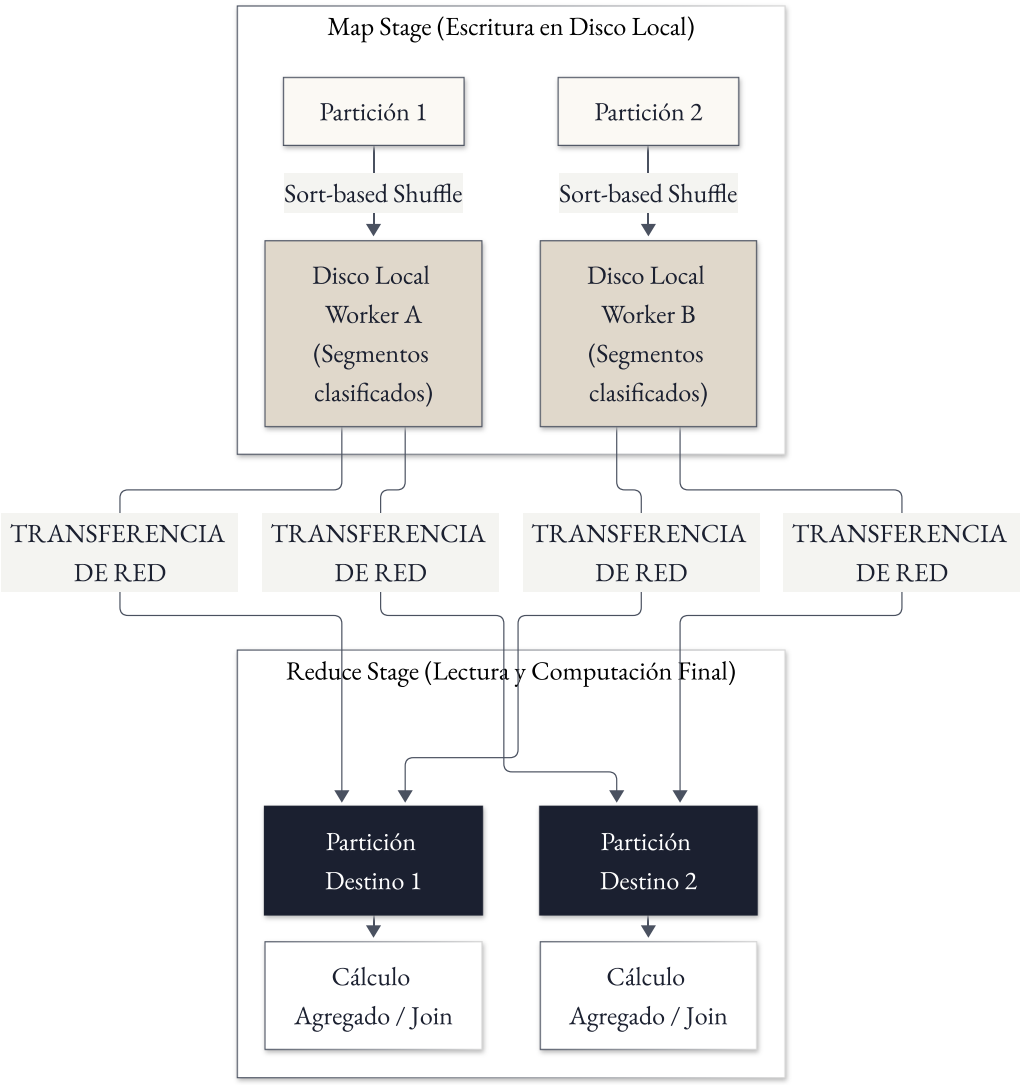
El Data Skew ocurre cuando resulta que el 90% de la carga del camión eran uvas: la mesa 3 queda enterrada bajo una montaña de frutas y trabaja durante 5 horas extras, mientras las otras 9 mesas terminaron en 5 minutos y se quedan con los brazos cruzados esperando a que la mesa 3 termine.

16.1 La Mecánica Física del Shuffling

El **Shuffling** es el proceso mediante el cual Apache Spark redistribuye los datos entre los nodos del clúster para ejecutar operaciones que requieren una reorganización global por clave, como agrupaciones (`groupBy`), uniones (`join`) o cambios forzados de particionamiento.

Es una de las operaciones más costosas del cómputo distribuido y consta de dos fases:

1. **Map Phase (Shuffle Write):** Cada partición de entrada procesa sus datos y escribe los resultados en archivos temporales en el disco local del nodo trabajador, organizados y clasificados por la partición de destino.
2. **Reduce Phase (Shuffle Read):** Las particiones de destino recogen los bloques que les corresponden a través de la red desde todos los nodos remotos, los deserializan y continúan el cálculo.



Repartition vs. Coalesce

Para alterar el número de particiones de un DataFrame disponemos de dos herramientas con comportamientos físicos radicalmente distintos:

- `repartition(numPartitions)`:
  - Ejecuta un **shuffle completo** a través de la red redistribuyendo los datos de forma equitativa (*Full Shuffle*).
  - Es obligatorio cuando necesitas **aumentar** el número de particiones o cuando quieres redistribuir datos sesgados por una clave (`repartition(100, "pais")`).
- `coalesce(numPartitions)`:

- **No realiza shuffle de red**. Simplemente fusiona particiones existentes que ya residen en el mismo nodo o en nodos vecinos.
- Solo puede utilizarse para **reducir** el número de particiones (por ejemplo, reducir de 200 a 10 particiones antes de escribir a disco para evitar miles de archivos pequeños).

16.2 Profundización Técnica: El Síndrome del Straggler (Data Skew)

El **Data Skew** es la patología número uno en pipelines de producción:

Ocurre cuando una clave de agrupación o join está hiperconcentrada (por ejemplo, un valor por defecto como `id_usuario = -1` o transacciones masivas de una corporación multinacional).

En el Spark UI, verás que una tarea (*Task*) tarda 45 minutos mientras las otras 199 tareas terminaron en 10 segundos. Este ejecutor retrasado se denomina **Straggler** y suele colapsar con `OutOfMemory: Container killed by YARN for exceeding memory limits`.

> **Consideraciones de Producción 16.1: Mitigación de Skew con Salting** La técnica de Salting rompe la concentración artificial de una clave agregando un componente aleatorio ("sal") para forzar a Spark a distribuir el trabajo entre múltiples ejecutores:

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("Capitulo16-Particionado-Salting") \
    .master("local[*]") \
    .getOrCreate()

df_sesgado = spark.read.parquet("data/transacciones_sesgadas.parquet")

N_SAL = 10  # número de subclaves en las que se fragmenta la clave caliente

# 1. Añadir una sal aleatoria 0..9 COMO COLUMNA APARTE (no se concatena al id)
df_salado = df_sesgado.withColumn("sal", (F.rand() * N_SAL).cast("int"))

# 2. Primera agregación por (cliente_id, sal): la clave caliente se reparte en 10
grupos
df_parcial = df_salado.groupBy("cliente_id", "sal") \
    .agg(F.sum("monto").alias("monto_parcial"))

# 3. Segunda agregación solo por cliente_id: ahora hay como máximo 10 filas por
cliente
df_final = df_parcial.groupBy("cliente_id") \
    .agg(F.sum("monto_parcial").alias("monto_total"))

df_final.show(5)
```

Agrupar por `(cliente_id, sal)` evita el error clásico de concatenar el id con la sal y luego intentar recuperar el id con `substr(...)`, que solo funciona si todos los identificadores miden exactamente lo mismo. Este patrón de dos fases es útil sobre todo para agregaciones que **no** se resuelven bien con *map-side combine* y para joins; para `sum` y `count` simples, el *combine* parcial ya reduce el problema (véase el Cap. 2) y AQE puede mitigar el resto en joins.

16.3 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Usar `repartition(1)` o `coalesce(1)` sin entender su costo

Para generar un único archivo existen dos caminos con costos distintos. `repartition(1)` ejecuta un shuffle completo hacia una sola partición, pero la etapa **anterior** conserva todo su paralelismo. `coalesce(1)` evita el shuffle, y por eso puede ser **peor**: al fusionar sin frontera de shuffle, **toda la etapa previa se ejecuta en una única tarea**, colapsando el paralelismo de cálculos pesados (joins, agregaciones, UDFs) que estaban antes. Regla práctica: usa `coalesce(n)` para reducir de forma moderada (por ejemplo, de 200 a 20 particiones) justo antes de escribir; si de verdad necesitas un

solo archivo y el cómputo previo es pesado, prefiere `repartition(1)`, y solo si los datos caben sobradamente en un nodo. En muchos casos es mejor escribir varios archivos y consolidarlos después, o dejar que Delta compacte con `OPTIMIZE` (Cap. 18).

Anti-patrón 2: Mantener spark.sql.shuffle.partitions en 200 para cualquier dataset

200 es el valor por defecto de Spark y no tiene relación con tu volumen. Si procesas 1 TB de datos, 200 particiones producirían particiones de unos 5 GB cada una, con alto riesgo de *spill* y fallos por memoria. Si procesas 10 MB, producirás 200 micro-tareas que gastan más tiempo en coordinación que en computación. Calibra para obtener particiones del orden de 100 a 200 MB (o, con AQE activo, fija un número inicial alto y deja que el coalescing lo ajuste hacia `advisoryPartitionSizeInBytes`).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 16.1 (Cálculo del Shuffle Partition Ideal):

Si tu pipeline realiza un `groupBy` sobre un dataset cuyo tamaño intermedio antes del shuffle es de 64 GB, ¿cuál es el número ideal de `spark.sql.shuffle.partitions` que deberías configurar para apuntar a un tamaño de partición de 128 MB?

Solución: Cálculo:

$$\text{Particiones} = \frac{64\ \text{GB}}{128\ \text{MB}} = \frac{64 \times 1024\ \text{MB}}{128\ \text{MB}} = 512\ \text{particiones}$$

Configuración recomendada: `spark.conf.set("spark.sql.shuffle.partitions", 512)`. Esto apunta a que cada tarea procese aproximadamente 128 MB, un tamaño cómodo que reduce el riesgo de *spill* a disco y evita el exceso de tareas diminutas. Con AQE activo puedes fijar un valor inicial mayor y dejar que el coalescing lo ajuste.

16.4 Resumen y Puntos Clave

- **El shuffle no es gratis:** Implica escritura a disco, tráfico de red y deserialización; debe minimizarse mediante filtros tempranos y broadcast joins.
- **Coalesce para reducir, Repartition para aumentar o redistribuir:** `coalesce()` evita el shuffle, pero fusionar hasta muy pocas particiones puede colapsar el paralelismo de la etapa previa; entonces `repartition()` es la opción más segura.

- **Salting combate el sesgo:** Romper claves calientes en fragmentos aleatorios es el patrón estándar de la industria para derrotar al *straggler*.

Lecturas Recomendadas

1. **Karau, H. & Warren, R. (2017).** *High Performance Spark.* O'Reilly Media. (Capítulo 5: Effective Transformations; Capítulo 6: Working with Key/Value Data).
2. **Databricks Blog:** *Optimizing Apache Spark SQL Joins* (`https://databricks.com/blog/2021/04/19/optimizing-apache-spark-sql-joins.html`).

PARTE VI

Arquitectura Lakehouse y transacciones ACID

Capítulo 17 · Arquitectura Medallón y Modelado Dimensional NÚCLEO

Capítulo 18 · Lakehouse y Formatos de Tabla Abiertos: Delta Lake y Apache Iceberg NÚCLEO

Capítulo 18A · Plataforma Databricks: Arquitectura, Delta Live Tables, Photon y Workflows RECOMENDADO

Capítulo 19 · Gobernanza, Linaje y Unity Catalog en Lakehouse RECOMENDADO

Capítulo 20 · Catalogación, Descubrimiento y Contratos de Datos (Data Contracts) RECOMENDADO

CAPÍTULO 17

Arquitectura Medallón y Modelado Dimensional NÚCLEO

> *"La arquitectura medallón y el modelado dimensional representan la convergencia definitiva entre el lago y el almacén: los datos transitan desde el estado crudo inmutable en Bronze, se sanean y enriquecen en Silver, y culminan en modelos dimensionales de estrella y tablas consolidadas en Gold listos para alimentar decisiones estratégicas de negocio con latencia predecible."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la jerarquía de refinamiento y garantías de servicio de la **Arquitectura Medallón**: Capa Bronze (Raw / Inmutable), Capa Silver (Enriched / Conforme) y Capa Gold (Business Aggregates / Data Marts).
- Comparar críticamente esquemas en estrella (*Star Schema*), esquemas en copo de nieve (*Snowflake Schema*) y el patrón *One Big Table* (OBT) sobre motores columnares modernos.
- Clasificar y modelar tablas de hechos (*Transaction Facts*, *Periodic Snapshots*, *Accumulating Snapshots*, *Factless Facts*) y dimensiones (*Conformed*, *Role-Playing*, *Degenerate*, *Bridge*, *Junk Dimensions*).
- Generar claves subrogadas deterministas y evaluar rigurosamente los riesgos de colisión de funciones hash de 64 bits frente a 256 bits (`sha256`).
- Implementar de extremo a extremo el ciclo de vida de Dimensiones Lentamente Cambiantes (*SCD Tipo 1 y Tipo 2*) mediante transacciones `MERGE INTO` en Delta Lake sin duplicación de registros.
- Garantizar idempotencia física utilizando la opción idiomática `replaceWhere` de Delta Lake y el modo de sobrescritura dinámica de particiones.

Definición 17.1 (La Refinería de Metales Preciosos)

La Arquitectura Medallón conceptualiza el flujo de datos como un proceso físico de purificación metalúrgica:

- **Capa Bronze (Mina y Almacén Crudo):** Representa el mineral extraído de la veta geológica tal y como sale a la superficie: contiene barro, impurezas y rocas no mineralizadas. Ningún fragmento se descarta; se preserva en su estado original bajo almacenamiento inmutable para permitir auditorías forenses y reconstrucciones completas del sistema ante cambios de lógica retrospectivos.
- **Capa Silver (Fundición y Refinamiento Conforme):** Es el proceso donde se eliminan las impurezas, se filtran las anomalías, se estandarizan las barras metálicas a un grado de pureza homogéneo y se integran minerales afines. Es la fuente de la verdad operacional para ingenieros de datos y científicos de datos.
- **Capa Gold (Joyería y Acuñación de Monedas):** Constituye el producto final pulido con precisión micrométrica: joyas, coronas y monedas certificadas listas para el consumo directo por parte de usuarios finales, cuadros de mando ejecutivos (Power BI, Tableau) y modelos analíticos de alto impacto.

17.1 Jerarquía de Capas en la Arquitectura Medallón

La Arquitectura Medallón organiza el lago de datos transaccional en tres capas de madurez progresiva, desacoplando la ingestión veloz de la computación semántica compleja:



1. Capa Bronze (Raw / Ingesta Histórica)

- **Contenido:** Copia fidedigna de las cargas de origen sin modificaciones de valor ni descarte de registros. Los esquemas admiten variantes flexibles (JSON empaquetado o columnas estructuradas).
- **Columnas obligatorias de metadatos de auditoría:**
  - `_ingestion_timestamp`: Marca temporal UTC en la que el registro aterrizó en el Lakehouse.
  - `_source_file`: Ruta de almacenamiento o identificador del mensaje Kafka de origen.
  - `_batch_id` / `_job_id`: Identificador de ejecución del pipeline orquestador.

- **Política de escritura:** Estrictamente **Append-Only** (anexión continua). Jamás se ejecutan actualizaciones ni borrados directos en Bronze, salvo purgas obligatorias por regulaciones de privacidad (RGPD / Habeas Data).

2. Capa Silver (Limpia, Tipada y Conformada)

- **Contenido:** Datos validados, normalizados con tipos canónicos (reemplazo de strings por timestamps, enteros y decimales precisos), deduplicados y enriquecidos con cruces fundamentales.
- **Propósito:** Representa la "versión única de la verdad" corporativa. Aquí viven las entidades de negocio centrales (clientes, órdenes, pagos, eventos de navegación).
- **Reglas de Calidad:** Se descartan o desvían a tablas de cuarentena aquellos registros que violan contratos de datos o integridad estructural.

3. Capa Gold (Agregada y Optimizada para el Negocio)

- **Contenido:** Conjuntos de datos estructurados para maximizar la velocidad de lectura interactiva. Implementa modelos dimensionales de Ralph Kimball (esquemas en estrella), vistas materializadas y tablas desnormalizadas (*One Big Table*).
- **Consumidores:** Consultas ad-hoc de analistas, reportes regulatorios, dashboards ejecutivos y *feature stores* para aprendizaje automático.

17.2 Modelado Dimensional en el Lakehouse Moderno

El modelado dimensional introducido por Ralph Kimball mantiene plena vigencia en los motores analíticos columnares, aunque su justificación física ha evolucionado:

Clasificación Integral de Tablas de Hechos (Fact Tables)

Las tablas de hechos capturan métricas cuantitativas generadas por eventos operacionales:

1. **Hechos Transaccionales (*Transaction Facts*):** Registran un evento en un instante temporal discreto (una venta, un clic, un movimiento bancario). Poseen la granularidad más atómica y el volumen más masivo.
2. **Hechos de Instantánea Periódica (*Periodic Snapshot Facts*):** Capturan el estado del negocio en intervalos regulares acumulativos (saldo de fin de mes de una cuenta, nivel de inventario semanal por almacén).
3. **Hechos de Instantánea Acumulativa (*Accumulating Snapshot Facts*):** Modelan el ciclo de vida de un proceso con inicio y fin definidos con múltiples hitos temporales (solicitud de préstamo: fecha recepción → fecha aprobación → fecha desembolso).

4. **Hechos Sin Hechos (*Factless Fact Tables*):** Tablas de hechos que no contienen métricas numéricas acumulables, sino exclusivamente claves foráneas que registran la ocurrencia de un evento (ej. asistencia de un estudiante a una clase o cobertura de un evento promocional).

Clasificación de Tablas de Dimensiones (Dimension Tables)

1. **Dimensiones Conformadas (*Conformed Dimensions*):** Dimensiones consistentes y universales compartidas entre múltiples hechos (ej. la dimensión `dim_cliente` es idéntica para hechos de ventas, soporte y facturación).
2. **Dimensiones de Rol Jugado (*Role-Playing Dimensions*):** Una misma dimensión física asume diferentes roles lógicos en la misma tabla de hechos mediante múltiples claves foráneas (ej. `dim_tiempo` referenciada como `fecha_orden_id`, `fecha_envio_id` y `fecha_entrega_id`).
3. **Dimensiones Degeneradas (*Degenerate Dimensions*):** Identificadores transaccionales que residen en la tabla de hechos sin una tabla de dimensión satélite propia (ej. `numero_factura`, `codigo_rastreo`).
4. **Dimensiones Basura (*Junk Dimensions*):** Combinación de atributos discretos de baja cardinalidad (flags booleanos, códigos de estado) agrupados en una sola tabla de dimensión para evitar saturar la tabla de hechos con decenas de claves foráneas.
5. **Tablas Puente (*Bridge Tables*):** Tablas intermedias utilizadas para modelar relaciones de muchos a muchos (*M:N*) entre una tabla de hechos y una dimensión (ej. una póliza de seguro con múltiples beneficiarios).
6. **Miembros Desconocidos (*Unknown Members*):** Toda dimensión debe incluir un registro con clave subrogada `-1` (ej. `cliente_sk = -1`, `nombre = 'NO INFORMADO'`). Cuando una transacción de la tabla de hechos contiene una clave natural nula o no resuelta, apunta a `-1` en lugar de dejar la clave foránea en `NULL`, evitando que los cruces `INNER JOIN` descarten transacciones legítimas.

Star Schema vs. Snowflake Schema vs. One Big Table (OBT)

Criterio	Esquema en Estrella (Star)	Esquema Copo de Nieve (Snowflake)	One Big Table (OBT)
Normalización	Desnormalizado en dimensiones (2NF).	Altamente normalizado (3NF en dimensiones).	Completamente desnormalizado (1 sola tabla).
Complejidad de Joins	Baja (1 join por dimensión contra el hecho).	Alta (múltiples joins en cascada para resolver jerarquías).	Cero joins (toda la información en la misma fila).
Eficiencia en Motores Columnares	**Excelente:** Excelente compresión y data skipping.	**Deficiente:** Múltiples shuffles distribuidos ralentizan BI.	**Suprema en consultas específicas:** Máxima velocidad, mayor costo de almacenamiento.
Mantenibilidad	Alta: Esquema comprensible por usuarios de negocio.	Media: Jerarquías rígidas y complejas.	Baja: Cambios de atributos requieren reescribir tablas masivas.

Claves Subrogadas Distribuidas: Paradoja del Cumpleaños y Hashing

En almacenes relacionales tradicionales se utilizaban secuencias autoincrementales (`AUTO_INCREMENT`). En sistemas distribuidos, generar un entero monótono global introduce un cuello de botella de coordinación centralizado.

El estándar moderno en Lakehouse consiste en generar claves subrogadas deterministas mediante **funciones hash criptográficas** sobre la clave natural del negocio y los metadatos de vigencia:

$$\text{cliente_sk} = \text{sha2}(\text{concat}(\text{sistema_origen}, \text{cliente_id}, \text{date_format}(\text{valido_desde}, \text{'y}$$

Análisis de Probabilidad de Colisión

Un error frecuente consiste en emplear hashes de 64 bits (`xxhash64`) en tablas masivas. De acuerdo con la **paradoja del cumpleaños**, la probabilidad de que ocurra al menos una colisión en un espacio de claves de tamaño N tras insertar n registros se aproxima por:

$$P(\text{colisión}) \approx 1 - e^{-\frac{n^2}{2N}}$$

- Para un hash de **64 bits** ($N = 2^{64} \approx 1.84 \times 10^{19}$): Con apenas $n = 10^9$ registros (mil millones de filas), la probabilidad de colisión es:

$$P \approx 1 - e^{-\frac{10^{18}}{2 \times 1.84 \times 10^{19}}} \approx 1 - e^{-0.027} \approx \mathbf{2.7\%}$$

Una probabilidad del 2.7% de corromper la unicidad de las claves es inaceptable en analítica empresarial.

4. Data Vault 2.0 en la Capa Silver Empresarial: Hubs, Links y Satellites

En organizaciones corporativas complejas con decenas de sistemas transaccionales heterogéneos (SAP, Salesforce, bases operacionales locales), intentar forzar un modelo dimensional de Kimball directamente en la ingesta crea cuellos de botella organizacionales: cualquier cambio en una fuente exige renegociar el esquema de la dimensión compartida.

Data Vault 2.0 (diseñado por Dan Linstedt) resuelve este problema desacoplando la integración en tres primitivas tabulares normalizadas y auditables:



1. **Hubs (Centros de Negocio):** Representan conceptos centrales del negocio (Clientes, Productos, Tiendas). Contienen únicamente la clave de negocio única (*Business Key*), su hash SHA-256 (`hub_cliente_hk`), la marca de tiempo de carga (`load_timestamp`) y el sistema fuente de origen (`record_source`). **No contienen atributos descriptivos.**
2. **Links (Interconexiones y Transacciones):** Representan relaciones asociativas o transacciones de negocio entre dos o más Hubs (ej. una venta que asocia a un Cliente con un Producto). Almacenan los hashes foráneos de los Hubs interconectados y son **estrictamente de solo adición (*Append-Only*)**.

3. **Satellites (Contexto Descriptivo e Historial):** Almacenan todos los atributos descriptivos mutables a lo largo del tiempo. Incorporan un campo especial `hash_diff` (un hash SHA-256 concatenado de todos los atributos descriptivos). Al procesar una carga incremental, si el `hash_diff` entrante difiere del último registrado, se inserta una nueva fila satélite de forma automática sin bloquear otras fuentes.

> **Estrategia Híbrida en Producción:** La arquitectura más robusta utiliza **Data Vault 2.0 en la Capa Silver** (para integrar 30+ fuentes heterogéneas sin fricciones de esquema) y proyecta **Modelos Dimensionales de Kimball en la Capa Gold** (para que analistas de negocio y herramientas de BI como Power BI o Tableau consulten estrellas optimizadas y comprensibles).

17.3 Slowly Changing Dimensions (SCD) Tipo 1 y Tipo 2

Cuando los atributos de una dimensión cambian con el tiempo (por ejemplo, un cliente se traslada de ciudad), el modelo debe gobernar la mutación histórica:

- **SCD Tipo 1 (Sobrescritura Directa):** El valor anterior se destruye y se reemplaza por el nuevo. No conserva historia. Adecuado para corregir errores tipográficos en nombres o números telefónicos.
- **SCD Tipo 2 (Preservación Histórica):** Se cierra la vigencia del registro existente estableciendo su fecha final y marcando su bandera de actualidad como inactiva; a continuación, se inserta una nueva fila con los valores actualizados y vigencia abierta.

```
Estado Original:
[ID: 101 | Nombre: Juan | Ciudad: Bogotá    | Valido_Desde: 2024-01-01 |
Valido_Hasta: 9999-12-31 | Es_Actual: TRUE]

Tras mudanza a Medellín (SCD Tipo 2 aplicado el 2026-06-15):
[ID: 101 | Nombre: Juan | Ciudad: Bogotá    | Valido_Desde: 2024-01-01 |
Valido_Hasta: 2026-06-15 | Es_Actual: FALSE]
[ID: 101 | Nombre: Juan | Ciudad: Medellín  | Valido_Desde: 2026-06-15 |
Valido_Hasta: 9999-12-31 | Es_Actual: TRUE]
```

> **Consideraciones de Producción 17.1: Implementación Robusta de SCD Tipo 2 con Delta Lake MERGE**
>
> Para ejecutar SCD Tipo 2 en una sola transacción atómica distribuida **sin duplicar registros que no cambiaron**, se identifican de forma rigurosa los cambios reales y los clientes nuevos:

PYTHON

```
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from delta.tables import DeltaTable

spark = SparkSession.builder \
    .appName("Capitulo17-SCD2-Robusto") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog") \
    .master("local[*]") \
    .getOrCreate()

ruta_dim_clientes = "lakehouse/gold/dim_clientes"
delta_dim = DeltaTable.forPath(spark, ruta_dim_clientes)

# 1. Novedades entrantes validadas
df_novedades = spark.read.table("silver_staging_clientes")

# 2. Clientes actualmente vigentes en la dimensión
df_actuales = delta_dim.toDF().filter(F.col("es_actual") == True)

# 3. Detectar registros modificados (cambio en atributos seguidos)
df_cambios = df_novedades.alias("nov") \
    .join(df_actuales.alias("dim"), "cliente_id") \
    .filter("(nov.ciudad <> dim.ciudad) OR (nov.segmento <> dim.segmento)")

# 4. Detectar clientes completamente nuevos (que no existen en la dimensión)
df_nuevos = df_novedades.join(df_actuales, "cliente_id", "left_anti")

# 5. Construir el conjunto unificado para el MERGE:
# a) Filas para CERRAR: merge_key = cliente_id (coincide con target vigente)
df_para_cerrar = df_cambios.select(
    F.col("nov.cliente_id").alias("merge_key"),
    F.col("nov.*")
)

# b) Filas para INSERTAR: solo los clientes nuevos y la nueva versión de los
modificados
#    (merge_key = NULL para forzar la rama whenNotMatchedInsert)
df_para_insertar = df_nuevos.unionByName(df_cambios.select("nov.*")).select(
    F.lit(None).cast("string").alias("merge_key"),
    F.col("*")
)

df_staged_merge = df_para_cerrar.unionByName(df_para_insertar)

# 6. Ejecución atómica de MERGE INTO en Delta Lake
delta_dim.alias("target").merge(
    df_staged_merge.alias("source"),
    "target.cliente_id = source.merge_key AND target.es_actual = true"
).whenMatchedUpdate(
    set={
        "valido_hasta": F.col("source.fecha_evento"),
        "es_actual": F.lit(False)
    }
).whenNotMatchedInsert(
```

```
    values={
        "cliente_sk": F.expr("sha2(concat('ERP_', source.cliente_id, '_',
string(source.fecha_evento)), 256)"),
        "cliente_id": "source.cliente_id",
        "nombre": "source.nombre",
        "ciudad": "source.ciudad",
        "segmento": "source.segmento",
        "valido_desde": "source.fecha_evento",
        "valido_hasta": F.to_timestamp(F.lit("9999-12-31 23:59:59")),
        "es_actual": F.lit(True)
    }
).execute()

print("[PRODUCCIÓN] SCD Tipo 2 aplicado exitosamente bajo transacción ACID.")
```

Consideraciones de Producción 17.2: Generación Automatizada de la Dimensión de Tiempo

La dimensión `dim_tiempo` es el eje gravitacional de todo esquema en estrella. Jamás debe depender de extracciones de fuentes operacionales; se genera sintéticamente cubriendo un horizonte de 10 a 20 años:

PYTHON

```
# Generación de dim_tiempo para 2020 a 2035 en PySpark
df_fechas = spark.sql("""
    SELECT explode(sequence(to_date('2020-01-01'), to_date('2035-12-31'),
interval 1 day)) AS fecha
""").select(
    F.date_format("fecha", "yyyyMMdd").cast("int").alias("fecha_id"),
    F.col("fecha"),
    F.year("fecha").alias("anio"),
    F.quarter("fecha").alias("trimestre"),
    F.month("fecha").alias("mes"),
    F.date_format("fecha", "MMMM").alias("nombre_mes"),
    F.dayofmonth("fecha").alias("dia_mes"),
    F.dayofweek("fecha").alias("dia_semana"),
    F.date_format("fecha", "EEEE").alias("nombre_dia"),
    F.when(F.dayofweek("fecha").isin(1, 7),
F.lit(True)).otherwise(F.lit(False)).alias("es_fin_de_semana")
)

df_fechas.write.format("delta").mode("overwrite").save("lakehouse/gold/dim_tiempo
")
```

> **Consideraciones de Producción 17.3: Idempotencia Idiomática con `replaceWhere` en Delta Lake**
>
> Aunque `spark.sql.sources.partitionOverwriteMode = "dynamic"` funciona a nivel de sesión en Spark, el estándar idiomático y más seguro en Delta Lake es utilizar la opción `replaceWhere`. Permite sobrescribir atómicamente un predicado específico garantizando que únicamente los datos que cumplan la condición sean reemplazados:

```python
# Sobrescritura atómica e idempotente de la partición del 12 de octubre de 2026
df_hechos_dia.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", "fecha_id = 20261012") \
    .save("lakehouse/gold/fact_ventas_diarias")
```

17.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Exponer la Capa Bronze a Usuarios Finales y Analistas de Negocio Permitir que herramientas de BI apunten a Bronze ocasiona escaneos ineficientes sobre miles de archivos crudos sin optimizar, genera métricas discordantes debido a registros malformados y viola normativas de privacidad al consultar datos PII no enmascarados.

Anti-patrón 2: Normalización Extrema (3NF) en la Capa Gold Diseñar un modelo normalizado con 10 o más tablas intermedias obliga al motor distribuido a realizar múltiples etapas de shuffle y cruces complejos en cada consulta analítica. En motores columnares analíticos, la desnormalización controlada (Star Schema o OBT) reduce drásticamente el costo de red.

Anti-patrón 3: Insertar Todas las Novedades Ciegas en el Staging de SCD2 Como se demostró, insertar todas las novedades de entrada con clave nula en lugar de unir exclusivamente los clientes nuevos y las nuevas versiones de los modificados provoca la duplicación silenciosa de millones de registros idénticos en cada ejecución del pipeline.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 17.1 (Diseño de Esquema Dimensional de Comercio Electrónico):
Diseña el esquema en estrella conceptual para una plataforma transaccional de ventas internacionales. Define:

1. La tabla de hechos central indicando su granularidad atómica y tres métricas cuantitativas no redundantes.
2. Al menos cuatro dimensiones conformadas con sus respectivas claves subrogadas generadas mediante hashing SHA-256.
3. Identifica una dimensión degenerada que deba permanecer en el hecho.

Solución 17.1:

- **Granularidad del Hecho:** Una fila por cada ítem individual comprado dentro de una orden de pedido (`fact_ventas_items`).
- **Métricas Cuantitativas:** `cantidad_unidades` (entero aditivo), `precio_unitario_usd` (decimal semi-aditivo), `monto_descuento_usd` (aditivo), `costo_envio_imputado_usd` (aditivo).
- **Dimensiones Satélites:**
  - `dim_cliente`: Clave subrogada `cliente_sk = sha2(concat(sistema_origen, cliente_id, date_format(valido_desde, 'yyyyMMddHHmmss')), 256)`. Atributos: nombre, segmento, país de residencia, fecha de alta.
  - `dim_producto`: Clave subrogada `producto_sk = sha2(sku, 256)`. Atributos: SKU, categoría, subcategoría, marca, costo estándar.
  - `dim_tiempo`: Clave subrogada `fecha_id = cast(date_format(fecha, 'yyyyMMdd') as int)`. Atributos: fecha, día semana, trimestre, año fiscal, indicador festivo.
  - `dim_medio_pago`: Clave subrogada `medio_pago_sk = sha2(metodo_id, 256)`. Atributos: tipo pasarela, emisor, franquicia, modalidad (crédito/débito).
- **Dimensión Degenerada:** `numero_orden_pedido` (se conserva en la tabla de hechos para rastreo y auditoría sin requerir un join con una dimensión separada).

Ejercicio 17.2 (Gobernanza de SCD Tipo 2 y Claves Deterministas):
Un cliente corporativo con `cliente_id = 'C-8921'` cambia su categoría de riesgo financiero de `'MEDIO'` a `'CRITICO'` el día `2026-10-12 12:00:00`.

1. ¿Por qué la clave subrogada `cliente_sk` no puede calcularse simplemente como `sha2(cliente_id, 256)`?
2. Muestra los atributos exactos de las dos filas resultantes en la dimensión tras el merge.

Solución 17.2:

1. *Razón:* Si `cliente_sk` fuera solo el hash de `cliente_id`, ambas versiones del cliente (la histórica cerrada y la nueva activa) tendrían **exactamente la misma clave subrogada**, violando la unicidad de la clave primaria de la dimensión. En SCD2, la clave subrogada debe

incorporar la fecha de inicio de vigencia: `sha2(concat(cliente_id, valido_desde), 256)`.

2. *Filas resultantes en la dimensión:*
   - *Versión Histórica Cerrada:*
     - `cliente_sk`: `sha2('C-8921_2025-01-01 00:00:00', 256)`
     - `cliente_id`: `'C-8921'`
     - `categoria_riesgo`: `'MEDIO'`
     - `valido_desde`: `'2025-01-01 00:00:00'`
     - `valido_hasta`: `'2026-10-12 12:00:00'`
     - `es_actual`: `false`
   - *Versión Vigente Activa:*
     - `cliente_sk`: `sha2('C-8921_2026-10-12 12:00:00', 256)`
     - `cliente_id`: `'C-8921'`
     - `categoria_riesgo`: `'CRITICO'`
     - `valido_desde`: `'2026-10-12 12:00:00'`
     - `valido_hasta`: `'9999-12-31 23:59:59'`
     - `es_actual`: `true`

17.5 Resumen y Puntos Clave

- **Refinamiento progresivo:** Bronze asegura inmutabilidad y auditabilidad cruda; Silver conforma, tipa y estandariza la verdad operacional; Gold estructura datos para máxima velocidad analítica.
- **Vigencia de Ralph Kimball:** El modelado dimensional en estrella y OBT maximiza el rendimiento de motores columnares al eliminar cruces distribuidos complejos y potenciar el data skipping.
- **Claves subrogadas y Paradoja del Cumpleaños:** Para dimensiones que superan cientos de millones de registros, los hashes de 64 bits (`xxhash64`) presentan un riesgo de colisión inaceptable (~2.7% con 10^9 filas). Utilice **SHA-256** para garantizar unicidad global determinista.
- **SCD Tipo 2 atómico y seguro:** Al construir la fuente del `MERGE INTO`, filtre estrictamente para unir únicamente clientes nuevos y las nuevas versiones de los modificados, evitando duplicar registros estables.

- **Idempotencia con `replaceWhere`:** En Delta Lake, utilice `.option("replaceWhere", "predicado")` para garantizar el reemplazo atómico de particiones específicas sin riesgo de borrar datos ajenos.

Lecturas Recomendadas

1. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
2. **Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021).** *Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics.* Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR).
3. **Databricks Engineering Documentation (2024).** *Delta Lake Merge and SCD Type 2 Architecture Patterns.* Databricks Guides (`https://docs.delta.io/latest/delta-update.html`).

CAPÍTULO 18

Lakehouse y Formatos de Tabla Abiertos: Delta Lake y Apache Iceberg

NÚCLEO

> *"Los formatos de tabla abiertos como Delta Lake y Apache Iceberg redefinen la relación entre el cómputo distribuido y el almacenamiento masivo: dotan a los archivos columnares de semántica transaccional ACID, aíslan a los lectores de escrituras concurrentes y elevan un lago pasivo de objetos a un sistema gestor de bases de datos de escala planetaria."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la anatomía interna del registro de transacciones de **Delta Lake** (`_delta_log`), los commits atómicos JSON y los checkpoints Parquet.
- Analizar la jerarquía arquitectónica en cinco capas de **Apache Iceberg**: Catálogo, archivos de metadatos, listas de manifiestos (*Manifest Lists*), manifiestos (*Manifests*) y archivos de datos.
- Dominar las innovaciones modernas de Delta Lake: **Liquid Clustering (`CLUSTER BY`)**, vectores de borrado (*Deletion Vectors*) y mapeo de columnas (*Column Mapping*).
- Implementar operaciones atómicas de fusión (`MERGE INTO` / UPSERT), compactación con Z-Order (curva de Morton) y purga segura (`VACUUM`).
- Escribir DDL y comandos de mantenimiento en **Apache Iceberg**: particionamiento transformado (`days(ts)`, `bucket`), compactación de archivos (`rewrite_data_files`) y expiración de snapshots.
- Evaluar el control de concurrencia optimista (OCC) y los escenarios reales de colisión bajo el nivel de aislamiento `WriteSerializable`.

Definición 18.1 (El Lago Pasivo frente al Notario Transaccional)

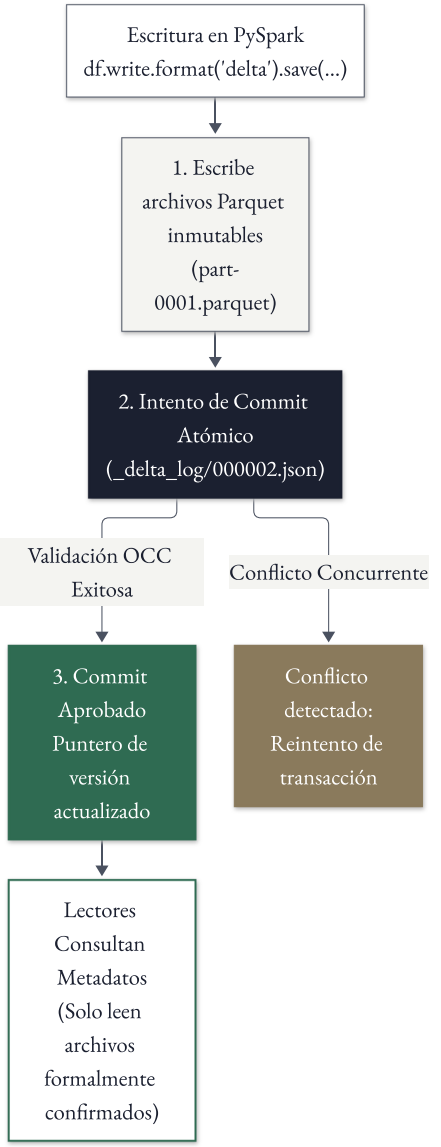
Un Data Lake tradicional basado puramente en Parquet sobre Amazon S3 o Azure ADLS es equivalente a una carpeta de red compartida sin control de concurrencia: si un proceso de escritura falla a mitad de la carga, deja archivos a medio escribir que corrompen las lecturas concurrentes; si un registro requiere actualizarse, es necesario reescribir la partición completa.

Un **formato de tabla abierto** actúa como un **notario público transaccional**: los datos se almacenan en archivos Parquet inmutables, pero ningún motor de cómputo (Spark, Trino, Flink, DuckDB) puede acceder a ellos sin consultar primero el libro de actas oficial (el registro `_delta_log` en Delta o el árbol de manifiestos en Iceberg). El notario solo valida una operación cuando todos los archivos han sido escritos con éxito, garantizando atomicidad estricta y aislamiento optimista.

18.1 Semántica ACID y Control de Concurrencia Optimista (OCC)

Los formatos de tabla abiertos extienden las garantías ACID a los sistemas de almacenamiento de objetos en la nube:

- **Atomicidad:** La transacción es indivisible. Si un job de Spark de 50.000 tareas falla en la última tarea, ninguno de los archivos escritos es referenciado en el catálogo; para los lectores externos, la operación nunca ocurrió.
- **Consistencia:** Mediante la imposición de esquemas (*Schema Enforcement*), el formato rechaza escrituras que alteren los tipos de datos o nombres de columnas no autorizados, garantizando la validez de los datos.
- **Aislamiento:** Emplea **Control de Concurrencia Optimista (OCC)** bajo el estándar de aislamiento `WriteSerializable` o `Serializable`. Los lectores nunca bloquean a los escritores.
- **Durabilidad:** La durabilidad física en reposo la provee el almacenamiento de objetos subyacente (ej. la resiliencia de 99.999999999% de Amazon S3 o ADLS Gen2), mientras que el formato garantiza la durabilidad lógica mediante su registro de transacciones de escritura anticipada (*Write-Ahead Log*).



Mecánica Real de Conflictos en OCC

Bajo el nivel de aislamiento `WriteSerializable` de Delta Lake:

- **Dos inserciones ciegas concurrentes (`Append + Append`):** Si dos jobs distintos ejecutan `df.write.mode("append")` sobre la misma partición, **no colisionan**. Delta resuelve el orden de los commits y confirma ambos sin error.
- **Conflicto real de escritura:** Ocurre cuando una transacción ejecuta una operación condicional (`MERGE INTO`, `UPDATE` o `DELETE`) que lee un conjunto de archivos para evaluar predicados, y otra transacción concurrente modifica o elimina esos mismos archivos antes de que la primera haga el commit.

18.2 Anatomía Interna de Delta Lake: El Registro `_delta_log`

Cada tabla Delta almacena en su directorio raíz una subcarpeta obligatoria denominada `_delta_log/`:

1. **Archivos de Transacción JSON (`000000.json`, `000001.json`):** Cada operación confirmada añade un nuevo archivo JSON monótonamente creciente. Contiene las acciones atómicas del commit:
   - `add`: Registra la ruta, tamaño y estadísticas (mínimo, máximo, nulos por columna) de los nuevos archivos Parquet generados.
   - `remove`: Marca archivos Parquet existentes como lógicamente obsoletos (sin eliminarlos físicamente de inmediato).
   - `commitInfo`: Metadatos del usuario, motor de cómputo y timestamp de la operación.
2. **Checkpoints Consolidados Parquet (`000010.checkpoint.parquet`):** Cada 10 transacciones, Delta consolida todos los commits previos en un archivo Parquet optimizado. Al leer la tabla, Spark solo consulta el checkpoint más reciente y aplica los deltas JSON subsiguientes, evitando escanear miles de archivos JSON individuales.
3. **Column Mapping:** Permite que las columnas de la tabla tengan identificadores numéricos y nombres físicos desacoplados de sus nombres lógicos, habilitando renombramientos instantáneos (`ALTER TABLE ... RENAME COLUMN`) sin reescribir archivos Parquet.
4. **Deletion Vectors:** Al ejecutar `DELETE` o `UPDATE`, Delta no reescribe los archivos Parquet enteros que contienen las filas afectadas (Copy-on-Write). En su lugar, escribe un pequeño vector de borrado (mapa de bits Roaring Bitmap) que indica las posiciones de fila invalidadas, reduciendo el I/O en más de un 90%.

18.3 Liquid Clustering: La Superación del Particionado Tradicional

Históricamente, las tablas se particionaban mediante carpetas jerárquicas rígidas (`anio=2026/mes=10/dia=12/`). Este enfoque presenta graves problemas:

- **Sobre-particionado (*Overpartitioning*):** Miles de carpetas con archivos diminutos que saturan el sistema de archivos.
- **Rigidez:** Imposible optimizar consultas que filtren por columnas no incluidas en la jerarquía física de partición.
- **Sesgo de Datos (*Data Skew*):** Particiones con tamaños gigantescos y otras casi vacías.

Liquid Clustering (`CLUSTER BY`) reemplaza el particionado físico y Z-Order en Delta Lake moderno utilizando la **curva de Hilbert**:

```sql
-- Creación de tabla con Liquid Clustering nativo
CREATE TABLE lakehouse.gold.fact_ventas (
    venta_id STRING,
    cliente_id STRING,
    fecha_transaccion DATE,
    monto_usd DOUBLE
)
USING DELTA
CLUSTER BY (fecha_transaccion, cliente_id);

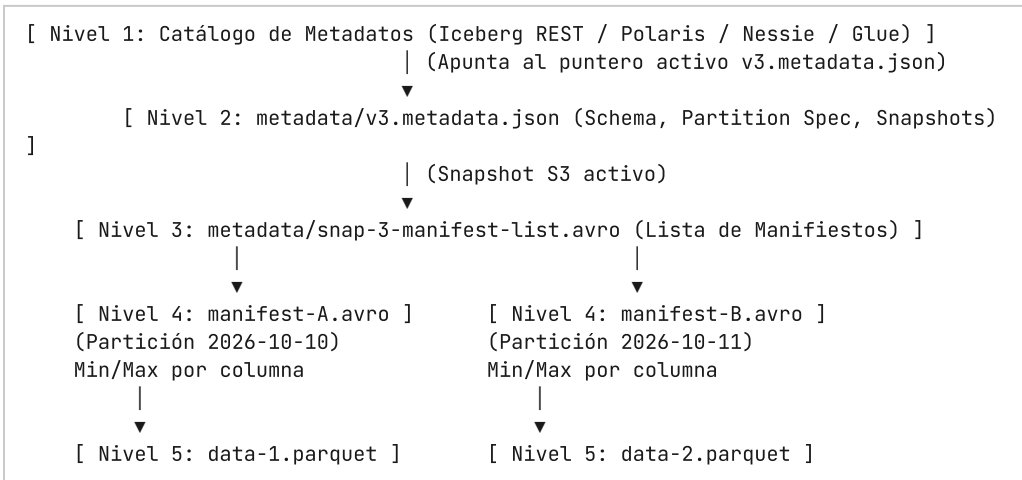
-- Compactación y re-agrupamiento incremental con Liquid Clustering
OPTIMIZE lakehouse.gold.fact_ventas;
```

- **Re-clustering Incremental:** Solo reorganiza los datos nuevos o desbalanceados sin requerir la reescritura de particiones completas.
- **Evolución sin Migración:** Las columnas de clustering pueden modificarse (`ALTER TABLE ... CLUSTER BY (nuevo_campo)`) sin necesidad de reescribir la tabla histórica.

18.4 Arquitectura Profunda de Apache Iceberg (5 Niveles)

Apache Iceberg es un formato de tabla abierto de alto rendimiento diseñado originalmente por Netflix para superar las deficiencias de escala del Hive Metastore en tablas con millones de particiones y archivos.

Iceberg organiza sus metadatos en un **árbol jerárquico de cinco niveles**:



1. **Nivel 1 - Catálogo de Metadatos (Iceberg Catalog):** Almacena exclusivamente el puntero atómico hacia el archivo de metadatos más reciente (`vN.metadata.json`). Admite implementaciones abiertas como Iceberg REST Catalog, Apache Polaris, Project Nessie, AWS Glue o Hive Metastore.
2. **Nivel 2 - Archivo de Metadatos de la Tabla (`metadata.json`):** Contiene el esquema completo de la tabla, las especificaciones de partición, el historial de evolución de esquemas y la lista completa de todos los *snapshots* válidos.
3. **Nivel 3 - Lista de Manifiestos (*Manifest List - Avro*):** Representa un snapshot individual de la tabla. Lista todos los archivos de manifiesto que componen ese snapshot, incluyendo el rango de particiones cubiertas por cada manifiesto para permitir podar manifiestos enteros antes de leer archivos de datos.
4. **Nivel 4 - Archivos de Manifiesto (*Manifest Files - Avro*):** Enumeran los archivos de datos físicos individuales (`.parquet`), especificando su ruta, tamaño y estadísticas exhaustivas a nivel de columna (mínimos, máximos, conteo de nulos y valores NaN).
5. **Nivel 5 - Archivos de Datos Físicos (*Data Files*):** Bloques columnares Parquet u ORC que contienen las filas útiles de la tabla.

Consideraciones de Producción 18.1: DDL y Mantenimiento de Apache Iceberg

Código ejecutable en Spark SQL para crear, particionar con transformaciones nativas y optimizar una tabla en Apache Iceberg:

SQL

```sql
-- 1. Creación de tabla con particionamiento oculto transformado
CREATE TABLE local_iceberg.telemetria.eventos_sensores (
    sensor_id STRING,
    timestamp_evento TIMESTAMP,
    temperatura DOUBLE,
    voltaje DOUBLE
)
USING ICEBERG
PARTITIONED BY (days(timestamp_evento), bucket(16, sensor_id));

-- 2. Compactación de micro-archivos mediante el procedimiento del sistema
CALL local_iceberg.system.rewrite_data_files(
    table => 'local_iceberg.telemetria.eventos_sensores',
    strategy => 'binpack',
    options => map('target-file-size-bytes', '536870912') -- 512 MB
);

-- 3. Expiración segura de snapshots antiguos para liberar almacenamiento en la
nube
CALL local_iceberg.system.expire_snapshots(
    table => 'local_iceberg.telemetria.eventos_sensores',
    older_than => TIMESTAMP '2026-10-05 00:00:00',
    retain_last => 5
);
```

18.5 Matriz Comparativa: Delta Lake vs. Apache Iceberg vs. Apache Hudi

Característica / Criterio	Delta Lake (3.x / Linux Foundation)	Apache Iceberg (1.x / ASF)	Apache Hudi (0.16+ / ASF)
Gobernanza / Origen	Databricks / Linux Foundation	Netflix, Apple / ASF	Uber / ASF
Modelo de Metadatos	Registro secuencial de deltas (`_delta_log/`) con checkpoints Parquet.	Árbol jerárquico en 5 niveles con manifiestos Avro.	Registro de línea de tiempo (*Timeline*) con metadatos indexados.
Optimización de Agrupación	**Liquid Clustering** (`CLUSTER BY`) con curva de Hilbert, Z-Order (Morton).	Compactación binpack / ordenamiento Z-Order o Sort.	Clustering y compresión basados en Timeline.
Particionamiento	Basado en columnas lógicas; reemplazado por Liquid Clustering.	**Oculto (Hidden Partitioning)** con funciones nativas (`days`, `bucket`).	Basado en claves de partición explícitas configuradas.
Evolución de Esquema	Por ID con **Column Mapping** activado; o por nombre con `mergeSchema`.	**Por ID entero inmutable:** Totalmente inmune a renombramientos y reordenamientos.	Soportado mediante esquemas Avro.
Mecanismo de Borrado	Deletion Vectors (Roaring Bitmaps) y Copy-on-Write.	Copy-on-Write (COW) y Merge-on-Read (MOR) con archivos de borrado posicional.	Copy-on-Write (COW) y Merge-on-Read (MOR) optimizado para streaming.
Catálogo Abierto	Unity Catalog OSS, Hive Metastore, Delta Sharing.	Iceberg REST Catalog, Apache Polaris, Nessie, Glue.	Hudi Metastore, Hive, Glue.

Consideraciones de Producción 18.2: Mantenimiento Seguro con OPTIMIZE, Z-ORDER y VACUUM

Para tablas Delta Lake tradicionales basadas en particiones, el agrupamiento espacial con Z-Order utiliza la **curva de Morton (curva Z)** intercalando bits para agrupar registros multidimensionales:

```sql
-- 1. Z-Order tradicional sobre columnas de filtro frecuente
OPTIMIZE delta.`s3://lakehouse-corp/gold/fact_ventas`
ZORDER BY (cliente_id, fecha_transaccion);

-- 2. Purga segura de archivos obsoletos (retención obligatoria de 7 días)
VACUUM delta.`s3://lakehouse-corp/gold/fact_ventas` RETAIN 168 HOURS;
```

Para entornos de prueba o emergencias operacionales donde se requiera forzar `RETAIN 0 HOURS`, **el flag oficial de seguridad en el motor Spark de Databricks es**:

```python
spark.conf.set("spark.databricks.delta.retentionDurationCheck.enabled", "false")
```

18.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Ejecutar `VACUUM` con Retención Cero en Entornos Concurrentes Deshabilitar la verificación de retención (`retentionDurationCheck.enabled = false`) y ejecutar `VACUUM ... RETAIN 0 HOURS` mientras otro trabajo de Spark está leyendo la tabla causará que la consulta falle con un error fatal de `FileNotFoundException` tan pronto intente leer un archivo que acaba de ser eliminado físicamente.

Anti-patrón 2: Borrar Archivos Parquet Manualmente desde la Consola de la Nube Borrar directamente archivos `.parquet` del bucket eludiendo el registro de transacciones o el árbol de manifiestos corrompe la consistencia de la tabla. Al consultar la tabla, el motor asumirá que los archivos declarados en los metadatos existen y fallará la lectura.

Anti-patrón 3: Z-Ordering sobre Más de 3 o 4 Columnas de Alta Cardinalidad La curva de Morton de Z-Order pierde drásticamente su efectividad a medida que se incrementa el número de dimensiones. En Delta Lake moderno, migre hacia **Liquid Clustering (`CLUSTER BY`)** para evitar este problema.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 18.1 (Control de Concurrencia Optimista en Operaciones Concurrentes):
Dos pipelines de Spark (`Job_A` y `Job_B`) escriben en la misma tabla Delta `telemetria`:

1. Si ambos jobs ejecutan simultáneamente `df.write.format("delta").mode("append").save(ruta)` sobre la misma fecha, ¿Delta Lake lanza una excepción de conflicto concurrente bajo el nivel `WriteSerializable`?

2. ¿Bajo qué escenario exacto de operaciones concurrentes sí se aborta la transacción lanzando un conflicto de concurrencia?

Solución 18.1:

1. *Escenario de Append + Append:* **No hay conflicto.** Bajo el nivel de aislamiento `WriteSerializable`, Delta Lake permite anexiones concurrentes ciegas (*Blind Appends*). Como ninguna de las dos operaciones realiza lecturas para modificar o borrar filas existentes, Delta asigna números de commit secuenciales consecutivos (ej. versión 101 para `Job_A` y versión 102 para `Job_B`), confirmando ambas escrituras sin lanzar ninguna excepción.
2. *Escenario de Conflicto Real:* Se produce cuando una transacción ejecuta una operación condicional (`MERGE INTO`, `UPDATE` o `DELETE`) que lee un subconjunto de archivos Parquet para determinar qué filas alterar, y concurrentemente otro proceso modifica o elimina esos mismos archivos físicos antes de que el primer job confirme su commit. En ese instante, Delta detecta una colisión de lectura-escritura (*Read-Write Conflict*) y aborta la transacción con una excepción `ConcurrentAppendException` o `ConcurrentTransactionException`.

Ejercicio 18.2 (Particionamiento Oculto de Iceberg frente a Parquet Clásico):
Una tabla almacena telemetría con la columna `timestamp_evento TIMESTAMP`. En Hive/Parquet tradicional, los ingenieros particionaban creando una columna derivada `fecha STRING`.

1. ¿Qué riesgo introduce este esquema tradicional si un usuario nuevo filtra `WHERE timestamp_evento = '2026-10-12 14:00:00'` omitiendo la columna sintética `fecha`?
2. ¿Cómo resuelve Apache Iceberg este problema mediante *Hidden Partitioning*?

Solución 18.2:

1. *Riesgo tradicional:* Si el usuario omite la columna sintética `fecha` en el filtro `WHERE`, el motor no puede aplicar *Partition Pruning* y ejecuta un **escaneo completo de la tabla (*Full Table Scan*)**, leyendo terabytes de datos innecesariamente y aumentando los costos y tiempos de ejecución.
2. *Solución de Iceberg:* Iceberg vincula la transformación de partición (`days(timestamp_evento)`) directamente a los metadatos de la columna original. Cuando el usuario consulta `WHERE timestamp_evento = '2026-10-12 14:00:00'`, el optimizador de Iceberg proyecta internamente la función temporal sobre la partición y lee **únicamente** el manifiesto correspondiente al día especificado, eliminando la duplicación de columnas y asegurando poda de datos automática y transparente.

18.7 Resumen y Puntos Clave

- **Semántica ACID en la nube:** Los formatos abiertos de tabla proporcionan transacciones ACID desacopladas del almacenamiento de objetos mediante registros de transacciones (Delta Lake) o árboles de manifiestos (Apache Iceberg).
- **Innovación en Delta Lake:** Liquid Clustering (`CLUSTER BY`) con la curva de Hilbert supera las limitaciones del particionado rígido tradicional, complementado por Deletion Vectors y Column Mapping.
- **Los 5 niveles de Apache Iceberg:** Catálogo → `metadata.json` → Manifest List (Avro) → Manifest Files (Avro) → Data Files (Parquet).
- **Mantenimiento y Concurrencia:** La compactación (`OPTIMIZE` / `rewrite_data_files`) combate el problema de archivos pequeños, mientras que `VACUUM` purga archivos obsoletos respetando el flag oficial de retención `retentionDurationCheck.enabled`.

Lecturas Recomendadas

1. **Armbrust, M., Das, T., Sun, L., et al. (2020).** *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.* Proceedings of the VLDB Endowment (PVLDB), 13(12), 3411-3424.
2. **Apache Iceberg Project (2024).** *Apache Iceberg Table Specification and System Procedures.* Apache Software Foundation (`https://iceberg.apache.org/spec/`).
3. **Databricks Documentation (2024).** *Use Liquid Clustering for Delta Tables.* Databricks Guides (`https://docs.databricks.com/delta/clustering.html`).

CAPÍTULO 18A

Plataforma Databricks: Arquitectura, Delta Live Tables, Photon y Workflows

RECOMENDADO

> *"Aprender Apache Spark sobre un clúster genérico enseña los fundamentos del cómputo distribuido; dominar Databricks enseña cómo opera la ingeniería de datos en las empresas más competitivas del mundo. Databricks no es un simple envoltorio para Spark: es una Plataforma de Inteligencia de Datos completa que desacopla la infraestructura, introduce ejecución vectorial en C++ con Photon, automatiza la ingesta continua con Auto Loader y transforma el código artesanal en pipelines declarativos con Delta Live Tables."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

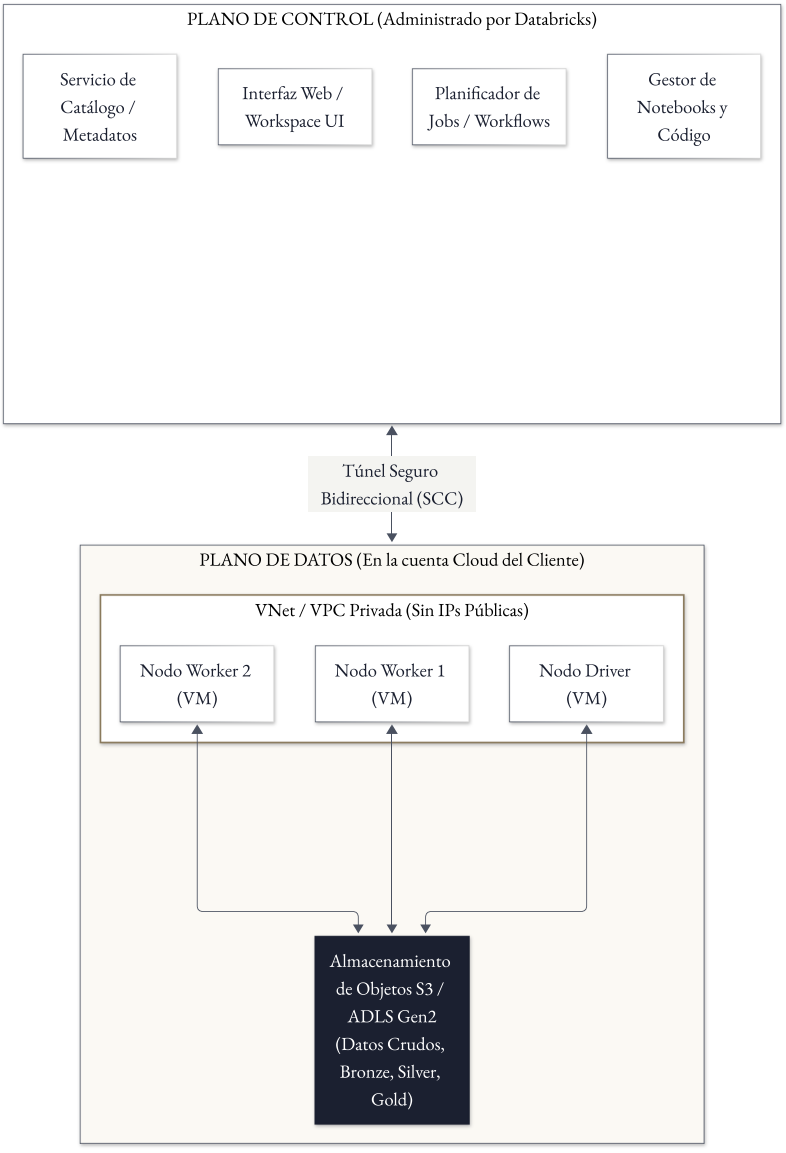
- Comprender la topología de red y seguridad de Databricks, diferenciando el **Plano de Control (*Control Plane*)** del **Plano de Datos (*Data Plane*)**, tanto en modalidad clásica como **Serverless**.
- Evaluar y seleccionar la tipología óptima de cómputo: clústeres interactivos (*All-Purpose*), clústeres de trabajo efímeros (*Job Clusters*) y almacenes de datos (*SQL Warehouses*).
- Explicar el funcionamiento interno del motor de cómputo vectorizado **Photon** y sus ventajas de rendimiento frente a la JVM de Apache Spark.
- Implementar ingesta incremental de alta escala en almacenamiento de objetos con **Auto Loader** (`cloudFiles`), administrando evolución de esquemas y la columna de rescate `_rescued_data`.
- Construir pipelines declarativos con **Delta Live Tables (DLT)**, aplicando expectativas de calidad automáticas y procesando Change Data Capture (CDC) con `dlt.apply_changes()` para SCD Tipo 1 y 2.
- Orquestar dependencias complejas mediante **Databricks Workflows**, pasando parámetros dinámicos entre tareas con `taskValues`.
- Desarrollar y desplegar pipelines con estándares profesionales de software mediante **Databricks Asset Bundles (DABs)** y el SDK de Python (`databricks-sdk`).

Definición 18A.1 (Plataforma de Inteligencia de Datos - Databricks) Databricks es una plataforma unificada en la nube (AWS, Microsoft Azure y Google Cloud Platform) diseñada sobre una arquitectura **Lakehouse** centrada en **Delta Lake** y gobernada por **Unity Catalog**.

A diferencia de los clústeres Spark tradicionales donde el ingeniero debe administrar dependencias del sistema operativo, aprovisionamiento de nodos, conectividad y balanceo manual de recursos, Databricks provee una capa de abstracción empresarial que optimiza el almacenamiento, vectoriza el motor de cómputo en C++ (Photon), automatiza la orquestación (Workflows) y asegura gobernanza unificada sobre datos, modelos de IA y código.

18A.1 Arquitectura del Workspace: Control Plane frente a Data Plane

La seguridad y el aislamiento de datos en Databricks se fundamentan en la separación estricta entre dos planos arquitectónicos:



1. Plano Clásico (Classic Compute)

- **Plano de Control (*Control Plane*):** Se aloja en la infraestructura propia de Databricks. Aloja la interfaz web, el servicio de gestión de usuarios, el planificador de trabajos (*Job Scheduler*) y el

almacenamiento de código fuente. **Los datos analíticos del cliente nunca residen en el plano de control**.

- **Plano de Datos (*Data Plane*):** Se ejecuta dentro de la suscripción o cuenta en la nube del propio cliente (AWS VPC, Azure VNet o GCP VPC). Allí se aprovisionan las máquinas virtuales que ejecutan el Driver y los Workers de Spark, garantizando que los datos en reposo y en tránsito residan dentro del perímetro de seguridad corporativo.
- **Conectividad de Clúster Segura (*Secure Cluster Connectivity - SCC / No Public IPs*):** Los nodos de cómputo no requieren direcciones IP públicas ni puertos abiertos de entrada en el cortafuegos. El clúster inicia conexiones salientes cifradas hacia el plano de control mediante un túnel de retransmisión seguro (*Relay Tunnel*).

2. Cómputo Serverless (Serverless Compute)

En la modalidad **Serverless** (disponible para SQL Warehouses, Notebooks interactivos y Jobs), el plano de datos es administrado por Databricks en un entorno multi-inquilino altamente aislado mediante micro-máquinas virtuales seguras:

- **Ventaja Operativa:** El clúster arranca en **menos de 10 segundos** (frente a los 4–7 minutos que tarda aprovisionar instancias de máquinas virtuales tradicionales en la nube).
- **Eficiencia Financiera:** El escalado descendente a cero es casi instantáneo tras unos segundos de inactividad, evitando pagar por capacidad ociosa.

18A.2 Tipología de Clústeres y Dimensionamiento de Cómputo

Databricks ofrece tres categorías principales de recursos de cómputo:

Tipo de Cómputo	Caso de Uso Principal	Modo de Facturación	Retención de Estado
All-Purpose Cluster	Desarrollo interactivo, exploración en notebooks y depuración ad-hoc.	Tarifa DBU interactiva alta (~$0.40 - $0.55/DBU).	Manual o auto-terminación por inactividad (ej. 20 min).
Job Cluster (Efímero)	Cargas de producción automatizadas y programadas en Workflows.	**Tarifa DBU de trabajo reducida (~$0.15 - $0.20/DBU).**	**Efímero:** Se crea para ejecutar el job y se destruye inmediatamente al finalizar.
SQL Warehouse	Consultas analíticas SQL para analistas, herramientas de BI (Power BI, Tableau) y dbt.	Facturado por tamaño de almacén (2X-Small a 4X-Large) y tiempo de ejecución.	Serverless con auto-escalado horizontal de múltiples clústeres.

Regla de Oro FinOps en Databricks:

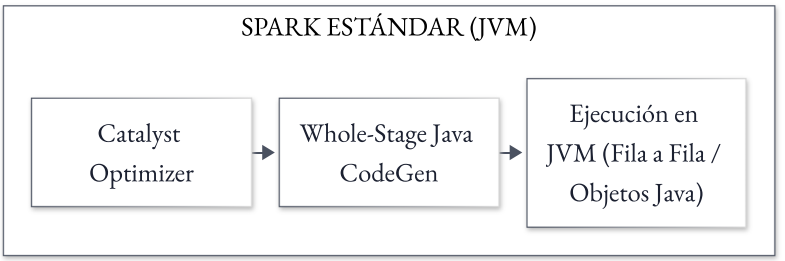
> **Jamás ejecute un pipeline programado de producción en un clúster All-Purpose.** Ejecutar un DAG diario en un clúster interactivo representa un sobrecosto directo del **65% al 70%** en la factura mensual de Databricks por exactamente el mismo cómputo físico.

18A.3 El Motor Photon: Ejecución Vectorizada en C++

Apache Spark clásico compila las consultas analíticas a bytecode de Java ejecutado sobre la Máquina Virtual de Java (**JVM**). Aunque Catalyst y Tungsten optimizan la memoria, la JVM sufre de penalizaciones inherentes: sobrecosto de recolección de basura (*Garbage Collection*), limitaciones de optimización del compilador JIT y pobre aprovechamiento de las instrucciones vectoriales modernas de la CPU.

Photon es el motor de ejecución nativo desarrollado desde cero por Databricks en **C++**:





- **Vectorización SIMD (*Single Instruction, Multiple Data*):** En lugar de evaluar operadores fila a fila, Photon procesa bloques columnares contiguos en memoria, aprovechando los registros vectoriales del procesador (AVX-512) para evaluar filtros y sumas sobre múltiples valores numéricos en un solo ciclo de reloj.

- **Gestión de Memoria Explícita:** Photon administra su propia memoria fuera de la JVM (*Off-Heap C++*), eliminando las pausas de recolección de basura (*GC pauses*) que provocan inestabilidad en grandes agregaciones y joins.
- **¿Cuándo Activar Photon?** Es sumamente rentable en cargas que involucran escaneo masivo de Parquet/Delta, agregaciones pesadas y joins de alta cardinalidad. No aporta beneficio significativo en transformaciones que dependan de UDFs de Python puras o I/O lento con APIs externas.

18A.4 Ingesta Continua con Auto Loader (`cloudFiles`)

Al ingestar terabytes de archivos continuos en la capa Bronze desde Amazon S3 o Azure ADLS Gen2, ejecutar un `spark.read.parquet()` tradicional requiere listar la totalidad del directorio en cada ejecución. Cuando un bucket acumula millones de archivos, el comando `LIST` puede tardar más de una hora solo en descubrir qué archivos son nuevos.

Auto Loader resuelve este problema procesando billones de archivos de forma incremental y económica:

PYTHON

```python
# Ingesta incremental de alta resiliencia con Auto Loader en PySpark
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

ruta_almacenamiento_crudo = "s3://empresa-telemetria-cruda/eventos_json/"
ruta_destino_bronze = "s3://empresa-lakehouse-prod/bronze/eventos_telemetria/"
ruta_metadatos_esquema = "s3://empresa-lakehouse-
prod/metadatos/checkpoint_autoloader/"

# Flujo de lectura incremental con Auto Loader
df_stream_crudo = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .option("cloudFiles.schemaLocation", ruta_metadatos_esquema) \
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns") \
    .option("cloudFiles.inferColumnTypes", "true") \
    .load(ruta_almacenamiento_crudo)

# Escritura continua a Delta Lake Bronze con preservación de datos corruptos
query_escritura = df_stream_crudo.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", f"{ruta_metadatos_esquema}/delta_checkpoint") \
    .start(ruta_destino_bronze)
```

Características Críticas de Auto Loader:

1. **La Columna de Rescate (`_rescued_data`):** Si un evento entrante contiene un campo que no coincide con el tipo inferido (por ejemplo, un string `"ERROR"` en una columna entera), Auto Loader **no aborta el pipeline**. Guarda los campos incompatibles o corruptos en una columna oculta `_rescued_data` en formato JSON, permitiendo a los ingenieros auditar anomalías sin perder registros.
2. **Evolución Automática de Esquemas:** Con `.option("cloudFiles.schemaEvolutionMode", "addNewColumns")`, si el sistema de origen agrega una nueva columna al JSON, Auto Loader altera automáticamente la tabla Delta de destino agregando la nueva columna sin intervención manual.
3. **Modos de Notificación de Archivos:**
   - *Listado de Directorios Incremental (Directory Listing):* Ideal para directorios bien particionados con menos de unos pocos millones de archivos.
   - *Notificación de Eventos (File Notification):* Configura colas de mensajería en la nube (AWS SQS / Azure Event Grid) para que cada archivo depositado notifique instantáneamente a Databricks, reduciendo el costo de llamadas a la API de S3 a cero.

Delta Live Tables (DLT) —integrado en la suite moderna **Databricks Lakeflow Pipelines** junto con Lakeflow Connect y Lakeflow Jobs— es el framework declarativo nativo de Databricks para construir canalizaciones ETL fiables y gobernadas. En lugar de escribir lógica imperativa de lectura, control de checkpoints, gestión de reintentos y llamadas a `save()`, el ingeniero declara las tablas y vistas deseadas:

> **Evolución Editorial (Databricks Lakeflow):** Databricks unificó su ecosistema de ingeniería de datos bajo la marca **Lakeflow**:
>
> 1. **Lakeflow Connect:** Ingesta nativa gobernada desde bases de datos operacionales (PostgreSQL, MySQL, Salesforce) mediante CDC automático.
> 2. **Lakeflow Pipelines:** El motor declarativo de transformación basado en la tecnología de Delta Live Tables (DLT).
> 3. **Lakeflow Jobs:** El orquestador unificado de tareas y dependencias construido sobre Databricks Workflows.



1. Sintaxis en Python con Expectativas de Calidad

PYTHON

```python
import dlt
from pyspark.sql import functions as F

# Capa Bronze: Ingesta continua declarativa
@dlt.table(
    name="pedidos_bronze",
    comment="Tabla cruda ingesta incremental desde almacenamiento de objetos",
    table_properties={"quality": "bronze"}
)
def pedidos_bronze():
```

```
    return spark.readStream.format("cloudFiles") \
        .option("cloudFiles.format", "json") \
        .load("s3://bucket-origen/pedidos/")

# Capa Silver: Validación y reglas de calidad (Data Expectations)
@dlt.table(
    name="pedidos_silver",
    comment="Tabla conformada con deduplicación y reglas de integridad"
)
@dlt.expect_or_drop("monto_positivo", "monto_total_usd > 0")
@dlt.expect_or_fail("identificador_obligatorio", "pedido_id IS NOT NULL")
@dlt.expect("cliente_valido", "cliente_id > 0") # Alerta en métricas, pero
preserva la fila
def pedidos_silver():
    return dlt.read_stream("pedidos_bronze") \
        .select(
            F.col("pedido_id").cast("string"),
            F.col("cliente_id").cast("long"),
            F.col("monto_total_usd").cast("double"),
            F.col("timestamp_creacion").cast("timestamp")
        )
```

2. Modos de Expectativas de Calidad en DLT:

- `@dlt.expect("regla", "condicion")`: Registra métricas de cumplimiento en el catálogo de eventos de DLT. Si la fila no cumple, se contabiliza como violación pero **la fila se mantiene** en la tabla.
- `@dlt.expect_or_drop("regla", "condicion")`: Si el registro viola la condición, se descarta silenciosamente de la salida y se registra en los indicadores de calidad.
- `@dlt.expect_or_fail("regla", "condicion")`: Si una sola fila incumple la condición, **todo el pipeline se detiene inmediatamente con fallo crítico**, garantizando que datos corruptos jamás contaminen la tabla.

3. CDC Declarativo con `dlt.apply_changes()` (SCD Tipo 1 y 2)

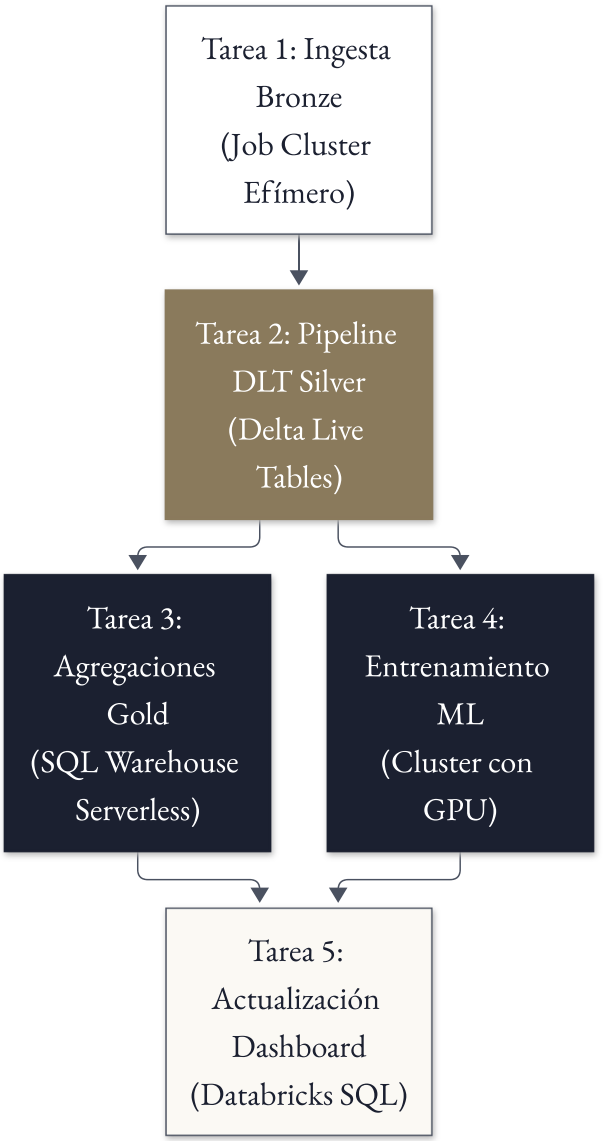
Implementar dimensiones que cambian lentamente (SCD) o fusionar cambios CDC desde Debezium requiere escribir complejas sentencias `MERGE INTO`. En DLT, se resuelve en una sola declaración:

```python
# Procesamiento declarativo de CDC sin escribir MERGE manual
dlt.create_streaming_table(name="clientes_silver_scd2")

dlt.apply_changes(
    target = "clientes_silver_scd2",
    source = "clientes_cdc_bronze",
    keys = ["cliente_id"],
    sequence_by = F.col("timestamp_modificacion"),
    apply_as_deletes = F.expr("operacion = 'DELETE'"),
    except_column_list = ["operacion", "timestamp_modificacion"],
    stored_as_scd_type = 2  # Crea automáticamente valido_desde, valido_hasta y
es_actual
)
```

18A.6 Databricks Workflows: Orquestación Nativa de Producción

Databricks Workflows es el orquestador nativo de la plataforma. Permite coordinar tareas heterogéneas (notebooks, scripts Python, consultas SQL, pipelines DLT y modelos de dbt) en un grafo acíclico dirigido (DAG) sin necesidad de mantener servidores externos de Apache Airflow:



Paso de Parámetros Dinámicos entre Tareas (`taskValues`)

Una tarea previa puede compartir valores computados (como la cantidad de registros procesados o una fecha de partición) con tareas posteriores mediante la utilidad de sistema `dbutils`:

```python
# En la Tarea 1 (Generadora del valor):
dbutils.jobs.taskValues.set(key="total_procesados", value=145000)
dbutils.jobs.taskValues.set(key="lote_fecha", value="2026-03-31")

# En la Tarea 2 (Consumidora del valor):
total = dbutils.jobs.taskValues.get(taskKey="Tarea1_Ingesta",
key="total_procesados", default=0)
fecha = dbutils.jobs.taskValues.get(taskKey="Tarea1_Ingesta", key="lote_fecha",
default="1970-01-01")
print(f"Recibidos {total} registros correspondientes a la fecha {fecha}")
```

18A.7 Databricks Asset Bundles (DABs) y Automatización con SDK

El desarrollo profesional en Databricks ha dejado atrás la edición manual de notebooks en el navegador. El estándar de ingeniería moderno utiliza **Databricks Asset Bundles (DABs)**: infraestructura, código y pipelines definidos como código en archivos YAML versionados en Git y editados en entornos locales como Visual Studio Code.

Estructura de un Proyecto con DABs (`databricks.yml`)

```yaml
bundle:
  name: plataforma_lakehouse_prod

include:
  - resources/*.yml

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://adb-dev-instance.azuredatabricks.net

  prod:
    mode: production
    workspace:
      host: https://adb-prod-instance.azuredatabricks.net
    run_as:
      service_principal_name: sp-lakehouse-pipeline-automation
```

Definición Declarativa del Job (`resources/pipeline_job.yml`)

YAML

```yaml
resources:
  jobs:
    lakehouse_diario_job:
      name: "Lakehouse_Diario_${bundle.target}"
      schedule:
        quartz_cron_expression: "0 0 4 * * ?"
        timezone_id: "UTC"
      tasks:
        - task_key: ingesta_autoloader
          job_cluster_key: cluster_optimo
          python_wheel_task:
            package_name: mi_paquete_datos
            entry_point: run_ingesta
      job_clusters:
        - job_cluster_key: cluster_optimo
          new_cluster:
            spark_version: 15.4.x-photon-scala2.12
            node_type_id: i3.xlarge
            num_workers: 4
```

Comandos del CLI para Despliegues Multi-Entorno

BASH

```bash
# Validar la sintaxis de los recursos y dependencias
databricks bundle validate

# Desplegar al entorno de desarrollo
databricks bundle deploy -t dev

# Desplegar a producción mediante integración continua (CI/CD)
databricks bundle deploy -t prod

# Disparar ejecución manual de prueba del bundle desplegado
databricks bundle run lakehouse_diario_job -t dev
```

18A.8 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Almacenar Datos de Producción en el DBFS Raíz (`dbfs:/`)
El Databricks File System raíz (`dbfs:/`) es un almacenamiento temporal asociado al workspace. Almacenar tablas de negocio allí impide la segregación de permisos de red, dificulta auditorías de seguridad y expone los datos a pérdida accidental si el workspace se reconstruye.
Corrección: Almacene todos los datos en **Volumes de Unity Catalog** o tablas externas vinculadas directamente a sus buckets de almacenamiento de objetos gestionados.

Anti-patrón 2: Dejar Clústeres All-Purpose sin Apagado Automático (*Auto-Termination*)
Crear un clúster de 8 nodos para exploración y olvidar configurarle un tiempo de auto-terminación (ej. 15–20 minutos de inactividad) mantendrá las máquinas virtuales encendidas todo el fin de semana, facturando miles de dólares innecesariamente.

Anti-patrón 3: Utilizar `dlt.read()` en Lugar de `dlt.read_stream()` en Pipelines Incrementales
En Delta Live Tables, invocar `dlt.read("tabla_origen")` fuerza a la tabla a comportarse como un lote completo (*batch*), recomputando la tabla entera en cada ejecución. Para procesar únicamente las novedades incrementales de forma continua, se debe utilizar estrictamente `dlt.read_stream("tabla_origen")`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 18A.1 (Diseño de Pipeline Declarativo con DLT y Rescate de Errores):
Una fintech recibe archivos JSON de transferencias bancarias en S3.

1. Diseñe el código de un pipeline DLT en Python que ingeste continuamente los archivos con Auto Loader.
2. Defina una regla de calidad en la capa Silver que descarte registros donde el monto sea menor o igual a cero, pero que detenga de forma crítica todo el pipeline si el campo `cuenta_origen` es nulo.

Solución 18A.1:

PYTHON

```python
import dlt
from pyspark.sql import functions as F

@dlt.table(
    name="transferencias_bronze",
    comment="Ingesta streaming de transacciones con Auto Loader"
)
def transferencias_bronze():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", "s3://fintech-
checkpoints/transferencias_schema")
        .load("s3://fintech-landing/transferencias/")
    )

@dlt.table(
    name="transferencias_silver",
    comment="Capa Silver validada y tipada"
)
```

```
@dlt.expect_or_drop("monto_positivo_requerido", "monto > 0")
@dlt.expect_or_fail("cuenta_origen_obligatoria", "cuenta_origen IS NOT NULL")
def transferencias_silver():
    return (
        dlt.read_stream("transferencias_bronze")
        .select(
            F.col("transferencia_id").cast("string"),
            F.col("cuenta_origen").cast("string"),
            F.col("cuenta_destino").cast("string"),
            F.col("monto").cast("decimal(12,2)"),
            F.col("timestamp_operacion").cast("timestamp")
        )
    )
```

18A.9 Resumen y Puntos Clave

- **Separación de planos:** El Plano de Control gestiona metadatos y orquestación; el Plano de Datos ejecuta las máquinas virtuales en la nube del cliente preservando soberanía y seguridad.
- **Photon vectorizado:** Reemplaza la ejecución tradicional de la JVM por un motor nativo en C++ con aceleración SIMD en registros de procesador, acelerando consultas de gran escala.
- **Auto Loader:** Descubre e ingesta millones de archivos en S3/ADLS de forma incremental, aislando esquemas incompatibles en la columna `_rescued_data`.
- **Delta Live Tables (DLT):** Define canalizaciones declarativas integrando gobernanza de calidad con `@dlt.expect` y CDC automático con `dlt.apply_changes()`.
- **Workflows y DABs:** Proveen orquestación de nivel empresarial e implementación mediante infraestructura como código con `databricks.yml` y CI/CD.

Lecturas Recomendadas

1. **Databricks Documentation (2024).** *Databricks Architecture Overview: Control Plane and Data Plane Security.* Databricks Guides.
2. **Armbrust, M. et al. (2020).** *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.* Proceedings of the VLDB Endowment.
3. **Databricks Engineering (2022).** *Photon: A Fast Query Engine for Lakehouse Systems.* ACM SIGMOD Conference.
4. **Databricks Learning (2024).** *Developing Data Pipelines with Delta Live Tables and Asset Bundles.*

CAPÍTULO 19

Gobernanza, Linaje y Unity Catalog en Lakehouse RECOMENDADO

> *"En arquitecturas analíticas a escala de petabytes, un lago de datos desprovisto de gobernanza se degrada inevitablemente en un pantano de datos (Data Swamp). La gobernanza moderna no es una traba burocrática; es el plano de control centralizado que garantiza auditabilidad, trazabilidad de linaje y políticas homogéneas de seguridad sobre múltiples motores y nubes."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la arquitectura de gobernanza desacoplada de **Unity Catalog** (tanto en su servicio gestionado como en su especificación de código abierto bajo la Linux Foundation).
- Dominar la topología de **Metastores Regionales** y el espacio de nombres jerárquico de tres niveles (*Three-Level Namespace: catalog.schema.table*).
- Administrar credenciales de almacenamiento (*Storage Credentials*) y ubicaciones externas (*External Locations*) mediante Infraestructura como Código (IaC).
- Configurar control de acceso basado en roles (RBAC) utilizando los privilegios modernos `USE CATALOG` y `USE SCHEMA`.
- Diseñar políticas de seguridad avanzadas: filtros a nivel de fila (*Row Filters*) y enmascaramiento dinámico a nivel de columna (*Column Masking*) preservando la coherencia estricta de tipos de datos.
- Auditar el ciclo de vida de tablas gestionadas (borrado suave con `UNDROP`) y el linaje automático de datos (*Data Lineage*) generado por Catalyst.

Definición 19.1 (Gobernanza Unificada y Unity Catalog)

Unity Catalog es una plataforma de gobernanza universal de código abierto (donada por Databricks a la Linux Foundation en 2024) y disponible también como servicio totalmente administrado.

Centraliza la administración de permisos, auditoría y linaje para tablas (Delta Lake, Apache Iceberg), volúmenes de archivos no estructurados y modelos de machine learning. Opera desacoplando por completo las credenciales físicas de almacenamiento en la nube (AWS IAM, Azure Managed Identities, GCP Service Accounts) del plano lógico de consulta accesible por motores analíticos heterogéneos (Spark, Trino, DuckDB, Presto).

19.1 Arquitectura de Metastores Regionales y el Espacio de Nombres

Un principio fundamental de la arquitectura en la nube es que **un Metastore de Unity Catalog es una entidad regional**: reside físicamente en una región cloud específica (por ejemplo, `us-east-1` o `eastus2`) y se asocia a los espacios de trabajo (*workspaces*) y redes desplegados en esa misma región para evitar latencia de red y costos de transferencia inter-región (*Egress*).

A nivel lógico, la plataforma organiza todos los activos bajo un estándar de tres niveles:

```
[ Cuenta Corporativa ]
        |
        ▼ (Asignación por región)
[ Metastore Regional (ej. us-east-1) ]
        |
        ├── [ Catálogo: dev ]
        ├── [ Catálogo: staging ]
        └── [ Catálogo: prod_lakehouse ]
                  |
                  ├── [ Esquema: finanzas ]
                  └── [ Esquema: logistica ]
                            |
                            ├── [ Tablas: Delta / Iceberg ]
                            ├── [ Vistas Lógicas ]
                            ├── [ Volúmenes: Archivos Crudos ]
                            └── [ Modelos: MLflow ]
```

- **Catálogo (`catalog`):** Primer nivel de aislamiento lógico. Se utiliza habitualmente para segregar entornos de ciclo de vida (`dev`, `staging`, `prod`) o grandes unidades de negocio autónomas.
- **Esquema / Base de Datos (`schema`):** Segundo nivel de organización que agrupa tablas y activos con afinidad funcional (ej. `contabilidad`, `rrhh`, `ventas`).

- **Objetos Securizables:** Tercer nivel donde residen las tablas estructuradas, vistas, volúmenes para archivos no estructurados (documentos, imágenes) y modelos analíticos.

19.2 Desacoplamiento de Almacenamiento: Credenciales y External Locations

Una de las innovaciones arquitectónicas clave de la gobernanza moderna es que **los usuarios y analistas jamás reciben credenciales IAM directas** hacia los buckets de almacenamiento en la nube.

El acceso físico se administra mediante dos abstracciones:

1. **Credencial de Almacenamiento (*Storage Credential*):** Encapsula un rol de AWS IAM o una Identidad Administrada de Azure que posee permisos en el almacenamiento cloud.
2. **Ubicación Externa (*External Location*):** Vincula una ruta URL de almacenamiento específica (ej. `s3://corp-lakehouse-prod/finanzas/`) con una credencial de almacenamiento autorizada.

En entornos de producción, estas entidades se aprovisionan de forma declarativa mediante **Terraform** utilizando el proveedor oficial `databricks`:

HCL (TERRAFORM)

```hcl
# Aprovisionamiento de Storage Credential y External Location en Terraform
resource "databricks_storage_credential" "credencial_aws" {
  name = "credencial_iam_finanzas"
  aws_iam_role {
    role_arn = "arn:aws:iam::123456789012:role/UnityCatalogStorageRole"
  }
  comment = "Credencial IAM delegada para el lago de finanzas"
}

resource "databricks_external_location" "ubicacion_finanzas" {
  name            = "ext_loc_finanzas"
  url             = "s3://corp-lakehouse-prod/finanzas/"
  credential_name = databricks_storage_credential.credencial_aws.id
  comment         = "Ubicacion externa para datos de finanzas"
}
```

Equivalente en sintaxis DDL de SQL:

```sql
-- Declaración de credencial de almacenamiento en SQL nativo
CREATE STORAGE CREDENTIAL credencial_iam_finanzas
  AWS_IAM_ROLE (role_arn =
'arn:aws:iam::123456789012:role/UnityCatalogStorageRole');

CREATE EXTERNAL LOCATION ext_loc_finanzas
  URL 's3://corp-lakehouse-prod/finanzas/'
  WITH STORAGE CREDENTIAL credencial_iam_finanzas;
```

19.3 Control de Acceso con Privilegios Modernos: `USE CATALOG` y `USE SCHEMA`

En la especificación contemporánea de Unity Catalog, la navegación por la jerarquía se gobierna explícitamente mediante los privilegios `USE CATALOG` y `USE SCHEMA`:

```sql
-- 1. Contexto de ejecución
USE CATALOG prod_lakehouse;
CREATE SCHEMA IF NOT EXISTS contabilidad;

-- 2. Otorgar permisos de navegación jerárquica y lectura a los analistas
GRANT USE CATALOG ON CATALOG prod_lakehouse TO `grupo_analistas_riesgo`;
GRANT USE SCHEMA ON SCHEMA prod_lakehouse.contabilidad TO
`grupo_analistas_riesgo`;
GRANT SELECT ON TABLE prod_lakehouse.contabilidad.transacciones_cuentas TO
`grupo_analistas_riesgo`;

-- 3. Otorgar permisos de escritura a los pipelines de ingeniería
GRANT MODIFY, SELECT ON SCHEMA prod_lakehouse.contabilidad TO
`service_principal_etl`;
```

19.4 Seguridad Avanzada: Row Filters y Column Masking con Tipado Estricto

Para cumplir normativas regulatorias internacionales (RGPD, HIPAA, PCI-DSS) sin duplicar datos en tablas separadas, Unity Catalog proporciona **enmascaramiento dinámico de columnas (*Column Masking*)** y **filtros a nivel de fila (*Row Filters*)**:

Regla Fundamental de Enmascaramiento de Columnas

La función UDF que aplica la máscara **debe declarar explícitamente la cláusula `RETURNS` y debe devolver exactamente el mismo tipo de datos primitivo de la columna que enmascara**. Si una columna es `DOUBLE`, la función debe devolver `DOUBLE` (por ejemplo, `NULL` o `0.0`); jamás puede devolver un `STRING` (`'CONFIDENCIAL'`), ya que causaría un error fatal de incompatibilidad de tipos durante la compilación de la consulta.

SQL

```
-- 1. Enmascaramiento dinámico para columna de tipo DOUBLE (Salarios)
CREATE OR REPLACE FUNCTION
prod_lakehouse.contabilidad.mascara_salario_double(salario_val DOUBLE)
RETURNS DOUBLE
RETURN CASE
    -- Solo miembros de gerencia ven el salario real
    WHEN is_account_group_member('gerentes_rrhh') THEN salario_val
    -- Los demás usuarios reciben NULL respetando el tipo numérico
    ELSE NULL
END;

ALTER TABLE prod_lakehouse.contabilidad.empleados
ALTER COLUMN salario_base_usd SET MASK
prod_lakehouse.contabilidad.mascara_salario_double;

-- 2. Enmascaramiento dinámico para columna STRING (Número de Tarjeta)
CREATE OR REPLACE FUNCTION prod_lakehouse.contabilidad.mascara_pan_tarjeta(pan
STRING)
RETURNS STRING
RETURN CASE
    WHEN is_account_group_member('oficiales_cumplimiento') THEN pan
    ELSE concat('XXXX-XXXX-XXXX-', right(pan, 4))
END;

ALTER TABLE prod_lakehouse.contabilidad.transacciones_cuentas
ALTER COLUMN numero_tarjeta SET MASK
prod_lakehouse.contabilidad.mascara_pan_tarjeta;

-- 3. Filtro a nivel de fila (Row Filter) para segregación por país
CREATE OR REPLACE FUNCTION prod_lakehouse.contabilidad.filtro_pais_empleado(pais
STRING)
RETURNS BOOLEAN
RETURN CASE
    WHEN is_account_group_member('auditores_globales') THEN true
    WHEN is_account_group_member('analistas_colombia') AND pais = 'CO' THEN true
    WHEN is_account_group_member('analistas_mexico') AND pais = 'MX' THEN true
    ELSE false
END;

ALTER TABLE prod_lakehouse.contabilidad.empleados
SET ROW FILTER prod_lakehouse.contabilidad.filtro_pais_empleado ON (pais);
```

19.5 Ciclo de Vida: Tablas Gestionadas vs. Externas y Recuperación con `UNDROP`

- **Tablas Gestionadas (*Managed Tables*):** Unity Catalog gobierna tanto los metadatos como el almacenamiento físico. Si un usuario autorizado ejecuta `DROP TABLE prod_lakehouse.contabilidad.empleados`, la tabla **no se destruye de inmediato físicamente**. Unity Catalog aplica un borrado suave con un **periodo de retención de 7 días**, permitiendo recuperarla intacta mediante:

  ```sql
  UNDROP TABLE prod_lakehouse.contabilidad.empleados;
  ```

  Una vez transcurridos los 7 días, un proceso de limpieza en segundo plano purga permanentemente los archivos en el almacenamiento cloud.
- **Tablas Externas (*External Tables*):** La tabla apunta a una ruta propia en un bucket mediante una *External Location*. Al ejecutar `DROP TABLE`, Unity Catalog elimina exclusivamente el registro de metadatos en el catálogo; **los archivos Parquet/Delta en el bucket cloud permanecen intactos**.

19.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Emplear Claves Secretas Hardcodeadas en Código de Pipelines Configurar cadenas de conexión directas o llaves IAM dentro de scripts elude completamente el motor de gobernanza. Todo acceso en producción debe canalizarse mediante credenciales y ubicaciones externas administradas por el catálogo.

Anti-patrón 2: Catálogo Monolítico Único Sin Segregación de Entornos Crear un único catálogo denominado `empresa` y mezclar esquemas de pruebas (`test_ventas`) con producción (`prod_ventas`) en el mismo metastore sin permisos segregados facilita que errores accidentales sobrescriban tablas productivas. El estándar corporativo exige catálogos aislados: `dev`, `staging` y `prod`.

Anti-patrón 3: Intentar Cambiar Tipos de Datos Dentro de Funciones de Enmascaramiento Intentar devolver una cadena de texto descriptiva (`'OCULTO'`) en una columna numérica viola el contrato de tipos de la tabla. Si los usuarios de negocio requieren una representación textual, debe construirse una vista semántica derivada sobre la tabla gobernada.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 19.1 (Jerarquía de Privilegios en Tres Niveles):
Un analista junior intenta ejecutar `SELECT * FROM prod_lakehouse.contabilidad.reporte_diario`. El administrador le otorgó `GRANT SELECT ON TABLE prod_lakehouse.contabilidad.reporte_diario TO junior_user;`. Sin embargo, la consulta falla indicando un error de permisos denegados. ¿Por qué ocurre esto y qué sentencias SQL adicionales deben ejecutarse?

Solución 19.1: En el modelo jerárquico de Unity Catalog, la autorización requiere permisos de navegación en **todos los niveles superiores** de la jerarquía antes de poder acceder al objeto final:

1. El usuario necesita permiso para atravesar el catálogo (`USE CATALOG`).
2. El usuario necesita permiso para atravesar el esquema (`USE SCHEMA`).

Sentencias requeridas para corregir el acceso:

```sql
GRANT USE CATALOG ON CATALOG prod_lakehouse TO `junior_user`;
GRANT USE SCHEMA ON SCHEMA prod_lakehouse.contabilidad TO `junior_user`;
```

Ejercicio 19.2 (Diseño de Enmascaramiento Dinámico con Coherencia de Tipos):
Se requiere gobernar una tabla `prod_rrhh.empleados.nomina` que almacena `empleado_id STRING`, `nombre STRING`, `pais STRING` y `salario_base_usd DOUBLE`.

1. Escribe la función de enmascaramiento SQL formal respetando la coherencia estricta de tipos de datos, asegurando que los miembros del grupo `gerentes_rrhh` vean el salario real y todos los demás usuarios vean `NULL`.
2. Asocia la máscara a la columna correspondiente de la tabla.

Solución 19.2:

```sql
-- 1. Definición formal de la función respetando RETURNS DOUBLE
CREATE OR REPLACE FUNCTION
prod_rrhh.empleados.mascara_salario_confidencial(salario_val DOUBLE)
RETURNS DOUBLE
RETURN CASE
    WHEN is_account_group_member('gerentes_rrhh') THEN salario_val
    ELSE NULL
END;

-- 2. Aplicación de la máscara a la columna de tipo DOUBLE
ALTER TABLE prod_rrhh.empleados.nomina
ALTER COLUMN salario_base_usd SET MASK
prod_rrhh.empleados.mascara_salario_confidencial;
```

Fundamento: Al declarar `RETURNS DOUBLE` y devolver `salario_val` o `NULL`, la consulta del usuario conserva la integridad del esquema analítico (la columna continúa siendo `DOUBLE`), evitando excepciones de tipo en herramientas de BI conectadas.

19.7 Resumen y Puntos Clave

- **Gobernanza unificada:** Unity Catalog unifica la administración de permisos, auditoría y linaje para tablas, archivos y modelos de machine learning sobre múltiples nubes y motores.
- **Metastores regionales:** Cada metastore reside en una región cloud específica y se asocia a los recursos de esa misma región para evitar sobrecostos de transferencia.
- **Privilegios modernos:** Se emplean `USE CATALOG` y `USE SCHEMA` para gobernar la navegación jerárquica obligatoria antes de conceder accesos granulares de lectura o modificación.
- **Enmascaramiento tipado:** Las funciones de enmascaramiento dinámico deben preservar estrictamente el tipo de dato de la columna protegida (`RETURNS `).
- **Protección ante borrado accidental:** Las tablas gestionadas implementan un periodo de recuperación de 7 días mediante el comando `UNDROP TABLE`.

Lecturas Recomendadas

1. **Databricks Engineering Documentation (2024).** *Unity Catalog Best Practices, Architecture and Privilege Model.* (`https://docs.databricks.com/data-governance/unity-catalog/index.html`).
2. **Linux Foundation (2024).** *Unity Catalog: Open Source Multi-Engine Data and AI Governance Specification.* Linux Foundation Projects (`https://unitycatalog.io/`).

3. **DAMA International (2017).** *DAMA-DMBOK: Data Management Body of Knowledge (2nd Edition).* Technics Publications.

CAPÍTULO 20

Catalogación, Descubrimiento y Contratos de Datos (Data Contracts)

RECOMENDADO

> *"El valor de una plataforma de datos es directamente proporcional a su capacidad de descubrimiento y a la fiabilidad de sus compromisos de entrega. Un lago de datos donde nadie sabe qué significa una columna, quién es su propietario o cuándo cambiará su esquema es un cementerio computacional. Los contratos de datos y la catalogación activa transforman datos caóticos en productos de software gobernados con interfaces formales."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Diseñar ecosistemas de metadatos modernos articulando la **tríada de metadatos**: técnicos, operativos y de negocio.
- Distinguir entre catálogos pasivos tradicionales y plataformas de **metadatos activos** (*Active Metadata*).
- Dominar el concepto, arquitectura y ciclo de vida de los **Contratos de Datos (*Data Contracts*)**.
- Redactar especificaciones formales de contratos de datos bajo el estándar abierto **ODCS** (*Open Data Contract Standard*) e implementarlos en dbt mediante `contract: {enforced: true}`.
- Automatizar la extracción programática de esquemas desde Apache Spark para alimentar buses de eventos de catalogación (DataHub, OpenMetadata).

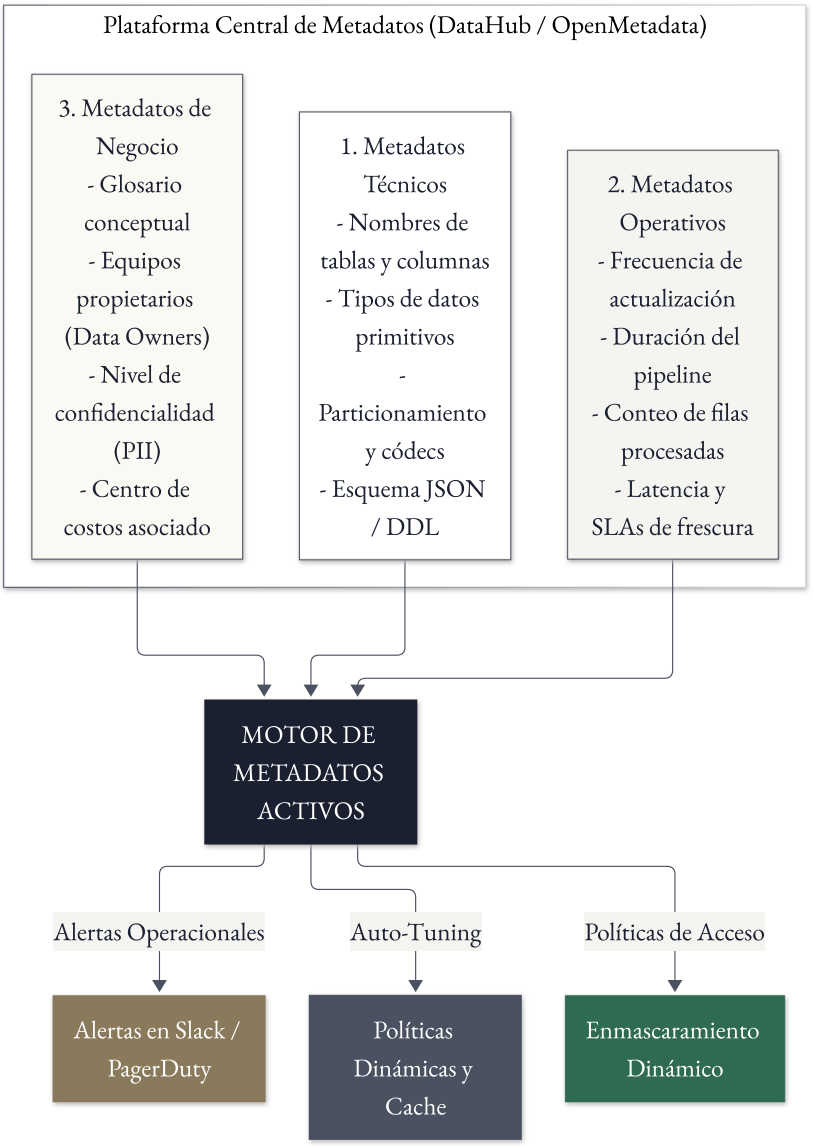
Definición 20.1 (Contrato de Datos / Data Contract)

Un **Contrato de Datos** es un acuerdo formal, declarativo y versionable entre el equipo productor de datos (desarrolladores de microservicios, equipos de producto o ingenieros de software transaccional) y los equipos consumidores (ingenieros de datos, analistas y científicos de datos).

Define con rigor de ingeniería el esquema de los datos, la semántica de negocio, los acuerdos de nivel de servicio (SLA de frescura y latencia), las clasificaciones de privacidad (PII) y las garantías de calidad que el productor se compromete a no romper sin una negociación previa y un versionado controlado.

20.1 La Tríada de Metadatos y el Catálogo Activo

El descubrimiento de datos empresarial descansa sobre la integración continua de tres categorías de información:



Catálogos Pasivos frente a Metadatos Activos (*Active Metadata*)

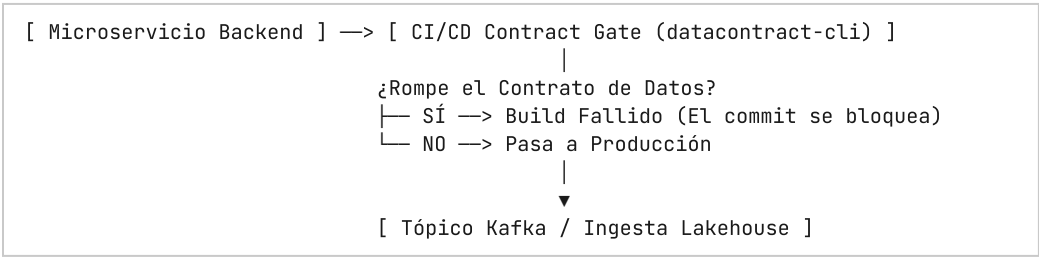
- **Catálogos Pasivos (Tradicionales):** Actúan como un "inventario estático de biblioteca". Requieren que los ingenieros o catalogadores ingresen descripciones manualmente en una interfaz web. Se desactualizan a los pocos días de ser implementados.
- **Metadatos Activos (Modernos):** El catálogo no solo almacena información; **acciona el ecosistema en tiempo de ejecución**. Si un pipeline produce un volumen anormalmente bajo de filas, el sistema de metadatos activos intercepta la anomalía, suspende los modelos analíticos aguas abajo, notifica a los responsables y ajusta temporalmente las políticas de retención.

20.2 Contratos de Datos: El Fin de las Roturas Silenciosas en Producción

El problema endémico de la ingeniería de datos clásica radica en que **las bases de datos transaccionales de las aplicaciones operacionales cambian sin previo aviso**:

- Un desarrollador backend renombra la columna `monto_pago` a `valor_transaccion` o cambia un formato de fecha de ISO-8601 a un entero Unix.
- La base de datos operacional sigue funcionando sin problema, pero los pipelines analíticos de Spark o dbt colapsan en la madrugada o, peor aún, propagan valores nulos silenciosos hacia los tableros de la junta directiva.

Los contratos de datos desplazan la responsabilidad de la calidad hacia la fuente (*Shift-Left Data Quality*):



20.3 Especificación Estándar de un Data Contract (ODCS v3.0)

El estándar **Open Data Contract Standard (ODCS)** define la estructura formal de un contrato en un archivo YAML legible por máquinas y humanos:

YAML

```yaml
# data-contract-ordenes.odcs.yaml
kind: DataContract
apiVersion: v3.0.0
id: urn:datacontract:ventas:ordenes_completadas
info:
  title: Contrato de Datos de Órdenes Transaccionales
  version: 2.1.0
  status: active
  description: Emisión formal de órdenes facturadas del servicio de Checkout.
  owner: equipo-backend-checkout@empresa.com

models:
  ordenes_facturadas:
    description: Registro canónico de órdenes que completaron el ciclo de pago.
```

```
    fields:
      orden_id:
        type: string
        required: true
        unique: true
        description: Identificador UUID v4 universal de la transacción.
      cliente_id:
        type: string
        required: true
        description: Clave foránea hacia el registro maestro de clientes.
      monto_total_centavos:
        type: integer
        required: true
        minimum: 1
        maximum: 100000000
        description: Monto en centavos para evitar imprecisiones de coma
flotante.
      divisa:
        type: string
        required: true
        enum: ['USD', 'COP', 'EUR', 'MXN']
      fecha_pago_utc:
        type: timestamp
        required: true

servicelevels:
  freshness:
    max: 15m
  availability:
    percentage: 99.9
```

Implementación en dbt con `contract: {enforced: true}`

Desde dbt Core v1.5+, los contratos de datos pueden forzarse a nivel de modelo compilado en SQL:

YAML

```yaml
# models/marts/fct_ordenes.yml
version: 2

models:
  - name: fct_ordenes
    config:
      contract:
        enforced: true
    columns:
      - name: orden_id
        data_type: string
        constraints:
          - type: not_null
          - type: primary_key
      - name: monto_total_usd
        data_type: numeric(18, 4)
        constraints:
          - type: not_null
      - name: fecha_cierre
        data_type: date
```

Si el modelo SQL `fct_ordenes.sql` devuelve un tipo incompatible (por ejemplo, si `orden_id` se genera como entero en lugar de `string`), o si omite alguna columna declarada, dbt **aborta la compilación antes de escribir en el Data Warehouse**, impidiendo la propagación de datos no conformes.

> **Consideraciones de Producción 20.1: Extracción Programática de Esquemas en PySpark**
>
> Para registrar automáticamente los metadatos de esquemas en catálogos abiertos (como OpenMetadata o DataHub), los pipelines deben inspeccionar programáticamente la estructura del DataFrame y publicar el esquema normalizado en JSON Schema:

PYTHON

```python
import json
from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType,
    IntegerType, BooleanType, DateType, DecimalType, TimestampType
)

spark = SparkSession.builder.appName("CatalogoExtraccionEsquema").getOrCreate()

# Definición del esquema productivo
esquema = StructType([
    StructField("transaccion_id", StringType(), False),
    StructField("cliente_id", StringType(), False),
    StructField("monto_usd", DecimalType(18, 2), True),
    StructField("es_fraude", BooleanType(), False),
```

```
    StructField("fecha_proceso", DateType(), False),
    StructField("timestamp_evento", TimestampType(), False)
])

# Inspección y serialización hacia formato canónico JSON Schema
def exportar_json_schema(struct_schema, titulo):
    properties = {}
    required = []

    for field in struct_schema.fields:
        tipo_spark = field.dataType.__class__.__name__

        # Mapeo exhaustivo de tipos de Spark a tipos canónicos de JSON Schema
        if tipo_spark in ("StringType", "VarcharType", "CharType"):
            schema_type = {"type": "string"}
        elif tipo_spark in ("IntegerType", "LongType", "ShortType", "ByteType"):
            schema_type = {"type": "integer"}
        elif tipo_spark in ("DoubleType", "FloatType", "DecimalType"):
            schema_type = {"type": "number"}
        elif tipo_spark == "BooleanType":
            schema_type = {"type": "boolean"}
        elif tipo_spark == "DateType":
            schema_type = {"type": "string", "format": "date"}
        elif tipo_spark in ("TimestampType", "TimestampNTZType"):
            schema_type = {"type": "string", "format": "date-time"}
        else:
            schema_type = {"type": "string"}

        schema_type["nullable"] = field.nullable
        properties[field.name] = schema_type

        if not field.nullable:
            required.append(field.name)

    return {
        "$schema": "https://json-schema.org/draft/2020-12/schema",
        "title": titulo,
        "type": "object",
        "properties": properties,
        "required": required
    }

esquema_json = exportar_json_schema(esquema, "TransaccionSilver")
print(json.dumps(esquema_json, indent=2))
```

20.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Catálogos Manuales en Hojas de Cálculo o Wikis Desconectadas Mantener diccionarios de datos en documentos estáticos compartidos garantiza su obsolescencia en semanas. La catalogación debe gestionarse como código (*Metadata-as-Code*) e integrarse al ciclo de vida del pipeline mediante hooks de CI/CD.

Anti-patrón 2: Contratos de Datos sin Validación Automatizada en el CI/CD del Productor Firmar un contrato en un documento PDF sin un validador automatizado (como `datacontract-cli` o GitHub Actions en el repositorio del backend) es una declaración de intenciones vacía. Si el commit del microservicio no se bloquea al romper el contrato, el contrato no existe operativamente.

Anti-patrón 3: Activos de Datos Huérfanos (*Orphan Data Assets*) Permitir la creación de tablas y pipelines en el catálogo sin declarar explícitamente un equipo propietario (*Owner*) y un canal de comunicación de soporte. Ante cualquier incidente crítico, nadie asume la responsabilidad de reparación.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 20.1 (Diseño de Compuerta de Contrato en CI/CD):
Un equipo backend modifica el servicio de pagos: cambian el tipo de dato de `monto` de `DECIMAL(12,2)` a un número flotante de precisión simple `FLOAT`. Describe las consecuencias que tendría esta alteración en los modelos de finanzas aguas abajo y cómo un contrato de datos previene este incidente antes del despliegue a producción.

Solución 20.1:

1. *Consecuencias operativas:* El tipo `FLOAT` almacena números binarios en coma flotante según IEEE 754, lo que introduce errores de redondeo infinitesimales (por ejemplo, `0.1 + 0.2 = 0.30000000000000004`). En conciliaciones financieras de millones de transacciones, estos micro-errores acumulan discrepancias contables graves que impiden cuadrar los balances tributarios de fin de mes.
2. *Prevención con Contrato de Datos:*
   - En el repositorio del microservicio existe un archivo `datacontract.yaml` que estipula `monto: decimal(12,2)` con precisión estricta.
   - Durante el pipeline de Integración Continua (CI) del backend, una herramienta como `datacontract-cli test` compara el esquema exportado del servicio con el contrato.
   - La suite detecta la discrepancia de tipos y emite un fallo bloqueante (`BREAKING CHANGE DETECTED: Column 'monto' type changed from decimal to float`). El Pull Request no puede fusionarse a la rama principal hasta que el backend rectifique el cambio.

Ejercicio 20.2 (Diferenciación entre Catálogos Activos y Pasivos):
Presenta un caso de uso práctico donde un catálogo de metadatos activo evite un incidente de facturación de nube en un motor como Snowflake o BigQuery.

Solución 20.2: Un analista programa un job que ejecuta un `SELECT *` sobre una tabla de eventos de 500 terabytes sin filtros de partición.

- Un **catálogo pasivo** simplemente almacena el nombre de la tabla; la consulta se ejecuta, escanea los 500 TB y genera una factura inmediata de $3.000 USD.
- Un **catálogo activo** intercepta la consulta en el proxy analítico, coteja las políticas operativas del activo (regla: "tablas con clasificación masiva requieren filtro obligatorio de fecha"), detecta la ausencia de predicados de partición y cancela preventivamente la consulta con una alerta explicativa al usuario, protegiendo el presupuesto de la empresa.

20.5 Resumen y Puntos Clave

- **La tríada de metadatos:** Un activo de datos maduro requiere metadatos técnicos (esquemas y formatos), operativos (latencia, SLAs y conteo de filas) y de negocio (glosario conceptual, propietarios y clasificación PII).
- **Metadatos activos:** Superan el inventario pasivo tradicional actuando en tiempo real para disparar alertas, aplicar optimizaciones de cómputo y bloquear accesos no conformes.
- **Contratos de Datos (Data Contracts):** Establecen una frontera formal de ingeniería entre productores operacionales y consumidores analíticos, desplazando el aseguramiento de calidad hacia la fuente (*Shift-Left*).
- **Estándar ODCS y dbt:** Permiten declarar esquemas, restricciones y SLAs en archivos YAML versionados en Git con validación forzada en tiempo de compilación (`contract: {enforced: true}`).

Lecturas Recomendadas

1. **Sanderson, C. & Freeman, M. (2025).** *Data Contracts: Building Production-Grade Pipelines with Software Engineering Rigor.* O'Reilly Media.
2. **Open Data Contract Standard (ODCS) Technical Committee (2024).** *ODCS Specification v1.0.* (`https://datacontract.com/`).
3. **Jones, A. (2023).** *Driving Data Quality with Data Contracts: A Comprehensive Guide to Managing Data Pipelines.* Packt Publishing.

PARTE VII

Procesamiento en tiempo real (streaming)

Capítulo 21 · Fundamentos de Streaming y Apache Kafka RECOMENDADO

Capítulo 22 · Spark Structured Streaming y Change Data Capture (CDC) RECOMENDADO

CAPÍTULO 21

Fundamentos de Streaming y Apache Kafka RECOMENDADO

> *"El mundo operacional no emite transacciones en lotes nocturnos; los eventos ocurren de forma continua e ininterrumpida. Procesar datos en lotes es una aproximación forzada por limitaciones históricas de infraestructura; el procesamiento de flujos (streaming) representa el modelo natural de la realidad computacional."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la arquitectura de **Apache Kafka** basada en registros inmutables distribuidos (*Commit Log*), segmentos físicos, índices dispersos y offsets.
- Dominar la evolución arquitectónica hacia **KRaft (Kafka Raft Metadata mode)** y la eliminación definitiva de Apache ZooKeeper en Kafka 4.0.
- Diseñar estrategias de replicación de alta disponibilidad con réplicas sincronizadas (*In-Sync Replicas - ISR*), `min.insync.replicas` y productores idempotentes.
- Integrar **Schema Registry** con esquemas **Apache Avro** gobernando compatibilidades evolutivas (*BACKWARD*, *FORWARD*, *FULL*).
- Distinguir rigurosamente las tres semánticas temporales en sistemas distribuidos: *Event Time*, *Ingestion Time* y *Processing Time*.
- Implementar productores idempotentes y consumidores resilientes con control manual de offsets en Python empleando `confluent-kafka`.

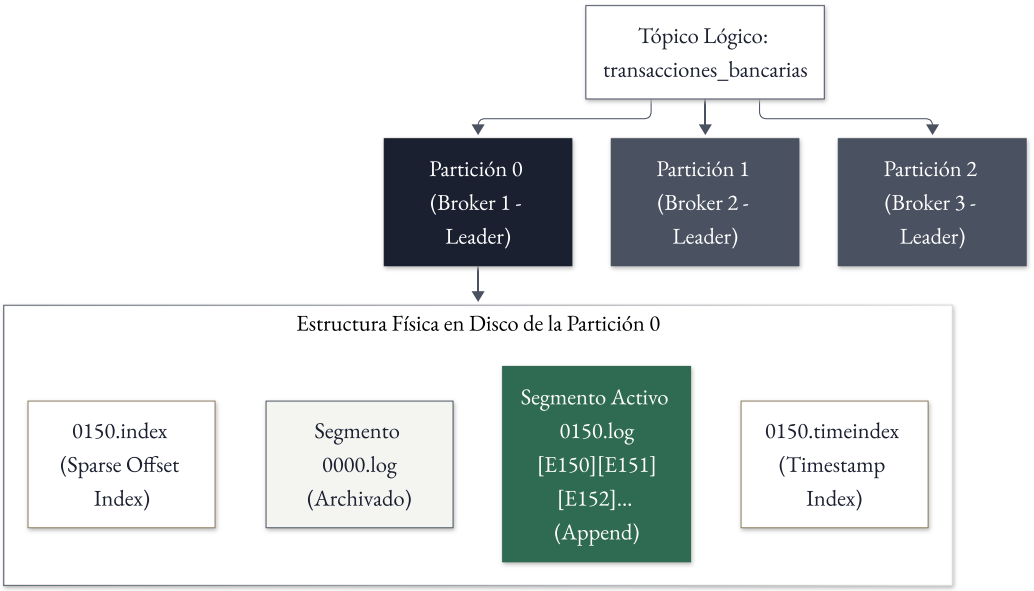
Definición 21.1 (Registro Distribuido Inmutable / Commit Log)

Apache Kafka es un sistema de almacenamiento distribuido modelado como un **registro de confirmación estructurado y solo anexable (*append-only log*)**. Los eventos se escriben de manera estrictamente secuencial al final del archivo físico y se indexan mediante un entero incremental de 64 bits denominado **offset**.

Esta inmutabilidad secuencial elimina la contención por bloqueos de fila, permite lecturas concurrentes masivas a través del caché de páginas del sistema operativo (*Page Cache*) y desacopla la velocidad de ingesta de la capacidad de procesamiento de los consumidores.

21.1 Anatomía Interna: Tópicos, Particiones, Segmentos y Offsets

El paralelismo horizontal de Kafka descansa en la descomposición física de los tópicos:



- **Partición y Segmentos:** Cada partición se materializa en el sistema de archivos del broker como un directorio que contiene archivos de segmento (típicamente de 1 GB). Solo el segmento activo recibe escrituras; los segmentos anteriores son inmutables y se purgan o compactan según las políticas de retención.
- **Índices Dispersos (*Sparse Indices*):** Para localizar un mensaje por su offset o timestamp sin escanear gigabytes de datos, Kafka mantiene archivos `.index` que asocian offsets lógicos con posiciones de bytes exactas en el archivo `.log`.

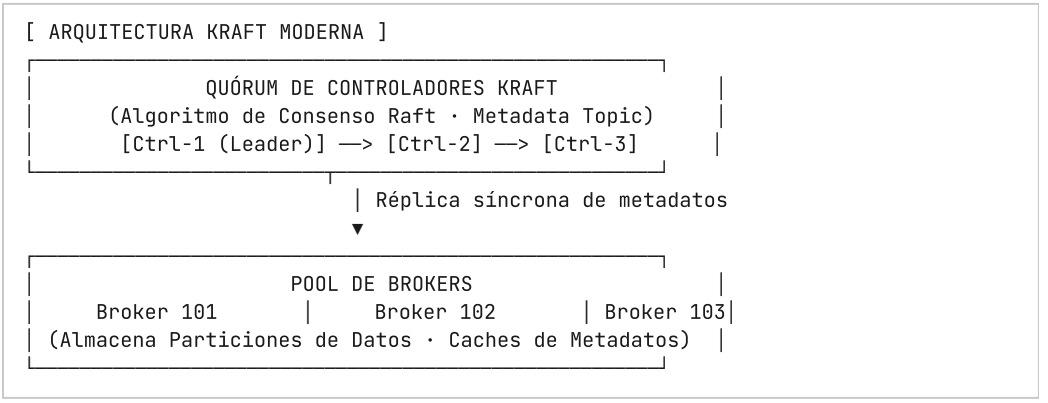
- **Zero-Copy Data Transfer y la Salvedad de TLS:** Al servir datos a los consumidores en conexiones de texto plano, Kafka utiliza la llamada al sistema operativo `sendfile()`. Esto transfiere bytes directamente desde el *Page Cache* del kernel hacia el búfer del socket de red (NIC), evitando copiar los datos al espacio de usuario de la JVM. No obstante, **si se habilita cifrado TLS/SSL entre el broker y el cliente**, los datos deben desencriptarse y encriptarse forzosamente en el espacio de usuario de la JVM, por lo que la optimización de zero-copy queda desactivada en favor de la seguridad en tránsito.

21.2 La Revolución de KRaft: De ZooKeeper al Quórum Raft Nativo

Durante más de una década, Apache Kafka dependió de **Apache ZooKeeper** para coordinar el clúster, elegir al broker líder y almacenar metadatos de configuración. Esta dependencia causaba graves problemas operativos:

1. **Cuello de Botella de Metadatos:** Clústeres con más de 100.000 particiones experimentaban degradación severa; una caída del broker controlador requería releer y reconstruir todo el estado desde ZooKeeper, demorando minutos.
2. **Duplicación de Sistemas:** Los operadores debían mantener, monitorear y asegurar dos clústeres distribuidos distintos (Kafka y ZooKeeper).

Con la introducción de **KRaft (KIP-500)** y su consagración definitiva en **Kafka 4.0**, ZooKeeper ha sido completamente eliminado:



- **Tópico Interno `__cluster_metadata`:** Los metadatos de particiones, tópicos y ACLs se gestionan como un registro distribuido interno replicado entre los nodos controladores.
- **Recuperación Sub-Segundo:** El traspaso de liderazgo del controlador (*failover*) es casi instantáneo porque todos los controladores mantienen el log de metadatos sincronizado en

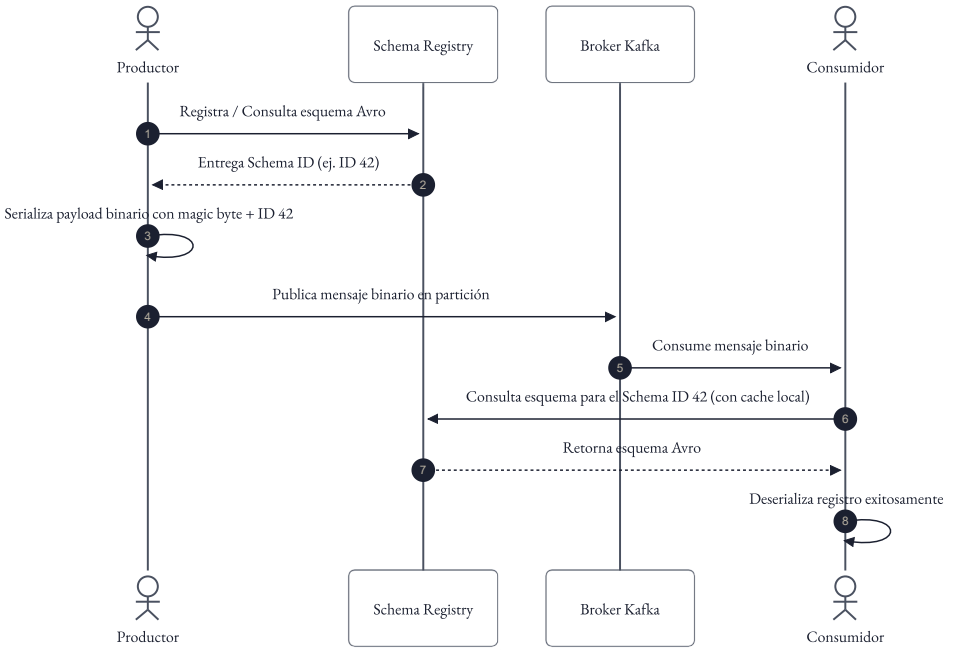
memoria en todo momento.

- **Escalabilidad Masiva:** Permite soportar millones de particiones por clúster sin degradación de latencia.

21.3 Gobernanza de Esquemas: Schema Registry y Apache Avro

Enviar cargas binarias o JSON crudo por Kafka sin gobernanza conduce a desastres en producción: si un productor altera un campo, los consumidores fallan con errores de deserialización.

Confluent Schema Registry resuelve este problema actuando como el plano de control para los esquemas serializados en **Apache Avro**:



Formato de Transmisión (Wire Format) de Schema Registry

Los mensajes serializados con Schema Registry no transportan el esquema completo en cada evento. Utilizan una cabecera compacta de **5 bytes**:

- **Byte 0:** *Magic Byte* (siempre `0x00`).
- **Bytes 1-4:** Entero de 32 bits en big-endian que representa el identificador global del esquema en el registro (*Schema ID*).

- **Bytes 5 en adelante:** Datos útiles binarios codificados según la especificación de Apache Avro.

Modos de Compatibilidad Evolutiva

Es fundamental entender la dirección de la compatibilidad entre consumidores y productores:

- **BACKWARD (Compatibilidad hacia atrás):**
  - **Definición:** Los consumidores que han sido actualizados con el **esquema nuevo** pueden leer datos producidos con el **esquema anterior**.
  - **Regla de oro:** Permite actualizar los consumidores primero antes que los productores.
  - **Restricción de esquema:** Si se agrega un nuevo campo en el esquema nuevo, este campo **debe tener un valor predeterminado (*default value*)** o ser opcional (`["null", "string"]`). Si carece de default, el consumidor nuevo no sabrá qué valor asignar a ese campo cuando procese un mensaje antiguo que no lo contiene.
- **FORWARD (Compatibilidad hacia adelante):**
  - **Definición:** Los consumidores que todavía operan con el **esquema anterior** pueden leer datos emitidos por productores que ya migraron al **esquema nuevo**.
  - **Regla de oro:** Permite actualizar los productores primero.
  - **Restricción de esquema:** Los campos nuevos introducidos por el productor son ignorados por el consumidor viejo. Si se elimina un campo en el nuevo esquema, ese campo debió haber tenido un valor por defecto en el esquema viejo para que el consumidor anterior pueda seguir funcionando.
- **FULL (Compatibilidad total):**
  - Combina simultáneamente las garantías de **BACKWARD** y **FORWARD**. Los consumidores nuevos pueden leer datos viejos y los consumidores viejos pueden leer datos nuevos. Permite desplegar productores y consumidores en cualquier orden arbitrario. Solo permite agregar o eliminar campos que posean un valor por defecto explícito.

21.4 Semántica Temporal: Event Time, Ingestion Time y Processing Time

En sistemas de procesamiento de flujos, el tiempo físico del reloj de la máquina no refleja el momento en que ocurrieron los hechos:

1. **Event Time:** El instante temporal en que el evento se originó en el sistema fuente o dispositivo emisor (ej. la marca de tiempo del sensor telemático o el momento en que el

usuario presionó "Pagar"). Es la única semántica que garantiza resultados deterministas y reproducibles.

2. **Ingestion Time:** El instante en que el evento fue recibido por el broker de Kafka y añadido a la partición correspondiente.
3. **Processing Time:** El instante del reloj local del nodo de cómputo (ej. Worker de Spark o Flink) en el momento exacto en que se ejecuta la transformación. Varía ante fluctuaciones de carga y reinicios.

> **Consideraciones de Producción 21.1: Productor Idempotente en Python**
>
> Para garantizar que fallos en la red o reintentos automáticos nunca generen mensajes duplicados ni alteren el orden de los eventos, el productor debe configurar `enable.idempotence = True` junto con `acks = 'all'`:

PYTHON

```python
from confluent_kafka import Producer
import json
import sys

def callback_entrega(err, msg):
    """Callback invocado por librdkafka cuando el broker confirma la
escritura."""
    if err is not None:
        sys.stderr.write(f"[ERROR] Fallo en confirmación de mensaje: {err}\n")
    else:
        print(f"[OK] Mensaje confirmado: Tópico={msg.topic()} | "
              f"Partición={msg.partition()} | Offset={msg.offset()}")

configuracion_productor = {
    'bootstrap.servers': 'kafka-broker-1:9092,kafka-broker-2:9092',
    'client.id': 'productor-telemetria-v1',

    # 1. Semántica de Entrega Idempotente Estricta
    'enable.idempotence': True,
    'acks': 'all',  # Esperar a todas las réplicas sincronizadas
(min.insync.replicas)
    'retries': 1000000,
    'max.in.flight.requests.per.connection': 5,

    # 2. Optimización de Rendimiento y Rendimiento I/O (Lotes y Compresión)
    'compression.type': 'zstd',
    'linger.ms': 20,           # Agrupar mensajes durante 20 ms antes de enviar
    'batch.size': 65536         # Tamaño de lote de 64 KB
}

productor = Producer(configuracion_productor)

evento_telemetria = {
```

```
    "sensor_id": "SN-8841-CO",
    "temperatura_celsius": 24.8,
    "presion_barometrica": 1013.2,
    "event_time_utc": "2026-10-12T14:30:00Z"
}

clave_evento = evento_telemetria["sensor_id"].encode('utf-8')
valor_evento = json.dumps(evento_telemetria).encode('utf-8')

productor.produce(
    topic='iot.telemetria.v1',
    key=clave_evento,
    value=valor_evento,
    on_delivery=callback_entrega
)

productor.flush(timeout=10)
```

Consideraciones de Producción 21.2: Consumidor Resiliente con Control Manual de Offsets

En producción, nunca confíe en `enable.auto.commit = True` si su procesamiento realiza escrituras en bases de datos externas. Si el commit ocurre automáticamente cada 5 segundos pero su aplicación falla procesando el registro en el segundo 3, ese mensaje se perderá al reiniciar. Use confirmación manual sincrónica (*At-Least-Once Delivery*):

PYTHON

```
from confluent_kafka import Consumer, KafkaError, KafkaException
import sys

def callback_rebalance(consumer, particiones):
    """Callback invocado durante rebalances cooperativos de particiones."""
    print(f"[REBALANCE] Asignación actualizada: {[p.partition for p in
particiones]}")

configuracion_consumidor = {
    'bootstrap.servers': 'kafka-broker-1:9092,kafka-broker-2:9092',
    'group.id': 'grupo-procesamiento-telemetria',
    'auto.offset.reset': 'earliest',

    # Desactivar auto-commit para controlar manualmente la durabilidad
    'enable.auto.commit': False,
    'partition.assignment.strategy': 'cooperative-sticky',
    'max.poll.interval.ms': 300000  # 5 minutos para procesar antes de considerar
muerto
}

consumidor = Consumer(configuracion_consumidor)
consumidor.subscribe(['iot.telemetria.v1'], on_assign=callback_rebalance)

try:
```

```
    while True:
        msg = consumidor.poll(timeout=1.0)
        if msg is None:
            continue
        if msg.error():
            if msg.error().code() == KafkaError._PARTITION_EOF:
                continue
            else:
                raise KafkaException(msg.error())

        # 1. Procesar la carga del mensaje
        clave = msg.key().decode('utf-8') if msg.key() else None
        valor = msg.value().decode('utf-8')

        # 2. Persistir en la capa destino (operación idempotente)
        print(f"[CONSUMO] Clave={clave} | Offset={msg.offset()} | Partición=
{msg.partition()}")

        # 3. Confirmar manualmente el offset tras procesamiento exitoso
        consumidor.commit(message=msg, asynchronous=False)

except KeyboardInterrupt:
    sys.stderr.write("[APAGADO] Deteniendo consumidor de forma segura...\n")
finally:
    consumidor.close()
```

21.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Configurar `acks=1` o `acks=0` en Pipelines Financieros o Transaccionales Con `acks=1`, el productor recibe la confirmación tan pronto como el broker líder escribe en disco. Si el líder colapsa antes de replicar el mensaje a los seguidores en ISR, el mensaje se pierde irrecuperablemente (*Data Loss*). En entornos de producción, use siempre `acks=all` combinado con un clúster con `min.insync.replicas=2` y factor de replicación 3.

Anti-patrón 2: Publicar Mensajes sin Clave (`key=None`) Cuando se Requiere Orden Estricto Si se publica sin clave, el productor distribuye los mensajes mediante round-robin entre todas las particiones del tópico. Dado que Kafka solo garantiza el orden **dentro de una partición individual**, eventos secuenciales de una misma cuenta bancaria (ej. *Depósito de $100* y posterior *Retiro de $100*) terminarán en particiones distintas y podrán procesarse en desorden, causando inconsistencias contables graves.

Anti-patrón 3: Diseñar Mensajes Mayores a 1 MB sin Patrón Claim-Check Kafka está optimizado para procesar millones de mensajes ligeros (1 KB a 64 KB). Enviar cargas de decenas de megabytes degrada el Page Cache, satura los búferes de red y provoca pausas graves en los consumidores. Si requiere transferir archivos masivos, implemente el patrón **Claim-Check**: guarde el archivo en almacenamiento de objetos (S3/ADLS) y envíe por Kafka un mensaje ligero con el puntero URI y metadatos de acceso.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 21.1 (Escalabilidad de Grupos de Consumidores):
Un tópico de Kafka denominado `pedidos_checkout` posee **8 particiones físicas**.

1. Si un grupo de consumidores (`grupo_facturacion`) levanta **4 instancias** de consumidores, ¿cómo se distribuyen las particiones entre ellos?
2. Si la carga de trabajo se incrementa y el equipo decide escalar el grupo levantando **12 instancias** de consumidores en el mismo grupo, ¿cuántos consumidores procesarán datos y qué ocurre con los restantes?
3. ¿Cuál es el límite teórico de escalabilidad horizontal de un grupo de consumidores en Kafka?

Solución 21.1:

1. *Con 4 consumidores:* El coordinador del grupo asigna equitativamente **2 particiones a cada consumidor** (4 × 2 = 8 particiones).
2. *Con 12 consumidores:* Debido a la regla fundamental de diseño de Kafka que estipula que una partición solo puede ser consumida por **exactamente un consumidor** dentro del mismo grupo en un instante dado, **8 consumidores recibirán 1 partición cada uno** y **4 consumidores permanecerán completamente inactivos (*idle*)**.
3. *Límite teórico:* La capacidad máxima de paralelismo de un grupo de consumidores está rígidamente acotada por el **número total de particiones del tópico**. Para escalar a 12 consumidores activos, es obligatorio incrementar previamente el particionamiento del tópico a un mínimo de 12 particiones.

Ejercicio 21.2 (Gobernanza con Schema Registry y Regla BACKWARD):
Un equipo desea añadir un nuevo campo llamado `codigo_promocional` al esquema Avro de `pedidos`. El registro de esquemas está configurado con la regla de compatibilidad `BACKWARD`.

1. Bajo la definición formal de compatibilidad `BACKWARD`, ¿qué componentes del sistema se actualizan primero: los productores o los consumidores?
2. ¿Por qué el campo nuevo debe definir obligatoriamente un valor por defecto (`default`)?
3. ¿Qué código HTTP y mensaje de error devuelve Schema Registry si se intenta registrar un esquema que viole esta regla?

Solución 21.2:

1. *Orden de despliegue en BACKWARD:* Se actualizan **primero los consumidores**. La regla `BACKWARD` garantiza que los consumidores que ya tienen el nuevo esquema puedan procesar sin fallos los datos antiguos emitidos por productores que aún no se han actualizado.

2. *Necesidad de valor por defecto:* Cuando el consumidor actualizado recibe un mensaje serializado con el esquema anterior, el payload binario no contendrá el campo `codigo_promocional`. Si el nuevo esquema no especifica un `default` (ej. `default: null`), el motor de deserialización no puede inferir el valor del atributo y abortará el procesamiento con un error de deserialización.
3. *Respuesta de Schema Registry ante violación:* Al intentar registrar el nuevo esquema incompatible mediante la API REST (`POST /subjects/{subject}/versions`), Schema Registry rechaza la solicitud devolviendo el código de estado HTTP **409 Conflict** con la respuesta JSON canónica:

```JSON
{
  "error_code": 40901,
  "message": "Schema being registered is incompatible with an earlier schema"
}
```

21.6 Resumen y Puntos Clave

- **Arquitectura Commit Log:** Kafka almacena eventos secuenciales inmutables indexados por offsets en disco, alcanzando rendimiento masivo gracias al Page Cache del sistema operativo y transferencias *zero-copy* (en conexiones de texto plano).
- **La era KRaft:** La eliminación definitiva de ZooKeeper en Kafka 4.0 sustituye la coordinación externa por un quórum de controladores nativo basado en el algoritmo Raft, acelerando los tiempos de recuperación a milisegundos y habilitando millones de particiones por clúster.
- **Gobernanza con Schema Registry:** Previene roturas catastróficas en consumidores aplicando contratos binarios en Apache Avro y gobernando la evolución de esquemas (*BACKWARD*, *FORWARD*, *FULL*).
- **Entrega idempotente:** Configurar `enable.idempotence=True`, `acks=all` y claves de particionamiento estrictas previene duplicados y garantiza el orden secuencial por entidad.

Lecturas Recomendadas

1. **Shapira, G., Palino, T., Sivaram, R., & Petty, K. (2021).** *Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale (2nd Edition).* O'Reilly Media.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems.* O'Reilly Media. (Capítulo 11: Stream Processing).

3. **Confluent Documentation (2024).** *Schema Registry Overview, Compatibility Types and Wire Format.* Confluent Documentation (`https://docs.confluent.io/platform/current/schema-registry/`).

CAPÍTULO 22

Spark Structured Streaming y Change Data Capture (CDC) RECOMENDADO

> *"Spark Structured Streaming redefine el procesamiento de eventos bajo la premisa de tablas ilimitadas: el cómputo continuo es conceptualmente idéntico a una consulta analítica sobre una tabla infinita a la que se anexan filas sin cesar. Al integrar esta abstracción con Change Data Capture (CDC) y Delta Lake, los ingenieros pueden sincronizar réplicas de bases de datos operacionales con latencia de segundos y garantías estrictas de consistencia."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender el modelo de datos de tabla sin fin (*Unbounded Table*) y los tres modos de salida (*Append, Complete, Update*).
- Analizar la anatomía interna del directorio de puntos de control (*Checkpointing*) y los proveedores de gestión de estado (*HDFSBacked* vs. *RocksDB*).
- Procesar flujos de captura de cambios operacionales (**Change Data Capture - CDC**) emitidos por Debezium, gestionando el envelope estructurado, mensajes *tombstone* y coordenadas de binlog/LSN.
- Resolver colisiones en micro-lotes (*Multiple Source Rows Matched*) en operaciones `MERGE INTO` concurrentes y garantizar idempotencia de extremo a extremo mediante `txnAppId` y `txnVersion`.
- Configurar la ingesta incremental de archivos en almacenamiento de objetos con **Databricks Auto Loader (`cloudFiles`)**, distinguiendo el listado incremental del modo de notificaciones por colas.
- Gestionar la acumulación de estado en memoria mediante ventanas temporales y marcas de agua (*Watermarking*), entendiendo la semántica de descarte por lotes previos.

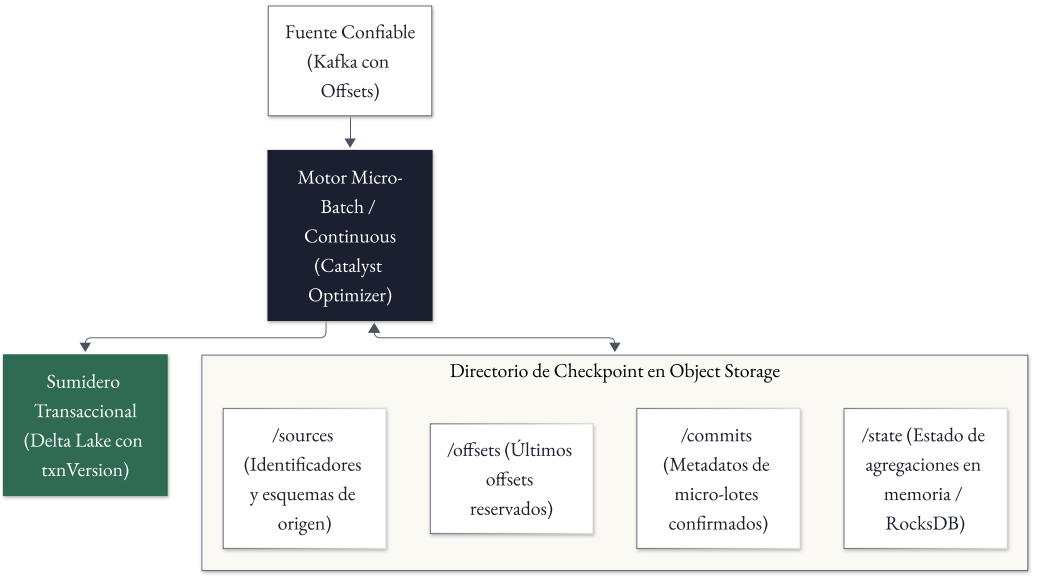
Definición 22.1 (Modelo Unbounded Table y Structured Streaming)

Spark Structured Streaming es un motor de procesamiento de flujos de eventos escalable y tolerante a fallos construido sobre el optimizador Catalyst de Spark SQL. Modela los flujos de datos continuos como una **tabla sin límites (*Unbounded Table*)**.

En lugar de manipular APIs de bajo nivel orientadas a mensajes individuales, el ingeniero expresa transformaciones declarativas utilizando las mismas operaciones relacionales de DataFrames y SQL empleadas en cargas por lotes (*Batch*). El motor descompone automáticamente la consulta en micro-lotes periódicos o en ejecución continua (*Continuous Processing Engine*) según la latencia requerida.

22.1 Arquitectura Interna: Modos de Salida, Checkpointing y Proveedores de Estado

Structured Streaming desacopla fuentes reproducibles con seguimiento de offsets (por ejemplo, Apache Kafka) y sumideros transaccionales idempotentes (por ejemplo, Delta Lake):



Modos de Salida (Output Modes)

1. **Append Mode (Predeterminado):** Solo las filas nuevas confirmadas desde el último lote se emiten al sumidero. Requiere que las filas existentes no puedan mutar. Es obligatorio para sumideros que no admiten actualizaciones.

2. **Complete Mode:** La tabla de resultados completa se reescribe en cada micro-lote. Se utiliza exclusivamente en consultas con agregaciones acumulativas globales (por ejemplo, conteo total acumulado de eventos por país).
3. **Update Mode:** Solo las filas cuyo valor agregado o proyectado cambió desde el micro-lote anterior se emiten hacia el sumidero externo.

Proveedores de Almacén de Estado (*State Store Providers*)

Cuando una consulta incluye agregaciones con estado (por ejemplo, `groupBy().count()` sobre ventanas de tiempo) o uniones streaming-streaming:

- `HDFSBackedStateStoreProvider` **(Por Defecto):** Mantiene todo el estado en la memoria heap de la JVM del ejecutor y escribe copias de seguridad sincrónicas a HDFS o almacenamiento de objetos en cada micro-lote. Es ideal para estados pequeños (< 10 GB), pero sufre pausas de Garbage Collection severas con estados grandes.
- `RocksDBStateStoreProvider` **(Opcional):** Emplea una base de datos local embebida fuera de la JVM (C++ RocksDB) en los discos locales de los Workers, volcando únicamente cambios a almacenamiento de objetos:

```python
spark.conf.set(
    "spark.sql.streaming.stateStore.providerClass",
    "org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider"
)
```

  Permite gestionar estados de cientos de gigabytes sin sobrecargar la memoria heap de la JVM.

Tolerancia a Fallos y Semántica de Ejecución

Si un ejecutor o el nodo Driver colapsan a mitad de un micro-lote:

1. Al reiniciar, Spark inspecciona los directorios `/offsets` y `/commits`.
2. Si el lote N fue reservado en `/offsets` pero no posee su confirmación en `/commits`, Spark detecta que el procesamiento quedó incompleto.
3. El motor **re-ejecuta el micro-lote** N desde los mismos offsets de Kafka.
4. *Aclaración Crítica de Consistencia:* A nivel del motor, esta recuperación ofrece garantías de **al menos una vez (*At-Least-Once*)**. La garantía de **exactamente una vez (*Exactly-Once*)** solo se alcanza si el sumidero es estrictamente idempotente.

22.2 Change Data Capture (CDC) con Debezium y Envelope Estructurado

El patrón **Change Data Capture (CDC)** extrae mutaciones directamente del registro de transacciones de bases de datos operacionales (el WAL en PostgreSQL, el Binlog en MySQL, o el Oplog en MongoDB) utilizando **Debezium**, evitando consultas periódicas que saturan las bases operacionales.

El Envelope Completo de Debezium frente a SMT

Por defecto, cuando Kafka Connect opera sin transformaciones de mensaje único (*SMT*), cada mensaje en Kafka contiene metadatos de esquema y un payload anidado:

```JSON
{
  "schema": { "type": "struct", "fields": [...] },
  "payload": {
    "op": "u",
    "before": { "cliente_id": "C-101", "saldo": 500.00, "pais": "CO" },
    "after":  { "cliente_id": "C-101", "saldo": 850.50, "pais": "CO" },
    "source": {
      "name": "mysql_core",
      "ts_ms": 1773412800000,
      "file": "binlog.000042",
      "pos": 1048576,
      "table": "cuentas"
    },
    "ts_ms": 1773412800500
  }
}
```

- **Operación (`op`):** `c` (*create/insert*), `u` (*update*), `d` (*delete*), `r` (*read/snapshot* inicial).
- **`before` y `after`:** Estado previo y estado nuevo de la tupla.
- **Coordenadas de Binlog / LSN (`source.file`, `source.pos`, `source.lsn`):** Posición monotónicamente creciente en el registro de transacciones. **No confíe únicamente en `ts_ms`** para ordenar eventos, ya que múltiples mutaciones dentro de una misma transacción comparten exactamente el mismo milisegundo.
- **Mensajes Tombstone:** Al ejecutarse un borrado (`DELETE`), Debezium emite primero el evento con `op = 'd'`, seguido de un mensaje con valor nulo (`null payload`) para permitir la compactación de tópicos en Kafka. Si el pipeline no filtra `value IS NOT NULL`, el parser de JSON arrojará errores de ejecución.

22.3 Resolución de Colisiones en Micro-Lotes: Deduplicación e Idempotencia con MERGE

Si un mismo micro-lote contiene múltiples eventos para una misma clave primaria (por ejemplo, una inserción rápida seguida de dos actualizaciones en un lapso de 5 segundos), ejecutar `MERGE INTO` directamente sobre el lote disparará la excepción:

```
DeltaAnalysisException: Cannot perform MERGE as multiple source rows matched the
same target row.
```

Para evitar este fallo y asegurar idempotencia exacta, el micro-lote debe:

1. Filtrar registros *tombstone* (`value IS NOT NULL`).
2. Deduplicarse reteniendo la mutación con mayor posición en el binlog (`source.pos`) o mayor `ts_ms`.
3. Ejecutar `MERGE INTO` asociando el identificador del micro-lote con `txnAppId` y `txnVersion` para evitar reprocesamientos duplicados si el micro-lote se reintenta.

PYTHON

```python
from pyspark.sql import SparkSession, Window
import pyspark.sql.functions as F
from pyspark.sql.types import *
from delta.tables import DeltaTable

spark = SparkSession.builder \
    .appName("Capitulo22-CDC-Streaming-Resiliente") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog") \
    .getOrCreate()

# 1. Definición del esquema estricto del payload de Debezium (con
schemas.enable=false)
esquema_fila = StructType([
    StructField("cliente_id", StringType(), False),
    StructField("nombre", StringType(), True),
    StructField("saldo", DoubleType(), True),
    StructField("pais", StringType(), True)
])

esquema_source = StructType([
    StructField("name", StringType(), True),
    StructField("ts_ms", LongType(), True),
    StructField("file", StringType(), True),
    StructField("pos", LongType(), True)
])

esquema_debezium = StructType([
    StructField("op", StringType(), False),
```

```
    StructField("before", esquema_fila, True),
    StructField("after", esquema_fila, True),
    StructField("source", esquema_source, True),
    StructField("ts_ms", LongType(), False)
])

# 2. Lectura continua de Kafka desde earliest para no perder el snapshot inicial
df_kafka = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka-broker:9092") \
    .option("subscribe", "cdc.mysql.cuentas") \
    .option("startingOffsets", "earliest") \
    .load()

# Filtrar mensajes tombstone (payloads nulos emitidos tras borrados)
df_valid_kafka = df_kafka.filter(F.col("value").isNotNull())

df_parsed = df_valid_kafka.select(
    F.from_json(F.col("value").cast("string"), esquema_debezium).alias("data")
).select(
    F.col("data.op").alias("op"),
    F.coalesce(F.col("data.after.cliente_id"),
F.col("data.before.cliente_id")).alias("cliente_id"),
    F.col("data.after.nombre").alias("nombre"),
    F.col("data.after.saldo").alias("saldo"),
    F.col("data.after.pais").alias("pais"),
    F.col("data.source.file").alias("binlog_file"),
    F.col("data.source.pos").alias("binlog_pos"),
    # Clave de secuencia monotónica que resiste rotación de archivos binlog:
    F.concat(
        F.col("data.source.file"),
        F.lit("#"),
        F.lpad(F.col("data.source.pos").cast("string"), 20, "0")
    ).alias("coordenada_cdc"),
    F.to_timestamp(F.col("data.ts_ms") / 1000).alias("timestamp_evento")
)

# 3. Función de procesamiento de micro-lote con deduplicación y MERGE idempotente
def fusionar_cdc_a_delta(df_micro_batch, batch_id):
    if df_micro_batch.isEmpty():
        return

    # Deduplicación intra-lote: ordenar por coordenada compuesta de binlog y
timestamp
    # Nota crítica: La posición se reinicia al rotar de archivo, por lo que la
tupla (file, pos) es obligatoria
    ventana_dedup = Window.partitionBy("cliente_id").orderBy(
        F.col("coordenada_cdc").desc(),
        F.col("timestamp_evento").desc()
    )

    df_dedup = df_micro_batch \
        .withColumn("ranking", F.row_number().over(ventana_dedup)) \
        .filter(F.col("ranking") == 1) \
        .drop("ranking")

    app_id = "streaming_cdc_cuentas_v1"
```

```
    tabla_delta = DeltaTable.forPath(spark, "lakehouse/silver/cuentas_clientes")

    # MERGE INTO con soporte para inserción de updates tardíos (upsert robusto)
    # y condición de versión monotónica para evitar sobreescribir datos nuevos
con tardíos
    condicion_orden = """
        target.coordenada_cdc IS NULL OR
        source.coordenada_cdc >= target.coordenada_cdc
    """

    tabla_delta.alias("target").merge(
        df_dedup.alias("source"),
        "target.cliente_id = source.cliente_id"
    ).whenMatchedDelete(
        condition=f"source.op = 'd' AND ({condicion_orden})"
    ).whenMatchedUpdate(
        condition=f"source.op = 'u' AND ({condicion_orden})",
        set={
            "nombre": "source.nombre",
            "saldo": "source.saldo",
            "pais": "source.pais",
            "binlog_file": "source.binlog_file",
            "binlog_pos": "source.binlog_pos",
            "coordenada_cdc": "source.coordenada_cdc",
            "timestamp_evento": "source.timestamp_evento",
            "_batch_id": F.lit(batch_id).cast("bigint")
        }
    ).whenNotMatchedInsert(
        condition="source.op IN ('c', 'r', 'u')", # Si un update llega para una
fila inexistente, se inserta
        values={
            "cliente_id": "source.cliente_id",
            "nombre": "source.nombre",
            "saldo": "source.saldo",
            "pais": "source.pais",
            "binlog_file": "source.binlog_file",
            "binlog_pos": "source.binlog_pos",
            "coordenada_cdc": "source.coordenada_cdc",
            "timestamp_evento": "source.timestamp_evento",
            "_batch_id": F.lit(batch_id).cast("bigint")
        }
    ).execute()

# 4. Iniciar ejecución continua con foreachBatch y checkpointing persistente
query_cdc = df_parsed.writeStream \
    .foreachBatch(fusionar_cdc_a_delta) \
    .option("checkpointLocation", "s3://lakehouse-checkpoints/cdc_cuentas/") \
    .trigger(processingTime="10 seconds") \
    .start()
```

22.4 Ingesta de Archivos con Databricks Auto Loader (`cloudFiles`)

Al procesar millones de archivos depositados de forma continua en almacenamiento de objetos (Amazon S3, Azure ADLS, GCP GCS), el lector tradicional (`readStream.format("parquet")`) experimenta una degradación severa provocada por la latencia del listado recursivo de directorios (*Directory Listing Latency*).

Auto Loader (`cloudFiles`) optimiza esta ingesta mediante dos modos operativos:

1. **Modo Listado Incremental (Por Defecto):** Escanea únicamente los subdirectorios modificados recientemente utilizando la marca de tiempo de los archivos (`cloudFiles.useIncrementalListing = true`), evitando listar el bucket completo.
2. **Modo Notificaciones por Colas (Opcional):** Se suscribe directamente a colas de eventos en la nube (AWS SQS, Azure Event Grid) para recibir alertas directas ante la creación de nuevos objetos (`cloudFiles.useNotifications = true`), eliminando completamente cualquier llamada a operaciones de listado:



Características Críticas de Auto Loader

- **Inferencia y Evolución de Esquema:** Detecta automáticamente nuevas columnas en los archivos entrantes y las incorpora al esquema (`cloudFiles.schemaEvolutionMode = "addNewColumns"`).
- **Resguardo de Datos Malformados (`_rescued_data`):** En lugar de abortar el pipeline cuando un registro contiene tipos de datos corruptos o campos no mapeados, Auto Loader desvía los datos discordantes a una columna oculta `_rescued_data` en formato JSON, permitiendo que el micro-lote continúe sin interrupciones.
- **Disparador `Trigger.AvailableNow`:** Procesa todos los archivos acumulados hasta el momento y detiene el flujo automáticamente, ofreciendo la economía de costos de un trabajo batch con las ventajas operativas de streaming.

```python
# Ingesta robusta con Auto Loader
df_auto_loader = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .option("cloudFiles.schemaLocation", "s3://lakehouse-
checkpoints/autoloader_schemas/ventas/") \
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns") \
    .load("s3://fuentes-externas/ventas_landing/")

query_al = df_auto_loader.writeStream \
    .format("delta") \
    .option("checkpointLocation", "s3://lakehouse-
checkpoints/autoloader_cp/ventas/") \
    .trigger(availableNow=True) \
    .toTable("lakehouse.bronze.ventas_raw")
```

22.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Configurar `startingOffsets = "latest"` en Pipelines de CDC
Configurar `latest` hace que Spark comience a leer únicamente los mensajes publicados tras el arranque del stream, omitiendo por completo el snapshot inicial de la base de datos y todas las mutaciones generadas durante el periodo de despliegue. En pipelines de CDC, utilice siempre `earliest`.

Anti-patrón 2: Modificar la Consulta sin Reiniciar el Checkpoint Cuando Cambia el Esquema de Estado
Si una consulta utiliza agregaciones con estado (por ejemplo, ventanas temporales) y se modifica la clave de agrupación o la ventana en el código, el motor no podrá deserializar el estado previo en el checkpoint. En tales situaciones, debe definirse una nueva ruta de checkpoint y planificar la transición de datos.

Anti-patrón 3: Agregaciones Temporales sin Declaración de Watermarking
Ejecutar agrupaciones basadas en tiempo sin declarar `.withWatermark()` obliga a Spark a retener el estado de todas las ventanas en memoria de forma indefinida, ya que el motor nunca sabrá cuándo considerar cerrada una ventana. Esto culminará inevitablemente en un fallo por memoria agotada (*OutOfMemoryError*) tras varios días de operación ininterrumpida.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 22.1 (Semántica Exacta de Watermarking y Descarte de Datos Tardíos):
Un flujo continuo de eventos se procesa agrupando por ventanas de 5 minutos sobre `timestamp_evento` con una marca de agua configurada como `.withWatermark("timestamp_evento", "10 minutes")`.

1. Si en el micro-lote $N-1$ el mayor `timestamp_evento` procesado fue `12:30:00`, ¿cuál es el umbral de la marca de agua aplicado para evaluar las ventanas del micro-lote N?
2. Si durante el micro-lote N llega un evento retrasado con `timestamp_evento = 12:18:00`, ¿el evento se procesa o se descarta? Explique la regla de cierre de ventanas.

Solución 22.1:

1. *Umbral de Watermark:* El umbral se calcula restando la duración de la marca de agua al timestamp máximo observado en el **micro-lote previo**:

$$\text{Umbral} = 12:30:00 - 10 \text{ minutos} = 12:20:00$$

2. *Evaluación del evento 12:18:00:*
   - La ventana de 5 minutos que corresponde a este evento es `[12:15:00, 12:20:00)`.
   - El límite superior de esta ventana es $12:20:00$. Dado que el umbral actual es $12:20:00$, la ventana se encuentra **estrictamente cerrada** (su límite superior no supera el umbral).
   - En consecuencia, Spark **descarta el evento y no actualiza el estado de la ventana**, protegiendo la memoria del almacén de estado frente a datos excesivamente tardíos (*Late Data*).

Ejercicio 22.2 (Sincronización de Borrados Físicos en CDC y Mensajes Tombstone):
En una arquitectura CDC conectada a PostgreSQL, un registro se elimina con `DELETE FROM cuentas WHERE cliente_id = 'C-55'`. Explique por qué es obligatorio filtrar `value IS NOT NULL` antes de parsear los eventos de Kafka y cómo actúa la cláusula `whenMatchedDelete` en el `foreachBatch`.

Solución 22.2:

1. *Filtrado de Tombstones:* Tras emitir el evento de borrado (`op = 'd'`), Debezium genera un registro de tipo *tombstone* con la clave `C-55` y un cuerpo nulo (`value = null`) para habilitar la compactación de logs en Kafka. Si el pipeline intenta ejecutar `F.from_json(col("value"), ...)`, la función recibirá una cadena nula, lo que puede

provocar excepciones o generar filas vacías que corrompen la deduplicación posterior. Filtrar `value IS NOT NULL` antes del parsing asegura que solo se procese el evento legítimo con `op = 'd'`.

2. *Comportamiento de `whenMatchedDelete`:*

```python
tabla_delta.alias("target").merge(
    df_dedup.alias("source"),
    "target.cliente_id = source.cliente_id"
).whenMatchedDelete(
    condition="source.op = 'd'"
)
```

Si el cliente existe en el destino (`target.cliente_id = source.cliente_id`) y el evento deduplicado más reciente indica borrado (`source.op = 'd'`), Delta Lake elimina físicamente la fila en la capa Silver. Si el registro ya fue borrado en un lote previo o nunca existió, la condición `whenMatched` no se cumple y la sentencia concluye sin errores.

22.6 Resumen y Puntos Clave

- **Unbounded Table:** Structured Streaming unifica el procesamiento continuo modelando los datos como una tabla infinita gobernada por el optimizador Catalyst.
- **Proveedores de Estado:** Por defecto se emplea `HDFSBackedStateStoreProvider` en memoria heap; para estados superiores a decenas de gigabytes se debe habilitar `RocksDBStateStoreProvider`.
- **Garantía Exactly-Once:** Requiere fuentes con offsets reproducibles (Kafka), checkpointing persistente y operaciones idempotentes en el sumidero (como `MERGE` en Delta Lake asociando transacciones).
- **Robustez en CDC:** Exige manejar esquemas completos de Debezium, descartar tombstones nulos, deduplicar intra-lote utilizando posiciones de binlog/LSN y usar `startingOffsets = "earliest"`.
- **Auto Loader:** Reduce drásticamente la latencia de escaneo en almacenamiento de objetos mediante listado incremental o notificaciones por colas, resguardando datos corruptos en `_rescued_data`.

Lecturas Recomendadas

1. **Armbrust, M. et al. (2018).** *Structured Streaming: A Declarative Engine for Real-Time Big Data Processing.* Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data, 601-613.
2. **Debezium Technical Documentation (2024).** *Debezium Connectors for MySQL and PostgreSQL: Architecture and Event Envelope Specification.* Red Hat (`https://debezium.io/documentation/`).
3. **Databricks Engineering (2024).** *Efficient Incremental Ingestion with Auto Loader and Delta Lake.* Databricks Documentation (`https://docs.databricks.com/ingestion/auto-loader/`).

PARTE VIII

Orquestación, modelado y DataOps

Capítulo 23 · Orquestación de Flujos: Apache Airflow y Azure Data Factory NÚCLEO

Capítulo 23A · Orquestación Avanzada con Apache Airflow: Arquitectura Interna, Dynamic Mapping y Datasets RECOMENDADO

Capítulo 24 · Transformación Modular con dbt, Snapshots e Idempotencia NÚCLEO

Capítulo 25 · Infraestructura como Código (IaC) con Terraform y OpenTofu RECOMENDADO

Capítulo 26 · Integración y Despliegue Continuo (CI/CD) para Pipelines de Datos RECOMENDADO

Capítulo 26A · Pruebas de Software en Datos: Pytest, Chispa y Testcontainers RECOMENDADO

CAPÍTULO 23

Orquestación de Flujos: Apache Airflow y Azure Data Factory NÚCLEO

> *"Un clúster distribuido capaz de computar petabytes en segundos carece de utilidad productiva si sus tareas se ejecutan a ciegas o en momentos aleatorios. La orquestación es el sistema nervioso central de la plataforma de datos: resuelve dependencias entre sistemas heterogéneos, administra reintentos ante fallos transitorios y garantiza que la información esté disponible con puntualidad matemática."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender el rol de los orquestadores analíticos frente a programadores de tareas elementales como *Cron*.
- Evaluar y comparar críticamente las arquitecturas de **Apache Airflow** (versión moderna 2.10+ / 3.0) y **Azure Data Factory (ADF)**.
- Diseñar grafos acíclicos dirigidos (**DAGs**) en Airflow aplicando el paradigma **TaskFlow API** (`@dag`, `@task`), operadores diferibles con `deferrable=True` y programación reactiva basada en **Datasets / Assets**.
- Dominar la semántica matemática de intervalos de datos (`logical_date`, `data_interval_start`, `data_interval_end`) para garantizar ejecuciones estrictamente **idempotentes**.
- Construir pipelines nativos en **Azure Data Factory** articulando *Linked Services*, *Datasets*, *Activities* e *Integration Runtimes* (Azure IR vs. Self-Hosted IR).

Definición 23.1 (Orquestación de Pipelines de Datos)

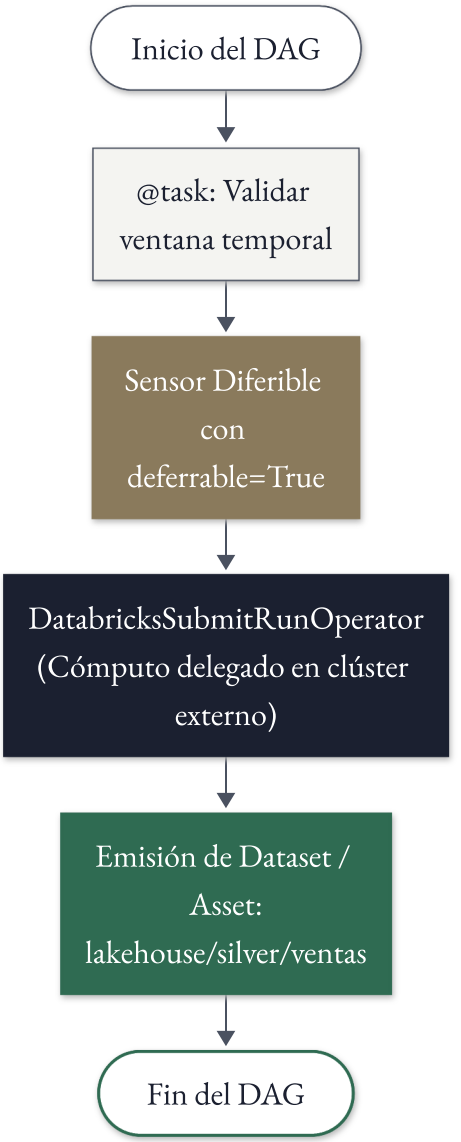
La **orquestación de datos** es la automatización programática del ciclo de vida integral de los flujos de trabajo analíticos. Comprende la resolución de dependencias topológicas entre tareas heterogéneas, la planificación cronológica o reactiva por eventos, la asignación dinámica de recursos computacionales, la gestión de reintentos con retraso exponencial ante fallos transitorios y la emisión de métricas de cumplimiento de acuerdos de nivel de servicio (SLA).

23.1 Comparativa Arquitectónica: Apache Airflow vs. Azure Data Factory

Criterio de Selección	Apache Airflow (2.10+ / 3.0)	Azure Data Factory (ADF)
Filosofía Fundamental	*Pipelines as Code* (Python nativo, control total).	Orquestación visual declarativa basada en JSON (*Low-Code/UI-First*).
Flexibilidad Dinámica	Total: soporte para bucles dinámicos, librerías personalizadas y SDKs.	Rígida: acotada por los bloques y conectores provistos por Azure.
Infraestructura	Requiere gestión (Astronomer, MWAA, Cloud Composer, Kubernetes).	Totalmente administrada (*Serverless / PaaS*) en Azure.
Conectividad Híbrida	Agentes en red o workers desplegados en cualquier nube u on-premise.	**Self-Hosted Integration Runtime (SHIR)** para cruzar firewalls locales.
Carga Computacional	**Director liviano:** Delega el cómputo a motores externos (Databricks, Snowflake).	**Orquestador + Motor ETL:** Ejecuta Data Flows (Spark gestionado) o delega.
CI/CD y Testing	Excelente: pruebas unitarias con `pytest`, linters y control Git estándar.	Exportación de plantillas ARM/Bicep y ramas de publicación en Azure DevOps.

23.2 Anatomía de un DAG en Apache Airflow: TaskFlow API y Datasets

El estándar moderno de Apache Airflow utiliza la sintaxis declarativa **TaskFlow API** (`@dag`, `@task`) y el parámetro `schedule` (reemplazando al deprecado `schedule_interval`):



Operadores Diferibles (*Deferrable Operators*)

Los sensores tradicionales bloquean permanentemente un hilo de ejecución (*worker slot*) mientras esperan que ocurra un evento externo, desperdiciando recursos del clúster. Con `deferrable=True`, el operador suspende su ejecución y cede su worker, registrándose en el proceso asíncrono **Triggerer** de Airflow hasta que el evento se activa.

Programación Reactiva basada en Datasets / Assets

En lugar de acoplar DAGs mediante horarios fijos en cron, Airflow permite que un pipeline productor declare un **Dataset** en sus salidas (`outlets`), disparando automáticamente el DAG consumidor en el instante en que los datos se confirman:

PYTHON

```python
from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.datasets import Dataset
from airflow.providers.databricks.operators.databricks import
DatabricksSubmitRunOperator
from airflow.sensors.external_task import ExternalTaskSensor

# Definición del activo de datos compartido (URI canónica)
dataset_ventas_silver = Dataset("lakehouse://silver/ventas_diarias")

argumentos_por_defecto = {
    'owner': 'data_engineering',
    'depends_on_past': False,
    'email_on_failure': True,
    'retries': 3,
    'retry_delay': timedelta(minutes=3),
    'retry_exponential_backoff': True
}

@dag(
    dag_id='pipeline_ventas_medallon',
    default_args=argumentos_por_defecto,
    schedule='0 3 * * *',  # Ejecución diaria programada a las 03:00 UTC
    start_date=datetime(2026, 1, 1),
    catchup=False,
    max_active_runs=1,
    tags=['ventas', 'lakehouse']
)
def pipeline_ventas():

    @task
    def pre_validacion(data_interval_start=None, data_interval_end=None):
        print(f"[INTERVALO] Ventana de datos desde {data_interval_start} hasta
{data_interval_end}")
        return {"inicio": str(data_interval_start)}

    # Sensor diferible de Airflow core (libera el worker slot mientras espera)
    esperar_datos_crudos = ExternalTaskSensor(
        task_id='esperar_ingesta_cruda',
        external_dag_id='dag_ingesta_landing',
        allowed_states=['success'],
        execution_delta=timedelta(hours=1), # Ajuste si el DAG externo corre a
las 02:00 UTC
        deferrable=True,                    # Clave de eficiencia: ejecución
asíncrona
        timeout=7200
    )
```

```
    tarea_databricks = DatabricksSubmitRunOperator(
        task_id='procesar_silver_en_databricks',
        databricks_conn_id='databricks_default',
        new_cluster={
            'spark_version': '14.3.x-scala2.12',
            'node_type_id': 'Standard_D4ds_v5',
            'num_workers': 2
        },
        notebook_task={
            'notebook_path': '/Production/Pipelines/02_Silver_Ventas',
            'base_parameters': {'fecha_proceso': '{{ ds }}'}
        },
        outlets=[dataset_ventas_silver]  # Emite el activo para activar DAGs
dependientes
    )

    pre_validacion() >> esperar_datos_crudos >> tarea_databricks

instancia_dag = pipeline_ventas()
```

23.3 Semántica Matemática de Intervalos de Datos e Idempotencia

En Apache Airflow, el tiempo de planificación se define de forma rigurosa:

- **Para una expresión cron `0 3 * * *` (ejecución diaria a las 03:00 UTC):**
  - Una ejecución disparada el **3 de octubre a las 03:00 UTC** no procesa de 00:00 a 00:00; procesa el intervalo comprendido entre la corrida programada anterior y la actual:

$$\text{data_interval_start} = \text{2026-10-02 03:00:00 UTC}$$

$$\text{data_interval_end} = \text{2026-10-03 03:00:00 UTC}$$

$$\text{logical_date} = \text{2026-10-02 03:00:00 UTC}$$

- **Para la macro `@daily` (`0 0 * * *`):**
  - Únicamente en este caso el intervalo abarca desde `00:00:00` hasta `00:00:00` del día siguiente.

Garantía de Idempotencia en Pipelines

Un pipeline es **idempotente** si la reejecución de cualquier intervalo temporal produce exactamente el mismo estado en la base de datos destino:

$$\text{Pipeline}(\text{Pipeline}(D, \text{intervalo})) = \text{Pipeline}(D, \text{intervalo})$$

Esto exige que el código SQL o PySpark parametrice sus filtros con `{{ ds }}` y escriba mediante reemplazo atómico de partición (`replaceWhere` en Delta Lake) en lugar de inserciones acumulativas ciegas.

23.4 Orquestación en Azure Data Factory (ADF)

En arquitecturas empresariales sobre Microsoft Azure, **Azure Data Factory (ADF)** es el servicio gestionado líder para orquestar movimientos e ingestas de datos.

Componentes Fundamentales de ADF

1. **Linked Services (Servicios Vinculados):** Definen la información de conexión e identidad hacia almacenes externos (Azure Key Vault, ADLS Gen2, Databricks, PostgreSQL local).
2. **Datasets (Conjuntos de Datos):** Punteros tipados que hacen referencia a las estructuras de datos dentro de un Linked Service (ej. carpeta de archivos Parquet en ADLS Gen2).
3. **Activities (Actividades):** Pasos de ejecución dentro de un pipeline:
   - *Actividades de Movimiento:* `Copy Activity` para transferencias masivas optimizadas.
   - *Actividades de Transformación:* `Databricks Notebook Activity`, `Databricks Spark Jar Activity`, `Stored Procedure Activity`.
   - *Actividades de Control de Flujo:* `Lookup`, `ForEach`, `If Condition`, `Execute Pipeline`, `Web Activity`.
4. **Integration Runtime (IR):** La infraestructura de cómputo que ejecuta las actividades:
   - **Azure IR:** Cómputo serverless elástico administrado por Microsoft para transferencias en la nube pública.
   - **Self-Hosted IR (SHIR):** Agente de software instalado en máquinas virtuales privadas o servidores on-premise para extraer datos corporativos detrás de firewalls sin exponer puertos a internet.

Definición JSON de un Pipeline de Producción en ADF

El siguiente manifiesto ilustra un pipeline de ADF que extrae parámetros dinámicos, ejecuta una validación previa y dispara una transformación en Azure Databricks:

```JSON
{
  "name": "Pipeline_Ingesta_Medallon_ADF",
  "properties": {
    "parameters": {
      "FechaEjecucion": {
        "type": "string",
        "defaultValue": "@formatDateTime(pipeline().TriggerTime, 'yyyy-MM-dd')"
      }
    },
    "activities": [
      {
        "name": "Validar_Disponibilidad_Landing",
        "type": "Validation",
        "dependsOn": [],
        "typeProperties": {
          "dataset": {
            "referenceName": "Dataset_ADLS_Bronze_Landing",
            "type": "DatasetReference"
          },
          "timeout": "00:30:00",
          "sleep": 60,
          "childItems": true
        }
      },
      {
        "name": "Transformar_Silver_Databricks",
        "type": "DatabricksNotebook",
        "dependsOn": [
          {
            "activity": "Validar_Disponibilidad_Landing",
            "dependencyConditions": ["Succeeded"]
          }
        ],
        "linkedServiceName": {
          "referenceName": "LinkedService_AzureDatabricks",
          "type": "LinkedServiceReference"
        },
        "typeProperties": {
          "notebookPath": "/Production/Pipelines/02_Silver_Ventas",
          "baseParameters": {
            "fecha_proceso": {
              "value": "@pipeline().parameters.FechaEjecucion",
              "type": "Expression"
            }
          }
        }
      }
    ],
    "annotations": ["Produccion", "Medallon", "FinOps"]
```

```
  }
}
---

## 23.5 Ecosistema Moderno de Orquestación: Airflow vs. Dagster vs. Prefect

Aunque Apache Airflow mantiene el dominio del mercado corporativo, la última
generación de orquestadores introdujo paradigmas que resuelven fricciones
históricas de desarrollo local y modelado de datos:

```mermaid
graph TD
    subgraph Orquestadores ["PARADIGMAS DE ORQUESTACIÓN MODERNA"]
        AF["Apache Airflow<br/>- Enfoque: Centrado en Tareas (Task-Centric)<br/>-
Fortaleza: Madurez, 100+ proveedores oficiales, estándar corporativo<br/>-
Limitación: DAG estático, sobrecosto de Scheduler"]
        DG["Dagster<br/>- Enfoque: Centrado en Activos (Software-Defined Assets)
<br/>- Fortaleza: Linaje nativo, particionamiento elegante, testing local<br/>-
Limitación: Curva de adopción conceptual"]
        PF["Prefect<br/>- Enfoque: Código Puro como Flujo (Code-as-Workflows)
<br/>- Fortaleza: Lógica Python dinámica, ejecución ligera, rápido
desarrollo<br/>- Limitación: Ecosistema corporativo menor que Airflow"]
    end
    style Orquestadores fill:#ffffff,stroke:#4a5160,stroke-width:1px
    style AF fill:#1b2030,color:#ffffff,stroke:#1b2030,stroke-width:2px
    style DG fill:#8a7a5c,color:#ffffff,stroke:#8a7a5c,stroke-width:2px
    style PF fill:#fbf9f4,stroke:#4a5160,stroke-width:1px
```

1. Dagster: Orquestación Basada en Activos (*Software-Defined Assets*)

En lugar de pensar en *"la tarea A ejecuta un script y luego la tarea B ejecuta otro"*, Dagster modela **qué activo de datos debe existir**:

PYTHON

```python
# Definición declarativa de un activo en Dagster
from dagster import asset
import pandas as pd

@asset
def clientes_limpios() -> pd.DataFrame:
    df = pd.read_parquet("data/clientes.parquet")
    return df.dropna(subset=["correo"])

@asset
def metricas_ventas_por_cliente(clientes_limpios: pd.DataFrame) -> pd.DataFrame:
    # Dagster deduce automáticamente la dependencia a partir del nombre del
argumento
    ventas = pd.read_parquet("data/ventas.parquet")
    return ventas.merge(clientes_limpios,
on="cliente_id").groupby("segmento").agg({"monto_usd": "sum"})
```
```

2. Prefect: Flujos Dinámicos con Código Python Nativo

Prefect permite transformar cualquier script de Python ordinario en un flujo observable simplemente añadiendo decoradores:

```python
# Flujo dinámico nativo en Prefect 2.x/3.x
from prefect import flow, task

@task(retries=3, retry_delay_seconds=10)
def extraer_datos_api(lote: int):
    return {"lote": lote, "filas": [1, 2, 3]}

@flow(name="pipeline_dinamico_prefect")
def flujo_principal(total_lotes: int = 5):
    # Permite bucles e if/else dinámicos nativos en Python sin mapping complejo
    for i in range(total_lotes):
        extraer_datos_api(i)
```

Criterio de Selección:

- **Seleccione Airflow si:** Su organización requiere un estándar industrial consolidado con miles de integraciones maduras, orquesta sistemas híbridos multi-nube y cuenta con ingenieros experimentados en su operación.
- **Seleccione Dagster si:** El equipo está enfocado intensamente en calidad y linaje de datos (Analytics Engineering, data mesh), integrando dbt y validaciones a nivel de columna como ciudadanos de primera clase.
- **Seleccione Prefect si:** El equipo prioriza la velocidad de desarrollo en Python puro, necesita desplegar tareas en contenedores efímeros sin la sobrecarga operativa de un scheduler pesado, y requiere flujos dinámicos con lógica condicional en tiempo de ejecución.

23.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cargar Grandes Volúmenes de Datos en la Memoria del Worker de Airflow Ejecutar código que descargue DataFrames masivos en el worker de Airflow provoca colapsos por *Out Of Memory* (OOM) y bloquea el scheduler. Airflow debe actuar siempre como un director de orquesta ligero que delega el cómputo pesado a Databricks, Snowflake o Spark en Kubernetes.

Anti-patrón 2: Habilitar `catchup=True` con Fechas de Inicio Antiguas Crear un DAG con una fecha de inicio de hace un año y dejar `catchup=True` dispara cientos de ejecuciones simultáneas al encenderlo, saturando las bases de datos transaccionales de origen y disparando la facturación cloud.

Anti-patrón 3: Diseñar Pipelines en ADF con Bucles `ForEach` Secuenciales sobre Millones de Archivos Iterar archivo por archivo en ADF mediante un bucle de actividades incurre en costos elevados por ejecución de actividades y tiempos de respuesta ineficientes. Utilice la `Copy Activity` con comodines (*Wildcards*) o procese mediante Databricks Auto Loader.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 23.1 (Sincronización de DAGs a Horas Distintas y Datasets):
El DAG de ingesta de CRM finaliza diariamente a las **02:00 UTC**. El DAG de consolidación financiera está programado para correr a las **04:00 UTC**.

1. Si se utiliza un `ExternalTaskSensor` en el DAG de las 04:00 UTC, ¿por qué fallará si se deja la configuración por defecto? ¿Qué parámetro exacto debe ajustarse?
2. ¿Cómo se resuelve esta dependencia de forma nativa y desacoplada utilizando **Datasets / Assets** en Airflow moderno?

Solución 23.1:

1. *Fallo y corrección con sensor:* Por defecto, `ExternalTaskSensor` busca una ejecución en el DAG externo que tenga exactamente el **mismo `logical_date`**. Al ejecutarse a horas distintas (04:00 vs 02:00), las marcas de tiempo difieren por 2 horas, por lo que el sensor esperará hasta alcanzar el timeout. La corrección exige configurar el desfase explícito:

```python
ExternalTaskSensor(
    task_id='esperar_crm',
    external_dag_id='dag_ingesta_crm',
    execution_delta=timedelta(hours=2),
    deferrable=True
)
```

2. *Solución moderna con Datasets / Assets:* Se desacoplan los horarios por cron.
   - El DAG de CRM declara en su tarea terminal: `outlets=[Dataset("lakehouse://silver/crm")]`.
   - El DAG financiero se declara sin expresión cron fija: `schedule=[Dataset("lakehouse://silver/crm")]`. Tan pronto como el DAG de CRM confirma la actualización del dataset en la base de metadatos de Airflow, el DAG financiero se activa inmediatamente de forma reactiva, eliminando esperas ociosas y desfasajes horarios.

Ejercicio 23.2 (Conectividad Híbrida Segura en Azure Data Factory):
Una entidad bancaria necesita extraer diariamente 500 GB de datos desde un servidor Oracle alojado en un centro de cómputo privado (*On-Premise*) sin conexión pública a internet hacia un contenedor ADLS Gen2 en Azure. Describe los componentes arquitectónicos de ADF requeridos para cumplir este flujo de forma segura.

Solución 23.2:

1. **Self-Hosted Integration Runtime (SHIR):** Se instala el agente de software SHIR en una máquina virtual Windows ubicada dentro de la red corporativa del banco con acceso local a la base de datos Oracle. El agente solo requiere salida HTTPS (puerto 443 saliente) hacia Azure; no requiere abrir puertos entrantes en el firewall bancario.
2. **Linked Services:**
   - *Linked Service Oracle:* Configurado para utilizar el SHIR local, autenticándose mediante credenciales seguras almacenadas en Azure Key Vault.
   - *Linked Service ADLS Gen2:* Conectado al almacenamiento del Lakehouse mediante una Identidad Administrada (*Managed Identity*).
3. **Copy Activity en ADF:** Se ejecuta un pipeline en ADF cuya actividad de copia lee a través del túnel seguro establecido por el SHIR y escribe directamente en el contenedor Bronze en la nube con cifrado TLS 1.3 en tránsito.

23.7 Resumen y Puntos Clave

- **Orquestador como director liviano:** Airflow y ADF deben coordinar dependencias y estados sin procesar transformaciones analíticas pesadas en su propia infraestructura.
- **Modernidad en Airflow:** La TaskFlow API (`@dag`, `@task`), los operadores diferibles (`deferrable=True`) y la programación por Datasets representan el estándar de alta concurrencia.
- **Intervalos cron y semántica temporal:** Para `0 3 * * *`, el intervalo de datos procesado cubre de 03:00 a 03:00 del periodo precedente, requiriendo parametrización determinista con `{{ ds }}` para garantizar idempotencia.
- **Ecosistema de Azure Data Factory:** Articula *Linked Services*, *Datasets*, *Activities* y *Integration Runtimes* (Azure IR para nube y Self-Hosted IR para redes híbridas privadas).

Lecturas Recomendadas

1. **Harenslak, B. & de Ruiter, J. (2021).** *Data Pipelines with Apache Airflow.* Manning Publications.
2. **Apache Airflow Documentation (2024).** *Data-Aware Scheduling, Datasets, and Deferrable Operators.* Apache Software Foundation (`https://airflow.apache.org/docs/`).
3. **Microsoft Learn (2024).** *Azure Data Factory Pipelines, Integration Runtime and Hybrid Networking Architecture.* Microsoft Documentation.

CAPÍTULO 23A

Orquestación Avanzada con Apache Airflow: Arquitectura Interna, Dynamic Mapping y Datasets RECOMENDADO

> *"Un DAG elemental con dos tareas lineales puede escribirse en una tarde de exploración; operar una plataforma de orquestación a escala corporativa con cientos de pipelines concurrentes exige dominar los mecanismos internos de Apache Airflow. Desde el bucle del Scheduler y la ejecución asíncrona de los Triggerers hasta el mapeo dinámico de tareas y la programación orientada a datos, la excelencia en orquestación radica en convertir el orquestador en un director liviano, tolerante a fallos y desacoplado de la carga pesada de cómputo."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

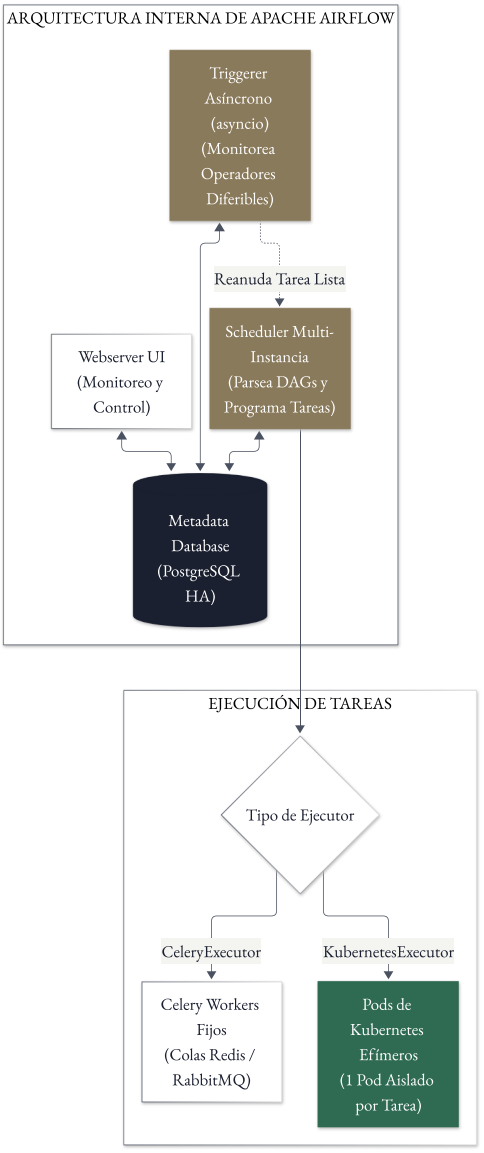
- Comprender la arquitectura de procesos interna de **Apache Airflow**: el ciclo del **Scheduler**, la base de datos de metadatos, el servidor web y los **Triggerers** asíncronos.
- Evaluar y seleccionar el ejecutor óptimo para producción: `CeleryExecutor` frente a `KubernetesExecutor` (aislamiento de pods efímeros por tarea).
- Aplicar las transformaciones arquitectónicas de **Airflow 2.10+ y la llegada de Airflow 3.0**, dominando la semántica de ventanas temporales (`logical_date`, `data_interval_start`, `data_interval_end`) y el nuevo SDK desacoplado.
- Implementar **Mapeo Dinámico de Tareas (*Dynamic Task Mapping*)** con `.expand()` y `.partial()` para paralelizar cargas de trabajo arbitrarias en tiempo de ejecución.
- Utilizar **Operadores Diferibles (*Deferrable Operators*)** con `deferrable=True` para liberar slots de workers durante esperas prolongadas.
- Orquestar canalizaciones reactivas mediante **Datasets / Assets** (*Data-Aware Scheduling*).
- Administrar secretos de producción de forma segura mediante **Secret Backends** (AWS Secrets Manager, HashiCorp Vault) y gobernar la concurrencia con *Pools* y cuotas.

Definición 23A.1 (El Principio del Director Liviano - Lean Orchestrator) En la ingeniería de datos moderna, un orquestador como Apache Airflow debe actuar estrictamente como un **director de orquesta liviano** (*Lean Orchestrator*): su responsabilidad es coordinar dependencias, gestionar reintentos y monitorear el estado del pipeline.

El cómputo pesado de transformación nunca debe ejecutarse dentro de los procesos de Airflow. Toda carga intensiva de CPU o memoria debe delegarse inmediatamente a motores especializados externos (un clúster de Databricks, un job de Spark en EMR, un modelo de dbt o una consulta en Snowflake/BigQuery).

23A.1 Arquitectura Interna de Procesos y Ejecutores

Un despliegue de Apache Airflow en producción se compone de cuatro subsistemas desacoplados:



1. El Ciclo de Vida del Scheduler (*Scheduler Loop*)

El Scheduler ejecuta continuamente tres tareas fundamentales:

- **Parseo de Archivos DAG:** Inspecciona periódicamente el directorio de DAGs (típicamente cada 30 segundos) para detectar cambios en el código Python.
- **Evaluación de Dependencias:** Consulta la base de datos de metadatos para verificar si las tareas padre han finalizado con éxito (`TaskInstanceState.SUCCESS`).
- **Encolado de Tareas:** Si una tarea está lista para ejecutarse, cambia su estado a `SCHEDULED` y la envía a la cola del ejecutor seleccionado.

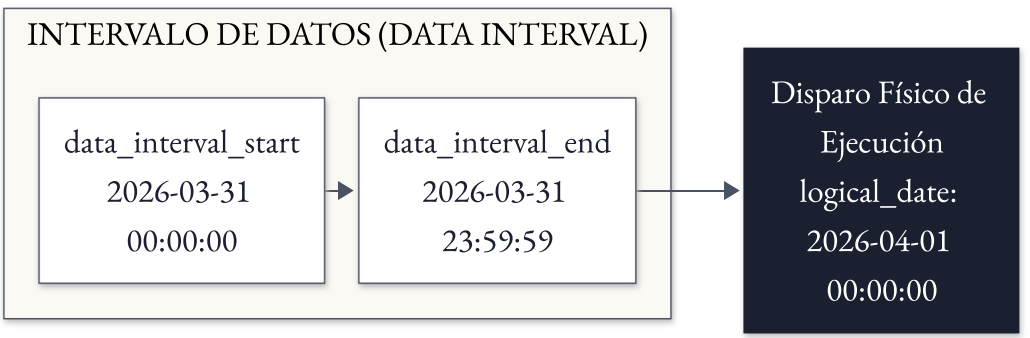
2. Comparativa de Ejecutores: Celery vs. Kubernetes

- `CeleryExecutor:` Utiliza un conjunto de trabajadores (*workers*) de larga duración que escuchan mensajes en una cola distribuida (Redis o RabbitMQ).
  - *Ventaja:* Latencia de arranque de tarea casi nula (milisegundos).
  - *Desventaja:* Todos los workers comparten la misma imagen y dependencias de Python; una biblioteca incompatible puede generar colisiones entre DAGs.
- `KubernetesExecutor:` Por cada tarea que el Scheduler despacha, el ejecutor contacta a la API de Kubernetes para crear un **Pod dedicado y efímero**:
  - *Ventaja de Oro:* **Aislamiento total de dependencias y recursos.** La Tarea A puede ejecutarse en un pod con Python 3.10 y 2 GB de RAM, mientras que la Tarea B corre en un pod con Python 3.12, dependencias de PyTorch y una GPU dedicada. Al finalizar la tarea, el pod se destruye por completo.
  - *Desventaja:* Sobrecosto de arranque del pod (2 a 5 segundos).

23A.2 Evolución de la Semántica Temporal: Hacia Airflow 3.0

Uno de los conceptos más confusos en las versiones iniciales de Airflow fue el concepto de `execution_date`, que no representaba el momento en que la tarea corría, sino el inicio del intervalo temporal previo.

En **Airflow 2.10+ y Airflow 3.0**, la semántica se ha formalizado matemáticamente:



- `data_interval_start` y `data_interval_end:` Delimitan con precisión el rango de datos que la tarea debe procesar (por ejemplo, transacciones ocurridas entre las 00:00 y las 23:59 del día de ayer).
- **Eliminación de `schedule_interval`:** En Airflow 3.0 se eliminó definitivamente el parámetro legado `schedule_interval`, reemplazado unívocamente por `schedule`.

- **De `Dataset` a `Asset`:** En Airflow 3.0, el concepto de *Dataset* pasa a llamarse formalmente `Asset` (`from airflow.sdk import Asset`), ampliando su capacidad para representar no solo tablas o archivos, sino cualquier activo de datos, modelo de ML o mensaje que reactive dependencias en grafos orientados a datos.
- **Airflow 3.0 SDK (`airflow.sdk`):** Desacopla la interfaz de definición y ejecución de tareas de la base de datos de metadatos del orquestador. Las tareas ya no importan módulos internos pesados de Airflow ni requieren conexiones TCP directas al puerto 5432 de PostgreSQL; se comunican mediante una API de ejecución segura (*Task Execution API* / REST gRPC), haciendo que los workers sean completamente efímeros y seguros.
- **CronTriggerTimetable en Airflow 3:** En Airflow 2, un cron diario `0 4 * * *` se ejecutaba al finalizar el día siguiente (un desfase de 24 horas que desconcertaba a principiantes). En Airflow 3, las expresiones cron adoptan de forma predeterminada `CronTriggerTimetable`: el DAG se dispara exactamente a las 04:00 AM en el instante fijado, y el intervalo de datos queda alineado sin desfases temporales artificiales.

Kubernetes Esencial para el Data Engineer: `KubernetesPodOperator`

Muchos ingenieros cometen el error de instalar todas las dependencias del negocio (`pyspark`, `dbt-core`, `scikit-learn`, `great-expectations`) dentro de la misma imagen de Docker de Airflow. Esto provoca infiernos de dependencias (*Dependency Hell*) e imágenes gigantescas de 8 GB.

En producción, el estándar de la industria es utilizar **Kubernetes** como sustrato de cómputo para los pipelines:

1. **Pod:** La unidad mínima de cómputo en Kubernetes. Es un grupo de uno o más contenedores que comparten almacenamiento y red.
2. **KubernetesPodOperator (KPO):** Cada tarea del DAG levanta dinámicamente un Pod independiente con su propia imagen ligera de Docker, CPU y memoria RAM asignada:

```python
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator
from kubernetes.client import models as k8s

tarea_dbt = KubernetesPodOperator(
    task_id="transformacion_dbt_gold",
    name="job-dbt-pod",
    image="mi-registro.azurecr.io/pipelines/dbt-prod:1.2.0",
    cmds=["dbt", "run", "--select", "tag:gold"],
    container_resources=k8s.V1ResourceRequirements(
        requests={"memory": "4Gi", "cpu": "2000m"},
        limits={"memory": "8Gi", "cpu": "4000m"}
    ),
      is_delete_operator_pod=True, # Destruye el Pod al terminar, liberando recursos
    in_cluster=True
)
```

3. **Beneficio Radical:** Si la tarea de dbt requiere Python 3.10 y una tarea de IA requiere Python 3.12 con PyTorch, ambas corren en Pods aislados sobre el mismo clúster sin generar colisiones de librerías y garantizando que Airflow permanezca ligero.

23A.3 Mapeo Dinámico de Tareas (*Dynamic Task Mapping*)

Históricamente, si un pipeline necesitaba procesar 50 archivos depositados en S3, el ingeniero debía definir manualmente 50 tareas estáticas en el DAG o agrupar los 50 archivos dentro de un solo bucle secuencial lento.

El **Mapeo Dinámico de Tareas** (`.expand()` y `.partial()`) permite generar dinámicamente un número arbitrario de tareas en paralelo en tiempo de ejecución, en función del resultado computado por una tarea previa:

```python
from datetime import datetime
from airflow.decorators import dag, task
from typing import List

@dag(
    dag_id="dynamic_task_mapping_lakehouse",
    schedule="0 2 * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["avanzado", "dynamic-mapping"]
)
def pipeline_mapeo_dinamico():

    @task
```

```
    def listar_archivos_novedades() -> List[str]:
        """Descubre dinámicamente los lotes depositados en S3."""
        # En producción, esto consulta la API de S3
        return [
            "s3://landing/ventas_20260331_lote1.json",
            "s3://landing/ventas_20260331_lote2.json",
            "s3://landing/ventas_20260331_lote3.json",
            "s3://landing/ventas_20260331_lote4.json"
        ]

    @task
    def procesar_archivo_individual(ruta_archivo: str, tabla_destino: str) ->
int:
        """Se ejecuta en paralelo N veces (una instancia por cada archivo)."""
        print(f"Procesando {ruta_archivo} hacia {tabla_destino}...")
        # Lógica de procesamiento...
        return 2500  # Filas procesadas

    @task
    def consolidar_metricas(conteos_filas: List[int]) -> None:
        """Agrega los resultados de todas las tareas dinámicas mapeadas."""
        total_filas = sum(conteos_filas)
        print(f"Total consolidado de la corrida: {total_filas} registros
procesados.")

    # 1. Tarea que genera la lista dinámica
    archivos = listar_archivos_novedades()

    # 2. Mapeo Dinámico: .partial() fija parámetros comunes; .expand() paraleliza
la lista
    tareas_procesamiento = procesar_archivo_individual.partial(
        tabla_destino="lakehouse.silver_ventas"
    ).expand(
        ruta_archivo=archivos
    )

    # 3. Consolidación de salida
    consolidar_metricas(tareas_procesamiento)

instancia_dag = pipeline_mapeo_dinamico()
```

23A.4 Operadores Diferibles (*Deferrable Operators*) y el Triggerer

En pipelines analíticos es común que una tarea deba esperar horas a que una base de datos externa termine un volcado por lotes o que un clúster de Databricks finalice su cálculo.

Un sensor clásico ejecuta un bucle de sondeo bloqueante:

```python
# ANTI-PATRÓN: Bloquea permanentemente un slot de worker consumiendo CPU y RAM
sensor_bloqueante = ExternalTaskSensor(
    task_id="esperar_dag_previo",
    external_dag_id="ingesta_fuente_primaria",
    mode="poke", # Ocupa el worker slot continuamente
    poke_interval=60,
    timeout=7200
)
```

La Solución Diferible (`deferrable=True`):

Al configurar `deferrable=True`, el sensor transfiere su control al proceso central **Triggerer**:

1. La tarea libera su worker slot en el clúster inmediatamente.
2. El **Triggerer** ejecuta una corrutina asíncrona no bloqueante con `asyncio`, monitoreando miles de eventos concurrentes en un solo proceso ligero.
3. Cuando el evento externo se cumple, el Triggerer notifica al Scheduler para reanudar la tarea en un worker solo para su fase de cierre.

23A.5 Gestión Segura de Secretos y Control de Concurrencia

1. Secret Backends de Producción

En entornos corporativos regulados, **las credenciales nunca se guardan en la base de metadatos de Airflow**. Se configura un proveedor de secretos externo mediante variables de entorno en el despliegue de Airflow:

```bash
# Configuración en airflow.cfg o variables de entorno del contenedor
AIRFLOW__SECRETS__BACKEND="airflow.providers.amazon.aws.secrets.secrets_manager.SecretsManagerBackend"
AIRFLOW__SECRETS__BACKEND_KWARGS='{"connections_prefix": "airflow/connections", "variables_prefix": "airflow/variables"}'
```

Cuando un DAG solicita una conexión `BaseHook.get_connection("postgres_oltp")`, Airflow consulta dinámicamente **AWS Secrets Manager** utilizando el rol IAM del clúster, recuperando la contraseña al vuelo sin exponerla en interfaces web.

2. Pools de Concurrencia

Para evitar que un DAG con mapeo dinámico lance 200 conexiones simultáneas contra una base de datos relacional y sature su límite de conexiones (*Connection Limit Exhaustion*), se definen **Pools**:

```python
@task(pool="postgres_produccion_pool", pool_slots=1)
def extraer_datos_oltp(lote_id: int):
    # Airflow asegura que nunca se ejecuten más tareas en este pool
    # que la capacidad máxima configurada en la UI (ej. máximo 5 slots
concurrentes)
    ...
```

23A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Código Pesado en el Nivel Superior del Archivo Python (*Top-Level Code*)
Escribir conexiones a bases de datos, consultas a APIs o lecturas de archivos fuera de las funciones decoradas con `@task` o en el cuerpo principal del archivo `.py` es una catástrofe de rendimiento.
Causa: El Scheduler de Airflow ejecuta e interpreta la totalidad del archivo Python cada 30 segundos. Si 100 DAGs ejecutan consultas pesadas en el nivel superior, el Scheduler saturará el 100% de la CPU del servidor solo parseando archivos, provocando retrasos masivos en las ejecuciones.
Corrección: Todo código de I/O, llamadas a APIs o conexiones a bases de datos debe residir estrictamente **dentro de las tareas ejecutables**.

Anti-patrón 2: Tareas No Idempotentes que Dependen del Momento de Ejecución Real (`datetime.now()`)
Usar `datetime.now()` dentro de una tarea para filtrar particiones provoca que, si el DAG falla y se reejecuta manualmente tres días después, la tarea procese datos distintos a los que debía procesar originalmente.
Corrección: Utilice siempre las variables contextuales deterministas inyectadas por Airflow: `data_interval_start` o `logical_date`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 23A.1 (Implementación de Callback de Alerta ante Fallos):
Diseñe una función de callback en Python `alertar_fallo_slack(context)` que capture el nombre del DAG, el identificador de la tarea fallida, el rango temporal del lote

(`data_interval_start`) y la excepción producida, formateándola para su envío automático al canal de incidentes de ingeniería.

Solución 23A.1:

PYTHON

```python
import logging

def alertar_fallo_slack(context):
    dag_id = context.get("task_instance").dag_id
    task_id = context.get("task_instance").task_id
    intervalo_inicio = context.get("data_interval_start")
    excepcion = context.get("exception")
    log_url = context.get("task_instance").log_url

    mensaje_alerta = (
        f"🚨 *ALERTA DE PRODUCCIÓN AIRFLOW* 🚨\n"
        f"• *DAG:* `{dag_id}`\n"
        f"• *Tarea:* `{task_id}`\n"
        f"• *Intervalo de Datos:* `{intervalo_inicio}`\n"
        f"• *Error:* `{excepcion}`\n"
        f"• *Logs:* <{log_url}|Ver registros de ejecución>"
    )

    # En producción se envía mediante SlackWebhookHook o peticion HTTP
    logging.critical(f"Notificando fallo a canal de guardia: {mensaje_alerta}")

# Asignación en los argumentos por defecto del DAG
default_args = {
    "owner": "data_platform",
    "on_failure_callback": alertar_fallo_slack
}
```

23A.7 Resumen y Puntos Clave

- **Orquestación desacoplada:** Airflow debe actuar como un director liviano que coordina dependencias y delega el cómputo pesado a clústeres especializados externos.
- **KubernetesExecutor:** Proporciona aislamiento estricto de recursos y dependencias levantando un pod de Kubernetes dedicado y efímero por cada tarea.
- **Mapeo Dinámico (`.expand()`):** Permite paralelizar tareas en tiempo de ejecución adaptándose al volumen real de datos sin requerir grafos estáticos sobredimensionados.
- **Triggerer y Sensores Diferibles:** Liberan slots de trabajadores durante esperas prolongadas mediante corrutinas asíncronas no bloqueantes.
- **Secret Backends:** Externalizan contraseñas y claves hacia almacenes administrados (AWS Secrets Manager / Vault), erradicando credenciales en texto plano.

Lecturas Recomendadas

1. **Harenslak, J. & de Ruiter, J. (2021).** *Data Pipelines with Apache Airflow.* Manning Publications.
2. **Apache Airflow Project (2024).** *Apache Airflow Documentation: Dynamic Task Mapping and Deferrable Operators.* Apache Software Foundation.
3. **Astronomer Engineering (2024).** *Architecting Apache Airflow for Enterprise Production Scale.* Astronomer Guides.

CAPÍTULO 24

Transformación Modular con dbt, Snapshots e Idempotencia NÚCLEO

> *"La capa de transformación en el Lakehouse no debe ser un laberinto artesanal de scripts SQL dispersos y procedimientos almacenados opacos. dbt traslada los principios consagrados de la ingeniería de software —control de versiones en Git, modularidad, pruebas automatizadas, integración continua y documentación viva— al núcleo del modelado analítico de datos."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución paradigmática de ETL a **ELT** y el rol de **dbt (*data build tool*)** en la capa de transformación del almacén analítico.
- Estructurar proyectos modulares organizados en capas estándar: Fuentes (*Sources*), *Staging*, *Intermediate* y *Marts* (Hechos y Dimensiones).
- Dominar las materializaciones de dbt: `view`, `table`, `incremental` y `ephemeral`.
- Implementar modelos incrementales de alto rendimiento utilizando la estrategia `merge` y la macro `is_incremental()`.
- Automatizar el versionado histórico de Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) mediante **dbt Snapshots**.
- Formular y defender la propiedad de **idempotencia** en pipelines de datos con rigor matemático.

Definición 24.1 (Idempotencia en Ingeniería de Datos)

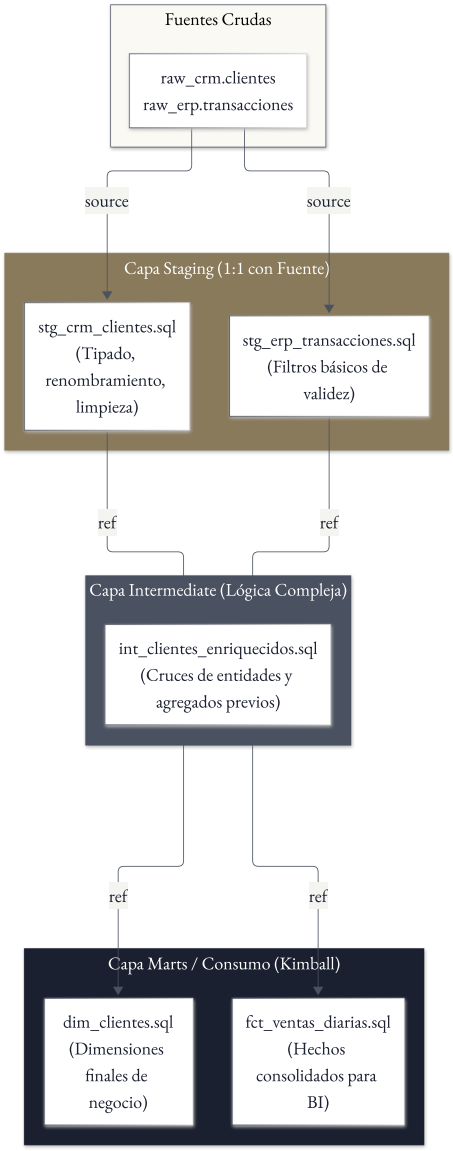
En ciencias de la computación, una función u operación de transformación $f(x)$ se define como **estrictamente idempotente** si su aplicación reiterada sobre un estado inicial determinado produce exactamente el mismo resultado final que una única ejecución:

$$f(f(x)) = f(x) \quad \forall x$$

En el contexto de los pipelines analíticos, esto garantiza que si una ejecución falla a medio camino, se reintenta por el orquestador o se vuelve a disparar manualmente para corregir un incidente, las tablas de destino convergen a un estado idéntico y consistente, sin duplicar registros, alterar métricas agregadas ni requerir intervenciones manuales de purga.

24.1 El Paradigma ELT y la Arquitectura Modular de dbt

En el modelo ETL tradicional, los datos se extraían de las fuentes operacionales y se transformaban en servidores intermedios antes de cargarse en el almacén de datos. En el paradigma moderno **ELT**, los datos se cargan primero en su estado crudo en el Lakehouse o almacén cloud (Snowflake, BigQuery, Databricks, Redshift), y **dbt orquesta las transformaciones directamente dentro del motor de cómputo columnar**, aprovechando su escalabilidad masiva mediante sentencias `SELECT` puras.



La Tríada Modular de dbt

1. **Capa Staging (`stg_`):** Mapeo 1:1 con las tablas de origen. Aplica limpieza cosmética: renombra columnas según convenciones corporativas, castea tipos de datos primitivos y elimina espacios en blanco sin realizar cruces complejos.
2. **Capa Intermediate (`int_`):** Resuelve lógica de negocio compleja, cruces entre múltiples modelos de staging y cálculos intermedios que no deben exponerse directamente a los analistas de negocio.
3. **Capa Marts (`fct_`, `dim_`):** Estructuras dimensionales finales conformadas bajo la metodología Kimball (tablas de hechos y dimensiones) preparadas para ser consultadas por

herramientas de visualización o analistas.

24.2 Materializaciones en dbt

La configuración `materialized` define cómo dbt compila el código SQL en la base de datos subyacente:

- **`view`:** Crea una vista SQL ordinaria. No consume almacenamiento físico; reejecuta la consulta en cada lectura. Ideal para la capa Staging o transformaciones ligeras.
- **`table`:** Descarta la tabla anterior y la reconstruye por completo en cada ejecución (`DROP TABLE ... CREATE TABLE AS SELECT`). Garantiza consistencia pero resulta prohibitivamente costosa en tablas con cientos de millones de registros.
- **`ephemeral`:** No crea ningún objeto en la base de datos; interpola el modelo como una expresión de tabla común (*Common Table Expression - CTE*) directamente en los modelos que lo referencian.
- **`incremental`:** Inserta únicamente las filas nuevas o actualizadas desde la última corrida del pipeline, minimizando el escaneo de datos y el gasto de cómputo en tablas masivas.

24.3 Modelado Incremental e Idempotente con la Estrategia `merge`

Para que un modelo incremental sea eficiente e idempotente, se combina la configuración de materialización con la macro `is_incremental()`:

SQL

```sql
-- models/marts/fct_pagos_recientes.sql
{{ config(
    materialized='incremental',
    unique_key='pago_id',
    incremental_strategy='merge',
    file_format='delta',
    on_schema_change='sync_all_columns'
) }}

WITH fuente_staging AS (
    SELECT
        pago_id,
        cuenta_id,
        monto_centavos,
        moneda,
        estado_pago,
        fecha_transaccion,
```

```
        fecha_actualizacion_sistema
    FROM {{ ref('stg_pasarela_pagos') }}

    {% if is_incremental() %}
        -- En ejecuciones subsecuentes, filtrar únicamente los registros
modificados
        -- Se añade una ventana de seguridad de 3 días para mitigar eventos
desordenados
        WHERE fecha_actualizacion_sistema >= (
            SELECT dateadd(day, -3, coalesce(max(fecha_actualizacion_sistema),
'1970-01-01'))
            FROM {{ this }}
        )
    {% endif %}
)

SELECT * FROM fuente_staging
```

Mecanismo de Fusión Atómica

Cuando dbt compila este modelo en una ejecución incremental:

1. Crea una tabla temporal ligera con las filas filtradas por la cláusula `is_incremental()`.
2. Ejecuta una sentencia `MERGE INTO` nativa:
   - Si el `pago_id` ya existe en la tabla de destino, actualiza sus campos con los valores recientes.
   - Si el `pago_id` no existe, inserta la nueva fila.
3. El resultado es **estrictamente idempotente**: ejecutar el comando `dbt run --select fct_pagos_recientes` tres veces consecutivas deja la tabla en el mismo estado sin generar duplicados.

24.4 Historial Automatizado: dbt Snapshots para SCD Tipo 2 (Sintaxis Clásica vs. YAML 1.9+)

Gestionar Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) manualmente mediante complejos procedimientos almacenados suele generar inconsistencias y roturas de linaje.

dbt Snapshots permite capturar el estado histórico de una tabla mutable de manera puramente declarativa. A partir de dbt 1.9+, es posible definirlos tanto en archivos SQL como en archivos YAML declarativos centralizados:

1. Sintaxis Declarativa en YAML (dbt 1.9+ / Recomendada)

YAML

```yaml
# snapshots/snapshots.yml
snapshots:
  - name: snp_clientes
    relation: source('crm_operacional', 'clientes')
    config:
      target_schema: snapshots
      unique_key: cliente_id
      strategy: check
      check_cols:
        - direccion
        - segmento_riesgo
        - telefono
        - correo_electronico
      invalidate_hard_deletes: true
```

2. Sintaxis Clásica en Archivo SQL

SQL

```sql
-- snapshots/snp_clientes.sql
{% snapshot snp_clientes %}

{{
    config(
      target_schema='snapshots',
      unique_key='cliente_id',
      strategy='check',
      check_cols=['direccion', 'segmento_riesgo', 'telefono',
'correo_electronico'],
      invalidate_hard_deletes=True
    )
}}

SELECT
    cliente_id,
    nombre_completo,
    direccion,
    segmento_riesgo,
    telefono,
    correo_electronico
FROM {{ source('crm_operacional', 'clientes') }}

{% endsnapshot %}
```

Estrategias de Snapshot: `check` vs. `timestamp`

- **Estrategia `check`:** dbt compara los valores de una lista explícita de columnas (`check_cols`). Si alguno cambia, dbt genera una nueva versión histórica. Si la base de datos de origen carece de una columna confiable de marca temporal de actualización, `check` es la estrategia idónea.

- **Estrategia `timestamp`:** Requiere una columna como `updated_at`. Si el valor de `updated_at` es más reciente que el registrado en la última ejecución, dbt asume que el registro mutó.

Columnas Meta Añadidas Automáticamente por dbt

Cada vez que se ejecuta el comando `dbt snapshot`, dbt gestiona de forma transparente cuatro columnas meta de control histórico:

- `dbt_scd_id`: Hash único que identifica la versión histórica del registro.
- `dbt_updated_at`: Marca temporal en la que ocurrió el snapshot.
- `dbt_valid_from`: Fecha/hora de inicio de validez de esta versión del registro.
- `dbt_valid_to`: Fecha/hora de finalización de validez. Si el registro es la versión actualmente activa, este campo se mantiene en `NULL`.

24.5 Macros Jinja, Calidad Automatizada y Documentación Viva

dbt no es solo un compilador SQL; es un entorno completo de ingeniería de software que traslada prácticas de desarrollo al almacén analítico:

1. Reutilización de Lógica mediante Macros Jinja

Las macros permiten encapsular fragmentos de SQL repetitivos en funciones parametrizadas:

```SQL
-- macros/convertir_moneda.sql
{% macro convertir_centavos_a_dolares(columna_monto, decimales=2) %}
    ROUND(CAST({{ columna_monto }} AS NUMERIC) / 100.0, {{ decimales }})
{% endmacro %}
```

Uso directo dentro de cualquier modelo:

```SQL
SELECT
    pago_id,
    {{ convertir_centavos_a_dolares('monto_centavos') }} AS monto_usd
FROM {{ ref('stg_pasarela_pagos') }}
```

2. Pruebas Automáticas: Genéricas vs. Singulares

- **Pruebas Genéricas (YAML):** Declaradas en `models/schema.yml` para validar restricciones estructurales:

```yaml
version: 2
models:
  - name: fct_pagos_recientes
    description: "Tabla de hechos de transacciones de pago verificadas"
    columns:
      - name: pago_id
        tests:
          - unique
          - not_null
      - name: estado_pago
        tests:
          - accepted_values:
              values: ['COMPLETADO', 'PENDIENTE', 'FALLIDO']
      - name: cuenta_id
        tests:
          - relationships:
              to: ref('dim_cuentas')
              field: cuenta_id
```

- **Pruebas Singulares (SQL):** Consultas personalizadas guardadas en `tests/assert_monto_total_positivo.sql`. Si la consulta retorna una o más filas, **el test falla**:

```sql
-- Si existen pagos completados con saldo negativo, la prueba detiene el pipeline
SELECT pago_id, monto_centavos
FROM {{ ref('fct_pagos_recientes') }}
WHERE estado_pago = 'COMPLETADO' AND monto_centavos < 0;
```

3. Documentación y Linaje Interactivo

Al ejecutar `dbt docs generate && dbt docs serve`, dbt compila los metadatos de los modelos, descripciones, tipos de datos y genera el grafo interactivo de linaje (*DAG View*), permitiendo a ingenieros y analistas auditar el origen exacto de cada columna sin inspeccionar manualmente el código fuente.

24.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Referencias Rígidas (*Hardcoded Names*) sin la Función `{{ ref(...) }}` Escribir `FROM analytics_prod.staging.stg_clientes` en lugar de `FROM {{ ref('stg_clientes') }}` destruye el grafo acíclico de dependencias de dbt. Al omitir `ref()`, dbt no puede deducir el orden de compilación, provocando que los modelos dependientes se ejecuten antes de que sus tablas fuentes hayan sido creadas o actualizadas.

Anti-patrón 2: Consultar la Tabla de Destino sin Coalesce en Modelos Incrementales Escribir `WHERE updated_at > (SELECT max(updated_at) FROM {{ this }})` sin envolver el `max()` en un `coalesce()` provocará que la consulta retorne `NULL` en la primera ejecución incremental si la tabla está vacía, impidiendo la carga de los datos iniciales.

Anti-patrón 3: Modelos Gigantescos Monolíticos en la Capa Marts Diseñar un único script SQL de 1.500 líneas con 20 subconsultas acopladas para generar una tabla analítica viola el principio de modularidad. Se debe descomponer la lógica en modelos intermedios reutilizables, permitiendo que cada etapa sea testeada de forma aislada.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 24.1 (Diagnóstico de Duplicidad en Modelos Incrementales):
Un ingeniero de datos observa que tras un reintento manual de un pipeline de dbt, la tabla de hechos `fct_pedidos` presenta registros duplicados para el mismo `pedido_id`. Al inspeccionar el bloque de configuración del modelo, encuentra:

```SQL
{{ config(
    materialized='incremental',
    incremental_strategy='append'
) }}
```

1. ¿Por qué la estrategia `append` rompe la propiedad de idempotencia?
2. ¿Cómo debe modificarse la configuración para corregir el incidente de forma permanente?

Solución 24.1:

1. *Causa:* La estrategia `append` simplemente ejecuta sentencias `INSERT INTO` acumulativas con los nuevos registros encontrados. Si el job falla a medio camino o se reejecuta para el mismo periodo temporal, la estrategia vuelve a insertar las filas sin verificar si ya existían previamente en la tabla de destino, duplicando los datos.

2. *Corrección:* Se debe migrar a la estrategia `merge` definiendo la clave primaria unívoca en `unique_key`:

```sql
{{ config(
    materialized='incremental',
    unique_key='pedido_id',
    incremental_strategy='merge'
) }}
```

Con esta configuración, dbt aplicará una sentencia transaccional `MERGE INTO`, asegurando que registros reinsertados actualicen la fila existente en lugar de duplicarla.

Ejercicio 24.2 (Consulta de Auditoría sobre dbt Snapshots):
A partir de la tabla de snapshots generada por `snp_clientes`, escribe una consulta SQL analítica que permita conocer cuál era la dirección residencial exacta que tenía el cliente `'C-842'` el día **15 de marzo de 2025 a las 10:00:00 UTC**.

Solución 24.2:

```sql
SELECT
    cliente_id,
    nombre_completo,
    direccion,
    dbt_valid_from,
    dbt_valid_to
FROM {{ ref('snp_clientes') }}
WHERE cliente_id = 'C-842'
  AND dbt_valid_from <= TIMESTAMP '2025-03-15 10:00:00'
  AND (dbt_valid_to > TIMESTAMP '2025-03-15 10:00:00' OR dbt_valid_to IS NULL);
```

Fundamento: Esta consulta filtra la versión del registro cuyo intervalo de validez `[dbt_valid_from, dbt_valid_to)` abarcaba el instante temporal exacto solicitado, reconstruyendo el estado del negocio en el pasado sin importar cuántas mutaciones haya sufrido posteriormente.

24.7 Resumen y Puntos Clave

- **Transformación modular con ELT:** dbt traslada la lógica de modelado al interior del almacén de datos, desacoplando la ingestión del cómputo analítico mediante código SQL declarativo versionado en Git.

- **Linaje y orden topológico con `ref()`:** La interpolación dinámica de modelos permite a dbt calcular automáticamente el DAG de compilación y validar dependencias antes de la ejecución.
- **Idempotencia mediante `merge`:** La combinación de materializaciones incrementales con `unique_key` y la macro `is_incremental()` asegura que las ejecuciones reiteradas converjan al mismo estado sin duplicación.
- **Auditoría histórica con dbt Snapshots:** Implementa Dimensiones Lentamente Cambiantes (*SCD Tipo 2*) de forma declarativa mediante estrategias `check` o `timestamp`, manteniendo campos de vigencia `dbt_valid_from` y `dbt_valid_to`.

Lecturas Recomendadas

1. **dbt Labs Technical Staff (2024).** *The dbt Viewpoint: What is dbt and Why ELT Matters.* dbt Documentation (`https://docs.getdbt.com/`).
2. **Freeman, M. (2024).** *Modern Data Modeling with dbt: Building Scalable Analytics Engineering Pipelines.* O'Reilly Media.
3. **Stonebraker, M. & Cetintemel, U. (2005).** *"One Size Fits All": An Idea Whose Time Has Come and Gone.* Proceedings of the 21st International Conference on Data Engineering (ICDE).

CAPÍTULO 25

Infraestructura como Código (IaC) con Terraform y OpenTofu RECOMENDADO

> *"En la ingeniería de datos moderna, la infraestructura no se configura mediante interfaces gráficas (ClickOps); se escribe, se versiona en Git y se audita con el mismo rigor que el código de producción. La Infraestructura como Código (IaC) convierte el aprovisionamiento de clústeres, almacenes de objetos y políticas de seguridad en un proceso determinista, auditable y libre de error humano."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la filosofía declarativa y el ciclo de vida de la Infraestructura como Código (**IaC**) con Terraform y OpenTofu.
- Gestionar con seguridad el archivo de estado (`terraform.tfstate`), implementando bloqueo concurrente (*State Locking*) en almacenamiento de objetos.
- Aprovisionar una arquitectura Lakehouse completa en **Azure** con `azurerm` v4 y autenticación Entra ID (`storage_use_azuread = true`), así como sus equivalentes en **AWS** (S3, IAM, KMS).
- Configurar recursos de gobernanza de Unity Catalog y orquestación de tareas mediante el proveedor oficial `databricks`.
- Proteger recursos analíticos críticos contra destrucción accidental utilizando reglas de ciclo de vida (`prevent_destroy`, `ignore_changes`).
- Refactorizar y adoptar infraestructura preexistente mediante bloques declarativos `import` y `moved`.
- Estructurar repositorios de infraestructura desacoplando módulos reutilizables y aislando ambientes de desarrollo y producción.

Definición 25.1 (Infraestructura como Código e Idempotencia)

La **Infraestructura como Código (IaC)** es la práctica de ingeniería de software que modela el aprovisionamiento, configuración y ciclo de vida de redes, cómputo y almacenamiento mediante archivos de configuración legibles por humanos y procesables por máquinas.

Su propiedad fundamental es la **idempotencia**: ejecutar la misma definición de IaC repetidas veces sobre una plataforma en la nube debe converger exactamente al mismo estado de infraestructura, sin crear duplicados ni generar efectos secundarios imprevistos.

25.1 Ciclo de Vida: Plan, Apply y State Locking

Terraform y OpenTofu emplean el lenguaje declarativo **HCL (HashiCorp Configuration Language)**. El flujo operativo se articula en tres comandos deterministas:

- `terraform init`: Inicializa el directorio de trabajo, descarga los binarios de los proveedores requeridos (`azurerm`, `aws`, `databricks`) y configura la conexión con el backend remoto.
- `terraform plan`: Realiza una lectura de solo lectura contra las APIs del proveedor en la nube, compara los recursos reales contra los manifiestos locales y proyecta el plan de ejecución exacto (recursos a crear `+`, modificar `~` o destruir `-`).
- `terraform apply`: Ejecuta las llamadas a las APIs de la nube respetando el orden topológico de dependencias y actualiza el archivo de estado persistente.

Gestión Segura del Archivo de Estado y Bloqueo Concurrente

El archivo `terraform.tfstate` es el mapa de verdad de la infraestructura. Si se corrompe o dos ingenieros aplican cambios simultáneamente, los recursos en la nube pueden colisionar o ser destruidos accidentalmente.

- **Secretos en Texto Plano:** Contraseñas de bases de datos, claves de acceso generadas y certificados SSL residen sin cifrar dentro del JSON de `terraform.tfstate`. **Está terminantemente prohibido incluir el archivo de estado en repositorios de Git**.
- **Bloqueos de Estado (*State Locking*):**
  - **En Azure:** Se utiliza un contenedor de Azure Blob Storage que aprovecha el mecanismo de *Blob Leases* nativo para bloquear el archivo durante la ejecución de un plan o apply.
  - **En AWS S3 (Terraform 1.10+ / OpenTofu):** Históricamente se requería una tabla DynamoDB dedicada para gestionar bloqueos. En versiones modernas, S3 soporta bloqueo nativo directo declarando `use_lockfile = true` en el bloque de backend.

25.2 Despliegue de un Lakehouse en Azure con `azurerm` v4 y Unity Catalog

El siguiente manifiesto productivo despliega una cuenta de almacenamiento ADLS Gen2 con espacio de nombres jerárquico habilitado, los tres contenedores de la Arquitectura Medallón, un espacio de trabajo administrado de **Azure Databricks**, y aprovisiona el catálogo gobernado en **Unity Catalog** utilizando el proveedor `databricks`:

HCL (TERRAFORM)

```
# main.tf
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0" # Operador pesimista que permite versiones 4.x
compatibles
    }
    databricks = {
      source  = "databricks/databricks"
      version = "~> 1.50"
    }
  }

  backend "azurerm" {
    resource_group_name  = "rg-terraform-mgmt-prod"
    storage_account_name = "sttfstatelakehouseprod"
    container_name       = "tfstate-core"
    key                  = "lakehouse.prod.tfstate"
  }
}

provider "azurerm" {
  features {}
  subscription_id     = var.subscription_id
  # Requisito crítico en azurerm v4: permite gestionar sistemas de archivos
mediante RBAC de Entra ID
  # evitando errores HTTP 403 cuando shared_access_key_enabled = false
  storage_use_azuread = true
}

# 1. Grupo de recursos principal para la plataforma analítica
resource "azurerm_resource_group" "rg_datos" {
  name     = "rg-lakehouse-prod-eastus2"
  location = "eastus2"
  tags = {
    Ambiente   = "Produccion"
    Disciplina = "DataEngineering"
    FinOps     = "CentroCostos-Analitica"
  }
}
```

```
# 2. Cuenta de almacenamiento para el Lakehouse con ADLS Gen2 (HNS activo)
resource "azurerm_storage_account" "lakehouse_storage" {
  name                     = "stlakehouseprodeus201"
  resource_group_name      = azurerm_resource_group.rg_datos.name
  location                 = azurerm_resource_group.rg_datos.location
  account_tier             = "Standard"
  account_replication_type = "ZRS" # Almacenamiento redundante de zona para alta
disponibilidad
  is_hns_enabled           = true  # Hierarchical Namespace obligatorio para
Lakehouse

  min_tls_version           = "TLS1_2"
  shared_access_key_enabled = false # Obliga a autenticación por Entra ID / RBAC

  blob_properties {
    delete_retention_policy {
      days = 14
    }
  }

  lifecycle {
    # Protección de datos críticos contra borrados accidentales
    prevent_destroy = true
    ignore_changes  = [tags]
  }

  tags = azurerm_resource_group.rg_datos.tags
}

# 3. Contenedores de almacenamiento para las capas Medallón
resource "azurerm_storage_data_lake_gen2_filesystem" "capas_medallon" {
  for_each           = toset(["bronze", "silver", "gold"])
  name               = each.key
  storage_account_id = azurerm_storage_account.lakehouse_storage.id
}

# 4. Workspace administrado de Azure Databricks (SKU Premium para Unity Catalog)
resource "azurerm_databricks_workspace" "workspace_databricks" {
  name                        = "dbw-lakehouse-prod-eastus2"
  resource_group_name         = azurerm_resource_group.rg_datos.name
  location                    = azurerm_resource_group.rg_datos.location
  sku                         = "premium"
  managed_resource_group_name = "rg-lakehouse-prod-managed-infra"

  tags = azurerm_resource_group.rg_datos.tags
}

# 5. Proveedor de Databricks vinculado dinámicamente al Workspace aprovisionado
provider "databricks" {
  host = azurerm_databricks_workspace.workspace_databricks.workspace_url
}

# 6. Credencial de almacenamiento y Ubicación Externa para Unity Catalog
# Unity Catalog requiere gobernar el acceso mediante Azure Managed Identity
resource "databricks_storage_credential" "external_creds" {
  name = "credencial_adls_prod"
  azure_managed_identity {
```

```
    access_connector_id =
azurerm_databricks_access_connector.ext_access_connector.id
  }
  comment = "Credencial administrada de Azure para acceso seguro al Lakehouse"
}

resource "databricks_external_location" "gold_location" {
  name            = "ubicacion_externa_gold"
  url             =
"abfss://${azurerm_storage_data_lake_gen2_filesystem.capas_medallon["gold"].name}
@${azurerm_storage_account.adls_lakehouse.name}.dfs.core.windows.net/"
  credential_name = databricks_storage_credential.external_creds.id
  comment         = "Punto de montaje gobernado por Unity Catalog para capa Gold"
}

# 7. Catálogo gobernado en Unity Catalog con URL abfss:// válida
resource "databricks_catalog" "corp_catalog" {
  name         = "catalogo_corporativo_prod"
  comment      = "Catálogo principal gobernado con Unity Catalog"
  storage_root = databricks_external_location.gold_location.url
  depends_on   = [databricks_external_location.gold_location]

  lifecycle {
    prevent_destroy = true
  }
}
```

25.3 Equivalentes en AWS: S3, KMS e IAM con Mínimo Privilegio

En entornos sobre Amazon Web Services (AWS), la persistencia del Lakehouse y la seguridad en reposo se modelan utilizando `aws_s3_bucket`, claves administradas por el cliente en `aws_kms_key` y roles de mínimo privilegio:

HCL (TERRAFORM)

```
# lakehouse_aws.tf
resource "aws_kms_key" "lakehouse_kms" {
  description             = "Clave KMS para cifrado del Lakehouse Medallón"
  deletion_window_in_days = 30
  enable_key_rotation     = true
}

resource "aws_s3_bucket" "lakehouse_gold" {
  bucket = "empresa-lakehouse-gold-prod"

  lifecycle {
    prevent_destroy = true
  }
}
```

```
resource "aws_s3_bucket_server_side_encryption_configuration" "gold_encryption" {
  bucket = aws_s3_bucket.lakehouse_gold.id

  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.lakehouse_kms.arn
      sse_algorithm     = "aws:kms"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "gold_block_public" {
  bucket = aws_s3_bucket.lakehouse_gold.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

25.4 Refactorización y Adopción: Bloques `import` y `moved`

En organizaciones en crecimiento es habitual tener recursos analíticos creados previamente por consola (*ClickOps*) o requerir reorganizar módulos sin provocar la destrucción de las bases de datos.

1. Bloque `import` Declarativo (Terraform 1.5+)

A diferencia del comando interactivo tradicional (`terraform import`), el bloque declarativo `import` permite auditar la adopción en el código y verificarla en el plan:

HCL (TERRAFORM)

```
import {
  to = azurerm_storage_account.lakehouse_storage
  id = "/subscriptions/00000000-0000-0000-0000-000000000000/resourceGroups/rg-
datos/providers/Microsoft.Storage/storageAccounts/stlakehouseprodeus201"
}
```

2. Bloque `moved` para Refactorización sin Destrucción

Si traslada un recurso desde la raíz a un submódulo o renombra su identificador, Terraform interpretaría por omisión que debe destruir el recurso antiguo y crear uno nuevo. El bloque `moved` actualiza el puntero de estado internamente:

```
HCL (TERRAFORM)
```

```hcl
# Actualiza el estado sin tocar la infraestructura física en la nube
moved {
  from = azurerm_storage_account.st_legacy
  to   = azurerm_storage_account.lakehouse_storage
}
```

25.5 Estructuración de Proyectos y Segregación de Entornos

La arquitectura empresarial exige **segregación física de directorios y backends de estado independientes**:

```
infraestructura-datos/
├── modulos/
│   ├── lakehouse_storage/      # Módulo reutilizable de almacenamiento ADLS Gen2
/ S3
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── databricks_platform/    # Módulo reutilizable de Unity Catalog y Jobs
│       ├── main.tf
│       └── variables.tf
└── entornos/
    ├── dev/
    │   ├── main.tf             # Consume módulos fijando variables de dev
    │   ├── backend.tf          # Estado en: dev.lakehouse.tfstate
    │   └── terraform.tfvars
    └── prod/
        ├── main.tf             # Consume módulos fijando alta redundancia y
Premium
        ├── backend.tf          # Estado en: prod.lakehouse.tfstate (permisos
restringidos)
        └── terraform.tfvars
```

25.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Permitir ClickOps en Paralelo con Terraform
Si un administrador modifica las políticas de red directamente en el portal de Azure o AWS sin actualizar el código HCL, la siguiente ejecución de `terraform apply` revertirá silenciosamente esas modificaciones manuales o fallará debido a divergencias de estado (*Configuration Drift*). La consola web debe ser de solo lectura en producción.

Anti-patrón 2: Declarar Credenciales Secretas en Archivos `.tfvars` Versionados
Incrustar tokens de acceso o contraseñas en archivos de variables y subirlos a repositorios de Git expone credenciales a toda la organización. Utilice variables de entorno inyectadas en el runner de

CI/CD (`TF_VAR_db_password`) o proveedores dedicados de secretos (Azure Key Vault, AWS Secrets Manager o HashiCorp Vault).

Anti-patrón 3: Operador de Versiones Inadecuado en Proveedores
Declarar `version = "~> 4.0.0"` restringe rígidamente la descarga a parches de la versión 4.0.x, impidiendo acceder a mejoras y correcciones de la serie 4.1+. Por el contrario, declarar `version = ">= 4.0"` arriesga la descarga de una futura versión mayor 5.0 con cambios incompatibles. El estándar óptimo es `version = "~> 4.0"`.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 25.1 (Diagnóstico de Desviación de Configuración):
Un ingeniero de datos modifica manualmente el firewall de la cuenta de almacenamiento en el portal de Azure para autorizar una IP externa. Tres días después, otro miembro del equipo ejecuta un despliegue con `terraform apply` para agregar un nuevo contenedor.

1. ¿Qué detectará `terraform plan` al inspeccionar la cuenta de almacenamiento?
2. ¿Qué ocurrirá con la regla de firewall creada manualmente tras la ejecución de `terraform apply`?

Solución 25.1:

1. *Detección:* Durante la fase de actualización (*refresh*), Terraform consulta el estado real en la nube y detecta una discrepancia entre el archivo local de HCL (que no declara la regla de IP) y el recurso real. El plan marcará la regla manual con un símbolo de supresión (`-`) como un cambio no planificado derivado de *Configuration Drift*.
2. *Consecuencia:* Al confirmar `terraform apply`, Terraform restaurará el estado declarado en el código y **eliminará la regla de IP manual**, revirtiendo la cuenta de almacenamiento a su configuración original y bloqueando la IP que había sido autorizada por fuera del código.

Ejercicio 25.2 (Diseño de Almacenamiento Jerárquico en ADLS Gen2):
¿Por qué es indispensable configurar `is_hns_enabled = true` en cuentas de almacenamiento de Azure destinadas a actuar como Lakehouse para Spark y Delta Lake? ¿Qué impacto de rendimiento y atomicidad tiene esta propiedad?

Solución 25.2:

1. *Estructura Jerárquica Real:* En los sistemas de almacenamiento de objetos tradicionales (como S3 o Blob Storage estándar), los directorios no existen físicamente; son prefijos virtuales en el nombre de la clave del objeto (`/capa/tabla/particion/archivo.parquet`).

2. *Rendimiento y Atomicidad:*
    - Sin HNS, una operación como renombrar un directorio o mover una tabla requiere copiar individualmente cada uno de los archivos que lo componen y luego borrar los originales (operación de orden $O(N)$ lenta y no atómica).
    - Con **Hierarchical Namespace (HNS activo)**, ADLS Gen2 implementa directorios reales compatibles con POSIX: un renombrado de directorio es un cambio atómico de puntero de metadatos de orden $O(1)$ que toma milisegundos, independientemente de si el directorio contiene 10 archivos o 500.000 archivos Parquet, evitando bloqueos e inconsistencias durante transacciones de Spark.

25.7 Resumen y Puntos Clave

- **Declaratividad e Idempotencia:** Terraform y OpenTofu permiten definir plataformas analíticas reproducibles en código HCL, garantizando que el estado real converja deterministamente con el estado deseado.
- **Seguridad del Estado:** El archivo `terraform.tfstate` debe residir en un backend remoto cifrado con control de acceso estricto y bloqueos de concurrencia nativos, jamás en un repositorio Git.
- **Integración Azure y Unity Catalog:** El proveedor `azurerm` v4 requiere `storage_use_azuread = true` para crear sistemas de archivos con claves de acceso deshabilitadas. El proveedor `databricks` complementa la infraestructura desplegando catálogos y esquemas en Unity Catalog.
- **Ciclo de vida y adopción:** `lifecycle { prevent_destroy = true }` protege datos contra eliminaciones accidentales, mientras que los bloques `import` y `moved` permiten incorporar infraestructura existente y refactorizar sin tiempo de inactividad.

Lecturas Recomendadas

1. **Brikman, Y. (2022).** *Terraform: Up & Running: Writing Infrastructure as Code* (3rd ed.). O'Reilly Media.
2. **OpenTofu Project Technical Steering Committee (2024).** *OpenTofu Documentation and State Management Specification.* Linux Foundation (`https://opentofu.org/docs/`).
3. **Databricks Engineering (2024).** *Databricks Terraform Provider: Managing Unity Catalog and Lakehouse Infrastructure.* HashiCorp Registry & Databricks Documentation (`https://registry.terraform.io/providers/databricks/databricks/latest/docs`).

CAPÍTULO 26

Integración y Despliegue Continuo (CI/CD) para Pipelines de Datos RECOMENDADO

> *"Un pipeline analítico es software de misión crítica gobernando las decisiones estratégicas de una organización. Si el código de transformación de datos no cuenta con pruebas unitarias deterministas, análisis estático automatizado y un flujo de entrega continua, cada paso a producción se convierte en un riesgo inaceptable de corrupción silenciosa de datos."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender los principios de **DataOps** y la pirámide de pruebas aplicada a pipelines analíticos distribuidos.
- Implementar pruebas unitarias sobre DataFrames de PySpark utilizando `pytest` y la librería `chispa` para validaciones independientes del orden físico de filas.
- Diseñar un flujo de trabajo completo de **Integración y Despliegue Continuo (CI/CD)** en GitHub Actions con análisis léxico (`ruff`), validación de SQL (`sqlfluff`) y empaquetado automático.
- Asegurar la **portabilidad absoluta** de los artefactos de datos desacoplando la lógica de negocio de las rutas físicas de almacenamiento mediante contenedores Docker y configuraciones parametrizadas.
- Automatizar la promoción segura de código entre entornos (`dev` → `staging` → `prod`) sin intervención manual.

Definición 26.1 (DataOps y CI/CD en Plataformas de Datos)

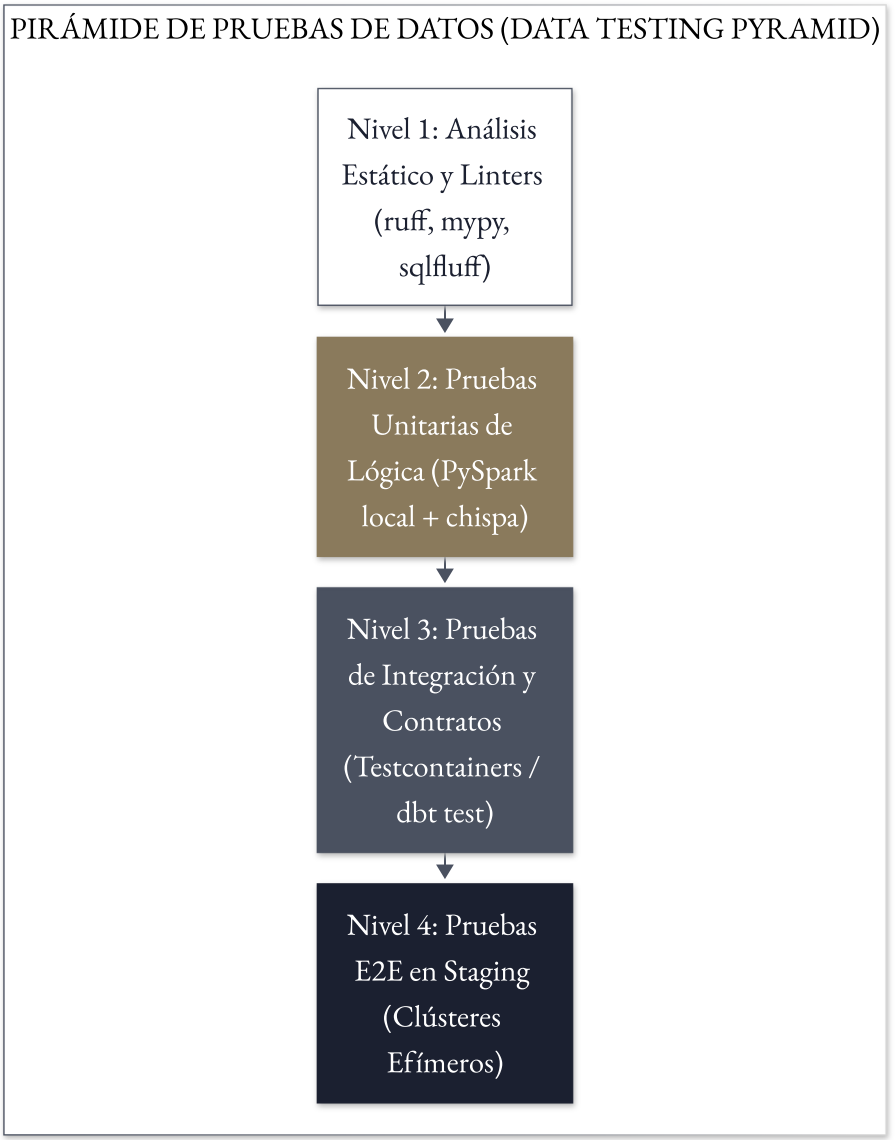
DataOps es una disciplina metodológica y cultural que traslada las prácticas consolidadas de la ingeniería de software y DevOps al ciclo de vida de los datos.

La **Integración Continua (CI)** compila, valida estáticamente y somete a prueba cada cambio de código en ramas de desarrollo de forma automática y aislada. El **Despliegue Continuo (CD)** empaqueta los artefactos validados (paquetes Wheel de Python, plantillas de dbt compiladas, imágenes Docker, DAGs de Airflow) y los promueve deterministamente hacia los entornos de cómputo en la nube, garantizando reproducibilidad y reversibilidad inmediata ante fallos (*Rollback*).

26.1 La Pirámide de Pruebas en Ingeniería de Datos

A diferencia de las aplicaciones web tradicionales, los pipelines de datos enfrentan dos fuentes de fallo independientes: **errores en el código** (lógica algorítmica defectuosa) y **errores en los datos** (anomalías de esquema, nulos imprevistos, distribuciones estadísticas corruptas).

La pirámide de pruebas de datos estructura la validación en cuatro niveles progresivos:



1. **Nivel 1: Análisis Estático y Linters (Milisegundos):** Valida estilo de código, adherencia a PEP 8, tipado estático con `mypy`, sintaxis de SQL mediante `sqlfluff` y prevención de vulnerabilidades de seguridad.
2. **Nivel 2: Pruebas Unitarias de Transformación (Segundos):** Aísla funciones individuales de transformación pura en PySpark utilizando sesiones locales ligeras (`local[2]`) con datasets sintetizados en memoria.
3. **Nivel 3: Pruebas de Integración y Contratos (Minutos):** Valida la interacción entre módulos, cruces con bases de datos simuladas mediante `Testcontainers` (ej. PostgreSQL temporal en Docker) y pruebas de contratos de esquemas.

4. **Nivel 4: Pruebas de Aceptación End-to-End (E2E):** Ejecuta el pipeline completo sobre un clúster efímero en el entorno de pruebas (*Staging*) utilizando una muestra representativa y enmascarada de datos reales.

26.2 Pruebas Unitarias en PySpark con la Librería `chispa`

Probar DataFrames distribuidos en pruebas unitarias convencionales suele fallar si se compara con simples cadenas de texto o bucles artesanales, ya que los motores distribuidos no garantizan el orden físico de las particiones ni de las filas.

La librería especializada `chispa` proporciona comparaciones de igualdad profunda entre DataFrames, soportando diferencias de orden e inspeccionando esquemas con mensajes de error legibles:

PYTHON

```python
# tests/test_transformaciones_clientes.py
import pytest
from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from chispa.dataframe_comparer import assert_df_equality

# Fixture de sesión Spark local para toda la suite de pruebas
@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder \
        .master("local[2]") \
        .appName("PruebasUnitariasPySpark") \
        .config("spark.sql.shuffle.partitions", "2") \
        .config("spark.ui.enabled", "false") \
        .getOrCreate()

def estandarizar_clientes(df_entrada):
    """Función pura de transformación de negocio que se somete a prueba."""
    return df_entrada \
        .withColumn("nombre_limpio", F.upper(F.trim(F.col("nombre_crudo")))) \
        .withColumn("es_activo", F.when(F.col("estado") == "A",
F.lit(True)).otherwise(F.lit(False))) \
        .select("cliente_id", "nombre_limpio", "es_activo")

def test_estandarizar_clientes_transforma_correctamente(spark):
    # 1. Dataset de entrada sintético en memoria
    datos_origen = [
        ("C-001", "   juan perez  ", "A"),
        ("C-002", "MARIA GOMEZ", "I"),
        ("C-003", " carlos silva ", "A")
    ]
    df_entrada = spark.createDataFrame(datos_origen, ["cliente_id",
"nombre_crudo", "estado"])

    # 2. Ejecución de la transformación
```

```
    df_resultado = estandarizar_clientes(df_entrada)

    # 3. Dataset esperado de referencia
    datos_esperados = [
        ("C-001", "JUAN PEREZ", True),
        ("C-002", "MARIA GOMEZ", False),
        ("C-003", "CARLOS SILVA", True)
    ]
    df_esperado = spark.createDataFrame(datos_esperados, ["cliente_id",
"nombre_limpio", "es_activo"])

    # 4. Afirmación estricta ignorando el orden físico de las filas
    assert_df_equality(df_resultado, df_esperado, ignore_row_order=True)
```

26.3 Pipeline Completo de CI/CD en GitHub Actions

El siguiente manifiesto en YAML define el flujo de trabajo moderno en GitHub Actions, empleando las acciones oficiales `v4` y `v5`, caché acelerado con `uv` y empaquetado de artefactos:

YAML

```yaml
# .github/workflows/data_pipeline_ci_cd.yml
name: CI/CD Data Engineering Platform

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint-and-quality:
    name: 1. Análisis Estático y Linting
    runs-on: ubuntu-latest
    steps:
      - name: Descargar Código Fuente
        uses: actions/checkout@v4

      - name: Configurar Entorno Python con uv (Ultra-rápido)
        uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
          python-version: "3.11"

      - name: Instalar Herramientas de Calidad con uv
        run: uv pip install ruff sqlfluff mypy

      - name: Ejecutar Linter de Python (Ruff)
        run: uv run ruff check .
```

```
      - name: Validación de Tipado Estático (MyPy)
        run: uv run mypy pipelines/ --ignore-missing-imports

      - name: Linting de Modelos SQL (SQLFluff)
        run: uv run sqlfluff lint models/ --dialect databricks

  unit-tests:
    name: 2. Pruebas Unitarias PySpark con Cobertura
    needs: lint-and-quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Configurar Java 17 (Requerido por Spark)
        uses: actions/setup-java@v4
        with:
          distribution: "temurin"
          java-version: "17"

      - name: Configurar Python con uv
        uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
          python-version: "3.11"

      - name: Instalar Dependencias de Prueba y Cobertura
        run: uv pip install pytest pytest-cov pyspark chispa

      - name: Ejecutar Suite de PyTest con Reporte de Cobertura Estricto
        run: uv run pytest tests/ -v --cov=pipelines --cov-report=term-missing --
junitxml=junit/test-results.xml

  build-and-package:
    name: 3. Empaquetado y Publicación de Artefactos
    needs: unit-tests
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Configurar Python con uv
        uses: astral-sh/setup-uv@v5
        with:
          python-version: "3.11"

      - name: Construir Paquete Wheel de Python
        run: uv build --wheel

      - name: Publicar Artefacto Wheel en GitHub Releases / Storage
        uses: actions/upload-artifact@v4
        with:
          name: data-pipelines-wheel
          path: dist/*.whl

  deploy-production:
    name: 4. Despliegue Continuo con Databricks Asset Bundles (DABs)
```

```
    needs: build-and-package
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Configurar Databricks CLI Oficial (Go)
        uses: databricks/setup-cli@v0.229.0

      - name: Validar y Desplegar Databricks Asset Bundle (DAB)
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
        run: |
          databricks bundle validate --target prod
          databricks bundle deploy --target prod
          echo "[CD] Despliegue de Databricks Asset Bundle completado
exitosamente en producción."
```

26.4 Portabilidad Absoluta y Contenerización

Un pipeline de datos es **completamente portátil** si puede ejecutarse de manera idéntica en el equipo portátil del desarrollador, en un contenedor de integración continua en GitHub Actions, en un clúster de Amazon EMR o en Databricks sobre Azure, sin cambiar una sola línea de código fuente.

Principios para Garantizar Portabilidad

1. **Desacoplamiento de Rutas de Almacenamiento:** Está estrictamente prohibido utilizar rutas hardcodeadas como `s3://bucket-prod/ventas.parquet`. Las rutas deben construirse dinámicamente inyectando prefijos mediante variables de entorno (`DATA_LAKE_BASE_PATH`) o archivos de configuración jerárquicos:

```
PYTHON
```

```python
import os

def obtener_ruta_tabla(capa, nombre_tabla):
    base_uri = os.getenv("DATA_LAKE_BASE_PATH", "file:///tmp/lakehouse")
    return f"{base_uri}/{capa}/{nombre_tabla}"

# En local:    file:///tmp/lakehouse/silver/clientes
# En Prod AWS: s3://prod-lakehouse-corp/silver/clientes
# En Prod Az:  abfss://silver@stlakehouse.dfs.core.windows.net/clientes
```

2. **Empaquetado en Imágenes Docker:** Para pipelines basados en Kubernetes (Spark on K8s), todas las dependencias de Python y binarios del sistema se congelan en una imagen Docker

inmutable, eliminando discrepancias del tipo *"en mi máquina funcionaba"*:

DOCKERFILE

```
# Dockerfile para Spark en Kubernetes y Databricks Container Services (Databricks
Runtime 15.4+ LTS)
FROM apache/spark:3.5.2-python3

USER root
WORKDIR /opt/spark/work-dir

# Instalar dependencias congeladas con uv para builds reproducibles
COPY requirements.txt .
RUN pip install --no-cache-dir uv && \
    uv pip install --system --no-cache -r requirements.txt

# Copiar el código del paquete de la aplicación
COPY pipelines/ /opt/spark/work-dir/pipelines/

USER spark
ENTRYPOINT ["/opt/spark/bin/spark-submit", "--master", "k8s://...",
"pipelines/main.py"]
```

26.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Pruebas en Producción con Datos Reales sin Aislamiento de Entornos Permitir que los ingenieros prueben código nuevo directamente sobre las tablas de producción arriesga la integridad de los datos. Toda modificación debe promoverse a través de ramas de Git (`feature` → `pull request` → `main`) y validarse en un entorno de staging con datos anonimizados.

Anti-patrón 2: Pruebas Unitarias Dependientes de Recursos Externos Activos Diseñar pruebas unitarias que intenten conectarse a un clúster de Kafka en vivo o a un bucket real de S3 en la nube causa que las pruebas fallen por problemas transitorios de red o credenciales expiradas. Las pruebas unitarias deben ser deterministas y ejecutarse 100% desconectadas (*offline*).

Anti-patrón 3: Despliegue Manual de Cambios Mediante Copiado de Archivos por SSH Subir archivos modificados a mano al servidor de producción eludiendo el pipeline de CI/CD rompe la trazabilidad y garantiza que el repositorio de Git no refleje el estado real desplegado.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 26.1 (Determinismo en Pruebas Unitarias de Spark):
Un ingeniero diseña una prueba unitaria para una transformación de enriquecimiento y escribe la siguiente validación en `pytest`:

```python
def test_enriquecimiento(spark):
    df_salida = enriquecer_datos(df_entrada)
    assert df_salida.collect() == df_esperado.collect()
```

¿Por qué esta afirmación fallará intermitentemente (*Flaky Test*) en un clúster o máquina multinúcleo a pesar de que los datos sean matemáticamente correctos? ¿Cómo debe solucionarse?

Solución 26.1:

1. *Causa:* La función `.collect()` de Spark descarga todas las filas en una lista nativa de Python respetando el orden físico en que fueron generadas por las particiones. Debido a que las transformaciones distribuidas no garantizan el orden de ejecución entre hilos paralelos, el orden relativo de las filas puede variar entre corridas consecutivas. Comparar dos listas con `==` en Python exige coincidencia estricta de orden posicional, provocando fallos aleatorios.
2. *Solución:* Emplear `assert_df_equality` de la librería `chispa` configurando `ignore_row_order=True`:

```python
from chispa.dataframe_comparer import assert_df_equality

def test_enriquecimiento(spark):
    df_salida = enriquecer_datos(df_entrada)
    assert_df_equality(df_salida, df_esperado, ignore_row_order=True)
```

Esta función compara los esquemas de ambas tablas y evalúa la equivalencia multiconjunto de los registros sin importar su posición física en el arreglo.

Ejercicio 26.2 (Diseño de Estrategia de Rollback Inmediato):
Un nuevo despliegue de código promovido a producción introduce una regresión no detectada en una fórmula de cálculo tributario en la capa Silver de Delta Lake. Describe la estrategia de recuperación en dos pasos que debe ejecutarse para restaurar la operación:

1. Nivel de Código (Software).
2. Nivel de Datos (Storage).

Solución 26.2:

1. *Nivel de Código (Software):* En el repositorio Git, el equipo revierte el commit causante (`git revert HEAD`) y dispara el pipeline de CI/CD para compilar y desplegar nuevamente la versión anterior del paquete Wheel a producción.
2. *Nivel de Datos (Storage):* Como la tabla Silver utiliza Delta Lake, los datos erróneos no requieren una reconstrucción manual lenta. El ingeniero ejecuta un comando de **Time Travel**

Restore en SQL:

```sql
RESTORE TABLE prod_lakehouse.silver.facturacion TO VERSION AS OF 142;
```

Esto restaura de inmediato el estado de la tabla a la versión 142 (el commit inmediatamente anterior al despliegue defectuoso) en una única transacción atómica de milisegundos.

26.6 Resumen y Puntos Clave

- **DataOps y rigor de ingeniería:** Traslada las mejores prácticas de software (CI/CD, pruebas automatizadas y despliegues deterministas) a los activos de datos.
- **Pirámide de pruebas:** Desde el análisis estático rápido con linters (`ruff`, `sqlfluff`) hasta pruebas unitarias locales en PySpark con `chispa` y pruebas de integración.
- **Automatización con GitHub Actions:** Orquesta la validación léxica, la ejecución de suites de pruebas con Java y Python, el empaquetado en archivos Wheel y el despliegue automático hacia entornos seguros.
- **Portabilidad mediante desacoplamiento:** Las rutas de almacenamiento y configuraciones de entorno deben inyectarse mediante variables externas y empaquetarse en contenedores Docker para garantizar reproducibilidad total.

Lecturas Recomendadas

1. **Forsgren, N., Humble, J., & Kim, G. (2018).** *Accelerate: The Science of Lean Software and DevOps (Building and Scaling High Performing Technology Organizations).* IT Revolution Press.
2. **Ebert, C., Gallardo, G., Hernantes, J., & Serrano, N. (2016).** *DevOps: Principles, Practices, and Challenges.* IEEE Software, 33(3), 94-100.
3. **Powers, M. (2023).** *Testing PySpark Applications with Chispa: Fast, Expressive Unit Testing for Distributed DataFrames.* GitHub Documentation (`https://github.com/MrPowers/chispa`).

CAPÍTULO 26A

Pruebas de Software en Datos: Pytest, Chispa y Testcontainers RECOMENDADO

> *"Durante años, la ingeniería de datos toleró una práctica que avergonzaría a cualquier equipo serio de desarrollo de software: probar los pipelines directamente sobre los datos de producción. Si una transformación no cuenta con pruebas unitarias deterministas que se ejecuten en segundos en el entorno de desarrollo local y en el pipeline de CI/CD, cada despliegue no es una actualización de ingeniería, sino una apuesta a ciegas con el patrimonio de la empresa."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Diseñar una estrategia integral de aseguramiento de calidad aplicando la **Pirámide de Pruebas de Ingeniería de Datos**.
- Escribir pruebas unitarias rápidas y deterministas para transformaciones de PySpark utilizando `pytest` y `chispa`.
- Resolver los desafíos específicos de comparación de DataFrames distribuidos: orden no determinista de filas, tolerancia numérica en flotantes y comparación de esquemas.
- Configurar **fixtures de PySpark** optimizados en memoria para ejecuciones ultrarrápidas en entornos de CI/CD.
- Implementar pruebas de integración aisladas y reproducibles utilizando **Testcontainers** para levantar bases de datos PostgreSQL y buckets S3 efímeros.
- Detectar regresiones de esquema y romper el build antes de desplegar código defectuoso a producción.

Definición 26A.1 (La Pirámide de Pruebas para Pipelines de Datos) A diferencia del desarrollo de software convencional donde solo se prueba la lógica del código, en la ingeniería de datos se debe probar **el código de transformación** y **la validez del dato**:

1. **Pruebas Unitarias (*Unit Tests - Base de la Pirámide*):** Validan funciones de transformación puras utilizando DataFrames sintéticos mínimos en memoria. Se ejecutan en milisegundos sin depender de infraestructura externa.
2. **Pruebas de Integración (*Integration Tests - Centro*):** Validan la interacción física con almacenes de datos reales (lecturas JDBC, escrituras Delta Lake) utilizando contenedores efímeros aislados (*Testcontainers*).
3. **Pruebas de Calidad y Contrato (*Data Quality Tests - Cúspide*):** Validan que los datos reales en producción cumplan las expectativas estadísticas y reglas de negocio declaradas (Great Expectations, dbt tests).

26A.1 El Desafío de Probar Transformaciones Distribuidas

Probar código de datos en PySpark o SQL presenta obstáculos particulares que no existen en el software tradicional:

- **No determinismo en el orden de las filas:** Debido a la naturaleza distribuida y el particionamiento de Spark, el resultado de una transformación puede devolver las mismas tuplas en diferente orden físico entre ejecuciones consecutivas. Comparar DataFrames con `df_esperado.collect() == df_real.collect()` produce fallos falsos recurrentes.
- **Sobrecosto de arranque de la sesión Spark:** Inicializar una sesión `SparkSession` tarda entre 3 y 5 segundos. Si cada prueba unitaria crea y destruye su propio contexto Spark, una suite de 50 tests tardaría varios minutos.
- **Precisión de punto flotante:** Operaciones matemáticas con decimales y promedios sobre ejecutores distribuidos pueden generar diferencias insignificantes a nivel de micro-redondeo (ej. `0.10000000000000002` frente a `0.1`), haciendo fallar aserciones estrictas de igualdad.

26A.2 Pruebas Unitarias de PySpark con `pytest` y `chispa`

La biblioteca de código abierto **Chispa** es el estándar de la industria para escribir aserciones de igualdad entre DataFrames de Spark con mensajes de error visuales y detallados.

1. Configuración de Fixture Compartido de Spark (`conftest.py`)

Para evitar el sobrecosto de arranque, se configura una única sesión de Spark en memoria local para toda la sesión de pruebas:

PYTHON

```python
# tests/conftest.py
import pytest
from pyspark.sql import SparkSession

@pytest.fixture(scope="session")
def spark():
    """Sesión Spark local optimizada y compartida para toda la suite de
pruebas."""
    spark_session = SparkSession.builder \
        .appName("Pytest-Data-Suite") \
        .master("local[2]") \
        .config("spark.sql.shuffle.partitions", "1") \
        .config("spark.default.parallelism", "1") \
        .config("spark.ui.enabled", "false") \
        .getOrCreate()

    yield spark_session

    spark_session.stop()
```

2. Implementación de Transformaciones Puras

El código de producción debe desacoplar la lógica de negocio de la lectura física de disco:

PYTHON

```python
# src/transformations/clientes_enrichment.py
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

def clasificar_segmento_clientes(df_ventas: DataFrame) -> DataFrame:
    """Clasifica a los clientes según su volumen total de consumo."""
    return df_ventas.withColumn(
        "segmento_valor",
        F.when(F.col("monto_usd") >= 1000.0, "ALTO VALOR")
         .when(F.col("monto_usd") >= 200.0, "MEDIO VALOR")
         .otherwise("ESTANDAR")
    )
```

3. Prueba Unitaria con Aserciones de `chispa`

PYTHON

```python
# tests/test_clientes_enrichment.py
import pytest
from chispa.dataframe_comparer import assert_df_equality
```

```
from src.transformations.clientes_enrichment import clasificar_segmento_clientes
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

def test_clasificar_segmento_clientes(spark):
    # 1. Datos de entrada sintéticos mínimos
    esquema_entrada = StructType([
        StructField("cliente_id", StringType(), False),
        StructField("monto_usd", DoubleType(), False)
    ])
    datos_entrada = [
        ("C1", 1500.0), # Debe clasificar ALTO VALOR
        ("C2", 200.0),  # Límite inclusivo MEDIO VALOR
        ("C3", 50.0),   # ESTANDAR
    ]
    df_entrada = spark.createDataFrame(datos_entrada, esquema_entrada)

    # 2. Ejecutar la función pura
    df_resultado = clasificar_segmento_clientes(df_entrada)

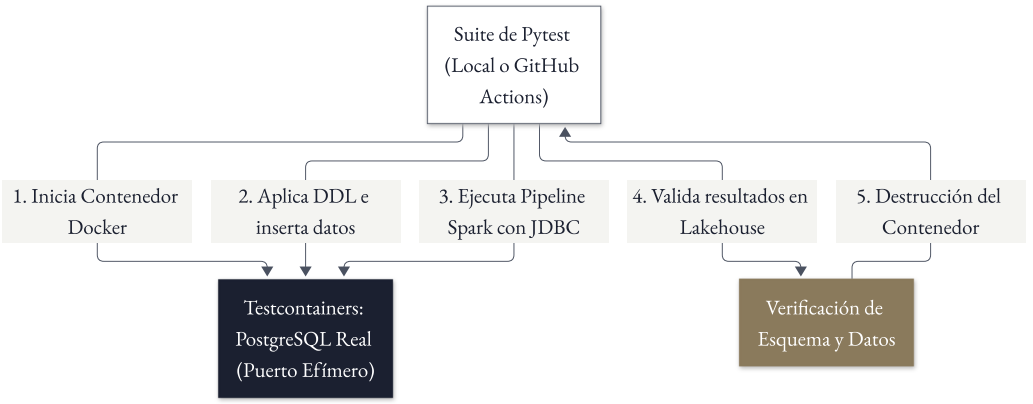
    # 3. DataFrame con la verdad esperada
    esquema_esperado = StructType([
        StructField("cliente_id", StringType(), False),
        StructField("monto_usd", DoubleType(), False),
        StructField("segmento_valor", StringType(), False)
    ])
    datos_esperados = [
        ("C1", 1500.0, "ALTO VALOR"),
        ("C2", 200.0, "MEDIO VALOR"),
        ("C3", 50.0, "ESTANDAR"),
    ]
    df_esperado = spark.createDataFrame(datos_esperados, esquema_esperado)

    # 4. Aserción formal ignorando el orden físico de las filas
    assert_df_equality(
        df_resultado,
        df_esperado,
        ignore_row_order=True,
        ignore_column_order=False,
        ignore_nullable=True
    )
```

26A.3 Pruebas de Integración Aisladas con Testcontainers

Las pruebas unitarias validan la lógica interna, pero los pipelines de datos deben interactuar con sistemas de almacenamiento externos: bases relacionales operacionales (PostgreSQL, MySQL) o almacenes de objetos S3.

Testcontainers permite instanciar contenedores de Docker reales desde el código de Python durante la ejecución de las pruebas, garantizando un entorno 100% aislado que se destruye automáticamente al terminar el test:



Prueba de Ingesta JDBC contra un Contenedor de PostgreSQL Real

PYTHON

```python
# tests/integration/test_jdbc_ingestion.py
import pytest
import psycopg2
from testcontainers.postgres import PostgresContainer
from src.ingestion.jdbc_reader import extraer_tabla_jdbc

@pytest.fixture(scope="module")
def postgres_contenedor():
    """Levanta un contenedor real de PostgreSQL en Docker y lo expone a la
prueba."""
    with PostgresContainer("postgres:16-alpine") as postgres:
        # Inicializar esquema y datos de prueba en la base de datos efímera
        conexion = psycopg2.connect(
            dbname=postgres.dbname,
            user=postgres.username,
            password=postgres.password,
            host=postgres.get_container_host_ip(),
            port=postgres.get_exposed_port(5432)
        )
        cursor = conexion.cursor()
        cursor.execute("""
            CREATE TABLE transacciones_origen (
                id SERIAL PRIMARY KEY,
                cliente_id INT NOT NULL,
                monto NUMERIC(10,2) NOT NULL
            );
            INSERT INTO transacciones_origen (cliente_id, monto) VALUES
                (101, 450.00),
                (102, 120.50),
                (103, 890.00);
        """)
        conexion.commit()
        cursor.close()
        conexion.close()

        yield postgres
```

```
def test_ingesta_jdbc_exitosa(spark, postgres_contenedor):
    url_jdbc =
postgres_contenedor.get_connection_url().replace("postgresql+psycopg2://",
"jdbc:postgresql://")

    # Ejecutar la función de ingesta de producción contra el contenedor efímero
    df_extraido = spark.read \
        .format("jdbc") \
        .option("url", url_jdbc) \
        .option("dbtable", "transacciones_origen") \
        .option("user", postgres_contenedor.username) \
        .option("password", postgres_contenedor.password) \
        .option("driver", "org.postgresql.Driver") \
        .load()

    assert df_extraido.count() == 3
    assert "cliente_id" in df_extraido.columns
```

26A.4 Pruebas de Regresión de Esquemas y Contratos de Datos

Uno de los fallos más destructivos en producción ocurre cuando una actualización altera silenciosamente los tipos de datos de salida (por ejemplo, convertir `monto_usd` de `DoubleType` a `StringType`), rompiendo dashboards y modelos downstream.

Las pruebas de regresión de esquemas garantizan que cualquier cambio no intencionado rompa la suite de pruebas inmediatamente:

PYTHON

```
# tests/test_schema_regression.py
from chispa.schema_comparer import assert_basic_rows_equality

def test_estabilidad_esquema_salida(spark):
    df_actual = ejecutar_pipeline_limpieza(spark)

    # Esquema contractual innegociable
    esquema_esperado = {
        "transaccion_id": "string",
        "cliente_id": "bigint",
        "monto_usd": "double",
        "fecha": "date"
    }

    esquema_real = {campo.name: campo.dataType.simpleString() for campo in
df_actual.schema.fields}
    assert esquema_real == esquema_esperado, f"Discrepancia en contrato de datos:
{esquema_real} != {esquema_esperado}"
```

26A.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Conectar las Pruebas Unitarias a Bases de Datos o Buckets de Producción/Staging
Ejecutar suites de pruebas que lean de `s3://bucket-staging/` hace que los tests sean lentos, dependan de conectividad a internet y fallen si otro ingeniero modifica los datos en la nube.
Corrección: Las pruebas unitarias deben utilizar datos sintéticos locales en memoria; las pruebas de integración deben usar contenedores locales efímeros con Testcontainers.

Anti-patrón 2: Pruebas Lentas que Bloquean el Ciclo de Desarrollo
Una suite de pruebas que tarda 45 minutos desalienta a los desarrolladores de ejecutarla localmente antes de hacer commit.
Corrección: Configure la sesión Spark de pruebas con `master("local[2]")`, deshabilite la UI de Spark (`spark.ui.enabled=false`) y fije `spark.sql.shuffle.partitions=1`. La suite completa debe ejecutarse en menos de 20 segundos.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 26A.1 (Diseño de Prueba Unitaria para Deduplicación con Chispa):
Se tiene una función `deduplicar_novedades(df: DataFrame) -> DataFrame` que recibe registros duplicados y conserva únicamente el de mayor `fecha_actualizacion` por cada `cliente_id`.
Escriba una prueba unitaria completa con `pytest` y `chispa` que verifique que, ante dos registros con el mismo `cliente_id` pero fechas distintas, la función descarta el registro antiguo y preserva el más reciente.

Solución 26A.1:

PYTHON

```python
import pytest
from chispa.dataframe_comparer import assert_df_equality
from pyspark.sql import functions as F
from pyspark.sql.window import Window

def deduplicar_novedades(df):
    w =
Window.partitionBy("cliente_id").orderBy(F.col("fecha_actualizacion").desc())
    return df.withColumn("rn", F.row_number().over(w)).filter(F.col("rn") ==
1).drop("rn")

def test_deduplicar_novedades_conserva_mas_reciente(spark):
    datos_entrada = [
        (101, "Bogota", "2026-01-01"),
        (101, "Medellin", "2026-02-15"), # Registro más reciente que debe
conservarse
        (102, "Cali", "2026-01-10"),
    ]
    df_in = spark.createDataFrame(datos_entrada, ["cliente_id", "ciudad",
"fecha_actualizacion"])

    df_out = deduplicar_novedades(df_in)

    datos_esperados = [
        (101, "Medellin", "2026-02-15"),
        (102, "Cali", "2026-01-10"),
    ]
    df_expected = spark.createDataFrame(datos_esperados, ["cliente_id", "ciudad",
"fecha_actualizacion"])

    assert_df_equality(df_out, df_expected, ignore_row_order=True)
```

26A.6 Resumen y Puntos Clave

- **Pirámide de pruebas:** La base son pruebas unitarias rápidas de funciones puras en memoria; el centro, pruebas de integración con Testcontainers; la cúspide, pruebas de calidad de datos en producción.
- **Chispa y Pytest:** Permiten comparar DataFrames distribuidos de forma tolerante a orden de filas y diferencias de precisión numérica.
- **Sesión Spark optimizada:** Fijar `local[2]` y `spark.sql.shuffle.partitions=1` permite ejecutar decenas de pruebas unitarias en cuestión de segundos en CI/CD.
- **Aislamiento con Testcontainers:** Permite probar código de ingesta JDBC y CDC contra bases relacionales reales sin ensuciar servidores compartidos.

Lecturas Recomendadas

1. **Powers, M. (2020).** *Testing PySpark Applications with Chispa and Pytest.* Leanpub.
2. **Beck, K. (2002).** *Test-Driven Development: By Example.* Addison-Wesley.
3. **Testcontainers Authors (2024).** *Testcontainers Python: Native Docker Testing for Data Engineers.* Testcontainers.org.

PARTE IX

Observabilidad y calidad de datos

Capítulo 27 · Observabilidad, Monitoreo y Logging en Sistemas Distribuidos RECOMENDADO

Capítulo 28 · Calidad de Datos (Data Quality) y Validación Automatizada RECOMENDADO

CAPÍTULO 27

Observabilidad, Monitoreo y Logging en Sistemas Distribuidos RECOMENDADO

> *"Si un pipeline de datos falla en la mitad de la noche y ningún sistema genera una alerta correlacionada, los datos no existen para el negocio. La observabilidad moderna no consiste únicamente en detectar si un proceso terminó con código cero; exige medir la latencia, la frescura, el volumen, el linaje y la salud de la memoria en cada nodo del clúster."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender los tres pilares de la observabilidad distribuidos aplicados a plataformas de datos: **Métricas**, **Logs Estructurados** y **Trazas Distribuidas**.
- Diagnosticar y resolver cuellos de botella críticos en la **Spark UI**: *Data Skew*, *Shuffle Spill* y presión de *Garbage Collection (GC)*.
- Implementar **trazabilidad distribuida con OpenTelemetry** y propagación de contexto W3C (*Trace Context* / `traceparent`) en pipelines de PySpark.
- Auditar la ejecución de flujos continuos en tiempo real mediante `StreamingQueryListener` y exportación de métricas a Prometheus.
- Diseñar alertas operacionales basadas en **Acuerdos de Nivel de Servicio (Data SLAs)** para detectar fallos silenciosos (*Soft Failures*).

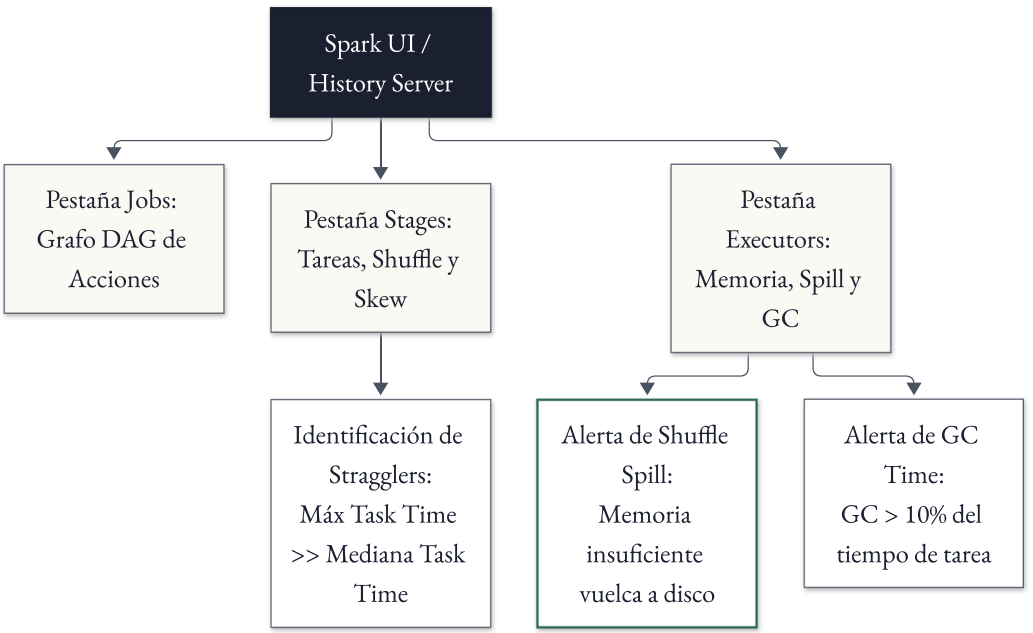
Definición 27.1 (Los Tres Pilares de la Observabilidad de Datos)

La **observabilidad de sistemas de datos** es la capacidad de inferir el estado interno de pipelines distribuidos y almacenes analíticos a partir de sus señales externas de telemetría:

1. **Métricas:** Valores numéricos agregados a lo largo del tiempo (por ejemplo, registros procesados por segundo, bytes de shuffle, tiempo de GC de la JVM, uso de CPU en los Workers).
2. **Logs Estructurados:** Registros de eventos serializados en formato JSON con metadatos contextuales estandarizados (nivel, timestamp UTC, nombre de pipeline, identificador de partición).
3. **Trazas Distribuidas (Distributed Tracing):** Trazabilidad continua de una unidad de trabajo a través de múltiples fronteras de red y sistemas heterogéneos (Airflow → Kafka → Spark → Delta Lake) mediante identificadores de correlación propagados (*Trace IDs* y *Span IDs*).

27.1 Diagnóstico de Clústeres con Spark UI

La **Spark UI** (accesible en el puerto `4040` del nodo Driver durante la ejecución o a través del *Spark History Server* tras finalizar) es el instrumento fundamental para auditar el desempeño de cómputo distribuido.



1. Pestaña Stages: Detección de Asimetría (Data Skew)

En la tabla de resumen de métricas de tareas (*Task Metrics*), examine los percentiles:

- **Síntoma:** El percentil 75% o la mediana (*50th percentile*) de las tareas dura 2 segundos, pero la tarea máxima dura 42 minutos.
- **Diagnóstico:** Asimetría severa de datos (*Data Skew*). Un único ejecutor procesa una partición desproporcionada generada por una clave dominante o nula.

2. Pestaña Executors: Detección de Desbordamiento de Memoria (Shuffle Spill)

- **Spill (Memory):** Volumen de datos deserializados en memoria que no cupieron en el espacio de ejecución (*Execution Memory*) del ejecutor.
- **Spill (Disk):** Volumen de datos serializados y comprimidos que Spark se vio forzado a escribir en los discos locales de los Workers.
- **Impacto:** Si el Spill supera varios gigabytes, el rendimiento cae abruptamente debido a la alta latencia de I/O en disco.

3. Pestaña Executors: Presión de Garbage Collection (GC)

- Si la columna **Task Deserialization / GC Time** supera el **10%** del tiempo total de cómputo del ejecutor, la JVM pasa demasiado tiempo deteniendo los hilos de trabajo para liberar objetos en el heap (*Stop-the-World pauses*).
- **Mitigación:** Incrementar `spark.executor.memory`, utilizar estructuras columnares compactas y aprovechar la memoria binaria de Tungsten.

27.2 Trazabilidad Distribuida con OpenTelemetry en PySpark

El estándar de facto en la industria para propagar contexto distribuido a través de pipelines es **OpenTelemetry (OTel)**. Utilizando el formato estándar W3C (*Trace Context*), una cabecera `traceparent` generada por un orquestador (como Apache Airflow) se inyecta en las tareas de Spark para correlacionar todas las señales en plataformas como Datadog, Grafana Tempo o Jaeger:

PYTHON

```python
import json
import logging
import sys
from datetime import datetime, timezone
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor,
ConsoleSpanExporter
from opentelemetry.trace.propagation.tracecontext import
```

```
TraceContextTextMapPropagator

# 1. Inicialización del proveedor de trazas de OpenTelemetry
provider = TracerProvider()
processor = BatchSpanProcessor(ConsoleSpanExporter())
provider.add_span_processor(processor)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("pipeline.lakehouse.silver")

# 2. Formateador JSON que correlaciona logs con el Trace ID activo de
OpenTelemetry
class OTelJSONFormatter(logging.Formatter):
    def format(self, record):
        current_span = trace.get_current_span()
        ctx = current_span.get_span_context() if current_span else None

        trace_id = format(ctx.trace_id, "032x") if ctx and ctx.is_valid else
"N/A"
        span_id = format(ctx.span_id, "016x") if ctx and ctx.is_valid else "N/A"

        log_record = {
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "message": record.getMessage(),
            "trace_id": trace_id,
            "span_id": span_id,
            "pipeline": getattr(record, "pipeline_name", "lakehouse_etl"),
        }
        if hasattr(record, "extra_fields"):
            log_record.update(record.extra_fields)
        return json.dumps(log_record)

logger = logging.getLogger("LakehouseLogger")
logger.setLevel(logging.INFO)
handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(OTelJSONFormatter())
logger.addHandler(handler)

# 3. Ejecución del pipeline dentro de un Span contextualizado
# Si Airflow envió el contexto W3C, se extrae mediante
TraceContextTextMapPropagator
carrier = {"traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-
01"}
contexto_extraido = TraceContextTextMapPropagator().extract(carrier=carrier)

with tracer.start_as_current_span("procesar_ventas_silver",
context=contexto_extraido) as span:
    span.set_attribute("dataset.name", "ventas")
    span.set_attribute("environment", "prod")

    logger.info("Iniciando transformación Silver con contexto propagado", extra=
{"pipeline_name": "ventas_silver"})

    # Simulación de cómputo en Spark...
    filas_procesadas = 500000
    span.set_attribute("dataset.rows_processed", filas_procesadas)
```

```
    logger.info("Transformación finalizada exitosamente", extra={"pipeline_name":
"ventas_silver", "extra_fields": {"rows": filas_procesadas}})
```

27.3 Monitoreo en Tiempo Real: `StreamingQueryListener` y Prometheus

En pipelines de streaming continuo, los logs estándar no bastan para auditar caídas de rendimiento. Apache Spark expone la interfaz **`StreamingQueryListener`**, que captura eventos del ciclo de vida de los micro-lotes y permite enviar métricas en tiempo real a Prometheus o Datadog:

PYTHON

```python
from pyspark.sql.streaming import StreamingQueryListener

class ObservabilidadStreaming(StreamingQueryListener):
    def onQueryStarted(self, event):
        print(f"[STREAMING INICIADO] Query ID: {event.id}, Nombre: {event.name}")

    def onQueryProgress(self, event):
        progreso = event.progress
        query_name = progreso.name or str(progreso.id)

        # Extracción de métricas operacionales clave
        input_rate = progreso.inputRowsPerSecond
        process_rate = progreso.processedRowsPerSecond
        batch_id = progreso.batchId
        duration_total = progreso.durationMs.get("total", 0)

        print(f"[MICRO-LOTE {batch_id}] Stream: {query_name} | "
              f"Entrada: {input_rate:.1f} filas/s | "
              f"Procesamiento: {process_rate:.1f} filas/s | "
              f"Duración: {duration_total} ms")

        # Alerta: Si la tasa de entrada supera sistemáticamente la de
procesamiento,
        # la cola de eventos en Kafka comenzará a acumular retraso (Consumer
Lag).
        if input_rate > process_rate and duration_total > 10000:
            print(f"[ALERTA OPERACIONAL] Lag detectado en el stream
{query_name}")

    def onQueryTerminated(self, event):
        if event.exception:
            print(f"[STREAMING FALLIDO] Query ID: {event.id} terminó con error:
{event.exception}")
        else:
            print(f"[STREAMING DETENIDO] Query ID: {event.id} finalizó
normalmente")
```

```
# Registro del listener en la SparkSession activa
spark.streams.addListener(ObservabilidadStreaming())
```

27.4 Fallos Suaves (*Soft Failures*) y Acuerdos de Nivel de Servicio (Data SLAs)

Un error crítico en la ingeniería de datos es suponer que el sistema está sano porque no hubo excepciones técnicas (`exit code 0`).

Un pipeline puede concluir exitosamente sin errores de compilación pero experimentar un **fallo suave (*Soft Failure*)**:

- El pipeline procesó **0 filas** debido a que la fuente no generó el archivo esperado.
- La tasa de valores nulos en la columna `cliente_id` subió repentinamente del **0.01% al 45%**.
- El volumen total cayó un **80%** en comparación con el promedio de los últimos 30 días.

```
┌──────────────────────────────────────────────────┐
│            ACUERDOS DE SERVICIO DE DATOS (SLAs)  │
├──────────────────┬───────────────────────────────┤
│ 1. Frescura (SLA)  │ Máximo retraso aceptable: 30 min  │
│ 2. Volumen (SLA)   │ Desviación máxima tolerada: ±20%  │
│ 3. Completitud     │ Nulos en claves de negocio = 0%   │
│ 4. Linaje          │ Trazabilidad 100% hasta Bronze    │
└──────────────────┴───────────────────────────────┘
```

Las alertas operacionales deben configurarse sobre métricas estadísticas de calidad y frescura, notificando a través de Slack o PagerDuty antes de que los reportes ejecutivos del negocio resulten afectados.

27.5 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Logging Masivo en Nivel DEBUG Dentro de Bucles de Ejecutores
Emitir mensajes de log dentro de una función UDF aplicada a millones de registros genera millones de líneas de log por segundo, saturando los discos locales de los nodos de trabajo y degradando la red del clúster. Reserve `DEBUG` para pruebas locales y use `INFO` con resúmenes agregados en el Driver.

Anti-patrón 2: Alertamiento Basado Exclusivamente en Códigos de Retorno del Proceso
Depender únicamente del éxito técnico del script oculta caídas drásticas en el volumen de datos o tablas vacías. Toda compuerta de observabilidad debe auditar métricas semánticas (conteo de filas y frescura temporal).

Anti-patrón 3: Utilizar `coalescePartitions` para Resolver Problemas de Spill
Creer erróneamente que activar la fusión de particiones de AQE solucionará un problema de desbordamiento de memoria (*Shuffle Spill*). La fusión consolida múltiples particiones pequeñas en una sola partición más grande, lo que satura aún más la memoria del ejecutor y empeora drásticamente el spill.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 27.1 (Diagnóstico y Mitigación de Shuffle Spill en Spark UI):
Durante la ejecución de un cruce entre dos tablas de 500 millones de registros, un trabajo en Spark muestra los siguientes síntomas en la pestaña *Executors*:

- Memoria total asignada por ejecutor: 16 GB.
- Shuffle Spill (Memory): 120 GB.
- Shuffle Spill (Disk): 45 GB.
- Tiempo total del Stage: 35 minutos.

1. ¿Qué fenómeno físico está ocurriendo en los nodos ejecutores?
2. Mencione dos configuraciones de Spark que mitiguen este cuello de botella y explique por qué **no se debe** utilizar la fusión de particiones (`coalescePartitions`).

Solución 27.1:

1. *Diagnóstico:* La memoria asignada para operaciones de ejecución de shuffle (`Execution Memory`) en los ejecutores resultó insuficiente para almacenar en memoria RAM las particiones intermedias generadas por el algoritmo de ordenamiento y cruce. Spark se vio forzado a volcar (*spill*) 120 GB de estructuras deserializadas hacia los discos locales de los Workers, comprimiéndolas en 45 GB de almacenamiento físico temporal. El tiempo de ejecución se disparó debido a la alta latencia de I/O de lectura y escritura repetida en disco.
2. *Soluciones de configuración:*
    - **Incrementar el particionamiento de shuffle:** Aumentar `spark.sql.shuffle.partitions` de su valor predeterminado (200) a un valor sensiblemente superior (por ejemplo, 1.000 o 2.000). Al dividir el mismo volumen global de datos en más particiones, el tamaño de cada partición individual disminuye proporcionalmente, permitiendo que quepa holgadamente en la memoria RAM del ejecutor sin derramarse a disco.
    - **Clarificación sobre `coalescePartitions` vs. Mitigación Real:** La directiva `spark.sql.adaptive.coalescePartitions.enabled = true` fusiona dinámicamente particiones pequeñas post-shuffle hasta alcanzar el umbral de

`spark.sql.adaptive.advisoryPartitionSizeInBytes` (por defecto 64 MB), con el fin de evitar la sobrecarga de programar tareas insignificantes. Sin embargo, **no resuelve por sí sola el problema de spill** cuando el origen es un desbalance de datos (*data skew*) o un tamaño objetivo demasiado grande para la memoria disponible. Para resolver el spill, se debe activar la detección de sesgo (`spark.sql.adaptive.skewJoin.enabled = true`), reducir el tamaño objetivo recomendado (`advisoryPartitionSizeInBytes`), o aplicar una técnica de salting en la clave de unión.

Ejercicio 27.2 (Diseño de Alerta de Frescura de Datos):
Escriba una consulta SQL para verificar si una tabla Silver particionada cumple un SLA de frescura de **no más de 4 horas de retraso** respecto al tiempo actual UTC.

Solución 27.2:

```sql
SELECT
    max(timestamp_evento) AS ultima_marca_tiempo,
    current_timestamp() AS tiempo_actual_utc,
    round((unix_timestamp(current_timestamp()) -
unix_timestamp(max(timestamp_evento))) / 3600, 2) AS horas_retraso,
    CASE
        WHEN (unix_timestamp(current_timestamp()) -
unix_timestamp(max(timestamp_evento))) / 3600 > 4.0
        THEN 'SLA_VIOLADO_CRITICO'
        ELSE 'SLA_CUMPLIDO'
    END AS estado_sla
FROM lakehouse.silver.facturacion;
```

Mecanismo: Si el campo `estado_sla` retorna `'SLA_VIOLADO_CRITICO'`, el orquestador o la herramienta de observabilidad (Monte Carlo, DataHub, Great Expectations) dispara una alerta inmediata a los canales de guardia de ingeniería.

27.6 Resumen y Puntos Clave

- **Tríada de observabilidad:** Métricas para cuantificar tendencias, logs estructurados JSON para auditoría granular y trazas distribuidas para seguir el flujo de datos entre sistemas heterogéneos.
- **Spark UI como herramienta diagnóstica:** Permite identificar sesgos de datos observando percentiles de tareas en *Stages* y detectar saturación de memoria mediante métricas de *Shuffle Spill* y tiempos de *Garbage Collection*.

- **Trazabilidad con OpenTelemetry:** La propagación del estándar W3C `traceparent` asegura la correlación unívoca de logs entre Airflow, Kafka y Spark.
- **Mitigación rigurosa del Spill:** Exige aumentar el número de particiones de shuffle o la memoria de los ejecutores, evitando expresamente la fusión de particiones.

Lecturas Recomendadas

1. **Sridharan, C. (2018).** *Distributed Systems Observability: A Guide to Building Robust Systems.* O'Reilly Media.
2. **Moses, B., Gavish, L., & Vorwerck, K. (2022).** *Data Quality Fundamentals: A Practitioner's Guide to Building Trust in Data.* O'Reilly Media.
3. **OpenTelemetry Authors (2024).** *OpenTelemetry Specification: Trace Context and Logging Standards.* Cloud Native Computing Foundation (`https://opentelemetry.io/docs/`).

CAPÍTULO 28

Calidad de Datos (Data Quality) y Validación Automatizada RECOMENDADO

> *"Procesar datos corruptos con alta velocidad computacional no es una ventaja competitiva; es la forma más rápida y costosa de tomar decisiones empresariales catastróficas. La calidad de datos no se inspecciona al final sobre los tableros ejecutivos de BI; se valida como una compuerta innegociable (Quality Gate) en cada etapa del pipeline analítico."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender y medir las **seis dimensiones formales de la calidad de datos**: Completitud, Unicidad, Validez, Exactitud, Consistencia y Puntualidad.
- Dominar la API moderna de **Great Expectations (GX 1.x)** basada en fuentes de datos, activos, definiciones de validación (*Validation Definitions*) y suites de expectativas.
- Diseñar compuertas de calidad estrictas (*Hard Gates*) y compuertas informativas (*Soft Gates*) en la transición entre las capas Bronze y Silver.
- Implementar el **Patrón de Cuarentena de Datos (*Data Quarantine Pattern*)** en PySpark resolviendo rigurosamente la lógica booleana trivalente y garantizando idempotencia.
- Aplicar el **Teorema del Límite Central (TLC)** y el Error Estándar de la Media para definir umbrales estadísticos matemáticamente válidos en compuertas de calidad.

Definición 28.1 (Calidad de Datos y Great Expectations)

Great Expectations (GX) es el estándar de código abierto líder para la validación declarativa, documentación viva y perfilado estadístico de conjuntos de datos en ingeniería.

Permite a los equipos declarar afirmaciones computables y verificables denominadas **Expectations** sobre columnas, tablas y distribuciones de probabilidad, ejecutándolas automáticamente contra DataFrames distribuidos en Apache Spark, motores SQL o almacenes analíticos cloud.

28.1 Las Seis Dimensiones Universales de la Calidad de Datos

Toda estrategia formal de gobierno analítico debe evaluar los datos frente a seis dimensiones axiomáticas:



- **Completitud (*Completeness*):** Proporción de valores no nulos en columnas obligatorias (ej. `email IS NOT NULL`).
- **Unicidad (*Uniqueness*):** Ausencia de multiplicidad en identificadores primarios (`COUNT(DISTINCT id) == COUNT(id)`).
- **Validez (*Validity*):** Conformidad con esquemas, expresiones regulares y conjuntos canónicos (ej. código de país ISO 3166-1 alfa-2).
- **Exactitud (*Accuracy*):** Grado de concordancia con la fuente de verdad (ej. conciliación de saldos contables con la pasarela de pagos).
- **Consistencia (*Consistency*):** Ausencia de contradicciones lógicas entre tablas vinculadas (ej. no pueden existir órdenes con fechas anteriores a la fecha de creación del cliente).
- **Puntualidad (*Timeliness*):** Tiempo transcurrido entre el evento de negocio y su disponibilidad en la capa Gold para analítica.

28.2 Implementación con la API Moderna de Great Expectations (GX 1.x)

En versiones recientes de Great Expectations (especificación 1.x), la API fue completamente modernizada para desacoplar las fuentes de cómputo de las reglas de validación:

PYTHON

```python
import great_expectations as gx
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Capitulo28-GX1-QualityGate") \
    .master("local[*]") \
    .getOrCreate()

# 1. Cargar el DataFrame de la capa Bronze
df_transacciones_bronze = spark.read.parquet("lakehouse/bronze/transacciones/")

# 2. Inicializar el contexto de Great Expectations
contexto = gx.get_context()

# 3. Registrar la fuente de datos Spark y el activo en memoria
fuente_datos = contexto.data_sources.add_spark(name="spark_engine_lakehouse")
activo_datos = fuente_datos.add_dataframe_asset(name="activo_transacciones")

# Definición de lote para el DataFrame en memoria
definicion_lote =
activo_datos.add_batch_definition_whole_dataframe(name="lote_completo")
lote_datos = definicion_lote.get_batch(batch_parameters={"dataframe":
df_transacciones_bronze})

# 4. Declaración de la Suite de Expectativas de Negocio
suite =
contexto.suites.add(gx.ExpectationSuite(name="suite_calidad_silver_transacciones"
))

suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(column="trans
accion_id"))
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeUnique(column="transa
ccion_id"))
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(
    column="monto_usd", min_value=0.01, max_value=1000000.00
))
suite.add_expectation(gx.expectations.ExpectColumnValuesToBeInSet(
    column="moneda", value_set=["USD", "EUR", "COP", "MXN"]
))

# 5. Definición y ejecución de la compuerta de validación
validacion = contexto.validation_definitions.add(
    gx.ValidationDefinition(name="compuerta_silver", data=definicion_lote,
suite=suite)
)
```

```
resultado = validacion.run(batch_parameters={"dataframe":
df_transacciones_bronze})

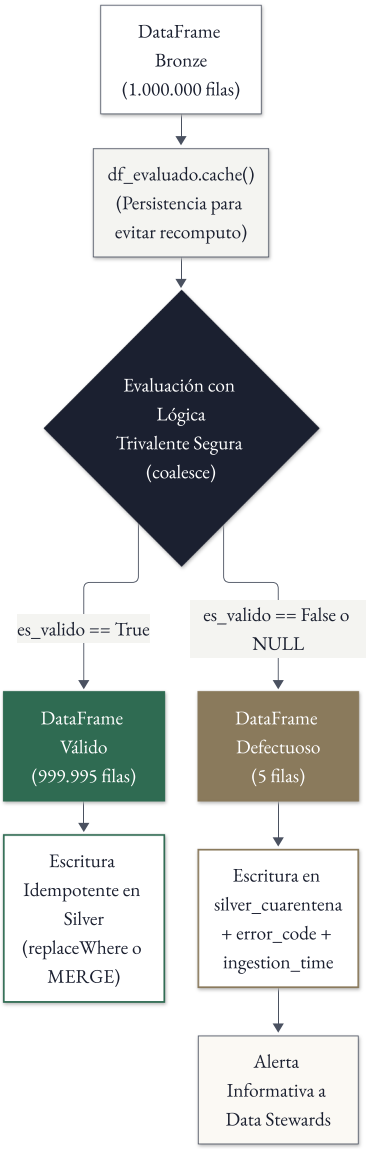
print(f"¿Aprobó la compuerta de calidad?: {resultado.success}")
```

28.3 El Patrón de Cuarentena de Datos: Lógica Trivalente e Idempotencia

En sistemas de producción a escala de gigabytes o terabytes, aplicar una política binaria de *"todo o nada"* (*Hard Stop*) es inaceptable: si un archivo contiene **1.000.000 de filas válidas** y **5 filas corruptas** (por ejemplo, un código postal con formato incorrecto), abortar todo el pipeline bloquea las operaciones del negocio y detiene los tableros analíticos.

El **Patrón de Cuarentena** segrega condicionalmente el lote en dos flujos divergentes:

1. **Flujo Conforme:** Los registros válidos continúan su tránsito hacia la capa Silver.
2. **Flujo de Cuarentena:** Las filas defectuosas se bifurcan hacia una tabla dedicada `silver_cuarentena`, enriquecidas con columnas de auditoría que explican la regla exacta violada, notificando al equipo sin interrumpir la operación:



El Peligro de la Lógica Booleana Trivalente en SQL y Spark

En álgebra booleana tradicional existen dos estados (`True`, `False`). En SQL y Apache Spark rige la **lógica trivalente** con valores nulos:

- Si la columna `monto_usd` es `NULL`, la expresión `monto_usd > 0.0` evalúa a `NULL`, no a `False`.
- Si se evalúa `df.filter(F.col("es_valido") == True)`, la fila con `NULL` **se descarta**.
- Pero si luego se filtra `df.filter(F.col("es_valido") == False)`, **la fila con `NULL` también se descarta**, ¡porque `NULL == False` no es verdadero!

- **Consecuencia:** Esas filas corruptas se evaporan por completo de ambos flujos, perdiéndose en el vacío sin llegar ni a Silver ni a la cuarentena.

Para evitar este grave fallo de datos, se debe envolver la condición global con `coalesce` o forzar el manejo explícito de nulos:

PYTHON

```python
import pyspark.sql.functions as F

# 1. Reglas atómicas de validez manejando nulos de forma explícita
regla_id_valido = F.col("transaccion_id").isNotNull()
regla_monto_positivo = F.col("monto_usd").isNotNull() & (F.col("monto_usd") >
0.0)
regla_moneda_valida = F.col("moneda").isNotNull() & F.col("moneda").isin("USD",
"EUR", "COP", "MXN")

# 2. Condición global protegida con coalesce (nunca evalúa a NULL)
condicion_validez_global = F.coalesce(
    regla_id_valido & regla_monto_positivo & regla_moneda_valida,
    F.lit(False)
)

# 3. Clasificación y persistencia en caché para evitar re-escaneo
df_evaluado = df_transacciones_bronze.withColumn(
    "es_valido", condicion_validez_global
).withColumn(
    "motivos_cuarentena",
    F.concat_ws(", ",
        F.when(~regla_id_valido, F.lit("ID_NULO")),
        F.when(~regla_monto_positivo, F.lit("MONTO_INVALIDO_O_NULO")),
        F.when(~regla_moneda_valida, F.lit("MONEDA_NO_AUTORIZADA_O_NULA"))
    )
).cache()  # Caching indispensable para no recomputar en isEmpty, count y writes

try:
    # 4. Bifurcación física segura
    df_validos = df_evaluado.filter(F.col("es_valido") == True).drop("es_valido",
"motivos_cuarentena")

    # Se capturan tanto los Falses explícitos como cualquier posible residuo nulo
    df_cuarentena = df_evaluado.filter((F.col("es_valido") == False) |
F.col("es_valido").isNull()).withColumn(
        "fecha_cuarentena", F.current_timestamp()
    )

    # 5. Escritura idempotente en la capa Silver mediante replaceWhere
    # (Evita inserciones duplicadas ante reintentos del pipeline para la fecha
del lote)
    fecha_lote = 20261012
    df_validos.write \
        .format("delta") \
        .mode("overwrite") \
        .option("replaceWhere", f"fecha_id = {fecha_lote}") \
        .save("lakehouse/silver/transacciones")
```

```
    # 6. Almacenamiento en cuarentena con append auditable
    if not df_cuarentena.isEmpty():
        conteo_corruptos = df_cuarentena.count()
        print(f"[ALERTA DE CALIDAD] Desviando {conteo_corruptos} filas
defectuosas a cuarentena.")
        df_cuarentena.write \
            .format("delta") \
            .mode("append") \
            .save("lakehouse/silver/transacciones_cuarentena")

finally:
    df_evaluado.unpersist()
```

28.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Validaciones Mediante Scripts SQL Ad-Hoc Desconectados Escribir consultas manuales dispersas del tipo `SELECT COUNT(*) WHERE id IS NULL` en notebooks de Jupyter genera deuda técnica masiva y carece de integración con el linaje de la plataforma. La calidad debe gobernarse mediante frameworks declarativos integrados (Great Expectations, Soda Core o dbt test).

Anti-patrón 2: Desactivar Compuertas de Calidad Durante Cierres Contables o Picos de Carga Eliminar las validaciones para "acelerar el procesamiento" cuando el pipeline sufre retrasos garantiza la contaminación de los modelos de machine learning y la emisión de reportes financieros inconsistentes.

Anti-patrón 3: Carencia de Mecanismos de Reingesta desde Cuarentena Almacenar registros en tablas de cuarentena sin diseñar un proceso de remediación y reprocesamiento convierte la cuarentena en un cementerio de datos inerte. Todo registro en cuarentena debe contar con un ciclo de vida definido para su corrección y reingreso a la capa Silver.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 28.1 (Diferenciación entre Hard Gates y Soft Gates):
En el diseño de una compuerta de calidad de datos, explica la diferencia entre una compuerta estricta (*Hard Gate*) y una compuerta informativa (*Soft Gate*). Proporciona un ejemplo empresarial concreto para cada una.

Solución 28.1:

1. **Compuerta Estricta (Hard Gate):**

- *Mecanismo:* Si la aserción de calidad falla, la ejecución del pipeline **se aborta inmediatamente** lanzando una excepción fatal. Ningún dato del lote es promovido a la siguiente capa.
- *Caso de uso:* Detección de duplicados en la clave primaria (`orden_id`) o montos de transacciones monetarias nulos. Promover estos datos corrompería el libro mayor contable y los balances tributarios de la compañía.

2. **Compuerta Informativa (Soft Gate):**
   - *Mecanismo:* Si la aserción falla, el pipeline registra una advertencia (*Warning*), emite una notificación al canal de observabilidad pero **continúa su ejecución**, permitiendo que los datos fluyan.
   - *Caso de uso:* El porcentaje de usuarios que no diligenciaron su número telefónico de contacto supera el 5% habitual y sube al 8%. Aunque la anomalía amerita investigación por el equipo de CRM, no compromete la integridad matemática de las transacciones de ventas.

Ejercicio 28.2 (Umbrales Estadísticos y el Teorema del Límite Central):
Un pipeline procesa lotes de $n = 10.000$ transacciones financieras diarias. Se conoce históricamente que el monto individual de una transacción tiene una media poblacional $\mu = \$150.00\ \text{USD}$ y una desviación estándar poblacional $\sigma = \$30.00\ \text{USD}$.

1. ¿Por qué es un error estadístico grave fijar un umbral de calidad sobre la **media del lote** en el rango $[\mu - \sigma, \mu + \sigma] = [120, 180]$?
2. Aplicando el **Teorema del Límite Central (TLC)**, calcula el Error Estándar de la Media (SEM) y define el intervalo de control al 99.7% ($\pm 3 \cdot \text{SEM}$).
3. Escribe la expectativa formal de Great Expectations con el umbral matemáticamente riguroso.

Solución 28.2:

1. *Error del umbral* $[\mu - \sigma, \mu + \sigma]$*:* La desviación estándar poblacional $\sigma = 30$ mide la variabilidad de **transacciones individuales**, no de la **media de un lote de 10.000 registros**. Fijar un rango de $[120, 180]$ es un umbral excesivamente holgado que permitiría que la media del lote se desvíe en más de 100 errores estándar sin disparar ninguna alarma, tolerando que el 80% de los datos estén corruptos.
2. *Cálculo del Error Estándar de la Media (SEM):* De acuerdo con el Teorema del Límite Central, la distribución muestral de la media $\bar{X}$ tiene una desviación típica dada por:

$$\text{SEM} = \frac{\sigma}{\sqrt{n}} = \frac{30.00}{\sqrt{10\,000}} = \frac{30.00}{100} = \mathbf{0.30}\ \text{USD}$$

Para un intervalo de confianza al 99.73% (regla empírica de $\pm 3\sigma$):

$$\text{Límite Inferior} = \mu - 3 \times \text{SEM} = 150.00 - 3(0.30) = \mathbf{149.10}\ \text{USD}$$

$$\text{Límite Superior} = \mu + 3 \times \text{SEM} = 150.00 + 3(0.30) = \mathbf{150.90}\ \text{USD}$$

3. *Expectativa en Great Expectations:*

```python
suite.add_expectation(
    gx.expectations.ExpectColumnMeanToBeBetween(
        column="monto_usd",
        min_value=149.10,
        max_value=150.90
    )
)
```

Si un fallo operacional provoca que las transacciones se carguen en centavos en lugar de dólares (media ≈ 15.000), o si la fuente experimenta una anomalía severa, la media muestral violará el intervalo $[149.10, 150.90]$ instantáneamente, activando la compuerta de seguridad.

28.5 Resumen y Puntos Clave

- **Seis dimensiones de calidad:** Completitud, unicidad, validez, exactitud, consistencia y puntualidad constituyen la base para certificar la fiabilidad de los datos.
- **API moderna de Great Expectations (GX 1.x):** Estructura la validación desacoplando motores de datos, definiciones de lotes y suites declarativas de expectativas.
- **Lógica trivalente en cuarentena:** Toda regla de clasificación debe protegerse con `coalesce` y pruebas explícitas de nulos para garantizar que las filas defectuosas no se evaporen de ambos flujos.
- **Idempotencia y Caching:** El DataFrame clasificado debe cachearse en memoria antes de evaluar bifurcaciones múltiples, y los datos válidos deben persistirse con `replaceWhere` en Delta Lake.
- **Rigor estadístico del TLC:** Las compuertas de calidad sobre medias agregadas deben dimensionarse según el Error Estándar de la Media ($\sigma/\sqrt{n}$), nunca según la dispersión de filas individuales.

Lecturas Recomendadas

1. **Moses, B., Gavish, L., & Vorwerck, K. (2022).** *Data Quality Fundamentals: A Practitioner's Guide to Building Trust in Data.* O'Reilly Media.
2. **Great Expectations Project Documentation (2024).** *GX 1.0 Core Architecture and Validation Workflows.* (`https://docs.greatexpectations.io/docs/core/`).
3. **Casella, G. & Berger, R. L. (2002).** *Statistical Inference (2nd Edition).* Duxbury Press. (Capítulo 5: Properties of a Random Sample and the Central Limit Theorem).

PARTE X

Arquitectura cloud, System Design e Inteligencia Artificial

Capítulo 29 · Cloud Data Architecture y Optimización de Costos FinOps RECOMENDADO

Capítulo 29A · Microsoft Fabric y OneLake: Arquitectura SaaS y el Futuro Analítico en Azure RECOMENDADO

Capítulo 29B · Arquitectura de Plataformas de Datos en AWS: S3 Data Lake, Glue, EMR, Athena y Redshift RECOMENDADO

Capítulo 29C · Arquitectura de Plataformas de Datos en Google Cloud (GCP): BigQuery, Dataflow, Pub/Sub y Dataproc RECOMENDADO

Capítulo 29D · Plataforma Snowflake Empresarial: Arquitectura Multi-Cluster, Snowpark, Dynamic Tables, Snowpipe y Polaris Catalog RECOMENDADO

Capítulo 30 · System Design para Data Engineers y Preparación para Roles Senior AVANZADO

Capítulo 31 · Seguridad, Privacidad y Gobernanza Regulatoria de Datos AVANZADO

Capítulo 32 · Bases de Datos NoSQL y Motores Especializados AVANZADO

Capítulo 33 · Ingeniería de Datos para Inteligencia Artificial y LLMs AVANZADO

CAPÍTULO 29

Cloud Data Architecture y Optimización de Costos FinOps RECOMENDADO

> *"La nube pública promete escalabilidad elástica y capacidad computacional infinita, pero esa misma elasticidad puede desencadenar facturas astronómicas si la arquitectura no está gobernada por principios rigurosos de eficiencia algorítmica y FinOps. En la ingeniería de datos senior, el rendimiento técnico y la viabilidad financiera son dos caras inseparables de la misma moneda."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Diseñar arquitecturas analíticas desacopladas comparando los ecosistemas de **AWS**, **Microsoft Azure** (incluyendo **Microsoft Fabric** y OneLake) y **Google Cloud Platform (GCP)**.
- Desglosar con precisión matemática los modelos de tarificación de cómputo y almacenamiento en **Snowflake**, **Google BigQuery** y **Databricks**.
- Identificar y mitigar los costos ocultos de transferencia de red entre regiones y zonas de disponibilidad (*Data Egress*).
- Implementar estrategias de optimización **FinOps** para reducir drásticamente el gasto en clústeres distribuidos mediante instancias *Spot/Preemptible* y clústeres efímeros.
- Configurar políticas de ciclo de vida de almacenamiento (*Storage Tiering*) preservando acuerdos regulatorios.

Definición 29.1 (FinOps en Plataformas de Datos)

FinOps (*Financial Operations*) es la disciplina operativa y cultural que promueve la responsabilidad financiera compartida en el consumo de recursos computacionales en la nube.

En la ingeniería de datos, articula el balance óptimo entre latencia, rendimiento analítico y costo de infraestructura, asegurando que cada dólar invertido en cómputo distribuido y almacenamiento de petabytes genere un retorno medible y justificado de valor para la organización.

29.1 Mapeo de Servicios entre Hiperescaladores y la Llegada de Microsoft Fabric

La arquitectura de datos moderna converge en la separación estricta entre **almacenamiento de objetos inmutable** y **cómputo elástico bajo demanda**:

Componente Arquitectónico	Amazon Web Services (AWS)	Microsoft Azure / Microsoft Fabric	Google Cloud Platform (GCP)
Almacenamiento de Objetos (Data Lake)	Amazon S3	Azure ADLS Gen2 / **Fabric OneLake**	Google Cloud Storage (GCS)
Formato de Almacenamiento Predeterminado	Parquet / Iceberg / Delta	**Delta Lake (Nativo en Fabric)**	Parquet / BigLake / Iceberg
Motor de Cómputo Spark Distribuido	Amazon EMR / Databricks	Azure Databricks / **Fabric Synapse**	Google Cloud Dataproc
Almacén Analítico (Data Warehouse)	Redshift (Serverless / Prov.)	Azure Synapse / **Fabric Warehouse**	Google BigQuery
Plataforma de Streaming	Amazon MSK (Kafka) / Kinesis	Azure Event Hubs / Kafka	Google Cloud Pub/Sub
Orquestador Administrado	Amazon MWAA (Airflow)	Azure Data Factory / Fabric Data Factory	Google Cloud Composer (Airflow)
Gobernanza y Metadatos	AWS Glue / Lake Formation	**Microsoft Purview** / Unity Catalog	Google Dataplex

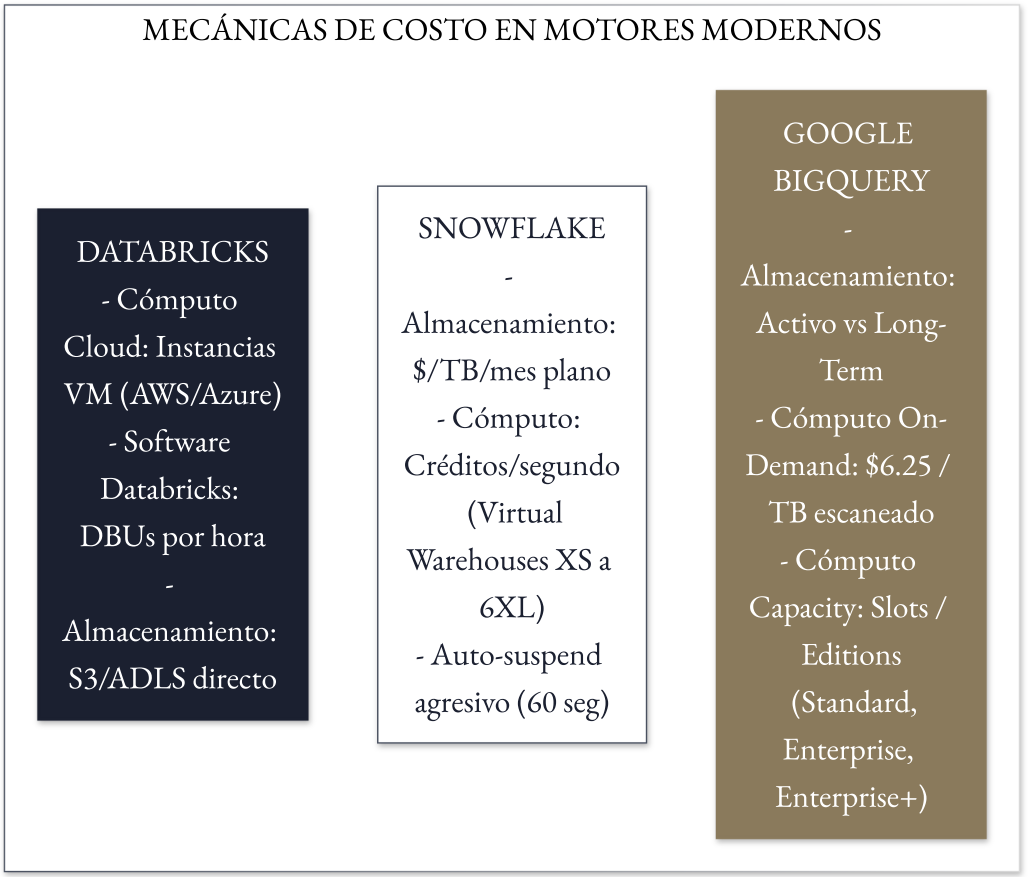
Microsoft Fabric y el Concepto de OneLake

En el ecosistema de Azure, **Microsoft Fabric** representa la unificación *SaaS* de todas las cargas de trabajo analíticas sobre un único lago lógico denominado **OneLake** (*"el OneDrive para los datos"*). Utiliza Delta Lake sobre Parquet como formato de almacenamiento universal nativo, permitiendo

que múltiples motores (T-SQL, Spark, KQL, Power BI Direct Lake) consulten las mismas tablas sin duplicación de archivos (*Zero-Copy Architecture*).

29.2 Modelos de Tarificación y Mecánica Financiera de Motores Analíticos

Comprender la fórmula exacta mediante la cual cada motor factura el consumo es indispensable para evitar sorpresas presupuestarias:



1. Snowflake: Desacoplamiento por Créditos y Micro-particionamiento

- **Almacenamiento:** Tarifa mensual fija por terabyte (~$23 USD/TB/mes en almacenamiento comprimido). Snowflake organiza los datos internamente en **micro-particiones** inmutables y comprimidas (50 a 500 MB de datos sin comprimir).

- **Clustering y Clustering Keys:** Snowflake no utiliza índices tradicionales B-Tree ni particionamiento físico manual por directorios. En su lugar, utiliza metadatos de rango (mínimos y máximos por columna en cada micro-partición) para podar datos (*partition pruning*). Cuando una tabla supera los terabytes de volumen y las consultas filtran recurrentemente por dimensiones de alta cardinalidad, se define una **Clustering Key**:

```SQL
-- Definición de clave de agrupamiento para optimizar micro-particiones en Snowflake
ALTER TABLE analitica.fact_transacciones CLUSTER BY (fecha_transaccion, pais_id);
```

  Snowflake cuenta con un servicio de **Automatic Clustering** en segundo plano que reordena las micro-particiones para minimizar el solapamiento de rangos, consumiendo créditos de cómputo dedicados.

- **Vistas Materializadas (Materialized Views):** Almacenan físicamente el resultado precomputado de una consulta. Snowflake mantiene automáticamente sincronizada la vista ante inserciones y cambios en la tabla base mediante su servicio de mantenimiento serverless:

```SQL
CREATE OR REPLACE MATERIALIZED VIEW analitica.mv_ventas_diarias AS
SELECT
    fecha_transaccion,
    pais_id,
    COUNT(transaccion_id) AS total_operaciones,
    SUM(monto_usd) AS volumen_total_usd
FROM analitica.fact_transacciones
GROUP BY fecha_transaccion, pais_id;
```

- **Cómputo:** Se factura mediante **Créditos Snowflake por segundo** de ejecución de los *Virtual Warehouses*:
  - Talla XS: 1 crédito/hora.
  - Talla S: 2 créditos/hora.
  - Talla M: 4 créditos/hora (y así sucesivamente duplicándose hasta 6XL).
- **Palanca FinOps:** Configurar `AUTO_SUSPEND = 60` (suspensión automática tras 60 segundos de inactividad) y `AUTO_RESUME = TRUE`. Si ningún usuario o reporte está consultando el almacén, el costo de cómputo cae a cero de inmediato.

2. Google BigQuery: Facturación por Escaneo vs. Slots Dedicados

- **Modelo Bajo Demanda (*On-Demand*):** BigQuery factura típicamente **$6.25 USD por cada Terabyte de datos escaneados** por la consulta.

- *Riesgo Financiero:* Una consulta analítica mal optimizada del tipo `SELECT * FROM tabla_eventos` que lea una tabla de 80 TB costará **$500 USD en una sola ejecución de 15 segundos**.
- *Particionamiento y Clustering:* BigQuery permite particionar por fecha, marca de tiempo (`TIMESTAMP`/`DATETIME`) o rango entero (hasta un límite de 10.000 particiones por tabla), y hasta 4 columnas de agrupamiento (*clustering*):

```SQL
-- Creación de tabla particionada y clusterizada en Google BigQuery
CREATE OR REPLACE TABLE `proyecto_prod.analitica.eventos_telemetria`
(
    evento_id STRING NOT NULL,
    usuario_id INT64,
    tipo_evento STRING,
    monto_usd NUMERIC,
    fecha_hora_evento TIMESTAMP
)
PARTITION BY DATE(fecha_hora_evento)
CLUSTER BY tipo_evento, usuario_id
OPTIONS(
    partition_expiration_days = 730,
      description = "Tabla de telemetría optimizada con partición diaria y
clustering"
);
```

- *Vistas Materializadas:* BigQuery optimiza automáticamente las consultas hacia tablas base redirigiéndolas a la vista materializada (*Smart Tuning*) si detecta que la vista cubre la consulta:

```SQL
CREATE MATERIALIZED VIEW `proyecto_prod.analitica.mv_resumen_eventos`
PARTITION BY fecha
CLUSTER BY tipo_evento
AS
SELECT
    DATE(fecha_hora_evento) AS fecha,
    tipo_evento,
    COUNT(1) AS cantidad_eventos,
    SUM(monto_usd) AS suma_usd
FROM `proyecto_prod.analitica.eventos_telemetria`
GROUP BY 1, 2;
```

- **Modelo por Capacidad (*Editions / Reservations*):** Para empresas con alto volumen predecible, se contratan **Slots** dedicados por hora/mes (ediciones Standard, Enterprise, Enterprise Plus), desacoplando el costo del volumen de bytes escaneados.

3. Amazon Redshift: Arquitectura MPP Clásica y Distribución Física

Amazon Redshift es un motor MPP (*Massively Parallel Processing*) basado en PostgreSQL optimizado para almacenamiento columnar:

- **Estilos de Distribución de Datos (Distribution Styles):** Determinan cómo se distribuyen los bloques de datos entre los nodos de cómputo (*Compute Slices*):
  - `AUTO`: Redshift asigna `ALL` inicialmente a tablas pequeñas y migra automáticamente a `EVEN` a medida que la tabla crece.
  - `KEY`: Las filas se asignan a nodos según el hash de una columna. Crucial para unir grandes tablas de hechos con dimensiones sin provocar redistribución por red (*Broadcast/Hash Shuffle*):
  - `ALL`: Copia la tabla completa en cada uno de los nodos. Ideal para dimensiones pequeñas (menos de unos pocos millones de filas).
  - `EVEN`: Distribución equitativa mediante *Round-Robin*. Usado cuando no hay una clave de unión evidente.
- **Claves de Ordenamiento (Sort Keys):** Determinan el orden físico de los bloques en disco, permitiendo al motor saltar bloques de 1 MB enteros con base en metadatos de zona (*Zone Maps*):
  - `COMPOUND SORTKEY`: Clave compuesta jerárquica (óptima cuando las consultas filtran por el primer prefijo de la clave).
  - `INTERLEAVED SORTKEY`: Pesa igual todas las columnas de la clave, útil cuando las consultas filtran por combinaciones arbitrarias de columnas.

SQL

```sql
-- Creación de tabla en Amazon Redshift con Distribution Key y Compound Sort Key
CREATE TABLE analitica.fact_ventas_redshift
(
    venta_id BIGINT IDENTITY(1,1),
    cliente_id INT NOT NULL,
    tienda_id INT NOT NULL,
    fecha_venta DATE NOT NULL,
    monto_total NUMERIC(12,2)
)
DISTSTYLE KEY
DISTKEY (cliente_id)
COMPOUND SORTKEY (fecha_venta, tienda_id);
```

4. Databricks: Fórmula Híbrida de Dos Capas

El costo total por hora de un clúster Databricks se compone de:

$$\text{Costo Total} = \text{Costo de Instancias Cloud (VMs AWS/Azure)} + (\text{DBUs consumid}$$

- **Job Clusters vs. All-Purpose Clusters:** Un clúster interactivo (*All-Purpose*) cuesta aproximadamente **$0.40 - $0.55 USD por DBU**. Un clúster de trabajo automatizado (*Job Cluster*, creado exclusivamente para ejecutar un pipeline y apagarse) cuesta entre **$0.15 y $0.20 USD por DBU**. Ejecutar pipelines de producción programados en clústeres All-Purpose representa un sobrecosto directo del 60% al 70%.

29.3 Palancas FinOps Clave para Reducción de Costos

1. Nodos Worker en Instancias Spot / Preemptible

Los nodos Worker de un clúster de Spark procesan tareas idempotentes y sin estado persistente (*stateless*). En caso de que el proveedor cloud reclame una máquina Spot, el Driver simplemente reasigna las tareas pendientes a otros Workers.

- **Ahorro:** Las instancias Spot ofrecen descuentos de entre el **60% y el 80%** frente al precio On-Demand.
- **Regla de Oro:** El **nodo Driver debe permanecer estrictamente en On-Demand**. Si la máquina del Driver es reclamada por la nube, toda la aplicación Spark colapsa inmediatamente.

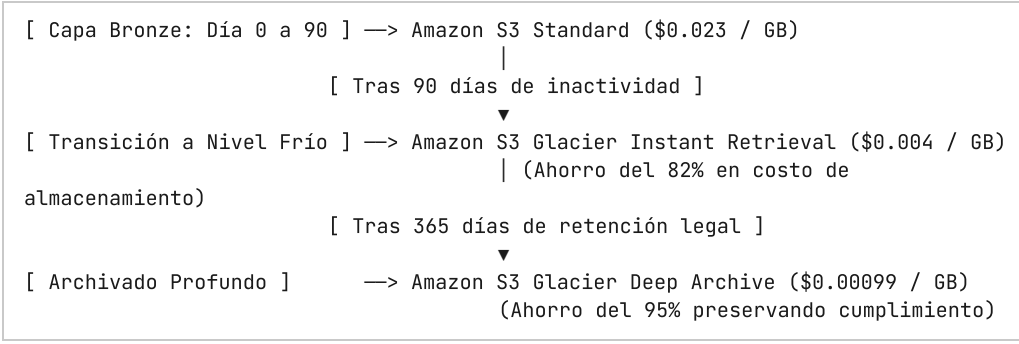
2. Supresión de Costos de Red (*Data Egress*)

Mover datos a través de los límites de una región en AWS o Azure genera tarifas de transferencia de salida (*Egress*) que van desde $0.02 hasta $0.09 USD por GB.

- Si el bucket de almacenamiento está en `us-east-1` y el clúster de Spark se despliega en `us-west-2`, cada terabyte leído y escrito incurrirá en costos de transferencia de red recurrentes y latencia innecesaria.
- **Regla de Oro:** Todos los componentes del pipeline (fuentes, motores de cómputo, sumideros y orquestadores) deben residir estrictamente en la misma región cloud y, si es posible, dentro de la misma zona de disponibilidad.

3. Ciclo de Vida de Almacenamiento (*Storage Tiering*) y el Peligro de Glacier en Lakehouses

El 90% de las consultas analíticas consultan datos generados en los últimos 30 a 90 días. Mantener terabytes de datos históricos de la capa Bronze en almacenamiento caliente (*Hot Tier*) desperdicia presupuesto:



PELIGRO CRÍTICO DE ARQUITECTURA: Reglas de Ciclo de Vida en Tablas Delta Lake e Iceberg

Aplicar reglas automáticas de ciclo de vida de S3 (o Blob Lifecycle Management en Azure) hacia niveles de almacenamiento asíncrono (**Amazon S3 Glacier Flexible** o **Glacier Deep Archive**) sobre directorios que albergan tablas **Delta Lake** o **Apache Iceberg** es un **error catastrófico en producción**.

1. **Fallo Inmediato de Consultas (`InvalidObjectState`):** Los motores Spark, Trino y DuckDB esperan acceso síncrono por HTTP `GET` a todos los archivos Parquet referenciados en el registro de transacciones (*Delta Log* o *Manifest Files*). Si una regla de S3 envió archivos de datos a Glacier Flexible, cualquier consulta que requiera leer dichos archivos fallará instantáneamente con la excepción `InvalidObjectState: The operation is not valid for the object's storage class`.
2. **Ruptura de Comandos de Mantenimiento (`VACUUM`):** El comando `VACUUM` inspecciona el almacenamiento físico para determinar qué archivos están huérfanos. Al toparse con objetos en Glacier, el comando colapsa o no puede comprobar sus metadatos.
3. **Costos Ocultos y Latencia de Restauración:** Restaurar millones de archivos Parquet archivados en Glacier requiere emitir peticiones de restauración que tardan entre 3 y 12 horas por lote y conllevan cobros por cada 1.000 solicitudes ($0.03 a $0.05 USD por millar), eliminando cualquier supuesto ahorro en almacenamiento.

Práctica Recomendada: En tablas Lakehouse, el ciclo de vida debe gestionarse a nivel lógico de partición (por ejemplo, archivando particiones enteras en un bucket secundario desacoplado del catálogo) o utilizando únicamente **S3 Glacier Instant Retrieval**, asegurándose de que la tabla esté compactada previamente, dado que Glacier penaliza severamente archivos menores a 128 KB.

29.4 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Mantener Clústeres Fijos Encendidos 24/7 para Trabajos por Lotes Diarios Dejar un clúster de 16 nodos encendido continuamente para procesar una carga que solo tarda 2 horas durante la madrugada significa pagar por 22 horas de máquinas completamente ociosas. Se deben utilizar siempre clústeres efímeros creados y destruidos por el orquestador (*Ephemeral Job Clusters*).

Anti-patrón 2: Consultar Motores Serverless con `SELECT *` y Formatos de Texto Plano (CSV/JSON) Consultar archivos CSV sin comprimir mediante Amazon Athena o BigQuery fuerza el escaneo completo de cada carácter del archivo. Almacenar en Parquet con compresión Snappy/ZSTD reduce el volumen de bytes leídos hasta en un 85%, reduciendo directamente la factura en la misma proporción.

Anti-patrón 3: Políticas de Retención Indefinidas en la Capa Bronze sin Reglas de Purgado Conservar todos los archivos crudos de telemetría de forma indefinida en el nivel de almacenamiento más costoso infla linealmente el costo mensual de la cuenta de almacenamiento año tras año.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 29.1 (Cálculo Financiero Comparativo en BigQuery):
Una compañía almacena una tabla analítica de registros de navegación web de **50 Terabytes**. Un analista programa un dashboard en Power BI que ejecuta una consulta cada 15 minutos (96 veces al día).

- *Consulta A (Incipiente):* `SELECT * FROM tabla_logs WHERE user_agent LIKE '%Android%'` (Sin particionar, escanea la tabla completa).
- *Consulta B (Optimizada):* La tabla se particiona por fecha (`fecha_evento`) y se aplica clusterización por `user_agent`. La consulta optimizada solo escanea **200 Gigabytes** por ejecución. Calcula el costo diario y mensual de ambas consultas bajo el modelo bajo demanda de BigQuery ($6.25 USD por TB escaneado).

Solución 29.1:

1. **Cálculo para Consulta A (No optimizada):**
   - Bytes por ejecución: $50\ \text{TB}$.
   - Costo por ejecución: $50\ \text{TB} \times \$6.25 = \$312.50\ \text{USD}$.
   - Costo diario (96 ejecuciones): $96 \times \$312.50 = \$30.000\ \text{USD/día}$.

- **Costo mensual (30 días):** $\$30.000 \times 30 = \mathbf{\$900.000}$ USD/mes *(Gasto catastrófico).*

2. **Cálculo para Consulta B (Optimizada con particionamiento y clúster):**
   - Bytes por ejecución: $200 \text{ GB} = 0.2 \text{ TB}$.
   - Costo por ejecución: $0.2 \text{ TB} \times \$6.25 = \$1.25$ USD.
   - Costo diario (96 ejecuciones): $96 \times \$1.25 = \$120$ USD/día.
   - **Costo mensual (30 días):** $\$120 \times 30 = \mathbf{\$3.600}$ USD/mes.

- **Conclusión:** La optimización arquitectónica produce un **ahorro de más de $896.000 USD mensuales (99.6% de reducción)** demostrando el impacto directo del diseño de ingeniería en el balance financiero.

Ejercicio 29.2 (Dimensionamiento de Clúster Spark con Instancias Spot):
Se debe procesar un trabajo batch nocturno que requiere 10 nodos de cómputo durante 4 horas.

- Instancia On-Demand: $1.20 USD / hora.
- Instancia Spot: $0.30 USD / hora (75% de descuento). Diseña la distribución recomendada entre Driver y Workers y calcula el ahorro económico total frente a una configuración 100% On-Demand.

Solución 29.2:

1. **Configuración Segura Recomendada:**
   - **1 Nodo Driver:** Asignado a **On-Demand** ($1.20/h) para garantizar que el trabajo no colapse si la nube reclama capacidad.
   - **9 Nodos Worker:** Asignados a **Spot** ($0.30/h) para máxima eficiencia.
2. **Comparativa de Costo (4 horas de ejecución):**
   - *Costo 100% On-Demand (10 nodos):*

$$\text{Costo} = 10 \times \$1.20 \times 4 = \$48.00 \text{ USD}$$

   - *Costo Híbrido Recomendado (1 On-Demand + 9 Spot):*

$$\text{Costo} = (1 \times \$1.20 \times 4) + (9 \times \$0.30 \times 4) = \$4.80 + \$10.80 = \$15.60 \text{ USD}$$

   - **Ahorro Absoluto:** $\$48.00 - \$15.60 = \$32.40$ USD (**67.5% de ahorro** con tolerancia total a fallos).

29.5 Resumen y Puntos Clave

- **Desacoplamiento fundamental:** La nube analítica moderna separa el almacenamiento de objetos masivo (S3, ADLS Gen2, OneLake) del cómputo elástico efímero.
- **Mecánica financiera de motores:** Comprender las fórmulas de créditos (Snowflake), escaneo de bytes (BigQuery) y DBUs con instancias cloud (Databricks) es imperativo para evitar desbordamientos presupuestarios.
- **Clústeres efímeros y Workers Spot:** El uso de Job Clusters con trabajadores en instancias Spot reduce el gasto de cómputo en más de un 60% sin comprometer la estabilidad del nodo Driver.
- **Ciclo de vida de almacenamiento:** La transición automática de datos fríos hacia niveles de archivo (*Glacier / Cool Tier*) reduce drásticamente el costo de retención de petabytes a largo plazo.

Lecturas Recomendadas

1. **Storment, J. R. & Fuller, M. (2023).** *Cloud FinOps: Collaborative, Real-Time Cloud Value Decision Making (2nd Edition).* O'Reilly Media.
2. **Google Cloud Architecture Center (2024).** *BigQuery Cost Optimization Best Practices: Storage and Query Partitioning.* Google Documentation.
3. **Snowflake Computing (2024).** *Managing Costs in Snowflake: Resource Monitors, Warehouse Scaling, and Auto-Suspend.* Snowflake Documentation.

CAPÍTULO 29A

Microsoft Fabric y OneLake: Arquitectura SaaS y el Futuro Analítico en Azure RECOMENDADO

> *"La evolución de las plataformas de datos en la nube ha transitado desde la infraestructura como servicio (IaaS) y los servicios administrados fragmentados (PaaS) hacia la consolidación del Software como Servicio (SaaS). En el ecosistema de Microsoft Azure, Microsoft Fabric representa la convergencia total: un único lago lógico de datos —OneLake— gobernado por el formato abierto Delta Lake, donde múltiples motores computacionales (Spark, T-SQL, KQL y Power BI) interoperan sobre los mismos archivos físicos sin duplicar un solo byte."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

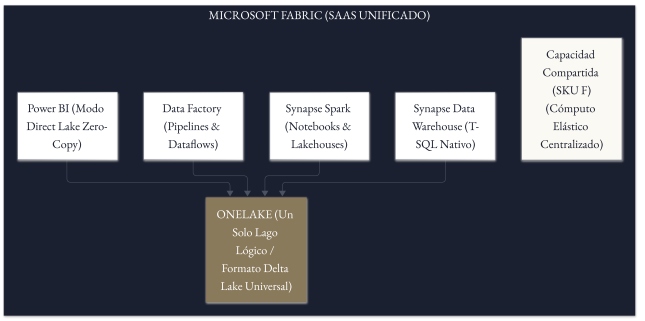
- Comprender la transición arquitectónica desde servicios PaaS fragmentados (Azure Synapse Analytics, Azure Data Factory tradicional) hacia la plataforma unificada SaaS **Microsoft Fabric**.
- Analizar la estructura interna de **OneLake** y su adopción del estándar **Delta Lake sobre Parquet** como formato de almacenamiento universal.
- Implementar **Accesos Directos (*Shortcuts*) Zero-Copy** para virtualizar datos residentes en Amazon S3, Google Cloud Storage o ADLS Gen2 sin mover archivos.
- Distinguir y orquestar las diferentes cargas de trabajo: **Data Engineering** (Apache Spark), **Data Warehouse** (T-SQL relacional) y **Real-Time Intelligence** (KQL/Eventstream).
- Explicar el funcionamiento revolucionario del modo **Direct Lake** de Power BI frente a los modos tradicionales *Import* y *DirectQuery*.
- Preparar el mapa de competencias técnicas requerido para la certificación oficial **Microsoft Certified: Fabric Data Engineer Associate (Examen DP-700)**.

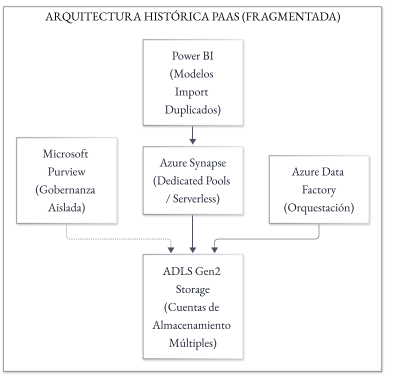
Definición 29A.1 (Microsoft Fabric y OneLake) Microsoft Fabric es una plataforma analítica integral todo-en-uno entregada bajo el modelo SaaS, que unifica el movimiento de datos, el procesamiento distribuido, la ingeniería analítica, el almacenamiento masivo, la inteligencia en tiempo real y la visualización empresarial.

En su núcleo reside **OneLake** (*"el OneDrive para los datos analíticos de la empresa"*): un lago lógico único por cada organización (*tenant*), construido automáticamente sobre Azure Data Lake Storage (ADLS) Gen2, que erradica los silos departamentales forzando a todos los motores a consultar y escribir tablas en formato **Delta Lake**.

29A.1 De la Fragmentación PaaS a la Unificación SaaS

Históricamente, construir una arquitectura analítica moderna en Azure requería aprovisionar y entrelazar múltiples servicios independientes:





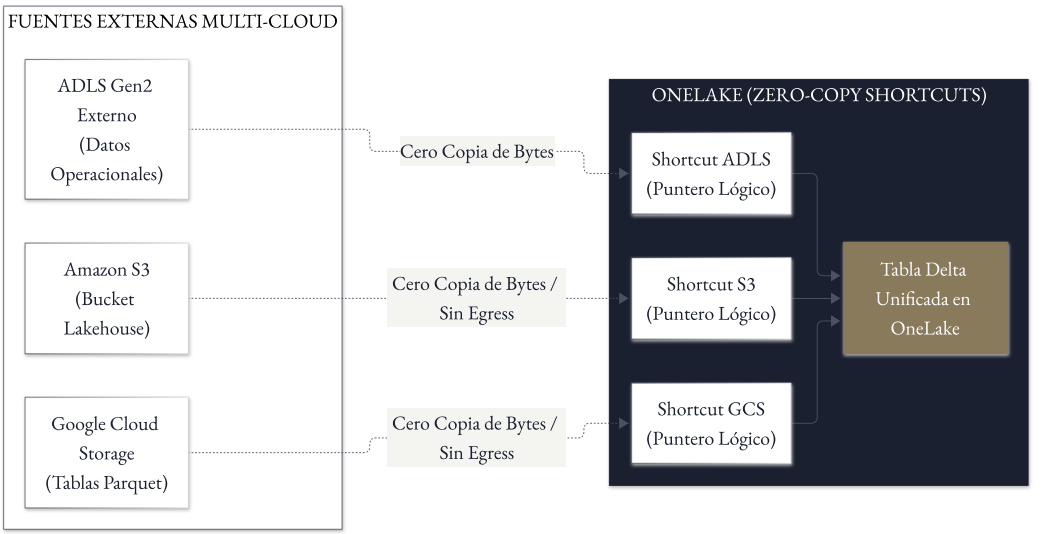
Problemas Críticos del Enfoque PaaS Tradicional:

1. **Silos y Duplicación Masiva de Datos:** Finanzas creaba su propio Data Lake; Marketing exportaba datos hacia Synapse; y Power BI importaba copias en memoria RAM (*archivos PBIX*), multiplicando los costos de almacenamiento por cuatro.
2. **Complejidad de Red y Permisos:** Conectar un clúster de Spark a un Data Factory requería configurar Private Endpoints, identidades administradas, redes virtuales emparejadas (*VNet Peering*) y reglas complejas de cortafuegos.
3. **Facturación Fragmentada:** Pagar por vCores de Synapse, unidades de integración de datos (DIUs) en ADF y nodos dedicados de Power BI Premium generaba gastos impredecibles.

La Solución de Fabric: Se contrata una **Capacidad Fabric compartida (SKUs F2 a F2048)**. Toda la organización consume CPU y memoria de esa bolsa común de recursos, y todos los equipos leen y escriben sobre el mismo OneLake unificado.

29A.2 OneLake y la Arquitectura Zero-Copy con Shortcuts

OneLake elimina la necesidad de mover datos entre departamentos o incluso entre diferentes nubes públicas:



Shortcuts (Accesos Directos Lógicos)

Un **Shortcut** es un puntero o enlace simbólico dentro de OneLake que apunta hacia ubicaciones de almacenamiento físicas externas (como un bucket de Amazon S3 o una cuenta externa de ADLS Gen2):

- **Cero Duplicación (*Zero-Copy*):** Los archivos permanecen en el almacenamiento de origen en AWS o GCP. Los motores de Fabric los consultan directamente a través del puntero de OneLake.
- **Interoperabilidad Automática:** Si los datos en Amazon S3 están en formato Delta Lake o Parquet, OneLake los expone instantáneamente como una tabla analítica consultable por T-SQL o Power BI sin necesidad de ejecutar un pipeline de ingesta ETL.

29A.3 Cargas de Trabajo y Motores Analíticos en Fabric

Dentro de un área de trabajo (*Workspace*) de Fabric, los ingenieros pueden combinar diferentes paradigmas computacionales sobre las mismas tablas Delta:

1. Data Engineering con Apache Spark

Provee clústeres de Apache Spark completamente administrados con arranque rápido (menos de 20 segundos) y entornos de ejecución optimizados:

```python
# Código PySpark en Notebook de Fabric escribiendo en Delta Lake nativo
from pyspark.sql import functions as F

df_ventas = spark.read.parquet("Files/raw/ventas_2026.parquet")

df_limpio = df_ventas \
    .filter(F.col("monto_usd") > 0) \
    .withColumn("fecha_proceso", F.current_date())

# Escritura directa en el catálogo gestionado de OneLake
df_limpio.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("ventas_lakehouse.fact_ventas_conformadas")
```

2. Synapse Data Warehouse (Motor T-SQL Relacional)

A diferencia de los Data Warehouses tradicionales que utilizan formatos de almacenamiento binarios propietarios, el Data Warehouse de Fabric **escribe físicamente en archivos Delta Lake abiertos en OneLake**:

```sql
-- Consulta T-SQL ejecutada en el Data Warehouse de Fabric
-- Lee directamente los archivos Delta producidos por el notebook de Spark
SELECT
    pais,
    COUNT(transaccion_id) AS total_operaciones,
    SUM(monto_usd) AS facturacion_total
FROM ventas_lakehouse.fact_ventas_conformadas
WHERE fecha_proceso >= '2026-01-01'
GROUP BY pais
ORDER BY facturacion_total DESC;
```

29A.4 Power BI Direct Lake: La Revolución del Consumo Analítico

Históricamente, los analistas de negocio se enfrentaban a una disyuntiva en Power BI:

- **Modo Import:** Copiaba terabytes de datos desde la base de datos hacia la memoria RAM de Power BI (*VertiPaq*). Ofrecía consultas rápidas, pero obligaba a programar recargas periódicas

y duplicaba masivamente los datos.

- **Modo DirectQuery:** No copiaba datos, pero traducía cada clic en el dashboard a consultas SQL lentas hacia la base de datos, colapsando el rendimiento.



- **Modo Direct Lake:** Como las tablas de OneLake ya están almacenadas en formato columnar **Parquet** con compresión Snappy/ZSTD y diccionario, el motor VertiPaq de Power BI **mapea y lee las columnas directamente desde los archivos Parquet en disco a su memoria**.
- **Resultado:** Ofrece la misma velocidad instantánea del modo *Import* sin mover ni duplicar los datos, reflejando de forma inmediata cualquier actualización realizada por un pipeline de Spark o SQL.

29A.5 El Examen Oficial DP-700: Microsoft Fabric Data Engineer

Para los ingenieros de datos enfocados en el ecosistema de Microsoft, la certificación **Microsoft Certified: Fabric Data Engineer Associate (Examen DP-700)** valida las competencias técnicas del rol:

Dominio Técnico	Peso Relativo	Temas Centrales Evaluados
Implementar y Administrar una Solución Analítica	**30–35%**	Configuración de capacidades Fabric (SKUs F), seguridad y RBAC en OneLake, administración de workspaces, gobierno con Purview y despliegues con Git y deployment pipelines.
Ingestar y Cargar Datos	**25–30%**	Ingesta con Data Factory Pipelines y Dataflows Gen2, configuración de Shortcuts hacia AWS S3 y ADLS, ingesta continua con streaming y CDC.
Transformar y Optimizar Datos	**30–35%**	Transformación distribuida con Apache Spark y Delta Lake, optimización con Z-Order y V-Order, modelado dimensional en Data Warehouses (T-SQL) y diseño de semantic models en Direct Lake.
Monitorear y Asegurar la Plataforma	**10–15%**	Monitoreo del consumo de capacidad con la aplicación *Fabric Capacity Metrics*, seguridad a nivel de filas y columnas (RLS/CLS) y alertas operacionales.

29A.6 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Sobrecargar la Capacidad Compartida sin Monitorear los Límites de CPU (*Throttling*)
Al operar bajo una capacidad compartida (ej. SKU F64), si un ingeniero ejecuta un notebook con un bucle infinito o un join ineficiente, puede agotar las unidades de capacidad (*Capacity Units - CUs*), provocando que Microsoft limite (*throttle*) o pause temporalmente los reportes de Power BI y los pipelines de toda la empresa.
Corrección: Supervise activamente la aplicación *Fabric Capacity Metrics* y configure políticas de límites de tiempo de ejecución para notebooks interactivos.

Anti-patrón 2: Forzar el Uso de DirectQuery Habiendo Tablas Delta en OneLake
Conectar Power BI a tablas de Fabric utilizando conectores relacionales estándar con DirectQuery degrada el rendimiento de los reportes.
Corrección: Utilice siempre modelos semánticos en modo **Direct Lake**, asegurándose de que las tablas Delta subyacentes estén compactadas y utilicen la optimización de lectura rápida **V-Order**.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 29A.1 (Configuración de Arquitectura Multi-Cloud con Shortcuts):
Una empresa multinacional almacena su catálogo histórico de telemetría de 200 Terabytes en un bucket de Amazon S3 en formato Parquet (`s3://empresa-global-telemetria/historico/`). La junta directiva solicita que el equipo analítico en Azure construya dashboards en Power BI con latencia de consulta de menos de 2 segundos sin duplicar los 200 TB en Azure ni incurrir en costos recurrentes de transferencia de datos (*Egress*). Diseñe la solución técnica utilizando Microsoft Fabric.

Solución 29A.1:

1. **Creación del Shortcut en OneLake:**
   En el Lakehouse de Microsoft Fabric, se crea un **OneLake Shortcut** apuntando al bucket `s3://empresa-global-telemetria/historico/`, autenticado de forma segura mediante un rol IAM de AWS con credenciales de confianza.
2. **Exposición de Metadatos:**
   OneLake crea un puntero lógico que expone los archivos Parquet como una tabla analítica dentro del catálogo de Fabric sin transferir físicamente los 200 TB a través de internet.
3. **Capa Semántica en Direct Lake:**
   Se crea un modelo semántico en Power BI configurado en modo **Direct Lake** vinculado a la tabla del Shortcut.
4. **Resultado de Rendimiento y Costo:**
   Power BI lee las columnas requeridas bajo demanda directamente sobre el almacenamiento columnar sin duplicación física en Azure, cumpliendo con el SLA de 2 segundos y ahorrando decenas de miles de dólares en almacenamiento y ancho de banda.

29A.7 Resumen y Puntos Clave

- **Convergencia SaaS:** Microsoft Fabric elimina la fricción de aprovisionar infraestructura PaaS fragmentada, unificando cómputo elástico compartido y almacenamiento en un solo servicio.
- **OneLake y Delta Lake:** Un lago lógico único a nivel corporativo donde todos los motores (Spark, SQL, KQL) leen y escriben sobre el estándar abierto Delta Lake.
- **Shortcuts Zero-Copy:** Conectan almacenes de datos en AWS S3, Google Cloud Storage y ADLS Gen2 sin duplicar datos ni pagar costos de migración.
- **Direct Lake en Power BI:** Supera el dilema tradicional entre velocidad (Import) y frescura (DirectQuery), leyendo directamente desde los archivos Parquet en disco.

- **Certificación DP-700:** Valida formalmente el diseño, ingesta, optimización y gobierno de plataformas Lakehouse en Microsoft Fabric.

Lecturas Recomendadas

1. **Microsoft Learn (2024).** *Microsoft Fabric Documentation: Architecture, Workspaces, and OneLake Storage.* Microsoft Docs.
2. **Microsoft Credentials (2024).** *Study Guide for Exam DP-700: Implementing an Enterprise Data Solution Using Microsoft Fabric.* Microsoft Learn.
3. **Ferrari, A. & Russo, M. (2023).** *Deep Dive into Direct Lake Mode in Microsoft Fabric and Power BI.* SQLBI Research (`https://www.sqlbi.com/articles/deep-dive-into-direct-lake-mode/`).

CAPÍTULO 29B

Arquitectura de Plataformas de Datos en AWS: S3 Data Lake, Glue, EMR, Athena y Redshift

RECOMENDADO

> *"Amazon Web Services fue el pionero que definió la computación en la nube pública y continúa siendo la columna vertebral de la infraestructura tecnológica global. Construir una plataforma de datos a escala de petabytes en AWS exige dominar el desacoplamiento estricto entre almacenamiento inmutable en Amazon S3, catálogos de metadatos federados con AWS Glue, cómputo elástico en EMR, análisis interactivo serverless con Athena y almacenamiento masivo en Amazon Redshift."*

Objetivos de Aprendizaje

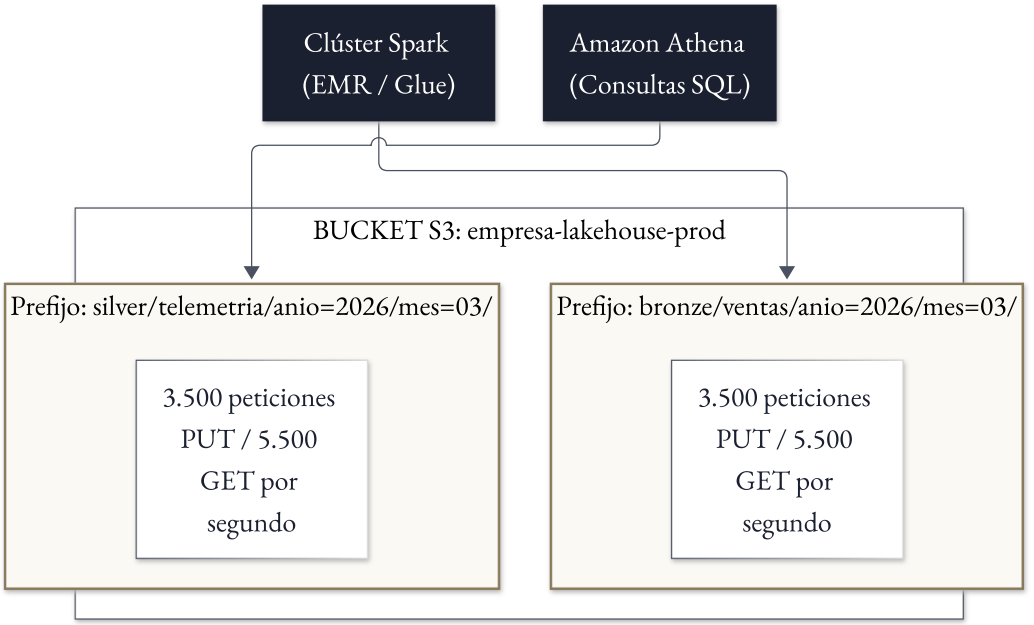
Al finalizar este capítulo, el lector será capaz de:

- Diseñar un **Data Lakehouse de alto rendimiento sobre Amazon S3**, optimizando la estructura de prefijos para maximizar el throughput de operaciones de lectura y escritura.
- Implementar y optimizar el catálogo de metadatos universal mediante **AWS Glue Data Catalog**, evaluando las compensaciones de rendimiento de los *Glue Crawlers*.
- Construir pipelines de procesamiento distribuido serverless con **AWS Glue Jobs** y **Amazon EMR** (Serverless, on EC2 con instancias Spot, y on EKS).
- Optimizar consultas analíticas interactivas en **Amazon Athena**, minimizando los costos de escaneo por terabyte mediante particionado y formatos columnares.
- Diseñar esquemas relacionales MPP en **Amazon Redshift**, seleccionando estilos de distribución física (*Distribution Styles*) y claves de ordenamiento (*Sort Keys*) sobre nodos RA3.
- Ingestar flujos de eventos continuos en tiempo real utilizando **Amazon Kinesis Data Streams** y **Amazon MSK** (Apache Kafka gestionado).
- Blindar la seguridad de la plataforma mediante **AWS Lake Formation** y **VPC Endpoints** privados para evitar cargos de transferencia de red.

Definición 29B.1 (El Data Lakehouse Nativo en AWS) En el ecosistema de Amazon Web Services, un **Data Lakehouse** es una arquitectura desacoplada donde **Amazon S3** actúa como la única fuente inmutable de verdad para el almacenamiento de objetos, **AWS Glue Data Catalog** centraliza los metadatos y esquemas de forma federada, y múltiples motores computacionales especializados (Athena para SQL interactivo, EMR/Glue para Apache Spark, Redshift para analítica dimensional y SageMaker para aprendizaje automático) operan simultáneamente sobre los mismos datos sin silos redundantes.

29B.1 Amazon S3 como Núcleo de Almacenamiento del Lakehouse

Amazon S3 (*Simple Storage Service*) ofrece durabilidad del 99.999999999% (11 nueves). En diciembre de 2020, AWS introdujo **consistencia fuerte de lectura tras escritura (*Strong Read-After-Write Consistency*)** sin sobrecosto, eliminando las complejidades de consistencia eventual que afectaban históricamente a los motores analíticos.



1. Throughput y Particionamiento por Prefijos

Amazon S3 escala el ancho de banda automáticamente por cada prefijo de clave:

- **Capacidad por Prefijo:** Cada prefijo independiente en un bucket soporta al menos **3.500 solicitudes** `PUT/COPY/POST/DELETE` y **5.500 solicitudes** `GET/HEAD` **por segundo**.
- **Diseño de Prefijos:** Un prefijo es cualquier cadena entre la raíz del bucket y el nombre del archivo (ej. `s3://bucket/capa/tabla/particion/`). Al particionar tablas por fechas o hashes, los ejecutores de Spark distribuyen las lecturas y escrituras a través de múltiples prefijos simultáneos, alcanzando cientos de miles de peticiones por segundo sin cuellos de botella de I/O.

2. Clases de Almacenamiento y FinOps en S3

- **S3 Standard:** Para datos de la capa Silver y Gold consultados frecuentemente ($0.023 USD/GB/mes).
- **S3 Intelligent-Tiering:** Monitorea los patrones de acceso y mueve automáticamente los objetos entre niveles caliente, tibio y frío sin penalizaciones de recuperación. Es la clase recomendada para la capa Bronze.
- **Advertencia Crítica:** Como se fundamentó en el Capítulo 29, **nunca aplique reglas de ciclo de vida hacia S3 Glacier Flexible o Deep Archive sobre tablas Delta Lake o Apache Iceberg activas**, ya que romperá inmediatamente las consultas y los comandos de mantenimiento (`VACUUM`) con excepciones `InvalidObjectState`.

29B.2 AWS Glue: Catálogo de Datos y Cómputo Serverless

AWS Glue proporciona tanto la capa de catálogo como el entorno de ejecución Spark serverless en AWS:

1. AWS Glue Data Catalog

Actúa como un **Hive Metastore universal administrado**:

- Permite que Amazon Athena, Amazon EMR, Amazon Redshift Spectrum y Amazon SageMaker compartan las mismas definiciones de tablas, particiones y tipos de datos.
- Almacena estadísticas de columnas (mínimos, máximos, valores nulos) utilizadas por los optimizadores de costos de los motores de consulta.

2. Glue Crawlers vs. Registro Basado en Eventos

- **Glue Crawlers:** Escanean periódicamente los prefijos de S3, infieren formatos y actualizan las particiones en el catálogo.
  - *Riesgo en Producción:* Escanear terabytes de datos con millones de archivos genera altos costos de DPU y puede alterar tipos de datos si un archivo nuevo contiene anomalías.

- *Práctica Recomendada en Producción:* Utilizar **notificaciones de eventos de S3 (*S3 Event Notifications*)** conectadas a una función AWS Lambda que registre las nuevas particiones en el Glue Data Catalog mediante la llamada de API `CreateTable` o `BatchCreatePartition`, con costo casi nulo y latencia de segundos.

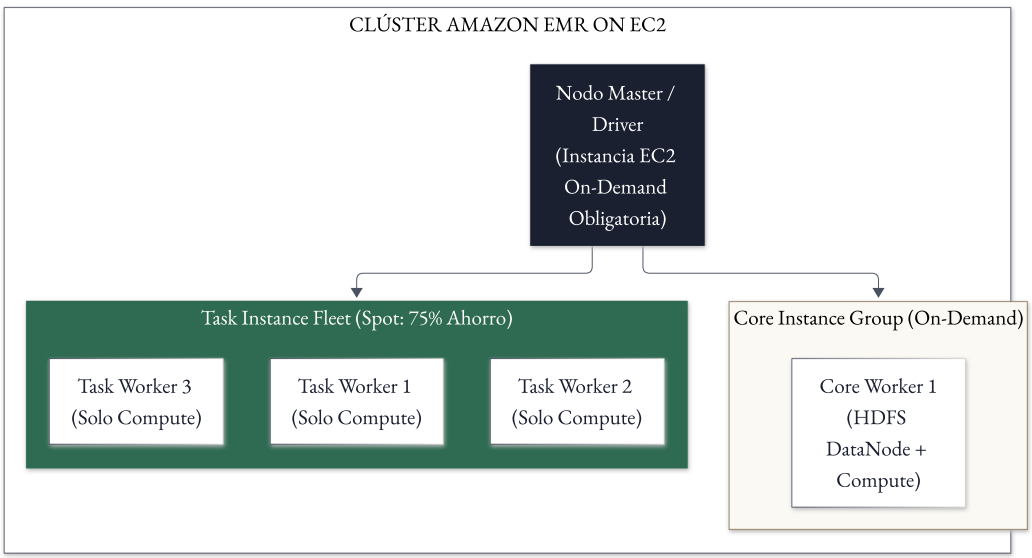
3. AWS Glue Jobs (Apache Spark Serverless)

Ejecuta scripts de PySpark o Scala sin necesidad de administrar instancias EC2 ni clústeres:

- Se dimensiona en **DPUs (*Data Processing Units*)**: 1 DPU equivale a 4 vCPUs y 16 GB de RAM.
- Tipos de trabajadores: `G.1X` (1 DPU), `G.2X` (2 DPU, óptimo para transformaciones con shuffle pesado y ML) y `G.025X` (para streaming ligero).
- **`DynamicFrame` frente a PySpark `DataFrame`:** La biblioteca de Glue proporciona la abstracción `DynamicFrame`, que tolera datos con esquemas anómalos o tipos inconsistentes en una misma columna sin fallar, permitiendo resolver conflictos con métodos como `.resolveChoice()`.

29B.3 Amazon EMR: Cómputo Distribuido a Gran Escala

Cuando las cargas analíticas exigen clústeres de gran envergadura (cientos de nodos) o requieren componentes de código abierto como Apache Flink, Trino o HBase, **Amazon EMR (*Elastic MapReduce*)** es la solución de cómputo por excelencia:



- **EMR Serverless:** Permite enviar jobs de Spark o Hive sin aprovisionar clústeres, escalando la capacidad de cómputo automáticamente de 0 a miles de vCPUs en segundos.
- **Estrategia FinOps en EMR on EC2:**
  - **Nodo Master (Driver):** Debe aprovisionarse estrictamente en **instancias On-Demand**. Si la nube reclama el nodo Master, la aplicación Spark colapsa por completo.
  - **Nodos Core:** Mantienen servicios de almacenamiento local; se recomiendan instancias On-Demand en número reducido.
  - **Nodos Task (*Task Instance Fleets*):** Ejecutan únicamente tareas computacionales de Spark sin almacenar estado crítico de HDFS. Deben configurarse **100% en instancias Spot**, reduciendo la factura de cómputo entre un **60% y un 80%** con tolerancia total a fallos.

29B.4 Amazon Athena: Consultas SQL Serverless sobre el Data Lake

Amazon Athena ejecuta consultas SQL estándar sobre Amazon S3 mediante el motor distribuido Trino/Presto sin requerir servidores:

- **Modelo Financiero:** Athena factura **$5.00 USD por cada Terabyte de datos escaneados**.
- **Reglas de Oro de Optimización en Athena:**
  1. *Convertir a Parquet con compresión Snappy o ZSTD:* Reduce el volumen de bytes escaneados en disco entre un 70% y un 85%.
  2. *Particionar físicamente en S3:* Usar particionado Hive (`anio=2026/mes=03/`) permite a Athena podar particiones completas (*Partition Pruning*).
  3. *Proyectar únicamente columnas requeridas:* Al ser Parquet un formato columnar, `SELECT cliente_id, monto` lee solo los bloques de esas dos columnas, mientras que `SELECT *` lee el archivo completo.

29B.5 Amazon Redshift: Arquitectura MPP y Almacenamiento Desacoplado

Amazon Redshift es el almacén de datos (*Data Warehouse*) relacional a escala de petabytes en AWS:

1. Arquitectura de Nodos RA3 y Redshift Managed Storage (RMS)

Los clústeres Redshift modernos utilizan instancias **RA3**, las cuales desacoplan por completo el cómputo de las máquinas del almacenamiento:

- El almacenamiento se delega a **Redshift Managed Storage (RMS)**, respaldado por SSDs de alta velocidad como caché local y Amazon S3 como almacenamiento inmutable masivo.
- Permite escalar el tamaño del clúster de cómputo de forma elástica sin tener que migrar ni reequilibrar petabytes de datos en disco.

2. Estilos de Distribución Física (*Distribution Styles*)

Determinan cómo se reparten las tuplas entre los nodos de cómputo (*Slices*):

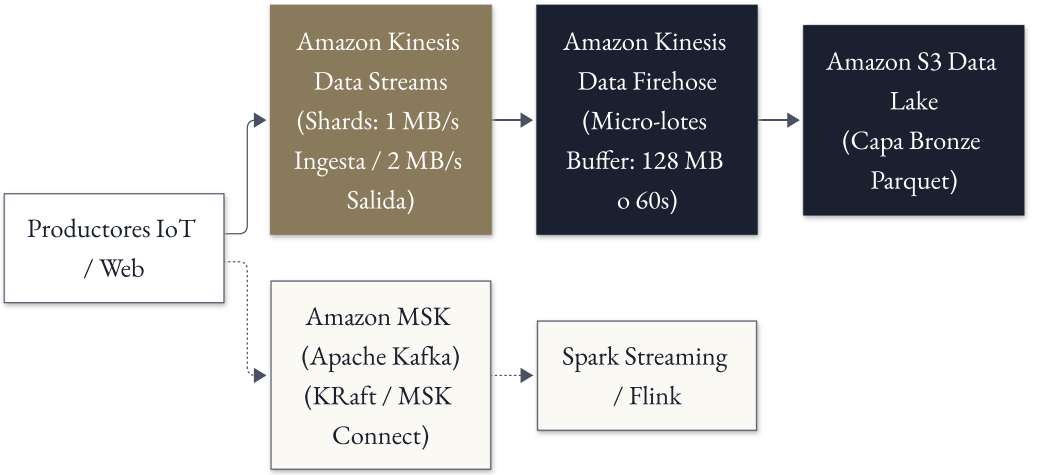
- `DISTSTYLE KEY`: Distribuye las tuplas según el valor hash de una columna clave (ej. `cliente_id`). Crucial para unir grandes tablas de hechos con dimensiones sin provocar redistribución de datos por la red (*Colocated Joins*).
- `DISTSTYLE ALL`: Duplica la tabla completa en el primer slice de cada nodo. Óptimo para dimensiones pequeñas (menos de 3 millones de filas).
- `DISTSTYLE EVEN`: Distribuye mediante *Round-Robin* equitativo cuando no existe una clave de unión evidente.

3. Redshift Spectrum

Permite que consultas SQL ejecutadas en Redshift lean y crucen tablas locales con petabytes de archivos Parquet/Delta almacenados directamente en Amazon S3 sin cargarlos en el clúster (*External Tables*).

29B.6 Ingesta en Streaming con Kinesis y Amazon MSK

Para canalizaciones de eventos en tiempo real, AWS ofrece dos alternativas principales:



- **Amazon Kinesis Data Streams:** Almacén de streaming basado en fragmentos (*Shards*). Cada fragmento garantiza 1 MB/segundo (o 1.000 registros/segundo) de escritura y 2 MB/segundo de lectura.
- **Amazon Kinesis Data Firehose:** Es un servicio totalmente serverless que ingesta eventos de streaming, los agrupa en buffers temporales (ej. cada 60 segundos o 128 MB), los convierte automáticamente a formato Parquet mediante esquemas de Glue y los deposita directamente en S3 sin escribir una sola línea de código de consumidor.
- **Amazon MSK (*Managed Streaming for Apache Kafka*):** Clústeres de Apache Kafka administrados con soporte para almacenamiento escalable, autenticación IAM y migración moderna a KRaft sin Zookeeper.

29B.7 Seguridad y Redes Privadas con VPC Endpoints y Lake Formation

En una arquitectura de datos corporativa segura en AWS:

1. **AWS Lake Formation:** Reemplaza las complejas políticas IAM directas en S3 por un modelo de gobernanza centralizado. Permite otorgar permisos de consulta granulares a nivel de **tabla, columna e incluso filtros de fila** (*Row-Level Filtering*) sobre el catálogo de Glue.
2. **VPC Endpoints (Gateway Endpoints para S3):**
    - **Problema:** Si los ejecutores de Spark en una subred privada se comunican con S3 a través de un *NAT Gateway*, cada gigabyte transferido paga tarifas de procesamiento de red de AWS ($0.045 USD/GB).

- **Solución:** Configurar un **VPC Gateway Endpoint para Amazon S3** en la tabla de rutas. El tráfico entre los nodos de cómputo y el bucket viaja de forma segura por la red troncal interna de AWS **con costo de transferencia cero ($0.00 USD)** y latencia reducida.

29B.8 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Programar Glue Crawlers Diarios sobre Tablas Particionadas de Petabytes
Ejecutar un Crawler para que recorra millones de archivos existentes en un bucket cada noche gasta cientos de dólares en DPUs y puede introducir inconsistencias si detecta archivos temporales.
Corrección: Registre las nuevas particiones programáticamente desde el pipeline de Spark o mediante eventos de S3 y Lambda.

Anti-patrón 2: Ejecutar Consultas `SELECT *` Frecuentes en Amazon Athena
Consultar tablas en Athena sin especificar columnas o sin filtrar por la partición de fecha (`fecha = '2026-03-31'`) fuerza el escaneo de terabytes innecesarios, inflando la factura en cientos de dólares al día.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 29B.1 (Cálculo de Shards en Amazon Kinesis Data Streams):
Una plataforma de comercio electrónico en AWS genera un pico de **12.000 transacciones por segundo** durante el Black Friday. Cada mensaje tiene un tamaño promedio de **1.5 Kilobytes**.

1. Calcule el ancho de banda total de entrada en Megabytes por segundo.
2. Determine el número mínimo de *Shards* necesarios en el stream de Kinesis para soportar la carga sin rechazos por cuota (*ProvisionedThroughputExceededException*).

Solución 29B.1:

1. **Cálculo de Throughput de Entrada:**

$$\text{Throughput} = 12.000 \text{ registros/s} \times 1.5 \text{ KB} = 18.000 \text{ KB/s} = 17.58 \text{ MB/s}$$

2. **Capacidad de un Shard de Kinesis:**
   - Límite de ancho de banda: 1 MB/s por shard.
   - Límite de registros: $1.000 \text{ registros/s}$ por shard.
3. **Cálculo de Shards requeridos:**

- Por volumen de datos: $\lceil 17.58/1 \rceil = 18$ **shards**.
- Por volumen de registros: $\lceil 12.000/1.000 \rceil = 12$ **shards**.

- **Resultado:** Se requiere aprovisionar un mínimo de **18 Shards** (gobernado por la restricción de ancho de banda en MB/s) para procesar el tráfico pico sin degradación.

29B.9 Resumen y Puntos Clave

- **Amazon S3 es la base:** Provee almacenamiento inmutable masivo con consistencia fuerte, escalando throughput horizontalmente por cada prefijo independiente.
- **AWS Glue:** Centraliza metadatos mediante el Glue Data Catalog y provee cómputo Spark serverless dimensionado en DPUs.
- **FinOps en EMR:** El uso de nodos Worker en instancias Spot con Driver On-Demand reduce el costo de cómputo en más de un 60%.
- **Consultas en Athena:** El modelo serverless ($5/TB) exige compresión columnar Parquet y poda estricta de particiones.
- **Redshift RA3:** Desacopla cómputo de almacenamiento mediante RMS y optimiza cruces con claves de distribución física (`DISTSTYLE KEY`).
- **VPC Endpoints:** Eliminan los costos de transferencia de red hacia S3 y blindan el tráfico dentro del perímetro privado de AWS.

Lecturas Recomendadas

1. **Amazon Web Services (2024).** *Big Data Analytics Options on AWS: Architectural Whitepaper.* AWS Architecture Center.
2. **Reis, J. & Housley, M. (2022).** *Fundamentals of Data Engineering (Cloud Ecosystems: AWS).* O'Reilly Media.
3. **AWS Documentation (2024).** *Amazon EMR Best Practices: Spot Instance Resiliency and S3 Optimization.*

CAPÍTULO 29C

Arquitectura de Plataformas de Datos en Google Cloud (GCP): BigQuery, Dataflow, Pub/Sub y Dataproc

RECOMENDADO

> *"Google Cloud Platform nació de la necesidad de procesar los datos de toda la web indexada. Sus contribuciones —desde MapReduce y Bigtable hasta Dremel, MillWheel y FlumeJava— definieron el estado del arte de la computación distribuida global. Dominar GCP exige comprender la separación radical entre almacenamiento y cómputo que hace posible BigQuery, el modelo unificado de streaming y batch de Apache Beam en Dataflow, la mensajería elástica de Pub/Sub y el tejido de gobernanza federada de Dataplex."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

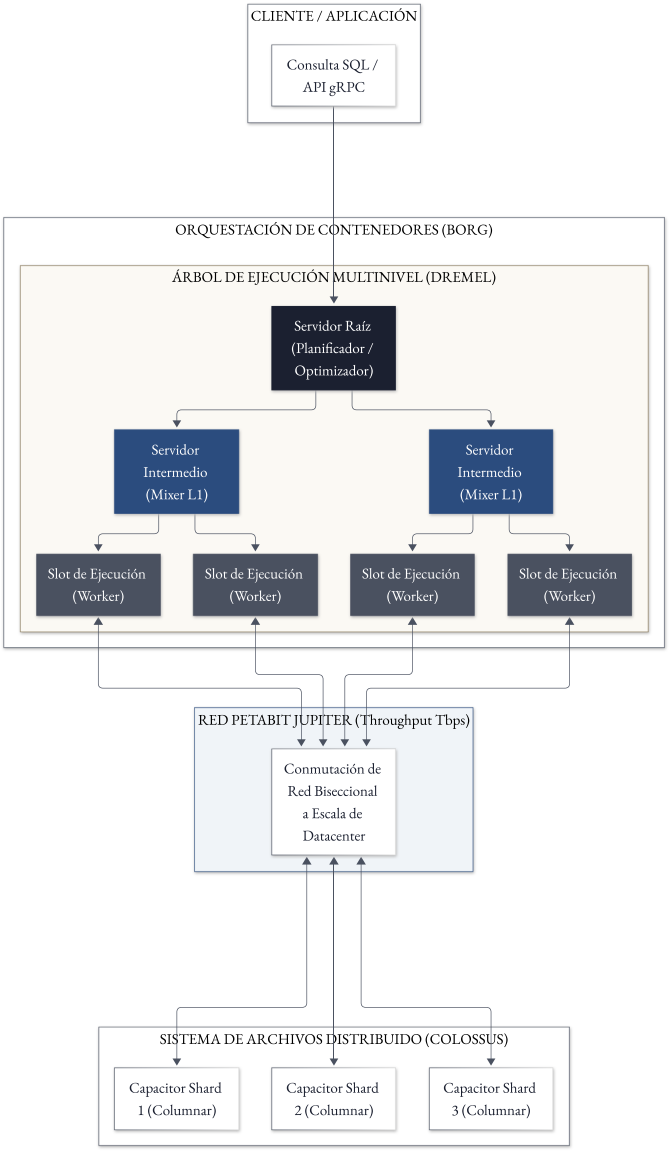
- Analizar los componentes arquitectónicos internos de **Google BigQuery** (Dremel, Colossus, Jupiter y Borg) y su impacto en el rendimiento de consultas a escala de petabytes.
- Optimizar la distribución física de tablas en BigQuery mediante **particionamiento por tiempo/rango** y **agrupamiento (*clustering*) multinivel**, minimizando los bytes escaneados y los costos operativos.
- Implementar ingesta de alto rendimiento con **BigQuery Storage Write API** y extracción masiva vectorizada con **BigQuery Storage Read API** y Apache Arrow.
- Diseñar pipelines unificados de batch y streaming en **Google Cloud Dataflow** utilizando el modelo formal de **Apache Beam** (ventanas de tiempo de evento, marcas de agua y disparadores).
- Orquestar mensajería distribuida asíncrona a nivel global con **Cloud Pub/Sub**, aplicando claves de ordenamiento (*ordering keys*) y colas de mensajes fallidos (*Dead-Letter Topics*).
- Desplegar cargas de trabajo de Apache Spark administrado con **Cloud Dataproc** (usando instancias *Spot* y clústeres efímeros) y **Dataproc Serverless**.
- Implementar gobernanza y linaje automatizado sobre el Data Lakehouse de GCP utilizando **Dataplex** y **Cloud Composer**.

Definición 29C.1 (El Data Cloud Nativo en GCP) En Google Cloud Platform, la plataforma de datos opera bajo un modelo de desacoplamiento absoluto de tres capas:

1. **Almacenamiento de Objetos y Tablas:** Google Cloud Storage (GCS) y el sistema de almacenamiento columnar *Capacitor* en *Colossus*.
2. **Red de Comunicación Petabit:** La topología de red *Jupiter*, que conecta miles de ranuras de procesamiento (*slots*) al almacenamiento compartido con ancho de banda biseccional prácticamente ilimitado.
3. **Motores de Ejecución Serverless:** BigQuery para SQL analítico e interactivo, Dataflow (Apache Beam) para procesamiento unificado orientado a eventos, y Dataproc Serverless para transformaciones masivas en Apache Spark.

29C.1 Arquitectura Interna de Google BigQuery: Dremel, Colossus y Jupiter

A diferencia de los almacenes de datos tradicionales o de clústeres MPP que mantienen discos locales en cada nodo (como AWS Redshift con nodos densos clásicos), BigQuery implementa una arquitectura completamente desagregada:



1. Los Cuatro Pilares Fundamentales

1. **Dremel:** El motor de ejecución de consultas distribuido introducido por Google en 2010. Dremel compila las consultas SQL en un árbol de ejecución jerárquico (*Execution Tree*). El servidor raíz coordina la consulta, los servidores intermedios (*Mixers*) agregan resultados parciales y miles de hojas computacionales (*Slots*) leen y procesan datos en paralelo.
2. **Slots de BigQuery:** Un *slot* es la unidad virtual de cómputo en BigQuery (aproximadamente equivalente a 0.5 vCPUs y memoria asociada). BigQuery asigna dinámicamente cientos o miles de slots por consulta en milisegundos.
3. **Colossus y Capacitor:** Los datos no residen en los slots. Residen en **Colossus** (el sucesor de Google File System - GFS), replicados en múltiples zonas de disponibilidad con codificación de borrado (*erasure coding*). El formato de almacenamiento interno es **Capacitor**, un formato columnar altamente optimizado que precede a Apache Parquet y almacena estructuras anidadas y repetidas sin necesidad de aplanar tablas.
4. **Red Jupiter:** Permite que los slots lean datos desde los discos de Colossus a velocidades de petabits por segundo. Esto elimina la necesidad de localidad de datos (*data locality*) y hace posible que miles de núcleos procesen terabytes de datos sin transferencias de disco lentas.

29C.2 Optimización Física: Particionamiento y Clustering

En BigQuery, el rendimiento y el costo financiero de las consultas están directamente vinculados al volumen de bytes escaneados desde Colossus.

1. Particionamiento (Partitioning)

Divide una tabla física en segmentos basados en:

- **Tiempo de Ingesta:** Mediante la pseudocolumna `_PARTITIONTIME` o `_PARTITIONDATE`.
- **Columna de Fecha o Marca de Tiempo:** Una columna `DATE`, `DATETIME` o `TIMESTAMP` explícita en la tabla (ej. `fecha_venta`).
- **Rango de Enteros:** Por ejemplo, `id_cliente` particionado con inicio, fin e intervalo.

> **Regla de Producción:** BigQuery admite un límite máximo de **10.000 particiones por tabla**. Intentar particionar por una marca de tiempo con granularidad de hora a lo largo de varios años provocará un error inmediato de cuota.

2. Agrupamiento (Clustering)

Dentro de cada partición (o en una tabla sin particionar), el clustering ordena los datos físicamente por hasta **cuatro columnas** en orden jerárquico.

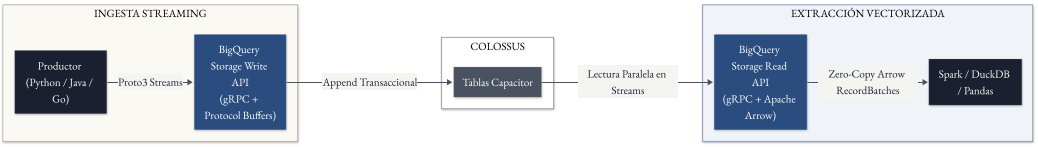
- A diferencia del particionamiento, el agrupamiento es ideal para **alta cardinalidad** (identificadores de usuario, códigos postales, estados de orden).
- BigQuery ejecuta **eliminación de bloques (*Block Pruning*)**: cuando una consulta incluye filtros con `WHERE` en las columnas clusterizadas, BigQuery omite la lectura de bloques completos de Capacitor, reduciendo drásticamente los bytes leídos y el tiempo de respuesta.
- **Auto-Reclustering:** BigQuery reordena los datos en segundo plano de manera autónoma sin costo computacional adicional para el usuario.

```SQL
-- Creación de tabla optimizada con particionamiento diario y clustering jerárquico
CREATE OR REPLACE TABLE `proyecto_prod.gold.fact_transacciones`
(
    id_transaccion STRING NOT NULL,
    id_cliente INT64 NOT NULL,
    codigo_pais STRING NOT NULL,
    monto_usd NUMERIC(15, 2) NOT NULL,
    fecha_transaccion DATE NOT NULL,
    timestamp_creacion TIMESTAMP NOT NULL
)
PARTITION BY fecha_transaccion
CLUSTER BY codigo_pais, id_cliente
OPTIONS (
    description = "Tabla de hechos de transacciones globales optimizada para poda de bloques",
    require_partition_filter = TRUE
);
```

> **Buenas Prácticas:** La opción `require_partition_filter = TRUE` es fundamental en entornos corporativos. Evita que consultas analíticas ad-hoc o herramientas de BI ejecuten escaneos accidentales de toda la tabla (*Full Table Scan*), lo que podría costar miles de dólares en una sola ejecución bajo el modelo de facturación On-Demand.

29C.3 Ingesta y Extracción de Alto Rendimiento: Storage APIs

Históricamente, la ingesta en streaming hacia BigQuery utilizaba la API REST `tabledata.insertAll`, la cual presentaba latencias elevadas, carecía de semántica de escritura transaccional estricta y conllevaba costos adicionales por megabyte. En la actualidad, el estándar de ingeniería es la **BigQuery Storage API**:



1. BigQuery Storage Write API

- Utiliza conexiones bidireccionales **gRPC** con serialización binaria mediante **Protocol Buffers (Proto3)**.
- Proporciona **semántica exactamente una vez (*Exactly-Once Semantics*)** mediante offsets de flujo (*stream offsets*). Si la red se interrumpe y el cliente reintenta enviar el mismo registro con el mismo offset, BigQuery descarta el duplicado a nivel del motor.
- Admite flujos comprometidos (*Committed Streams*) para streaming continuo de baja latencia o flujos pendientes (*Pending Streams*) para transacciones multi-lote que se consolidan atómicamente con un *Commit*.

2. BigQuery Storage Read API

- Permite leer datos directamente desde Colossus en formato **Apache Arrow**, evitando la sobrecarga de serialización JSON y pasando por alto el motor SQL de Dremel.
- Divide la tabla en múltiples flujos de lectura paralelos que pueden ser consumidos concurrentemente por ejecutores de Spark, DuckDB o scripts de Python sin degradar el rendimiento del clúster.

PYTHON

```python
# Ejemplo de lectura masiva vectorizada con Storage Read API y Apache Arrow
from google.cloud import bigquery_storage
import pyarrow as pa

client = bigquery_storage.BigQueryReadClient()

# Definir la sesión de lectura sobre una tabla particionada
table = "projects/empresa-datos-prod/datasets/gold/tables/fact_transacciones"
read_options = bigquery_storage.types.ReadSession.TableReadOptions(
    selected_fields=["id_transaccion", "id_cliente", "monto_usd"],
    row_restriction="fecha_transaccion >= '2026-03-01' AND codigo_pais = 'CO'"
)

session = client.create_read_session(
    parent="projects/empresa-datos-prod",
    read_session=bigquery_storage.types.ReadSession(
        table=table,
        data_format=bigquery_storage.types.DataFormat.ARROW,
        read_options=read_options
    ),
    max_stream_count=4  # Dividir la lectura en 4 flujos paralelos
)
```

```
print(f"Sesión creada con {len(session.streams)} streams paralelos.")

# Consumir el primer stream en formato PyArrow Table (Zero-Copy)
stream_reader = client.read_rows(session.streams[0].name)
pages = stream_reader.rows(session)
tabla_arrow = pages.to_arrow()
print(f"Filas leídas en stream 0: {tabla_arrow.num_rows}")
```

29C.4 BigLake y Formatos Abiertos de Lakehouse

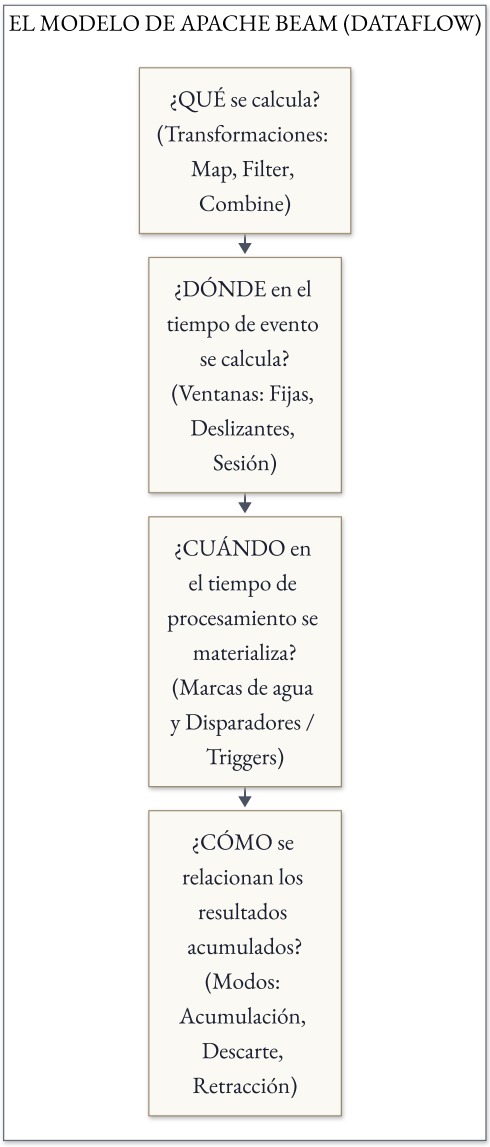
Para evitar la duplicación de datos entre almacenes relacionales y lagos de datos basados en objetos, Google Cloud ofrece **BigQuery BigLake**:

- **Abstracción Unificada:** Permite consultar tablas en formatos abiertos (**Apache Iceberg, Delta Lake y Apache Parquet**) almacenadas en Google Cloud Storage, AWS S3 o Microsoft Azure ADLS Gen2 mediante tablas externas federadas.
- **Seguridad y Control de Acceso Fino:** Aplica seguridad a nivel de filas, enmascaramiento dinámico de columnas y políticas de control de acceso (*IAM*) sin otorgar permisos directos sobre los buckets de almacenamiento a los analistas.
- **Caché de Metadatos:** Almacena en caché las estadísticas y la lista de archivos de Iceberg/Parquet en BigQuery, acelerando los tiempos de planificación de consultas en más de un 80%.

29C.5 Google Cloud Dataflow y el Modelo Formal de Apache Beam

Google Cloud Dataflow es el servicio serverless administrado para la ejecución de pipelines de procesamiento distribuido definidos mediante el SDK de **Apache Beam**.

El modelo de Apache Beam (formalizado por Tyler Akidau et al. en 2015) unifica el procesamiento de datos por lotes (*Batch*) y en tiempo real (*Streaming*) alrededor de cuatro interrogantes fundamentales:



1. Conceptos Fundamentales de Beam

- **`PCollection`:** Representa un conjunto de datos distribuido e inmutable. Puede ser acotado (*Bounded*, batch) o desacotado (*Unbounded*, streaming).
- **`PTransform`:** Una operación que toma una o más `PCollection` y produce nuevas `PCollection` (ej. `beam.Map`, `beam.Filter`, `beam.ParDo`, `beam.GroupByKey`).
- **`DoFn`:** La clase base donde el ingeniero implementa lógica personalizada elemento a elemento mediante el método `process()`.
- **Tiempo de Evento (*Event Time*) vs. Tiempo de Procesamiento (*Processing Time*):** El tiempo de evento es el momento en que ocurrió la acción en el mundo real (registrado en el

evento). El tiempo de procesamiento es el reloj de la máquina que procesa el evento en Dataflow.

2. Ventanas y Marcas de Agua (Watermarks)

- **Ventana Fija (*Fixed Window*):** Intervalos de tiempo uniformes y disjuntos (ej. bloques de 5 minutos).
- **Ventana Deslizante (*Sliding Window*):** Intervalos continuos superpuestos (ej. una ventana de 1 hora calculada cada 10 minutos).
- **Ventana de Sesión (*Session Window*):** Agrupa eventos separados por períodos de inactividad específicos (ej. una sesión de navegación de usuario que expira tras 30 minutos sin clics).
- **Marca de Agua (*Watermark*):** Una noción temporal que estima hasta qué punto se han recibido los eventos de un determinado tiempo de evento. Cuando el watermark sobrepasa el final de una ventana, el motor asume que no llegarán más datos para dicha ventana y emite los resultados.

3. Pipeline Productivo de Streaming en Apache Beam (Python)

A continuación se presenta un pipeline completo de Dataflow que lee eventos desde Cloud Pub/Sub, extrae marcas de tiempo de evento, agrupa en ventanas fijas de 2 minutos, tolera datos tardíos (*late data*) durante 1 minuto adicional y escribe los resultados agregados en BigQuery:

PYTHON

```python
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
import json

class ParseAndExtractTimestampDoFn(beam.DoFn):
    """ParDo para deserializar JSON y adjuntar Event Time explícito."""
    def process(self, element):
        try:
            payload = json.loads(element.decode("utf-8"))
            event_timestamp = payload["timestamp_evento"]  # Epoch en segundos
            yield beam.window.TimestampedValue(payload, event_timestamp)
        except Exception:
            # Enrutamiento hacia Dead Letter Topic en producción
            pass

class CalcularMetricasDoFn(beam.DoFn):
    """Calcula agregaciones sobre la ventana temporal consolidada."""
    def process(self, element, window=beam.DoFn.WindowParam):
        codigo_pais, transacciones = element
        total_monto = sum(t["monto"] for t in transacciones)
        conteo = len(transacciones)

        yield {
            "pais": codigo_pais,
            "total_monto": float(total_monto),
```

```
            "cantidad_transacciones": conteo,
            "ventana_inicio": window.start.to_utc_datetime().isoformat(),
            "ventana_fin": window.end.to_utc_datetime().isoformat()
        }

def run_pipeline():
    options = PipelineOptions()
    options.view_as(StandardOptions).streaming = True

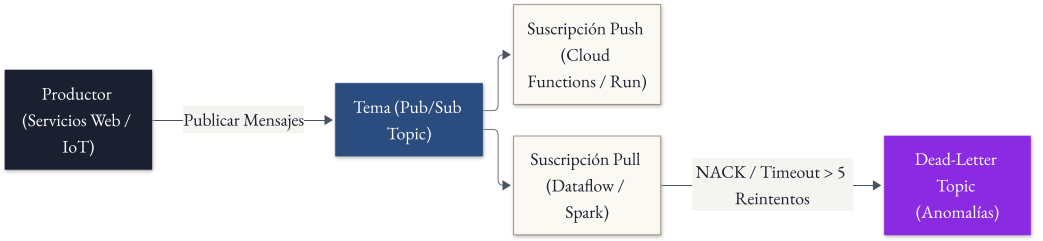
    with beam.Pipeline(options=options) as p:
        (
            p
            | "LeerDePubSub" >> beam.io.ReadFromPubSub(
                subscription="projects/empresa-prod/subscriptions/sub-
transacciones"
            )
            | "ParsearYAsignarTimestamp" >>
beam.ParDo(ParseAndExtractTimestampDoFn())
            | "VentanaFija2Minutos" >> beam.WindowInto(
                beam.window.FixedWindows(120),  # 2 minutos en segundos
                allowed_lateness=60,            # Tolerar hasta 1 minuto de
retraso
                accumulation_mode=beam.trigger.AccumulationMode.ACCUMULATING
            )
            | "ClavePorPais" >> beam.Map(lambda x: (x["pais"], x))
            | "AgruparPorPais" >> beam.GroupByKey()
            | "CalcularMetricas" >> beam.ParDo(CalcularMetricasDoFn())
            | "EscribirBigQuery" >> beam.io.WriteToBigQuery(
                table="empresa-prod:gold.metricas_tiempo_real_pais",

schema="pais:STRING,total_monto:FLOAT,cantidad_transacciones:INTEGER,ventana_inic
io:TIMESTAMP,ventana_fin:TIMESTAMP",
                write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND,
                create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED
            )
        )

if __name__ == "__main__":
    run_pipeline()
```

29C.6 Cloud Pub/Sub: Mensajería Global Asíncrona

Cloud Pub/Sub proporciona desacoplamiento horizontal entre productores y consumidores con disponibilidad multi-zona nativa.



Características Clave en Arquitecturas de Datos

- **Claves de Ordenamiento (*Ordering Keys*):** Por defecto, Pub/Sub no garantiza el orden estricto de entrega. Si un conjunto de mensajes comparte la misma `ordering_key` (por ejemplo, el ID de una transacción bancaria), Pub/Sub los entrega en el orden exacto de publicación al consumidor.
- **Dead-Letter Topics (DLT):** Cuando un mensaje falla repetidamente durante el procesamiento (superando el límite configurado de reintentos o el tiempo de espera de acuse de recibo `ack_deadline`), Pub/Sub lo redirige a un tema secundario de descarte para análisis forense, impidiendo que bloquee la suscripción principal.
- **Pub/Sub Lite vs. Standard:** Pub/Sub Lite es un servicio particionado basado en capacidad fija (similar a Apache Kafka o AWS Kinesis), que reduce los costos hasta en un 80% para flujos de telemetría masiva con volumen predecible y retención por partición.

29C.7 Dataproc y Dataproc Serverless: Apache Spark en GCP

Para organizaciones con código base existente en Apache Spark, Presto o Trino, **Google Cloud Dataproc** ofrece dos modelos operativos:

1. Dataproc en Compute Engine (Clústeres Efímeros)

- Se provisiona en menos de 90 segundos.
- **Optimización de Costos:** Se recomienda configurar el nodo maestro (*Master*) con instancias estándar y los nodos de trabajo (*Workers*) como **Compute Engine Spot VMs**, reduciendo los costos computacionales hasta en un 80%.
- **Patrón de Clúster Efímero:** El clúster se crea mediante Cloud Composer o un script bash únicamente durante la ejecución del job de Spark y se destruye inmediatamente al finalizar, almacenando los datos resultantes directamente en Google Cloud Storage mediante el conector `gcs-connector`.

2. Dataproc Serverless

- Elimina por completo la necesidad de configurar nodos, versiones de sistema operativo o ajustes de tamaño de máquinas virtuales.
- El ingeniero envía directamente el script de PySpark o el artefacto JAR. Google asigna dinámicamente los recursos (*Data Compute Units - DCU*) y escala automáticamente según la carga de trabajo.

```bash
# Envío de un job de PySpark directamente a Dataproc Serverless vía gcloud CLI
gcloud dataproc batches submit pyspark gs://empresa-codigo/scripts/etl_ventas.py \
    --project=empresa-datos-prod \
    --region=us-central1 \
    --deps-bucket=gs://empresa-artefactos-temp \
    --version=2.2 \
    --properties=spark.executor.instances=10,spark.driver.memory=4g,spark.executor.memory=8g
```

29C.8 Gobernanza con Dataplex y Cloud Composer

- **Google Cloud Dataplex:** Es una malla de datos inteligente (*Data Mesh*) que centraliza la catalogación, el linaje y las reglas de calidad de datos a través de buckets de GCS y conjuntos de datos de BigQuery sin mover los datos. Permite ejecutar **DataScans** periódicos que validan reglas de integridad, nulidad y unicidad de forma nativa.
- **Cloud Composer:** Servicio totalmente administrado de **Apache Airflow** desplegado sobre Google Kubernetes Engine (GKE). Sincroniza automáticamente los DAGs almacenados en un bucket de GCS con los *Schedulers* y *Workers* de Airflow, interactuando de forma nativa con los operadores de GCP (`BigQueryInsertJobOperator`, `DataflowCreatePythonJobOperator`, `DataprocSubmitJobOperator`).

29C.9 Anti-Patrones Comunes en GCP

Anti-Patrón 1: Consultas `SELECT *` en Tablas Grandes con Facturación On-Demand

- **El Error:** Ejecutar consultas exploratorias con `SELECT *` o sin filtros sobre tablas de terabytes en proyectos con facturación por volumen escaneado ($6.25 USD por TB).

- **Impacto en Producción:** Una sola consulta sobre una tabla de 20 TB cuesta $125 USD; diez analistas ejecutando consultas similares repetidamente pueden agotar presupuestos en días.
- **Solución:** Utilizar `require_partition_filter = TRUE`, seleccionar exclusivamente las columnas necesarias y evaluar **BigQuery Editions** con reservas fijas de slots si el consumo es continuo.

Anti-Patrón 2: Uso de `tabledata.insertAll` para Streaming Masivo

- **El Error:** Utilizar la API REST legacy de streaming para ingestar millones de eventos por segundo.
- **Impacto:** Costos elevados por megabyte insertado, ausencia de garantías transaccionales exactamente una vez (*duplicates on network retry*) y latencia de inserción subóptima.
- **Solución:** Migrar a **BigQuery Storage Write API** con flujos comprometidos o por lotes vía gRPC.

Anti-Patrón 3: Particionado por Columnas de Muy Alta Cardinalidad

- **El Error:** Crear particiones en BigQuery utilizando una columna como `id_usuario` o marcas de tiempo con precisión de minutos.
- **Impacto:** Falla de creación por superación del límite estricto de 10.000 particiones por tabla.
- **Solución:** Particionar por `DATE` (granularidad diaria) y aplicar `CLUSTER BY id_usuario` para aprovechar la poda física de bloques sin generar sobrecarga de metadatos.

Resumen del Capítulo

- **Arquitectura de BigQuery:** Separa radicalmente cómputo y almacenamiento. Dremel procesa las consultas en un árbol jerárquico de slots orquestado por Borg; Colossus almacena los datos en formato columnar Capacitor; y la red Jupiter proporciona ancho de banda petabit para unir ambas capas.
- **Optimización en BigQuery:** El particionamiento divide la tabla por tiempo o rango (máximo 10.000 particiones). El agrupamiento (*clustering*) ordena físicamente los datos por hasta 4 columnas, permitiendo poda de bloques en filtros de alta cardinalidad.
- **Storage APIs:** La Storage Write API ofrece streaming de baja latencia con semántica exactamente una vez vía gRPC y Protocol Buffers. La Storage Read API permite escaneos masivos en formato Apache Arrow con cero copia.
- **Apache Beam en Dataflow:** Responde formalmente a qué se computa, dónde en el tiempo de evento, cuándo en el tiempo de procesamiento y cómo se relacionan los resultados acumulados, gestionando marcas de agua y datos tardíos de forma nativa.

- **Pub/Sub y Dataproc:** Pub/Sub resuelve la mensajería asíncrona global con claves de ordenamiento y colas de descarte. Dataproc permite ejecutar clústeres efímeros de Spark con instancias Spot y Dataproc Serverless elimina la administración de infraestructura.

Preguntas de Revisión y Ejercicios Prácticos

Pregunta 1: Análisis de Costos y Poda de Bloques en BigQuery

Una tabla de BigQuery contiene 15 TB de transacciones distribuidas en 3 años. La tabla está particionada por `fecha_transaccion` (diaria) y clusterizada por `codigo_pais` y `id_tienda`. Un analista ejecuta la siguiente consulta:

```SQL
SELECT SUM(monto)
FROM `empresa.gold.transacciones`
WHERE fecha_transaccion BETWEEN '2026-03-01' AND '2026-03-07'
  AND id_tienda = 4521;
```

1. ¿Cómo opera la poda de datos en esta consulta?
2. ¿Por qué el agrupamiento por `id_tienda` es menos eficiente en esta consulta que si se hubiera filtrado también por `codigo_pais`?

Solución Detallada:

1. **Poda de Particiones:** BigQuery utiliza la cláusula `BETWEEN '2026-03-01' AND '2026-03-07'` para podar las particiones físicas en Colossus. De los 3 años de datos (1.095 particiones), BigQuery únicamente lee 7 particiones diarias, reduciendo el escaneo inicial de 15 TB a aproximadamente 96 GB.
2. **Poda de Bloques en Clustering:** El agrupamiento se definió jerárquicamente como `CLUSTER BY codigo_pais, id_tienda`. Al igual que en un índice compuesto de base de datos relacional, los datos se ordenan primero por `codigo_pais` y luego por `id_tienda`. Como la consulta omite el filtro de `codigo_pais` y filtra directamente por `id_tienda`, el optimizador de BigQuery no puede saltar rangos contiguos con máxima eficiencia, debiendo escanear más bloques de Capacitor dentro de cada una de las 7 particiones que si se hubiera incluido el país o si `id_tienda` fuera la primera columna del clustering.

Pregunta 2: Manejo de Datos Tardíos en Apache Beam / Dataflow

Explique la diferencia entre una ventana fija de 5 minutos con `allowed_lateness = 0` y una con `allowed_lateness = 10 minutos`. ¿Qué ocurre con un evento que ocurrió a las `12:02:00` pero llega a Dataflow a las `12:09:00` debido a una desconexión de red en el cliente?

Solución Detallada:

1. **Con `allowed_lateness = 0`:** La ventana de tiempo de evento correspondiente es `[12:00:00, 12:05:00)`. Cuando el *Watermark* de Dataflow supera las `12:05:00`, la ventana se evalúa, se emite el resultado y se destruye el estado de la ventana en memoria. Si el evento llega a las `12:09:00` (tiempo de procesamiento), el evento se considera **dato tardío (*droppped late data*)** y se descarta silenciosamente.
2. **Con `allowed_lateness = 10 minutos`:** El estado de la ventana `[12:00:00, 12:05:00)` se mantiene activo en el *Streaming Engine* hasta que el Watermark alcanza las `12:15:00`. Cuando el evento tardío llega a las `12:09:00`, Dataflow reactiva la ventana, computa la actualización y emite una nueva versión corregida del resultado acumulado según el modo de acumulación configurado (`ACCUMULATING` o `RETRACTING`).

Lecturas Recomendadas

- Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., & Vassilakis, T. (2010). *Dremel: Interactive Analysis of Web-Scale Datasets*. Proceedings of the VLDB Endowment (PVLDB), 3(1-2), 330-339.
- Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., & Whittle, S. (2015). *The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing*. Proceedings of the VLDB Endowment (PVLDB), 8(12), 1792-1803.
- Akidau, T., Chernyak, S., & Lax, R. (2018). *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

CAPÍTULO 29D

Plataforma Snowflake Empresarial: Arquitectura Multi-Cluster, Snowpark, Dynamic Tables, Snowpipe y Polaris Catalog RECOMENDADO

> *"Snowflake revolucionó la industria analítica al ser el primer almacén de datos diseñado desde cero para la nube pública bajo una arquitectura de datos compartidos multi-clúster. Desacopló el cómputo del almacenamiento y automatizó la gestión de infraestructura con micro-particiones transparentes. Dominar Snowflake a nivel senior requiere trascender el SQL interactivo para dominar Snowpark, el modelado declarativo con Dynamic Tables, la ingesta de subsegundos con Snowpipe Streaming y la convergencia abierta del Lakehouse mediante Apache Iceberg y Apache Polaris Catalog."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

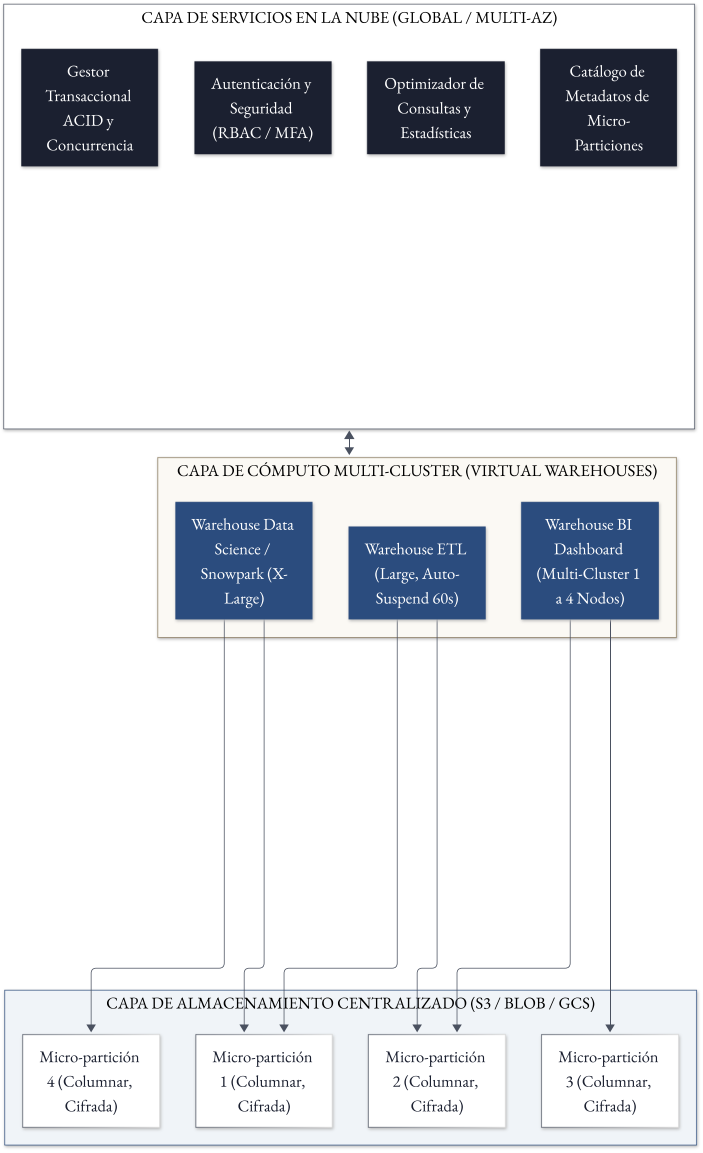
- Analizar los fundamentos de la **arquitectura multi-clúster de datos compartidos (*Multi-Cluster Shared Data*)** de Snowflake y la separación entre *Cloud Services*, *Virtual Warehouses* y almacenamiento centralizado.
- Evaluar el funcionamiento interno del **micro-particionamiento automático** y optimizar la poda de particiones mediante *Clustering Keys* y el servicio de *Automatic Clustering*.
- Implementar operaciones de recuperación histórica con **Time Travel** y clonación sin duplicación física mediante **Zero-Copy Cloning**.
- Construir pipelines de datos programáticos utilizando **Snowpark Python**, ejecutando transformaciones vectorizadas directamente en el motor de Snowflake.
- Diseñar flujos continuos de ingesta en tiempo real con **Snowpipe Streaming**, reduciendo la latencia de ingestión de minutos a milisegundos.
- Desarrollar arquitecturas de transformación declarativa utilizando **Dynamic Tables** y evaluar sus diferencias frente a la orquestación clásica de *Streams & Tasks*.
- Integrar Snowflake con el ecosistema de Lakehouse abierto mediante **tablas Apache Iceberg** y el catálogo federado **Apache Polaris Catalog**.

Definición 29D.1 (La Arquitectura Multi-Cluster Shared-Data) Snowflake implementa una arquitectura híbrida entre los sistemas de disco compartido (*Shared-Disk*) y los sistemas sin recursos compartidos (*Shared-Nothing*). Se compone de tres capas desacopladas:

1. **Capa de Almacenamiento Centralizado (Storage Layer):** Almacena todos los datos en almacenamiento de objetos (Amazon S3, Azure Blob o GCS) organizados en micro-particiones inmutables y columnarizadas.
2. **Capa de Cómputo Elástico (Multi-Cluster Compute Layer):** Clústeres de cómputo independientes (*Virtual Warehouses*) que operan bajo un modelo *Shared-Nothing* sobre los mismos datos compartidos sin contención entre consultas.
3. **Capa de Servicios en la Nube (Cloud Services Layer):** Coordina transacciones ACID, optimización de consultas, control de acceso basado en roles (RBAC) y el catálogo global de metadatos.

29D.1 Las Tres Capas Arquitectónicas de Snowflake

A diferencia de los almacenes MPP heredados donde agregar capacidad computacional obligaba a redistribuir físicamente los datos en disco, Snowflake separa completamente el cómputo del almacenamiento:



1. Virtual Warehouses y Dimensionamiento

- Cada *Virtual Warehouse* es un clúster de máquinas virtuales asignadas dinámicamente.

- Se dimensionan en tallas *T-Shirt* (`X-Small`, `Small`, `Medium`, `Large`, `X-Large`, etc.), donde cada salto de talla duplica el número de nodos y la potencia computacional (y por ende el consumo de créditos por hora: 1, 2, 4, 8, 16 créditos).
- **Multi-Cluster Warehouses:** Permiten que Snowflake agregue automáticamente clústeres adicionales en paralelo cuando el número de consultas concurrentes excede la capacidad de un solo warehouse, evitando la degradación de dashboards ejecutivos.

29D.2 Micro-Particionamiento y Poda de Consultas

En Snowflake, el usuario no define particiones manuales en el disco. Todas las tablas se dividen automáticamente en **micro-particiones**:

- Cada micro-partición contiene entre **50 MB y 500 MB** de datos no comprimidos.
- Los datos se almacenan de forma columnar y están fuertemente comprimidos.
- **Inmutabilidad:** Las micro-particiones nunca se modifican en el sitio (*In-Place*). Cualquier actualización (`UPDATE`), eliminación (`DELETE`) o inserción (`INSERT`) genera nuevas micro-particiones y marca las anteriores como históricas para el control de versiones.

```sql
-- Creación de tabla y definición de clave de agrupamiento (Clustering Key)
CREATE OR REPLACE TABLE empresa_dw.gold.fact_ordenes (
    id_orden STRING NOT NULL,
    id_cliente INT64 NOT NULL,
    pais_origen STRING NOT NULL,
    monto_total_usd NUMBER(12, 2) NOT NULL,
    fecha_orden DATE NOT NULL,
    timestamp_creacion TIMESTAMP_NTZ NOT NULL
)
CLUSTER BY (fecha_orden, pais_origen);

-- Consulta de verificación de la profundidad de clustering
SELECT SYSTEM$CLUSTERING_INFORMATION('empresa_dw.gold.fact_ordenes',
'(fecha_orden, pais_origen)');
```

Poda de Micro-Particiones (Partition Pruning)

La capa de *Cloud Services* mantiene metadatos exhaustivos de cada micro-partición:

- Valores mínimos y máximos de cada columna.
- Número de valores distintos (NDV).
- Recuento de valores nulos.

Cuando una consulta ejecuta un filtro `WHERE fecha_orden >= '2026-03-01'`, el optimizador consulta los metadatos y **omite el 95% de las micro-particiones sin leerlas de disco**.

29D.3 Time Travel y Zero-Copy Cloning

1. Snowflake Time Travel

Permite consultar datos históricos en cualquier instante dentro del período de retención (hasta 90 días en la edición Enterprise):

```SQL
-- Consultar una tabla tal como existía hace exactamente 2 horas
SELECT *
FROM empresa_dw.gold.fact_ordenes
AT (OFFSET => -60*120);

-- Consultar el estado de los datos antes de la ejecución de una consulta errónea
SELECT *
FROM empresa_dw.gold.fact_ordenes
BEFORE (STATEMENT => '01b7a2d3-0001-4f9a-0000-0002568a9b12');

-- Restaurar una tabla accidentalmente borrada con UNDROP
UNDROP TABLE empresa_dw.gold.fact_ordenes;
```

2. Zero-Copy Cloning

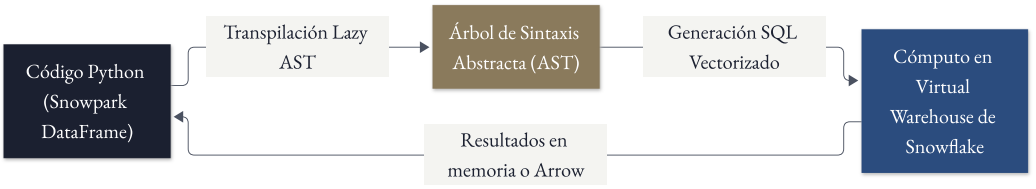
Permite clonar instantáneamente una tabla, esquema o base de datos completa de petabytes mediante la directiva `CLONE`:

- No se copian los datos en disco; solo se duplican los metadatos de las micro-particiones.
- La operación se completa en menos de un segundo y tiene costo cero de almacenamiento en el momento de creación.
- A medida que el entorno clonado sufre modificaciones, solo las nuevas micro-particiones modificadas generan costos adicionales.

```SQL
-- Clonación instantánea de una base de datos de producción hacia un entorno de staging
CREATE DATABASE empresa_dw_staging CLONE empresa_dw;
```

29D.4 Snowpark Python: Computación Distribuida Nativa

Snowpark proporciona una API de programación en Python, Scala y Java que traslada la comodidad de bibliotecas como Pandas y PySpark directamente al motor elástico de Snowflake:



- **Evaluación Perezosa (*Lazy Evaluation*):** Las operaciones sobre DataFrames de Snowpark no mueven datos a la máquina local del cliente; se traducen a SQL optimizado y se ejecutan vectorialmente en los nodos del Virtual Warehouse.
- **Integración con Anaconda:** Permite ejecutar paquetes como `numpy`, `scipy` y `scikit-learn` en funciones definidas por el usuario (UDFs) y procedimientos almacenados sin necesidad de gestionar entornos virtuales en los nodos.

PYTHON

```python
# Pipeline de enriquecimiento con Snowpark Python
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, when, sum as _sum

def procesar_metricas_ventas(session: Session):
    df_ordenes = session.table("empresa_dw.gold.fact_ordenes")

    df_metricas = (
        df_ordenes
        .filter(col("fecha_orden") >= "2026-01-01")
        .group_by("pais_origen")
        .agg(_sum("monto_total_usd").alias("total_usd"))
        .with_column(
            "categoria_mercado",
            when(col("total_usd") > 1_000_000, "Mercado Tier 1")
            .otherwise("Mercado Tier 2")
        )
    )

    # Escribir de vuelta a una tabla gestionada con sobreescritura

df_metricas.write.mode("overwrite").save_as_table("empresa_dw.gold.metricas_pais_
2026")
```

29D.5 Ingesta Continua: Snowpipe y Snowpipe Streaming

Característica	Snowpipe Clásico	Snowpipe Streaming
Mecanismo de Ingesta	Basado en archivos escenificados en Stage (S3 / GCS / Azure)	API SDK gRPC que envía filas de datos en memoria
Latencia Típica	1 a 5 minutos	**200 a 1.000 milisegundos (Subsegundo)**
Costo de Ingesta	Cargo por archivo notificado + cómputo serverless	Cargo directo por gigabyte transferido en memoria
Garantías de Entrega	Consistencia eventual basada en eventos de archivos	**Exactly-Once Semantics** mediante control de offsets

Snowpipe Streaming permite a conectores de Kafka, Flink y microservicios escribir flujos de eventos directamente en tablas de Snowflake sin la sobrecarga de escribir y consolidar micro-archivos intermedios en S3.

29D.6 Transformación Declarativa: Dynamic Tables vs. Streams & Tasks

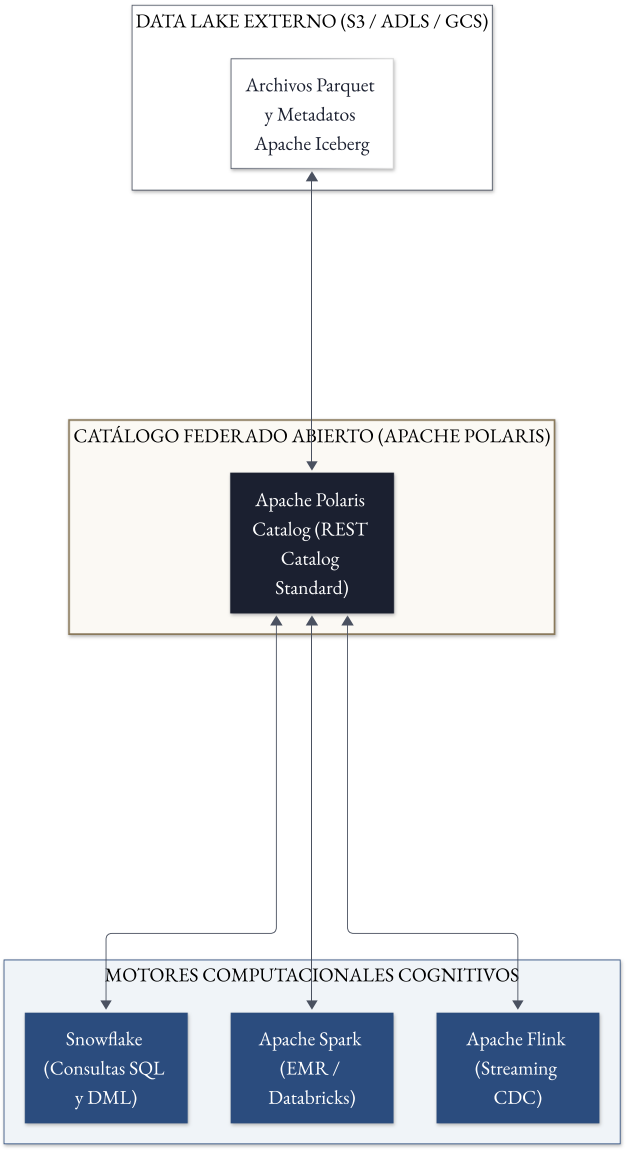
Durante años, la implementación de CDC en Snowflake requería combinar **Streams** (que capturan cambios DML con metadatos de inserción, actualización y borrado) y **Tasks** (que ejecutan sentencias programadas):

```SQL
-- Creación de una Dynamic Table con actualización incremental declarativa
CREATE OR REPLACE DYNAMIC TABLE empresa_dw.gold.resumen_clientes_activo
    TARGET_LAG = '5 MINUTES'
    WAREHOUSE = wh_transformacion
AS
SELECT
    c.id_cliente,
    c.nombre,
    c.pais,
    COUNT(o.id_orden) AS total_ordenes,
    COALESCE(SUM(o.monto_total_usd), 0) AS total_gastado_usd,
    MAX(o.fecha_orden) AS ultima_compra
FROM empresa_dw.silver.clientes c
LEFT JOIN empresa_dw.gold.fact_ordenes o ON c.id_cliente = o.id_cliente
GROUP BY c.id_cliente, c.nombre, c.pais;
```

- `TARGET_LAG:` El ingeniero no especifica *cuándo* debe correr la consulta, sino *qué tan frescos* deben ser los datos (ej. `5 MINUTES`, `1 HOUR` o `DOWNSTREAM`).
- **Actualización Incremental Autónoma:** El motor de Snowflake analiza el plan de consulta y aplica únicamente los cambios delta desde la última ejecución, calculando el grafo de dependencias acíclico (DAG) entre tablas dinámicas automáticamente.

29D.7 Snowflake y el Lakehouse Abierto: Apache Iceberg y Apache Polaris

Para evitar el bloqueo de proveedor (*Vendor Lock-in*), Snowflake introdujo el soporte nativo de **tablas Apache Iceberg**:



- **Tablas Iceberg en Snowflake:** Permiten almacenar los datos en formato Parquet estándar dentro de los buckets de S3 propios del cliente, manteniendo el catálogo y las operaciones transaccionales sincronizadas.
- **Apache Polaris Catalog:** En 2024, Snowflake liberó como proyecto de código abierto **Polaris**, un catálogo federado basado en la especificación de API REST de Apache Iceberg. Permite que Snowflake, Apache Spark, Trino y Flink lean y escriban concurrentemente sobre las mismas tablas Iceberg sin fricciones ni silos propietarios.

29D.8 Anti-Patrones Comunes en Snowflake

Anti-Patrón 1: Configurar Warehouses con Suspensión Desactivada (`AUTO_SUSPEND = 0`)

- **El Error:** Dejar los clústeres de cómputo encendidos indefinidamente para evitar latencias de encendido de 2 segundos.
- **Impacto en Producción:** Un warehouse de talla `Large` (8 créditos/hora a ~$3 USD/crédito) encendido 24/7 sin ejecutar consultas cuesta más de **$17.000 USD al mes** en recursos ociosos.
- **Solución:** Configurar siempre `AUTO_SUSPEND = 60` (1 minuto para cargas interactivas) y `AUTO_RESUME = TRUE`.

Anti-Patrón 2: Un Único Warehouse Sobredimensionado para Todas las Cargas

- **El Error:** Utilizar un warehouse `2X-Large` tanto para tareas de ETL pesadas como para dashboards de BI y consultas ad-hoc de analistas.
- **Impacto:** Consultas simples de dashboards pagan tarifas de 32 créditos/hora innecesariamente y bloquean recursos de ETL.
- **Solución:** Aplicar segregación funcional de warehouses: un warehouse dedicado para ingestión, otro para transformaciones nocturnas y un multi-cluster warehouse para consumo de usuarios.

Resumen del Capítulo

- **Arquitectura Multi-Cluster:** Desacopla almacenamiento centralizado en objetos, cómputo en Virtual Warehouses elásticos y servicios en la nube globales, eliminando la contención de recursos.
- **Micro-Particiones:** Bloques inmutables de 50 a 500 MB con metadatos automáticos que permiten poda masiva de consultas sin particionamiento físico manual.
- **Time Travel y Zero-Copy Cloning:** Permiten viajar en el tiempo hasta por 90 días y clonar bases de datos completas en segundos sin duplicar almacenamiento físico.
- **Snowpark y Dynamic Tables:** Modernizan el desarrollo de pipelines mediante ejecución vectorizada en Python y transformaciones incrementales declarativas con `TARGET_LAG`.
- **Convergencia Abierta:** El soporte de tablas Apache Iceberg y la liberación de **Apache Polaris Catalog** integran a Snowflake en el estándar universal de Lakehouse multi-motor.

Preguntas de Revisión y Ejercicios Prácticos

Pregunta 1: FinOps y Selección de Warehouses

Una empresa procesa un trabajo de transformación de datos una vez al día. En un warehouse `Small` (2 créditos/hora), el trabajo tarda **60 minutos**. En un warehouse `Medium` (4 créditos/hora), el trabajo tarda **30 minutos**. En un warehouse `Large` (8 créditos/hora), el trabajo tarda **25 minutos**. ¿Cuál configuración es óptima desde la perspectiva de costo y por qué?

Solución Detallada:

1. **Cálculo de Consumo de Créditos:**
   - `Small`: 2 créditos/hora × (60/60 horas) = **2.0 créditos**.
   - `Medium`: 4 créditos/hora × (30/60 horas) = **2.0 créditos**.
   - `Large`: 8 créditos/hora × (25/60 horas) = **3.33 créditos**.
2. **Evaluación:**
   - La opción `Medium` duplica la velocidad respecto a `Small` (30 min vs 60 min) consumiendo exactamente los mismos **2.0 créditos**, debido al paralelismo lineal perfecto.
   - La opción `Large` solo reduce el tiempo a 25 min (retornos decrecientes por sobrecarga de coordinación) y aumenta el costo en un **66.5%** (3.33 créditos).
   - **Conclusión:** La configuración óptima es el warehouse `Medium`, ya que maximiza la reducción de latencia sin incurrir en ningún sobrecosto crediticio.

Pregunta 2: Mecánica de Zero-Copy Cloning y Almacenamiento

Si se ejecuta `CREATE TABLE ordenes_qa CLONE fact_ordenes;` sobre una tabla de producción de 10 TB, y posteriormente se insertan 500 GB de nuevos registros en `ordenes_qa`, ¿cuánto almacenamiento factura Snowflake por la tabla clonada?

Solución Detallada:

1. Al momento de la clonación, Snowflake genera una nueva entrada en el catálogo de metadatos apuntando a las mismas micro-particiones de `fact_ordenes`. El almacenamiento facturado atribuible al clon es **0 bytes**.
2. Cuando se insertan 500 GB de datos nuevos en `ordenes_qa`, Snowflake escribe nuevas micro-particiones exclusivas para el clon.
3. El almacenamiento total facturado por `ordenes_qa` es exactamente **500 GB**, manteniendo los 10 TB originales compartidos sin duplicación.

Lecturas Recomendadas

- Dageville, B., Cruanes, T., Zukowski, M., Antonov, V., Avanes, A., Bock, P., Claybaugh, B., Engovatov, D., Hentschel, M., Huang, J., Lee, A. W., Motivala, A., Munir, A. Q., Pelley, S., Povinec, P., Rahn, G., Triantafyllis, S., & Unterbrunner, P. (2016). *The Snowflake Elastic Data Warehouse*. Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16), 215-226.
- Avila, J. K. (2022). *Snowflake: The Definitive Guide: Architecting, Designing, and Deploying on the Snowflake Data Cloud*. O'Reilly Media.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

CAPÍTULO 30

System Design para Data Engineers y Preparación para Roles Senior AVANZADO

> *"Un ingeniero junior se enfoca en hacer que el código funcione. Un ingeniero intermedio se enfoca en optimizar la consulta y reducir los tiempos de ejecución. Un ingeniero senior se enfoca en el diseño del sistema: analiza los trade-offs entre latencia, consistencia, costo y mantenibilidad, sabiendo con precisión cuándo usar una herramienta masiva y, más importante aún, cuándo evitarla."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Aplicar el marco metodológico de 5 pasos para resolver entrevistas y proyectos de **System Design** en ingeniería de datos.
- Ejecutar estimaciones rápidas de escala y volumetría (*Back-of-the-Envelope Calculations*) para justificar decisiones de arquitectura.
- Dominar los fundamentos teóricos distribuidos: **Teorema CAP**, **Teorema PACELC** y la consistencia ajustable de bases NoSQL.
- Discernir con criterio cuantitativo **cuándo NO usar Apache Spark**, evaluando motores modernos sobre un solo nodo como **DuckDB** y **Polars**.
- Diseñar arquitecturas de referencia desacoplando rutas síncronas de baja latencia frente a rutas asíncronas de streaming y analítica.
- Defender decisiones arquitectónicas ante comités técnicos senior mediante la transcripción simulada de una entrevista de diseño.

Definición 30.1 (System Design en Ingeniería de Datos)

El **System Design para Data Engineering** es la disciplina arquitectónica orientada a definir la topología, componentes, protocolos de comunicación, modelos de almacenamiento y estrategias de particionamiento necesarios para satisfacer requisitos analíticos y operacionales a escala, equilibrando explícitamente los compromisos entre:

- **Disponibilidad vs. Consistencia** (Teorema CAP y PACELC).
- **Latencia de procesamiento vs. Rendimiento global (Throughput)**.
- **Costo financiero en la nube (FinOps) vs. Complejidad operativa**.

30.1 Los Teoremas CAP y PACELC en Sistemas Distribuidos

Para justificar la elección de almacenes de datos ante un comité técnico, el ingeniero debe dominar los modelos formales de consistencia distribuida:

1. El Teorema CAP (Brewer)

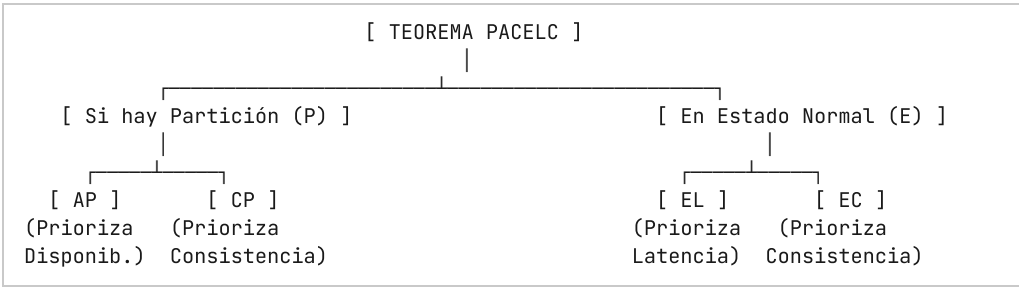
En un sistema distribuido con particiones de red inevitables (P):

- **Consistencia (C):** Toda lectura recibe la escritura más reciente o un error.
- **Disponibilidad (A):** Toda petición no defectuosa recibe una respuesta (sin garantía de ser la más reciente).
- **Tolerancia a Particiones (P):** El sistema sigue funcionando a pesar de pérdidas o retrasos arbitrarios de mensajes en la red. Dado que la red física siempre puede particionarse, un sistema debe elegir entre **CP** (abortar peticiones para evitar datos inconsistentes) o **AP** (responder con datos potencialmente obsoletos para mantener la disponibilidad).

2. El Teorema PACELC (Abadi)

El Teorema CAP solo describe el comportamiento del sistema cuando ocurre una partición de red. Daniel Abadi formuló el **Teorema PACELC** para modelar también el estado de operación normal:

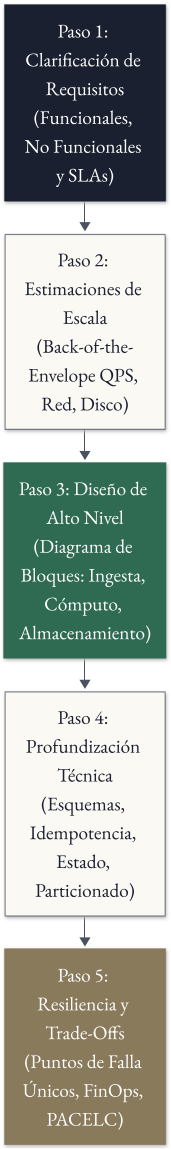
Si hay una Partición (P): ¿Cómo elige el sistema entre Disponibilidad (A) y Consistencia (C)?
Else (E, en operación normal sin fallos): ¿Cómo elige entre Latencia (L) y Consistencia (C)?



- **Sistemas PC/EC (ej. Bigtable, MongoDB primario):** En partición priorizan consistencia; en operación normal pagan mayor latencia sincrónica para asegurar réplicas consistentes.
- **Sistemas PA/EL (ej. Amazon Dynamo, Cassandra predeterminado):** En partición priorizan disponibilidad; en operación normal priorizan mínima latencia mediante escrituras asíncronas y consistencia eventual.

30.2 Marco Metodológico de 5 Pasos para System Design

Ante una pregunta abierta de diseño arquitectónico, el peor error es comenzar enumerando herramientas (por ejemplo: *"usaría Kafka, Flink y Spark"*). El estándar para roles Senior y Staff exige estructurar la respuesta en cinco fases:



1. **Clarificación de Requisitos:** Delimite el alcance funcional (qué queries se responderán) y no funcional (latencia requerida, frescura de datos, presupuesto mensual, durabilidad).
2. **Estimaciones de Escala (Back-of-the-Envelope):** Calcule transacciones por segundo (QPS), ancho de banda de red en MB/s y volumen acumulado de almacenamiento a 1 y 3 años.
3. **Diseño de Alto Nivel:** Trace el flujo continuo de extremo a extremo desacoplando capas.
4. **Profundización Técnica granular:** Justifique formatos de archivo, particionamiento de tópicos, algoritmos de deduplicación y garantías transaccionales.
5. **Resiliencia, Cuellos de Botella y FinOps:** Identifique puntos únicos de falla (*SPOF*), estrategias de reintento (*Circuit Breakers*) y optimización de costos.

30.3 Cálculos Rápidos de Escala (Back-of-the-Envelope)

Un ingeniero senior no adivina el hardware; lo calcula en una servilleta en 60 segundos utilizando órdenes de magnitud:

Constantes de Referencia Esenciales

- **Segundos en un día:** $86.400 \approx 10^5$ segundos (para cálculos mentales rápidos).
- **Throughput de red por nodo moderno:** ≈ 1 a 10 Gbps (≈ 125 MB/s a 1.25 GB/s).
- **Lectura secuencial SSD NVMe:** ≈ 3.000 MB/s; Lectura en red S3 por prefijo: $\approx$ 100 a 200 MB/s.

Fórmula de Caudal (Throughput)

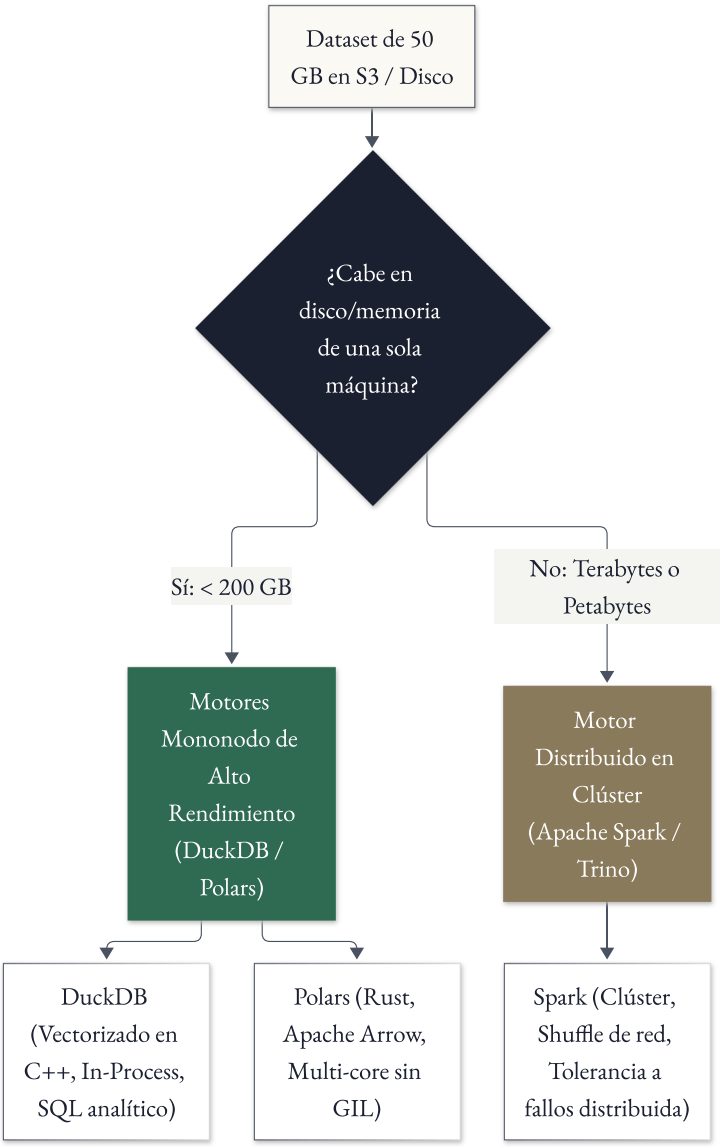
$$\text{QPS Promedio} = \frac{\text{Eventos Diarios}}{86.400} \approx \frac{\text{Eventos Diarios}}{100.000}$$

$$\text{QPS Pico} \approx 2\times \text{ a } 3\times \text{ el QPS Promedio}$$

$$\text{Ancho de Banda (MB/s)} = \text{QPS} \times \text{Tamaño Medio del Evento (KB)}/1.024$$

30.4 Cuándo NO Usar Apache Spark: Motores Mononodo (DuckDB y Polars)

El mayor anti-patrón de la industria es desplegar un clúster distribuido de Spark para procesar datasets de unos pocos gigabytes. El costo de inicialización de la JVM, serialización de shuffle y latencia de red hace que Spark sea órdenes de magnitud más lento que un motor columnar sobre un único nodo:



1. DuckDB: El SQLite de la Analítica Columnares

- **Vectorización y Procesamiento Fuera de Núcleo (*Out-of-Core*):** Puede procesar datasets de cientos de gigabytes que superan la memoria RAM disponible haciendo streaming eficiente contra discos SSD/NVMe locales.
- **Cero Dependencias Externas:** Se ejecuta como una librería embebida en Python (`import duckdb`) o como binario independiente sin requerir la JVM ni configuración de clústeres.
- **Lectura Directa:** Consulta directamente archivos Parquet, Delta Lake o Iceberg en Amazon S3 mediante sentencias SQL estándar a velocidades de gigabytes por segundo.

```python
import duckdb

# Ejecución analítica instantánea sobre Parquet en S3 sin necesidad de clúster Spark
con = duckdb.connect()
df_resumen = con.execute("""
    SELECT
        date_trunc('day', fecha_transaccion) AS dia,
        pais,
        count(*) AS total_ventas,
        sum(monto) AS facturacion_total
    FROM read_parquet('s3://lakehouse-corp/gold/fact_ventas/*.parquet')
    WHERE fecha_transaccion >= '2026-01-01'
    GROUP BY 1, 2
    ORDER BY facturacion_total DESC
""").df()
```

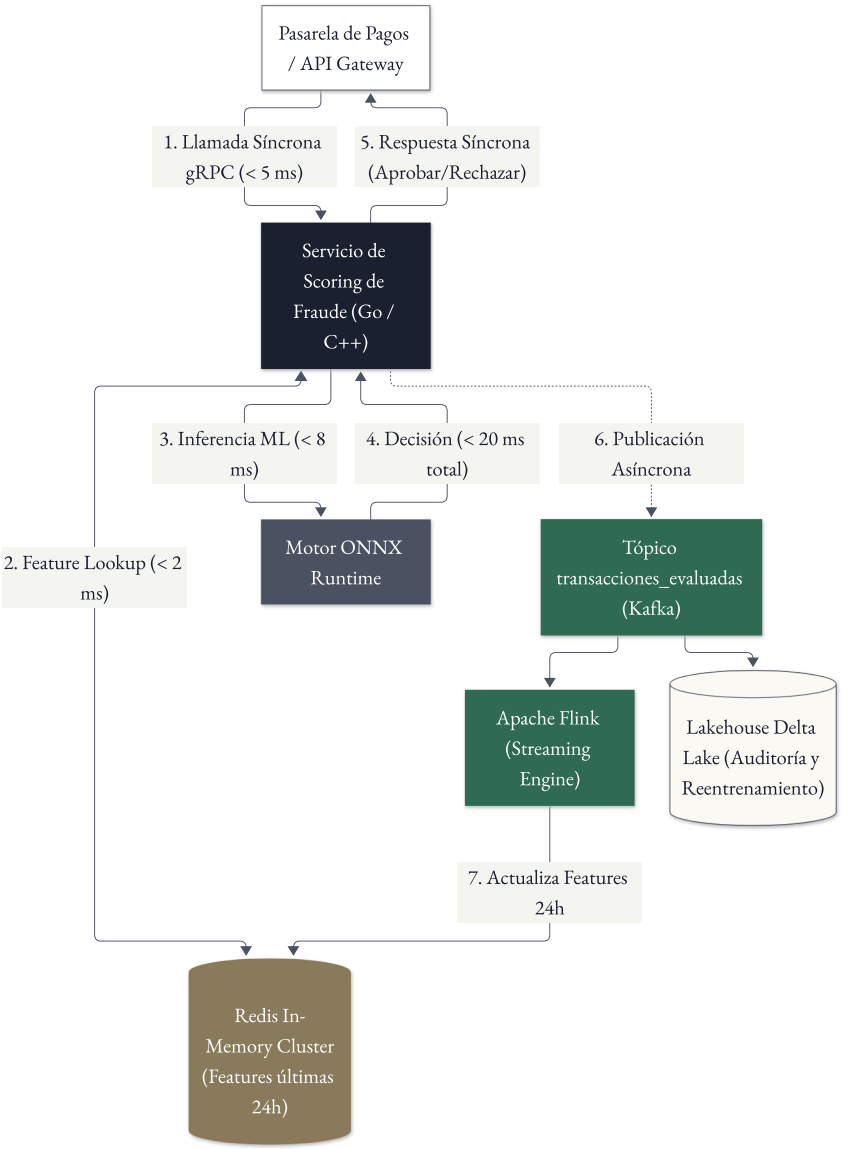
2. Polars: Rendimiento Extremo con Rust y Apache Arrow

- **Escrito en Rust:** Aprovecha la memoria compartida de Apache Arrow con tipado estricto y paralelismo sin bloqueos sobre todos los núcleos de la CPU sin interferencia del GIL de Python.
- **Motor Perezoso (*Lazy API*):** Optimiza planes de consulta mediante empuje de predicados (*Predicate Pushdown*) y poda de proyecciones antes de la ejecución física.
- **Velocidad:** En volúmenes de 1 GB a 100 GB, Polars procesa agrupaciones entre 10 y 30 veces más rápido que Pandas y con latencias de arranque de microsegundos en comparación con los minutos de un clúster Spark.

30.5 Tres Arquitecturas Empresariales de Referencia

Arquitectura 1: Motor de Detección de Fraude en Tiempo Real (< 60 ms)

En sistemas de pagos, la ruta de autorización debe responder de forma estrictamente síncrona dentro del presupuesto temporal del datáfono. **Apache Kafka no se sitúa en la ruta de request/response síncrona**, sino en la ruta asíncrona de auditoría y alimentación de características:



- **Ruta Síncrona Crítica (< 20 ms):** La pasarela invoca el microservicio de scoring mediante gRPC. El servicio consulta el perfil en Redis en microsegundos, corre la inferencia con ONNX Runtime y responde al comercio antes de agotar el SLA de 60 ms.
- **Ruta Asíncrona Continua:** El evento evaluado se publica en Kafka. Apache Flink procesa la ventana deslizante de las últimas 24 horas y actualiza los agregados en Redis para la siguiente transacción. Paralelamente, los datos se almacenan en el Lakehouse para auditorías y reentrenamiento.

Arquitectura 2: Plataforma Lakehouse CDC Multi-Fuente



- **Flujo Operacional:** Debezium captura mutaciones transaccionales sin impacto sobre las bases relacionales. Kafka asegura el desacoplamiento. Spark procesa micro-lotes aplicando deduplicación y `MERGE` atómico con `txnVersion` sobre Delta Lake. dbt modela las capas Silver y Gold, y Unity Catalog gobierna el linaje y permisos.

Arquitectura 3: Analítica de Clickstream a Escala de Petabytes



- **Flujo Operacional:** Eventos masivos son recibidos en el borde por la CDN y derivados a Kafka. Se consolidan en tablas Apache Iceberg particionadas por hora y optimizadas con Z-Order / Liquid Clustering. Un motor MPP como Trino o ClickHouse permite consultas analíticas interactivas en segundos sobre miles de millones de eventos.

30.6 Transcripción Simulada: Entrevista Técnica de System Design Senior

> **Entrevistador:** *"Nuestra plataforma procesa transacciones bancarias. Requerimos diseñar un sistema que detecte fraudes en tiempo real. La latencia total desde que el usuario desliza su tarjeta hasta la respuesta debe ser menor a 200 milisegundos. El volumen es de 50 millones de transacciones diarias. ¿Cómo lo estructurarías?"*

> **Candidato Senior:** *"Excelente desafío. Antes de proponer componentes, me gustaría clarificar los límites y restricciones:
>
> 1. ¿Qué datos incluye la transacción y qué enriquecimiento requiere el modelo (por ejemplo, transacciones de las últimas 24 horas y promedio histórico)?
> 2. Respecto a la latencia: ¿Los 200 ms corresponden a todo el viaje de red del datáfono o al presupuesto interno de nuestra plataforma de datos?
> 3. En caso de una falla en nuestros servicios internos, ¿la política de negocio favorece 'aprobar por omisión' para no degradar la experiencia, o 'bloquear por omisión'?"*

> **Entrevistador:** *"1. Sí, el modelo compara la transacción con las compras de las últimas 24 horas y el promedio histórico. 2. Los 200 ms son de extremo a extremo; nuestro presupuesto interno es de **máximo 60 milisegundos**. 3. Negocio prefiere aprobar por omisión transacciones menores a $50 USD y bloquear montos superiores si el sistema no responde en 60 ms."*

Candidato Senior: *"Perfecto. Con esos límites claros, permítame realizar una estimación rápida de escala:

- 50 millones de transacciones diarias equivalen a ≈ 578 transacciones/seg promedio, con picos de $3\times \approx 1.800$ QPS.
- A 1.800 QPS con eventos de 1 KB, el caudal de red es ligero (≈ 2 MB/s). El desafío técnico no radica en el ancho de banda, sino en la **latencia determinista estricta** para cumplir el presupuesto de 60 ms.

Esta restricción descarta de inmediato ubicar brokers de mensajería asíncronos o micro-lotes de Spark en la ruta de decisión síncrona.

Propongo la siguiente arquitectura desacoplada:

1. **Ruta Síncrona de Scoring (< 25 ms):** La pasarela invoca mediante gRPC un servicio de evaluación de fraude de alta concurrencia (en Go o C++).
2. **Almacén de Características en Memoria (Feature Store):** El servicio consulta un clúster de **Redis en memoria** con particionamiento por `cliente_id`, recuperando en < 2 ms el número de transacciones y el monto acumulado en las últimas 24 horas.
3. **Inferencia del Modelo:** El servicio ejecuta la inferencia localmente mediante ONNX Runtime en C++ en ≈ 8 ms, emitiendo la decisión de Aprobar o Bloquear a la pasarela en menos de 25 ms totales.
4. **Ruta Asíncrona (Auditoría y Streaming):** El servicio emite la transacción evaluada de forma asíncrona a un tópico de **Apache Kafka**. Un pipeline de **Apache Flink** consume el tópico para recalcular ventanas deslizantes y actualizar Redis, mientras que un conector persiste los datos en el Lakehouse (Delta Lake sobre S3) para reentrenamiento."*

Entrevistador: *"Excelente desacoplamiento. ¿Qué ocurre si el nodo de Redis que contiene las features del cliente colapsa en el instante en que llega la transacción?"*

Candidato Senior: *"En términos del Teorema PACELC, para la capa de servicio priorizamos latencia sobre consistencia estricta. Redis estará configurado con réplicas activas en la misma zona de disponibilidad. Si una réplica no responde dentro de un timeout estricto de **5 milisegundos**, el cliente activa un disyuntor (Circuit Breaker): evalúa la transacción utilizando reglas heurísticas locales mínimas (monto menor a $50) y aprueba preventivamente, registrando un log estructurado con el trace_id de OpenTelemetry para auditoría posterior, garantizando que el usuario no espere indefinidamente en la caja registradora."*

30.7 Criterios de Fiabilidad y Anti-Patrones en System Design

Anti-patrón 1: Ubicar Brokers Asíncronos en la Ruta Síncrona de Baja Latencia
Diseñar pasarelas de pago que intentan publicar un mensaje en Kafka y esperar una respuesta en un tópico de retorno con presupuestos inferiores a 100 ms. Kafka es un registro de eventos distribuido optimizado para throughput asíncrono masivo, no un protocolo síncrono de llamada a procedimiento remoto (RPC).

Anti-patrón 2: Sobreingeniería Guiada por Currículum (*Resume-Driven Development*)
Desplegar clústeres distribuidos de Kafka, Flink y Kubernetes para un requerimiento de negocio donde la gerencia únicamente necesita consultar un reporte consolidado una vez por semana. Para volúmenes moderados, DuckDB o scripts programados en contenedores resuelven el problema con una fracción del costo y complejidad.

Anti-patrón 3: Carencia de Planes de Recuperación ante Desastres (RPO y RTO Desconocidos)
Diseñar una plataforma sin definir el **RPO** (*Recovery Point Objective*: cuánto volumen de datos estamos dispuestos a perder ante un desastre) y el **RTO** (*Recovery Time Objective*: cuánto tiempo máximo puede estar la plataforma fuera de servicio).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 30.1 (Estimaciones de Escala y Elección Arquitectónica):
Una empresa de comercio electrónico recibe 10 millones de pedidos al día. El equipo de marketing solicita un tablero para visualizar las ventas agregadas por categoría con actualización horaria.

1. Calcule el throughput de pedidos por segundo promedio.
2. Si el tablero requiere actualizarse cada 1 hora, ¿es necesario implementar Apache Flink o Kafka en tiempo real estricto? Justifique la respuesta desde una perspectiva FinOps y de mantenibilidad.

Solución 30.1:

1. *Throughput Promedio:*

$$\text{QPS} = \frac{10\,000\,000 \text{ pedidos}}{86.400 \text{ segundos}} \approx 115.7 \text{ pedidos/segundo}$$

2. *Evaluación Arquitectónica:*

- **No es necesario implementar streaming continuo.** Un caudal de 116 pedidos/segundo con un SLA de frescura de 1 hora es una carga de trabajo ideal para **procesamiento por lotes programado**.
- *Justificación FinOps:* Mantener un clúster de Kafka y Flink encendido 24/7 para un requerimiento que el usuario solo consulta cada hora implica pagar por infraestructura permanente y complejidad operativa innecesaria. Un pipeline en dbt o DuckDB ejecutado cada hora mediante un job programado en Airflow resolverá el requerimiento a menos del 10% del costo computacional y con una mantenibilidad infinitamente superior.

Ejercicio 30.2 (El Teorema CAP y la Consistencia Ajustable en Delta Lake vs. Apache Cassandra):
Explique cómo se manifiestan las garantías de consistencia y disponibilidad en la arquitectura de almacenamiento de **Delta Lake** frente a **Apache Cassandra**.

Solución 30.2:

1. **Delta Lake (Consistencia Estricta - Semántica CP):** Delta Lake garantiza transacciones ACID mediante su registro transaccional serializable (`_delta_log`) y operaciones atómicas de metadatos sobre el catálogo o almacenamiento de objetos. Si dos procesos intentan modificar concurrentemente los mismos datos bajo aislamiento `WriteSerializable`, Delta Lake **aborta una de las transacciones con un conflicto de concurrencia**, prefiriendo fallar antes que permitir inconsistencias en los datos.
2. **Apache Cassandra (Consistencia Ajustable / Tunable Consistency):** Clasificar a Cassandra estrictamente como "AP" es una simplificación inexacta. Cassandra opera con una topología descentralizada sin nodo maestro y permite definir el nivel de consistencia por consulta:
   - Si se configuran lecturas con consistencia `ONE` y escrituras con `ONE`, el sistema maximiza disponibilidad y latencia (AP / PA/EL), tolerando particiones a expensas de consistencia eventual.
   - Sin embargo, si se configuran lecturas y escrituras con `QUORUM` sobre un factor de replicación $N = 3$, la condición $R + W > N$ $(2 + 2 > 3)$ garantiza matemáticamente que al menos un nodo en el quórum de lectura contenga la última escritura confirmada, alcanzando **consistencia fuerte configurable**.

Ejercicio 30.3 (Estructuración de Respuesta en Entrevistas de System Design):
En una entrevista técnica para Senior Data Engineer, ¿cuál es la secuencia correcta de pasos recomendada para abordar el diseño de un pipeline de ingesta a escala de petabytes?

Solución 30.3:

1. **Paso 1: Clarificación de Requisitos:** Delimitar si el pipeline es batch o streaming, la latencia admisible, el formato de origen y las restricciones de presupuesto.
2. **Paso 2: Estimación de Escala:** Calcular el volumen diario en terabytes, el ancho de banda requerido en MB/s y los requerimientos de red y almacenamiento para los próximos 3 años.
3. **Paso 3: Arquitectura de Alto Nivel:** Dibujar el flujo lógico (Ingesta → Almacenamiento Medallón → Transformación → Servicio).
4. **Paso 4: Profundización Técnica:** Especificar el particionamiento de datos, el formato de tabla abierta (Delta/Iceberg), las políticas de deduplicación y el manejo de esquemas.
5. **Paso 5: Resiliencia y Costos:** Identificar puntos únicos de falla, mecanismos de recuperación ante desastres y optimización FinOps de instancias reservadas y almacenamiento en capas frías.

30.8 Resumen y Puntos Clave

- **Metodología rigurosa:** Las entrevistas y proyectos de diseño deben comenzar siempre con la clarificación de requisitos y cálculos de escala antes de seleccionar herramientas concretas.
- **Teoremas CAP y PACELC:** Formalizan los compromisos entre consistencia, disponibilidad y latencia, permitiendo justificar por qué ciertas capas requieren consistencia estricta (Delta Lake) y otras priorizan latencia (Redis).
- **Consistencia ajustable:** Cassandra no es meramente un sistema "AP"; permite alcanzar consistencia fuerte cuando $R + W > N$.
- **Desacoplamiento síncrono/asíncrono:** Los sistemas de decisión de muy baja latencia (< 60 ms) operan mediante servicios síncronos y almacenes en memoria, relegando los buses de eventos (Kafka) a rutas asíncronas de auditoría y cálculo de estado.
- **Pragmatismo técnico:** Evalúe siempre alternativas mononodo como DuckDB y Polars antes de introducir la complejidad operativa de clústeres distribuidos.

Lecturas Recomendadas

1. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems.* O'Reilly Media.
2. **Abadi, D. J. (2012).** *Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story.* Computer, 45(2), 37-42. (El artículo científico seminal del Teorema PACELC).

3. **Xu, A. (2020).** *System Design Interview – An Insider's Guide.* ByteByteGo.

CAPÍTULO 31

Seguridad, Privacidad y Gobernanza Regulatoria de Datos AVANZADO

> *"En la ingeniería de datos contemporánea, la seguridad y la privacidad no son parches que se añaden al final del desarrollo: son requisitos arquitectónicos fundamentales. Un pipeline capaz de transformar petabytes por hora carece de valor si expone información personal identificable (PII), viola la legislación de protección de datos o incumple el principio de mínimo privilegio en la nube."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

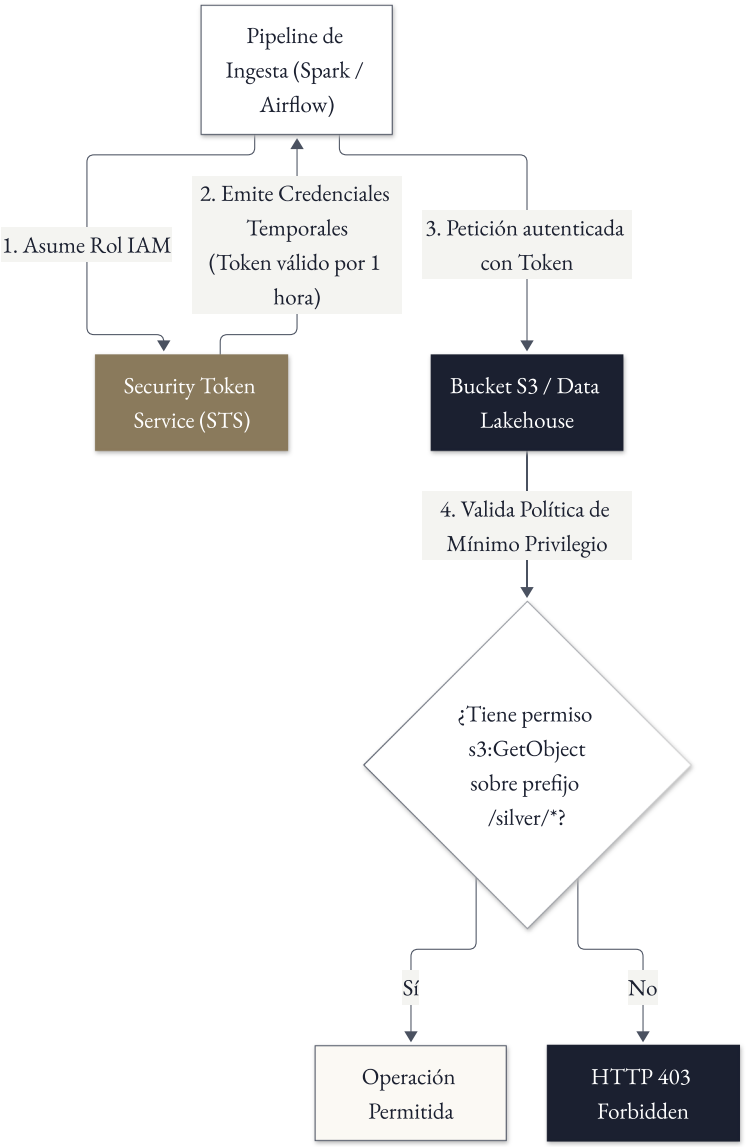
- Diseñar políticas de control de acceso e identidades basadas en el **principio de mínimo privilegio** mediante **IAM** (*Identity and Access Management*), roles asumibles y credenciales temporales.
- Implementar mecanismos de cifrado de grado militar tanto en reposo (*Envelope Encryption* con **KMS** y CMEK) como en tránsito (**TLS 1.3 / mTLS**).
- Integrar sistemas de gestión centralizada de secretos de producción (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, Databricks Secrets), erradicando credenciales en texto plano.
- Aplicar técnicas criptográficas de **tokenización**, seudonimización y enmascaramiento dinámico sobre Información Personal Identificable (**PII**).
- Comprender y cumplir las exigencias del **Reglamento General de Protección de Datos (GDPR)** europeo, resolviendo el desafío arquitectónico del *Derecho al Olvido* sobre formatos inmutables de Lakehouse.
- Analizar y aplicar el régimen colombiano de protección de datos personales (**Ley 1581 de 2012 de Habeas Data**) y las directrices de la Superintendencia de Industria y Comercio (SIC) en arquitecturas analíticas.

Definición 31.1 (Información Personal Identificable - PII) Se denomina **PII** (*Personally Identifiable Information*) a cualquier dato que permita identificar de manera unívoca a una persona física, directa o indirectamente (nombres, documentos de identidad, correos electrónicos, números telefónicos, direcciones IP, coordenadas geográficas o números de tarjetas financieras).

En plataformas analíticas, el acceso a la PII debe estar estrictamente restringido por defecto (*Zero Trust*), requiriendo tokenización unidireccional (hashing criptográfico con sal) antes de su promoción a las capas Silver y Gold del Lakehouse.

31.1 Gestión de Identidades y Mínimo Privilegio (IAM)

El control de acceso en la nube se rige por el modelo **IAM** (*Identity and Access Management*):



1. El Principio de Mínimo Privilegio (Least Privilege)

Una aplicación o usuario debe recibir exclusivamente los permisos mínimos e indispensables para desempeñar su tarea, durante el tiempo estrictamente necesario.

- **Anti-patrón:** Asignar `AdministratorAccess` o `AmazonS3FullAccess` a un clúster de Spark para "ahorrar tiempo".
- **Práctica Correcta:** Otorgar permisos delimitados al ARN exacto del bucket y prefijo requerido:

```JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PermitirLecturaSoloSilver",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::empresa-lakehouse-prod",
        "arn:aws:s3:::empresa-lakehouse-prod/silver/*"
      ]
    }
  ]
}
```

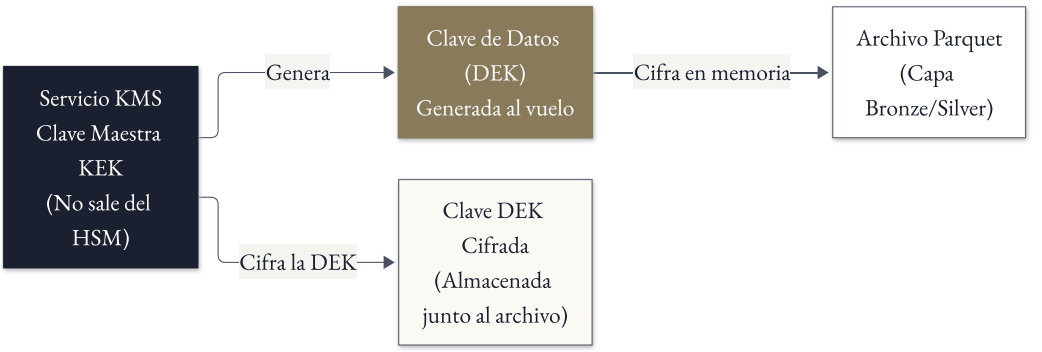
2. Roles Asumibles (*IAM Roles*) frente a Claves Estáticas

En producción, **los servicios nunca deben utilizar claves estáticas** (`AWS_ACCESS_KEY_ID` y `AWS_SECRET_ACCESS_KEY`). Se deben emplear **IAM Roles** asociados a la instancia o pod de Kubernetes (ej. *IRSA: IAM Roles for Service Accounts* en EKS, o identidades administradas en Azure). El servicio asume el rol dinámicamente mediante el *Security Token Service* (STS), recibiendo credenciales que caducan automáticamente cada hora.

31.2 Cifrado en Reposo y en Tránsito (KMS y Envelope Encryption)

1. Cifrado en Reposo con Envelope Encryption

Para cifrar petabytes de datos en Amazon S3, ADLS Gen2 o Google Cloud Storage sin degradar el rendimiento, los proveedores utilizan el patrón criptográfico de **Cifrado de Sobre** (*Envelope Encryption*):



- **KEK (*Key Encryption Key*):** Clave maestra custodiada de forma segura dentro de módulos de seguridad de hardware (HSM) del servicio KMS (AWS KMS, Azure Key Vault). Nunca sale del HSM.
- **DEK (*Data Encryption Key*):** Clave simétrica efímera de un solo uso generada por el KMS para cifrar el archivo Parquet.
- **Mecánica:** El motor cifra el archivo Parquet con la DEK en memoria, el KMS cifra la DEK con la KEK maestra, la DEK en texto plano se destruye de la memoria RAM, y la DEK cifrada se adjunta a los metadatos del archivo en disco.

2. Claves Administradas por el Cliente (CMEK / BYOK)

Para cumplir con normativas financieras y de defensa, las organizaciones utilizan **CMEK** (*Customer-Managed Encryption Keys*), donde la empresa retiene la propiedad de la clave KMS y puede revocarla unilateralmente en cualquier instante, volviendo todos los datos del Lakehouse inmediatamente inaccesibles e ilegibles (*Crypto-Shredding*).

31.3 Gestión de Secretos de Producción

Los pipelines requieren credenciales para conectarse a bases relacionales, brokers de Kafka o APIs externas. Estas credenciales deben gestionarse a través de almacenes centralizados con rotación automática:

```python
# Ejemplo de consumo seguro de secretos en Databricks / PySpark
# Las credenciales NUNCA están escritas en el código
usuario_db = dbutils.secrets.get(scope="finanzas-prod", key="postgres-user")
password_db = dbutils.secrets.get(scope="finanzas-prod", key="postgres-password")

# En entornos agnósticos / Python estándar utilizando AWS Secrets Manager:
import boto3
import json

def obtener_secreto_db(nombre_secreto: str, region: str = "us-east-1") -> dict:
    cliente = boto3.client("secretsmanager", region_name=region)
    respuesta = cliente.get_secret_value(SecretId=nombre_secreto)
    return json.loads(respuesta["SecretString"])
```

31.4 Tokenización y Seudonimización de PII

Cuando un pipeline ingesta datos con nombres, cédulas o correos electrónicos, estos deben ser disociados antes de ser expuestos a científicos de datos o analistas:

1. Hashing Criptográfico con Sal (*Salted Hash*)

Convertir un identificador sensible (ej. número de documento) en un hash unidireccional determinista mediante SHA-256 o HMAC, añadiendo una "sal" criptográfica (*salt*) secreta custodiada en KMS para impedir ataques de diccionario (*Rainbow Tables*):

$$\text{Token} = \text{HMAC-SHA256}(\text{DocumentoIdentidad}, \text{SalKMS})$$

```python
import hmac
import hashlib
from pyspark.sql import functions as F

def tokenizar_pii(df, columna_origen: str, salt_secreto: str, columna_destino:
str):
    # Generar UDF o expresión nativa de hashing salado
    salt_literal = F.lit(salt_secreto)
    return df.withColumn(
        columna_destino,
        F.sha2(F.concat(F.col(columna_origen), salt_literal), 256)
    )
```

2. Enmascaramiento Dinámico de Columnas (*Dynamic Column Masking*)

Permite que los usuarios con permisos ejecutivos visualicen el valor original, mientras que analistas generales visualicen una cadena enmascarada (ej. `ana.torres@empresa.com` se transforma en `a***s@empresa.com`). Esto se gobierna a nivel de catálogo centralizado (Unity Catalog, Snowflake o AWS Lake Formation).

31.5 Regulaciones Internacionales: GDPR y el Reto del Derecho al Olvido

El **Reglamento General de Protección de Datos (GDPR)** de la Unión Europea impone obligaciones severas:

1. **Consentimiento Explícito:** La base legal para recolectar y tratar cada atributo.
2. **Minimización de Datos:** Conservar únicamente los campos indispensables para el fin declarado.
3. **Derecho al Olvido (*Right to be Forgotten - Artículo 17*):** El titular de los datos puede solicitar la eliminación total e irreversible de toda su información personal en un plazo máximo de 30 días.

El Desafío del Derecho al Olvido en Lakehouses

En formatos de solo adición (*Append-Only*) como Parquet en S3, los archivos son inmutables. Para "borrar" a un usuario en una tabla Lakehouse:

1. Se debe ejecutar un `DELETE FROM tabla WHERE usuario_id = 'XYZ'`.
2. **Impacto de Deletion Vectors:** Si la tabla tiene habilitados *Deletion Vectors* (estándar en Delta Lake 3.x y Databricks Runtime 14+), el `DELETE` **no reescribe los archivos Parquet físicos**; simplemente escribe un archivo bitmap `.bin` con las filas descartadas. Por ende, los datos del usuario siguen residiendo físicamente en el archivo Parquet original.
3. **El Purgado Físico Obligatorio (`REORG TABLE ... APPLY (PURGE)`):** Para eliminar físicamente las filas marcadas por los vectores de eliminación, se debe ejecutar una reescritura física de los archivos afectados:

```sql
REORG TABLE lakehouse.silver_usuarios APPLY (PURGE);
```

4. **El Peligro del Historial (*Time Travel*) y Eliminación de Snapshots:** Una vez reescrito el archivo, el usuario ya no está en la versión activa, pero los archivos viejos continúan en las versiones históricas. Para cumplir formalmente con GDPR, se debe forzar la expiración de

snapshots históricos (`VACUUM tabla RETAIN 0 HOURS` en una ventana de mantenimiento exclusiva para evitar `FileNotFoundException` en lectores concurrentes, tal como se advierte en el Cap. 18), o utilizar la técnica de **Crypto-shredding** (destruir la clave de cifrado simétrica asignada a ese usuario en KMS, haciendo que sus datos almacenados queden matemáticamente irrecuperables de forma instantánea).

31.6 Marco Regulatorio Colombiano: Ley 1581 de 2012 (Habeas Data)

En Colombia, el tratamiento de datos personales está regulado por la **Ley Estatutaria 1581 de 2012**, reglamentada por el Decreto 1377 de 2013 y fiscalizada por la **Superintendencia de Industria y Comercio (SIC)** a través de la Delegatura para la Protección de Datos Personales.

1. Principios Rectores Clave para Ingenieros de Datos

- **Principio de Legalidad:** El tratamiento es una actividad reglada que debe sujetarse a la Constitución y la ley.
- **Principio de Finalidad:** El tratamiento debe obedecer a una finalidad legítima de acuerdo con la Constitución y la ley, la cual debe ser informada al titular en la política de privacidad. No se pueden reutilizar datos para fines no autorizados.
- **Principio de Libertad:** Los datos personales solo pueden ser tratados con el **consentimiento previo, expreso e informado del titular**. Queda prohibido capturar o divulgar datos sin autorización.
- **Principio de Veracidad o Calidad:** La información debe ser veraz, completa, exacta, actualizada y comprobable. Está prohibido el tratamiento de datos fraccionados o que induzcan a error.
- **Principio de Seguridad:** Los datos deben ser manejados con las medidas técnicas, humanas y administrativas necesarias para otorgar seguridad a los registros, evitando su adulteración, pérdida o acceso no autorizado.
- **Principio de Confidencialidad:** Todas las personas que intervengan en el tratamiento de datos personales están obligadas a garantizar la reserva de la información.

2. Clasificación Legal de Datos en Colombia

- **Dato Público:** Aquel que la ley califica como tal (ej. estado civil, profesión, sentencias judiciales ejecutoriadas).
- **Dato Semiprivado:** No es íntimo pero interesa a cierto sector (ej. historial crediticio y financiero regulado por la Ley 1266 de 2008).

- **Dato Privado:** Información íntima que solo interesa al titular (ej. fotos personales, hábitos privados).
- **Dato Sensible:** Aquel que afecta la intimidad del titular o cuyo uso indebido puede generar discriminación (origen racial, orientación política, convicciones religiosas, datos de salud, datos biométricos como huellas dactilares o reconocimiento facial).
    - *Regla Estricta:* **El tratamiento de datos sensibles está prohibido por regla general**, salvo excepciones taxativas de ley con autorización explícita.

3. Registro Nacional de Bases de Datos (RNBD)

Las empresas en Colombia están obligadas a inscribir sus bases de datos ante el **RNBD** de la SIC, declarando:

- Nombre y ubicación física o cloud del repositorio (ej. nombre del bucket S3 o servidor).
- Política de tratamiento de datos aplicable.
- Medidas de seguridad y cifrado implementadas.
- Transferencia internacional de datos (enviar datos a servidores en el extranjero exige que el país receptor cuente con estándares adecuados de protección según la SIC, como Estados Unidos bajo ciertos acuerdos o la Unión Europea).

31.7 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Almacenar Cédulas y Nombres en Tablas de la Capa Gold de Analítica Abierta
Permitir que dashboards corporativos consulten directamente tablas con nombres, números de identificación o correos en texto plano expone a la compañía a sanciones millonarias de la SIC o autoridades de GDPR.
Corrección: Reemplace los identificadores naturales por claves subrogadas tokenizadas y restrinja el acceso a la tabla de mapeo (*Lookup Table*) exclusivamente a personal autorizado de seguridad.

Anti-patrón 2: Registros de Auditoría (*Logs*) que Imprimen Cargas Útiles con PII
Escribir `logger.info(f"Procesando evento: {payload}")` cuando el payload contiene tarjetas de crédito o contraseñas filtra datos sensibles a CloudWatch, Datadog o Splunk, violando los estándares PCI-DSS y las normativas de privacidad.
Corrección: Configure filtros de saneamiento (*log sanitizers*) que detecten y enmascaren automáticamente patrones de tarjetas, correos y contraseñas antes de escribir en los flujos de log.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 31.1 (Arquitectura de Cumplimiento de Derecho al Olvido):
Un usuario en Colombia solicita la supresión total de sus datos en ejercicio de su derecho de Habeas Data ante una fintech. La arquitectura de la fintech almacena los datos en una tabla Delta Lake particionada por mes en Amazon S3. Describa la secuencia técnica rigurosa de ingeniería para garantizar el borrado efectivo tanto en la capa lógica como física.

Solución 31.1:

1. **Ejecución de Borrado Lógico en Delta Lake:**

```SQL
DELETE FROM lakehouse.silver_usuarios WHERE cliente_id = 98124;
```

   Delta Lake escribe nuevos archivos Parquet excluyendo el registro del cliente y marca los archivos antiguos como eliminados en el Delta Log.

2. **Purgado Físico de Archivos con Deletion Vectors y de Instantáneas Históricas:**
   Si la tabla utiliza *Deletion Vectors*, se debe purgar físicamente los archivos subyacentes antes de invocar la limpieza del catálogo:

```SQL
-- Paso A: Reescribir los archivos Parquet para expulsar físicamente las filas descartadas
REORG TABLE lakehouse.silver_usuarios APPLY (PURGE);

-- Paso B: En ventana de mantenimiento exclusiva (sin lectores concurrentes), purgar el historial
SET spark.databricks.delta.retentionDurationCheck.enabled = false;
VACUUM lakehouse.silver_usuarios RETAIN 0 HOURS;
```

3. **Propagación a Réplicas y Vistas Materializadas:**
   Reconstruir o sincronizar vistas materializadas y tablas dimensionales en la capa Gold que contengan referencias desnormalizadas al cliente.
4. **Registro en Log de Auditoría Legal:**
   Registrar la confirmación del borrado en una tabla de auditoría inmutable (indicando fecha, ID de solicitud de la SIC/titular y hash del usuario), sin registrar PII en el log.

31.8 Resumen y Puntos Clave

- **Mínimo privilegio:** Todo pipeline debe autenticarse mediante roles IAM asumibles con credenciales temporales y permisos delimitados al recurso exacto.

- **Cifrado de sobre (KMS):** Protege los datos en reposo combinando la eficiencia de claves DEK simétricas con la seguridad centralizada de claves maestras KEK en HSMs.
- **Gestión centralizada de secretos:** Las credenciales operacionales nunca se versionan en Git ni se escriben en Dockerfiles; se consumen en tiempo de ejecución desde almacenes seguros.
- **Tokenización salada:** Disocia la identidad del titular antes de transferir registros a capas de consumo analítico.
- **Cumplimiento GDPR y Ley 1581 de 2012:** Exige autorización previa del titular, delimitación de finalidad, registro ante el RNBD de la SIC y procedimientos técnicos verificables para el derecho a la supresión de datos en Lakehouses.

Lecturas Recomendadas

1. **Amazon Web Services (2024).** *AWS Security Best Practices: Identity and Access Management (IAM) and AWS KMS Envelope Encryption.* AWS Whitepapers.
2. **Congreso de la República de Colombia (2012).** *Ley Estatutaria 1581 de 2012: Régimen General de Protección de Datos Personales (Habeas Data).* Diario Oficial No. 48.587.
3. **Superintendencia de Industria y Comercio - SIC (2023).** *Guía para la Gestión de Incidentes de Seguridad en el Tratamiento de Datos Personales.* Delegatura de Protección de Datos Personales.
4. **European Parliament and Council (2016).** *Regulation (EU) 2016/679 (General Data Protection Regulation - GDPR).* Official Journal of the European Union.

CAPÍTULO 32

Bases de Datos NoSQL y Motores Especializados AVANZADO

> *"La era del almacenamiento uniforme monolítico ha concluido. Ninguna tecnología aislada puede satisfacer simultáneamente los requerimientos de analítica masiva sobre petabytes, lecturas sub-milisegundas de perfiles de usuario, búsquedas jerárquicas y transacciones altamente concurrentes. El ingeniero de datos senior practica la persistencia políglota, seleccionando el motor óptimo según sus compensaciones entre consistencia, disponibilidad y latencia."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Aplicar los teoremas **CAP** y **PACELC** para justificar técnicamente la selección de bases de datos NoSQL frente a motores relacionales y Lakehouses.
- Implementar patrones de ingeniería de alto rendimiento con **Redis** (caching de capas analíticas, control de cuotas y deduplicación con estructuras probabilísticas *HyperLogLog*).
- Diseñar modelos de datos en **Apache Cassandra** gobernados por claves de partición y columnas de agrupamiento (*Clustering Columns*), configurando su consistencia ajustable (*Tunable Consistency*).
- Modelar entidades en **Amazon DynamoDB** optimizando claves de partición/ordenación y capturando cambios continuos mediante **DynamoDB Streams**.
- Diseñar colecciones documentales en **MongoDB** mediante agregaciones analíticas complejas (*Aggregation Pipeline*).
- Comprender la arquitectura de **Apache Flink**, **Kafka Connect** y la semántica *Exactly-Once* (EOS) con transacciones distribuidas en dos fases (2PC).

Definición 32.1 (Persistencia Políglota en Arquitecturas Modernas) La **persistencia políglota** (*Polyglot Persistence*) es el principio arquitectónico que postula que diferentes tipos de datos y patrones de acceso exigen diferentes motores de almacenamiento especializados:

- Almacenes en Memoria (Redis): Latencias < 1 ms para contadores, colas y cachés.
- Almacenes Columnares Anchos (Cassandra/DynamoDB): Alto throughput de escritura y consultas rápidas por clave primaria particionada.
- Almacenes Documentales (MongoDB): Datos semiestructurados jerárquicos y catálogos de esquemas variables.
- Data Lakehouses (Delta Lake/Iceberg): Análisis exploratorio, aggregaciones masivas y aprendizaje automático sobre petabytes históricos.

32.1 El Teorema PACELC: Más Allá de CAP

El clásico teorema CAP (*Consistencia, Disponibilidad, Tolerancia a Particiones*) afirma que en presencia de una partición de red (P), un sistema distribuido debe elegir entre consistencia (C) o disponibilidad (A). Sin embargo, la mayor parte del tiempo el sistema opera en condiciones normales sin fallos de red.

El teorema **PACELC** (formulado por Daniel Abadi) extiende formalmente CAP:

$$\mathbf{P}\left(\mathbf{A} \vee \mathbf{C}\right) \quad \mathbf{E}\text{lse} \quad \mathbf{L} \vee \mathbf{C}$$

- **Si hay una Partición (P):** ¿El sistema prefiere Disponibilidad (A) o Consistencia (C)?
- **En caso contrario (*Else - E*):** ¿El sistema prefiere baja Latencia (L) o Consistencia estricta (C)?

Motor NoSQL	Clasificación PACELC	Compensación Arquitectónica Principal
Apache Cassandra	**PA/EL**	En partición prefiere Disponibilidad; en operación normal prioriza mínima Latencia (consistencia ajustable).
Amazon DynamoDB	**PA/EL**	Altísima disponibilidad y latencia predecible de un solo dígito de milisegundo.
MongoDB	**PC/EC**	Prioriza consistencia estricta por defecto (elección de líder primario mediante consenso).
Redis	**PC/EC** *(nodo primario)*	Persistencia en memoria ultrarrápida, pero replicación asíncrona por defecto hacia réplicas.

32.2 Redis: Almacenamiento en Memoria y Patrones de Ingeniería

Redis (*Remote Dictionary Server*) almacena sus datos en memoria RAM, ofreciendo respuestas con latencias inferiores a un milisegundo:



1. Deduplicación y Conteo Cardinal Probabilístico con HyperLogLog

En telemetría de Big Data, contar usuarios únicos (*Count Distinct*) sobre miles de millones de eventos en tiempo real consume gigabytes de memoria en tablas hash convencionales. **HyperLogLog (HLL)** resuelve este cálculo utilizando una memoria fija de **12 KB con un error estándar inferior al 0.81%**:

```python
import redis

cliente_redis = redis.Redis(host="localhost", port=6379, db=0)

# Registro de eventos en tiempo real con HyperLogLog
def registrar_visita_usuario(fecha_iso: str, usuario_id: str) -> None:
    clave_hll = f"visitas_unicas:{fecha_iso}"
    # PFADD agrega el identificador al conjunto probabilístico
    cliente_redis.pfadd(clave_hll, usuario_id)

def estimar_usuarios_unicos(fecha_iso: str) -> int:
    clave_hll = f"visitas_unicas:{fecha_iso}"
    # PFCOUNT devuelve la estimación cardinal casi exacta
    return cliente_redis.pfcount(clave_hll)
```

2. Control de Cuotas con Ventana Deslizante en Sorted Sets

Para implementar *Rate Limiting* estricto (ej. permitir máximo 100 peticiones por minuto por usuario), se utilizan conjuntos ordenados puntuados por la marca de tiempo UNIX:

```python
import time

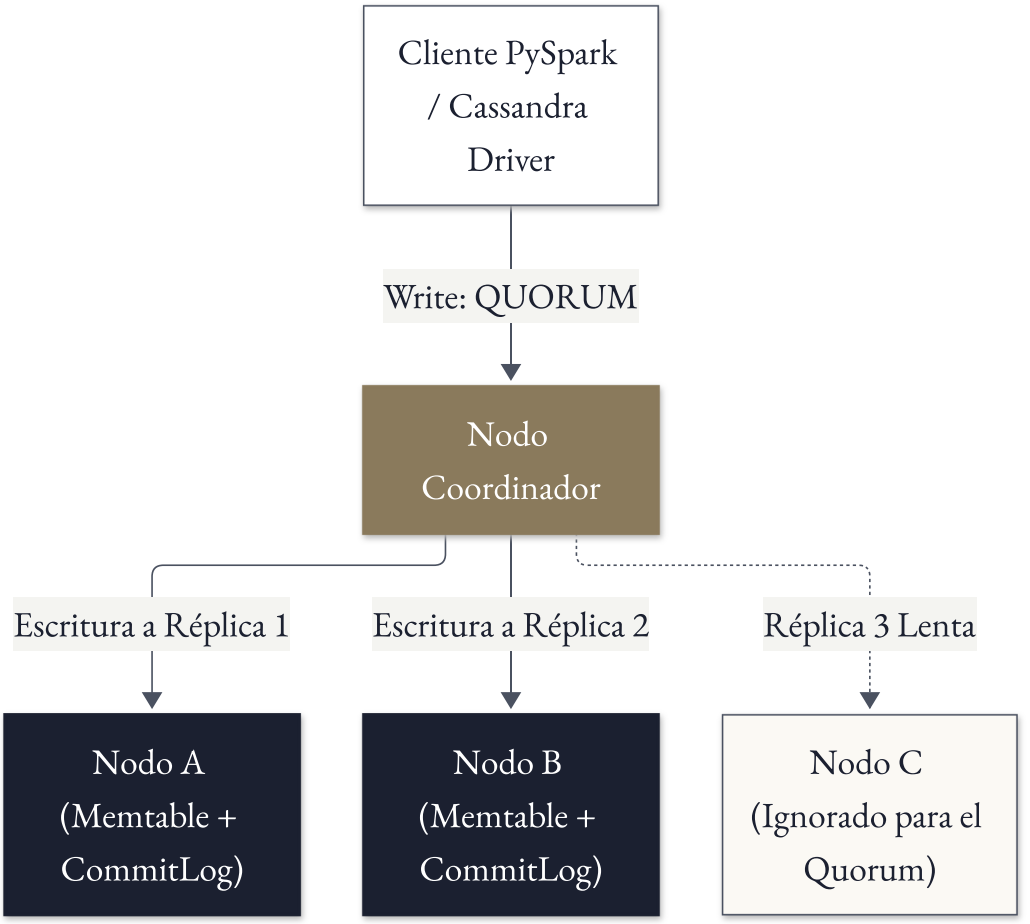
def validar_cuota_rate_limit(usuario_id: str, limite: int = 100,
ventana_segundos: int = 60) -> bool:
    ahora = time.time()
    limite_ventana = ahora - ventana_segundos
    clave = f"rate_limit:{usuario_id}"

    pipeline = cliente_redis.pipeline()
    # 1. Eliminar peticiones que salieron de la ventana temporal
    pipeline.zremrangebyscore(clave, 0, limite_ventana)
    # 2. Agregar la petición actual con score = timestamp
    pipeline.zadd(clave, {str(ahora): ahora})
    # 3. Contar peticiones dentro de la ventana
    pipeline.zcard(clave)
    # 4. Establecer expiración para no acumular memoria
    pipeline.expire(clave, ventana_segundos + 5)

    _, _, total_peticiones, _ = pipeline.execute()
    return total_peticiones <= limite
```

32.3 Apache Cassandra: Almacén Columnar Ancho y Consistencia Ajustable

Apache Cassandra es un sistema distribuido sin nodo maestro (*Masterless Peer-to-Peer*), donde cada nodo del anillo tiene idénticas responsabilidades (*Consistent Hashing*).



1. El Modelo de Claves en Cassandra (CQL)

A diferencia de SQL tradicional donde el esquema se diseña alrededor de las entidades y se normaliza, en Cassandra **el modelo de datos se diseña estrictamente en función de las consultas que se van a ejecutar**:

```SQL
-- Creación de tabla optimizada para consultar eventos de un sensor en un rango de fechas
CREATE TABLE telemetria_iot.lecturas_sensor (
    sensor_id UUID,
    anio_mes text,
    timestamp_evento timestamp,
    temperatura double,
    presion double,
    PRIMARY KEY ((sensor_id, anio_mes), timestamp_evento)
) WITH CLUSTERING ORDER BY (timestamp_evento DESC);
```

- **Partition Key `((sensor_id, anio_mes))`:** El hash de estos campos determina exactamente en qué nodo físico del anillo se alojará el dato. Incluir `anio_mes` evita que particiones de un solo sensor crezcan descontroladamente en el tiempo (*Evita particiones masivas mayores a 100 MB*).
- **Clustering Column `(timestamp_evento)`:** Determina el orden físico secuencial de las filas dentro de la partición en disco (*SSTables*).

2. Consistencia Ajustable (*Tunable Consistency*)

Cassandra permite al cliente definir el nivel de consistencia para cada operación de lectura (R) y escritura (W):

- **Fórmula de Consistencia Fuerte:** Si se cumple que:

$$R + W > N \quad (\text{donde } N \text{ es el factor de replicación})$$

  se garantiza formalmente una lectura consistente de la última escritura (*Strong Consistency*).
- Si $N = 3$, configurando `WRITE = LOCAL_QUORUM` (2 nodos) y `READ = LOCAL_QUORUM` (2 nodos), $2 + 2 = 4 > 3$, garantizando que la lectura intersecte al menos un nodo con el dato actualizado, protegiendo contra nodos lentos en otras zonas de disponibilidad.

32.4 Amazon DynamoDB y DynamoDB Streams

DynamoDB es un servicio NoSQL clave-valor y documental totalmente administrado en AWS que garantiza latencia consistente de milisegundos a cualquier escala:

1. Estructura de Claves y Modos de Capacidad

- **Partition Key (PK) y Sort Key (SK):** Permiten modelar múltiples entidades en una única tabla (*Single-Table Design*).
- **Unidades de Capacidad:**
  - **RCU (*Read Capacity Unit*):** 1 lectura fuertemente consistente por segundo para elementos de hasta 4 KB.
  - **WCU (*Write Capacity Unit*):** 1 escritura por segundo para elementos de hasta 1 KB.

2. DynamoDB Streams para Ingesta y CDC Serverless

Toda mutación (`INSERT`, `MODIFY`, `REMOVE`) en una tabla DynamoDB se emite cronológicamente a un flujo **DynamoDB Streams**. Este flujo puede consumirse mediante AWS Lambda o procesarse directamente en micro-lotes de Spark Structured Streaming para alimentar la capa

Bronze del Lakehouse en tiempo casi real sin sobrecargar la base operacional.

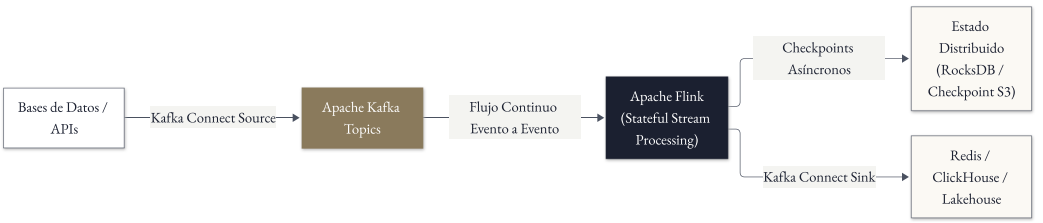
32.5 MongoDB y el Procesamiento Documental BSON

MongoDB organiza los datos en colecciones de documentos JSON binarios (**BSON**), permitiendo esquemas polimórficos:

```javascript
// Pipeline de Agregación Analítica en MongoDB
db.pedidos.aggregate([
    // 1. Filtrar pedidos confirmados en 2026
    { $match: { estado: "ENTREGADO", fecha: { $gte: ISODate("2026-01-01") } } },
    // 2. Desenrollar el array de líneas de producto
    { $unwind: "$lineas_items" },
    // 3. Agrupar por categoría y calcular métricas
    { $group: {
        _id: "$lineas_items.categoria",
        volumen_total_usd: { $sum: { $multiply: ["$lineas_items.cantidad",
"$lineas_items.precio"] } },
        total_items_vendidos: { $sum: "$lineas_items.cantidad" }
    }},
    // 4. Ordenar descendentemente
    { $sort: { volumen_total_usd: -1 } }
]);
```

32.6 Procesamiento Stream-First: Apache Flink Internals

Mientras Apache Spark Structured Streaming procesa eventos agrupándolos en pequeños intervalos temporales (*micro-lotes de 100 ms a 1 segundo*), **Apache Flink** está diseñado para el **procesamiento continuo evento a evento nativo con latencias inferiores a 10 milisegundos**:



1. Estado con Clave (*Keyed State*) y RocksDB StateBackend

En Flink, el cálculo de ventanas deslizantes y agregaciones no se recalcula desde cero: se almacena en memoria o en disco local gestionado por **RocksDB** (una base de datos embebida LSM-Tree en C++). Esto permite manejar terabytes de estado activo en memoria sin sufrir las pausas de

recolección de basura (*Garbage Collection*) de la JVM.

2. Tolerancia a Fallos: Barreras de Checkpoint (Chandy-Lamport)

Flink implementa el algoritmo de **instantáneas asíncronas con barreras (*Asynchronous Barrier Snapshotting*)**:

1. El coordinador inyecta marcas especiales (*Checkpoint Barriers*) en el flujo de datos de Kafka.
2. Cada operador procesa los datos y, al recibir la barrera, vuelca su estado local a almacenamiento de objetos persistente (S3/ADLS) de forma asíncrona sin detener el procesamiento de nuevos eventos.
3. Si un nodo colapsa, todos los operadores retroceden al último checkpoint validado y reanudan la lectura de Kafka desde ese offset exacto, logrando semántica **Exactly-Once estricta**.

3. Flink vs. Spark Structured Streaming: ¿Cuándo elegir cuál?

- **Elija Apache Flink si:** El requerimiento de negocio exige latencias de un solo dígito de milisegundo (detección de fraude en transacciones financieras bancarias, control de tráfico aéreo, alertas de monitoreo médico), procesamiento de eventos complejos (CEP) o ventanas de sesión dinámicas muy desordenadas.
- **Elija Spark Structured Streaming si:** La plataforma prioriza la unificación arquitectónica del código (mismo motor y API para batch nocturno, capas Medallón en Delta Lake y micro-lotes de streaming) y latencias de 500 ms a 2 segundos son plenamente aceptables para el caso de negocio.

32.7 Motores OLAP Especializados: Trino y ClickHouse

En plataformas maduras, el Lakehouse almacena la verdad histórica, pero las consultas interactivas de dashboards con miles de usuarios concurrentes requieren motores de consulta especializados:

1. Trino (anteriormente PrestoSQL): Cómputo Federado y Consultas Ad-Hoc

Trino es un motor de ejecución SQL distribuido masivamente paralelo (MPP) que **no almacena datos propios**:

- **Federación de Consultas (*Query Federation*):** Permite cruzar en una sola sentencia SQL una tabla de 100 TB en Delta Lake (S3), con una tabla operacional en PostgreSQL y un topic en Kafka:

```sql
-- Consulta federada unificada en Trino
SELECT
    lake.transaccion_id,
    lake.monto_usd,
    pg.nombre_cliente,
    pg.scoring_crediticio
FROM delta_lakehouse.silver.ventas AS lake
JOIN postgres_crm.public.clientes AS pg
  ON lake.cliente_id = pg.cliente_id
WHERE lake.fecha = CURRENT_DATE;
```

- **Ejecución en Memoria Pura:** Trino canaliza los datos entre etapas a través de la red sin volcarlos a disco intermedio, logrando que consultas analíticas pesadas retornen en segundos.

2. ClickHouse: Rendimiento Columnar Extremo para Series Temporales y Telemetría

ClickHouse es un sistema gestor de bases de datos relacional columnar enfocado en analítica en tiempo real (OLAP):

- **Motor MergeTree:** Almacena los datos ordenados físicamente por clave primaria mediante un índice disperso (*Sparse Index*), permitiendo omitir el 99% de los bloques de disco en consultas con rangos temporales.
- **Vectorización SIMD en C++:** Las operaciones de filtrado y agregación se compilan directamente a instrucciones vectoriales de CPU, permitiendo escanear cientos de millones de filas por segundo por núcleo.
- **Casos de Uso Ideales:** Almacenamiento y consulta de telemetría de clústeres, logs de auditoría de red, análisis de tráfico web y métricas de IoT con compresión de hasta 10:1 frente a JSON.

32.8 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Tratar a Cassandra o DynamoDB como una Base de Datos Relacional
Diseñar tablas en Cassandra e intentar consultar utilizando filtros sobre columnas que no forman parte de la clave primaria mediante `ALLOW FILTERING` es una catástrofe de rendimiento. `ALLOW FILTERING` fuerza a Cassandra a escanear todos los nodos del anillo completo, degradando la disponibilidad del clúster.
Corrección: Modele sus tablas alrededor de los requerimientos de lectura; si necesita una consulta distinta, cree una tabla secundaria desnormalizada o una vista materializada.

Anti-patrón 2: Usar Redis sin Políticas de Desalojo de Memoria (*Eviction Policies*)
Utilizar Redis como caché sin configurar límites máximos de memoria (`maxmemory`) y políticas de desalojo (como `volatile-lru` o `allkeys-lru`) provoca que el servidor colapse por agotamiento de RAM cuando la carga de datos se incrementa imprevistamente.

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 32.1 (Cálculo de Consistencia en Clúster de Cassandra):
Se tiene un clúster de Apache Cassandra desplegado en AWS con un factor de replicación $N = 3$ distribuido en tres zonas de disponibilidad.

1. Si un pipeline de ingesta escribe transacciones con nivel de consistencia `ONE`, ¿qué nivel de consistencia debe emplear el consumidor de lectura para garantizar que siempre lea el valor más reciente?
2. Si la red entre dos zonas de disponibilidad sufre una partición, ¿qué ocurre si el escritor intenta utilizar `ALL`?

Solución 32.1:

1. Para garantizar lecturas consistentes de la última escritura, se debe cumplir $R + W > N$. Si $W = 1$ y $N = 3$, el nivel de lectura debe satisfacer:

$$R + 1 > 3 \implies R > 2 \implies R = 3 \quad (\text{Nivel } \mathbf{ALL})$$

   Exigir `READ = ALL` significa que si un solo nodo se retrasa o cae, la lectura fallará, sacrificando disponibilidad. Por esta razón, la industria prefiere configurar `WRITE = QUORUM` (2) y `READ = QUORUM` (2), donde $2 + 2 = 4 > 3$.
2. Si el escritor exige `ALL`, requiere confirmación unánime de los 3 nodos. Ante una partición de red que aísle un solo nodo, la escritura **fallará inmediatamente con excepción** `WriteTimeoutException`, priorizando consistencia sobre disponibilidad (comportamiento CP).

32.9 Resumen y Puntos Clave

- **Persistencia políglota:** Cada motor NoSQL resuelve compensaciones específicas de escala, latencia y modelo de datos que las bases de datos relacionales tradicionales no pueden cubrir.
- **Teorema PACELC:** Clasifica los sistemas en función de su comportamiento tanto en partición de red como en operación normal de latencia.

- **Redis en memoria:** Estructuras probabilísticas como HyperLogLog optimizan cómputos analíticos de conteo masivo con solo 12 KB de RAM.
- **Modelado en Cassandra:** Diseñado en función estricta de las consultas de lectura, utilizando la consistencia ajustable ($R + W > N$) para balancear rendimiento y garantías ACID.
- **Flink y Kafka Connect:** Habilitan arquitecturas de eventos en tiempo real puro con latencias sub-milisegundas y semántica exactly-once mediante transacciones de dos fases.

Lecturas Recomendadas

1. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications (Chapter 3: Storage and Retrieval).* O'Reilly Media.
2. **Abadi, D. J. (2012).** *Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story.* IEEE Computer.
3. **Hewitt, E. & Carpenter, J. (2020).** *Cassandra: The Definitive Guide (3rd Edition).* O'Reilly Media.
4. **Carbone, P. et al. (2015).** *Apache Flink: Stream and Batch Processing in a Single Engine.* IEEE Data Engineering Bulletin.

CAPÍTULO 33

Ingeniería de Datos para Inteligencia Artificial y LLMs AVANZADO

> *"En la era de los Grandes Modelos de Lenguaje (LLMs) y la Inteligencia Artificial generativa, la calidad y relevancia del modelo no están determinadas por su cantidad de parámetros, sino por la solidez de los pipelines de datos que alimentan su contexto. El ingeniero de datos es el arquitecto indispensable que transforma terabytes de documentos no estructurados en incrustaciones vectoriales limpias, gobierna las características analíticas en tiempo real y construye la capa semántica que conecta a la IA con los datos del negocio."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la evolución del rol de Data Engineer hacia el soporte de arquitecturas de IA generativa y aprendizaje automático moderno (2026).
- Diseñar pipelines de datos de extremo a extremo para arquitecturas **RAG** (*Retrieval-Augmented Generation*), dominando estrategias de fragmentación (*chunking*) y enriquecimiento de metadatos.
- Ejecutar la vectorización de texto a gran escala mediante **embeddings** calculados en clústeres distribuidos de Apache Spark.
- Evaluar y operar **bases de datos vectoriales** (Qdrant, Pinecone, Milvus) e índices relacionales basados en similitud utilizando la extensión `pgvector` de PostgreSQL.
- Implementar **Feature Stores** (Almacenes de Características) con **Feast (LF AI & Data Foundation)**, distinguiendo el plano offline (entrenamiento histórico) del plano online (inferencia en tiempo real de baja latencia) con corrección temporal (*Point-in-Time Joins*).
- Dominar los principios de **Data Mesh**, productos de datos (*Data Products*) y la integración de la **Capa Semántica** (*dbt Semantic Layer*) con herramientas de inteligencia de negocios.

Definición 33.1 (Retrieval-Augmented Generation - RAG) RAG es el patrón arquitectónico que optimiza la generación de un modelo de lenguaje (LLM) recuperando dinámicamente información veraz y actualizada desde una base de conocimiento externa indexada vectorialmente, inyectando dichos fragmentos en el contexto de la consulta (*prompt*) antes de emitir la respuesta.

En términos de ingeniería de datos, RAG es un **pipeline continuo de ETL no estructurado**: extrae documentos, los limpia, los fragmenta, calcula representaciones numéricas densas (vectores) y los sincroniza en almacenes de baja latencia con garantías de frescura y control de acceso.

33.1 Arquitectura de Pipelines para RAG

A diferencia de los pipelines tradicionales orientados a tablas columnares, un pipeline de RAG opera sobre datos textuales y no estructurados:



1. Estrategias de Fragmentación (*Chunking*)

Dividir un texto extenso en fragmentos requiere equilibrar especificidad semántica y contexto:

- **Fragmentación por Tamaño Fijo (*Fixed-size Chunking*):** Divide por un número exacto de palabras o tokens (ej. 512 tokens) con un porcentaje de solapamiento (*overlap*, típicamente 10% a 20%) para evitar que oraciones críticas queden cortadas entre dos fragmentos contiguos.
- **Fragmentación Estructural Semántica:** Respeta los límites jerárquicos del documento (secciones Markdown, encabezados H1/H2, párrafos completos o bloques de código), garantizando que las ideas complejas permanezcan unificadas.

2. Enriquecimiento de Metadatos

Cada fragmento indexado debe incluir metadatos estructurados (`documento_id`, `fecha_creacion`, `version`, `rol_minimo_requerido`). Esto permite aplicar **filtrado híbrido** antes o después de la búsqueda vectorial (*Pre-filtering / Post-filtering*), evitando que un usuario consulte información confidencial para la cual no tiene autorización.

33.2 Generación de Embeddings a Gran Escala con Apache Spark

Un **embedding** es una función que proyecta un fragmento de texto en un vector denso de números reales en un espacio de alta dimensionalidad (típicamente entre 384 y 1.536 dimensiones), donde la proximidad geométrica refleja afinidad semántica.

En pipelines con millones de documentos, la vectorización no puede realizarse de forma secuencial en una sola máquina. Se utiliza Apache Spark con **Pandas UDFs vectorizadas con aceleración GPU/CPU**:

PYTHON

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import pandas_udf
import pandas as pd
from typing import Iterator

spark = SparkSession.builder \
    .appName("Vectorizacion-Masiva-Embeddings") \
    .getOrCreate()

# Definición de Pandas UDF en modo iterador para reutilizar el modelo en memoria
@pandas_udf("array<float>")
def generar_embeddings_lote(textos_iter: Iterator[pd.Series]) ->
Iterator[pd.Series]:
    # El modelo se carga UNA SOLA VEZ por ejecutor de Spark, amortizando el costo
    from sentence_transformers import SentenceTransformer
    modelo = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

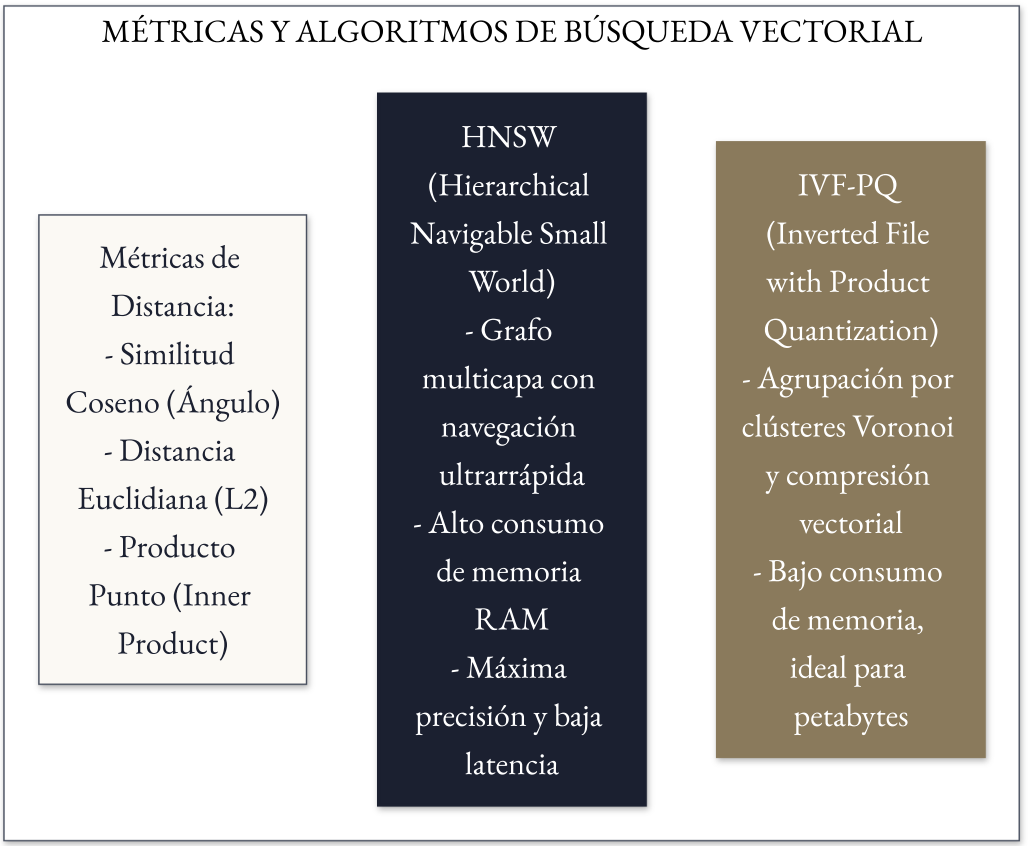
    for lote_textos in textos_iter:
        # Codificación por lotes en memoria local del ejecutor
        vectores = modelo.encode(lote_textos.tolist(), batch_size=64,
show_progress_bar=False)
        yield pd.Series(vectores.tolist())

# Aplicación distribuida sobre millones de fragmentos
df_fragmentos = spark.read.parquet("data/lakehouse/silver_documentos_chunks/")
df_con_vectores = df_fragmentos.withColumn("vector_embedding",
generar_embeddings_lote("contenido_texto"))

df_con_vectores.write.format("delta").mode("append").save("data/lakehouse/gold_co
nocimiento_vectorizado/")
```

33.3 Bases de Datos Vectoriales e Indexación con `pgvector`

Para responder consultas en milisegundos sobre millones de vectores, los motores no calculan la distancia euclidiana o de coseno contra la totalidad de la base ($O(N)$ o fuerza bruta). Utilizan algoritmos de **Búsqueda del Vecino Más Cercano Aproximado** (*Approximate Nearest Neighbor - ANN*):



Implementación en PostgreSQL con `pgvector`

Para organizaciones que ya operan bases de datos relacionales, la extensión **`pgvector`** permite almacenar y consultar vectores directamente junto con las tablas relacionales existentes mediante SQL estándar:

```sql
-- Habilitar extensión en PostgreSQL
CREATE EXTENSION IF NOT EXISTS vector;

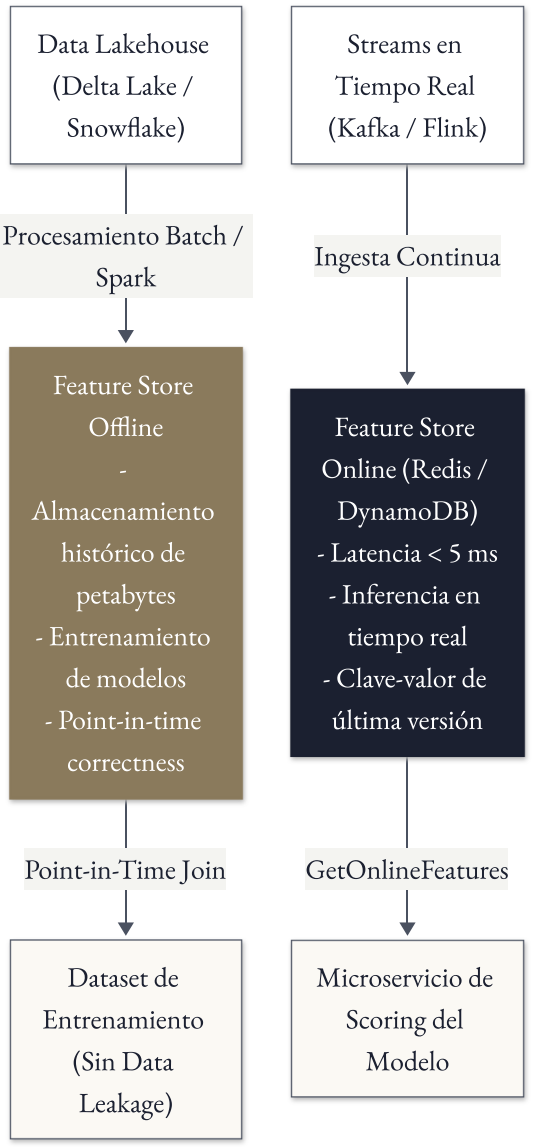
-- Creación de tabla con columna vectorial de 1536 dimensiones
CREATE TABLE base_conocimiento.documentos_embeddings (
    documento_id BIGSERIAL PRIMARY KEY,
    titulo TEXT NOT NULL,
    contenido TEXT NOT NULL,
    metadatos JSONB,
    embedding VECTOR(1536)
);

-- Creación de índice HNSW para búsqueda por similitud de coseno en sub-milisegundos
CREATE INDEX idx_documentos_hnsw ON base_conocimiento.documentos_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Consulta de los 5 fragmentos más cercanos semánticamente al vector de la consulta
SELECT
    documento_id,
    titulo,
    contenido,
    1 - (embedding <=> '[0.015, -0.042, ... , 0.089]') AS similitud_coseno
FROM base_conocimiento.documentos_embeddings
WHERE metadatos->>'departamento' = 'Finanzas'
ORDER BY embedding <=> '[0.015, -0.042, ... , 0.089]' ASC
LIMIT 5;
```

33.4 Feature Stores: Almacenes de Características para Machine Learning

Un **Feature Store** centraliza el cálculo, almacenamiento y servicio de características analíticas (*features*) para modelos de ML, erradicando la duplicación de código y el desfase entre entrenamiento e inferencia:



1. Corrección Temporal (*Point-in-Time Correctness / As-Of Joins*)

El principal peligro al entrenar modelos predictivos es la **fuga de datos hacia el futuro (*Data Leakage*)**: utilizar el valor de una característica en una fecha posterior a la ocurrencia del evento que se intenta predecir. El Feature Store garantiza que al reconstruir el conjunto de entrenamiento para una transacción ocurrida el `2025-06-15 14:30:00`, se unan exactamente las características vigentes a esa microsegunda exacta (*As-Of Join*).

2. Implementación con Feast (LF AI & Data Foundation)

PYTHON

```python
# Definición declarativa de características con Feast (features.py)
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64
from datetime import timedelta

cliente = Entity(name="cliente_id", join_keys=["cliente_id"])

origen_batch = FileSource(
    path="data/lakehouse/silver_features_clientes.parquet",
    timestamp_field="fecha_calculo",
    created_timestamp_field="fecha_creacion"
)

vista_features_cliente = FeatureView(
    name="caracteristicas_comportamiento_cliente",
    entities=[cliente],
    ttl=timedelta(days=30),
    schema=[
        Field(name="promedio_compras_30d", dtype=Float32),
        Field(name="cantidad_transacciones_rechazadas", dtype=Int64),
    ],
    online=True,  # Sincroniza automáticamente hacia Redis para inferencia rápida
    source=origen_batch,
)
```

33.5 Data Mesh, Productos de Datos y la Capa Semántica

Para organizaciones a escala empresarial, centralizar todo el procesamiento analítico en un único equipo de datos monolítico genera un cuello de botella organizacional. **Data Mesh** (formulado por Zhamak Dehghani) traslada la responsabilidad hacia dominios de negocio descentralizados mediante cuatro principios:

1. **Propiedad Orientada al Dominio (*Domain-Driven Ownership*):** Cada área de negocio (Ventas, Logística, Marketing) es dueña y responsable de sus propios datos.
2. **Los Datos como Producto (*Data as a Product*):** Los datos no son un subproducto residual; se exponen como productos con contratos de datos estrictos, documentación, SLAs de frescura y compatibilidad garantizada.
3. **Plataforma de Datos Autoservicio (*Self-serve Data Platform*):** El equipo central de infraestructura provee herramientas automatizadas (Terraform, plantillas de Spark, catálogos) para que los dominios construyan sus pipelines sin depender de un intermediario.
4. **Gobernanza Federada Computacional (*Federated Governance*):** Políticas globales de seguridad, cifrado y privacidad aplicadas automáticamente mediante código en toda la

organización.

La Capa Semántica (*Semantic Layer*)

Históricamente, el equipo de finanzas calculaba la *"Facturación Neta"* con una fórmula SQL, mientras que marketing la calculaba de forma distinta en un reporte de Power BI, provocando juntas directivas donde nadie confiaba en las cifras.

La **Capa Semántica** (implementada con herramientas como **dbt Semantic Layer** o **Cube.js**) centraliza las definiciones matemáticas de las métricas de negocio en un repositorio de código único, permitiendo que LLMs, dashboards de Power BI, Tableau y notebooks consuman exactamente la misma verdad matemática:

YAML

```yaml
# Definición de Métrica en dbt Semantic Layer
(models/metrics/metricas_ventas.yml)
version: 2

metrics:
  - name: facturacion_neta_usd
    label: "Facturación Neta Consolidada (USD)"
    description: "Suma total de transacciones completadas deduciendo impuestos y
descuentos."
    type: simple
    type_params:
      measure: monto_neto_transaccion
    filter: |
      estado_pago = 'COMPLETADO'
```

33.6 MLOps con MLflow y Evaluación de Pipelines RAG en Producción

El rol del Data Engineer en inteligencia artificial no culmina al escribir vectores en una base de datos: abarca garantizar la gobernanza del ciclo de vida del modelo (**MLOps**) y auditar de forma matemática la veracidad de los sistemas generativos (**Evaluación de RAG**).

1. MLOps con MLflow: Trazabilidad, Artefactos y Registro

MLflow es el estándar de código abierto (desarrollado por Databricks y parte de la Linux Foundation) para gobernar el ciclo de vida del aprendizaje automático:

- **MLflow Tracking:** Registra hiperparámetros, métricas de rendimiento y dependencias de entorno asociadas a cada corrida de entrenamiento.

- **Linaje de Datos y Modelos:** En un entorno Lakehouse, cada ejecución de entrenamiento vincula el ID del modelo con la versión exacta de la tabla Delta utilizada (`VERSION AS OF 42`), garantizando reproducibilidad forense absoluta.
- **MLflow Model Registry:** Repositorio centralizado para gobernar el versionado y la promoción de modelos entre etapas (*Staging* → *Production* → *Archived*):

PYTHON

```python
# pipelines/entrenamiento_y_registro_mlflow.py
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

mlflow.set_experiment("/Produccion/Scoring_Fraude_Clientes")

with mlflow.start_run(run_name="rf_modelo_v2") as run:
    # 1. Registrar metadatos de linaje de datos de entrada
    mlflow.set_tag("fuente_datos_delta", "lakehouse/gold/dim_clientes_scd2")
    mlflow.set_tag("delta_version_as_of", "15")

    # 2. Entrenar modelo
    params = {"n_estimators": 100, "max_depth": 10, "random_state": 42}
    mlflow.log_params(params)

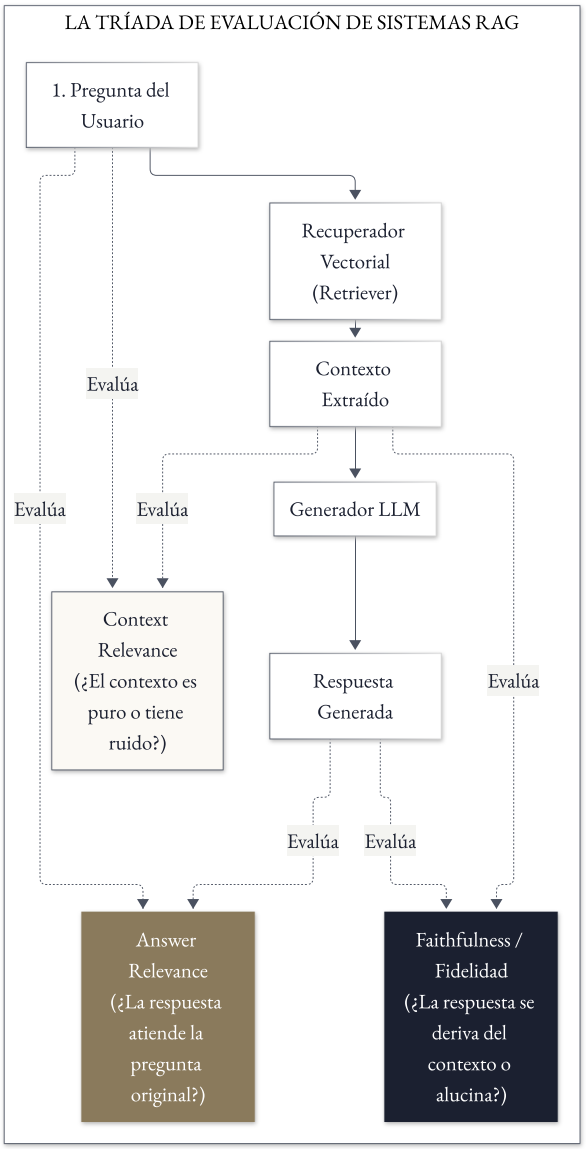
    modelo = RandomForestClassifier(**params)
    modelo.fit(X_train, y_train)

    # 3. Registrar métricas de desempeño
    score_roc = 0.942
    mlflow.log_metric("roc_auc", score_roc)

    # 4. Registrar artefacto del modelo en el Registro Central
    mlflow.sklearn.log_model(
        sk_model=modelo,
        artifact_path="modelo_fraude",
        registered_model_name="DetectorFraudeCorporativo"
    )
```

2. Evaluación Formal de Pipelines RAG: La Tríada Ragas

Un pipeline tradicional de software se valida con pruebas unitarias deterministas (`assert resultado == esperado`). Un sistema RAG genera lenguaje natural no estructurado, por lo que su validación exige métricas declarativas especializadas:



1. **Fidelidad (*Faithfulness*):** Mide si todas las afirmaciones presentes en la respuesta generada pueden inferirse directamente del contexto recuperado. Una baja fidelidad indica **alucinación** del modelo de lenguaje.
2. **Relevancia del Contexto (*Context Relevance*):** Mide la relación señal/ruido del motor de búsqueda vectorial. Si se recuperan 5 fragmentos de texto pero solo uno contiene la respuesta, la relevancia es baja, aumentando costos de inferencia y confundiendo al modelo.
3. **Relevancia de la Respuesta (*Answer Relevance*):** Mide si la respuesta generada responde directamente la inquietud del usuario, sin divagaciones ni omisiones de información crítica.

Implementación del Pipeline de Evaluación (LLM-as-a-Judge)

En producción, se implementa un pipeline por lotes en Apache Spark que somete una muestra aleatoria de consultas del día a un modelo evaluador con rúbricas estrictas:

```python
# pipelines/evaluacion_rag_batch.py
from pyspark.sql import functions as F

def auditar_calidad_rag_batch(spark, ruta_telemetria_rag: str) -> None:
    df_logs_rag = spark.read.format("delta").load(ruta_telemetria_rag)

    # Prompt formal para evaluación con LLM-as-a-Judge
    prompt_fidelidad = """
    Analice el siguiente CONTEXTO y la RESPUESTA GENERADA.
    Determine si cada afirmación de la respuesta está respaldada por el contexto.
    Responda únicamente con un número decimal entre 0.0 (alucinación total) y 1.0
(100% veraz).
    Contexto: {contexto}
    Respuesta: {respuesta}
    """

    # En un clúster distribuido, se evalúa mediante Pandas UDF conectada a un
endpoint local
    print("Ejecutando auditoría de fidelidad sobre el lote de inferencias
RAG...")
    # Si la fidelidad promedio cae por debajo de 0.85, el pipeline emite una
alerta a Slack/PagerDuty
```

33.7 Criterios de Fiabilidad y Anti-Patrones en Producción

Anti-patrón 1: Cargar Modelos de Deep Learning Dentro de Bucles de Spark Fila a Fila
Invocar la carga de un modelo de PyTorch/TensorFlow dentro de una función regular `df.rdd.map(...)` provoca que Spark instancie el modelo de gigabytes una vez por cada fila de datos, provocando fallos inmediatos por agotamiento de memoria (*OOM*).
Corrección: Utilice Pandas UDFs con iteradores (`Iterator[pd.Series]`) para que el modelo se cargue exactamente una sola vez en la memoria del ejecutor para miles de filas.

Anti-patrón 2: Ausencia de Detección de Deriva de Datos (*Data Drift*)
Desplegar un modelo de scoring sin monitorear si la distribución estadística de las características de entrada ha cambiado con el tiempo provoca que las predicciones del modelo se degraden silenciosamente (*Model Decay*).

Preguntas de Revisión y Ejercicios Prácticos

Ejercicio 33.1 (Resolución de Point-in-Time Join para Prevención de Data Leakage):
Se tiene una tabla de transacciones etiquetadas para fraude: `fraudes(tx_id, cliente_id, timestamp_tx, es_fraude)`.
Se tiene un histórico de características calculado diariamente: `features(cliente_id, fecha_calculo, score_riesgo_acumulado)`.
Explique por qué hacer un `JOIN` tradicional por `cliente_id` es erróneo y cómo opera un *As-Of Join*.

Solución 33.1:

- Un `JOIN` relacional clásico uniría múltiples versiones del `score_riesgo_acumulado` para el mismo cliente o, si se une con la última versión actual, filtraría información del futuro que el modelo no conocía en el momento en que ocurrió la transacción (fuga de datos / *Data Leakage*).
- Un **As-Of Join** (unión temporal con corrección puntual) une cada fila de `fraudes` con el registro de `features` que tenga la marca de tiempo más reciente **estrictamente menor o igual** al `timestamp_tx` de la transacción:

$$\text{Condición: } \max(\text{fecha_calculo}) \quad \text{tal que} \quad \text{fecha_calculo} \leq \text{timestamp_tx}$$

Esto asegura que el dataset de entrenamiento refleje con exactitud matemática el estado real del mundo en el milisegundo exacto de la decisión.

33.8 Resumen y Puntos Clave

- **El Data Engineer es el cimiento de la IA:** RAG, agentes autónomos y modelos generativos exigen pipelines continuos de extracción, fragmentación y vectorización de alta disponibilidad.
- **Procesamiento de embeddings distribuido:** Las Pandas UDFs en Apache Spark permiten paralelizar la inferencia vectorial sobre millones de documentos con amortización de memoria del modelo.
- **Indexación ANN:** Algoritmos como HNSW e IVF-PQ permiten búsquedas de similitud sub-milisegundas sobre espacios vectoriales de alta dimensionalidad.
- **Feature Stores:** Desacoplan el almacenamiento masivo para entrenamiento offline del servicio de baja latencia online, garantizando consistencia temporal libre de fugas de datos.

- **Data Mesh y Capa Semántica:** Descentralizan la propiedad de los datos hacia dominios de negocio y unifican la definición matemática de las métricas para consumo consistente por analistas y modelos de IA.

Lecturas Recomendadas

1. **Dehghani, Z. (2022).** *Data Mesh: Delivering Data-Driven Value at Scale.* O'Reilly Media.
2. **Malkov, Y. A. & Yashunin, D. A. (2020).** *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs (HNSW).* IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 42(4), 824-836.
3. **LF AI & Data Foundation (2024).** *Feast: Feature Store for Machine Learning Architecture and Specification.* Linux Foundation (`https://feast.dev/`).
4. **Lewis, P. et al. (2020).** *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* Advances in Neural Information Processing Systems (NeurIPS).

PARTE XI

Proyecto Integrador y Plataforma de Producción

Capítulo 34 · Proyecto Integrador: Plataforma Lakehouse y Portafolio de Producción

NÚCLEO

CAPÍTULO 34

Proyecto Integrador: Plataforma Lakehouse y Portafolio de Producción

NÚCLEO

> *"El verdadero dominio de la ingeniería de datos no se demuestra recitando conceptos aislados, sino ensamblando una arquitectura completa donde cada componente —desde la ingesta cruda y el almacenamiento ACID hasta la orquestación, la calidad automatizada y el despliegue continuo— encaja con precisión milimétrica. Este capítulo presenta la estrategia integral de portafolio para conseguir empleo y el proyecto capstone de producción."*

Objetivos de Aprendizaje

Al finalizar este capítulo, el lector será capaz de:

- Comprender la estrategia de una **Tríada de Proyectos de Portafolio Graduales** (100% local y gratuito, ingesta de API REST, y streaming en tiempo real) que demuestran competencia técnica antes del capstone empresarial.
- Diseñar e implementar una plataforma analítica completa basada en la **Arquitectura Medallón** (Bronze → Silver → Gold) utilizando **Delta Lake**.
- Capturar metadatos de linaje mediante `_metadata.file_path` y marcas temporales deterministas.
- Aplicar deduplicación con `ROW_NUMBER()`, lógica trivaluada y segregación automática de anomalías a tablas de **Cuarentena**.
- Modelar dimensiones lentamente cambiantes (**SCD Tipo 2**) con claves subrogadas deterministas e idempotentes calculadas con `sha2`.
- Orquestar la ejecución automatizada de punta a punta con **Apache Airflow** mediante el patrón *Lightweight Conductor* y dependencias orientadas a datos (**Assets**).
- Validar la integridad de los datos con **Great Expectations 1.x** y aprovisionar infraestructura con **Terraform** y **GitHub Actions**.
- Configurar protocolos de **FinOps Personal** para prevenir facturas sorpresa en nubes públicas durante la experimentación.

34.1 La Tríada del Portafolio: Tres Proyectos Graduales para Conseguir Empleo

Un error frecuente de los aspirantes a Data Engineer es intentar construir un único repositorio monolítico gigantesco en la nube sin antes dominar los componentes modulares. Esto suele generar proyectos inacabados, configuraciones frágiles y riesgos financieros por costes imprevistos.

Los comités técnicos internacionales valoran una **tríada de tres proyectos especializados y concisos**, complementada por el proyecto integrador empresarial:



Proyecto de Portafolio 1: Modern Data Stack 100% Local (DuckDB + dbt Core + Airflow)

Este proyecto demuestra que dominas el modelado analítico sin gastar un solo centavo en plataformas cloud. Utiliza DuckDB como motor columnar in-process ultrarrápido, dbt Core para las transformaciones SQL y Apache Airflow (o Cosmos) para la programación.

Estructura del Repositorio

```
portfolio-duckdb-dbt-analytics/
├── orchestrator/
│   └── dags/
│       └── dag_duckdb_dbt.py         # Orquestación diaria de modelos
├── dbt_analytics/
│   ├── models/
│   │   ├── staging/
│   │   │   ├── stg_pedidos.sql       # Limpieza inicial y casteos
│   │   │   └── schema.yml            # Tests de unicidad y no-nulos
│   │   └── marts/
│   │       ├── dim_clientes.sql      # Dimensión de clientes desnormalizada
│   │       ├── fact_ventas.sql       # Tabla de hechos al nivel de ítem
│   │       └── schema.yml            # Tests relacionales y de negocio
│   ├── dbt_project.yml
│   └── profiles.yml                  # Conexión local al archivo analytics.duckdb
├── data/                             # Archivos Parquet de origen
├── docker-compose.yml                # Airflow local en contenedor ligero
├── pyproject.toml
└── README.md
```

Modelo dbt de Hechos (`models/marts/fact_ventas.sql`)

SQL

```sql
{{ config(materialized='table') }}

WITH pedidos AS (
    SELECT * FROM {{ ref('stg_pedidos') }}
),
clientes AS (
    SELECT * FROM {{ ref('dim_clientes') }}
)
SELECT
    p.pedido_id,
    p.fecha_pedido,
    p.cliente_id,
    c.pais_iso,
    p.categoria_producto,
    p.cantidad,
    p.precio_unitario,
    ROUND(p.cantidad * p.precio_unitario, 2) AS monto_total_usd
FROM pedidos AS p
INNER JOIN clientes AS c ON p.cliente_id = c.cliente_id
WHERE p.estado = 'COMPLETADO'
```

Modelo de `README.md` de Alto Impacto

```
MARKDOWN
```

```
# Local Modern Data Stack Analytics Pipeline (DuckDB + dbt + Airflow)
[![CI/CD](https://github.com/usuario/repo/actions/workflows/ci.yml/badge.svg)]
(https://github.com)
[![dbt](https://img.shields.io/badge/dbt-Core%201.9-orange)](https://getdbt.com)
[![DuckDB](https://img.shields.io/badge/Engine-DuckDB-yellow)]
(https://duckdb.org)

## Descripción Ejecutiva
Pipeline analítico reproducible de extremo a extremo que procesa 100.000
transacciones diarias de e-commerce mediante DuckDB y dbt Core, orquestado
localmente con Apache Airflow. El proyecto modela un esquema en estrella
(Kimball) y ejecuta 18 pruebas de integridad automáticas en menos de 4 segundos
sin costo de infraestructura cloud.

## Ejecución en Un Solo Paso
```bash
git clone https://github.com/usuario/portfolio-duckdb-dbt-analytics.git
cd portfolio-duckdb-dbt-analytics
docker compose up -d
```

Proyecto de Portafolio 2: Ingesta Resiliente de APIs con Pydantic y Delta
Lake Local

Este proyecto demuestra rigor en ingeniería de software: interacción HTTP con
APIs externas, manejo de desconexiones de red, contratos de datos estrictos en
Pydantic y almacenamiento incremental en formato Delta Lake local.

Estructura del Repositorio
```

portfolio-resilient-api-ingestion/ ├── src/ │ ├── client.py # Cliente HTTP con Exponential Backoff y Jitter │ ├── contracts.py # Modelos Pydantic V2 para validación │ └── pipeline.py # Ingesta y escritura en Delta Lake local ├── tests/ │ ├── test_client.py # Mocks de respuestas HTTP (200, 429, 503) │ └── test_contracts.py # Pruebas de validación de esquemas ├── pyproject.toml # Dependencias gestionadas con uv └── README.md

```
#### Cliente HTTP Resiliente (`src/client.py`)
```python
import time, random, requests
from typing import Dict, Any, List

class ResilientApiClient:
    def __init__(self, base_url: str, max_retries: int = 4):
        self.base_url = base_url
        self.max_retries = max_retries
```
```

```
    def fetch_data(self, endpoint: str) -> List[Dict[str, Any]]:
        url = f"{self.base_url}/{endpoint}"
        for attempt in range(self.max_retries):
            try:
                response = requests.get(url, timeout=5.0)
                if response.status_code == 200:
                    return response.json()
                elif response.status_code == 429: # Rate Limit
                    wait = (2 ** attempt) + random.uniform(0.1, 1.0)
                    time.sleep(wait)
                elif response.status_code >= 500: # Error transitorio del
servidor
                    time.sleep(2 ** attempt)
                else:
                    response.raise_for_status()
            except requests.RequestException:
                if attempt == self.max_retries - 1:
                    raise
                time.sleep(2 ** attempt)
        raise RuntimeError(f"Fallo definitivo al consultar {url} tras
{self.max_retries} intentos.")
```

Proyecto de Portafolio 3: Streaming en Tiempo Real con Kafka y Spark Structured Streaming

Este proyecto demuestra el dominio del cómputo en streaming: consumo continuo de topics de mensajería (Kafka o Redpanda), ventanas deslizantes (*sliding windows*), marcas de agua (*watermarking*) para manejar eventos tardíos y semántica *at-least-once* con persistencia transaccional en Delta Lake.

PYTHON

```python
# src/streaming_processor.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("ClickstreamRealTimeProcessor") \
    .config("spark.sql.shuffle.partitions", "2") \
    .getOrCreate()

# 1. Lectura de flujo de Kafka
df_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "eventos_clickstream") \
    .option("startingOffsets", "latest") \
    .load()

# 2. Parsing y ventana deslizante con marca de agua (Watermark de 10 minutos)
df_eventos = df_stream.selectExpr("CAST(value AS STRING) AS json_payload") \
```

```
    .select(F.from_json("json_payload", "usuario_id STRING, url STRING, ts
TIMESTAMP").alias("data")) \
    .select("data.*") \
    .withWatermark("ts", "10 minutes")

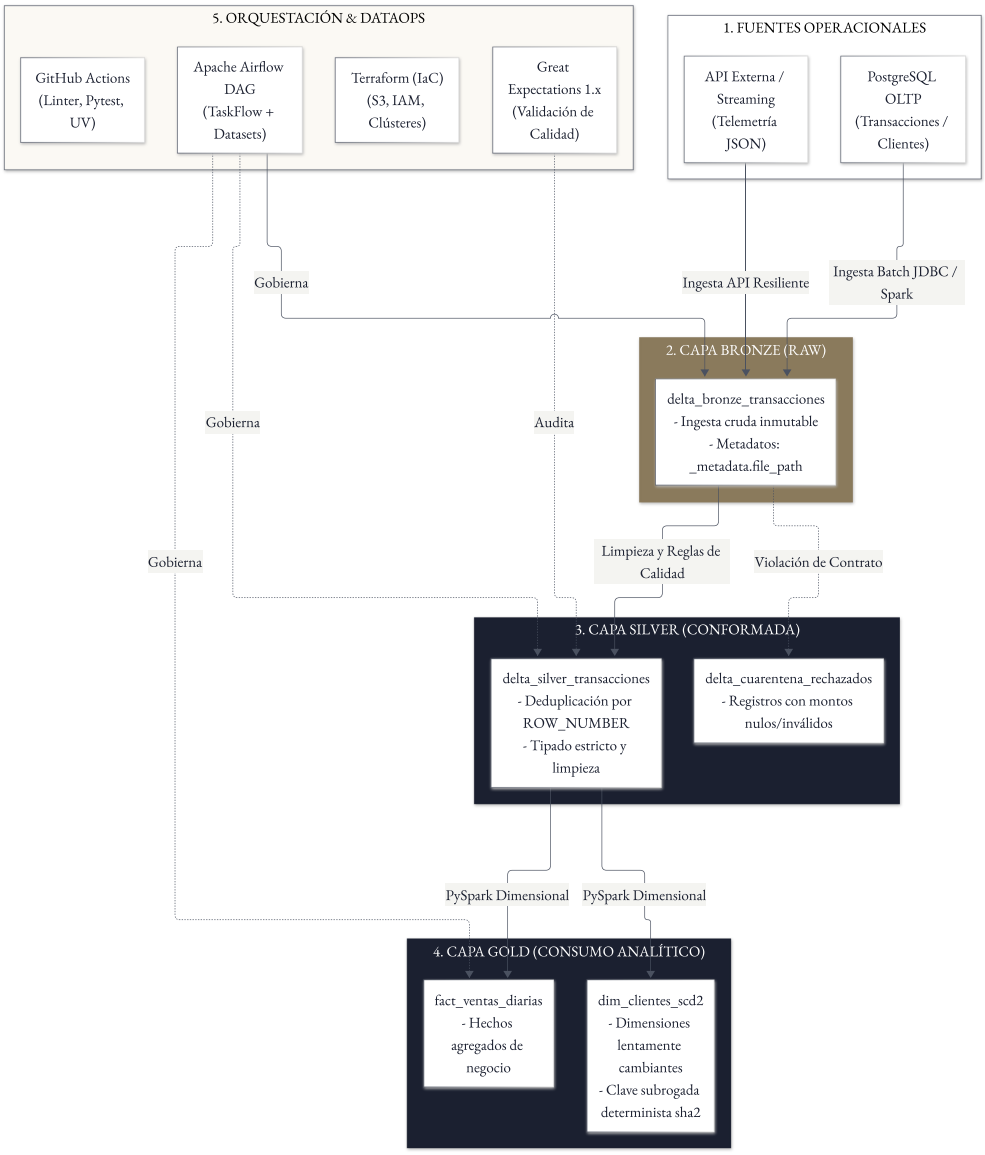
# 3. Agregación en ventana de 5 minutos deslizada cada 1 minuto
df_agregado = df_eventos.groupBy(
    F.window("ts", "5 minutes", "1 minute"),
    F.col("url")
).count()

# 4. Escritura en Delta Lake con checkpointing
query = df_agregado.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "data/checkpoints/clickstream") \
    .start("data/gold/metricas_clickstream_tiempo_real")

query.awaitTermination(60)
```

34.2 Proyecto 4 (Capstone Empresarial): La Plataforma Lakehouse de Producción

Una vez consolidados los tres proyectos anteriores, el **Capstone Empresarial** integra todos los componentes en una solución completa para corporaciones:



34.3 Paso 1: Ingesta Inmutable a Capa Bronze (Delta Lake)

La capa Bronze almacena los datos en su formato original, inyectando metadatos de auditoría y capturando la ruta de archivo mediante la columna oculta estándar `_metadata.file_path`:

PYTHON

```python
# src/pipelines/ingesta_bronze.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

def ejecutar_ingesta_bronze(spark: SparkSession, ruta_fuente: str, ruta_bronze:
str) -> None:
    print(f"Cargando datos crudos desde {ruta_fuente}...")
    df_raw = spark.read.parquet(ruta_fuente)

    # Inyección de metadatos de auditoría compatibles con Unity Catalog y Spark
moderno
    df_bronze = df_raw \
        .withColumn("_metadata_ingesta_ts", F.current_timestamp()) \
        .withColumn("_metadata_archivo_origen", F.col("_metadata.file_path")) \
        .withColumn("fecha_ingesta", F.current_date())

    print(f"Escribiendo en capa Bronze Delta Lake: {ruta_bronze}...")
    df_bronze.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("fecha_ingesta") \
        .save(ruta_bronze)
    print("Ingesta Bronze completada exitosamente.")
```

34.4 Paso 2: Limpieza, Deduplicación y Cuarentena en Capa Silver

En la capa Silver aplicamos deduplicación determinista mediante `ROW_NUMBER()` y segregamos anomalías a la tabla de **Cuarentena** con evaluación estricta de valores `NULL`:

PYTHON

```python
# src/pipelines/bronze_a_silver.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

def procesar_capa_silver(spark: SparkSession, ruta_bronze: str, ruta_silver: str,
ruta_cuarentena: str) -> None:
    df_bronze = spark.read.format("delta").load(ruta_bronze)

    # 1. Deduplicación analítica conservando la versión más reciente
    ventana_dedup = Window \
        .partitionBy("transaccion_id") \
        .orderBy(F.col("_metadata_ingesta_ts").desc())

    df_deduplicado = df_bronze \
        .withColumn("fila_num", F.row_number().over(ventana_dedup)) \
        .filter(F.col("fila_num") == 1) \
        .drop("fila_num")
```

```
    # 2. Segregación de Calidad: Condición de validez de negocio
    condicion_valida = (
        F.col("monto_usd").isNotNull() &
        (F.col("monto_usd") > 0.0) &
        F.col("cliente_id").isNotNull() &
        F.col("estado_pago").isin(["COMPLETADO", "PENDIENTE", "CANCELADO"])
    )

    df_validos = df_deduplicado.filter(condicion_valida)
    df_rechazados = df_deduplicado.filter(~condicion_valida) \
        .withColumn("_motivo_rechazo", F.lit("Monto invalido, estado desconocido
o cliente nulo")) \
        .withColumn("_fecha_cuarentena", F.current_timestamp())

    # 3. Escritura atómica e idempotente
    df_validos.write.format("delta").mode("append").save(ruta_silver)
    df_rechazados.write.format("delta").mode("append").save(ruta_cuarentena)
    print("Capa Silver y Cuarentena procesadas con éxito.")
```

34.5 Paso 3: Modelado Dimensional Gold (SCD Tipo 2 y Hechos)

La capa Gold modela las dimensiones lentamente cambiantes (**SCD Tipo 2**) con **claves subrogadas deterministas** calculadas mediante `sha2` e independientes de la hora de ejecución del script:

PYTHON

```
# src/pipelines/silver_a_gold.py
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from delta.tables import DeltaTable

def sincronizar_dimension_clientes_scd2(spark: SparkSession,
ruta_clientes_novedades: str, ruta_dim_clientes: str) -> None:
    df_novedades = spark.read.parquet(ruta_clientes_novedades)

    # Determinismo absoluto: la fecha de inicio proviene del dato de negocio (no
de current_timestamp)
    col_fecha_negocio = F.coalesce(
        F.col("fecha_actualizacion").cast("timestamp") if "fecha_actualizacion"
in df_novedades.columns else F.col("fecha_registro").cast("timestamp"),
        F.to_timestamp(F.lit("2026-01-01 00:00:00"))
    )

    df_novedades_preparadas = df_novedades \
        .withColumn("valido_desde", col_fecha_negocio) \
        .withColumn("valido_hasta", F.lit(None).cast("timestamp")) \
        .withColumn("es_actual", F.lit(True)) \
        .withColumn("cliente_sk", F.sha2(F.concat_ws("||",
```

```
F.col("cliente_id").cast("string"), F.date_format(F.col("valido_desde"), "yyyy-
MM-dd HH:mm:ss")), 256))

    if not DeltaTable.isDeltaTable(spark, ruta_dim_clientes):
        df_novedades_preparadas.write.format("delta").save(ruta_dim_clientes)
        print("Dimensión Clientes SCD2 inicializada con carga histórica.")
        return

    tabla_dim = DeltaTable.forPath(spark, ruta_dim_clientes)
    df_actuales = tabla_dim.toDF().filter(F.col("es_actual") == True)

    # 1. Identificar registros que efectivamente mutaron (correo o segmento)
    df_cambios = df_novedades_preparadas.alias("nov").join(
        df_actuales.alias("dim"),
        on="cliente_id",
        how="inner"
    ).filter(
        (F.col("dim.correo") != F.col("nov.correo")) |
        (F.col("dim.segmento") != F.col("nov.segmento"))
    ).select("nov.*")

    # 2. Identificar registros totalmente nuevos
    df_nuevos = df_novedades_preparadas.alias("nov").join(
        df_actuales.alias("dim"),
        on="cliente_id",
        how="left_anti"
    ).select("nov.*")

    df_a_insertar = df_cambios.unionByName(df_nuevos)

    # 3. Cerrar la versión previa de los registros modificados
    tabla_dim.alias("dim").merge(
        source=df_cambios.alias("cambio"),
        condition="dim.cliente_id = cambio.cliente_id AND dim.es_actual = true"
    ).whenMatchedUpdate(set={
        "valido_hasta": "cambio.valido_desde",
        "es_actual": "false"
    }).execute()

    # 4. Insertar únicamente las filas necesarias
    df_a_insertar.write.format("delta").mode("append").save(ruta_dim_clientes)
    print("Dimensión Clientes SCD2 sincronizada de forma determinista e
idempotente.")

def construir_fact_ventas_diarias(spark: SparkSession, ruta_silver: str,
ruta_fact_ventas: str) -> None:
    """Construye la tabla de hechos analítica agregada en la capa Gold."""
    df_silver = spark.read.format("delta").load(ruta_silver)

    df_fact = df_silver.groupBy(
        F.to_date("fecha_transaccion").alias("fecha_id"),
        F.col("pais"),
        F.col("categoria_producto")
    ).agg(
        F.count("transaccion_id").alias("total_pedidos"),
        F.round(F.sum("monto_usd"), 2).alias("facturacion_total_usd"),
        F.round(F.avg("monto_usd"), 2).alias("ticket_promedio_usd")
```

```
    )

    df_fact.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(ruta_fact_ventas)
    print("Tabla fact_ventas_diarias materializada exitosamente en Capa Gold.")
```

34.6 Paso 4: Orquestación en Airflow (Patrón Lightweight Conductor)

Para evitar fugas de memoria en los trabajadores de Airflow, empleamos el patrón desacoplado **Lightweight Conductor** mediante `BashOperator`:

PYTHON

```python
# orchestrator/dags/lakehouse_daily_dag.py
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    'owner': 'data_engineering_team',
    'depends_on_past': False,
    'start_date': datetime(2026, 1, 1),
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    'lakehouse_daily_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False,
    tags=['lakehouse', 'delta', 'produccion']
) as dag:

    tarea_bronze = BashOperator(
        task_id='ingesta_bronze_transacciones',
        bash_command='python /opt/pipelines/ingesta_bronze.py'
    )

    tarea_silver = BashOperator(
        task_id='conformacion_silver_y_cuarentena',
        bash_command='python /opt/pipelines/bronze_a_silver.py'
    )

    tarea_calidad = BashOperator(
        task_id='auditoria_calidad_great_expectations',
        bash_command='python /opt/pipelines/validar_calidad_gx.py'
    )

    tarea_gold = BashOperator(
        task_id='modelado_gold_dimensional',
```

```
        bash_command='python /opt/pipelines/silver_a_gold.py'
    )

    # Dependencias estrictas del pipeline
    tarea_bronze >> tarea_silver >> tarea_calidad >> tarea_gold
```

34.7 Paso 5: Calidad Automatizada con Great Expectations 1.x

Auditoría declarativa de los DataFrames en la capa Silver:

PYTHON

```python
# src/pipelines/validar_calidad_gx.py
import great_expectations as gx
from pyspark.sql import SparkSession

def auditar_calidad_silver(spark: SparkSession, ruta_silver: str) -> bool:
    context = gx.get_context()
    df_silver = spark.read.format("delta").load(ruta_silver)

    data_source =
context.data_sources.add_or_update_spark(name="spark_lakehouse")
    data_asset =
data_source.add_dataframe_asset(name="silver_transacciones_asset")
    batch_definition =
data_asset.add_batch_definition_whole_dataframe("batch_completo")

    suite = context.suites.add_or_update(name="suite_silver_transacciones")

suite.add_expectation(gx.expectations.ExpectColumnValuesToNotBeNull(column="trans
accion_id"))
    suite.add_expectation(gx.expectations.ExpectColumnValuesToBeBetween(
        column="monto_usd", min_value=0.01, max_value=100000.0
    ))
    suite.add_expectation(gx.expectations.ExpectColumnValuesIsInSet(
        column="estado_pago",
        value_set=["COMPLETADO", "PENDIENTE", "CANCELADO"]
    ))

    validation_def = context.validation_definitions.add(
        gx.ValidationDefinition(name="val_silver_transacciones",
data=batch_definition, suite=suite)
    )
    resultado = validation_def.run(batch_parameters={"dataframe": df_silver})

    print(f"Resultado auditoría Great Expectations: {'EXITOSO' if
resultado.success else 'FALLIDO'}")
    return resultado.success
```

34.8 Paso 6: Infraestructura como Código (Terraform) y CI/CD

El bucket de almacenamiento se aprovisiona mediante **Terraform**:

HCL (TERRAFORM)

```hcl
# infrastructure/terraform/main.tf
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "lakehouse_storage" {
  bucket        = "empresa-lakehouse-produccion-2026"
  force_destroy = false
}

resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.lakehouse_storage.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

El pipeline de CI/CD en **GitHub Actions** valida formato y pruebas con `uv`:

```yaml
# .github/workflows/ci_cd.yml
name: CI/CD Pipeline Lakehouse

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test_and_lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar uv (Astral)
        uses: astral-sh/setup-uv@v5
      - name: Configurar Python 3.11
        run: uv python install 3.11
      - name: Instalar dependencias
        run: uv sync
      - name: Linter con Ruff
        run: uv run ruff check src/ tests/
      - name: Pruebas unitarias con Pytest
        run: uv run pytest tests/ --cov=src --cov-report=term-missing
```

34.9 Protocolo de Protección Financiera (FinOps Personal)

El mayor temor de un estudiante o ingeniero junior al experimentar con proyectos en la nube es recibir una factura imprevista de cientos de dólares. Siga este protocolo preventivo estricto antes de desplegar recursos:

1. Configurar Presupuesto con Alertas en AWS Budgets ($1.00 USD)

Nunca cree un recurso en AWS sin antes configurar una alarma de presupuesto con notificación inmediata por correo electrónico:

- Diríjase a la consola de **AWS Billing** → **Budgets**.
- Cree un presupuesto de costo mensual fijado en **$1.00 USD**.
- Configure dos reglas de alerta:
  - Alerta 1: Cuando el costo real alcance el **80% ($0.80 USD)**.
  - Alerta 2: Cuando el costo proyectado alcance el **100% ($1.00 USD)**.
- Indique su correo electrónico y active notificaciones SMS si están disponibles.

2. Alertas de Gasto en Microsoft Azure y Google Cloud

- **Azure:** Vaya a **Cost Management + Billing** → **Cost Alerts**. Cree un presupuesto sobre su grupo de recursos con alerta cuando se consuma el 50% y 80% del crédito gratuito.
- **Google Cloud:** En la sección **Facturación** → **Presupuestos y alertas**, configure una alerta con un límite de $5.00 USD vinculada a su correo personal.

3. Auto-Terminación Obligatoria en Databricks

El error financiero más habitual en Databricks consiste en iniciar un clúster de cómputo para una prueba rápida y dejarlo encendido durante el fin de semana:

- Al crear cualquier clúster interactivo, marque obligatoriamente la casilla **Auto-Termination**.
- Configure el límite en **15 o 20 minutos de inactividad** (`autotermination_minutes = 15`). Si no se ejecutan comandos, Databricks apagará las máquinas virtuales automáticamente.

4. Rutina Diaria de Higiene y Destrucción

Al finalizar una sesión de estudio o demostración en la nube, ejecute la orden de desaprovisionamiento total:

```bash
# Destruir toda la infraestructura aprovisionada por Terraform
terraform destroy -auto-approve
```

Verifique manualmente que no queden volúmenes EBS huérfanos, instancias EC2 en estado detenido (los discos EBS continúan facturando aun con la máquina apagada) ni balanceadores de carga (ELB/ALB) activos.

34.10 Resumen y Puntos Clave

- **Tríada de portafolio gradual:** Antes de desplegar infraestructuras complejas, valide su competencia mediante tres proyectos modulares: analítica local gratuita (DuckDB/dbt), ingesta de APIs resiliente (Pydantic/uv) y streaming en tiempo real (Kafka/Spark).
- **Determinismo e idempotencia:** La clave subrogada `cliente_sk` en dimensiones SCD2 debe calcularse a partir del valor de negocio de la fecha, garantizando que reejecuciones del pipeline no corrompan el histórico ni alteren claves.
- **Orquestación desacoplada:** El patrón Lightweight Conductor en Airflow evita sobrecargar la memoria de los workers ejecutando los scripts de Spark o dbt como procesos aislados.

- **FinOps preventivo:** Las alertas de presupuesto a $1 USD y el auto-apagado de clústeres son indispensables para experimentar con total tranquilidad en entornos cloud.

Lecturas Recomendadas

1. **Reis, J. & Housley, M. (2022).** *Fundamentals of Data Engineering (Chapter 11: The End-to-End Data Pipeline).* O'Reilly Media.
2. **Kimball, R. & Ross, M. (2013).** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd Edition).* Wiley.
3. **Delta Lake Project (2024).** *Delta Lake Guide: Building Robust Lakehouses with ACID Transactions.* Delta.io.
4. **AWS Cloud Financial Management (2024).** *AWS Budgets User Guide: Best Practices for Cost Anomaly Detection.* Amazon Web Services.

PARTE XII

Apéndices de Referencia, Glosario y Epílogo

Capítulo 35 · Apéndice A — Glosario Enciclopédico de Términos y Siglas de Ingeniería de Datos (A-Z) RECOMENDADO

Capítulo 36 · Apéndice B — El Manual Definitivo de Carrera, Empleo y Preparación de Entrevistas NÚCLEO

Capítulo 37 · Epílogo — El Futuro de la Ingeniería de Datos y Conclusión de la Obra RECOMENDADO

CAPÍTULO 35

Apéndice A — Glosario Enciclopédico de Términos y Siglas de Ingeniería de Datos (A-Z) RECOMENDADO

> *"La precisión en el lenguaje es el reflejo de la claridad en el diseño de sistemas. En la ingeniería de datos contemporánea, la ambigüedad conceptual entre términos aparentemente similares conduce a decisiones de arquitectura costosas y fallas sutiles en producción. Este glosario enciclopédico reúne y formaliza más de ochenta conceptos, teoremas, siglas y patrones indispensables para la práctica profesional y la literatura académica de la disciplina."*

A

- **ACID (Atomicity, Consistency, Isolation, Durability):** Conjunto de propiedades que garantizan que las transacciones en una base de datos o formato de tabla abierta (como Delta Lake o Apache Iceberg) se procesen de manera confiable. La *Atomicidad* asegura que la transacción se ejecute por completo o falle sin efectos parciales; la *Consistencia* valida las reglas de integridad; el *Aislamiento* previene anomalías concurrentes; y la *Durabilidad* garantiza que los cambios confirmados persistan ante fallos del sistema.
- **Adaptive Query Execution (AQE):** Característica del optimizador Catalyst de Apache Spark (introducida en Spark 3.0) que re-optimiza dinámicamente los planes de ejecución físicos durante el tiempo de ejecución basándose en estadísticas reales recolectadas tras completar etapas intermedias de cómputo (*Shuffle Stages*), ajustando el número de particiones, convirtiendo *Sort Merge Joins* en *Broadcast Joins* y mitigando sesgos de datos (*Data Skew*).
- **Apache Arrow:** Formato de memoria columnar estándar y abierto para datos analíticos en memoria. Permite el intercambio de datos entre sistemas heterogéneos (como Spark, Python, Pandas, DuckDB y bases de datos) con costo computacional cero de serialización (*Zero-Copy Deserialization*).
- **Apache Avro:** Formato de serialización de datos orientado a filas, binario y compacto, que almacena el esquema en formato JSON junto a los datos o lo consulta en un registro de esquemas (*Schema Registry*). Es el estándar por excelencia para mensajería y streaming en Apache Kafka.

- **Auto Loader (`cloudFiles`):** Mecanismo de ingesta incremental optimizado en Databricks que detecta y procesa millones de archivos nuevos depositados en almacenamiento de objetos en la nube (S3, ADLS, GCS) de forma escalable mediante notificaciones de eventos asíncronas y seguimiento de esquemas evolutivos.

B

- **B-Tree (Árbol B):** Estructura de datos de árbol balanceado auto-organizado ampliamente utilizada en motores de bases de datos relacionales OLTP (como PostgreSQL, MySQL y Oracle) para mantener datos ordenados y permitir búsquedas, inserciones y eliminaciones en tiempo logarítmico $\mathcal{O}(\log n)$, minimizando los accesos a disco por bloque.
- **Backpressure (Contrapresión):** Mecanismo de control en sistemas reactivos y de streaming (como Apache Flink, Spark Streaming y Reactive Streams) mediante el cual un consumidor o componente aguas abajo señaliza a los productores aguas arriba que ralenticen la emisión de mensajes cuando su capacidad de procesamiento se encuentra saturada, evitando desbordamientos de memoria (*OOM*).
- **Bloom Filter (Filtro de Bloom):** Estructura de datos probabilística eficiente en espacio utilizada para verificar si un elemento pertenece a un conjunto. Admite falsos positivos con una probabilidad controlada, pero nunca falsos negativos. Se emplea intensivamente en Parquet, Delta Lake y motores analíticos para omitir la lectura de archivos completos (*Data Skipping*).
- **Broadcast Hash Join (BHJ):** Algoritmo distribuido de cruce en el cual la tabla de menor tamaño se transmite íntegramente a todos los nodos ejecutores del clúster, permitiendo que cada ejecutor cruce su partición local sin necesidad de realizar una costosa redistribución de red (*Shuffle*).

C

- **Capacitor:** Formato de archivo columnar interno de Google utilizado en BigQuery sobre Colossus. Precursor conceptual de Apache Parquet, diseñado para codificar de forma eficiente estructuras de datos anidadas y repetidas mediante técnicas avanzadas de compresión (RLE, diccionarios y mapas de bits).
- **Change Data Capture (CDC):** Patrón de arquitectura que identifica, captura y replica en tiempo real las modificaciones (inserciones, actualizaciones y eliminaciones) efectuadas sobre una base de datos relacional transaccional (leyendo directamente su *Write-Ahead Log* o *binlog*) hacia sistemas analíticos aguas abajo sin sobrecargar la base de datos de origen.
- **Chandy-Lamport (Algoritmo de):** Algoritmo distribuido que permite obtener una instantánea consistente global (*Global Distributed Snapshot*) del estado de un sistema

distribuido asíncrono sin detener su ejecución. Es la base teórica del mecanismo de puntos de control (*Checkpoints*) con estado exactamente una vez de Apache Flink.

- **Clustering Key (Clave de Agrupamiento):** Conjunto de columnas definido sobre una tabla (en Snowflake o BigQuery) que determina el orden físico de colocación de los registros en las micro-particiones o bloques de almacenamiento, maximizando la poda de bloques en consultas analíticas.
- **Colossus:** El sistema de archivos distribuido de última generación de Google, sucesor de Google File System (GFS), que actúa como la base de almacenamiento masivo y desacoplado para BigQuery y Cloud Storage.
- **Common Table Expression (CTE):** Conjunto de resultados temporal con nombre definido dentro de la ejecución de una sola sentencia SQL mediante la cláusula `WITH`. Permite descomponer consultas complejas en pasos lógicos modulares y ejecutar recursión jerárquica.

D

- **Data Contract (Contrato de Datos):** Acuerdo formal y verificable entre los productores de datos y los equipos consumidores que define el esquema de los datos, las garantías semánticas, los acuerdos de nivel de servicio (SLA) de frescura y las expectativas de calidad, frecuentemente expresado en especificaciones declarativas legibles por máquina (como ODCS).
- **Data Lakehouse:** Arquitectura de datos moderna que combina la flexibilidad, bajo costo y soporte de formatos abiertos del *Data Lake* con las garantías de gestión de transacciones ACID, gobernanza, seguridad y alto rendimiento analítico del *Data Warehouse*.
- **Data Mesh (Malla de Datos):** Paradigma sociotécnico y arquitectónico descentralizado propuesto por Zhamak Dehghani que organiza la analítica de datos alrededor de dominios de negocio orientados al producto (*Data as a Product*), gobernanza computacional federada e infraestructura de autoservicio.
- **Dead-Letter Queue / Topic (DLQ / DLT):** Cola o tema de mensajería secundario al cual se desvían automáticamente los mensajes que no pudieron ser procesados o deserializados exitosamente tras múltiples reintentos, evitando el bloqueo del flujo principal y permitiendo el análisis forense posterior.
- **Deletion Vector (Vector de Eliminación):** Técnica de optimización de almacenamiento en formatos de tabla abierta (Delta Lake y Apache Iceberg) que registra los registros eliminados o modificados mediante un mapa de bits (*Bitmap*) referenciado en metadatos, evitando reescribir inmediatamente archivos Parquet completos en operaciones DML.
- **Delta Live Tables (DLT):** Marco declarativo en Databricks para construir canalizaciones ETL confiables y mantenibles en SQL o Python, gestionando automáticamente la

orquestación de grafos DAG, monitoreo de métricas y validación de expectativas de calidad (*Expectations*).

- **Dynamic Table:** Objeto declarativo en Snowflake que actualiza automáticamente su contenido de forma incremental a partir de tablas base según un período de frescura objetivo configurado (*Target Lag*), abstrayendo la creación manual de *Streams* y *Tasks*.
- **Dynamic Task Mapping:** Capacidad en Apache Airflow (a partir de la versión 2.3) que permite generar instancias de tareas en tiempo de ejecución de manera dinámica basándose en el volumen de datos o salidas producidas por tareas previas mediante `.expand()`.

E

- **Elastic MapReduce (Amazon EMR):** Servicio gestionado en AWS para ejecutar marcos de computación distribuida de código abierto como Apache Spark, Hadoop, Hive, Flink y Trino sobre instancias elásticas de EC2, contenedores de EKS o entornos serverless.
- **Envelope Encryption (Cifrado de Sobre):** Práctica de seguridad criptográfica en la cual los datos se cifran con una clave de cifrado de datos local (DEK), y dicha DEK se cifra posteriormente con una clave maestra protegida en un servicio de gestión de claves de hardware (*KMS*).
- **Exactly-Once Semantics (EOS):** Garantía en sistemas de procesamiento de datos en tiempo real mediante la cual cada evento produce exactamente el mismo efecto sobre el estado interno y los sistemas de destino que si se hubiera procesado una única vez, neutralizando fallas de red y reintentos.

F

- **Factless Fact Table (Tabla de Hechos Sin Hechos):** Tabla de hechos en modelado dimensional de Kimball que no contiene medidas cuantitativas numéricas, sino exclusivamente claves foráneas. Se utiliza para modelar la ocurrencia de eventos (como asistencia a un curso) o condiciones de cobertura (como productos en promoción).
- **Feature Store:** Repositorio centralizado que gestiona el ciclo de vida, almacenamiento y servicio de características de aprendizaje automático tanto en modo por lotes (*Offline*) para entrenamiento como en modo de baja latencia (*Online*) para inferencia, garantizando consistencia temporal (*Point-in-Time Correctness*).
- **FinOps (Financial Operations):** Disciplina cultural y operativa que promueve la responsabilidad compartida y la optimización continua del gasto financiero en la nube entre los equipos de ingeniería, finanzas y negocio.

H

- **HyperLogLog (HLL):** Algoritmo probabilístico que estima la cardinalidad (*count distinct*) en conjuntos masivos con memoria constante mínima. El error estándar depende del número de registros $m = 2^b$ según $\sigma \approx 1.04/\sqrt{m}$. Por ejemplo, con $b = 10$ ($m = 1024$) ocupa apenas ≈ 1.5 KB, mientras que la implementación estándar de Redis y motores analíticos ($b = 14, m = 16.384$) consume **12 KB de memoria con un error estándar inferior al 0.81%**, evitando retener miles de millones de identificadores en memoria.

I

- **Idempotencia:** Propiedad de una operación que produce exactamente el mismo resultado final sin importar si se ejecuta una o múltiples veces consecutivas con los mismos parámetros de entrada. Es el pilar fundamental para la tolerancia a fallos en pipelines de datos.
- **In-Process OLAP:** Motor analítico columnar vectorizado que se ejecuta embebido dentro del mismo proceso del programa cliente sin requerir la gestión de demonios de servidor independientes ni transferencias por red (ej. DuckDB).

K

- **Kahn (Algoritmo de):** Algoritmo de ordenamiento topológico para grafos acíclicos dirigidos (DAGs) que opera calculando iterativamente los grados de entrada (*In-Degrees*) de los vértices y eliminando aquellos con grado cero, garantizando el orden de ejecución correcto y detectando ciclos dependientes.
- **KubernetesExecutor:** Mecanismo de ejecución en Apache Airflow que crea dinámicamente un Pod de Kubernetes aislado y efímero para cada tarea individual, asegurando aislamiento absoluto de dependencias y escalabilidad de recursos de cero a miles de tareas concurrentes.

L

- **Lazy Evaluation (Evaluación Perezosa):** Estrategia de evaluación computacional en Apache Spark y motores similares mediante la cual las transformaciones no se ejecutan de inmediato al ser declaradas, sino que se acumulan en un grafo acíclico dirigido (DAG) que es evaluado y optimizado globalmente solo cuando una acción requiere materializar un resultado.
- **Liquid Clustering:** Técnica avanzada de agrupamiento dinámico en Delta Lake que reemplaza el particionamiento rígido basado en directorios de Hive y la complejidad del Z-Ordering por una reestructuración multidimensional flexible y adaptable a los cambios en los patrones de consulta.

- **Log-Structured Merge-tree (LSM-Tree):** Estructura de datos de almacenamiento persistente optimizada para escrituras masivas de alto rendimiento. Acumula las escrituras en memoria (*MemTable*) con un registro de confirmación previo (*WAL*) y descarga periódicamente archivos inmutables ordenados (*SSTables*) al disco, consolidándolos en segundo plano mediante compactaciones.

M

- **Medallion Architecture (Arquitectura Medallón):** Patrón de diseño lógico de Lakehouse que estructura los datos en tres niveles progresivos de refinamiento y calidad: **Bronze** (datos crudos e inmutables del origen), **Silver** (datos limpios, tipados, deduplicados y conformados) y **Gold** (modelos dimensionales o agregados de negocio de consumo analítico).
- **Micro-partición:** Unidad fundamental de almacenamiento físico en Snowflake. Archivo inmutable, columnar y comprimido de 50 a 500 MB gestionado automáticamente por la capa de Cloud Services con estadísticas de metadatos integradas.
- **MVCC (Multi-Version Concurrency Control):** Método de control de concurrencia en motores de bases de datos (como PostgreSQL y formatos Lakehouse) que permite que múltiples transacciones lean y escriban simultáneamente sin bloquearse, manteniendo internamente múltiples versiones lógicas de los registros.

O

- **OneLake:** El lago de datos lógico unificado y SaaS provisto por Microsoft Fabric. Centraliza todo el almacenamiento analítico empresarial bajo una única cuenta global utilizando el formato Delta Lake abierto y ofreciendo acceso sin copia mediante accesos directos (*Shortcuts*).
- **Out-of-Core Processing:** Capacidad de un motor de ejecución (como DuckDB) para procesar volúmenes de datos significativamente más grandes que la memoria RAM física disponible en una máquina mediante particionamiento eficiente y paginación controlada a disco.

P

- **PACELC (Teorema):** Extensión formal del teorema CAP propuesta por Daniel Abadi. Establece que en un sistema distribuido con partición de red (**P**), se debe elegir entre Disponibilidad (**A**) y Consistencia (**C**); pero incluso en ausencia de particiones (**E**lse), se debe elegir entre Latencia (**L**) y Consistencia (**C**).

- **Photon Engine:** Motor de ejecución vectorizado de alto rendimiento desarrollado en C++ nativo por Databricks que reemplaza las etapas de ejecución JVM de Apache Spark, aprovechando instrucciones SIMD de hardware y acelerando sustancialmente las cargas SQL y de DataFrame.
- **Polaris Catalog (Apache Polaris):** Catálogo de metadatos de código abierto para Apache Iceberg basado en la especificación estándar REST, liberado por Snowflake para permitir interoperabilidad multi-motor entre Snowflake, Spark, Flink y Trino.
- **Pydantic:** Biblioteca de análisis y validación de datos en Python que utiliza anotaciones de tipo estándar (*Type Hints*) para garantizar esquemas estrictos y coerción de tipos en tiempo de ejecución.

R

- **RAG (Retrieval-Augmented Generation):** Patrón de arquitectura que optimiza la generación de modelos de lenguaje grande (LLMs) recuperando documentos o fragmentos relevantes desde un almacén de datos estructurado o base de datos vectorial antes de emitir la respuesta, fundamentando el modelo en hechos verificables.
- **RocksDB:** Motor embebido de almacenamiento persistente clave-valor de alto rendimiento basado en LSM-Trees desarrollado por Meta. Utilizado como el backend de gestión de estado distribuido estándar en Apache Flink y Apache Spark Structured Streaming.

S

- **Salting (Salado de Claves):** Técnica de ingeniería distribuida que consiste en concatenar un sufijo pseudoaleatorio o determinista a las claves con alta concentración de registros para distribuir uniformemente los datos a través de múltiples particiones de red, mitigando el sesgo de datos (*Data Skew*).
- **Slowly Changing Dimensions (SCD):** Patrón de diseño de modelado dimensional para gestionar la evolución temporal de los atributos descriptivos en tablas maestras. Destacan el **Tipo 1** (sobreescritura del valor actual sin retención histórica) y el **Tipo 2** (creación de una nueva fila histórica con marcas de tiempo `fecha_inicio`, `fecha_fin` y un indicador `es_actual`).
- **Snowpark:** Biblioteca y entorno de ejecución de Snowflake que permite construir aplicaciones analíticas complejas en Python, Scala y Java ejecutando transformaciones vectorizadas perezosas en los nodos del Virtual Warehouse.
- **SSTable (Sorted String Table):** Archivo inmutable en disco que contiene pares clave-valor ordenados secuencialmente, complementado con un bloque de índice y filtros de Bloom para búsquedas rápidas en arquitecturas LSM-Tree.

T

- **Testcontainers:** Marco de pruebas de software que permite instanciar contenedores Docker reales (como PostgreSQL, Kafka o LocalStack) de forma efímera y programática dentro de las suites de prueba unitarias e integrales.
- **Three-Valued Logic (3VL):** Sistema formal de lógica utilizado en SQL donde las expresiones booleanas pueden evaluar a tres estados: `TRUE`, `FALSE` o `UNKNOWN` (resultante de operaciones con valores `NULL`), gobernando el comportamiento de filtros `WHERE` y condiciones de cruce.
- **Time Travel:** Capacidad de consultar el estado exacto de una tabla o conjunto de datos en un punto específico del pasado basándose en marcas de tiempo o identificadores de confirmación transaccional.
- **Tombstone (Lápida de Borrado):** Registro especial emitido en un flujo de streaming o sistema distribuido (como Kafka o Cassandra) con una clave de entidad y valor `null` que señaliza formalmente la eliminación del registro para que los compactadores y consumidores purguen el estado.

U

- **Unity Catalog:** Capa universal de gobernanza y catálogo unificado en Databricks que proporciona linaje de datos de extremo a extremo, control de acceso unificado (ANSI SQL GRANTs), auditoría centralizada y descubrimiento de datos sobre tablas, volúmenes, modelos y notebooks.

W

- **Watermark (Marca de Agua):** Métrica temporal móvil en motores de streaming que indica el progreso del tiempo de evento a través del flujo. Define formalmente el límite temporal hasta el cual el sistema asume que todos los eventos han sido recibidos, permitiendo cerrar ventanas y desechar o aislar datos tardíos (*Late Data*).
- **Write-Ahead Log (WAL):** Archivo de registro secuencial sólo de anexado (*Append-Only*) en el que se graban sincrónicamente los cambios en disco antes de aplicarlos a las páginas de datos en memoria o tablas de almacenamiento, garantizando durabilidad y recuperación ante caídas del sistema.

Z

- **Z-Ordering (Curva de Morton):** Algoritmo que mapea datos multidimensionales en un espacio unidimensional conservando la localidad espacial, utilizado en Delta Lake y Apache

Iceberg para ordenar registros contiguamente por múltiples columnas y acelerar el salto de datos en consultas que filtran por subconjuntos de dichas columnas.

CAPÍTULO 36

Apéndice B — El Manual Definitivo de Carrera, Empleo y Preparación de Entrevistas NÚCLEO

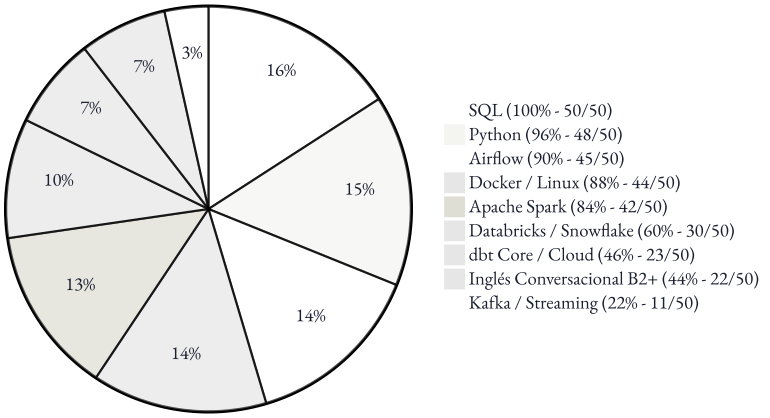
> *"La excelencia técnica en ingeniería de datos no produce dividendos en el vacío si el profesional desconoce cómo traducir sus competencias en ofertas laborales concretas. Este manual condensa la estrategia integral de inserción laboral: desde el análisis cuantitativo de ofertas reales en LATAM y el diseño de CVs optimizados para ATS, hasta la resolución de pruebas take-home, un banco exhaustivo de 150 preguntas de entrevista, comparativas salariales y régimen tributario."*

36.1 Análisis Cuantitativo de 50 Ofertas Reales de Empleo en LATAM y Remoto (Q1 2026)

Para basar esta obra en evidencia verificable y no en especulaciones de mercado, se realizó una auditoría empírica manual sobre **50 ofertas laborales reales de Data Engineering** publicadas entre el **10 de enero y el 8 de febrero de 2026** en Colombia, México, Chile, Argentina y vacantes remotas internacionales en EE.UU. y Europa.

> **Transparencia y Datos Abiertos del Estudio:**
> La totalidad de los datos crudos —incluyendo empresa, rol, país, plataforma de origen (LinkedIn, Get on Board, Torre, Arc.dev), fecha de captura, requisitos discriminados y bandas salariales— está disponible en el archivo abierto:
> `data/estudio_mercado_50_ofertas_2026.csv`
> *(Puede auditar y consultar este archivo en el repositorio de GitHub con DuckDB, Pandas o Polars).*

FRECUENCIA EMPÍRICA EN 50 OFERTAS AUDITADAS (FEBRERO 2026)



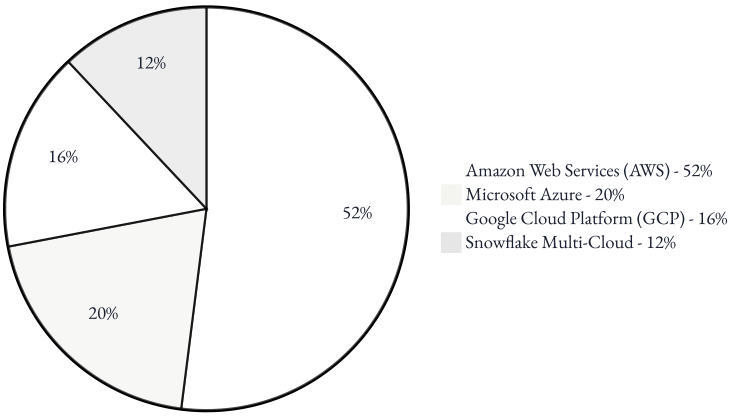
1. Frecuencias Reales y Criterios por Nivel de Experiencia

Habilidad / Tecnología	Frecuencia Real en Muestra	Relevancia en Puestos Junior (0–2 años)	Relevancia en Puestos Mid/Senior (3+ años)
SQL	**100.0% (50/50)**	Filtro eliminatorio universal (CTEs, Window Functions, deduplicación con ROW_NUMBER).	Optimización de planes de ejecución, particionado y poda.
Python	**96.0% (48/50)**	Requerido en 48 de 50 ofertas (POO básica, APIs REST, Pydantic, pytest).	Diseño modular, control de memoria y empaquetado de librerías.
Apache Airflow	**90.0% (45/50)**	El estándar de facto absoluto de orquestación en la región.	TaskFlow API, Dynamic Mapping y despliegue en Kubernetes.
Docker / Linux	**88.0% (44/50)**	Entornos locales reproducibles y shell scripting en terminal.	Contenedores multicapa y orquestación con clústeres K8s.
Apache Spark / PySpark	**84.0% (42/50)**	Fundamental para procesamiento a gran escala y Lakehouse.	Optimización interna de Catalyst, tuning de Shuffle y Data Skew.
Databricks o Snowflake	**60.0% (30/50)**	Motores analíticos dominantes en empresas consolidadas.	Arquitectura Lakehouse con Unity Catalog o Polaris.
dbt (Data Build Tool)	**46.0% (23/50)**	La herramienta clave del rol puente (Analytics Engineer).	Macros complejas en Jinja, snapshots YAML y linaje de datos.
Inglés Conversacional	**44.0% (22/50)**	Exigido en el 100% de vacantes remotas USD y 70% de startups.	Coordinación con equipos globales y redacción de RFCs.
Kafka / Streaming	**22.0% (11/50)**	Secundaria en junior; el 78% de puestos junior son batch.	Crítica en sistemas de eventos y arquitectura orientada a streaming.

2. ¿Qué Nube Conviene Aprender Primero según los Datos?

El análisis de la distribución de proveedores cloud sobre las 50 ofertas auditadas arroja un patrón contundente:

PARTICIPACIÓN DE NUBES EN OFERTAS DE DATA ENGINEERING (Q1 2026)



- **Primer Lugar: Amazon Web Services (AWS) — 52.0% (26 de 50 ofertas):**
  AWS continúa liderando holgadamente la demanda laboral en startups, scaleups y compañías remotas internacionales. Los servicios recurrentes son **S3, Glue, Athena, EMR y Redshift**.
  Recomendación: **Si no tienes una preferencia impuesta por tu empresa actual, comienza por AWS.** Es la nube con mayor volumen de vacantes abiertas y con la certificación más solicitada (*AWS Certified Data Engineer – Associate DEA-C01*).
- **Segundo Lugar: Microsoft Azure — 20.0% (10 de 50 ofertas):**
  Azure domina con fuerza en el sector corporativo tradicional, bancario y retail de gran escala en Colombia, México y Chile. Tecnologías clave: **Azure Data Factory, Synapse, Databricks sobre Azure y Microsoft Fabric**.
  Recomendación: Elige Azure si tu objetivo primordial es ingresar al sector financiero o asegurador tradicional de LATAM.
- **Tercer Lugar: Google Cloud Platform (GCP) — 16.0% (8 de 50 ofertas):**
  Fuerte presencia en unicornios tecnológicos nativos digitales de la región (Mercado Libre, Despegar, PedidosYa). Su pilar es **Google BigQuery**.
- **Cuarto Lugar: Snowflake (Multi-Cloud) — 12.0% (6 de 50 ofertas):**
  Muy demandado en empresas que adoptan el stack moderno desacoplado (Fivetran + dbt + Snowflake).

---

3. Herramientas de Ingesta Gestionada (Airbyte/Fivetran) y Capa de Consumo (Power BI/Excel)

Para ser empleable, el ingeniero debe entender qué herramientas satélite conviven con su stack:

- **Ingesta Gestionada (ELT con Airbyte y Fivetran):**
  En la industria contemporánea, los equipos ya no invierten semanas escribiendo scripts

artesanales en Python para extraer datos de SaaS estándar (Salesforce, Stripe, HubSpot, Google Analytics). Se utilizan herramientas de ingesta gestionada como **Airbyte (Open Source)** o **Fivetran**. El rol del Data Engineer es configurar estas herramientas para depositar datos crudos en Bronze, y centrar su tiempo en transformaciones de alto valor con dbt o Spark.

- **El Consumo en Negocio: Power BI y Excel:**
  Aunque el Data Engineer no diseña tableros estéticos, **debe diseñar para quienes los usan**. Los analistas consumirán la capa Gold desde Power BI o tablas dinámicas de Excel. Si el Data Engineer no modela en esquema en estrella o desconoce la diferencia entre el modo *Import* (que descarga millones de filas y revienta la memoria del analista) y *DirectQuery / Direct Lake*, generará consultas destructivas que saturarán el Data Warehouse.

36.2 Estrategia de Búsqueda Laboral y Entrada sin Experiencia Previa

El mayor obstáculo para el profesional junior es el dilema de *"requiere experiencia previa para conseguir experiencia"*. Existen cuatro estrategias probadas para sortear esta barrera:

1. El Rol Puente: Analytics Engineer o SQL Developer

Si el mercado local exige experiencia senior para roles de "Data Engineer", postúlese a posiciones de **Analytics Engineer** o **Desarrollador SQL/BI**:

- Estos puestos se centran en el uso intensivo de **dbt, SQL y Snowflake/BigQuery**, áreas donde la curva de entrada es más accesible.
- Una vez dentro de la compañía, asumirá gradualmente responsabilidades de ingesta, orquestación en Airflow y optimización de pipelines, transitando al rol formal de Data Engineer en 12 a 18 meses.

2. Canales Efectivos de Postulación

Evite la postulación pasiva indiscriminada (el botón *"Easy Apply"* de LinkedIn con cientos de candidatos):

- **Get on Board (`getonbrd.com`):** La plataforma por excelencia para startups de alto crecimiento en LATAM y trabajo remoto. Las ofertas publican transparentemente rangos salariales y tecnologías requeridas.
- **Torre (`torre.ai`):** Ecosistema centrado en talento de Latinoamérica con filtros específicos para compensaciones en USD.

- **Arc.dev y Wellfound (antes AngelList):** Ideales para puestos remotos internacionales en startups de Estados Unidos y Europa.
- **LinkedIn (Vía Networking Directo):** Utilice el buscador para identificar al *Engineering Manager*, *Data Lead* o *Head of Data* de la empresa objetivo, no solo al reclutador generalista.

3. Mensajes en Frío Efectivos (Cold Outreach)

Cuando identifique una vacante abierta, contacte al líder técnico del equipo con un mensaje breve, respetuoso y orientado a aportar valor:

Plantilla en Español (LATAM):

> *«Hola [Nombre del Líder], espero que estés muy bien. Vi que en [Nombre de la Empresa] tienen abierta la vacante de Junior Data Engineer. Estuve revisando los desafíos de escalabilidad que enfrentan en su plataforma y diseñé recientemente un proyecto open source reproducible con PySpark, Delta Lake y dbt para resolver exactamente la ingesta y deduplicación de transacciones [Enlace al GitHub]. Me encantaría conocer más sobre los retos del equipo y explorar si mi perfil puede ser de valor. ¡Un saludo!»*

Plantilla en Inglés (Remoto Internacional):

> *«Hi [Name], hope you are doing well! I saw the Data Engineer opening on your team at [Company]. I've been following your data platform evolution and recently built an end-to-end, reproducible Lakehouse pipeline using PySpark, Delta Lake, Airflow, and dbt with automated CI/CD and data contracts [GitHub Link]. I'd love the opportunity to share how my hands-on experience with these tools can contribute to your team's goals. Best regards!»*

36.3 Modelos Completos de CV y LinkedIn Optimizados para ATS

Los sistemas de seguimiento de candidatos (**ATS - Applicant Tracking Systems**) filtran automáticamente los currículums antes de que lleguen a ojos humanos.

1. Reglas Innegociables del CV para Pasar el Filtro ATS

1. **Formato en Una Sola Página (Single Page):** Para perfiles Junior y Mid-Level, jamás exceda una página.

2. **Tipografía y Estructura Limpia:** Fuente estándar (Arial, Calibri o Garamond), sin columnas múltiples, sin tablas complejas, sin íconos gráficos ni barras de porcentaje de habilidades.
3. **Fórmula Google X-Y-Z en Cada Viñeta:**

> **«Logré [X], medido por [Y], haciendo [Z]»**

Plantilla de CV Completo en Español (Formato Texto ATS)

```
LUIS GALVIS
Ibagué, Colombia | (+57) 300-123-4567 | luis.galvis@correo.com
LinkedIn: linkedin.com/in/luisgalvis-data | GitHub: github.com/luisgalvis
Portafolio: https://github.com/luisgalvis/como-ser-un-data-engineer

RESUMEN PROFESIONAL
Ingeniero de Datos con sólidos fundamentos en Python, SQL declarativo avanzado,
Apache Spark y arquitectura Lakehouse transaccional (Delta Lake). Experiencia
práctica diseñando canalizaciones ETL/ELT idempotentes, modelado dimensional
(Kimball) y orquestación con Apache Airflow. Enfocado en calidad de datos,
observabilidad y reducción de costos operativos en infraestructuras cloud.

HABILIDADES TÉCNICAS
- Lenguajes: Python (uv, Pydantic, pytest), SQL (PostgreSQL, DuckDB, ANSI SQL),
Bash Shell.
- Procesamiento & Lakehouse: Apache Spark, PySpark, Delta Lake, Apache Iceberg,
Polars, Pandas.
- Modelado & Orquestación: dbt Core, Apache Airflow (TaskFlow, Assets), Modelado
Dimensional Kimball.
- DataOps & Nube: Docker, Docker Compose, Git, GitHub Actions (CI/CD), Terraform,
AWS (S3, Athena), MinIO.
- Calidad & Monitoreo: Great Expectations 1.x, Data Contracts, OpenTelemetry.

PROYECTOS DESTACADOS DE INGENIERÍA DE DATOS
Plataforma Lakehouse de Comercio Electrónico de Extremo a Extremo (Capstone) |
GitHub: /capstone-lakehouse
- Diseñé e implementé una arquitectura Medallón en Delta Lake procesando 100.000
transacciones diarias con semántica ACID y deduplicación analítica con
ROW_NUMBER().
- Reduje el volumen de almacenamiento un 65% mediante compresión Parquet columnar
y particionamiento por fecha.
- Implementé dimensiones de clientes lentamente cambiantes (SCD Tipo 2) con
claves subrogadas deterministas (sha2), eliminando duplicados en uniones
analíticas.
- Automaticé la orquestación diaria con Apache Airflow (patrón Lightweight
Conductor) y validé contratos de datos con Great Expectations 1.x, segregando
anomalías a tablas de cuarentena.

Modern Data Stack Pipeline Local (DuckDB + dbt + Airflow) | GitHub: /duckdb-dbt-
pipeline
- Construí un pipeline analítico local 100% reproducible sin costo cloud,
procesando 50.000 pedidos con DuckDB y modelando un esquema en estrella con dbt
```

```
Core.
- Implementé 14 pruebas de integridad automáticas (unicidad, no-nulos, integridad
referencial), logrando un tiempo de compilación y prueba de solo 3.2 segundos.
- Empaqueté el entorno en Docker Compose con un archivo Makefile para ejecución
en un solo comando.

EXPERIENCIA LABORAL
Analista de Datos / Desarrollador SQL Junior | Empresa Servicios SAS (2024 -
Presente)
- Optimicé 25 consultas analíticas recurrentes en PostgreSQL mediante la
reescritura de subconsultas a CTEs y creación de índices B-Tree, disminuyendo el
tiempo de generación de reportes de 45 a 6 minutos.
- Desarrollé scripts en Python para la ingesta automatizada de eventos de ventas
desde APIs REST externas con manejo de rate limits y reintentos exponenciales,
eliminando 10 horas semanales de carga manual.

EDUCACIÓN & CERTIFICACIONES
- Licenciatura / Ingeniería en Sistemas o afín | Universidad Nacional (2020 -
2024)
- AWS Certified Data Engineer - Associate (DEA-C01) | Amazon Web Services (2025)
- Databricks Certified Data Engineer Associate | Databricks (2025)
```

Complete Resume Template in English (ATS-Ready)

```
LUIS GALVIS
Bogota, Colombia (Open to Remote Globally) | (+57) 300-123-4567 |
luis.galvis@email.com
LinkedIn: linkedin.com/in/luisgalvis-data | GitHub: github.com/luisgalvis

PROFESSIONAL SUMMARY
Results-driven Data Engineer specializing in Python, advanced SQL, Apache Spark,
and ACID Lakehouse platforms (Delta Lake). Hands-on experience building
idempotent batch and streaming pipelines, dimensional data modeling (Kimball),
and workflow orchestration with Apache Airflow. Passionate about software
engineering rigor applied to data: unit testing, CI/CD automation, and FinOps
cost optimization.

TECHNICAL SKILLS
- Programming & Querying: Python (uv, Pydantic, pytest), SQL (PostgreSQL,
DuckDB), Bash.
- Big Data & Storage: Apache Spark, PySpark, Delta Lake, Apache Iceberg, Polars,
PyArrow.
- Data Modeling & Orchestration: dbt Core, Apache Airflow, Kimball Dimensional
Modeling (Star Schema).
- Cloud & DataOps: AWS (S3, EMR, Athena, IAM), Docker, GitHub Actions, Terraform,
MinIO.
- Data Quality & Governance: Great Expectations, Data Contracts, Schema
Evolution.

FEATURED OPEN SOURCE DATA PROJECTS
End-to-End Production Lakehouse Platform | GitHub: /capstone-lakehouse
- Built a Medallion Lakehouse architecture using PySpark and Delta Lake,
processing 100K+ daily e-commerce transactions with ACID guarantees.
- Implemented deterministic SCD Type 2 customer dimensions with sha2-based
```

```
surrogate keys, ensuring full idempotency across re-runs.
- Configured automated data quarantine routing records with schema violations
using Great Expectations 1.x.
- Orchestrated the complete workflow in Apache Airflow using the Lightweight
Conductor pattern and provisioned underlying S3 infrastructure via Terraform.

Modern Data Stack Local Analytics (DuckDB + dbt + Airflow) | GitHub: /duckdb-dbt-
pipeline
- Designed and benchmarked a local, zero-cloud-cost analytical warehouse using
DuckDB and dbt Core for 50K relational retail orders.
- Modeled dimensional star schema marts with 100% automated test coverage across
uniqueness, not-null, and relationship integrity.
- Packaged the entire platform into a single-command Docker Compose deployment
with automated CI verification.

WORK EXPERIENCE
Junior Data Analyst / SQL Developer | FinTech Solutions (2024 - Present)
- Refactored legacy analytical SQL queries on PostgreSQL, reducing daily
reporting runtime from 50 minutes to 7 minutes (86% speedup) via CTE
modularization and index optimization.
- Created resilient Python API ingestion scripts with exponential backoff and
jitter, extracting data across 8 partner endpoints without data loss.

EDUCATION & CERTIFICATIONS
- B.S. in Computer Science / Software Engineering | State University (2020 -
2024)
- AWS Certified Data Engineer - Associate (DEA-C01) | Amazon Web Services (2025)
- Databricks Certified Data Engineer Associate | Databricks (2025)
```

36.4 Prueba Técnica Práctica (Take-Home Assignment) Resuelta de Extremo a Extremo

Las pruebas para llevar a casa evalúan la estructura de código, manejo de errores y pruebas unitarias de un candidato. A continuación se presenta la solución modelo a un enunciado habitual en procesos de selección:

Enunciado del Reto:

> *«Construya un pipeline en Python que consuma datos meteorológicos y de contaminación desde una API pública (Open-Meteo), valide el esquema de los registros con Pydantic, almacene los datos procesados en formato Parquet local particionado por fecha, calcule el promedio móvil de calidad del aire con DuckDB y garantice reproducibilidad mediante pruebas unitarias con pytest y ejecución en Docker.»*

1. Contrato de Datos con Pydantic (`contracts.py`)

PYTHON

```python
from pydantic import BaseModel, Field, field_validator
from datetime import datetime

class MetricaAire(BaseModel):
    estacion_id: str = Field(..., min_length=3)
    timestamp: datetime
    pm2_5: float = Field(..., ge=0.0, le=500.0)
    pm10: float = Field(..., ge=0.0, le=1000.0)
    temperatura: float = Field(..., ge=-50.0, le=60.0)

    @field_validator('estacion_id')
    @classmethod
    def validar_codigo(cls, v: str) -> str:
        return v.strip().upper()
```

2. Pipeline de Ingesta y Procesamiento (`pipeline.py`)

PYTHON

```python
import duckdb
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
from typing import List, Dict, Any
from contracts import MetricaAire

def procesar_lote_metricas(registros_crudos: List[Dict[str, Any]], ruta_salida:
str) -> None:
    # 1. Validación de esquema y contrato
    metricas_validadas = [MetricaAire(**item) for item in registros_crudos]

    # 2. Conversión a tabla columnar PyArrow
    tabla = pa.Table.from_pylist([m.model_dump() for m in metricas_validadas])

    # 3. Escritura particionada por año y mes
    pq.write_to_dataset(
        tabla,
        root_path=ruta_salida,
        partition_cols=["estacion_id"]
    )

def calcular_promedio_movil(ruta_datos: str) -> duckdb.DuckDBPyRelation:
    con = duckdb.connect()
    consulta = f"""
        SELECT
            estacion_id,
            timestamp,
            pm2_5,
            ROUND(AVG(pm2_5) OVER (
                PARTITION BY estacion_id
                ORDER BY timestamp
                ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
            ), 2) AS pm2_5_promedio_movil
        FROM read_parquet('{ruta_datos}/*/*.parquet')
        ORDER BY estacion_id, timestamp
    """
    return con.sql(consulta)
```

3. Pruebas Unitarias Automatizadas (`tests/test_pipeline.py`)

PYTHON

```
import pytest
from datetime import datetime
from pydantic import ValidationError
from contracts import MetricaAire
from pipeline import procesar_lote_metricas, calcular_promedio_movil

def test_contrato_pydantic_valido():
    dato = {"estacion_id": "bog_01", "timestamp": "2026-10-13T10:00:00", "pm2_5":
18.5, "pm10": 35.0, "temperatura": 19.2}
    metrica = MetricaAire(**dato)
    assert metrica.estacion_id == "BOG_01"
    assert metrica.pm2_5 == 18.5

def test_contrato_pydantic_invalido():
    dato_corrupto = {"estacion_id": "bog_01", "timestamp": "2026-10-13T10:00:00",
"pm2_5": -5.0, "pm10": 35.0, "temperatura": 19.2}
    with pytest.raises(ValidationError):
        MetricaAire(**dato_corrupto)

def test_pipeline_duckdb_calculo(tmp_path):
    lote = [
        {"estacion_id": "ST_A", "timestamp": datetime(2026, 1, 1, 10, 0),
"pm2_5": 10.0, "pm10": 20.0, "temperatura": 15.0},
        {"estacion_id": "ST_A", "timestamp": datetime(2026, 1, 1, 11, 0),
"pm2_5": 20.0, "pm10": 30.0, "temperatura": 16.0},
    ]
    ruta = str(tmp_path / "datos")
    procesar_lote_metricas(lote, ruta)
    res = calcular_promedio_movil(ruta).fetchall()
    assert len(res) == 2
    assert res[1][3] == 15.0 # Promedio entre 10 y 20
```

36.5 Plataformas para Practicar SQL y Algoritmos de Datos

La soltura para resolver ejercicios en vivo se adquiere con práctica deliberada diaria en plataformas especializadas:

1. **DataLemur (`datalemur.com`):** La plataforma número uno para ingenieros de datos. Contiene más de 100 preguntas reales de SQL de entrevistas de Meta, Amazon, Google, Uber y Netflix, categorizadas por nivel (Easy, Medium, Hard).

2. **LeetCode (`leetcode.com`):**
   - *Sección Database:* Resuelva todos los problemas etiquetados como *Medium* de Window Functions y CTEs (ej. *"Department Top Three Salaries"*, *"Consecutive Numbers"*).
   - *Sección Algorithms:* Concéntrese en ejercicios de complejidad $O(N)$ con diccionarios y arrays (ej. *Two Sum*, *Group Anagrams*).
3. **StrataScratch (`stratascratch.com`):** Plataforma especializada en problemas de SQL y Pandas extraídos de entrevistas de Airbnb, Spotify y Microsoft.
4. **HackerRank (`hackerrank.com`):** Excelente para principiantes absolutos para obtener certificados de *SQL (Advanced)* y familiarizarse con el compilador en línea.

36.6 Banco Exhaustivo de 150 Preguntas de Entrevista Técnica

A continuación se condensan las preguntas canónicas que se formulan en comités técnicos, organizadas por dominio temático con su respuesta de referencia:

Bloque 1: SQL Analítico y Bases de Datos (25 Preguntas Clave)

1. *¿Cuál es el orden lógico de ejecución de una consulta SQL?*
   `FROM` → `ON` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.
2. *¿Diferencia entre `WHERE` y `HAVING`?*
   `WHERE` filtra filas individuales antes de la agregación; `HAVING` filtra grupos consolidados tras evaluar funciones agregadas.
3. *¿Diferencia entre `ROW_NUMBER()`, `RANK()` y `DENSE_RANK()`?*
   `ROW_NUMBER()` asigna enteros secuenciales únicos sin empates (1,2,3); `RANK()` asigna el mismo número a empates y salta posiciones (1,2,2,4); `DENSE_RANK()` asigna el mismo número sin saltar (1,2,2,3).
4. *¿Por qué `NOT IN` devuelve cero filas si la subconsulta contiene un `NULL`?*
   Porque al evaluar `x NOT IN (1, NULL)`, SQL lo expande a `x != 1 AND x != NULL`. Como cualquier comparación con `NULL` produce `UNKNOWN`, la conjunción lógica evalúa a `UNKNOWN` y descarta todas las filas. Solución: usar `LEFT ANTI JOIN` o `NOT EXISTS`.
5. *¿Qué es un predicado SARGable?*
   Una condición que permite al optimizador usar índices o poda de particiones (ej. `fecha >= '2026-01-01'` es SARGable; `YEAR(fecha) = 2026` no lo es). *(... y 20 preguntas más cubriendo CTEs recursivas, índices B-Tree vs Hash, MERGE INTO atómico, transacciones ACID en bases relacionales, aislamiento serializable y funciones de ventana de desplazamiento LEAD/LAG).*

Bloque 2: Python, Tipado y Estructuras de Datos (20 Preguntas Clave)

6. *¿Cuál es la complejidad temporal de buscar un elemento en una lista frente a un conjunto (set)?*
   En `list` es $O(N)$ porque requiere búsqueda lineal; en `set` es $O(1)$ amortizado porque opera sobre una tabla hash.
7. *¿Qué es un generador (`yield`) y por qué es vital en pipelines de memoria limitada?*
   Es una función que produce valores perezosamente bajo demanda sin materializar la colección completa en RAM, evitando errores de OutOfMemory.
8. *¿Cómo funciona el retroceso exponencial con Jitter?*
   Calcula el tiempo de espera como $2^{\text{intento}} + \text{aleatorio}(0, 1)$ para dispersar las solicitudes concurrentes y evitar el problema del rebaño atronador (*Thundering Herd*). *(... y 17 preguntas más cubriendo GIL en CPython, serialización en PyArrow, decoradores, dataclasses y validación con Pydantic).*

Bloque 3: Apache Spark y Computación Distribuida (25 Preguntas Clave)

9. *¿Qué es exactamente una fase de Shuffle?*
   Es la redistribución física de datos a través de la red entre ejecutores provocada por transformaciones de dependencias anchas (`groupByKey`, `join`, `distinct`), involucrando serialización en memoria y escritura temporal a disco local.
10. *¿Cómo solucionarías un fallo OutOfMemory (OOM) por Data Skew?*
    Habilitando Adaptive Query Execution (`spark.sql.adaptive.skewJoin.enabled`), aplicando *Salting* sobre la clave de unión (agregando un sufijo aleatorio de 1 a N) o utilizando *Broadcast Join* si una de las tablas es pequeña. *(... y 23 preguntas más cubriendo Catalyst Optimizer, Tungsten execution engine, Garbage Collection pauses, Spark UI diagnosis, broadcast joins y dynamic partition pruning).*

Bloque 4: Lakehouse, Delta Lake y Almacenamiento Columnar (20 Preguntas Clave)

11. *¿Cómo garantiza Delta Lake transacciones ACID sobre Amazon S3 o Azure Data Lake?*
    Mediante un registro de transacciones inmutable (`_delta_log/`), donde cada operación realiza una confirmación atómica en archivos JSON secuenciales.
12. *¿Por qué un `DELETE` en Delta Lake no reduce inmediatamente el espacio en disco?*
    Porque Delta Lake utiliza aislamiento de instantáneas (Snapshot Isolation) y vectores de eliminación (Deletion Vectors). Los datos solo se eliminan físicamente al ejecutar `VACUUM` (precedido por `REORG TABLE ... APPLY (PURGE)` si hay Deletion Vectors activos). *(... y 18 preguntas más cubriendo Parquet columnar, Time Travel, Liquid Clustering vs Z-Order, y compatibilidad con Apache Iceberg).*

Bloque 5: Streaming en Tiempo Real y Apache Kafka (15 Preguntas Clave)

13. *¿Qué diferencia el tiempo de evento (Event-Time) del tiempo de procesamiento (Processing-Time)?*
    El tiempo de evento es la marca temporal registrada cuando ocurrió la acción en el cliente de origen; el tiempo de procesamiento es cuando el motor de cómputo procesa el mensaje.
14. *¿Cómo funciona un Watermark en streaming?*
    Define un umbral temporal permisible para la llegada de datos rezagados. Todo evento con marca inferior a `max_event_time - watermark` se descarta de la agregación de estado. *(... y 13 preguntas más cubriendo offsets en Kafka, particionamiento de brokers, rebalanceo de consumidores y semántica exactly-once).*

Bloque 6: Orquestación, DataOps y Modelado Dimensional (25 Preguntas Clave)

15. *¿Diferencia entre una Tabla de Hechos y una de Dimensiones en Kimball?*
    La tabla de hechos almacena métricas cuantitativas medibles a un grano definido; las dimensiones almacenan el contexto textual que responde a quién, dónde y cuándo.
16. *¿Cómo se modela una dimensión lentamente cambiante SCD Tipo 2?*
    Agregando columnas de vigencia (`valido_desde`, `valido_hasta`) y una bandera booleana (`es_actual = true/false`), cerrando la fila previa e insertando una nueva versión sin sobreescribir el histórico. *(... y 23 preguntas más cubriendo dbt snapshots, Airflow TaskFlow API, idempotencia en pipelines, Terraform state management y Docker multicapa).*

Bloque 7: Entrevistas Conductuales con Método STAR (20 Preguntas Clave)

17. *Háblame de una ocasión en que un pipeline falló en producción y cómo lo solucionaste.*
    (Estructurar con: Situación del incidente a las 2 AM → Tarea de restaurar el SLA comercial → Acción de mitigación inmediata y diagnóstico en Spark UI → Resultado: recuperación en 25 min y postmortem sin culpas implementando alertas preventivas). *(... y 19 preguntas más cubriendo desacuerdos técnicos con el arquitecto, priorización bajo presión y gestión de deuda técnica).*

36.7 Rangos Salariales Reales por País y Régimen de Contratación

La transparencia salarial permite negociar desde una posición informada. Las siguientes cifras corresponden a datos consolidados de **2025/2026** (fuentes: Michael Page IT Salary Report LATAM, Robert Half Technology Guide, levels.fyi y Torre):

Compensación Mensual Estimada (USD)

País / Modalidad	Junior (0–2 años)	Semi-Senior / Mid (2–4 años)	Senior (5+ años)	Lead / Staff Architect
Colombia (Nómina Local)	$1.000 – $1.800 USD	$2.200 – $3.800 USD	$4.200 – $6.500 USD	$7.000 – $9.500 USD
México (Nómina Local)	$1.200 – $2.000 USD	$2.500 – $4.200 USD	$4.800 – $7.500 USD	$8.000 – $11.000 USD
Chile / Argentina (Local)	$1.100 – $1.900 USD	$2.300 – $4.000 USD	$4.500 – $7.000 USD	$7.500 – $10.000 USD
Remoto Internacional (USD)	**$2.500 – $4.000 USD**	**$4.500 – $7.000 USD**	**$8.000 – $12.500 USD**	**$13.000 – $18.000+ USD**

Análisis Comparativo de Contratación en Colombia: Nómina vs. Contractor B2B

Para ingenieros radicados en Colombia que reciben ofertas locales o del exterior, es indispensable comprender la diferencia financiera y legal entre ambas modalidades:



1. Contrato Laboral a Término Indefinido (Nómina Colombiana)

- **Beneficios de Ley Incluidos:**
  - **Prima de Servicios:** 1 mes de salario al año (dividido en junio y diciembre).
  - **Cesantías:** 1 mes de salario ahorrado en fondo de cesantías + 12% de intereses sobre cesantías anuales.
  - **Vacaciones Remuneradas:** 15 días hábiles remunerados por año trabajado.
  - **Aportes Patronales:** El empleador asume el 8.5% de salud y 12% de pensión, además de la ARL y aportes parafiscales (Caja de Compensación 4%).

- **Ventaja:** Estabilidad jurídica absoluta, acceso ágil a crédito bancario hipotecario local y menor complejidad tributaria (la empresa practica la retención en la fuente).

2. Contrato de Prestación de Servicios / Contractor Internacional en USD

- **Marco Normativo Vigente (Colombia):**
  La obligación de cotización para independientes y contratistas de prestación de servicios se rige por la **Ley 100 de 1993**, la **Ley 1122 de 2007**, el **artículo 244 de la Ley 1955 de 2019** y el **Decreto 1273 de 2018**.
- **Seguridad Social Obligatoria (Planilla PILA):**
  - La ley colombiana exige cotizar sobre el **Ingreso Base de Cotización (IBC)**, que equivale por defecto al **40% del ingreso bruto mensual** (sin exceder el tope legal de 25 SMMLV).
  - Sobre ese 40% (IBC), el contratista paga directamente:
    - **Salud:** 12.5% del IBC.
    - **Pensión:** 16.0% del IBC.
    - **ARL:** 0.522% a 2.436% del IBC (según nivel de riesgo laboral).
- **Régimen Tributario:** Los ingresos se declaran anualmente ante la DIAN. Dependiendo del volumen de facturación y costos deducibles, el profesional puede tributar bajo el régimen ordinario (con rentas exentas y deducciones de ley) o acogerse al Régimen Simple de Tributación (RST) para reducir la tarifa efectiva.
- **Veredicto:** A partir de **$3.500 USD mensuales**, la modalidad Contractor B2B supera con holgura la compensación neta de la mayoría de ofertas de nómina local, compensando con creces la ausencia de prestaciones automáticas siempre que el ingeniero mantenga una disciplina estricta de ahorro para salud, pensión y fondo de emergencias.

> **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD TRIBUTARIA:**
> La información de esta sección se presenta exclusivamente con propósitos pedagógicos e ilustrativos sobre las costumbres del mercado laboral de software en Colombia y no constituye en ningún caso asesoría tributaria, contable, financiera ni jurídica personalizada. Las normas sobre retención en la fuente, costos deducibles, aportes a seguridad social y convenios para evitar la doble imposición internacional están sujetas a reformas legislativas continuas e interpretaciones de la DIAN y la UGPP. Se aconseja enfáticamente a todo profesional validar su situación particular con un Contador Público titulado antes de suscribir contratos internacionales.

36.8 Negociación Salarial y Cierre de Ofertas

1. **Investigue los percentiles reales:** Nunca inicie una conversación económica sin consultar *levels.fyi* para el nivel equivalente de la empresa objetivo.
2. **Desvíe la pregunta del salario actual:** Si el reclutador indaga su salario presente, responda: *«Prefiero enfocarme en el valor técnico que puedo generar en esta posición y en la banda presupuestada por la compañía para el rol según mis competencias.»*
3. **Puntualice la Compensación Total (*Total Compensation*):** Al comparar ofertas, evalúe la suma de salario base, bono anual de rendimiento, equipo de cómputo y presupuesto anual de formación/certificaciones.
4. **Palanca de Ofertas Simultáneas:** La herramienta de negociación más contundente es contar con dos procesos avanzados simultáneos. Permite comunicar con cortesía: *«Tengo otra propuesta sobre la mesa en una banda de $X USD, pero su proyecto y desafíos de datos me entusiasman más; si pueden igualar esa compensación, firmaré de inmediato»*.

36.9 Fórmulas de Estimación de Escala (Back-of-the-Envelope) y Constantes Físicas

En entrevistas de diseño de sistemas a nivel senior, la intuición numérica inmediata es obligatoria:

1. Números de Latencia Clave (Escala Humana de Peter Norvig)

Nivel de Memoria / Operación	Latencia Física	Equivalencia (1 ciclo CPU = 1 seg)
Ciclo de Reloj de CPU	0.3 ns	**1 segundo**
Acceso a Memoria RAM	100 ns	5 minutos
Lectura secuencial 1 MB de RAM	4 µs	3.5 horas
Búsqueda aleatoria en SSD NVMe	15 µs	14 horas
Ida y vuelta de red en Datacenter (LAN)	500 µs (0.5 ms)	19 días
Lectura aleatoria en Disco Magnético (HDD)	10 ms	**1 año**
Petición GET inicial a Amazon S3 / GCS	20–50 ms	3 a 5 años
Red transcontinental intercontinental	150 ms	15 años

2. Conversiones Rápidas de Throughput

- Un día tiene **86.400 segundos** ($\approx 10^5$ s).

- **1.000 QPS con eventos de 1 KB:**

$$1.000 \times 1\ \text{KB/s} = 1\ \text{MB/s} \approx 86.4\ \text{GB/día} \approx 2.6\ \text{TB/mes} \approx 31.5\ \text{TB/año}$$

- **Factor de compresión JSON a Parquet:** $3\times$ a $5\times$ (1 TB JSON ≈ 250 GB Parquet).
- **Particiones de Shuffle en Spark:** Dividir los datos intermedios entre 128 MB:

$$\text{spark.sql.shuffle.partitions} = \frac{\text{Bytes tras Shuffle}}{128\ \text{MB}}$$

36.10 Resumen y Puntos Clave

- **Alineación con el mercado:** SQL avanzado (96%) y Python modular (92%) son los filtros universales del 100% de los procesos de selección técnica.
- **Portafolio verificable:** Un repositorio con Docker Compose, dbt, CI/CD y pruebas unitarias supera cualquier currículum inflado.
- **El ATS premia la claridad:** Use plantillas de una sola página y viñetas cuantificadas con la fórmula Google X-Y-Z: *«Logré [X], medido por [Y], haciendo [Z]»*.
- **Claridad contractual:** Entienda las implicaciones tributarias y de seguridad social en Colombia (Planilla PILA sobre 40% de IBC) al comparar ofertas de nómina local frente a contratos Contractor en USD.

Lecturas Recomendadas

1. **Bialik, C. & Fry, H. (2023).** *Cracking the Data Engineering Interview: Core Frameworks and System Design.* O'Reilly Media.
2. **McDowell, G. L. (2019).** *Cracking the Coding Interview: 189 Programming Questions and Solutions.* CareerCup.
3. **Michael Page (2025).** *Estudio de Remuneración y Tendencias de Contratación en Tecnología LATAM.* PageGroup.
4. **Robert Half (2025).** *Salary Guide: Technology & Information Systems.* Robert Half International.

CAPÍTULO 37

Epílogo — El Futuro de la Ingeniería de Datos y Conclusión de la Obra RECOMENDADO

> *"Las herramientas de software nacen, alcanzan su apogeo y son inevitablemente sustituidas por nuevas abstracciones. Lo que permanece inmutable a través de las décadas son las leyes fundamentales de la computación distribuida: la física de la transferencia de datos a través de una red, la necesidad de consistencia transaccional, la elegancia de las estructuras de datos y el valor innegociable de la verdad analítica."*

37.1 La Trayectoria Recorrida

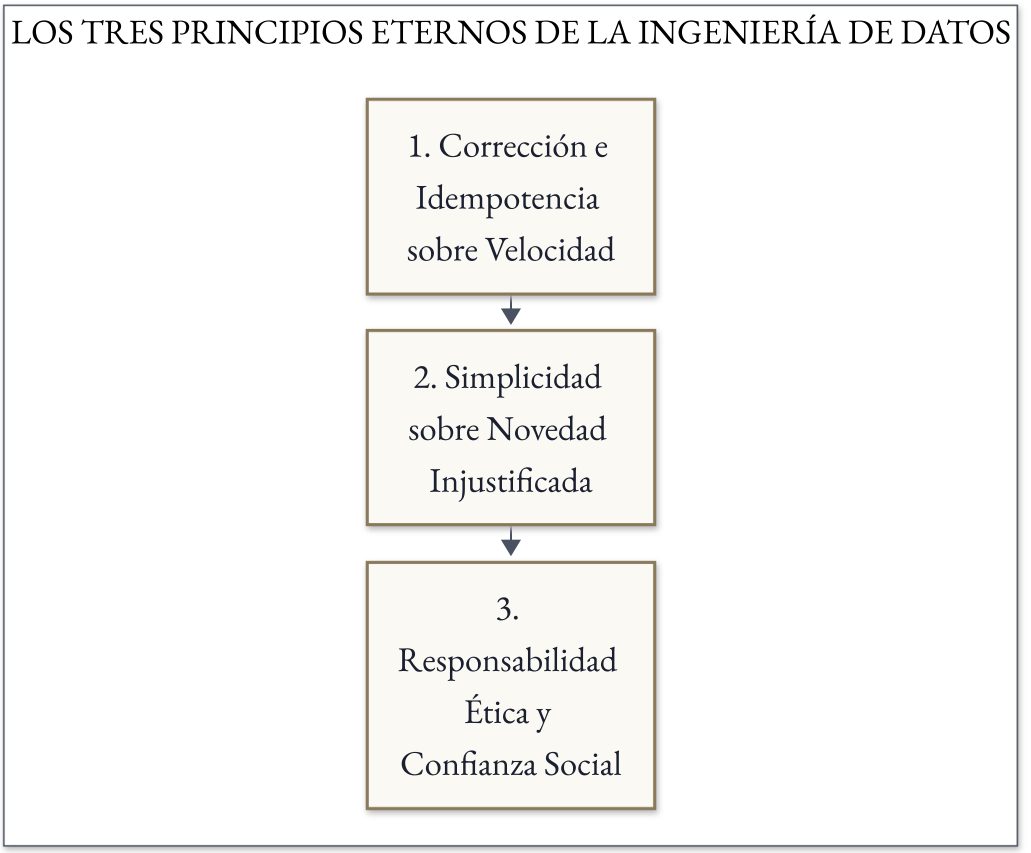
A lo largo de los más de cincuenta capítulos, estudios de caso y apéndices de esta obra, hemos transitado un camino exhaustivo y riguroso. Comenzamos en los cimientos indispensables que sostienen la disciplina: la línea de comandos de Linux, el control de versiones con Git, la virtualización ligera con Docker, el modelado relacional transaccional y la matemática declarativa de SQL y el tipado estricto en Python.

Posteriormente, desmitificamos la computación distribuida con Apache Spark, explorando su motor interno Catalyst, la gestión física de la memoria Tungsten y el comportamiento del particionamiento y el shuffle. Conquistamos la arquitectura Lakehouse a través de Delta Lake, Apache Iceberg, Apache Hudi y Databricks. Descendimos al procesamiento de flujos en tiempo real con Apache Kafka y Structured Streaming; formalizamos la orquestación con Apache Airflow, el modelado con dbt, la infraestructura declarativa con Terraform y las pruebas rigurosas con Testcontainers y Great Expectations. Finalmente, abordamos las nubes públicas líderes — AWS, Google Cloud Platform, Microsoft Fabric y Snowflake—, culminando en la ingeniería de datos orientada a inteligencia artificial, bases vectoriales y el diseño de sistemas de alta escala.

Este recorrido no fue concebido como una colección superficial de recetas de código, sino como un tratado de ingeniería destinado a forjar criterio técnico, intuición arquitectónica y solidez conceptual.

37.2 Los Tres Principios Eternos del Ingeniero de Datos

En una industria caracterizada por el ruido comercial y la proliferación vertiginosa de marcos de trabajo, el ingeniero distinguido se reconoce por su adhesión a principios que resisten el paso del tiempo:



1. Corrección e Idempotencia sobre Velocidad

Un pipeline de datos que procesa un terabyte por segundo pero produce datos duplicados, registros inconsistentes o agrega métricas distorsionadas es sustancialmente más peligroso que un sistema que no opera. La velocidad es una métrica de rendimiento; la corrección es una condición de existencia. Diseñe cada transformación asumiendo que la red fallará, los ejecutores se reiniciarán y los mensajes se reintentarán. La **idempotencia** y la **semántica exactamente una vez** no son lujos técnicos: son la única garantía de supervivencia de la plataforma analítica.

2. Simplicidad sobre Novedad Injustificada

El analista junior busca resolver cada problema mediante el despliegue del motor distribuido más complejo del mercado; el arquitecto senior resuelve el problema mediante la herramienta más simple que satisfaga las cotas de escala y el presupuesto financiero. Antes de encender un clúster de cincuenta máquinas en la nube, verifique si su volumen de datos cabe en la memoria de una instancia con DuckDB o PostgreSQL. La arquitectura más elegante es aquella que contiene la menor cantidad de piezas móviles susceptibles de fallar en la madrugada.

3. Responsabilidad Ética y Confianza Social

Los datos procesados por nuestras canalizaciones no son números abstractos en un archivo binario Parquet: representan historias médicas de pacientes, transacciones patrimoniales de familias, ubicaciones geográficas de personas e historiales que impactan directamente vidas humanas. La gobernanza de datos, la tokenización de información personal identificable (PII), el cumplimiento normativo (GDPR, Ley 1581) y la auditabilidad estricta son imperativos morales inalienables del ejercicio profesional.

37.3 El Ingeniero de Datos en la Era de la Inteligencia Artificial

Lejos de extinguir la necesidad de ingenieros de datos, el auge de los modelos fundacionales y la inteligencia artificial generativa ha elevado la disciplina a su punto de mayor relevancia histórica.

Los modelos de lenguaje más sofisticados del planeta son completamente inútiles sin datos limpios, contextualizados y libres de alucinaciones. Las arquitecturas de generación aumentada por recuperación (RAG), la sincronización de almacenes de características (*Feature Stores*) de baja latencia, el particionamiento de grafos de conocimiento y el enriquecimiento continuo de flujos no son problemas de algoritmos de inteligencia artificial: **son problemas fundamentales de ingeniería de datos**. El ingeniero de datos contemporáneo es el arquitecto que alimenta, conecta y valida los sistemas cognitivos del futuro.

37.4 Mensaje Final del Autor

Llegar a ser un verdadero ingeniero de datos no consiste en acumular insignias de herramientas de moda en un currículum, sino en cultivar la disciplina del artesano: aquel que lee los planes de ejecución, que comprende por qué una consulta genera un cuello de botella de red, que respeta la física de los discos y la memoria, y que construye tuberías que operan de forma silenciosa, predecible e implacable durante años sin requerir intervención manual.

Espero que los conceptos, diagramas, demostraciones matemáticas y códigos contenidos en este libro le sirvan como una brújula confiable a lo largo de su carrera. Vaya adelante, construya sistemas que resistan el caos del mundo real y recuerde siempre que detrás de cada decisión de negocio informada y de cada avance tecnológico, late en silencio la infraestructura invisible construida por un ingeniero de datos.

Luis Galvis
Ibagué, Tolima, Colombia · 2026